

GAME DEVELOPMENT MASTERBOOK

Game Development Masterbook

A Complete Guide to Plan, Create,
Publish and Succeed in Video Games

Edition: 1st Edition, 2026

Language: Bengali with English Terms

Target: Beginner to Advanced

GAME DEVELOPMENT MASTERBOOK

Game Development Masterbook

A Complete Guide to Plan, Create, Publish
and Succeed in Video Games

1st Edition, 2026

PART 1

PART 1 -- GAME DEVELOPMENT kii?

PART 1 -- GAME DEVELOPMENT kii?

****Learning Goal:**** aii chapter shaessae aapanai jaaanabaena Game Development-aira sabaaadhaana bhaitataika dhaaaranaaa, parataitai paeshaadaara bhauumaikaaa aiba Studio taaipaera paaarathakaya

1.1 Game kii?

****Game (gaema)**** halao aikatai naiyama-bhaitataika kaaathaaamao (Rule-based Framework) yaekhaanae aika baaa aikaaadhaika khaelaoyaaada (Player) nairadaissata udadaeshaya arajanaera janaya saidadhaanata naeya aiba saei saidadhaanataera phalaaaphala bhaoga karae

Game-aira mauula upaadaana (Core Elements):

Game-aira dharana:

1. ****Board Game (baorada gaema)**** -- Chess, Ludo, Monopoly
2. ****Card Game (kaaarada gaema)**** -- Poker, Uno
3. ****Sports (khaelaadhaulaaa)**** -- Cricket, Football
4. ****Video Game (bhaidaiu gaema)**** -- Mobile, PC, Console games
5. ****Tabletop RPG (taebailatapa RPG)**** -- Dungeons & Dragons
6. ****Puzzle Game (paaajala gaema)**** -- Rubik's Cube

****Pro Tip:**** aikatai bhaaalao Game haauyaaara janaya shaudhau Rules yathaessata naya -- khaelaoyaaadake ****Fun (majaaa)**** baodha karatae habae aitaai Game-aira sabachaeyae gaurautabapauurana baaishaissataya

1.2 Video Game kaiibhaabae kaaaja karae?

Video Game halao kamapaiutaaara, maobaaaila, baaa Console-ai chalaaa aikatai daijaitaaala Game aitai mauulata tainatai satarae kaaaja karae:

Game Loop (gaema laupa):

GAME LOOP (gaema laupa)
INPUT (inapauta)
DOWN

```

PROCESS (parasaesa)
  DOWN
RENDER (raenadaaara)
  DOWN
OUTPUT (aautapauta)
  DOWN
[Loop back to INPUT]

```

baisataaaraita bayaaakhayaaa:

Step 1: Input (inapauta)

Player-aira kaaachha thaekae tathaya sagaraha karaaa haya

- Keyboard, Mouse, Controller, Touch Screen
- udaaaharana: W/A/S/D chaaapalae player character saaamane yaaaya

Step 2: Process (parasaesa)

Game Engine inapauta parasaesa karae

- Physics Calculation (padaaarathabaidayaaara gananaaa)
- Collision Detection (sagharassa sanaakatakarana)
- AI Decision Making (AI saidadhaaanata garahana)
- State Management (abasathaaa bayabasathaaapanaaa)

Step 3: Render (raenadaaara)

gaaaaaphaikasa taairai karaaa haya

- 3D Models daekhaanaao
- Textures laaagaaanao
- Lighting saeta karaaa
- Animation chaaalaaanao

Step 4: Output (aautapauta)

Player-kae phalaaaphala daekhaanao haya

- Screen-ai chhabai daekhaaa
- Speaker thaekae Sound baaajaaa
- Controller-ai Vibration haauyaaa

Frame Rate (pharaema raeta):

1 saekaenada = 60 Frames (FPS)
aikatai Frame = 16.67 milliseconds

- ****30 FPS**** -- Minimum acceptable
- ****60 FPS**** -- Standard
- ****120 FPS**** -- Competitive gaming
- ****144+ FPS**** -- Esports

1.3 Game Development kaii?

****Game Development (gaema daebhaelapamaenata)**** halao aikatai Video Game taairaira samapauurana parakaraiyaaa aitai shaudhau Programming naya -- aitaе thaaakae Design, Art, Audio, Testing, Marketing aiba Business saba kaichhau

Game Development Pipeline (gaema daebhaelapamaenata paaaipalaaaina):

```

  IDEA    > RESEARCH > PITCH
(dhaaaranaaa)    (gabaessanaaa)    (paicha)
PRODUC- < GDD    < PROTO-
TION      (nakashaaa)    TYPE
(upaaadana)    (paraotao)
  ALPHA    > BETA    > LAUNCH
(aalaphaaa)    (baitaaa)    (maukatai)

```

Development Phases-aira baisataaaraita:

1.4 Game Designer kait?

****Game Designer (gaema daijaainaaara)**** halaena saei bayakatai yainai Game-aira naiyama, galapa, charaitara, paraibaesha aiba saaamagaraika abhaijanyataaa paraikalapanaaa karaena

Game Designer-aira kaaaja:

```

  GAME DESIGNER
[X] Game Concept (gaema dhaaaranaaa) taaairai
[X] Rules (naiyama) laekhaaa
[X] Story (galapa) laekhaaa
[X] Mechanics (maekaaanaikasa) daijaainaa
[X] Level Layout (maanachaitara) paraikalapanaaa
[X] Balance (bhaarasaaamaya) thaika karaaa
[X] Playtest (palaetaesata) paraichaaalanaaa
[X] Documentation (dakaumaenataeshana) laekhaaa

```

Game Designer-aira dharana:

1. ****Lead Designer (paradhaana daijaainaaara)**** -- saaamagaraika daaayaitaba
2. ****Systems Designer (saisataemasa daijaainaaara)**** -- Economy, Progression
3. ****Level Designer (laebhaela daijaainaaara)**** -- maanachaitara au paraibaesha
4. ****Narrative Designer (nayaaraetaibha daijaainaaara)**** -- galapa au Dialogue
5. ****Combat Designer (kamabayaaata daijaainaaara)**** -- yaudadha padadhatai
6. ****UI/UX Designer**** -- inataaaraphaesa daijaainaa

Required Skills:

- Creativity (sarijanashailataaa)
- Problem Solving (samasayaaa samaaadhaana)
- Communication (yaogaaayaoga)
- Documentation (dakaumaenataeshana)
- Playtesting (palaetaesata)

- Basic Programming (maaulaika paraogaraaamai)
-

1.5 Game Programmer kii?

****Game Programmer (gaema paraogaraaamaara)**** halaena saei bayakatai yainai Game Designer-aira dhhaaranaaagaulaokae kaodae rauupaaanata karaena

Game Programmer-aira dharana:

Required Skills:

- C# / C++ / Python
 - Data Structures & Algorithms
 - Physics & Math
 - Engine Knowledge (Unity/Unreal)
 - Debugging
 - Optimization
-

1.6 Game Artist kii?

****Game Artist (gaema aarataisata)**** halaena saei bayakatai yainai Game-aira sakala darishayamaana upaaadaana taairai karaena

Game Artist-aira dharana:

```
GAME ARTIST
2D ARTIST:
- Concept Art (kanasaepata aarata)
- Sprite (saparaaaita)
- UI Art (iuuui aarata)
- Texture (taekasachaaara)
3D ARTIST:
- 3D Modeling (3D madaelai)
- Texturing (taekasachaaarai)
- Rigging (raigai)
- Animation (ayaaanaimaeshana)
TECHNICAL ARTIST:
- Shaders (shaedaaara)
- VFX (bhaijayauyaaala iphaekata)
- Optimization (apataimaaaijaeshana)
```

1.7 Level Designer kii?

****Level Designer (laebhaela daijaaainaaara)**** halaena saei bayakatai yainai Game-aira parataitai Level/Map/Stage taairai karaena

Level Designer-aira kaaaja:

- Map Layout (maaanachaitaraera bainayaaasa) paraikalapananaa
- Player Path (khaelaoyaaadaera patha) sajanyaaayaita karaaa
- Encounter Placement (shatarau sathaaapana) karaaa
- Puzzle Design (paaajala daijaaaina) karaaa
- Environmental Storytelling (paraibaeshagata galapa) taairai karaaa
- Checkpoint (chaekapayaenata) saeta karaaa
- Secret Areas (gaopana ailaaakaaa) taairai karaaa
- Lighting Setup (aalaora bainayaaasa) karaaa

Level Design Process:

Concept -> Blockout -> Greybox -> Art Pass -> Lighting -> Polish -> Playtest

1.8 Game Producer kii?

****Game Producer (gaema paradaiusaaara)**** halaena saei bayakatai yainai Game Development-aira saaamagaraika parakaraiaaaa paraichaaalananaa karaena

Producer-aira daaayaitaba:

GAME PRODUCER

Project Management (parakalapa bayabasathaaapanaaa)
Budget Management (baajaeta)
Team Management (dala bayabasathaaapanaaa)
Schedule Management (samayasauuchaii)
Milestone Tracking (maaaailasataona)
Risk Management (jhaukai)
Stakeholder Communication
Documentation

Producer-aira dharana:

1. ****Executive Producer**** -- saaamagaraika tatatabaaabadhaana
 2. ****Lead Producer**** -- daainaika bayabasathaaapanaaa
 3. ****Associate Producer**** -- sahaaayataaa
 4. ****Associate Producer**** -- sahaaayataaa
-

1.9 Audio Designer kii?

****Audio Designer (adaiau daijaaainaaara)**** halaena saei bayakatai yainai Game-aira sakala Sound Effect, Music aiba Voice-over taairai karaena

Audio Designer-aira kaaaja:

1.10 QA Tester kii?

****QA Tester (kaiuai taesataaara)**** halaena saei bayakatai yainai Game-aira Bug au Error khaujae baera karaena

QA Tester-aira kaaaja:

- Bug Report (baaaga raipaorata) laekhaaa
- Reproduction Steps (paunaraupaaadana padadhatai) laekhaaa
- Severity Classification (taiibarataaa sharaenaiibaibhaaaga)
- Regression Testing (paunaraaabaritatai paraiikassaaa)
- Performance Testing (karamakassamataaa paraiikassaaa)
- Compatibility Testing (saaamanyajasaya paraiikassaaa)
- Usability Testing (bayabahaaarayaogayataaa paraiikassaaa)

Bug Severity Levels:

Critical -- Game Crash, Data Loss

Major -- Major Feature Broken

Minor -- Minor Issue, Workaround Available

Trivial -- Cosmetic Issue

1.11 Solo Game Developer banaaama Team

Solo Developer (aikaka daebhaelapaaara):

SOLO DEVELOPER

aikajana bayakatai saba kaichhau karaena:

[x] Design

[x] Programming

[x] Art

[x] Audio

[x] Testing

[x] Marketing

ADVANTAGES:

[x] Full Control (pauurana naiyanatarana)

[x] No Communication Overhead

[x] Full Profit (samapauurana laaabha)

[x] Flexible Schedule (nanaanaiya samayasauuchaii)

DISADVANTAGES:

Slow Development (dhaira upaadana)

Skill Gaps (dakassataaara abhaaaba)

Burnout Risk (baarananaaataera jhaukai)

Limited Scope (saiimaita paraisara)

Team (dala):

GAME DEVELOPMENT TEAM

aikaaadhaika bayakatai aikasaaathae kaaaja karaena:

[x] Specialized Roles (baishaessaaayaita bhauumaikaaa)

[x] Parallel Work (samaaanataraaaala kaaaja)

- [X] Higher Quality (uchachamaana)
 - [X] Larger Scope (bada paraisara)
- DISADVANTAGES:
- Communication Cost
 - Budget Required
 - Coordination Challenges
 - Revenue Sharing

Comparison Table:

1.12 Indie Studio banaaama AAA Studio

Indie Studio (inadai sataudaiau):

****Definition:**** chhaota, sabaaadhaina, saiimaita baaajaetae kaaaja karaaa aikatai Game Development Studio

- INDIE STUDIO
- Team Size: 1-20
 - Budget: \$0-\$500K
 - Games: Small to Medium
 - Focus: Innovation, Creativity
 - Timeline: 6-24 months
 - Revenue: \$10K-\$10M
- Examples:
- [X] Stardew Valley (1 person)
 - [X] Hollow Knight (3 people)
 - [X] Celeste (5 people)
 - [X] Among Us (5 people)
- ADVANTAGES:
- [X] Creative Freedom
 - [X] Quick Decisions
 - [X] Lower Risk
 - [X] Unique Games
- DISADVANTAGES:
- [X] Limited Resources
 - [X] Limited Marketing Budget
 - [X] Hard to Compete with AAA

AAA Studio (aiaiai sataudaiau):

****Definition:**** bada, bahau-jaaataiia, uchacha baaajaetae kaaaja karaaa Game Development Studio

- AAA STUDIO
- Team Size: 100-1000+
 - Budget: \$10M-\$500M+
 - Games: Large Scale
 - Focus: Quality, Polish, Sales
 - Timeline: 3-7 years
 - Revenue: \$100M-\$2B+
- Examples:
- [X] Rockstar Games (GTA V)
 - [X] Naughty Dog (Last of Us)
 - [X] CD Projekt Red (Cyberpunk 2077)
 - [X] EA (FIFA, Madden)
- ADVANTAGES:

- [X] Large Budget
- [X] Large Team
- [X] High Quality
- [X] Global Marketing
- DISADVANTAGES:
- [X] High Risk
- [X] Slow Development
- [X] Less Creative Freedom
- [X] High Expectations

1.13 Game Development Team Structure Diagram



Practical Exercise (paraaayaogaika anaushailana):

1. aapanaaara pachhanadaera 3tai Game laikhauna
2. parataitai Game-aira janaya 3tai karae Role chaihanaaita karauna
3. aikatai Game baechhae naina aiba taaara Team Structure aakauna

Common Mistakes (saaadhhaarana bhaula):

- Game Development shaudhau Programming manae karaaa
- aikaaai saba karaaara chaessataaa karaaa
- Planning chhaaadaaaa saaathae saaathae kaaaja shaurau karaaa
- Scope bada karae phaelaaa
- Quality Assurance upaekassaaa karaaa

Pro Tips:

- parataitai Role-aira kaaaja samaparakae jaaanauna

- chhaota kaaaja daiyae shaurau karauna
 - dalae kaaaja karaaara abhaijanyataaaa naina
 - Game Development Community-tae yaoga daina
 - parataidaina kaichhau natauna shaikhauna
-

Chapter Summary (saaarasakassaepa):

- Game halao naiyama-bhaitataika maithasakaraiyaaa
- Video Game halao daijaitaaala Game yaaa Input-Process-Render Loop daiyae kaaaja karae
- Game Development halao samapauurana Game taairaira parakaraiyaaa
- parataitai Role (Designer, Programmer, Artist, etc.) aira naijasaba daaayaitaba aachhae
- Solo Developer aiba Team-aira paaarathakaya aachhae
- Indie Studio aiba AAA Studio-aira paaarathakaya aachhae
- aikatai Game Development Team-aira Structure baojhaaa jaurai

PART 2

PART 2 -- GAME IDEA baera karaaara padadhatai

PART 2 -- GAME IDEA baera karaaara padadhatai

****Learning Goal:**** aii chapter shaessae aapanai jaaanabaena kaiibhaaabaek aikatai natauna, unique aiba marketable Game Idea taairai karatae haya

2.1 kaiibhaaabaek natauna Game Idea khaujatae haya?

Idea Sources (dhaaaranaaara usa):

```
IDEA SOURCES (usa)
Other Games -- anayaaanaya Game thaekae
Movies/TV -- sainaemaaa/taibhai thaekae
Books -- bai thaekae
Real Life -- baaasataba jaiibana thaekae
Music -- sagaiita thaekae
Dreams -- sabapana thaekae
News -- sabaaada thaekae
Brainstorming -- masataissakadaaa
Random Generator -- rayaaanadama
"What If?" -- "yadai... haya?"
```

Idea Generation Methods:

****Method 1: Mashup (mayaaashaaapa)****

dautai baaa tataodhaika Game-aira dhaaaranaaa aikataraita karauna

```
Horror + Puzzle = Silent Hill
FPS + Survival = Left 4 Dead
RPG + Roguelike = Hades
Platformer + Metroidvania = Hollow Knight
```

****Method 2: Constraint (baaadhaaa)****

naijaekae kaichhau baaadhaaa daina

- shaudhau aikatai Button daiyae khaelaaa yaaaya aimana Game
- shaudhau aikatai Color-aira Game
- shaudhau Sound-aira upara nairabharashaiila Game
- 5 mainaitaera madhayae shaessa haya aimana Game

****Method 3: Inversion (baiparaiitakarana)****

aikatai Game-aira dhaaaranaaa ulatao karauna

- Mario: Princess kae baaachaaanao -> parainasaesa haisaebae khaelaaa
- Horror: bhaya paaaauyaaa -> bhaya daekhhaanao
- FPS: shatarau maaaraaa -> shataraukae baaachaaanao

****Method 4: Scale (paraisara)****

Game-aira paraisara paraibaratana karauna

- bada Game -> chhaota Game (Indie version)
- chhaota Game -> bada Game (AAA version)
- Serious Game -> Silly version
- Realistic -> Stylized

2.2 Unique Concept taairai karaaara padadhatai

aikatai Concept Unique karaaara upaaaya:

****1. Mechanic (maekaaanaika)****

Game-aira mauula khaelaaara padadhatai unique karauna

- Portal: darajaaa taairai karaaara Mechanic
- Tetris: aakaaara maelaanao

****2. Theme (thaima)****

Game-aira baissayabasatau unique karauna

- Papers, Please: Border Control Officer
- Return of the Obra Dinn: Mystery Investigation

****3. Setting (saetai)****

Game-aira jaga unique karauna

- Outer Wilds: mahaaakaaasha
- Celeste: paaahaaadaera chauudaaaya

****4. Presentation (upasathaaapana)****

Game-aira daekhhaara dharana unique karauna

- Limbo: kaaalao-saaadaaa Silhouette
- Cuphead: 1930s Cartoon Style

2.3 Genre Selection (dharana nairabaaachana)

Game Genres-aira taaalaikaaa:

Genre Selection Framework:

GENRE SELECTION FRAMEWORK

1. aapanaaara pachhanadaera Genre kaii?
-> yaaa bhaaalaobaaasaena taaa taairai karauna
2. aapanaaara Skill Set kaii?
-> yaaa jaaanaena taaa bayabahaaara karauna

3. Market-ai kaona Genre chalachhae?
-> Trend daekhauna
 4. kaona Genre Competition kama?
-> Niche khaujauna
 5. kaona Genre-tae Innovation samabhaha?
-> natauna kaichhau karauna
-

2.4 Target Audience (lakassaya darashaka)

Audience Demographics:

Audience Psychographics:

AUDIENCE PSYCHOGRAPHICS

Core Gamer:

- 4+ hours/week
- High-end PC/Console
- Willing to pay \$60+
- Values depth and challenge

Casual Gamer:

- 1-2 hours/week
- Mobile/Switch
- Free-to-play preferred
- Values fun and relaxation

Hardcore Gamer:

- 10+ hours/week
- Multiple platforms
- Early Adopter
- Values competition and mastery

Family Gamer:

- Plays with kids/family
 - Switch, Mobile
 - Family-friendly content
 - Values co-op and sharing
-

2.5 Platform Selection (palayaaatapharama nairabaaachana)

Platform Comparison:

Platform Selection Decision:

PLATFORM SELECTION DECISION

aapanaaara Game-aira dharana kaii?

Casual/Mobile -> Android/iOS

Core/PC -> Steam

Console -> Switch/PlayStation

Indie -> Itch.io + Steam

AAA -> All Platforms

aapanaaara baaajaeta kata?

\$0 -> Itch.io, Google Play

\$100 -> Steam
\$1000+ -> Steam + Console
\$10000+ -> All Platforms

2.6 Core Fantasy (mauula kalapanaaa)

Core Fantasy halao saei anaubhauutai yaaa Player khaelaaara samaya paetae chaaaya

Core Fantasy Examples:

Core Fantasy Framework:

Player wants to feel: _____
When they: _____
In a world that: _____

Example:

When they: Hunt monsters in a dark forest
In a world that: Is corrupted by evil forces

Player

2.7 Player Motivation (khaelaoyaaadaera anauparaeranaaaa)

Bartle's Player Types:

BARTLE'S PLAYER TYPES

ACHIEVER (arajanakaaaraii):

- paurasakaaara paetae chaaaya
- lakassaya pauurana karatae chaaaya
- Level Up karatae chaaaya

Example: RPG Games

EXPLORER (anausanadhaaanakaaaraii):

- natauna jaaayagaaa khaujatae chaaaya
- Secret khaujae baera karatae chaaaya
- Map pauraotaaa daekhatae chaaaya

Example: Open World Games

SOCIALIZER (saaamaajaika):

- anayadaera saaathae khaelatae chaaaya
- banadhau taairai karatae chaaaya
- dalae kaaaja karatae chaaaya

Example: MMO Games

KILLER (parataiyaogaii):

- anayadaera haaaraaatae chaaaya
- parataiyaogaitaaaya jaitatae chaaaya
- shakatai paradarashana karatae chaaaya

Example: Competitive Games

Motivation Examples:

2.8 Gameplay Loop (gaemapalae laupa)

****Gameplay Loop**** halao Player-aira paunaraaabaritata karaiyaaakalaaapaera dhaaaraaa

Core Loop Example (Mario):

```
MARIO GAMEPLAY LOOP
chalauna (Run)
DOWN
laaaphaaai (Jump)
DOWN
shatarau maaarao (Kill Enemy)
DOWN
kayaena sagaraha (Collect Coin)
DOWN
shakatai paaaau (Power Up)
DOWN
chauudaaya paauchhaaaau (Reach Flag)
DOWN
[natauna Level shaurau]
```

Core Loop Template:

```
CORE GAMEPLAY LOOP
PLAYER ACTION (khaelaoyaaadaera kaaaja)
DOWN
CHALLENGE (chayaaalaenyaja)
DOWN
REWARD (paurasakaaara)
DOWN
PROGRESSION (agaragatai)
DOWN
NEW CHALLENGE (natauna chayaaalaenyaja)
DOWN
[REPEAT]
```

2.9 Hook (hauka)

****Hook**** halao saei aikatai baaishaissataya yaaa Player-kae aakarissata karae

Hook Examples:

Hook Framework:

This game is like [Familiar Game] but with [Unique Twist]

****Examples:****

This game is like Dark Souls but in space
This game is like Candy Crush but with horror elements

This g

2.10 USP (Unique Selling Proposition)

USP halao saei baaishaissataya yaaa aapanaaara Game-kae anayaaanaya Game thaekae aalaaadaaaa karae

USP Examples:

USP Framework:

Our game is the only game that [Unique Feature] for [Target Audience] who want to [Core Fa

Example:

for horror fans who want a truly terrifying experience.

2.11 Theme, Setting, Art Style

Theme (thaima):

Theme halao Game-aira mauula baaarataaa

Setting (saetai):

Setting halao Game-aira samaya au sathaaana

Art Style (shaailapaika dharana):

2.12 IDEA Formula

IDEA FORMULA	
I	= Genre (dharana)
D	= Player Fantasy (kalapanaaa)
E	= Unique Mechanic (ananaya maekaaanaika)
A	= Theme + Audience (thaima + darashaka)
IDEA	= Genre + Player Fantasy
	+ Unique Mechanic + Theme
	+ Audience

2.13 20tai Game Concept Examples

Concept 1: "Shadow Walker"

Concept 2: "Pixel Kingdom"

Concept 3: "Space Chef"

Concept 4: "Time Loop Detective"

Concept 5: "Ghost Taxi"

Concept 6: "Monster Farmer"

Concept 7: "Neon Runner"

Concept 8: "Memory Palace"

Concept 9: "Deep Sea Colony"

Concept 10: "Bullet Heaven"

Concept 11: "Forest Whisperer"

Concept 12: "Mech Rider"

Concept 13: "Dream Architect"

Concept 14: "Zombie Chef"

Concept 15: "Quantum Thief"

Concept 16: "Pirate's Legacy"

Concept 17: "Sound Puzzle"

Concept 18: "Tower of Dreams"

Concept 19: "AI Garden"

Concept 20: "Horror Hotel"

Practical Exercise:

1. IDEA Formula bayabahaaara karae 3tai Game Concept taairai karauna
 2. parataitai Concept-aira janaya:
 - Genre nairabaaachana karauna
 - Target Audience sajanyaaayaita karauna
 - Core Fantasy laikhauna
 - Unique Hook khaujae baera karauna
 - Art Style baechhae naina
 3. aikatai Game Concept baechhae naiyae Pitch taairai karauna
-

Common Mistakes:

- anayadaera Game kapai karaaa
 - Scope khauba bada raaakhaaa
 - Market Research naaa karaaa
 - Target Audience naaa baojhaaa
 - Unique Hook naaa thaaakaaa
-

Pro Tips:

- chhaota daiyae shaurau karauna
 - yaaa jaaanaena taaa karauna
 - Market-ai kaonatai chalachhae taaa daekhauna
 - naijaera pachhanadaera Game taairai karauna
 - parataidaina natauna dhaaaranaaa laikhauna
-

Chapter Summary:

- Game Idea khaujae baera karaaara anaeka padadhatai aachhae
- IDEA Formula: Genre + Player Fantasy + Unique Mechanic + Theme + Audience
- Market Research jaraurai

- Unique Hook thaaakaaa darakaaara
- 20tai Example daeayaaa hayaechhae

PART 3

Chapter 3

adhayaaaya 3: GAME MARKET RESEARCH -- gaema maaarakaeta raisaaaracha

shaekhaara udadaeshaya (Learning Goal)

aai adhayaaaya shaessae aapanai jaaanabaena:

- Game Market Research kaii aiba kaena aitai atayanata gaurautabapauurana
 - Competitor Research kaibhaaabaekarabaena (Steam, Google Play, Console)
 - Genre Research aiba Player Review Analysis padadhatai
 - Pricing Research aiba Sales Estimation kaaushala
 - Wishlist Strategy kaibhaaabaekabaaasatabaaayana karabaena
 - Competitive Analysis Table taairai karatae shaikhabaena
-

dhaaaranaaa bayaaakhayaaa (Concept Explanation)

Game Market Research kaii?

Game Market Research halao aapanaaara gaema taairai karaaara aagae aiba samayae samayae maaarakaeta baishalaessana karaaara parakaraiyaaa aitai aapanaaakae jaaanaaya:

- kaona dharanaera gaema aikhana janapariya
- aapanaaara parataiyaogaiiraaa kaii karachhae
- khaelaoyaaadaraaa kaii chaaichhae
- kaona maaarakaetae kaona dharanaera gaema saphala hachachhae
- aapanaaara gaemaera janaya sathaika paraaaisa payaenata kaii habae

kaena Market Research gaurautabapauurana?

WHY MARKET RESEARCH MATTERS

Without Research	With Research
Random Game Idea	Data-Driven Idea
Wrong Audience	Targeted Audience
Wrong Pricing	Optimal Pricing

Too Much Competition	Market Gap Found
Low Sales	Higher Chance of Success
Development Waste	Focused Development
Result: 90% Failure	Result: Much Higher Success Rate

Competitor Research Methods

1. Steam Research (PC Games)

Steam halao baishabaera barihatatama PC Game Distribution Platform aikhaana thaekae aapanai parataiyaogaiidaera samaparaka gaurautabapaurana tathaya sagaraha karatae paaarabaena

****Steam thaekae yaaa daekhabaena:****

- Game Store Pages: parataitai gaemaera Steam Store Page daekhauna
- Review Scores: Positive/Negative reviews gananaaa karauna
- Player Count: Concurrent Players aiba Total Players
- Tags: gaemaera kaona Tags baeshai janaparaia
- Price History: SteamDB thaekae paraaaaisa haisatarai daekhauna
- Sales Data: VG Insights, SteamSpy thaekae saelasa daetaaa sagaraha karauna

****Steam Research taulasa:****

- SteamDB (steamdb.info): paraaaaisa haisatarai, saelasa daetaaa
- SteamSpy: saelasa aisataimaeshana, palaeyaaara daetaaa
- VG Insights: raebhaiu ainaaalaaisaisa, saelasa aisataimaeshana
- Steam Review Analysis: palaeyaaara samasayaaa au pachhanada baojhaaa

2. Google Play Research (Mobile Games)

Google Play Store halao Android maobaaaila gaemaera janaya barihatatama Platform

****Google Play thaekae yaaa daekhabaena:****

- Downloads: kata baaara Download hayaechhae (1M+, 10M+, 100M+)
- Rating: 4.5+ raetai katagaulao gaemae aachhae
- Reviews: palaeyaaararaaa kii balachhae
- Top Charts: kaona gaema Top tae aachhae
- Category Charts: kaona Category ai kaona gaema aachhae
- Ad Strategy: kaona gaema kii dharanaera Ad bayabahaaara karachhae

****Mobile Research taulasa:****

- Sensor Tower: App Download & Revenue Data
- data.ai (App Annie): Market Intelligence
- AppFollow: Review Analysis
- Mobile Action: Keyword Research

3. Console Research (PlayStation, Xbox, Nintendo)

Console maaarakaeta baojhaaa saamaaanaya bhainana

****Console thaekae yaaa daekhabaena:****

- PlayStation Store, Xbox Store, Nintendo eShop
- Featured Games: kaona gaema Featured hachachhae
- Exclusive Games: kaona Platform ai kii Exclusive aachhae
- Price Ranges: kaona Price Range ai baeshai baikarai hachachhae

Genre Research

Genre Research halao baojhaaa kaona dharanaera gaema aikhana janaparaiya aiba kaona Genre ai kama parataiyaogaitaaa aachhae

****Genre Research padadhatai:****

1. ****Genre Popularity Check****: Steam Tags, Google Play Categories daekhauna
2. ****Genre Trends****: kaona Genre aikhana baaadachhae, kaona Genre kamachhae
3. ****Genre Saturation****: kaona Genre ai baeshai gaema aachhae
4. ****Genre Gap****: kaona Genre ai bhaaalao gaema kama aachhae
5. ****Hybrid Genres****: dauyaera baeshai Genre mailaiya natauna kaichhau taairai

Genre Research Framework		
Step 1: List Popular Genres		
Genre	Popularity	Competition
Action	High	Very High
RPG	High	High
Puzzle	Medium	Medium
Horror	Medium	Low-Medium
Strategy	Medium	Medium
Step 2: Find Gaps		
- Where demand is HIGH but supply is LOW		
- Where quality is LOW but players want BETTER		
Step 3: Identify Opportunities		
- Hybrid Genres (e.g., Horror + Puzzle)		
- Underserved Platforms		
- Niche Audiences		

Player Review Analysis

Player Review Analysis halao palaeyaaaradaera raibhaiu padae baojhaaa taaaraaa kii pachhanada karae aiba kii samasayaaa paaaya

****Review Analysis padadhatai:****

1. ****Positive Review Analysis**** (5-Star, Positive):
 - palaeyaaararaaa kaona Feature pachhanada karaechhae?
 - kaona Mechanics bhaaalao kaaaja karaechhae?
 - kaona Element manaoyaoga dharae raekhaechhae?
2. ****Negative Review Analysis**** (1-Star, Negative):
 - palaeyaaararaaa kaona Problem paeyaeachhae?
 - kaona Feature naegaetaibha raibhaiu ainaechhae?
 - kaona Issue baaarabaaara uthae aisaechhae?
3. ****Common Themes****:

- sabachaeyae baeshai kaona Problem balaaa hayaechhae?
- sabachaeyae baeshai kaona Feature parashasaaa karaaa hayaechhae?

REVIEW ANALYSIS WORKFLOW

100 Reviews Read

Positive (70)	Negative (30)
What Worked?	What Failed?
- Great Combat	- Boring Story
- Good Art	- Too Short
- Fun Puzzles	- Bugs

ACTIONABLE INSIGHTS

- Focus on Combat & Art
- Write Better Story
- Make Game Longer
- Test Thoroughly

Pricing Research

Pricing Research halao sathaika daaama nairadhaaarana karaaa yaaa palaeyaaararaaa daitae raaajaii habae aiba aapanai laaabhabaaana habaena

****Pricing Research padadhatai:****

1. ****Competitor Price Analysis****: aikai Genre aira gaemaera daaama daekhauna
2. ****Price Sensitivity****: palaeyaaararaaa kata daitae raaajaii
3. ****Value Proposition****: aapanaaara gaemaera mauulaya kata
4. ****Regional Pricing****: baibhainana daeshae baibhainana daaama
5. ****Discount Strategy****: kakhana aiba kata Discount daebaena

****Pricing Tiers:****

- ****Premium (\$15-60)****: Full-featured game
- ****Mid-tier (\$5-15)****: Good quality, smaller scope
- ****Budget (\$1-5)****: Simple but fun
- ****Free-to-Play****: Free with in-app purchases
- ****Early Access (\$10-30)****: In-development, discounted

Sales Estimation

Sales Estimation halao aapanaaara gaema katataukau baikarai hatae paaarae taaa anaumaaana karaaa

****Sales Estimation padadhatai:****

1. ****Platform Data****: SteamSpy, Sensor Tower thaekae daetaaa sagaraha
2. ****Comparable Games****: aikai dharanaera gaemaera saelasa daekhauna
3. ****Marketing Reach****: kata maaanaussa aapanaaara gaemaera kathaaa jaaanabae
4. ****Conversion Rate****: katajana Wishlisted thaekae Buy karabae
5. ****Revenue Projection****: maota aaya anaumaaana

Wishlist Strategy

Wishlist halao Steam aira aikatai Feature yaekhaaanae palaeyaaararaaa taaadaera pachhanadaera gaema saebha karae raaakhae

****Wishlist Strategy padadhatai:****

1. ****Early Announcement****: gaema ghaossanaaaa karauna yata taaadaaataaadai samabhaha
2. ****Steam Page Setup****: Steam Store Page taairai karauna
3. ****Wishlist Call-to-Action****: saba jaaayagaaaya "Wishlist Now" balauna
4. ****Demo Release****: Demo daiyae Wishlists baaadaaana
5. ****Marketing Push****: parataitai Marketing Moment ai Wishlist chaaaibaena

WISHLIST FUNNEL

Awareness (100,000 people see your game)
(10% interested)

Interest (10,000 people click)
(30% wishlist)

Wishlist (3,000 people wishlist)
(20% buy at launch)

Purchase (600 people buy)
(10% review)

Reviews (60 reviews)

Key Metrics:

- Wishlist to Purchase: 15-25% (good)
- Awareness to Wishlist: 3-10% (good)

Competitive Analysis Table

naichaera taebailae 5tai Real Indie Game Example daeauyaaa halao yaegaulao thaekae aapanai shaikhatae paaarabaena:

Competitive Analysis Table baishalaessana

1. Hollow Knight (Team Cherry)

- ****kaena saphala****: Metroidvania Genre ai daaarauna kaaaja karaechhae Hand-drawn Art Style, Tight Combat, aiba Massive World aira samanabaya
- ****shaikassaaa****: Polished Core Mechanics aiba Massive Content aira samanabaya daiiraghamaeyaaadai saphalataaa naishachaita karae
- ****anaukaranayaogaya****: yadai aapanai Metroidvania baaanaaatae chaaana, Hollow Knight aira matao Combat System aiba Exploration taairai karauna

2. Stardew Valley (ConcernedApe)

- ****kaena saphala****: aikajana Solo Developer aira saaaphalaya Farming Simulation aira saaathae RPG Elements yaoga karae gabhaiirataaa ainaechhae
- ****shaikassaaa****: Simplicity + Depth = saphalataaa aikatai Simple Concept kae gabhaiira karae taulauna
- ****anaukaranayaogaya****: Pixel Art + Farming Loop + Social System = Proven Formula

3. Hades (Supergiant Games)

- ****kaena saphala****: Roguelike Format ai Narrative dhaokaaanao parataitai Run ai Story Progress haya

- ****shaikassaaa****: Narrative + Gameplay Integration shakataishaaalaili palaeyaaarakae Story jaaanaaara janaya khaelatae ichachhauka karaaana
- ****anaukaranayaogaya****: Quality over Quantity aiba Genre-Blending kaaushala

4. Celeste (Maddy Thorson)

- ****kaena saphala****: Accessibility Features yaemana Assist Mode daiyae saba dharanaera palaeyaaarakae anatarabhaukata karaechhae
- ****shaikassaaa****: Accessibility = Larger Audience shaudhau Hardcore Gamers naya, sabaaaikae khaelaara sauyaoga daina
- ****anaukaranayaogaya****: Emotional Storytelling + Tight Controls + Accessibility

5. Among Us (InnerSloth)

- ****kaena saphala****: Simple Mechanics + Social Element = Viral Potential Streaming-friendly Gameplay
- ****shaikassaaa****: sahaja Mechanics au Social Interaction bhaaaairaaala hatae paaarae Multiplayer Focus karauna
- ****anaukaranayaogaya****: Simple Design + Social Deduction + Content Updates

Practical Exercise

anaushailana 1: Competitor Research

****nairadaeshanaaa****

1. Steam ai yaaana aiba aapanaaara pachhanadaera Genre aira 5tai gaema khaujauna
2. parataitai gaemaera janaya naichaera tathaya sagaraha karauna:
 - Game Name
 - Developer
 - Release Date
 - Price
 - Review Score (Positive %)
 - Key Features
 - Player Complaints
3. aikatai Comparison Table taairai karauna

anaushailana 2: Review Analysis

****nairadaeshanaaa****

1. aikatai janaparaiya gaemaera 50tai Positive aiba 50tai Negative Review padauna
2. naichaera Categories ai bhaaaga karauna:
 - ****Positive Themes****: palaeyaaararaaa kaii pachhanada karaechhae
 - ****Negative Themes****: palaeyaaararaaa kaii samasayaaa paeyaeachhae
 - ****Common Complaints****: sabachaeyae baeshai kaona Problem balaaa hayaechhae
3. aapanaaara gaemaera janaya 3tai Actionable Insight laikhauna

anaushailana 3: Pricing Research

****nairadaeshanaaa:****

1. aikai Genre aira 10tai gaemaera daaama daekhauna
 2. Price Distribution taairai karauna:
 - katagaulao \$0-5
 - katagaulao \$5-10
 - katagaulao \$10-15
 - katagaulao \$15-20
 - katagaulao \$20+
 3. aapanaara gaemaera janaya sathaika Price Point nairadhaaarana karauna
-

Common Mistakes (saaadhaaarana bhaula)

1. Research naaa karae saraaasarai Development shaurau karaaa

bhaula: "aamaara aikatai aaidaiyaaa aachhae, aikhanai kaoda laikhai"

sathaika: "parathamae maaarakaeta raisaaaracha karai, taaarapara aaidaiyaaa Validate kara

2. shaudhau Positive Review padaaa

bhaula: "aai gaemaera 90% Positive Review aachhae, taaahalae bhaaalao"

sathaika: "Negative Review gaulao padai, saekhaana thaekae samasayaaa baujhai"

3. Competitor kae Copy karaaa

bhaula: "aai gaemataaa saphala hayaechhae, aamai aikai kapai karai"

sathaika: "aai gaemataaa kaena saphala hayaechhae, aamai kaiibhaaaba bhainana karatae pa

4. Market Trends upaekassaaa karaaa

bhaula: "Trends tao paraibaratana haya, aamai yaaa chaaai taaai baaanaaaba"

sathaika: "Trends daekhai, taaara saaathae aamaara Vision mailaiyae chalai"

5. Regional Pricing naaa karaaa

bhaula: "saba daeshae aikai daaama raaakhai"

sathaika: "baibhainana daeshae baibhainana Regional Price raaakhai"

6. Wishlist Strategy naaa thaaakaaa

bhaula: "gaema taairai halae Steam ai daiyae daeba"

sathaika: "gaema taairaira aagae Steam Page baaanaiyae Wishlist jamaaa karai"

Pro Tips (paeshaadaara taipasa)

1. SteamSpy Data sathaikabhaaaba bayabahaaara karauna

SteamSpy sabasamaya sathaika daetaaa daeya naaa tabae Range Estimate haisaebae bayabahaaar VG Insights aira saaathae Compare karauna

2. Player Review thaekae Feature Idea naina

parataitai Negative Review aikatai Feature Idea

"aai Feature thaaakalae aamai 5 Star daitaama" -- aai dharanaera Review khaujauna

3. Genre Trends tarayaaaka karauna

Trends Change haya Horror Genre aikhana Popular, taaahalae kai 2 bachhara para thaaakabae?
Long-term Trends daekhauna, Short-term naya

4. Competitor aira Weakness khaujauna

parataitai gaemaera kamaitai kaichhau aapanaaara gaema saei kamaitai dauura karatae paaara
aitaaai aapanaaara USP (Unique Selling Point) hatae paaarae

5. Demo daiyae Validate karauna

Concept Phase ai Demo baaanaiyae Player Feedback naina

Demo thaekae Wishlist jamaaa karauna

6. Market Research kae Ongoing Process baaanaana

aikabaaara raisaaaracha karae chhaedae daebaena naaa

Development Cycle saaaaaa dharae raisaaaracha chaaalaiyae yaaana

Chapter Summary (adhayaaaya saaaraaasha)

mauula paaayaenata:

1. ****Game Market Research**** halao saphala gaema taairaira parathama dhaaapa
2. ****Competitor Research**** karauna (Steam, Google Play, Console) -- parataiyaogaiidaera samaparakaе jaaanauna
3. ****Genre Research**** karauna -- kaona Genre ai kaothaaaya Gap aachhae khaujauna
4. ****Player Review Analysis**** karauna -- palaeyaaararaaa kaii chaaaichhae baojhauna
5. ****Pricing Research**** karauna -- sathaika daaama nairadhhaarana karauna
6. ****Sales Estimation**** karauna -- baikarai anaumaaana karauna
7. ****Wishlist Strategy**** taairai karauna -- aagaaama palaeyaaara taairai karauna

parabaratai adhayaaaya:

adhayaaaya 4 ai aamaraaa daekhabao ****Game Concept Document**** kaibhaaabae laikhabaena -- aapanaaara
gaemaera paura dhaaaranaaa aikatai Document ai kaibhaaabae saaajaaabaena

****manae raaakhabaena****: bhaaalao Market Research chhaadaaa bhaaalao Game taairai samabhaba naya aapanaaara samaya
baaachaaatae aiba saphalataaara samabhaabanaaa baaadaatae parataitai dhaaapae raisaaaracha karauna

PART 4

adhayaaaya 4: GAME CONCEPT DOCUMENT -- gaema kanasaepata dakaumaenata

adhayaaaya 4: GAME CONCEPT DOCUMENT -- gaema kanasaepata dakaumaenata

shaekhaara udadaeshaya (Learning Goal)

aii adhayaaaya shaessae aapanai jaaanabaena:

- Game Concept Document kaii aiba kaena aitai parayaojana
 - aikatai Professional Game Concept Document kaibhaaabaek laikhabaena
 - parataitai Section aira gaurautaba aiba kaibhaaabaek pauurana karabaena
 - Horror Game aira aikatai Complete Sample Concept Document
-

dhaaaranaaa bayaaakhayaaa (Concept Explanation)

Game Concept Document kaii?

Game Concept Document (GCD) halao aapanaara gaemaera samapauurana dhaaaranaara aikatai Written Document aitai aikatai Blueprint yaaa aapanaakaek aiba aapanaara taimakae daekhaaya gaemataaa kaemana habae

WHY GAME CONCEPT DOCUMENT?

Without Concept Document:

- Ideas remain vague in your mind
- Team members have different understanding
- Scope keeps changing
- Difficult to communicate to others
- No clear direction

With Concept Document:

- Clear vision written down
- Everyone on same page
- Scope is defined
- Easy to pitch to publishers/investors
- Reference document throughout development

GDD thaekae GCD kaibhaaabaek aalaaadaaa?

- **GDD (Game Design Document)**: paura gaemaera baisataaaraita Design -- 50-200+ parissathaaa
- **GCD (Game Concept Document)**: gaemaera sakassaipata dhaaaranaaa -- 2-5 parissathaaa

GCD halao GDD aira sakassaipata sasakarana Development shaurau karaaara aagae GCD laikhauna, GDD parae baaanaaabaena

Game Concept Document aira Sections

1. Game Title (gaemaera naaama)

aapanaaara gaemaera naaama kii habae? naaamatai hatae habae:

- sahaja uchachaaaranayaogaya
- manae raaakhaaa sahaja
- gaemaera Theme parataiphalaita karae
- ananaya (Unique)

udaaaharana:

- "Phantom's Grasp" (Horror)
- "Pixel Legends" (RPG)
- "Speed Demon" (Racing)

2. One-Line Pitch (aika laaainaera paicha)

aapanaaara gaemakae aika laaainae baojhaaana aitai habae aapanaaara "Elevator Pitch"

Formula:

[Game

udaaaharana:

- "Phantom's Grasp is a Survival Horror game where you survive a haunted asylum using only your wits and limited resources."
- "A puzzle game where you manipulate time to solve impossible challenges."
- "An RPG where your choices literally reshape the world around you."

3. Short Description (sakassaipata baibarana)

3-5 laaainae aapanaaara gaema baojhaaana aitai Steam Store Page Description aira matao habae

aikatai bhaaalao Short Description ai thaaakabae:

- Game Concept (1 laaaina)
- Key Features (2-3 baulaeta)
- Unique Selling Point (1 laaaina)
- Call to Action (1 laaaina)

4. Genre (dharana)

aapanaaara gaema kaona Genre aira anataragata?

****Genre nairadhaaarana karauna:****

- Primary Genre: mauula Genre (Horror, Action, RPG)
- Sub-Genre: upa-Genre (Survival Horror, Action RPG)
- Tags: atairaikata Tags (Souls-like, Metroidvania, Roguelike)

5. Platform (palayaaatapharama)

aapanaaara gaema kaona Platform ai Release habae?

****Platform Options:****

- PC (Windows, Mac, Linux)
- Mobile (Android, iOS)
- Console (PlayStation, Xbox, Nintendo Switch)
- Web Browser

6. Target Audience (lakassaya sharaotaaa)

aapanaaara gaema kaaara janaya?

****Target Audience sajanyaaa:****

- Age Range: 16-35
- Gamer Type: Casual, Core, Hardcore
- Interests: Horror fans, RPG enthusiasts, Puzzle lovers
- Platform Preference: PC gamers, Mobile gamers

7. Game Length (gaemaera daairaghaya)

aapanaaara gaema katakassana chalabae?

****Game Length Categories:****

- Short (2-5 hours): \$5-15 price range
- Medium (8-15 hours): \$15-25 price range
- Long (20+ hours): \$25-60 price range
- Endless: Multiplayer, Roguelike

8. USP (Unique Selling Point)

aapanaaara gaema anayadaera thaekae kaiibhaaabaee aalaaadaaaa?

****USP Examples:****

- Unique Mechanic: "Time manipulation mechanic"
- Art Style: "Hand-painted watercolor visuals"
- Story: "Story that changes based on player psychology"
- Innovation: "First horror game with smell-based mechanics"

9. Core Gameplay (mauula gaemapalae)

aapanaaara gaemae palaeyaaara paradhaaanata kaii karabae?

****Core Gameplay Description:****

- Primary Action: Explore, Fight, Solve, Build
- Secondary Actions: Craft, Socialize, Collect
- Game Loop: Explore -> Discover -> Challenge -> Reward

10. Story (galapa)

aapanaaara gaemae kaonao galapa aachhae kai?

****Story Elements:****

- Setting: gaemataaaa kaothaaaya ghatachhae?
- Characters: kaaaraaaa aachhae?
- Conflict: mauula samasayaaa kaii?
- Goal: palaeyaaaraera lakassaya kaii?

11. Visual Style (bhaijauyaaala sataaaila)

gaemataaaa kaemana daekhaaaba?

****Visual Style Options:****

- Realistic
- Stylized
- Pixel Art
- Low Poly
- Hand-drawn
- 2D/3D

12. Audio Style (adaiau sataaaila)

gaemaera saaaunada kaemana habae?

****Audio Elements:****

- Music Style: Orchestral, Electronic, Ambient
- Sound Effects: Realistic, Stylized
- Voice Acting: Yes/No, Full/Partial

13. Monetization (aayaera padadhatai)

gaema thaekae kaiibhaaaba? taaakaaa aaya habae?

****Monetization Models:****

- Premium: aikabaaarae paemaenata
- Free-to-Play: bainaaamauulayae + In-App Purchase

- Subscription: maaasaika phai
- DLC/Expansions: atairaikata Content

14. Development Scope (daebhaelapamaenata sakaopa)

gaemataaa katataaa bada?

****Scope Categories:****

- Micro: 1-3 months, 1 developer
- Small: 3-6 months, 1-3 developers
- Medium: 6-18 months, 3-10 developers
- Large: 18+ months, 10+ developers

Sample Concept Document: Horror Game

naichae aikatai Complete Sample Concept Document daeayuyaaa halao:

GAME CONCEPT DOCUMENT
"PHANTOM'S GRASP"

Game Title

****PHANTOM'S GRASP****

One-Line Pitch

"Survive a haunted psychiatric hospital where your sanity is your most valuable resource and every shadow hides a deadly secret."

Short Description

****Phantom's Grasp**** is a First-Person Survival Horror game set in an abandoned psychiatric hospital. Players must navigate dark corridors, solve environmental puzzles, and survive against supernatural entities while managing their sanity meter.

****Key Features:****

- ****Sanity System****: Your mental state affects gameplay -- hallucinations, distorted reality, and unpredictable events
- ****Dynamic Haunting****: The hospital changes each playthrough -- no two experiences are the same
- ****Resource Scarcity****: Limited flashlight batteries, few healing items -- every decision matters
- ****Multiple Endings****: Your choices determine which of the 5 endings you unlock

****Unique Selling Point****: "The only horror game where your fear literally changes the game world."

Genre

- ****Primary****: Survival Horror
- ****Sub-Genre****: Psychological Horror, Exploration Horror

- **Tags:** Atmospheric, Story Rich, Difficult, First-Person

Platform

- **Primary:** PC (Steam)
- **Secondary:** PlayStation 5, Xbox Series X|S
- **Tertiary:** Nintendo Switch (if scope allows)

Target Audience

- **Age:** 18-35
- **Gamer Type:** Core to Hardcore
- **Interests:** Horror games, Thriller movies, Psychological stories
- **Platform:** PC and Console gamers
- **Play Style:** Solo, Immersive, Story-driven

Game Length

- **Main Story:** 8-12 hours
- **Completionist:** 15-20 hours
- **Replay Value:** High (Multiple endings, Randomized events)

USP (Unique Selling Point)

"Sanity-Driven Horror"

Unlike traditional horror games with jump scares, Phantom's Grasp uses a Sanity System that dynamically changes the game world based on your mental state. The more scared you are, the more the hospital changes -- creating a truly personalized horror experience.

Core Gameplay

- Primary Actions:**
- **Explore:** Navigate through the hospital's dark corridors and rooms
 - **Survive:** Avoid or escape from supernatural entities
 - **Solve:** Environmental puzzles to progress
 - **Manage:** Balance sanity, health, and resources

Game Loop:

Manage Sanity -> Find Resource -> Progress -> Repeat

Story

- Setting:** Blackwood Psychiatric Hospital, abandoned in 1972 after a mysterious incident killed 47 patients and staff.
- Protagonist:** Dr. Sarah Chen, a psychologist investigating her sister's disappearance at Blackwood.
- Conflict:** Something ancient stirs within the hospital's walls. The boundaries between reality and nightmare blur as Sarah delves deeper into the hospital's dark past.
- Goal:** Uncover the truth about Blackwood, find Sarah's sister, and escape alive.

Themes: Sanity vs. Madness, Truth vs. Delusion, Guilt and Redemption

Visual Style

- **Perspective:** First-Person
- **Art Direction:** Realistic with Psychological Horror Elements
- **Lighting:** Dynamic shadows, limited flashlight
- **Color Palette:** Desaturated, cold tones with occasional warm highlights
- **Reference Games:** Outlast, Amnesia, P.T.

Audio Style

- **Music:** Ambient, Minimalistic, Unnerving
- **Sound Effects:** Realistic environmental sounds, distorted audio when sanity drops
- **Voice Acting:** Full voice acting for main characters
- **Reference:** Silent Hill 2, Layers of Fear

Monetization

- **Model:** Premium (\$24.99)
- **DLC Plan:** Free content updates for 6 months post-launch
- **Season Pass:** Optional expansion (\$9.99) with new story chapter
- **No Microtransactions:** Pure horror experience

Development Scope

- **Team Size:** 5 people
- **Development Time:** 18 months
- **Engine:** Unreal Engine 5
- **Budget:** \$200,000
- **Milestone:** Vertical Slice in 6 months

Practical Exercise

anaushailana 1: aapanaaara naijaera Concept Document laikhauna

nairadaeshanaaa:

1. aapanaaara gaemaera janaya uparaera saba Sections pauurana karauna
2. parataitai Section kamapakassae 2-3 laaainae laikhauna
3. One-Line Pitch taairai karauna
4. 5 janakae daiyae Read karaaana aiba Feedback naina

anaushailana 2: One-Line Pitch Practice

nairadaeshanaaa:

naichaera Formula bayabahaaara karae 5tai bhainana One-Line Pitch laikhauna:

****Example:****

"Shadow Realm is a Roguelike RPG where you fight through procedurally generated dungeons with a card-based combat system."

anaushailana 3: USP Definition

****nairadaeshanaaa:****

1. aapanaaara gaemaera 3tai Potential USP laikhauna
2. parataitai USP kae 1 laaainae Explain karauna
3. baechhae naina sabachaeyae shakataishaaalarii aikatai

Common Mistakes (saaadhaaarana bhaula)

1. atairaikata Vague haauyaaa

bhaula: "aikatai bhaaalao gaema habae"

sathaika: "8-12 ghanataaara First-Person Horror gaema, Sanity System saha"

2. Feature Overload

bhaula: "saba Feature thaaakabae -- Crafting, Multiplayer, PvP, PvE..."

sathaika: "mauula 3tai Feature ai Focus karauna, baaakaigaulao Optional"

3. Comparable Games naaa baolaaa

bhaula: "aamaaara gaema ananaya, airakama kaichhau naei"

sathaika: "Outlast + Sanity System = Phantom's Grasp"

4. Unrealistic Scope

bhaula: "3 maaasae paurao RPG baaanaaabao"

sathaika: "18 maaasae Horror Game, 5 janaera taima"

5. Story baaa Mechanic aikatai baaada daeauyaaa

bhaula: "Story laagabae naaa, shaudhau Gameplay aachhae"

sathaika: "Story aiba Gameplay dautaoi gaurautabapaurana"

Pro Tips (paeshaadaara taipasa)

1. Comparable Games sabasamaya bayabahaaara karauna

Publishers aiba Investors baojhatae chaaaya "aitaaa kaaara matao?"

"Outlast + Phasmophobia = Our Game" -- aibhaaabae baojhaaaana

2. One-Line Pitch Perfect karauna

aikatai bhaaalao One-Line Pitch 30 saekanaadae gaema baojhaaatae paaarae

aitai aapanaaara sabachaeyae gaurautabapaurana Sentence

3. Concept Document kae Living Document baaanaaana

yata baeshai jaaanabaena, Concept Document aapadaeta karauna
aitai paraibaratana haauyaaa sabaaabhaaabaika

4. Visual Reference yaoga karauna

shaudhau laikhae naya, Reference Images yaoga karauna
Mood Board taairai karauna

5. Target Audience kae Specific karauna

"sabaaara janaya" -- aitari kaaaja karae naaa
"18-35 bachhara bayasaii Horror Game paraemaidara janaya" -- aitari kaaaja karae

6. Development Scope yathaaarathabaaadaii raaakhauna

aapanaaara parathama gaema yata chhaota habae, tata bhaaalao
"Scope Down, Quality Up" -- aitaai manatara

Chapter Summary (adhayaaaya saaraaasha)

mauula paaayaenata:

1. **Game Concept Document** halao aapanaaara gaemaera samapauurana dhaaaranaaara Written Record
2. **14tai Section** aachhae yaegaulao pauurana karatae habae
3. **One-Line Pitch** halao sabachaeyae gaurautabapauurana -- aitari Perfect karauna
4. **Comparable Games** bayabahaaara karae baojhaaana aapanaaara gaema kaemana habae
5. **USP** nairadhaaarana karauna -- aapanaaara gaema kaiibhaaabaee aalaaadaaa
6. **Scope** yathaaarathabaaadaii raaakhauna -- chhaota shaurau karauna
7. **Concept Document** kae Living Document baaanaana -- naiyamaita aapadaeta karauna

parabarataii adhayaaaya:

adhayaaaya 5 ai aamaraaa daekhabao **Game Pitch** kaibhaaabaee karabaena -- maaatara 30 saekaenadae
aapanaaara gaema kaaaukae baojhaaanao

***manae raaakhabeena**: aikatai bhaaalao Concept Document chhaadaaa bhaaalao Game taairai kathaina aapanaaara
dhaaaranaaake laikhae raaakhauna -- aitari aapanaaara gaaaida habae samapauurana Development Cycle jaudae*

PART 5

adhayaaaya 5: GAME PITCH -- 30 saekadena Game baohhaanao

adhayaaaya 5: GAME PITCH -- 30 saekadena Game baohhaanao

shaekhaara udadaeshaya (Learning Goal)

aii adhayaaaya shaessae aapanai jaaanabaena:

- Game Pitch kaii aiba kaena aitai gaurautabapauurana
- 30 saekadena, 1 mainaita, 3 mainaita, 5 mainaita Pitch kaibhaaaba taairai karabaena
- Pitch Structure kaii aiba kaibhaaaba bayabahaara karabaena
- Professional Pitch kaibhaaaba daebaena

dhaaranaaa bayaaakhayaaa (Concept Explanation)

Game Pitch kaii?

Game Pitch halao apanaara gaemakae sakassaipatabhaaaba kaaaukae baohhaanaora kaaushala aitai aikatai Presentation yaekhaanae aapanai 30 saekadena thaekae 5 mainaitaera madhayae apanaara gaemaera sabakaichhau baohhaana

WHY GAME PITCH MATTERS

Without Pitch:

Cannot explain your game quickly
Publishers/Investors lose interest
Team members confused about direction
Miss opportunities

With Good Pitch:

Grab attention in 30 seconds
Convince publishers to fund your game
Align team vision
Get Media Coverage

Pitch Structure

aikatai bhaaalao Game Pitch ai 7tai mauula Element thaaakae:

GAME PITCH STRUCTURE

1. Problem (samasayaaa)

kaona Problem Solve karachhae?
2. Fantasy (kalapanaaaa)

palaeyaaara kaii anaubhaba karabae?
3. Gameplay (gaemapalae)

palaeyaaara kaii karabae?
4. Unique Feature (ananaya baaishaissataya)

anayadaera thaekae kaiibhaaabaee aalaaadaaa?
5. Audience (darashaka)

kaaara janaya?
6. Market Opportunity (maaarakaeeta sauyaoga)

kaena aikhana? kaena aikhaanaae?
7. Business Model (bayabasaayaika madaela)

kaiibhaaabaee taaakaaa aaya habae?

Pitch Versions

Version 1: 30-Second Pitch (ilaebhaetara paicha)

****udadaeshaya****: laiphatae baaa sakassaipata samayae gaema baojhaaanao

****Structure:****

****Formula:****

Imagine [Game Concept].
Our game [Unique Feature].
It's like [Comparable Game] but [Differentiation]."

****Example (Horror Game):****

Imagine a horror game where YOUR fear literally changes the game world.

Our game 'Phantom's Grasp' uses a Sanity System -- the more scared you get, the more the hospital distorts and changes around you.

It's like Outlast meets Phasmophobia, but the horror adapts to YOU.

Can I show you a 30-second demo?"

****Example (RPG Game):****

Imagine an RPG where your decisions don't just change dialogue -- they physically change the landscape, NPCs, and even gameplay mechanics.

Our game 'Echoes of Choice' uses a Propagation System where one choice creates ripple effects throughout the entire game world.

It's like The Witcher 3 meets BioShock, but with real consequences.

Want to see how it works?"

Version 2: 1-Minute Pitch (darauta paicha)

****udadaeshaya****: baeshai tathaya daiyae gaema baojhaaanao

****Structure:****

[Comparable Games] + [Target Audience] + [Business Model] + [Call to Action]

****Example (Horror Game):****

actually made you afraid -- not just startled, but genuinely afraid?

[Pause for response]

Most horror games rely on jump scares. They startle you, then you move on. The fear never lasts.

What if I told you about a game where YOUR fear becomes the game itself?

'Phantom's Grasp' is a First-Person Survival Horror game set in an abandoned psychiatric hospital. But here's what makes it different: we have a Sanity System.

The more scared you get, the more the hospital changes. Doors appear where there were walls. Hallways stretch longer. Monsters become more aggressive. Your fear literally reshapes reality.

Think Outlast meets Phasmophobia, but the horror adapts to your psychology.

Our target audience is 18-35 year old horror fans who want more than just jump scares. This is a Premium game at \$24.99 on PC and Console.

We're a team of 5 with 18 months development time. We're looking for a publishing partner who understands the horror genre.

Can I show you our Vertical Slice demo?"

Version 3: 3-Minute Pitch (pauurana paicha)

****udadaeshaya****: baisataaaraita baojhaaanao, Publisher baaa Investor aira janaya

****Structure:****

[Problem - 30 sec]
[Solution/Game Concept - 45 sec]
[Unique Features - 45 sec]
[Market Opportunity - 30 sec]
[Comparable Games - 15 sec]
[Target Audience - 15 sec]
[Development Status - 15 sec]
[Business Model - 15 sec]
[Call to Action - 15 sec]

****Example (Horror Game):****

"Have you ever played a horror game that truly scared you -- not just startled you with a jump scare, but made you afraid to continue? Most horror games today rely on cheap tactics. Let me tell you about a game that brings back REAL fear."

[PROBLEM - 30 sec]

"The horror genre is oversaturated with jump scare compilations. Players are desensitized. They play horror games for content, not because they're actually scared. Reviewers complain that modern horror games lack genuine atmosphere. Players want to be afraid, but games keep giving them the same old formula."

[SOLUTION/GAME CONCEPT - 45 sec]

"Phantom's Grasp is a First-Person Survival Horror game that brings back genuine psychological horror. Set in Blackwood Psychiatric Hospital, abandoned since 1972 after a mysterious incident killed 47 people.

You play as Dr. Sarah Chen, a psychologist investigating her sister's disappearance. As you explore the hospital, you'll encounter puzzles, collect clues, and survive against supernatural entities.

But here's what makes it special: our Sanity System."

[UNIQUE FEATURES - 45 sec]

"Our Sanity System tracks your psychological state in real-time.

When your sanity drops:

- Hallucinations appear -- things that aren't really there
- The hospital physically changes -- new rooms, shifting corridors
- Enemies become more aggressive and harder to predict
- Sound design distorts -- voices become whispers, footsteps echo

This means every player has a DIFFERENT experience based on how they react to fear. Some players might trigger hallucinations early, others might maintain sanity longer.

We also have Dynamic Haunting -- the hospital layout changes each playthrough. No two playthroughs are identical."

[MARKET OPPORTUNITY - 30 sec]

"The horror game market is booming. Games like Phasmophobia, Layers of Fear, and Outlast have proven there's huge demand. Phasmophobia alone made over \$100 million. But most of these games rely on multiplayer or jump scares.

There's a gap for single-player, psychological horror with deep mechanics. That's exactly where Phantom's Grasp fits."

[COMPARABLE GAMES - 15 sec]

"Our closest comparables are Outlast (atmosphere, setting), Layers of Fear (psychological horror), and Phasmophobia (sanity-based mechanics). We're Outlast's tension meets Layers of Fear's psychological depth."

[TARGET AUDIENCE - 15 sec]

"Our target is 18-35 year old core to hardcore gamers who love horror. These are players who've played every Outlast, every Amnesia, and want something that genuinely scares them again."

[DEVELOPMENT STATUS - 15 sec]

"We're a team of 5 developers. We've been in development for 12 months. Our Vertical Slice is complete and playable. We're seeking a publishing partner for Q3 2027 release."

[BUSINESS MODEL - 15 sec]

"Premium game at \$24.99 on Steam, PlayStation 5, and Xbox Series X. No microtransactions -- pure horror experience. Post-launch DLC planned for 12 months."

[CALL TO ACTION - 15 sec]

"Can I show you our 5-minute gameplay demo?
I'd love to hear your thoughts on the Sanity System."

Version 4: 5-Minute Pitch (baisataaaraita paicha)

****udadaeshaya****: pauurana baishalaessana, Demo saha

****Structure:****

- [0:30-1:00] Problem
- [1:00-2:00] Solution & Game Concept
- [2:00-3:00] Unique Features & Mechanics
- [3:00-3:30] Market Analysis
- [3:30-4:00] Comparable Games & Differentiation
- [4:00-4:30] Target Audience & Marketing
- [4:30-4:45] Development Status & Team
- [4:45-5:00] Business Model & Call to Action

****Key Points for 5-Minute Pitch:****

- parataitai Section aira janaya sathaika samaya dharauna
- Demo daekhaaana yadai samabhaha haya
- Questions aira janaya parasatauta thaaakauna
- Technical Details raaakhauna kainatau Overwhelm karabaena naaa
- Passion daekhaaana -- aapanai kaena aii gaema baaanaaachachhaena

Pitch Structure Breakdown

PITCH TIMING GUIDE

- 30-Second Pitch:
 - Hook: 5 sec Concept: 10 sec
 - Unique: 10 sec CTA: 5 sec
- 1-Minute Pitch:
 - Hook: 10 sec Problem: 10 sec
 - Solution: 15 sec Unique: 10 sec
 - Market: 10 sec CTA: 5 sec
- 3-Minute Pitch:
 - Hook: 15 sec Problem: 30 sec
 - Solution: 45 sec Features: 45 sec
 - Market: 30 sec Comparables: 15 sec
 - Audience: 15 sec Status: 15 sec
 - Business: 15 sec CTA: 15 sec
- 5-Minute Pitch:
 - Hook: 30 sec Problem: 30 sec
 - Solution: 60 sec Features: 60 sec
 - Market: 30 sec Comparables: 30 sec
 - Audience: 30 sec Status: 15 sec
 - Business: 15 sec CTA: 15 sec

Pitch Tips by Situation

Publisher Pitch

- Focus on: Market Opportunity, Sales Potential, Comparable Games
- Show: Vertical Slice Demo
- Mention: Release Timeline, Marketing Plan
- Avoid: Too much technical detail, personal stories

Investor Pitch

- Focus on: Business Model, Revenue Potential, ROI

Show: Financial Projections
Mention: Team Background, Market Size
Avoid: Game mechanics deep dive, art style discussion

Media Pitch

Focus: Unique Features, Story, Visual Style
Show: Screenshots, Trailer
Mention: Release Date, Platform
Avoid: Business details, financial projections

Team Pitch

Focus: Vision, Gameplay, Creative Direction
Show: Concept Art, Prototypes
Mention: Development Timeline, Roles
Avoid: Business details, market analysis

Practical Exercise

anaushailana 1: 30-Second Pitch Write

****nairadaeshanaaa:****

1. aapanaaara gaemaera janaya 30 saekaenadaera Pitch laikhauna
2. Formula bayabahaaara karauna: `Hook + Concept + Unique Feature + CTA`
3. Mirror aira saaamanae parayaaakataisa karauna
4. taaaimaaara raaakhauna -- 30 saekaenadaera baeshai habae naaa

anaushailana 2: Pitch Variations

****nairadaeshanaaa:****

aikai gaemaera janaya 4tai bhainana Pitch laikhauna:

- 30-Second Pitch
- 1-Minute Pitch
- 3-Minute Pitch
- 5-Minute Pitch

anaushailana 4: Pitch Practice

****nairadaeshanaaa:****

1. 5 jana banadhaukae aapanaaara 30-Second Pitch shaonaaana
2. taaadaera jajjanyaasaaa karauna: "aapanai kai aii gaema khaelatae chaaana? kaena?"
3. Feedback naina aiba Pitch aapadaeta karauna

Common Mistakes (saaadhaaarana bhaula)

1. atairaikata Technical haauyaaa

bhaua: "aamaraa Custom Physics Engine bayabahaaara karachhai..."
sathai: "aamaadaera Sanity System aapanaara Fear Level Track karae"

2. Feature Overload

bhaua: "Crafting, Multiplayer, PvP, PvE, Open World, 100+ Hours..."
sathai: "maula Feature: Sanity System yaa Horror Experience badalae daeya"

3. Passion naaa daekhaanao

bhaua: Monotone voice, no eye contact, reading from paper
sathai: Enthusiastic voice, eye contact, direct communication

4. Comparable Games naaa jaaanaaa

bhaua: "airakama aagae kaichhau hayana"
sathai: "Outlast + Layers of Fear + Sanity System = Phantom's Grasp"

5. CTA (Call to Action) naaa daeayaaa

bhaua: Pitch shaessa karae chaupa karae basae thaaakaaa
sathai: "aapanai kai Demo daekhatae chaaana? aamaadaera saaathae Partner hatae chaaana"

6. Audience naaa baojhaaa

bhaua: sabaaara janaya aikai Pitch
sathai: Publisher, Investor, Media -- paratayaekaera janaya aalaaadaaa Pitch

Pro Tips (paeshaadaara taipasa)

1. Hook paaaraphaekata karauna

parathama 5 saekanaa sabachaeyae gaurautabapaurana
Question daiyae shaurau karauna -- manaoyaoga kaaadauna

2. Narrative Voice bayabahaaara karauna

shaudhau Feature taaalaikaaa naya -- galapa balauna
"Let me tell you about a game that..." -- aibhaabae shaurau karauna

3. Visuals bayabahaaara karauna

shaudhau kathaaa naya, Screenshot daekhaana
Demo chaaalaaana yadai samabhava haya

4. Time raesapaekata karauna

30 saekanaa Pitch 35 saekanaa halae samasayaaa
Practice karauna -- sathai samayae shaessa karauna

5. Passion daekhaana

aapanai yadai naijaei Excited naaa hana, anaya kae habae naaa
aapanaara gaemakae bhaaalaobaasauna

6. Questions aira janaya parasatauta thaaakauna

Pitch shaessae Questions aasabae

parataitai Question aira Answer parasatauta raaakhauna

Chapter Summary (adhayaaaya saaaraaasha)

mauula paaayaenata:

1. **Game Pitch** halao aapanaaara gaemakae sakassaipatabhaaaba baajhaanaora kaaushala
2. **4tai Version** aachhae: 30-sec, 1-min, 3-min, 5-min
3. **7tai Structure Element**: Problem, Fantasy, Gameplay, Unique Feature, Audience, Market, Business Model
4. **Audience anauyaaayai Pitch** paraibaratana karauna -- Publisher, Investor, Media
5. **Hook** sabachaeyae gaurautabapauurana -- parathama 5 saekaenadae manaoyaoga kaaadauna
6. **Passion** daekhaana -- aapanai yadai Excited naaa hana, anaya habae naaa
7. **Practice** karauna -- parataidaina parayaaakataisa karauna

parabarataii adhayaaaya:

adhayaaaya 6 ai aamaraa daekhabao **Game Design Document (GDD)** -- paura gaemaera Design kaibhaaaba Document karabaena

***manae raaakhabaena**: aikatai bhaaalao Game Pitch aapanaaara gaemaera sabachaeyae bada haaataiyaaara 30 saekaenadae baajhaatae paaaralae aapanai yaekaonao Publisher baa Investor aira manaoyaoga kaaadatae paaarabaena*

PART 6

adhayaaaya 6: GAME DESIGN DOCUMENT (GDD) -- gaema daijaaaina dakaumaenata

adhayaaaya 6: GAME DESIGN DOCUMENT (GDD) -- gaema daijaaaina dakaumaenata

shaekhaaara udadaeshaya (Learning Goal)

aii adhayaaaya shaessae aapanai jaaanabaena:

- Game Design Document (GDD) kaii aiba kaena aitai parayaojana
 - GDD aira 36tai Section aiba parataitai kaibhaaaba paaurana karabaena
 - parataitai Section aira saaathae Example saha bayaaakhayaaa
 - GDD kaibhaaaba Maintain aiba Update karabaena
-

dhaaaranaaa bayaaakhayaaa (Concept Explanation)

Game Design Document (GDD) kaii?

GDD halao aapanaaara gaemaera samapauurana Blueprint -- paurao gaema kaibhaaaba kaaaja karabae, kaemana daekhaaaba, palaeyaaara kaii karabae, sabakaichhau aikatai Document ai aitai aapanaaara taimaera janaya Reference Guide

WHY GDD MATTERS

Without GDD:

- Team members have different interpretations
- Features keep changing randomly
- No clear development direction
- Difficult to onboard new team members
- Scope creep without documentation

With GDD:

- Single source of truth for entire team
- Clear, documented decisions
- Reference throughout development
- Easy to communicate vision
- Controlled scope with documentation

GDD aira dharana (Types of GDD)

GDD TYPES

High-Level GDD (Concept Phase):
5-10 pages
Core vision and concept
Target audience and market
Basic gameplay description
Detailed GDD (Pre-Production):
30-50 pages
Complete mechanics description
Level design overview
Art and audio direction
Technical requirements
Living GDD (Production):
50-200+ pages
Updated as development progresses
Version controlled
Referenced by all departments

GDD aira 36tai Section

Section 1: Game Overview (gaemaera saarakassaepa)

bayaaakhayaaa: gaematai sakassaepae kaii -- aikatai Paragraph ai

kaibhaaabae pauurana karabaena:

- gaemaera naaama
- Genre
- Platform
- Target Audience
- Game Length
- Price Point

udaaaharana:

Genre: First-Person Survival Horror
Platform: PC (Steam), PlayStation 5, Xbox Series X|S
Target Audience: 18-35 year old horror fans
Game Length: 8-12 hours main story, 15-20 hours completionist
Price: \$24.99

Section 2: Vision (darissatai)

bayaaakhayaaa: aapanaaara gaemaera mauula darissatai kaii -- kaena aitai taairai karachhaena?

kaibhaaabae pauurana karabaena:

- Vision Statement (1-2 laaaina)
- Core Experience (palaeyaaara kaii anaubhaba karabae)
- Reference Games (airakama gaema thaekae inspiration naiyaechhai)

udaaaharana:

not just startles."

Core Experience: Players should feel isolated, paranoid, and genuinely afraid to continue. The game should make them question what's real.

References: Outlast (atmosphere), Layers of Fear (psychological), Amnesia (sanity mechanics)

Section 3: Design Pillars (daijaaaina satamabha)

bayaaakhayaaa: aapanaaara gaemaera 3-5tai mauula naitai yaaa saba Design Decision kae nairadaesha karabae

kaibhaaabae pauurana karabaena:

- parataitai Pillar aira naaama
- sakassaipata bayaaakhayaaa
- aitai kaibhaaabae gaemae parataiphalaita habae

udaaaharana:

-- Players never know what's real. Hallucinations blend with reality.

Pillar 2: "Player Agency Matters"

-- Every choice has consequences. No hand-holding.

Pillar 3: "Atmosphere Over Action"

-- Quiet moments build tension. Combat is last resort.

Pillar 4: "Respect the Player"

-- No cheap jumpscares. Fair difficulty. Rewarding exploration.

Section 4: Target Audience (lakassaya sharaotaaa)

bayaaakhayaaa: aapanaaara gaema kaaara janaya?

kaibhaaabae pauurana karabaena:

- Age Range
- Gamer Type (Casual/Core/Hardcore)
- Interests
- Platform Preference
- Play Style
- Gaming Habits

udaaaharana:

Interests: Horror games, thriller movies, mystery novels

Play Style: Solo, immersive, story-driven

Platform: PC and Console gamers

Gaming Habits: 2-4 hours per session, evening/night play

Section 5: Genre (dharana)

bayaaakhayaaa: aapanaaara gaema kaona Genre aira?

kaibhaaabae pauurana karabaena:

- Primary Genre
- Sub-Genre

- Genre Blend (yadai thaaakae)
- Tags

****udaaaharana:****

Sub: Psychological Horror, Exploration Horror
Blend: Horror + Puzzle + Narrative
Tags: Atmospheric, Story Rich, Difficult, First-Person, Singleplayer

Section 6: Platforms (palayaaatapharama)

****bayaaakhayaaa:**** gaematai kaona Platform ai Release habae?

****kaibhaaabae pauurana karabaena:****

- Primary Platform
- Secondary Platforms
- Minimum Hardware Requirements
- Recommended Hardware Requirements

****udaaaharana:****

Secondary: PlayStation 5, Xbox Series X|S
Tertiary: Nintendo Switch (if scope allows)

PC Minimum: Windows 10, Intel i5-8400, 8GB RAM, GTX 1060
PC Recommended: Windows 11, Intel i7-10700, 16GB RAM, RTX 3060

Section 7: Core Gameplay (mauula gaemapalae)

****bayaaakhayaaa:**** palaeyaaara paradhaaanata kii karabae?

****kaibhaaabae pauurana karabaena:****

- Primary Actions (mauula kaaaja)
- Secondary Actions (paaarashaba kaaaja)
- Core Loop (mauula chakara)
- Player Motivation (kaena karabae)

****udaaaharana:****

- Explore: Navigate dark corridors and rooms
- Survive: Avoid or escape supernatural entities
- Solve: Environmental puzzles to progress
- Manage: Balance sanity, health, and resources

Core Loop:
Explore -> Discover Clue -> Encounter Threat -> Solve Puzzle ->
Manage Sanity -> Find Resource -> Progress -> Repeat

Section 8: Gameplay Loop (gaemapalae laupa)

****bayaaakhayaaa:**** palaeyaaara kii baaarabaaara karabae?

****kaibhaaabae pauurana karabaena:****

- Micro Loop (paratai mauhauurata)
- Macro Loop (paratai Session)
- Meta Loop (Long-term)

****udaaaharana:****

Move -> Check Corner -> Listen -> React -> Continue

Macro Loop (1 hour):
Enter Area -> Explore -> Find Key -> Unlock Door ->
Face Challenge -> Solve -> Get Reward -> New Area

Meta Loop (Full Game):
Investigate Clue -> Unlock Area -> Learn Truth ->
Face Boss -> Overcome -> Story Progress -> Repeat

Section 9: Mechanics (maekaaanaikasa)

****bayaaakhayaaa:**** gaemaera naiyamakaaanauna kaii?

****kaibhaaabae pauurana karabaena:****

- parataitai Mechanic aira naaama
- kaibhaaabae kaaaja karae
- kaena gaurautabapauurana
- Example

****udaaaharana:****

- How it works: Tracks player's mental state (0-100)
- When sanity drops below 50: Visual distortions begin
- When sanity drops below 25: Hallucinations appear
- When sanity reaches 0: Game Over
- Recovery: Find Sanity Items, solve puzzles correctly

Mechanic 2: Resource Management

- Flashlight: Limited battery, must find batteries
- Healing: Limited medkits, must find or craft
- Inventory: 12 slots, must prioritize items

Section 10: Player Controls (palaeyaaara kanataraola)

****bayaaakhayaaa:**** palaeyaaara kaiibhaaabae gaema khaelabae?

****kaibhaaabae pauurana karabaena:****

- Keyboard/Mouse Controls
- Controller Layout
- Mobile Touch Controls (yadai parayaojaya)

****udaaaharana:****

WASD - Move
Mouse - Look
Left Click - Interact/Use
Right Click - Flashlight Toggle
Space - Interact

Shift - Sprint
C - Crouch
Tab - Inventory
M - Map
Esc - Pause Menu

Section 11: Camera (kayaaamaeraaa)

bayaaakhayaaa: gaemae kayaaamaeraaa kaemana thaaakabae?

kaibhaaabae pauurana karabaena:

- Perspective (First/Third Person)
- Camera Behavior
- Field of View
- Camera Effects

udaaaharana:

FOV: 75 degrees (adjustable 60-90)
Camera Shake: Subtle, increases with low sanity
Head Bob: Minimal, realistic walking motion
Camera Effects:
- Low Sanity: Slight chromatic aberration, film grain
- Very Low Sanity: Screen distortion, color shifts

Section 12: Character System (charaitara saisataema)

bayaaakhayaaa: palaeyaaara Character aiba NPCs kaemana?

kaibhaaabae pauurana karabaena:

- Player Character
- Main NPCs
- Antagonists
- Character Progression

udaaaharana:

Name: Dr. Sarah Chen
Age: 32
Background: Psychologist, former Blackwood employee
Motivation: Find missing sister
Personality: Analytical, determined, haunted by past

Main NPCs:
- Dr. James Morrison: Former colleague, mentor figure
- Patient Zero: Mysterious entity, main antagonist
- Dr. Elena Vasquez: Previous researcher, left audio logs

Section 13: Enemy System (shatarau saisataema)

bayaaakhayaaa: gaemae kaonao Enemy thaaakalae taaaraaa kaemana?

kaibhaaabae pauurana karabaena:

- Enemy Types

- Behavior Patterns
- Difficulty Scaling
- Player Countermeasures

****udaaaharana:****

- Appearance: Humanoid shadow, distorts light
- Behavior: Patrols corridors, attracted to noise
- Detection: Sound and sight
- Counter: Hide, stay quiet, use distractions
- Weakness: Light sources reduce effectiveness

Enemy Type 2: The Phantom

- Appearance: Glitching humanoid, shifts between forms
- Behavior: Appears randomly, blocks paths
- Detection: Sanity-based (appears when sanity is low)
- Counter: Maintain sanity, solve puzzles to banish

Section 14: Combat (yaudadha)

****bayaaakhayaaa:**** gaemae kaonao yaudadha thaaakalae taaa kaemana?

****kaibhaaabaee pauurana karabaena:****

- Combat Type
- Weapons/Tools
- Damage System
- Combat Flow

****udaaaharana:****

Primary Approach: Run, hide, outsmart

Weapons: None (survival horror, not action)

Tools: Flashlight (temporary stun), Distraction items

Damage System: No health bar for enemies, instant kill on contact

Combat Flow: Detect -> Panic -> Hide -> Wait -> Escape

Section 15: Inventory (inabhaenatarai)

****bayaaakhayaaa:**** palaeyaaara kaii jainaisa bahana karabae?

****kaibhaaabaee pauurana karabaena:****

- Inventory Size
- Item Types
- Item Management
- Storage System

****udaaaharana:****

Item Types:

- Key Items: Story progression
- Consumables: Medkits, batteries, sanity pills
- Documents: Clues, lore, codes
- Tools: Lockpicks, crowbar (temporary)

Storage: Safe rooms have storage boxes
 Weight System: None (simplified inventory)

Section 16: Health (sabaaasathaya)

****bayaaakhayaaa:**** palaeyaaara kaiibhaaaba Damage paaaba aiba kaiibhaaaba nairaaamaya habae?

****kaiibhaaaba pauurana karabaena:****

- Health System
- Damage Sources
- Healing Items
- Death Consequence

****udaaaharana:****

Damage Sources:

- Enemy contact: 25-50 damage
- Traps: 10-20 damage
- Fall damage: 5-15 damage
- Sanity effects: damage (hallucinations cause mistakes)

Healing:

- Medkit: Restores 50 HP (rare)
- Bandage: Restores 25 HP (uncommon)
- Natural: Slow regeneration in safe rooms

Death: Reload from last checkpoint (no permadeath)

Section 17: Weapons (asatara)

****bayaaakhayaaa:**** gaemae kaonao asatara thaaakalae taaa kaemana?

****kaiibhaaaba pauurana karabaena:****

- Weapon Types
- Ammunition
- Weapon Management
- Weapon Progression

****udaaaharana (yadai thaaakae):****

- Function: Temporarily stuns enemies
- Battery: Drains quickly
- Upgrades: Extended battery, wider beam

Weapon 2: Distraction Items

- Rock: Throw to create noise
- Smoke bomb: Create visual cover
- Firecrackers: Attract enemies to location

No lethal weapons -- survival horror, not action

Section 18: Skills (dakassataaa)

****bayaaakhayaaa:**** palaeyaaara kaonao Skill Upgrade paaaba kai?

****kaibhaaabaepauurana karabaena:****

- Skill Tree
- Skill Types
- Skill Progression
- Skill Effects

****udaaaharana:****

High Sanity Abilities:

- Sharp Focus: Better flashlight battery life
- Calm Mind: Slower sanity drain
- Keen Eye: Highlight interactable objects

Low Sanity Abilities (Risky):

- Danger Sense: Detect enemies through walls
- Phantom Walk: Move quieter
- Time Slip: Brief slow-motion (distorted reality)

Trade-off: Stay sane for safety, or embrace madness for power

Section 19: Progression (paragaraeshana)

****bayaaakhayaaa:** palaeyaaara kaiibhaaabaepaigaiyae yaaabae?**

****kaibhaaabaepauurana karabaena:****

- Progression Type
- Unlock System
- Difficulty Scaling
- Completion Criteria

****udaaaharana:****

Unlock System:

- Key Items unlock new areas
- Clues unlock story progression
- Tools unlock new puzzle types

Difficulty Scaling:

- Enemies become more aggressive as sanity drops
- Puzzles become more complex in later chapters
- Resources become scarcer

Completion:

- 5 different endings based on choices
- Collectibles reveal full story
- Speed run mode unlocked after first completion

Section 20: Economy (arathanaiitai)

****bayaaakhayaaa:** gaemae kaonao Currency baaa Economy aachhae kai?**

****kaibhaaabaepauurana karabaena:****

- Currency Type
- Earning Method

- Spending Method
- Economy Balance

****udaaaharana:****

Resources:

- Batteries: Found, not purchased
- Medkits: Found, not purchased
- Sanity Pills: Found, not purchased
- Key Items: Found through exploration

No shops, no trading, no currency

Pure survival -- find what you need, use it wisely

Section 21: Level Design (laebhaela daijaaaina)

****bayaaakhayaaa:**** gaemaera maaanachaitara kaemana thaaakabae?

****kaibhaaabaes pauurana karabaena:****

- Level Structure
- Level Theme
- Level Progression
- Level Design Principles

****udaaaharana:****

Chapters:

1. Arrival: Hospital entrance, learn basics
2. Ward A: Patient rooms, first enemy encounter
3. Ward B: Restricted area, more complex puzzles
4. Basement: Dark, claustrophobic, high tension
5. Tower: Final area, truth revealed

Design Principles:

- Every room has purpose (story, gameplay, or resource)
- No empty corridors -- every space tells a story
- Multiple paths where possible
- Safe rooms every 10-15 minutes

Section 22: World Design (baishaba daijaaaina)

****bayaaakhayaaa:**** gaemaera World kaemana?

****kaibhaaabaes pauurana karabaena:****

- World Setting
- World Rules
- World Building
- Environmental Storytelling

****udaaaharana:****

Atmosphere: Abandoned, decaying, supernatural

World Rules:

- Time flows differently inside

- Sanity affects perception
- The hospital "remembers" past events

Environmental Storytelling:

- Patient records scattered around
- Blood stains and damage tell past events
- Audio logs reveal history
- Environmental details hint at truth

Section 23: Story (galapa)

****bayaaakhayaaa:**** gaemaera galapa kaii?

****kaibhaaabaepauurana karabaena:****

- Story Synopsis
- Story Structure
- Key Plot Points
- Storytelling Methods

****udaaaharana:****

to find her missing sister. She discovers the hospital's dark past and faces a supernatural entity that feeds on fear.

Structure: 5 Acts

1. Arrival: Learn the hospital, find first clues
2. Discovery: Uncover past experiments
3. Confrontation: Face the entity, learn its nature
4. Revelation: Understand the truth about her sister
5. Escape: Final choice -- ending based on sanity and choices

Storytelling Methods:

- Environmental storytelling
- Audio logs
- Flashbacks
- NPC dialogue (limited)
- Player choices

Section 24: Quest System (kauyaesata saisataema)

****bayaaakhayaaa:**** gaemae kaonao Quest thaaakalae taaa kaemana?

****kaibhaaabaepauurana karabaena:****

- Quest Types
- Quest Structure
- Quest Tracking
- Quest Rewards

****udaaaharana:****

Structure:

- Main Quest: Find sister, escape hospital
- Side Clues: Optional lore documents
- Environmental Puzzles: Unlock areas

No traditional quest log -- player tracks progress through:

Synops

Quest

- Journal (auto-updates with discoveries)
- Map (marks key locations)
- Environmental cues

Rewards:

- Story progression
- New areas unlocked
- Key items found
- Lore revealed

Section 25: UI/UX (iuaai/iuaikasa)

****bayaaakhayaaa:**** gaemaera Interface kaemana?

****kaibhaaabae pauurana karabaena:****

- HUD Design
- Menu Design
- Navigation
- Accessibility

****udaaaharana:****

Only visible:

- Flashlight battery (bottom left)
- Current item (bottom center)
- Sanity effect (visual distortion, not number)

Menus:

- Main Menu: Atmospheric, hospital corridor
- Pause Menu: Simple, clean
- Inventory: Grid-based, 12 slots
- Map: Found in-game, not always accessible

Accessibility:

- Colorblind modes
- Subtitles for all dialogue
- Control remapping
- Screen shake toggle

Section 26: Audio (adaiau)

****bayaaakhayaaa:**** gaemaera Sound kaemana?

****kaibhaaabae pauurana karabaena:****

- Music Style
- Sound Effects
- Voice Acting
- Audio Design Principles

****udaaaharana:****

- Exploration: Quiet, unsettling drones
- Tension: Rising strings, distorted sounds
- Chase: Intense, but brief
- Safe Rooms: Calm, brief relief

HUD: M

Music

Sound Effects: Realistic with horror distortion

- Footsteps: Change based on surface
- Doors: Creaking, slamming
- Ambient: Wind, dripping, distant sounds
- Sanity Effects: Distorted audio, reversed sounds

Voice Acting:

- Full voice acting for Sarah (protagonist)
- Partial for key NPCs
- Audio logs for backstory
- No combat voice lines (immersion)

Section 27: Visual Style (bhaijauyaaala sataaaila)

****bayaaakhayaaa:**** gaematai kaemana daekhaaaba?

****kaibhaaaba pauurana karabaena:****

- Art Direction
- Color Palette
- Lighting
- Visual Effects

****udaaaharana:****

Style Reference: Outlast, Layers of Fear

Color Palette:

- Base: Desaturated, cold tones
- Warm: Occasional warm light (hope)
- Horror: Red, distorted colors (danger)
- Sanity: Shifted, unnatural colors

Lighting:

- Dynamic shadows
- Limited flashlight (cone of light)
- Darkness is the enemy
- Safe rooms have warm, stable lighting

Visual Effects:

- Film grain (subtle)
- Chromatic aberration (low sanity)
- Screen distortion (very low sanity)
- Environmental effects (fog, dust, rain)

Section 28: Save System (saebha saisataema)

****bayaaakhayaaa:**** gaema kaibhaaaba saebha habae?

****kaibhaaaba pauurana karabaena:****

- Save Type
- Save Points
- Auto-Save
- Load System

****udaaaharana:****

Art D

Save

Save Points: Safe rooms (tape recorders)
Auto-Save: At chapter transitions, major events
Manual Save: Only at save points

Load: Load from any checkpoint
Restart: Can restart from any chapter

Death: Reload from last checkpoint
No permadeath -- punishment is progress loss, not game over

Section 29: Settings (saetaisa)

****bayaaakhayaaa:**** gaemae kaonao Settings thaaakalae taaa kii?

****kaibhaaabaes pauurana karabaena:****

- Graphics Settings
- Audio Settings
- Control Settings
- Gameplay Settings

****udaaaharana:****

- Resolution: 720p-4K
- Fullscreen/Windowed
- Quality: Low/Medium/High/Ultra
- VSync: On/Off
- FOV: 60-90

Audio:

- Master Volume
- Music Volume
- SFX Volume
- Voice Volume
- Subtitles: On/Off, Size

Controls:

- Key Remapping
- Mouse Sensitivity
- Controller Vibration
- invert Y-axis

Gameplay:

- Difficulty: Normal/Hard/Nightmare
- Subtitles: On/Off
- Colorblind Mode: None/Deuteranopia/Protanopia
- Screen Shake: On/Off

Section 30: Accessibility (ayaaakasaesaibailaitai)

****bayaaakhayaaa:**** saba dharanaera palaeyaaaraera janaya Game kaibhaaabaes Accessible habae?

****kaibhaaabaes pauurana karabaena:****

- Visual Accessibility
- Audio Accessibility
- Motor Accessibility
- Cognitive Accessibility

****udaaaharana:****

- Colorblind modes
- High contrast UI option
- Scalable text size
- Screen reader support (menus)

Audio:

- Visual sound indicators
- Subtitles for all dialogue
- Visual cues for audio events

Motor:

- Control remapping
- One-handed mode option
- Auto-run option
- Hold/Toggle options

Cognitive:

- Adjustable difficulty
- Puzzle hints option
- Simplified UI mode
- Clear objectives display

Section 31: Monetization (aayaera padadhatai)

****bayaaakhayaaa:**** gaema thaekae kaiibhaaabae taaakaaa aaya habae?

****kaiibhaaabae pauurana karabaena:****

- Base Game Price
- DLC Plan
- Microtransactions (yadai thaaakae)
- Season Pass

****udaaaharana:****

No microtransactions
No loot boxes
No battle pass

DLC Plan:

- Free content updates for 6 months
- Story Expansion: \$9.99 (4 hours new content)
- Costume Pack: \$4.99 (cosmetic only)

Revenue Projection:

- Year 1: 50,000 units x \$24.99 = \$1.25M
- Year 2: 30,000 units x \$24.99 = \$750K
- DLC: 20% attach rate

Section 32: Analytics (ayaaanaaalaitaikasa)

****bayaaakhayaaa:**** gaemae kaonao Data Collection thaaakalae taaa kaii?

****kaiibhaaabae pauurana karabaena:****

- Data to Track
- Privacy Policy

- Player Consent

****udaaaharana:****

- Session length
- Death locations
- Puzzle completion time
- Sanity levels at key moments
- Choices made
- Completion rate

Privacy:

- All data anonymous
- No personal information
- Opt-in only
- GDPR compliant

Purpose: Improve game balance, identify pain points

Section 33: Technical Requirements (paraaakatanaika parayaojanaiiyataaa)

****bayaaakhayaaa:**** gaemaera janaya kaona Technology laaagabae?

****kaibhaaabae pauurana karabaena:****

- Game Engine
- Programming Language
- Middleware
- Third-Party Tools

****udaaaharana:****

Language: C++, Blueprints
Middleware: Wwise (Audio), FMOD
Tools: Perforce (Version Control), Jira (Project Management)

Technical Features:

- Lumen (Global Illumination)
- Nanite (Virtual Geometry)
- MetaHuman (Character rendering)
- Chaos Physics (Destruction)

Section 34: Performance Requirements (karamadakassataaara parayaojanaiiyataaa)

****bayaaakhayaaa:**** gaema katataaa Smooth chalatae habae?

****kaibhaaabae pauurana karabaena:****

- Target FPS
- Resolution Targets
- Loading Times
- Memory Usage

****udaaaharana:****

- PC: 60 FPS (minimum 30)
- Console: 30 FPS (Performance Mode: 60 FPS)
- Quality vs Performance options

Resolution:

- PC: 720p to 4K (scalable)
- PS5: 4K/30 or 1440p/60
- Xbox Series X: 4K/30 or 1440p/60

Loading Times:

- Initial Load: <10 seconds
- Area Transition: <5 seconds
- Death/Reload: <3 seconds

Memory:

- RAM: 8GB minimum, 16GB recommended
- VRAM: 4GB minimum, 8GB recommended

Section 35: QA (Quality Assurance)

****bayaaakhayaaa:**** gaemaera Quality kaibhaaaba Ensure karaaa habae?

****kaibhaaaba pauurana karabaena:****

- Testing Types
- Bug Tracking
- Playtesting
- Quality Metrics

****udaaaharana:****

- Functional Testing: Does everything work?
- Performance Testing: Does it run smoothly?
- Compatibility Testing: Works on all platforms?
- Usability Testing: Is it fun and fair?
- Localization Testing: All languages correct?

Bug Tracking: Jira

Priority: Critical > Major > Minor > Trivial

Playtesting:

- Internal: Weekly playtesting sessions
- External: Beta testing with 100+ players
- Focus Groups: Horror fans for feedback

Quality Metrics:

- Crash Rate: <0.1%
- Bug Count: <50 major bugs at launch
- Frame Rate: Consistent 30/60 FPS
- Loading Time: <10 seconds

Section 36: Release Plan (railaija paraikalapanaaa)

****bayaaakhayaaa:**** gaema kakhana aiba kaiibhaaaba Release habae?

****kaibhaaaba pauurana karabaena:****

- Development Timeline
- Milestones
- Release Date
- Marketing Plan

****udaaaharana:****

- Pre-Production: 3 months (Concept, Prototype)
- Production: 12 months (Full development)
- Post-Production: 3 months (Polish, Testing)
- Total: 18 months

Milestones:

- Month 3: Vertical Slice Complete
- Month 6: Alpha (All features implemented)
- Month 9: Beta (Feature complete, polishing)
- Month 12: Gold (Ready for release)
- Month 15: Launch

Release:

- Platform: Steam, PlayStation Store, Xbox Store
- Date: Q3 2027
- Marketing: 3 months pre-launch campaign

Post-Launch:

- Day 1 Patch
- Week 1: Community feedback
- Month 1: First content update
- Month 3: Story DLC announcement

GDD Structure Overview

GDD 36 SECTIONS OVERVIEW

FOUNDATION (Sections 1-6):		
1. Game Overview	2. Vision	3. Design Pillars
4. Target Audience	5. Genre	6. Platforms
CORE GAMEPLAY (Sections 7-9):		
7. Core Gameplay	8. Gameplay Loop	9. Mechanics
PLAYER EXPERIENCE (Sections 10-12):		
10. Player Controls	11. Camera	12. Character System
GAME SYSTEMS (Sections 13-20):		
13. Enemy System	14. Combat	15. Inventory
16. Health	17. Weapons	18. Skills
19. Progression	20. Economy	21. Level Design
22. World Design	23. Story	24. Quest System
PRESENTATION (Sections 25-27):		
25. UI/UX	26. Audio	27. Visual Style
TECHNICAL (Sections 28-34):		
28. Save System	29. Settings	30. Accessibility
31. Monetization	32. Analytics	33. Technical Req
34. Performance Req	35. QA	36. Release Plan

Practical Exercise

anaushailana 1: Complete GDD Write

- **nairadaeshanaaa:****
1. aapanaaara gaemaera janaya 36tai Section pauurana karauna
 2. parataitai Section kamapakassae 3-5 laaainae laikhauna
 3. Example daina parataitai Section ai

4. maota GDD 30-50 parissathaaa haauyaaa uchaita

anaushailana 2: Design Pillars Definition

nairadaeshanaaa:

1. aapanaaara gaemaera janaya 3-5tai Design Pillar laikhauna
2. parataitai Pillar kae 2-3 laaainae Explain karauna
3. daekhauna parataitai Pillar kai satayaii gaurautabapaurana

anaushailana 3: Gameplay Loop Diagram

nairadaeshanaaa:

1. aapanaaara gaemaera Micro, Macro, aiba Meta Loop laikhauna
2. Text-based Diagram taairai karauna
3. Loop gaulao kai satayaii Flowing?

Common Mistakes (saaadhaaarana bhaula)

1. GDD naaa laekhaaa

bhaulta: "maukhae balae dailaaama, sabaaai jaaanae"
sathaika: "sabakaichhau Document karauna -- manae raaakhaaa yaaaya naaa"

2. Over-Specification

bhaulta: "parataitai paikasaela nairadhaaaraita"
sathaika: "Direction daina, Flexible thaaakauna"

3. GDD Update naaa karaaa

bhaulta: "GDD aikabaaara laikhae chhaedae daiyaechhai"
sathaika: "GDD kae Living Document baaanaana, naiyamaita aapadaeta karauna"

4. Vision Statement naaa thaaakaaa

bhaulta: "gaemataaa bhaaalao habae, aitai yathaessata"
sathaika: "Clear Vision Statement laikhauna -- kaena aii gaema?"

5. Design Pillars naaa thaaakaaa

bhaulta: "saba Feature yaoga karao"
sathaika: "Pillars daiyae Filter karauna -- kaii gaurautabapaurana?"

Pro Tips (paeshaadaaara taipasa)

1. GDD kae Version Control karauna

Git baaa Perforce bayabahaaara karauna
parataitai paraibaratana Track karauna

2. Visual References yaoga karauna

shaudhau laikhae naya, Screenshot, Concept Art yaoga karauna
Reference Images GDD kae samaridadha karae

3. Regular Updates daina

sapataaahae aikabaaara GDD Review karauna
natauna Decision gaulao Document karauna

4. Team Input naina

shaudhau aapanai naya, paurao taima Input daika
GDD sabaaara janaya -- sabaaai mailae taairai karauna

5. GDD kae Reference baaanaana

Development saaaraaa GDD daekhauna
"aamaraaa kai aii Direction ai yaaachachhai?" -- naiyamaita yaaachaaai karauna

Chapter Summary (adhayaaaya saaaraaasha)

mauula paaayaenata:

1. **GDD** halao aapanaara gaemaera samapauurana Blueprint
2. **36tai Section** aachhae -- parataitai gaurautabapauurana
3. **Foundation** thaekae shaurau karauna -- Vision, Pillars, Audience
4. **Core Gameplay** Section sabachaeyae baeshai gaurautabapauurana
5. **Technical** Sections gaulao Production Phase ai darakaaara habae
6. **Living Document** -- GDD naiyamaita aapadaeta karauna
7. **Team Input** naina -- GDD sabaaara mailae taairai karauna

parabarataii adhayaaaya:

adhayaaaya 7 ai aamaraaa daekhabao **Core Gameplay Loop** -- palaeyaaara karii baaarabaaara karabae aiba kaena
thaaamatae paaarabae naaa

manae raaakhabaena: aikatai bhaaalao GDD chhaadaaa bada gaema taairai samabhaba naya aapanaara GDD aapanaara
sabachaeyae bada banadhau -- aitaikae Maintain karauna

PART 7

adhayaaaya 7: CORE GAMEPLAY LOOP -- mauula gaemapalae laupa

adhayaaaya 7: CORE GAMEPLAY LOOP -- mauula gaemapalae laupa

shaekhaara udadaeshaya (Learning Goal)

aii adhayaaaya shaessae aapanai jaaanabaena:

- Core Gameplay Loop kii aiba kaena aitai gaurautabapauurana
- mauula Loop: Player -> Explore -> Discover -> Challenge -> Reward -> Upgrade -> Repeat
- 10tai Genre-Specific Gameplay Loop Diagram
- aapanaara gaemaera janaya Perfect Loop kaibhaaaba taairai karabaena

dhaaranaaa bayaaakhayaaa (Concept Explanation)

Core Gameplay Loop kii?

Core Gameplay Loop halao saei mauula kaaajagaulao yaaa palaeyaaara baaarabaaara karae -- aiba karatae chaaaya aitai gaemaera "Addiction Factor" -- yaaa palaeyaaarakae baaarabaaara khaelatae baaadhaya karae

CORE GAMEPLAY LOOP IMPORTANCE

Without Good Loop:

Players quit after 10 minutes
No reason to continue playing
Game feels boring and repetitive
No sense of progression

With Good Loop:

Players want to play "just one more round"
Clear motivation to continue
Satisfying progression
Long-term engagement

mauula Loop: Player -> Explore -> Discover -> Challenge -> Reward -> Upgrade -> Repeat

MASTER GAMEPLAY LOOP
PLAYER

(Start)
 EXPLORE
 Navigate
 DISCOVER
 Find
 CHALLENGE
 Face
 REWARD
 Receive
 UPGRADE
 Improve
 REPEAT <
 Continue

Loop Elements baisataaaraita bayaaakhayaaa:

1. Player (palaeyaaara)

- gaema shaurau karae
- Initial State: kama Skill, kama Resources
- Motivation: kaaatauuhala, chayaaalaenyaja, Achievement

2. Explore (anausanadhaaana)

- natauna jaaayagaaaya yaaaya
- chaaarapaaashae taaakaaaya
- patha khaujae baera karae

3. Discover (aabaissakaaara)

- natauna Item paaaya
- Clue khaujae paaaya
- Secret aabaissakaaara karae

4. Challenge (chayaaalaenyaja)

- Enemy aira saaathae ladaaai
- Puzzle samaaadhaaana
- Obstacle paaara karaaa

5. Reward (paurasakaaara)

- XP paaaya
- Item paaaya
- Achievement Unlock haya

6. Upgrade (unanatai)

- Level Up haya
- New Skill paaaya
- Equipment aapagaraeda haya

7. Repeat (paunaraaabaritatai)

- aabaaara shaurau karae
- kainatau aikhana aagaera chaeyae shakataishaaalaih

10 Genre-Specific Gameplay Loops

1. Horror Game Loop

```

                                HORROR GAME LOOP

Player
(Afraid)
Explore > Hide > Listen
(Dark)      (Silent)      (Tense)
              Discover      Survivor
              (Clue)       (Lucky)
              Escape       Fear
              > (Run!)      (More!)
              Sanity
              Drops
              World
              Changes
              Loop

```

Key Elements:

- Tension -> Release -> Tension cycle
- Fear as resource/consequence
- Exploration rewarded with story/clues
- Survival creates engagement

2. FPS Game Loop

```

                                FPS GAME LOOP

Player
(Armed)
Move
(Map)
Spot > Aim > Shoot
(Enemy)      (Focus)      (Fire!)
                          Kill
                          (XP!)
                          Loot
                          (Items)
                          Level
                          Up!
                          New
                          Weapon
Unlock

```

Key Elements:

- Quick action -> Immediate reward
- Weapon progression motivates play
- Skill improvement = better performance
- Kill -> Loot -> Upgrade -> Kill cycle

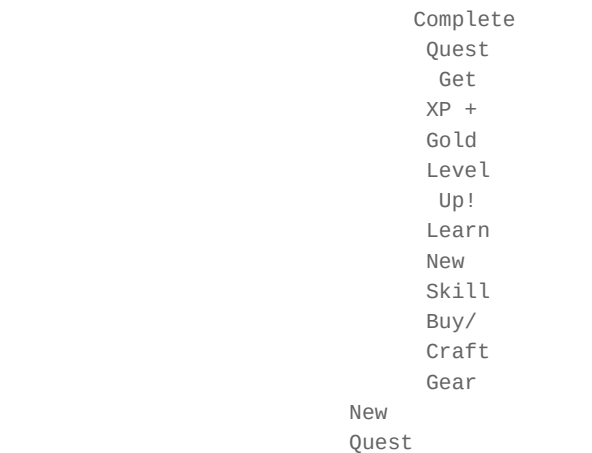
3. RPG Game Loop

```

                                RPG GAME LOOP

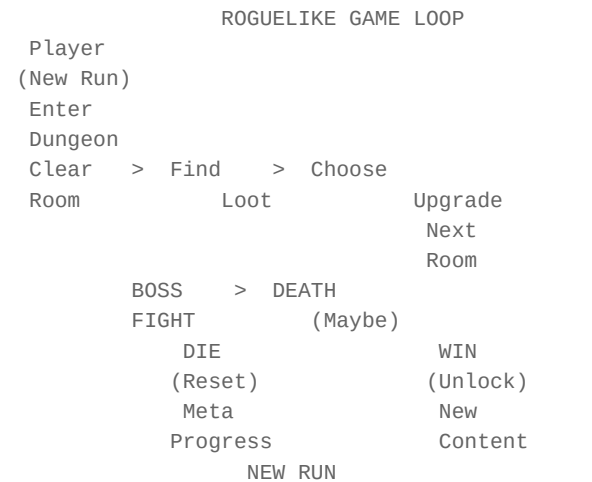
Player
(Hero)
Accept
Quest
Travel > Fight > Solve
(Explore)      (Combat)      (Puzzle)

```



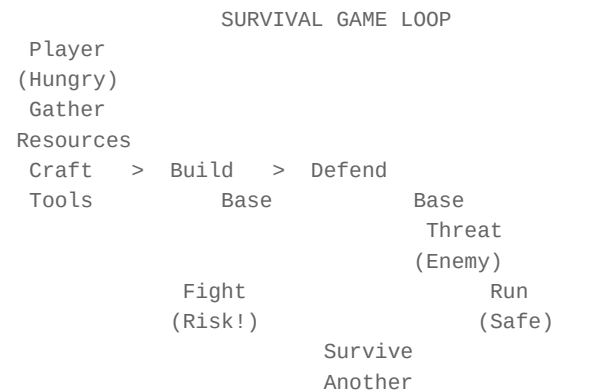
- Key Elements:
- Quest-driven exploration
 - Character progression (levels, skills, gear)
 - Story reveals through quests
 - Long-term investment in character

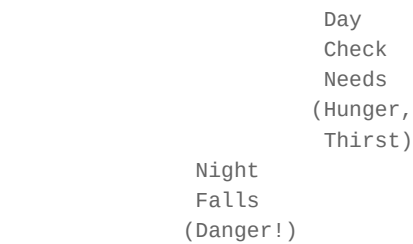
4. Roguelike Game Loop



- Key Elements:
- Death is not end -- it's progression
 - Randomized content = fresh each run
 - Quick runs encourage "one more try"
 - Meta-progression across runs

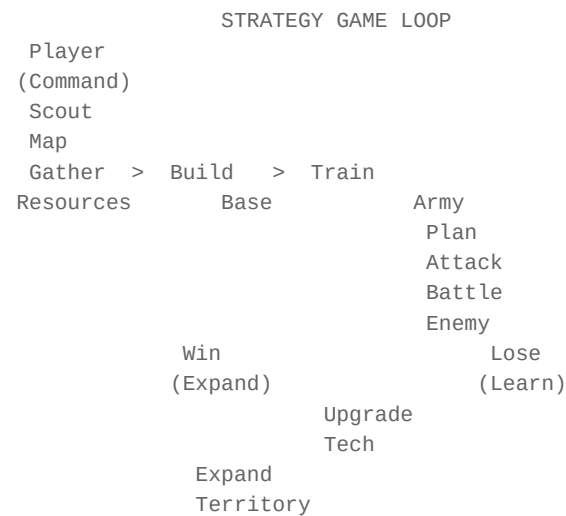
5. Survival Game Loop





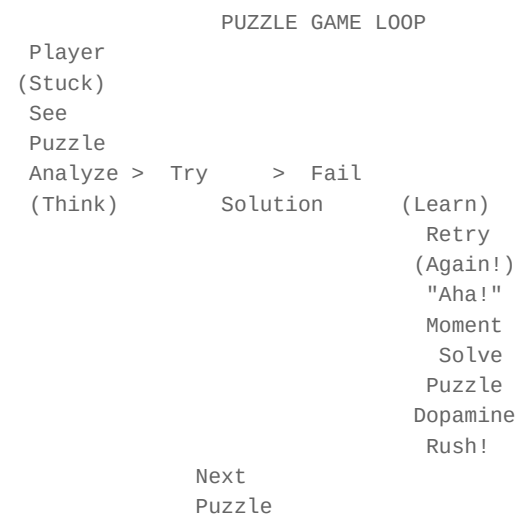
- Key Elements:
- Constant resource management
 - Threat creates urgency
 - Base building = investment
 - Day/Night cycle creates rhythm

6. Strategy Game Loop



- Key Elements:
- Resource management + Military strategy
 - Long-term planning rewarded
 - Information (scouting) is power
 - Economic + Military balance

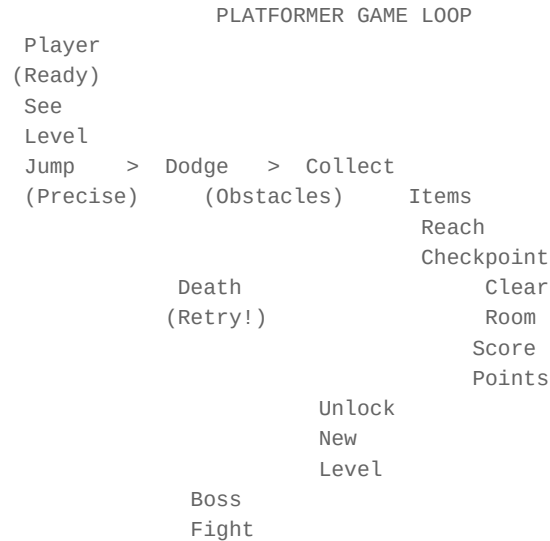
7. Puzzle Game Loop



- Key Elements:
- Struggle -> Insight -> Satisfaction

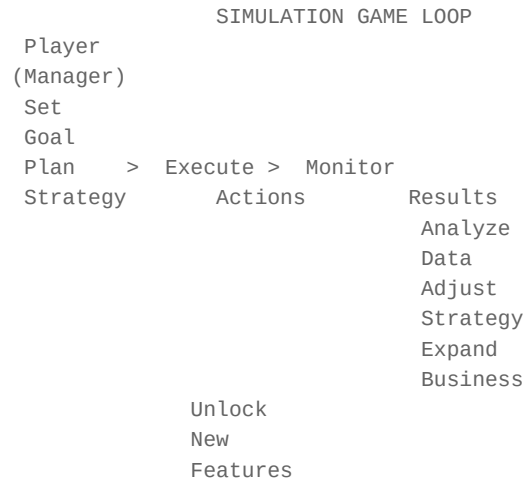
- Difficulty gradually increases
- "Aha!" moment is the reward
- Each puzzle teaches new mechanic

8. Platformer Game Loop



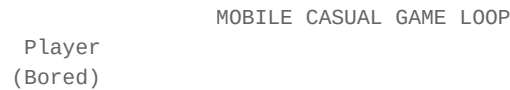
- Key Elements:
- Precision mechanics (jump timing)
 - Trial and error learning
 - Muscle memory development
 - Mastery = Speed + Style

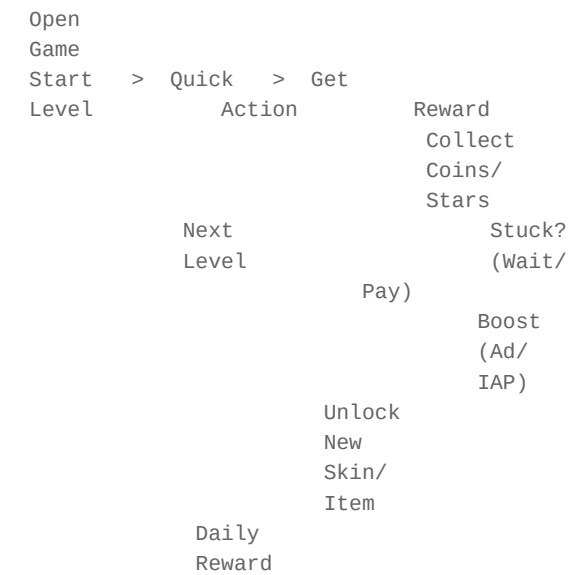
9. Simulation Game Loop



- Key Elements:
- Long-term planning and patience
 - Data-driven decisions
 - Gradual complexity increase
 - "One more day/turn" addiction

10. Mobile Casual Game Loop





- Key Elements:
- Quick sessions (2-5 minutes)
 - Simple controls (tap/swipe)
 - Immediate feedback
 - Social features (leaderboards, sharing)
 - Monetization through ads/IAP

Loop Comparison Table

GAMEPLAY LOOP COMPARISON				
Genre	Session	Reward	Difficulty	Addiction
Horror	Long	Story	Medium	Fear
FPS	Short	Kill	High	Adrenaline
RPG	Long	Level	Medium	Progress
Roguelike	Short	Meta	Very High	"One More"
Survival	Long	Survive	High	Urgency
Strategy	Long	Expand	Very High	Mastery
Puzzle	Medium	"Aha!"	Medium	Insight
Platformer	Short	Clear	High	Muscle
Simulation	Long	Growth	Medium	Patience
Mobile Casual	V.Short	Score	Low	Habit

Practical Exercise

anaushailana 1: Design Your Loop

- **nairadaeshanaaa:****
1. aapanaaaara gaemaera Genre nairadhaaarana karauna
 2. Master Loop (Player -> Explore -> Discover -> Challenge -> Reward -> Upgrade -> Repeat) aira parataitai Step aapanaaaara gaemae Apply karauna
 3. Text-based Diagram taairai karauna
 4. Loop kai Addictive? naijaekae jaijanyaasaaaa karauna

anaushailana 2: Analyze Existing Games

****nairadaeshanaaa:****

1. aapanaaara pachhanadaera 3tai gaema khaelauna
2. parataitai gaemaera Core Loop baera karauna
3. Diagram aakaaarae laikhauna
4. kaena aii Loop kaaaja karae?

anaushaiilana 3: Loop Improvement

****nairadaeshanaaa:****

1. aikatai Bad Loop aira Example laikhauna
2. baujhauna kaena aitai kaaaja karae naaa
3. aitaikae Good Loop ai Convert karauna
4. paraibaranagaulao bayaaakhayaaa karauna

Common Mistakes (saaadhaaarana bhaula)

1. Loop naaa thaaakaaa

bhaula: "gaema aachhae, kainatau kaonao Loop naei"
sathaika: "parataitai gaemaera aikatai Core Loop thaaakatae habae"

2. Loop khauba jataila haauyaaa

bhaula: "10tai Step aira Loop"
sathaika: "3-5tai Step aira Simple Loop"

3. Reward naaa thaaakaaa

bhaula: "Challenge aachhae, kainatau Reward naei"
sathaika: "parataitai Challenge aira parae Reward daina"

4. Repetition aira saiimaaa naaa jaaanaaa

bhaula: "aikai Loop 100 baaara baaarabaaara"
sathaika: "Loop Variation daina -- aikai kainatau bhainana"

5. Progression naaa daekhhaanao

bhaula: "palaeyaaara baujhatae paaarachhae naaa sae katataukau aigaiyae gaechhae"
sathaika: "Progress Bar, Level Up, Achievement -- daiyae daekhhaana"

Pro Tips (paeshaaadaara taipasa)

1. "One More Turn" Test karauna

aapanaaara Loop kai aimana yae palaeyaaara balabae "aaraekatau khaelai"?
yadai naaa -- Loop Improve karauna

2. Micro Loop Perfect karauna

30 saekaenadaera Loop yadai bhaaalao naaa haya, lamabaaa Loop kaaaja karabae naaa
parataitai Action ai Satisfying Feedback daina

3. Delayed Gratification bayabahaaara karauna

sabakaichhau taaakassanaika naaa hatae habae
kaichhau Reward aachhae yaaa parae aasabae -- aitaai Motivation taairai karae

rii. Loop Variation daina

aikai Loop 100 baaara baorai
paratai 10 mainaitae kaichhau natauna yaoga karauna

5. Player Psychology baujhauna

- Autonomy: Player naijae Decision naika
- Competence: Player bhaaalao hachachhae baojhatae paaarachhae
- Relatedness: Player gaemaera saaathae Connected

aii 3tai Need pauurana karauna

Chapter Summary (adhayaaaya saaraaasha)

mauula paaayaenata:

1. **Core Gameplay Loop** halao gaemaera sabachaeyae gaurautabapauurana asha
2. **Master Loop**: Player -> Explore -> Discover -> Challenge -> Reward -> Upgrade -> Repeat
3. **10tai Genre** aira Loop aalaaadaabhaaaba kaaaja karae
4. **Loop must be Addictive** -- "One More Turn" Test karauna
5. **Micro Loop** (30 sec) sabachaeyae gaurautabapauurana
6. **Progression** daekhaana -- Player katataukau aigaiyae gaechhae
7. **Variation** daina -- aikai Loop baaarabaaara naya, bhainanabhaaaba daina

parabaratai adhayaaaya:

adhayaaaya 8 ai aamaraa daekhabao **Scope Management** -- aapanaaara gaemakae chhaota aiba Manageable kaibhaaaba raaakhabaena

manae raaakhabaena: aikatai bhaaalao Game Loop chhaadaa gaema baorai aapanaaara Loop kae Addictive, Satisfying, aiba Progressively Challenging baaanaana

PART 8

adhayaaaya 8: SCOPE MANAGEMENT -- sakaopa mayaaanaejamaenata

adhayaaaya 8: SCOPE MANAGEMENT -- sakaopa mayaaanaejamaenata

shaekhaara udadaeshaya (Learning Goal)

aii adhayaaaya shaessae aapanai jaaanabaena:

- Scope Management kii aiba kaena itai Indie Developer aira janaya sabachaeyae gaurautabapauurana
 - MVP, Vertical Slice, Prototype, Minimum Feature Set, Nice-to-Have Features, Cut List kii
 - MoSCoW Framework (Must Have / Should Have / Could Have / Won't Have) kaibhaaaba bayabahaara karabaena
 - Practical Examples saha Scope Control
-

dhaaaranaaa bayaaakhayaaa (Concept Explanation)

Scope Management kii?

Scope Management halao apanaara gaemaera kaaajakae naiyanatarana karaa -- kii karatae habae, kii karaa yaaabae naa, aiba kakhana thaaamatae habae

WHY SCOPE MANAGEMENT MATTERS

Without Scope Management:

Project never finishes
Features keep getting added
Budget exceeds limits
Team burns out
Game releases incomplete or not at all

With Scope Management:

Clear boundaries
Focused development
On-time delivery
Quality over quantity
Finished game > Perfect game

The Scope Problem for Indie Developers

INDIE DEVELOPER'S DILEMMA

Phase 1: "I have a great idea!"
 Excitement: 100% Scope: Small
 Phase 2: "Let me add this feature too..."
 Excitement: 90% Scope: Medium
 Phase 3: "And this! And this!"
 Excitement: 70% Scope: Large
 Phase 4: "This is too much work..."
 Excitement: 40% Scope: Very Large
 Phase 5: "I'll never finish this..."
 Excitement: 10% Scope: Massive
 Phase 6: "Abandoned project"
 Excitement: 0% Scope: Game Over
 THIS IS WHY SCOPE MANAGEMENT MATTERS!

Key Terms (mauula shabadaaabalaii)

1. MVP (Minimum Viable Product) -- nayauunatama baaasatabasamamata panaya

MVP halao aapanaaara gaemaera sabachaeyae sahaja, kaaajaera sasakarana -- yaetai daiyae paraaathamaika Gameplay Experience daeayuaa yaaaya

****MVP kaena darakaaara:****

- darauta paraotaotaaipa taairai karauna
- Idea Validate karauna
- Player Feedback naina
- Development Direction nairadhaaarana karauna

****MVP Example:****

1 corridor
 1 enemy type
 1 puzzle
 Basic controls
 15-minute gameplay

What MVP does NOT include:

Multiple levels
 Multiple enemy types
 Complex puzzles
 Story cinematics
 Voice acting

2. Vertical Slice -- ulamaba salaaaisa

Vertical Slice halao aapanaaara gaemaera aikatai chhaota asha yaaa paura gaemaera Quality, Style, aiba Mechanics daekhaaaya aitari paura gaemaera "Taste"

****Vertical Slice kaena darakaaara:****

- Publisher/Investor kae daekhaaana
- Art Style Validate karauna
- Gameplay Feel yaaachaaai karauna

- Technical Feasibility paramaana karauna

****Vertical Slice Example:****

1 Complete Level (15-30 minutes)
 All Core Mechanics Working
 Final Art Quality
 Sound Design Complete
 UI/UX Implemented
 Save System Working
 1 Enemy Type (Final Quality)
 1 Puzzle Type (Final Quality)

What Vertical Slice does NOT include:

Multiple levels
 Multiple enemy types
 Full story
 All features

3. Prototype -- paraotaotaaipa

Prototype halao aikatai Quick aiba Dirty Version yaaa shaudhau Gameplay Mechanics Test karae Art, Sound, Polish -- kaichhaui thaaakae naaa

****Prototype kaena darakaaara:****

- Idea Test karauna
- Fun kainaaa yaaachaaai karauna
- Mechanics Validate karauna
- darauta Fail karauna (yadai kaaaja naaa karae)

****Prototype Example:****

Grey-box level (no art)
 Basic movement controls
 Simple enemy AI
 Placeholder sounds
 5-minute gameplay test

What Prototype does NOT include:

Any art
 Sound design
 UI
 Story
 Polish

4. Minimum Feature Set -- nayauunatama phaichaaara saeta

Minimum Feature Set halao aapanaaara gaemaera janaya sabachaeyae kama Features yaaa daiyae gaematai kaaaja karabae aiba Fun habae

****Minimum Feature Set Example:****

Movement (Walk, Run, Crouch)
 Flashlight
 Inventory (6 slots)
 Sanity System
 1 Enemy Type

3 Puzzle Types
 Save System
 Basic UI

NOT in Minimum Set:
 Crafting System
 Skill Tree
 Multiple Endings
 Photo Mode
 New Game+

5. Nice-to-Have Features -- bhaaalao halao thaaakata

Nice-to-Have Features halao saei Features yaaa gaemae thaaakalae bhaaalao habae, kainatau chhaaadalaeeu gaema kaaaja karabae

****Nice-to-Have Example:****

Photo Mode
 New Game+ Mode
 Multiple Endings (5 endings)
 Collectibles
 Achievements
 Speed Run Mode
 Commentary Mode

Horro

6. Cut List -- kaaata laisata

Cut List halao saei Features yaaa aapanai saidadhaanata naiyaechhaena gaemae thaaakabae naaa aigaulao bhabaissayataera janaya Save karauna

****Cut List Example:****

Multiplayer Mode (taaadaera janaya -- parae hatae paaarae)
 Procedural Generation (Scope baeshai hayae yaaaya)
 Crafting System (gaemaera Theme aira saaathae mailae naaa)
 Open World (anaeka bada parajaekata)
 Voice Acting for ALL NPCs (kharacha baeshai)

Horro

MoSCoW Framework

MoSCoW Framework halao Features kae 4tai Category ai bhaaaga karaaara padadhatai:

MoSCoW FRAMEWORK

M -- MUST HAVE (abashayai thaaakatae habae)
 chhaaadalaeeu gaema kaaaja karabae naaa
 MVP aira asha
 Release Gate
 100% Required
 S -- SHOULD HAVE (thaaakaaa uchaita)
 gaemakae bhaaalao karabae
 MVP aira parae yaoga karauna
 Important but not critical
 90% Target
 C -- COULD HAVE (thaaakalae bhaaalao)
 Bonus Feature

Time thaaakalae karauna
 Nice-to-have
 50% Stretch Goal
 W -- WON'T HAVE (aibaaara naya)
 aii Release ai naya
 Future Update aira janaya
 Out of Scope
 0% This Time

MoSCoW Application Example: Horror Game

HORROR GAME -- MoSCoW TABLE

MUST HAVE (M):

First-person movement
 Flashlight mechanic
 Sanity system
 1 enemy type
 3 puzzle types
 5 levels
 Save system
 Basic UI (health, sanity, inventory)
 Sound design

SHOULD HAVE (S):

2 additional enemy types
 Environmental storytelling
 Multiple endings (3)
 Audio logs
 Keyboard/controller support
 Accessibility features

COULD HAVE (C):

Photo mode
 New Game+ mode
 Collectibles (20)
 Speed run timer
 Commentary mode
 2 additional puzzle types

WON'T HAVE (W):

Multiplayer
 Procedural generation
 Crafting system
 Open world
 Full voice acting
 VR support

MoSCoW Application Example: RPG Game

RPG GAME -- MoSCoW TABLE

MUST HAVE (M):

Character movement
 Combat system (basic)
 10 levels
 Skill system (3 skills)
 Inventory system
 Save/load system
 1 story quest line
 Basic UI

SHOULD HAVE (S):

5 additional skills
 Side quests (10)
 Crafting system (basic)

NPC dialogue system
 Equipment system
 Controller support
 COULD HAVE (C):
 Skill tree (full)
 20 side quests
 Advanced crafting
 Multiple endings
 Photo mode
 New Game+ mode
 WON'T HAVE (W):
 Multiplayer
 Open world
 Procedural dungeons
 Full voice acting
 Mount system
 Housing system

Scope Control Methods

1. The "One More Feature" Trap

THE ONE MORE FEATURE TRAP

Developer: "The game needs just ONE more feature..."
 Month 1: "Let's add a crafting system"
 Month 2: "Now we need recipes for crafting"
 Month 3: "And materials to gather"
 Month 4: "And a UI for the crafting menu"
 Month 5: "And balance all the recipes"
 Month 6: "And test everything"
 Result: 6 months spent on ONE feature
 SOLUTION: Before adding ANY feature, ask:
 "Is this in my MUST HAVE list?"
 If NO -> Add to Cut List

2. Feature Priority Matrix

FEATURE PRIORITY MATRIX

HIGH IMPACT

MUST HAVE	SHOULD
(Do First)	HAVE
	(Do Next)

HIGH

EFFORT	SHOULD	COULD
	HAVE	HAVE
	(Consider)	(If Time)

LOW IMPACT

HIGH IMPACT + LOW EFFORT = DO FIRST
 HIGH IMPACT + HIGH EFFORT = PLAN CAREFULLY
 LOW IMPACT + LOW EFFORT = DO IF TIME
 LOW IMPACT + HIGH EFFORT = CUT

3. The 80/20 Rule (Pareto Principle)

THE 80/20 RULE

80% of game's fun comes from 20% of features

Example:

- Movement: 80% of fun, 10% of work
 - Combat: 70% of fun, 20% of work
 - Crafting: 30% of fun, 30% of work
 - Photo Mode: 5% of fun, 10% of work
 - Multiplayer: 40% of fun, 40% of work
- STRATEGY: Focus on the 20% that gives 80% fun
Don't spend 40% effort on 5% fun
-

Practical Exercise

anaushailana 1: MoSCoW Your Game

****nairadaeshanaaa:****

1. aapanaaara gaemaera saba Features laikhauna
2. parataitakae Must/Should/Could/Won't ai bhaaaga karauna
3. Must Have laisata yaaachaaai karauna -- kai satayaii darakaaara?
4. Won't Have laisata daekhauna -- kaonagaulao parae karatae paaaraena?

anaushailana 2: Prototype Your Game

****nairadaeshanaaa:****

1. 1 sapataaahaera madhayae aapanaaara gaemaera Prototype baaanaana
2. Grey-box level, basic controls, placeholder art
3. 5 janakae khaelaiyae daekhauna
4. Feedback naina: "aitaaa kai Fun?"

anaushailana 3: Cut List Exercise

****nairadaeshanaaa:****

1. aapanaaara Feature List thaekae 5tai Feature baechhae naina
2. jaijanyaasaaa karauna: "aitai chhaaadalae gaema kai kaaaja karabae?"
3. yadai hayaaa -- Cut List ai yaoga karauna
4. yadai naaa -- Must Have ai raaakhauna

anaushailana 4: Time Estimation

****nairadaeshanaaa:****

1. aapanaaara Must Have Features aira Time Estimate karauna
 2. maota Time x 2 = Realistic Time
 3. yadai Time Limit aira baaairae haya -- Scope kamaana
-

Common Mistakes (saaadhaaarana bhaula)

1. "Perfect" Game baaanaanaora chaessataaa

bhaula: "sabakaichhau Perfect haauyaaa darakaaara"
sathaika: "Finished game > Perfect game"

2. Feature Creep

bhauLa: "aaraekataaa Feature yaoga karai..."
sathaika: "Must Have laisata daekhauna -- saekhaanae naei taaahalaе naya"

3. Scope anaumaaana naaa karaaa

bhauLa: "3 maaasae hayae yaaabae"
sathaika: "Time x 2 = Realistic Estimate"

4. Cut List naaa baaanaanao

bhauLa: "saba Feature raaakhaba"
sathaika: "kaonagaulao Cut karaba -- saetaaa aagae nairadhaaarana karauna"

5. MVP baaada daeayaaa

bhauLa: "paurao gaema aikasaaathae baaanaaaba"
sathaika: "parathamae MVP baaanaana, taaarapara Expand karauna"

6. "Just One More" Syndrome

bhauLa: "shaudhau aikataaa aaraekataaa Feature yaoga karai"
sathaika: "Feature Freeze -- nairadhaaaraita Feature aira para natauna naya"

Pro Tips (paeshaadaara taipasa)

1. "Kill Your Darlings"

aapanaara pachhanadaera Feature yadai Must Have laisatae naei, kaaatauna bayakataigata sayaoga baaada daina -- gaemaera janaya kaii bhaaalao saetaaai karauna

2. Two-Week Rule

parataitai Feature aira janaya Maximum 2 sapataaaha
2 sapataaahaera baeshai laagachhae? Scope kamaana baaa Cut karauna

3. Vertical Slice First

MVP aira aagae Vertical Slice baaanaana
aitai daekhaaabaе gaemataaa kaemana habae
Publisher/Investor kae daekhaanaora janaya darakaaara

4. Weekly Scope Check

paratai sapataaahaе Feature List daekhauna:
- kai natauna yaoga hayaechhae? kaena?
- kai Cut karatae habae?
- Timeline kai On Track?

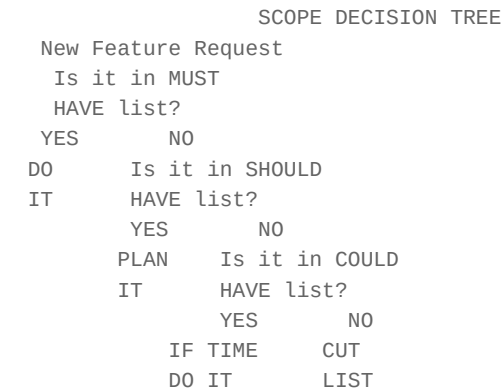
5. Say "No" More Often

parataitai Feature Request ai "No" balauna
"No" balatae paaaraena naaa? "Not Now" balauna
"Could Have" laisatae raaakhauna

6. Celebrate Small Wins

parataitai Must Have Feature shaessa halae Celebrate karauna
aitai Motivation raaakhabae
Progress daekhauna -- katataukau hayaechhae

Scope Management Decision Tree



Feature Cut Examples

FEATURE CUT EXAMPLES

Game: Horror Game

Original Scope:

- 15 levels
- 8 enemy types
- Crafting system
- Skill tree
- 5 endings
- Photo mode
- Multiplayer
- Procedural generation

After Scope Management:

MUST HAVE:

- 5 levels [x]
- 3 enemy types [x]
- Sanity system [x]
- Save system [x]

CUT:

- Crafting system (not essential for horror)
- Skill tree (keep simple)
- Photo mode (nice but not needed)
- Multiplayer (too much scope)
- Procedural generation (too complex)

Result: Finish in 12 months instead of 24+ months

Chapter Summary (adhayaaaya saaaraaasha)

mauula paaayaenata:

1. ****Scope Management**** Indie Developer aira sabachaeyae bada chayaaalaenyaja

2. **MVP**: sabachaeyae sahaja, kaaajaera Version -- parathamae aitai baaanaana
3. **Vertical Slice**: gaemaera "Taste" -- Publisher kae daekhaanaora janaya
4. **Prototype**: Quick Test -- Fun kainaaa yaaachaaai karauna
5. **MoSCoW Framework**: Must/Should/Could/Won't -- Features bhaaaga karauna
6. **Cut List**: kaona Features Cut karabaena -- aitaana aagae nairadhaaraana karauna
7. **"One More Feature" Trap**: aidaiyae chalauna -- Feature Freeze karauna
8. **Finished Game > Perfect Game**: gaema shaessa karauna, Perfect naya

parabarataii adhayaaaya:

adhayaaaya 9 ai aamaraaa daekhabao **Prototyping** -- darauta aiba sahajabhaaabae Game Prototype kaibhaabae taairai karabaena

***manae raaakhabaena**: "Scope Down, Quality Up" -- aapanaara gaemakae chhaota aiba Focused raaakhauna aikatai chhaota, bhaaalao gaema anaeka bhaaalao aikatai bada, asamaapata gaemaera chaeyae*

PART 9

adhayaaaya 9: GAME DEVELOPMENT ROADMAP

adhayaaaya 9: GAME DEVELOPMENT ROADMAP

shaekhaara udadaeshaya (Learning Goal)

aai adhayaaayae aapanai shaikhabaena aikatai Game Development parakalapakae shaurau thaekae shaessa parayanata kaiibhaaaba paraichaaalanaaa karatae haya aamaraaa 11tai saunairadaissata Phase naiyae aalaochanaaa karabao, parataitai Phase aira Goal, Tasks, Deliverables, Team, Tools, Timeline, Common Mistakes, aiba Exit Criteria baisataaaraitabhaaabaebaujhabao

9.1 Game Development Roadmap kaena darakaaara?

aikatai Game Develop karaaa haaajaaarao chhaota chhaota kaaajaera samanabaya aikajana Solo Developer haona baaa aikatai bada Studio haona, sabaaarai aikatai Clear Roadmap darakaaara Roadmap chhaaadaaa Game Development haya:

- ****Chaotic****: kaonao paraikalapananaa chhaaadaaa kaaaja shaurau karalae samayaera apachaya haya
- ****Unfocused****: kaonatai aagae karabaena kaonatai parae jaaanaaa thaaakae naaa
- ****Over-budget****: samaya au aratha dautaoi nassata haya
- ****Burnout****: atairaikata kaaajaera chaaapae maaanasaiika kalaaanatai haya
- ****Incomplete****: anaeka gaema shaessa haya naaa kaaarana kaonao Phase samapanana haya naaa

Roadmap aapanaaakae daeya:

- parataitai dhaaapae kaii karatae habae taaa sapassata thaaakae
 - kakhana kaona Phase shaessa karatae habae
 - taima sadasayadaera kaaaja bhaaagaaabhaaagai karaaa yaaaya
 - Risk chaihanaite karae aagae thaekae samaaadhaaana karaaa yaaaya
-

9.2 Game Development Roadmap taekasata-baesada taaaimalaaaina daayaaagaraama

Game Development Roadmap - 11tai Phase

Phase 1: IDEA

Goal: gaemaera mauula dhaaaranaaa nairadhaaarana

Time: 1-2 sapataaaha

Phase 2: RESEARCH

Goal: baaajaaara, parayaukatai, aiba parataiyaogaitaaa baishalaessana

Time: 2-4 sapataaaha

Phase 3: PRE-PRODUCTION

Goal: Game Design Document (GDD) taairai aiba paraikalapananaa

Time: 4-8 sapataaaha

Phase 4: PROTOTYPE

Goal: gaemaera mauula Mechanic paraiikassaaa karaaa

Time: 2-4 sapataaaha

Phase 5: VERTICAL SLICE

Goal: gaemaera aikatai chhaota kainatau samapauurana Playable Section

Time: 4-8 sapataaaha

Phase 6: PRODUCTION

Goal: gaemaera samapauurana Content taairai

Time: 6-18 maaasa

Phase 7: ALPHA

Goal: gaema Playable kainatau asamapauurana, Bug Fix shaurau

Time: 2-4 sapataaaha

Phase 8: BETA

Goal: gaema Feature Complete, shaudhau Bug Fix aiba Polish

Time: 2-6 sapataaaha

Phase 9: RELEASE CANDIDATE

Goal: gaema Release-aira janaya parasatauta

Time: 1-2 sapataaaha

Phase 10: LAUNCH

Goal: gaema paaabalaisha aiba maaarakaetai

Time: 1-2 sapataaaha

Phase 11: POST-LAUNCH

Goal: gaema aapadaeta, Patch, aiba Player Feedback

Time: 3-12 maaasa (athabaaa anairadaissatakaaala)

Total Estimated Timeline: 10-24 maaasa (Medium-scale Game)

9.3 parataitai Phase baisataaaraita aalaochanaaa

Phase 1: IDEA (dhaaaranaaa)

****Goal:****

gaemaera mauula dhaaaranaaa (Core Concept) nairadhaaarana karaaa kaona dharanaera gaema baaanaaatae chaaana, kaaara janaya baaanaaatae chaaana, aiba gaematai kaiibhaaabae ananaya habae taaa saidadhaaanata naeayyaaa

****Tasks:****

- gaemaera mauula dhaaaranaaa laikhauna (One-sentence Pitch)
- Genre nairadhaaarana karauna (RPG, Action, Puzzle, Horror, itayaaadai)
- Target Audience chaihanaite karauna
- USP (Unique Selling Point) sajanyaaayaita karauna
- paraaathamaika Game Loop chainataaa karauna
- Reference Games laisata taairai karauna

****Deliverables:****

- One-page Game Concept Document
- Game Pitch Document (1-2 parissathaaa)
- kamapakassae 3tai Reference Game aira taaalaikaaa

****Team:****

- Solo Developer: naijaei saba karabaena
- Small Team: Game Designer + Creative Director
- Large Team: Game Director + Lead Designer + Producer

****Tools:****

- Google Docs / Microsoft Word (laekhaaara janaya)
- Miro / Trello (Brainstorming aira janaya)
- Notion (Documentation aira janaya)

****Estimated Time:**** 1-2 sapataaaha****Common Mistakes:****

- dhaaaranaaa khaubai bada karae phaelaaa (Scope baeshai bada)
- shaudhaumaaatara Trend anausarana karaaa (Copying)
- Target Audience nairadhaaarana naaa karaaa
- USP sapassata naaa thaaakaaa
- aikaaadhaika dhaaaranaaaara madhayae daolaaa

****Exit Criteria:****

- gaemaera mauula dhaaaranaaa sapassata au sakassaipata
- Genre aiba Target Audience nairadhaaaraita
- USP sajanyaaayaita

Phase 2: RESEARCH (gabaessanaaa)

****Goal:****

baajaara baishalaessana, parataiyaogaii gaema samaiikassaaa, aiba parayaukatai paraiikassaaa karaaa

****Tasks:****

- Market Research karauna (kata dharanaera gaema aachhae, kata baikarai haya)
- Competitor Analysis karauna (shaiirassa 10 parataiyaogaii gaema samaiikassaaa)
- Technology Assessment karauna (Engine nairabaaachana: Unity, Unreal, Godot)
- Platform nairadhaaarana karauna (PC, Mobile, Console, Web)
- Budget Estimate taairai karauna
- Team Requirements nairadhaaarana karauna
- Legal Requirements paraiikassaaa karauna (Copyright, Trademark)

****Deliverables:****

- Market Research Report (5-10 parissathaaa)

- Competitor Analysis Spreadsheet
- Technology Stack Document
- Preliminary Budget Estimate
- Team Requirements Document

****Team:****

- Solo Developer: naijaei Research karabaena
- Small Team: Producer + Lead Designer
- Large Team: Producer + Market Researcher + Technical Lead

****Tools:****

- SteamSpy / Steam (PC Game Market Data)
- Sensor Tower / App Annie (Mobile Market Data)
- Google Trends (Trend Analysis)
- Trello / Asana (Research Organization)

****Estimated Time:** 2-4 sapataaaha******Common Mistakes:****

- atairaikata Research karae kaaaja shaurau naaa karaaa (Analysis Paralysis)
- shaudhaumaaatara naijaera pachhanadaera gaema baishalaessana karaaa
- baaajaaaraera parakarita chaaahidaaaa upaekassaaa karaaa
- parayaukatai samasayaaa samaaadhaaana naaa karae aigaiyae yaaaauyaaa
- Budget abaaasataba haisaaaba karaaa

****Exit Criteria:****

- baaajaaaraera chaaahidaaaa samaparakae sapassata dhaaaranaaaa
- parayaukatai samasayaaa samaaadhaaana karaaa hayaechhae
- paraaathamaika Budget au Timeline nairadhaaaraita

Phase 3: PRE-PRODUCTION (pauuraba-upaaadana)

****Goal:****

gaemaera samapauurana paraikalapanaaa taairai karaaa Game Design Document (GDD), Technical Design Document (TDD), aiba Art Bible laekhaaa

****Tasks:****

- Game Design Document (GDD) laikhauna
- Technical Design Document (TDD) laikhauna
- Art Style Guide / Art Bible taairai karauna
- Sound Design Document taairai karauna
- Story aiba Narrative Design laikhauna
- Level Design aira paraikalapanaaa karauna
- UI/UX Wireframe taairai karauna
- Project Schedule taairai karauna

- Risk Assessment karauna
- Milestone Definitions nairadhaaarana karauna

****Deliverables:****

- Game Design Document (GDD) (20-100 parissathaaa)
- Technical Design Document (TDD)
- Art Bible / Style Guide
- Sound Design Document
- Level Design Concepts
- UI/UX Wireframes
- Project Schedule with Milestones
- Risk Register

****Team:****

- Solo Developer: saba kaichhau naijaei karatae habae (chaaapaera samaya)
- Small Team: Game Designer + Technical Artist + Programmer
- Large Team: Game Designer + Level Designer + Technical Artist + Producer + QA Lead

****Tools:****

- Google Docs / Microsoft Word (GDD laekhaara janaya)
- Miro / Lucidchart (Flowcharts aira janaya)
- Figma / Adobe XD (UI Wireframe aira janaya)
- Trello / Jira (Task Management aira janaya)
- Git / Perforce (Version Control aira janaya)

****Estimated Time:** 4-8 sapataaha******Common Mistakes:****

- GDD khaubai lamabaaa laekhaaa (100+ parissathaaa yaaa kaeu padabae naaa)
- GDD aitari baisataaaraita laekhaaa yae paraibaratana karaaa kathaina
- Technical Constraints naaa baibaechanaaa karaaa
- Art Style consistency naaa thaaakaaa
- Milestones asapassata raaakhaaa
- paraikalapanaaaya samaya baeshai kharacha karae kaaaja shaurau naaa karaaa

****Exit Criteria:****

- GDD samapanana aiba taimaera sabaaai baujhaechhae
- Technical Architecture nairadhaaaraita
- Art Style saidadhaaanata hayaechhae
- Project Schedule baaasatabasamamata
- paraaathamaika Risk Mitigation Plan taairai

Phase 4: PROTOTYPE (paraotaotaaipa)

****Goal:****

gaemaera mauula Gameplay Mechanic paraiikassaaa karaaa darauta aiba saaasharayaii mauulayae gaematai kaaaja karae kainaaa taaa yaaachaaai karaaa

****Tasks:****

- Core Game Mechanic baaanaaana
- Player Controller taairai karauna
- maaulaika Input System saetaaapa karauna
- maaulaika Physics saetaaapa karauna
- Placeholder Art bayabahaaara karauna (Programmer Art)
- Core Game Loop paraiikassaaa karauna
- Fun Factor yaaachaaai karauna
- Player Feedback sagaraha karauna
- Iteration karauna (paraibaratana karae aabaaara paraiikassaaa)

****Deliverables:****

- Working Prototype (Playable Build)
- Gameplay Test Report
- Feedback Summary
- Iteration Log
- saidadhaaanata: parakalapatai aigaiyae yaaabae naaakai baaataila habae

****Team:****

- Solo Developer: naijaei paraotaotaaipa baaanaabaena
- Small Team: Programmer + Game Designer
- Large Team: 2-3 Programmers + Game Designer + QA Tester

****Tools:****

- Unity / Unreal Engine / Godot (Game Engine)
- Blender / Maya (yadai 3D Model darakaaara haya)
- Aseprite / Photoshop (yadai 2D Art darakaaara haya)
- FMOD / Wwise (yadai Sound darakaaara haya)
- Git (Version Control)

****Estimated Time:** 2-4 sapataaaha******Common Mistakes:****

- paraotaotaaipae atairikata Polish karaaa (samaya apachaya)
- paraotaotaaipa samapauurana gaemaera matao karaara chaessataaa karaaa
- shaudhaumaaatara "Fun" naaaau hatae paaarae taaa maenae naaa naeayuaa
- baaarabaaara Feature yaoga karaaa (Feature Creep)
- parayaaapata Testing naaa karaaa
- aikatai Prototype ai aatakae thaaakaaa yakhana anaekagaulao paraiikassaaa parayaojana

****Exit Criteria:****

- Core Mechanic kaaaja karachhae

- gaematai "Fun" balae manae hachachhae
 - Player Feedback itaibaaachaka
 - parakalapatai aigaiyae yaaaauyaaara saidadhaaanata garihaiita hayaechhae
-

Phase 5: VERTICAL SLICE (bhaaarataikaaala salaaaisa)

Goal:

gaemaera aikatai chhaota kainatau samapauurana Playable Section taairai karaaa yaaa daekhaaaya gaematai chauudaaanata rauupae kaemana habae

Tasks:

- aikatai Complete Level baaa Section taairai karauna
- Final Art Style bayabahaaara karauna
- Player Character aira samapauurana Animation taairai karauna
- Enemy AI taairai karauna
- Combat System baaasatabaaayana karauna
- UI System taairai karauna
- Sound Design samapanana karauna
- Basic VFX taairai karauna
- Lighting Setup karauna
- Save/Load System baaasatabaaayana karauna
- Win/Lose Condition saetaaapa karauna

Deliverables:

- Vertical Slice Build (Playable Demo)
- Performance Benchmark
- Art Pipeline Validation
- Technical Pipeline Validation
- saidadhaaanata: Full Production shaurau karaaa habae naaakai naaa

Team:

- Solo Developer: saimaita samayae saimaita kaaaja karatae habae
- Small Team: All Team Members
- Large Team: All Department Leads + Core Team

Tools:

- Game Engine (Unity/Unreal/Godot)
- All Art Software (Photoshop, Blender, Maya)
- Sound Software (FMOD, Wwise, Audacity)
- Version Control (Git, Perforce)
- Project Management (Jira, Trello)

****Estimated Time:**** 4-8 sapataaaha

Common Mistakes:

- Vertical Slice khaubai bada karae phaelaaa
- "Production Quality" ai samaya naaa daiyae "Demo Quality" raaakhaaa
- shaudhaumaaatara Visual aira daikae manaoyaoga daeautyaaa, Gameplay upaekassaaa karaaa
- Performance samasayaaa laukaaanao
- parayaaapata QA Testing naaa karaaa

****Exit Criteria:****

- Vertical Slice samapauurana Playable
- Art Quality chauudaaanata gaemaera matao
- Performance garahanayaogaya
- taima Full Production shaurau karaaara janaya parasatauta
- Funding/Investment paelae aii Build bayabahaaara karaaa yaaabae

Phase 6: PRODUCTION (upaaadana)

****Goal:****

gaemaera samapauurana Content taairai karaaa aitai sabachaeyae lamabaaa Phase

****Tasks:****

- All Levels taairai karauna
- All Characters taairai karauna
- All Enemies taairai karauna
- All Weapons/Items taairai karauna
- All Animations taairai karauna
- All Sounds/Music taairai karauna
- All UI Screens taairai karauna
- Story Sequences taairai karauna
- Dialogue System samapanana karauna
- Quest System samapanana karauna
- Skill Tree/Progression System samapanana karauna
- Achievement System taairai karauna
- Options Menu taairai karauna
- Tutorial Section taairai karauna
- naiyamaita QA Testing karauna
- Performance Optimization karauna

****Deliverables:****

- Feature Complete Build
- All Art Assets
- All Sound/Music Assets
- All Levels
- Complete UI
- Full Story/Narrative

- Regular Build Reports

****Team:****

- Solo Developer: sabachaeyae chaaapaera samaya (6-18 maaasa)
- Small Team: All Team Members pauuranakaaala
- Large Team: All Departments samapauurana sakaraiya

****Tools:****

- Game Engine
- All Art Software
- Sound/Music Software
- Version Control
- Bug Tracking System (Jira, Bugzilla)
- Continuous Integration (Jenkins, GitHub Actions)
- Performance Profiling Tools

****Estimated Time:** 6-18 maaasa**

****Common Mistakes:****

- Feature Creep (natauna Feature yaoga karatae thaaakaaa)
- Perfectionism (aikatai jainaisae atairaikata samaya daeayuaa)
- Burnout (atairaikata kaaajaera chaaapa)
- Communication Breakdown (taimaera madhayae yaogaaayaoga bhaengae padaaa)
- Scope Creep (gaemaera aakaaara baaadatae thaaakaaa)
- Quality Compromise (samayamatao shaessa naaa karae maaana kamaanao)
- Testing kama karaaa

****Exit Criteria:****

- saba Feature taairai samapanana
- saba Content taairai samapanana
- gaema Playable aiba Bug-Free (paraaaya)
- Performance garahanayaogaya
- Alpha Build parasatauta

Phase 7: ALPHA (aalaphaaa)

****Goal:****

gaema Playable kainatau asamapauurana Bug Fix shaurau haya abhayanataraiina Testing shaurau haya

****Tasks:****

- Feature Freeze ghaossanaaa karauna (natauna Feature yaoga habae naaa)
- saba Known Bugs taaalaikaaabhaukata karauna
- Bug Fix shaurau karauna
- Internal QA Testing shaurau karauna
- Performance Profiling shaurau karauna

- Memory Leak Detection karauna
- Crash Reporting saetaaapa karauna
- Game Balance paraiikassaaa karauna
- Difficulty Curve paraiikassaaa karauna
- Audio Balance paraiikassaaa karauna
- Localization shaurau karauna (yadai parayaojana)

****Deliverables:****

- Alpha Build (Internal)
- Bug Report Database
- Performance Report
- Game Balance Report
- Known Issues List

****Team:****

- Solo Developer: Bug Fix aiba Optimization ai manaoyaoga
- Small Team: Programmers + QA
- Large Team: QA Team + Programmers + Designers (baaakai taima saaahaaayaya karae)

****Tools:****

- Bug Tracking System (Jira, Bugzilla)
- Performance Profiler (Unity Profiler, Unreal Insights)
- Memory Profiler
- Crash Reporter
- QA Testing Tools

****Estimated Time:**** 2-4 sapataaaha****Common Mistakes:****

- Feature Freeze naaa maaanaaa
- shaudhaumaaatara "Showstopper" Bugs Fix karaaa, baaakai upaekassaaa karaaa
- Performance Optimization ai samaya daeayyaaa naaa
- Internal Testing aira maaanadanada narama raaakhaaa
- Crash Reports upaekassaaa karaaa

****Exit Criteria:****

- saba Known Bugs taaalaikaaabhaukata
- saba Showstopper Bugs Fix karaaa hayaechhae
- Performance sathaitaishaiila
- Game samapauurana Playable
- Beta Build parasatauta

Phase 8: BETA (baetaaa)

****Goal:****

game Feature Complete shaudhau Bug Fix aiba Polish baaakai bahai Testing shaurau haya

****Tasks:****

- External Beta Testing shaurau karauna
- Player Feedback sagaraha karauna
- Bug Fix chaaalaiyae yaaana
- Final Polish shaurau karauna
- Visual Polish karauna
- Audio Polish karauna
- UI Polish karauna
- Performance Final Optimization karauna
- Localization Review karauna
- Compliance Testing karauna (Platform Requirements)
- Rating Submission parasatauta karauna (ESRB, PEGI)
- Marketing Material taairai karauna

****Deliverables:****

- Beta Build (External)
- Player Feedback Report
- Bug Fix Report
- Final Polish Report
- Compliance Report
- Rating Submission

****Team:****

- Solo Developer: Bug Fix aiba Polish
- Small Team: All Team Members
- Large Team: QA Team + Marketing Team + PR Team

****Tools:****

- Beta Distribution Platform (Steam Beta, TestFlight)
- Feedback Collection Tools (Survey, Forum)
- All Development Tools

****Estimated Time:**** 2-6 sapataaaha****Common Mistakes:****

- baeshai Beta Testers naaa naeauyaaa
- Player Feedback gaurautaba naaa daeauyaaa
- Last Minute Feature Additions
- Polish Phase ai samaya naaa daeauyaaa
- Rating Submission samayamatao naaa karaaa
- Marketing shaurau naaa karaaa

****Exit Criteria:****

- saba Critical Bugs Fix karaaa
 - Player Feedback samaaadhaana karaaa
 - Performance chauudaaanata
 - Compliance samapanana
 - Release Candidate Build parasatauta
-

Phase 9: RELEASE CANDIDATE (railaija kayaaanadaidaeta)

****Goal:****

gaema Release-aira janaya parasatauta shaudhaumaaatara Critical Bugs thaaakalae Fix karaaa habae

****Tasks:****

- Final Build taairai karauna
- Platform Certification Testing karauna (PlayStation, Xbox, Nintendo)
- Final QA Testing karauna
- Final Performance Testing karauna
- Final Localization Review karauna
- Final Audio Mastering karauna
- Final Visual Polish karauna
- Release Notes laikhauna
- Marketing Campaign parasatauta karauna
- Distribution Setup karauna
- Launch Day Plan taairai karauna

****Deliverables:****

- Release Candidate Build
- Platform Certification Results
- Final QA Report
- Release Notes
- Marketing Materials
- Launch Day Checklist

****Team:****

- Solo Developer: saba shaessa parasatautai
- Small Team: All Team Members
- Large Team: All Departments + Marketing + PR

****Tools:****

- Platform Specific Tools (PlayStation Dev Kit, Xbox Dev Kit)
- All Development Tools
- Marketing Tools (Social Media, Press Kit)
- Distribution Platforms (Steam, App Store, Console Stores)

****Estimated Time:**** 1-2 sapataaaha

****Common Mistakes:****

- shaessa mauhauratae Bug Fix karaara chaessataaa
- Platform Certification ai bayaratha haauyaaa
- Marketing Material samayamatao parasatauta naaa haauyaaa
- Launch Day Plan naaa thaaakaaa
- Server Setup (yadai Online Game haya)

****Exit Criteria:****

- Platform Certification samapanana
- saba Critical Bugs Fix karaaa
- Final Build parasatauta
- Marketing Materials parasatauta
- Launch Day Plan parasatauta

Phase 10: LAUNCH (lanyacha)

****Goal:****

gaema paaabalaisha aiba maaarakaetai kaaarayakarama paraichaaalanaaa

****Tasks:****

- Game paaabalaisha karauna
- Launch Day Monitoring karauna
- Server Monitoring karauna (yadai Online Game haya)
- Player Support paradaaana karauna
- Social Media Monitoring karauna
- Press Coverage Monitor karauna
- First Day Sales Data sagaraha karauna
- Player Reviews Monitor karauna
- Emergency Bug Fix parasatauta raaakhauna
- Day One Patch parasatauta raaakhauna

****Deliverables:****

- Published Game
- Launch Day Report
- Sales Report
- Player Feedback Summary
- Media Coverage Summary
- Day One Patch (yadai parayaojana)

****Team:****

- Solo Developer: saba kaichhau naijaei karatae habae
- Small Team: All Team Members
- Large Team: All Departments + Marketing + PR + Support

****Tools:****

- Distribution Platforms (Steam, App Store, Console Stores)
- Analytics Tools
- Social Media Management Tools
- Customer Support Tools
- Monitoring Tools

****Estimated Time:**** 1-2 sapataaaha

****Common Mistakes:****

- Server Crash (Online Games)
- khaaaraaapa Player Support
- Negative Reviews aira saaathae khaaaraaapa aacharana
- Day One Patch naaa thaaakaaa
- Marketing Campaign ai samasayaaa
- Sales Expectation abaaasataba haauyaaa

****Exit Criteria:****

- Game saphalabhaaabae paaabalaisha hayaechhae
- Server Stable (yadai Online Game haya)
- Player Support sakaraiya
- Emergency Plans parasatauta

Phase 11: POST-LAUNCH (paosata-lanyacha)

****Goal:****

gaema aapadaeta, Patch, DLC, aiba Player Feedback bhaitataika unanatai

****Tasks:****

- Player Feedback sagaraha karauna
- Bug Reports samaaadhaana karauna
- Patches parakaaasha karauna
- DLC/Expansion paraikalapanaaa karauna
- Community Management karauna
- Content Updates parakaaasha karauna
- Performance Updates parakaaasha karauna
- New Features (yadai parayaojana)
- Season Events (yadai parayaojana)
- Analytics Review karauna
- Player Retention Analysis karauna
- Monetization Analysis karauna
- Next Project Planning shaurau karauna

****Deliverables:****

- Regular Patches
- DLC/Expansion Content
- Community Updates
- Analytics Reports
- Player Retention Reports
- Next Project Proposal

****Team:****

- Solo Developer: saiimaita aapadaeta
- Small Team: Smaller Maintenance Team
- Large Team: Live Service Team + Community Team

****Tools:****

- Live Service Tools
- Analytics Platforms
- Community Management Tools
- Customer Support Tools

****Estimated Time:**** 3-12 maaasa (athabaaa anairadaissatakaaala)

****Common Mistakes:****

- Player Feedback upaekassaaa karaaa
- Patches samayamatao naaa daeauyaaa
- Community Management upaekassaaa karaaa
- DLC aira janaya atairaikata chaaaraja karaaa
- gaema chhaedae natauna parakalapae chalaе yaaaauyaaa
- Player Expectation Management naaa karaaa

****Exit Criteria:****

- Player Base Stable
- Revenue Target pauurana
- Community Happy
- Next Project Ready

9.4 Phase Comparison Table

9.5 Practical Example: Indie Game Development Roadmap

dharauna aapanai aikatai Indie Horror Game baaanaataae chaaana aikhaaanae aikatai baaasatabasamamata Timeline:

Indie Horror Game Development Timeline

Month 1-2: IDEA + RESEARCH
Horror Sub-genre nairadhaaarana (Psychological, Survival, etc.)

Reference Games (Outlast, Amnesia, Phasmophobia)
Target Platform (PC first, Console later)
Budget Estimation

Month 3-4: PRE-PRODUCTION

GDD laekhaaa (30-50 parissathaaa)
Art Style (Dark, Realistic)
Sound Design Plan (Ambient Horror)
Story & Characters
Level Design Concepts

Month 5: PROTOTYPE

Player Controller (First Person)
Movement & Interaction
Basic Enemy AI
Horror Atmosphere Test
Fun Factor Test

Month 6-7: VERTICAL SLICE

One Complete Level
Final Art Style
Enemy Behavior
Puzzle System
Save System
UI System

Month 8-14: PRODUCTION

5-7 Complete Levels
All Enemies
All Puzzles
Full Story
Complete Sound Design
Complete Music
Full UI

Month 15: ALPHA

Feature Freeze
Bug Fix
Internal Testing
Performance Optimization

Month 16: BETA

External Testing
Player Feedback
Final Bug Fix
Polish

Month 17: RELEASE CANDIDATE

Final Build
Platform Certification
Marketing Push
Press Kit

Month 17 (End): LAUNCH

Release on Steam
Launch Day Monitoring
Player Support
Community Management

Month 18+: POST-LAUNCH

Bug Patches
Player Feedback Updates
DLC Planning
Next Game Planning

Total: 17-18 maaasa (Solo/Small Team)

9.6 Exercise (anaushailana)

Exercise 1: Personal Roadmap

aapanaaara naijaera aikatai Game Development Project-aira janaya aikatai Roadmap taairai karauna:

- parataitai Phase aira janaya samaya nairadhaaarana karauna
- parataitai Phase aira Tasks laisata karauna
- Milestones saeta karauna
- Risk Assessment karauna

Exercise 2: Timeline Analysis

aikatai Published Game-aira Development Timeline samaparakaе gabaessanaаа karauna (yaemana: Hollow Knight, Celeste, Hades) taaadaera kaona Phase ai kata samaya laegaechhae taaa laikhauna

Exercise 3: Risk Assessment

aapanaaara parakalapaera janaya kamapakassae 5tai Potential Risk chaihanaite karauna aiba parataitaira janaya Mitigation Plan taairai karauna

9.7 Common Mistakes Summary (saaadhaaarana bhaula)

1. **No Planning**: paraikalapanaaa chhaaadaаа saraаasarai kaaaja shaurau karaаа
2. **Scope Creep**: gaemaera aakaaara dhairae dhairae baaadaiyae daeаuyaaa
3. **Perfectionism**: aikatai Phase ai atairaikata samaya daeаuyaaa
4. **Ignoring Testing**: parayaaapata Testing naаа karaаа
5. **Poor Communication**: taimaera madhayae yaogaaayaoga bhaaangaaa
6. **No Milestones**: Milestone saeta naаа karae kaaaja karaаа
7. **Ignoring Feedback**: Player/Tester Feedback upaekassaaa karaаа
8. **Burnout**: atairaikata kaaajaera chaaapae kalaaanata haаuyaaa
9. **Skipping Phases**: kaonao Phase aidaiyae yaaaаuyaaa
10. **Late Marketing**: shaessa samayae Marketing shaurau karaаа

9.8 Pro Tips

1. **Start Small**: chhaota gaema daiyae shaurau karauna, bada gaema naаа
2. **Iterate**: parataibaaara unanatai karauna, paaaraphaekata hatae chaessataаа naаа karauna
3. **Document Everything**: saba kaichhau laikhae raaakhauna
4. **Communicate**: taimaera saaathae naiyamaita kathaaa balauna
5. **Test Early, Test Often**: yata taaadaаataаadai samabhaba Testing shaurau karauna
6. **Set Realistic Goals**: baaasatabasamamata lakassaya saeta karauna
7. **Take Breaks**: bairatai naina, Burnout aidaiyae chalauna
8. **Learn from Mistakes**: bhaula thaekae shaikhauna

9. ****Stay Focused****: aikatai kaaaja shaessa karae paraera kaaajae yaaana
10. ****Have Fun****: majaaa karauna, aitaii sabachaeyae gaurautabapauurana
-

9.9 Summary (saaarasakassaepa)

Game Development Roadmap halao aikatai gaema taairaira samapauurana patha 11tai Phase ai bhaaaga karae aamaraaa parataitai dhaapae kaii karatae habae, kata samaya laaagabae, aiba kaii bhaula aidaaatae habae taaa jaaanatae paaarai sabachaeyae gaurautabapauurana baissaya halao:

- ****paraikalapanaaa karauna****: chhaota thaekae shaurau karauna, dhairae dhairae baaadaaana
- ****Phase aidaiyae yaaana naaa****: parataitai Phase aira gaurautaba aachhae
- ****Testing karauna****: yata taaadaaataaadaai samabhaba Testing shaurau karauna
- ****Feedback naina****: Player aiba Tester daera kathaaa shaunauna
- ****Flexible thaaakauna****: paraikalapanaaa paraibaratana karatae hatae paaarae
- ****Enjoy karauna****: Game Development majaaara haauyaaa uchaita

manae raaakhabaena, *****A delayed game is eventually good, but a rushed game is forever bad.***** - Shigeru Miyamoto

parabarataii adhayaaayae aamaraaa Solo Indie Game Development samaparakee baisataaaraita aalaochanaaa karabao

PART 10

adhayaaaya 10: SOLO INDIE GAME DEVELOPMENT

adhayaaaya 10: SOLO INDIE GAME DEVELOPMENT

shaekhhaara udadaeshaya (Learning Goal)

aai adhayaaayae aapanai shaikhabaena kaiibhaaabaee aikaaa aikajana (Solo Developer) Game Develop karatae paaaraena daijaaaina, paraogaraaamai, Art, Animation, UI, Audio, Level Design, QA, aiba Marketing sabakaichhau aikaaa kaiibhaaabaee paraichhaalanaaa karabaena taaa baisataaaraita aalaochanaaa karabao

10.1 Solo Developer kae?

Solo Developer halaena aimana aikajana bayakatai yainai aikaaa aikajana paura Game Development parakaraiyaaa paraichhaalanaaa karaena tainai aikai saaathae Designer, Programmer, Artist, Sound Designer, aiba Marketer

****Solo Developer haauiyaaara saubaidhaa:****

- samapauurana Creative Control
- naijaera gataitae kaaaja karaaara sabaaadhainataaa
- kaonao maitai baaa Communication Overhead naei
- Profit samapauurana naijaera (yadai gaema baikarai haya)
- naijaera Skill unanata karaaara sauyaoga

****Solo Developer haauiyaaara chayaalaenyaja:****

- saba kaichhau aikaaa karatae haya
 - Skill Gap thaaakatae paaarae
 - Burnout aira jhaukai baeshai
 - Time Management kathaina
 - Motivation dharae raaakhaaa kathaina
 - baaajaaarajaaatakarana (Marketing) kathaina
-

10.2 1 Person Game Studio Workflow Diagram

1 PERSON GAME STUDIO WORKFLOW

SOLO DEVELOPER (You + You)		
DESIGN	DEVELOP	ART
* Concept	* Code	* 2D/3D
* GDD	* Engine	* Anim
* Level	* Physics	* UI Art
* Balance	* AI	* VFX
AUDIO	QA	MARKETING
* SFX	* Testing	* Social
* Music	* Bug Fix	* Steam
* Voice	* Polish	* Press
* Ambient	* Balance	* Community
LAUNCH		

Daily Workflow: Morning (Design/Code) -> Afternoon (Art/Audio)
Evening (QA/Marketing) -> Night (Learning)

10.3 parataitai kaaaja paraichaaalanaaaa karaaara padadhatai

10.3.1 Design (daijaaaina)

****kaii karatae habae:****

- Game Concept laikhauna
- Game Design Document (GDD) taairai karauna
- Gameplay Mechanics daijaaaina karauna
- Level Layout paraikalapananaaaa karauna
- Difficulty Curve paraikalapananaaaa karauna
- Player Experience Flow paraikalapananaaaa karauna

****Solo Developer aira janaya Tips:****

- GDD chhaota raaakhauna (10-20 parissathaaa)
- shaudhamaaataara Essential Information laikhauna
- Reference Games bayabahaaara karauna
- naijaei Playtest karauna
- Player Feedback naina

****Tools:****

- Google Docs / Notion (GDD laekhaaara janaya)
- Trello / Notion (Task Management)
- Miro (Brainstorming)
- Pen & Paper (Quick Sketch)

****Time Allocation:**** 10-15% samaya

10.3.2 Programming (paraogaraaamai)

****kaii karatae habae:****

- Game Engine Setup karauna
- Player Controller laikhauna
- Game Logic laikhauna
- UI System taairai karauna
- Save/Load System taairai karauna
- Enemy AI laikhauna
- Physics System saetaaapa karauna
- Input System saetaaapa karauna
- Audio System saetaaapa karauna
- Optimization karauna

****Solo Developer aira janaya Tips:****

- aikatai Engine ai Master haona (Unity, Godot, athabaaa Unreal)
- Reusable Code laikhauna
- Version Control (Git) bayabahaaara karauna
- Asset Store bayabahaaara karauna
- Documentation laikhauna
- Test-Driven Development anausarana karauna

****Tools:****

- Unity / Godot / Unreal Engine
- Visual Studio / VS Code
- Git / GitHub
- Blender (3D Modeling)

****Time Allocation:**** 30-40% samaya

****Recommended Learning Path:****

Intermediate: Design Patterns, State Machines, Event Systems

Advanced: Optimization, Shaders, Networking, Platform-specific

Beginn

10.3.3 Art (shailapa)

****kaii karatae habae:****

- Character Design karauna
- Environment Art taairai karauna
- UI Art taairai karauna
- Props taairai karauna
- Texture taairai karauna
- Animation taairai karauna
- VFX taairai karauna
- Concept Art taairai karauna

****Solo Developer aira janaya Tips:****

- aikatai Consistent Art Style baechhae naina
- Simple Art Style baechhae naina (Pixel Art, Low Poly, Stylish)
- Asset Store bayabahaaara karauna (Placeholder haisaebae)
- Free Assets bayabahaaara karauna shaekhaaara janaya
- Commission Artist (yadai baaajaaara thaaakae)
- AI Art Tools bayabahaaara karatae paaaraena (Concept aira janaya)

****Recommended Art Styles for Solo Dev:****

Medium: Low Poly 3D, Hand-drawn 2D, Voxel
Hardest: Realistic 3D, High-res 2D, Stylized 3D

****Tools:****

- Aseprite (Pixel Art)
- Blender (3D Modeling - Free)
- GIMP / Krita (2D Art - Free)
- Substance Painter (Texture)
- Spine / DragonBones (2D Animation)
- Adobe After Effects (VFX)

****Time Allocation:**** 20-30% samaya

10.3.4 Animation (ayaaanaimaeshana)

****kaii karatae habae:****

- Character Animation taairai karauna
- Environment Animation taairai karauna
- UI Animation taairai karauna
- VFX Animation taairai karauna
- Cutscene Animation taairai karauna

****Solo Developer aira janaya Tips:****

- Limited Animation bayabahaaara karauna (kama Frame)
- Procedural Animation bayabahaaara karauna
- Animation Libraries bayabahaaara karauna
- Simple Animation daiyae shaurau karauna
- Tweening/Interpolation bayabahaaara karauna

****Tools:****

- Blender (3D Animation)
- Spine (2D Animation)
- DragonBones (2D Animation)
- Adobe Animate (2D Animation)

****Time Allocation:**** 5-10% samaya

10.3.5 UI (iujaara inataaaraphaesa)

****kaii karatae habae:****

- Main Menu taairai karauna
- HUD (Heads-Up Display) taairai karauna
- Inventory Screen taairai karauna
- Settings Menu taairai karauna
- Pause Menu taairai karauna
- Dialogue System UI taairai karauna
- Loading Screen taairai karauna
- Victory/Game Over Screen taairai karauna

****Solo Developer aira janaya Tips:****

- Simple aiba Clean UI taairai karauna
- Player Feedback daekhauna
- UI Kit bayabahaaara karauna
- Responsive Design raaakhauna
- Accessibility baibaechanaaaa karauna

****Tools:****

- Figma / Adobe XD (UI Design)
- Game Engine UI System
- UI Asset Packs

****Time Allocation:**** 5-10% samaya

10.3.6 Audio (adaiau)

****kaii karatae habae:****

- Sound Effects (SFX) taairai karauna
- Background Music (BGM) taairai karauna
- Ambient Sounds taairai karauna
- Voice Over (yadai parayaojana) raekarada karauna
- Audio Mixing karauna
- Audio Mastering karauna

****Solo Developer aira janaya Tips:****

- Free Sound Libraries bayabahaaara karauna
- Royalty-Free Music bayabahaaara karauna
- naijae Sound Design shaikhauna
- Simple Audio daiyae shaurau karauna
- Audio Middleware (FMOD, Wwise) bayabahaaara karauna

****Free Resources:****

Music: Free Music Archive, Incompetech, Bensound
Tools: Audacity (Free), LMMS (Free), Reaper (Cheap)

****Tools:****

- Audacity (Free Sound Editor)
- LMMS (Free DAW)
- Reaper (Affordable DAW)
- FMOD / Wwise (Audio Middleware)

****Time Allocation:**** 5-10% samaya

10.3.7 Level Design (laebhaela daijaaaina)

****kaii karatae habae:****

- Level Layout paraikalapanaaa karauna
- Level Flow daijaaaina karauna
- Puzzle Design karauna
- Enemy Placement karauna
- Item Placement karauna
- Checkpoint System saetaapa karauna
- Environment Storytelling karauna
- Difficulty Progression paraikalapanaaa karauna

****Solo Developer aira janaya Tips:****

- chhaota Level daiyae shaurau karauna
- Blockout parathamae karauna (Basic Shapes)
- Playtest karauna parataitai Level
- Player Feedback naina
- Iteration karauna

****Tools:****

- Game Engine Level Editor
- ProBuilder (Unity)
- BSP Brushes (Unreal)

****Time Allocation:**** 10-15% samaya

10.3.8 QA (kaoyaalaitai aishauraenasa)

****kaii karatae habae:****

- naijaei Playtest karauna
- Bug Reports laikhauna
- Bug Fix karauna

- Performance Testing karauna
- Compatibility Testing karauna
- User Experience Testing karauna
- Balance Testing karauna
- Accessibility Testing karauna

****Solo Developer aira janaya Tips:****

- naijae thaekae dauurae sarae gaiyae Playtest karauna
- Friends/Family kae Playtest karaana
- Online Community thaekae Feedback naina
- Bug Tracking System bayabahaaara karauna
- Prioritize Bugs (Critical, Major, Minor)

****Tools:****

- Trello / Notion (Bug Tracking)
- Steam Beta Testing
- Discord Community

****Time Allocation:**** 5-10% samaya

10.3.9 Marketing (maaraketai)

****kaii karatae habae:****

- Steam Page taairai karauna
- Social Media Accounts taairai karauna
- Regular Updates daina (Development Logs)
- Trailer taairai karauna
- Screenshots taairai karauna
- Press Kit taairai karauna
- Influencer Outreach karauna
- Community Building karauna
- Email List taairai karauna
- Steam Next Fest Participate karauna

****Solo Developer aira janaya Tips:****

- Development aira saaathae saaathae Marketing shaurau karauna
- Regular Content Share karauna
- Authentic thaaakauna
- Player Community taairai karauna
- Free Marketing Channels bayabahaaara karauna

****Free Marketing Channels:****

Reddit: r/gamedev, r/indiegaming, r/indiegames
 YouTube: Devlog Videos, Dev Interviews

Twitter

Discord: Community Server
TikTok: Short Gameplay Clips
Steam: Steam Page, Steam Next Fest
itch.io: Free Demo Distribution

****Tools:****

- Twitter/X (Social Media)
- Reddit (Community)
- YouTube (Video Content)
- Discord (Community)
- OBS Studio (Screen Recording - Free)
- DaVinci Resolve (Video Editing - Free)

****Time Allocation:**** 10-15% samaya

10.4 Weekly Schedule Example (saaapataaahaika sauuchai)

SOLO DEVELOPER WEEKLY SCHEDULE

SATURDAY (shanaibaaara) - DESIGN & PLANNING DAY

09:00 - 10:00 Weekly Planning & Goals
10:00 - 12:00 GDD Updates & Design Work
12:00 - 13:00 Lunch Break
13:00 - 15:00 Level Design Planning
15:00 - 16:00 Research & Reference Gathering
16:00 - 17:00 Review & Week Prep

SUNDAY (rabaibaaara) - PROGRAMMING DAY

09:00 - 10:00 Code Review & Bug Fixes
10:00 - 12:00 Core Feature Development
12:00 - 13:00 Lunch Break
13:00 - 15:00 New Feature Implementation
15:00 - 16:00 Testing & Debugging
16:00 - 17:00 Documentation

MONDAY (saomabaaara) - PROGRAMMING DAY

09:00 - 10:00 Code Review & Bug Fixes
10:00 - 12:00 AI/Physics Development
12:00 - 13:00 Lunch Break
13:00 - 15:00 UI System Development
15:00 - 16:00 Save/Load System
16:00 - 17:00 Optimization

TUESDAY (mangalabaaara) - ART DAY

09:00 - 10:00 Concept Art
10:00 - 12:00 Character/Environment Art
12:00 - 13:00 Lunch Break
13:00 - 15:00 Animation Work
15:00 - 16:00 UI Art
16:00 - 17:00 Texture & Material

WEDNESDAY (baudhabaaara) - ART & AUDIO DAY

09:00 - 10:00 Art Polish
10:00 - 12:00 VFX Creation
12:00 - 13:00 Lunch Break
13:00 - 14:00 Sound Effects
14:00 - 15:00 Music Selection

15:00 - 16:00 Audio Integration

16:00 - 17:00 Audio Mixing

THURSDAY (barihasapataibaaara) - LEVEL DESIGN DAY

09:00 - 10:00 Level Layout

10:00 - 12:00 Level Building

12:00 - 13:00 Lunch Break

13:00 - 15:00 Level Polish

15:00 - 16:00 Puzzle Integration

16:00 - 17:00 Level Testing

FRAYDAY (shaukarabaaara) - QA & MARKETING DAY

09:00 - 10:00 Bug Testing

10:00 - 12:00 Bug Fixes

12:00 - 13:00 Lunch Break

13:00 - 14:00 Playtesting

14:00 - 15:00 Social Media Updates

15:00 - 16:00 Devlog Writing

16:00 - 17:00 Community Engagement

Weekly Total: 45-50 ghanataaa (Part-time) / 60-70 ghanataaa (Full-time)

10.5 Time Management Strategies (samaya bayabasathaaapanaaa)

Pomodoro Technique

25 mainaita kaaaja -> 5 mainaita bairatai -> 25 mainaita kaaaja -> 5 mainaita bairatai

-> 4tai Pomodoro aira para 15-30 mainaita lamabaaa bairatai

Time Blocking

- nairadaissata samayae nairadaissata kaaaja karauna
- aika kaaaja thaekae anaya kaaajae saaaita karabaena naaa
- Calendar bayabahaaara karauna
- Reminders saeta karauna

The 2-Minute Rule

- yadai kaonao kaaaja 2 mainaitaera kama samayae shaessa karaaa yaaaya, taaahalae saaathae saaathae karauna
- aita Task List chhaota thaaakae

Eisenhower Matrix

	Urgent	Not Urgent
Important	DO FIRST (Bugs, Deadlines)	SCHEDULE (New Features, Learning)
Not Important	DELEGATE (Minor Tasks)	ELIMINATE (Distractions)

10.6 Skill Prioritization (dakassataaara agaraaadhaikaaara)

SKILL PRIORITIZATION FOR SOLO DEVELOPER

Priority 1 (ESSENTIAL): aigaulao chhaaadaaa Game baaanaanao samabhaba naaa
Programming (Core Logic, Player Controller)
Game Design (Mechanics, Balance, Flow)
Basic Art (Placeholder, UI)
Basic Audio (SFX, Ambient)

Priority 2 (IMPORTANT): aigaulao gaemakae bhaaalao karae
Level Design
UI/UX Design
Animation (Basic)
Sound Design
QA Testing

Priority 3 (NICE TO HAVE): aigaulao gaemakae chamakaaara karae
Advanced Art (High Quality)
Advanced Animation
Advanced Audio (Original Music)
VFX
Advanced AI

Priority 4 (OUTSOURCE): samabhaba halae baaairae karaaana
Professional Music Composition
Voice Acting
Professional Art
Marketing Campaign
Localization

Learning Order: Programming -> Game Design -> Basic Art ->
Level Design -> UI/UX -> Audio -> Advanced Art

10.7 Tool Recommendations (saranyajama saupaaaraisha)

Game Engine

Beginner Friendly:
Godot (Free, Open Source, GDScript)
Unity (Free tier, C#, Large Community)
Construct (Browser-based, Visual Scripting)

Professional:
Unreal Engine (Free, C++/Blueprints, AAA Quality)
GameMaker (2D Focus, GML)

Art Tools

2D Art:
Aseprite (Pixel Art - Paid but worth it)
GIMP (Free Photoshop Alternative)
Krita (Free, Digital Painting)
Piskel (Free Pixel Art Online)

3D Modeling:
Blender (Free, Industry Standard)
Blockbench (Free, Low Poly)

Animation:
Spine (2D Animation - Paid)
DragonBones (2D Animation - Free)
Blender (3D Animation - Free)

Audio Tools

DAW (Digital Audio Workstation):

- LMMS (Free)
- Audacity (Free Sound Editor)
- Reaper (Affordable)
- FL Studio (Professional)

Sound Effects:

- Freesound.org (Free)
- ZapSplat (Free)
- SoundBible (Free)

Productivity Tools

Project Management:

- Trello (Free)
- Notion (Free)
- Obsidian (Free)
- Jira (Free tier)

Version Control:

- Git + GitHub (Free)
- GitLab (Free)
- Plastic SCM (Free for small teams)

Documentation:

- Google Docs (Free)
- Notion (Free)
- Obsidian (Free)

10.8 Solo Game Examples (Solo Developer daera saphala gaema)

SUCCESSFUL SOLO DEVELOPER GAMES		
Game	Developer	Genre
Stardew Valley	Eric Barone	Farming RPG
Undertale	Toby Fox	RPG
Celeste	Maddy Thorson	Platformer
Cave Story	Daisuke Amaya	Action
Hyper Light Drifter	Alex Preston	Action RPG
Shovel Knight	Sean Velasco	Platformer
Hollow Knight	Team Cherry (3)	Metroidvania
Papers, Please	Lucas Pope	Simulation
Return of the Obra	Lucas Pope	Mystery
Baba Is You	Arvi Teikari	Puzzle

Lesson: Start small, focus on core mechanics, polish everything

10.9 Exercise (anaushailana)

Exercise 1: Self Assessment

naijaera Skills aira aikatai Assessment karauna:

- kaona Skill ai aapanai bhaaalao?

- kaona Skill ai aapanai daurabala?
- kaona Skill shaikhatae habae?
- kaona kaaaja Outsource karatae paaaraena?

Exercise 2: Weekly Schedule

aapanaaara naijaera janaya aikatai Weekly Schedule taairai karauna:

- aapanai kata ghanataaa Game Development karatae paaarabaena?
- kaona samaya kaona kaaaja karabaena?
- bairatai kakhana naebaena?

Exercise 3: Tool Selection

aapanaaara parakalapaera janaya Tools baechhae naina:

- Game Engine kaonatai bayabahaaara karabaena?
- Art Tools kaonagaulao bayabahaaara karabaena?
- Audio Tools kaonagaulao bayabahaaara karabaena?

10.10 Common Mistakes Summary (saaadhaaarana bhaula)

1. ****Scope Too Big****: parathama gaemaei bada gaema baaanaanaora chaessataaa
2. ****No Learning****: parayaaapata shaekharaa aagaei kaaaja shaurau karaaa
3. ****Perfectionism****: aikatai jainaisae atairaikata samaya daeuyaaa
4. ****No Marketing****: shaudhaumaaatara Development ai manaoyaoga daeuyaaa
5. ****No Community****: Player Community taairai naaa karaaa
6. ****Burnout****: atairaikata kaaajaera chaaapae kalaaanata haauyaaa
7. ****No Feedback****: Player Feedback naaa naeuyaaa
8. ****Skipping QA****: parayaaapata Testing naaa karaaa
9. ****No Version Control****: Git naaa bayabahaaara karaaa
10. ****No Plan****: paraikalapananaa chhaadaaa kaaaja shaurau karaaa

10.11 Pro Tips

1. ****Start Tiny****: sabachaeyae chhaota gaema daiyae shaurau karauna (Pong, Breakout)
 2. ****Finish Games****: gaema shaessa karauna, Perfect karatae yaaabaena naaa
 3. ****Learn by Doing****: parayaaakataisa karae shaikhauna
 4. ****Join Communities****: Reddit, Discord, Twitter ai Active thaaakauna
 5. ****Share Progress****: naijaera Progress Share karauna
 6. ****Take Breaks****: naiyamaita bairatai naina
 7. ****Set Deadlines****: naijaera janaya Deadline saeta karauna
 8. ****Celebrate Wins****: chhaota chhaota jaya udayaaapana karauna
 9. ****Stay Motivated****: Motivation bajaaaya raaakhauna
 10. ****Have Fun****: majaaa karauna, aitai sabachaeyae gaurautabapauurana
-

10.12 Summary (saaarasakassaepa)

Solo Indie Game Development halao aikatai chayaaalaenyajai kainatau majaaara yaaataraaa aikaaa aikajana sabakaichhau karaaa samabhaba, tabae aira janaya darakaaara:

- **Skill Development**: parayaojanaiiya Skills shaikhauna
- **Time Management**: samaya sausharingakhalabhaaaba bayabasathaaapanaaa karauna
- **Tool Selection**: sathaika Tools baechhae naina
- **Scope Management**: chhaota gaema daiyae shaurau karauna
- **Marketing**: Development aira saaathae Marketing karauna
- **Community**: Player Community taairai karauna
- **Health**: naijaera sabaaasathayaera yatana naina

manae raaakhabaena, **"The best game to make is the one you can finish."** chhaota shaurau karauna, dhairae dhairae bada karauna, aiba sabachaeyae gaurautabapauurana - **majaaa karauna!**

parabarataii adhayaaayae aamaraaa Prototype samaparakaе baіsataaaraita aalaochanaaa karabao

PART 11

adhayaaaya 11: PROTOTYPE

adhayaaaya 11: PROTOTYPE

shaekhaara udadaeshaya (Learning Goal)

aai adhayaaayae aapanai shaikhabaena Prototype kii, kaena darakaaara, kakhana baaanaatae haya, aiba kaiibhaaaba 7 dainae aikatai Complete Prototype taairai karatae paaaraena aichhaadaaaaau Prototype ai kii thaaakabae aiba kii thaaakabae naaa taaa baisataaaraita aalaochanaaa karabao

11.1 Prototype kii? (What is a Prototype?)

Prototype halao aikatai gaemaera mauula Gameplay Mechanic-aira darauta, saaasharayai mauulayaera aiba asamapauurana sasakarana aitai gaemataira "Fun" kainaaa taaa paraiikassaaa karaara janaya taairai karaaa haya

****Prototype aira mauula baaishaissataya:****

- darauta taairai karaaa haya (daina thaekae sapataaaha parayanata)
- saaasharayai mauulayae taairai karaaa haya
- Placeholder Art bayabahaara karaaa haya (Programmer Art)
- shaudhaumaatara Core Mechanic phaokaaasa karaaa haya
- paraibaratana sahaja haya
- "Fun" paraiikassaaa karaaa haya
- Risk kamaaaya

****Prototype aira udaaaharana:****

- aikatai chhaota Box daiyae Player Character
 - Colored Cubes daiyae Environment
 - Simple Sounds daiyae Feedback
 - Basic UI daiyae Score/Health
-

11.2 kakhana Prototype baaanaatae habae? (When to make a Prototype?)

WHEN TO MAKE A PROTOTYPE

YES, Make a Prototype When:

aapanaaara aikatai natauna Game Idea aachhae
 aapanai aikatai natauna Mechanic Test karatae chaaana
 aapanai Fundraising karatae chaaana (Investors/Publishers)
 aapanai Game Jam Participate karachhaena
 aapanai natauna Technology Test karatae chaaana
 aapanai Player Feedback naitae chaaana
 aapanai natauna Team Member kae Train karatae chaaana
 aapanai baikalapa Mechanics Compare karatae chaaana

NO, Don't Make a Prototype When:

aapanaaara GDD itaimadhyae Complete aiba Approved
 aapanai Vertical Slice Phase ai aachhaena
 aapanaaara pauurabaera Prototype aikhanao Valid
 samaya/arathaera abhaaaba aachhae
 aapanai shaudhau "Perfect" gaema baaanaatae chaaana
 aapanai atairaikata Feature Test karatae chaaana (Scope Creep)

11.3 Prototype aira Goal (Purpose)

****mauula Goals:****

1. ****Fun Test****: gaematai majaaara kainaaa taaa paraiikassaaa karaaa
2. ****Feasibility Test****: parayaukataigata daika thaekae samabhaba kainaaa taaa paraiikassaaa karaaa
3. ****Market Test****: Player daera gaematai pachhanada karabae kainaaa taaa paraiikassaaa karaaa
4. ****Risk Reduction****: bada samaya/aratha bainaiyaogaera aagae Risk kamaanao
5. ****Learning****: gaematai samaparakae natauna kaichhau shaekhaa
6. ****Communication****: taima/Investors kae gaemaera dhaaaranaaa baojhanaanao

11.4 Prototype ai kaii thaaakabae aiba kaii thaaakabae naaa

PROTOTYPE: WHAT SHOULD BE IN vs NOT

WHAT SHOULD BE IN A PROTOTYPE:

Core Gameplay Mechanic
 Basic Player Movement
 Basic Input Handling
 Basic Physics
 Placeholder Art (Cubes, Shapes)
 Basic Sound Effects (Optional)
 Simple UI (Score, Health)
 Win/Lose Condition (Basic)
 Basic Enemy Behavior (if needed)
 Level Layout (Blockout)
 Fun Factor

WHAT SHOULD NOT BE IN A PROTOTYPE:

Final Art/Textures
 Complex Animations
 Full Sound Design
 Complete Music
 Advanced UI Design
 Save/Load System

- Achievement System
 - Full Story/Narrative
 - Tutorial System
 - Options Menu
 - Multiple Levels
 - Complex AI
 - Multiplayer System
 - Monetization System
 - Localization
 - Platform Optimization
 - Marketing Materials
- Rule of Thumb: Prototype ai shaudhaumaaatara "Fun" Test karauna

11.5 7-Day Prototype Challenge (7 dainaera paraotaotaaaipa chayaaalaenyaja)

Challenge Overview

aii Challenge ai aamaraaa 7 dainae aikatai Complete Prototype taairai karabao aamaraaa aikatai Simple 2D Platformer Game baaanaaabao yaaatae Player, Enemy, Platforms, Collectibles, aiba Basic UI thaaakabae

7-DAY PROTOTYPE CHALLENGE

- Game: Simple 2D Platformer
- Engine: Unity / Godot (Free)
- Goal: Complete playable prototype in 7 days
- Time: 3-5 hours per day (Total: 21-35 hours)

Day 1: Project Setup & Player Movement

Goal: Game Engine Setup aiba Player Character aira Basic Movement taairai karaaa

Tasks:

- Game Engine Install au Setup karauna
 - New Project Create karauna
 - Project Structure saetaaapa karauna
 - Scripts Folder
 - Scenes Folder
 - Prefabs Folder
 - Art Folder
 - Audio Folder
 - Version Control (Git) Initialize karauna
 - Basic Scene Setup karauna
- Afternoon (2 ghanataaa):
- Player Character Create karauna (Square/Circle)
 - Player Movement Script laikhauna
 - Horizontal Movement (Left/Right)
 - Jump (Spacebar)
 - Basic Collision Detection
 - Camera Follow Script laikhauna
 - Basic Physics Setup karauna

Mornin

Evening (1 ghanataaa):
Player Movement Test karauna
Bug Fix karauna
Controls Adjust karauna
Day 1 Progress Document karauna

****Deliverables:****

- Working Player Character
- Basic Movement (Left, Right, Jump)
- Camera Following Player
- Basic Physics

****Common Mistakes:****

- atairaikata Physics saetaaapa karaaa
- Camera Movement ai samaya baeshai daeauyaaa
- Multiple Movement Styles aikasaaathae baaanaanaao
- Input System ai jatailataaa yaoga karaaa

Day 2: Platforms & Environment

****Goal:**** Platforms, Ground, aiba Basic Environment taairai karaaa

****Tasks:****

Ground Platform Create karauna
Multiple Platforms Create karauna
Static Platforms
Moving Platforms (Optional)
Different Sizes
Platform Collision Setup karauna
Level Layout Design karauna

Afternoon (2 ghanataaa):
Platform Variations Create karauna
Small Platforms
Medium Platforms
Large Platforms
Moving Platforms
Level Boundaries Set karauna
Left Boundary
Right Boundary
Top Boundary
Bottom Boundary (Death Zone)
Camera Bounds Set karauna
Basic Environment Art (Colored Shapes)

Evening (1 ghanataaa):
Platform Layout Test karauna
Jump Height/Distance Test karauna
Level Flow Test karauna
Day 2 Progress Document karauna

****Deliverables:****

- Multiple Platform Types

Morni

- Level Layout
- Camera Bounds
- Death Zone
- Basic Environment

****Common Mistakes:****

- Platforms khaubai kathaina baaa sahaja raaakhaaa
- Camera Bounds saeta naaa karaaa
- Death Zone naaa raaakhaaa
- Level Flow naaa baibaechanaaa karaaa

Day 3: Collectibles & Score System

****Goal:**** Collectible Items, Score System, aiba Basic UI taairai karaaa

****Tasks:****

Collectible Item Create karauna (Star/Coin)
 Collectible Script laikhauna
 Pickup Detection
 Score Increase
 Visual Feedback (Rotation/Animation)
 Sound Effect (Optional)
 Multiple Collectibles Place karauna
 Collectible Placement Test karauna

Afternoon (2 ghanataaa):

Score System Script laikhauna
 Score Variable
 Score Increase Function
 Score Display
 Basic UI Create karauna
 Score Display (Text)
 Health Display (Optional)
 Simple HUD
 UI Script laikhauna
 UI Positioning & Styling

Evening (1 ghanataaa):

Collectible System Test karauna
 Score System Test karauna
 UI Display Test karauna
 Day 3 Progress Document karauna

****Deliverables:****

- Collectible Items
- Score System
- Basic UI/HUD
- Score Display
- Visual & Audio Feedback

****Common Mistakes:****

Mornin

- Collectible Detection samasayaaa
 - Score Update naaa haauyaaa
 - UI Display samasayaaa
 - Collectible Placement khaaaraaapa
-

Day 4: Enemy & Hazard

****Goal:**** Basic Enemy, Hazards, aiba Player Death saisataema taairai karaaa

****Tasks:****

Basic Enemy Create karauna (Square/Circle)
Enemy Movement Script laikhauna
 Patrol Movement (Left-Right)
 Platform Boundary Detection
 Simple AI
Enemy-Player Collision Detection
Enemy Death Animation (Optional)

Afternoon (2 ghanataaa):

Player Health System Script laikhauna
 Health Variable
 Damage Function
 Death Function
 Respawn Function
Hazards Create karauna
 Spikes
 Lava (Death Zone)
 Falling Objects (Optional)
Hazard-Player Collision
Damage/Death Logic

Evening (1 ghanataaa):

Enemy Behavior Test karauna
Health System Test karauna
Death/Respawn Test karauna
Game Over Condition Test karauna
Day 4 Progress Document karauna

****Deliverables:****

- Basic Enemy
- Patrol Behavior
- Health System
- Damage System
- Death/Respawn System
- Hazards

****Common Mistakes:****

- Enemy AI khaubai jataila karaaa
 - Health System samasayaaa
 - Respawn Position bhaula
 - Collision Detection samasayaaa
-

Mornin

Day 5: Win/Lose Conditions & Game Flow

****Goal:**** Win/Lose Conditions, Game Flow, aiba Basic Game State Management taairai karaaa

****Tasks:****

Win Condition Script laikhauna
All Collectibles Pickup (or)
Reach End Point (or)
Defeat Enemy Boss (Optional)
Win Screen Create karauna
"You Win!" Text
Score Display
Restart Button
Win Condition Logic
Win Screen Display

Afternoon (2 ghanataaa):

Lose Condition Script laikhauna
Health = 0
Fall in Death Zone
Time Out (Optional)
Game Over Screen Create karauna
"Game Over" Text
Score Display
Restart Button
Game State Management
Playing State
Paused State
Win State
Game Over State
Pause Functionality (Optional)

Evening (1 ghanataaa):

Win Condition Test karauna
Lose Condition Test karauna
Game Flow Test karauna
Restart Functionality Test karauna
Day 5 Progress Document karauna

****Deliverables:****

- Win Condition
- Lose Condition
- Win Screen
- Game Over Screen
- Restart Functionality
- Game State Management

****Common Mistakes:****

- Win/Lose Conditions sapassata naaa thaaakaaa
- Game Flow samasayaaa
- Restart Button kaaaja naaa karaaa
- Game State Management samasayaaa

Mornin

Day 6: Polish & Sound

****Goal:**** Basic Polish, Sound Effects, aiba Visual Improvements karaaa

****Tasks:****

Sound Effects Add karauna

Jump Sound

Collect Sound

Damage Sound

Death Sound

Win Sound

Enemy Sound

Audio Manager Script laikhauna

Sound Volume Control

Afternoon (2 ghanataaa):

Visual Polish karauna

Player Animation (Idle, Run, Jump)

Enemy Animation (Idle, Patrol)

Collectible Animation (Rotate, Glow)

Particle Effects (Optional)

Screen Flash (Damage/Death)

UI Polish karauna

Better Fonts

Better Colors

Better Layout

Camera Effects (Optional)

Evening (1 ghanataaa):

Sound Test karauna

Visual Test karauna

Overall Feel Test karauna

Bug Fix karauna

Day 6 Progress Document karauna

****Deliverables:****

- Sound Effects
- Basic Animations
- Visual Polish
- UI Improvements
- Audio Manager

****Common Mistakes:****

- Sound Volume asama
- Animation samasayaaa
- Visual Clutter
- Sound Loop samasayaaa

Day 7: Final Testing & Build

****Goal:**** chauudaaanata Testing, Bug Fix, aiba Build taairai karaaa

****Tasks:****

Complete Playtest karauna
 Start to Finish Play
 All Mechanics Test
 Edge Cases Test
 Difficulty Test
Bug List Create karauna
Critical Bug Fix karauna
Balance Adjustments

Afternoon (2 ghanataaa):
 Final Polish karauna
 Missing Features Add
 Bug Fix Continue
 Performance Check
 Final Touch
Build Settings Configure karauna
Build Create karauna
Build Test karauna

Evening (1 ghanataaa):
 Final Playtest karauna
 Screenshot/Video Capture karauna
 Documentation Complete karauna
 Feedback Form taairai karauna
 Day 7 Progress Document karauna

****Deliverables:****

- Final Prototype Build
- Bug Report
- Screenshot/Video
- Documentation
- Feedback Form

****Common Mistakes:****

- shaessa samayae Feature Add karaaa
- Build Settings bhaula
- Final Testing ai samaya kama daeauyaaa
- Documentation naaa karaaa

11.6 7-Day Challenge Summary Table

7-DAY PROTOTYPE CHALLENGE SUMMARY		
Day	Focus Area	Key Deliverables
1	Setup & Player Movement	Player, Movement, Camera
2	Platforms & Environment	Platforms, Level Layout, Bounds
3	Collectibles & Score	Collectibles, Score, UI
4	Enemy & Hazard	Enemy, Health, Damage, Death
5	Win/Lose & Game Flow	Win/Lose Screens, Game States
6	Polish & Sound	Sound, Animation, Visual Polish
7	Final Testing & Build	Final Build, Bug Fix, Documentation
Total Time: 21-35 ghanataaa (3-5 ghanataaa/daina)		
Difficulty: Beginner Friendly		

Engine: Unity / Godot (Free)

11.7 Different Types of Prototypes

TYPES OF PROTOTYPES

1. PROOF OF CONCEPT (PoC)
Goal: paramaana karauna yae kaonao kaichhau samabhava
Time: 1-3 daina
Example: AI Pathfinding Test
Quality: khaubai Basic
 2. PLAYABLE PROTOTYPE
Goal: gaematai majaaara kainaaa taaa paraikassaaa karauna
Time: 1-4 sapataaaha
Example: Full Gameplay Test
Quality: Basic but Playable
 3. VISUAL PROTOTYPE
Goal: gaematai kaemana daekhaaabaee taaa daekhaaana
Time: 2-4 sapataaaha
Example: Art Style Test
Quality: Visual Focus
 4. TECHNICAL PROTOTYPE
Goal: parayaukatai samasayaaa samaaadhaana karauna
Time: 1-4 sapataaaha
Example: Networking Test
Quality: Technical Focus
 5. PAPER PROTOTYPE
Goal: daijaaaana paraikassaaa karauna (kaaagajae)
Time: 1-2 daina
Example: Board Game Version
Quality: Non-digital
-

11.10 Exercise (anaushailana)

Exercise 1: Prototype Planning

aikatai Game Idea baechhae naina aiba aira Prototype Plan taairai karauna:

- kaona Mechanic Test karatae chaaana?
- kata dainae baaanaabaena?
- kii kii thaaakabae Prototype ai?
- kii kii thaaakabae naaa?

Exercise 2: Prototype Creation

7-Day Challenge anasarana karae aikatai Prototype taairai karauna:

- parataidainaera Tasks Complete karauna
- Progress Document karauna
- Final Build taairai karauna

- Feedback naina

Exercise 3: Prototype Evaluation

aapanaaara Prototype Evaluate karauna:

- gaematai kai majaaara?
- kaonao samasayaaa aachhae?
- kii kii paraibaratana karatae habae?
- gaematai aigaiyae yaaaauyaaara yaogaya?

11.11 Common Mistakes Summary (saaadhaaarana bhaula)

1. ****Over-scoping****: Prototype ai atairaikata Feature yaoga karaaa
2. ****Perfect Art****: Placeholder Art aira chaeyae Perfect Art baaanaanao
3. ****No Testing****: Prototype Playtest naaa karaaa
4. ****No Documentation****: Progress naaa laekhaaa
5. ****Feature Creep****: baaarabaaara Feature yaoga karaaa
6. ****No Feedback****: Player Feedback naaa naeayaaa
7. ****Time Overrun****: nairadhaaaraita samayaera baeshai samaya daeayaaa
8. ****No Fun****: gaematai majaaara naaa halaeau aigaiyae yaaaauyaaa
9. ****Skipping Core****: Core Mechanic chhaaadaaa anaya kaichhautae samaya daeayaaa
10. ****No Decision****: Prototype aira para Result saidadhaaanata naaa naeayaaa

11.12 Pro Tips

1. ****Time-box****: nairadaissata samayaera madhayae shaessa karauna
2. ****Simple Art****: sabachaeyae sahaja Art bayabahaara karauna
3. ****Core Focus****: shaudhaumaaatara Core Mechanic ai manaoyaoga daina
4. ****Test Early****: yata taaadaataadai samabhaba Playtest karauna
5. ****Be Honest****: gaematai majaaara naaa halae sabaiikaaara karauna
6. ****Kill Early****: gaematai majaaara naaa halae baaatila karauna
7. ****Document****: saba kaichhau laikhae raaakhauna
8. ****Share****: Prototype Share karauna Feedback paetae
9. ****Iterate****: paraibaratana karae aabaaara paraiikassaaa karauna
10. ****Have Fun****: majaaa karauna, aitai sabachaeyae gaurautabapauurana

11.13 Summary (saaarasakassaepa)

Prototype halao Game Development-aira atayanata gaurautabapauurana aikatai Phase aitai aapanaaakae darauta, saaasharayaii mauulayae aiba kama Risk-ai gaemaera "Fun" paraiikassaaa karatae saaahaaayaya karae

****mauula paaatha:****

- Prototype halao darauta, saaasharayaii mauulayaera paraiikassaaa
- shaudhaumaaatara Core Mechanic phaokaaasa karauna

- Placeholder Art bayabahaaara karauna
- nairadaissata samayaera madhayae shaessa karauna
- Player Feedback naina
- "Fun" naaa halae baaataila karauna
- Documentation raaakhauna

manae raaakhabaena, *****Fail fast, learn fast.***** aikatai bhaaalao Prototype aapanaaakae anaeka samaya aiba aratha baaachaaatae paaarae

parabarataii adhayaaayae aamaraaa Vertical Slice samaparakae baisataaaraita aalaochanaaa karabao

PART 12

adhayaaaya 12: VERTICAL SLICE

adhayaaaya 12: VERTICAL SLICE

shaekhaara udadaeshaya (Learning Goal)

aai adhayaaayae aapanai shaikhabaena Vertical Slice kaii, kaena darakaaara, aiba kaiibhaaabaek aikatai chhaota kainatau samapauurana Playable Section taairai karatae haya aamaraaa Environment, Player, Enemy, Combat, UI, Audio, VFX, Lighting, Save, Win/Lose - sabakaichhau baisataaaraita aalaochanaaa karabao

12.1 Vertical Slice kaii? (What is a Vertical Slice?)

Vertical Slice halao gaemaera aikatai chhaota kainatau samapauurana Playable Section yaaa daekhaaya gaematai chauudaaanata rauupae kaemana habae aitaai Prototype aira parae aiba Full Production aira aagae taairai karaaa haya

****Vertical Slice vs Prototype:****

PROTOTYPE:	VERTICAL SLICE:
Goal: Fun Test	Goal: Complete Section Test
Quality: Basic/Placeholder	Quality: Near-Final
Art: Programmer Art	Art: Final Art Style
Sound: Basic/None	Sound: Complete Sound Design
UI: Minimal	UI: Complete UI
Time: Days to Weeks	Time: Weeks to Months
Scope: Core Mechanic Only	Scope: One Complete Section
Decision: Should we continue?	Decision: Ready for Production?
Risk: Low	Risk: Medium

****Vertical Slice aira udadaeshaya:****

- **Complete Section Test****: gaematai chauudaaanata rauupae kaemana habae taaa daekhaaana
 - **Pipeline Validation****: Art, Audio, Programming Pipeline kaaaja karachhae kainaaa yaaachaaai karauna
 - **Quality Benchmark****: chauudaaanata gaemaera maaana kaemana habae taaa nairadhaaarana karauna
 - **Team Validation****: taima kai Production-aira janaya parasatauta taaa yaaachaaai karauna
 - **Funding/Investment****: Investors/Publishers kae daekhaaanaora janaya
 - **Marketing****: Trailers, Screenshots taairai karaaara janaya
-

12.2 Vertical Slice kaena darakaaara?

WHY VERTICAL SLICE IS NEEDED

1. RISK REDUCTION

Prototype: "Fun" aachhae kainaaa jaaanaaa gaechhae

Vertical Slice: "Quality" arajana samabhaha kainaaa jaaanaaa gaechhae

Full Production: baishabaaasa thaaakabae yae gaematai samabhaha

2. PIPELINE VALIDATION

Art Pipeline: Assets taairai thaekae Engine ai Import

Audio Pipeline: Sound Design thaekae Implementation

Programming Pipeline: Code Structure & Architecture

Level Design Pipeline: Blockout thaekae Final

3. QUALITY BENCHMARK

Visual Quality Standard Set karauna

Audio Quality Standard Set karauna

Gameplay Quality Standard Set karauna

Performance Standard Set karauna

4. TEAM CONFIDENCE

taima baujhabae katataukau kaaaja baaakai

taima baujhabae kaona Tools darakaaara

taima baujhabae kata samaya laaagabae

taima Confidence paaabae

5. FUNDING & INVESTMENT

Publishers kae daekhaanaora janaya

Investors kae daekhaanaora janaya

Kickstarter/Crowdfunding aira janaya

Press/Media Coverage aira janaya

12.3 Vertical Slice ai kaii kaii thaaakabae?

VERTICAL SLICE CHECKLIST

ENVIRONMENT:

One Complete Level/Section

Final Art Style

Proper Lighting

Environmental Details

Props & Decorations

Atmosphere & Mood

Performance Optimization

PLAYER:

Final Character Model

Complete Animations

Idle

Walk

Run

Jump

Attack

Damage

Death

Special Actions

Player Controller

Physics System

Camera System

Player Feedback

ENEMY:

- Final Enemy Model
- Enemy Animations
- Enemy AI
 - Patrol Behavior
 - Chase Behavior
 - Attack Behavior
 - Death Behavior
- Enemy Spawning
- Enemy Variations (if needed)

COMBAT:

- Attack System
- Damage System
- Health System
- Combat Feedback
 - Hit Effects
 - Damage Numbers
 - Screen Shake
 - Sound Effects
- Combat Balance

UI:

- Main Menu
- HUD (Health, Score, etc.)
- Pause Menu
- Settings Menu
- Win/Lose Screens
- UI Animations

AUDIO:

- Sound Effects
 - Player Sounds
 - Enemy Sounds
 - Environment Sounds
 - Combat Sounds
 - UI Sounds
- Background Music
- Ambient Sounds
- Audio Mixing

VFX:

- Particle Effects
 - Hit Effects
 - Death Effects
 - Environment Effects
 - UI Effects
- Screen Effects
 - Screen Shake
 - Screen Flash
 - Post Processing
- Lighting Effects

LIGHTING:

- Main Lighting Setup
- Ambient Lighting
- Dynamic Lighting (if needed)
- Light Mapping (if needed)
- Performance Optimization

SAVE SYSTEM:

- Basic Save/Load
- Auto Save (if needed)

Save Data Management

WIN/LOSE:

Win Condition

Lose Condition

Win Screen

Lose Screen

Restart Functionality

12.4 Step-by-Step Guide with Timeline

VERTICAL SLICE CREATION TIMELINE (8 WEEKS)

WEEK 1: PLANNING & PREPARATION

Day 1-2: Scope Definition

kaona Level/Section baaanaabaena saidadhaanata karauna

kaii kaii Features thaaakabae taaa laikhauna

Timeline nairadhaaarana karauna

Resources nairadhaaarana karauna

Day 3-4: Blockout

Level Layout Blockout karauna

Basic Shapes bayabahaaara karauna

Player Path nairadhaaarana karauna

Key Areas Identify karauna

Day 5-7: Greybox

Detailed Greybox taairai karauna

Collision Setup karauna

Basic Lighting Set karauna

Playtest karauna

WEEK 2: PLAYER & CORE MECHANICS

Day 1-3: Player Character

Final Character Model Import

Animation Setup

Player Controller Update

Physics Fine-tuning

Day 4-5: Core Mechanics

Combat System Finalize

Interaction System

Ability System (if needed)

Input System Polish

Day 6-7: Testing

Player Controls Test

Mechanics Test

Bug Fix

Balance Adjustments

WEEK 3: ENEMY & AI

Day 1-3: Enemy Character

Final Enemy Model Import

Enemy Animation Setup

Enemy AI Implementation

Enemy Behavior Polish

Day 4-5: Combat Integration

Enemy-Player Combat

Damage System Integration

Health System Integration

Combat Feedback

Day 6-7: Testing

- Enemy Behavior Test
- Combat Balance Test
- Bug Fix
- Difficulty Adjustment

WEEK 4: ENVIRONMENT & ART

- Day 1-3: Environment Art
 - Final Art Assets Import
 - Texture Application
 - Material Setup
 - Prop Placement
- Day 4-5: Lighting
 - Main Lighting Setup
 - Ambient Lighting
 - Dynamic Lighting
 - Light Baking (if needed)
- Day 6-7: Visual Polish
 - Visual Effects
 - Post Processing
 - Atmosphere Setup
 - Performance Check

WEEK 5: UI & AUDIO

- Day 1-3: UI Implementation
 - Main Menu
 - HUD
 - Pause Menu
 - Win/Lose Screens
- Day 4-5: Audio Implementation
 - Sound Effects Integration
 - Music Integration
 - Ambient Sounds
 - Audio Mixing
- Day 6-7: Testing
 - UI Navigation Test
 - Audio Test
 - Bug Fix
 - Polish

WEEK 6: VFX & POLISH

- Day 1-3: VFX Implementation
 - Particle Effects
 - Screen Effects
 - Lighting Effects
 - Combat VFX
- Day 4-5: Visual Polish
 - Animation Polish
 - UI Polish
 - Environment Polish
 - Overall Polish
- Day 6-7: Testing
 - VFX Test
 - Visual Test
 - Bug Fix
 - Performance Check

WEEK 7: SAVE SYSTEM & FLOW

- Day 1-3: Save System
 - Save/Load Implementation
 - Auto Save Setup
 - Save Data Management
 - Save System Test

Day 4-5: Game Flow
Game State Management
Level Transitions
Win/Lose Flow
Restart Functionality
Day 6-7: Testing
Save/Load Test
Game Flow Test
Bug Fix
Polish

WEEK 8: FINAL TESTING & BUILD

Day 1-3: Final Testing
Complete Playtest
All Mechanics Test
Performance Profiling
Bug Fix
Day 4-5: Final Polish
Missing Features
Visual Polish
Audio Polish
UI Polish
Day 6-7: Build & Documentation
Final Build Creation
Build Testing
Documentation
Presentation Materials

Total Time: 8 sapataaaha (Full-time) / 16 sapataaaha (Part-time)

Team Size: 3-8 jana (Small Team)

12.5 Detailed Section Descriptions

12.5.1 Environment (paraibaesha)

****kaii karatae habae:****

- aikatai Complete Level/Section taairai karauna
- Final Art Style bayabahaaara karauna
- Proper Lighting saetaaapa karauna
- Environmental Details yaoga karauna
- Props & Decorations Place karauna
- Atmosphere & Mood taairai karauna

****Best Practices:****

- Blockout parathamae karauna
- Art Assets parae Import karauna
- Performance baibaechanaaaa karauna
- Level Flow paraikassaaa karauna
- Player Guidance daina (Lighting, Color)

****Common Mistakes:****

- atairaikata Detail yaoga karaaaa

- Performance upaekassaaa karaaa
- Player Path asapassata raaakhaaa
- Art Style Inconsistency

12.5.2 Player (khaelaoyaaada)

****kaii karatae habae:****

- Final Character Model bayabahaaara karauna
- Complete Animations taairai karauna
- Player Controller Update karauna
- Physics System Fine-tune karauna
- Camera System Polish karauna
- Player Feedback yaoga karauna

****Required Animations:****

Walk	haaataaa
Run	daaudaaanao
Jump	laaaphaaanao
Fall	padaaa
Attack	aakaramana
Damage	kassataigarasata haauyaaa
Death	maritayau
Crouch	haaatau gaedae thaaakaaa
Interact	maithasakaraiyaaa

Idle

****Common Mistakes:****

- Animation asamapauurana raaakhaaa
- Input Lag thaaakaaa
- Camera Issues
- Physics Problems

12.5.3 Enemy (shatarau)

****kaii karatae habae:****

- Final Enemy Model Import karauna
- Enemy Animation Setup karauna
- Enemy AI Implement karauna
- Enemy Behavior Polish karauna
- Enemy Variations taairai karauna
- Enemy Spawning System taairai karauna

****Enemy AI States:****

Patrol	nairadaissata pathae chalaaa
Chase	Player kae taaadaaa karaaa
Attack	aakaramana karaaa

Idle

Flee	paaalaaanao
Death	maritayau

****Common Mistakes:****

- AI khaubai jataila baaa sahaja raaakhaaa
- Pathfinding samasayaaa
- Animation samasayaaa
- Balance samasayaaa

12.5.4 Combat (yaudadha)

****kaii karatae habae:****

- Attack System Implement karauna
- Damage System Implement karauna
- Health System Implement karauna
- Combat Feedback yaoga karauna
- Combat Balance karauna
- Combat VFX yaoga karauna

****Combat Feedback Types:****

Audio	Hit Sounds, Damage Sounds, Death Sounds
Haptic	Controller Vibration (Console)
Physical	Screen Shake, Knockback
UI	Health Bar Change, Damage Indicators

****Common Mistakes:****

- Combat anaubhaba karaaa yaaaya naaa
- Damage asama
- Health System samasayaaa
- Feedback aparayaaapata

12.5.5 UI (iujaara inataaaraphaesa)

****kaii karatae habae:****

- Main Menu taairai karauna
- HUD (Heads-Up Display) taairai karauna
- Pause Menu taairai karauna
- Settings Menu taairai karauna
- Win/Lose Screens taairai karauna
- UI Animations yaoga karauna

****UI Elements:****

HUD	Health, Score, Ammo, Minimap
Pause Menu	Resume, Settings, Quit

Settings	Audio, Graphics, Controls
Win Screen	Score, Stars, Restart
Lose Screen	Score, Retry, Quit

****Common Mistakes:****

- UI Navigation kathaina
- UI Design apaurana
- UI Animations naei
- Responsive Design naei

12.5.6 Audio (adaiau)

****kaii karatae habae:****

- Sound Effects Implement karauna
- Background Music Implement karauna
- Ambient Sounds Implement karauna
- Audio Mixing karauna
- Audio Mastering karauna
- Audio Manager taairai karauna

****Required Sound Effects:****

Enemy	Patrol, Attack, Damage, Death
Combat	Hit, Miss, Critical
UI	Click, Hover, Open, Close
Environment	Wind, Water, Birds, Creaks

****Common Mistakes:****

- Sound Volume asama
- Sound Loop samasayaaa
- Missing Sound Effects
- Audio Performance Issues

12.5.7 VFX (bhaijayauyaaala iphaekatasa)

****kaii karatae habae:****

- Particle Effects taairai karauna
- Screen Effects taairai karauna
- Lighting Effects taairai karauna
- Combat VFX taairai karauna
- Environment VFX taairai karauna
- UI Effects taairai karauna

****Common VFX Types:****

Screen	Screen Shake, Screen Flash, Damage Vignette
--------	---

Lighting	Dynamic Lights, Light Flares, Shadows
Environment	Dust, Fog, Rain, Snow
UI	Button Press, Score Pop, Transition

****Common Mistakes:****

- VFX atairaikata
- Performance Issues
- VFX adarishaya
- VFX Timing samasayaaa

12.5.8 Lighting (aalao)

****kaii karatae habae:****

- Main Lighting Setup karauna
- Ambient Lighting Set karauna
- Dynamic Lighting Implement karauna
- Light Baking karauna (yadai parayaojana)
- Shadow Setup karauna
- Performance Optimization karauna

****Lighting Types:****

Point	baaatai, aagauna (Local Lights)
Spot	taracha, phaokaaasada laaaaita
Area	Window Light, Soft Shadows
Ambient	saaadhaaarana aalao (Global Illumination)

****Common Mistakes:****

- Lighting atairaikata ujajabala baaa anadhakaaara
- Performance Issues (Too Many Lights)
- Shadow Artifacts
- Inconsistent Lighting

12.5.9 Save System (saebha saisataema)

****kaii karatae habae:****

- Basic Save/Load System taairai karauna
- Auto Save Setup karauna (yadai parayaojana)
- Save Data Management karauna
- Save System Test karauna
- Save File Management karauna

****Save Data Types:****

Level Data	Completed Levels, Stars, Unlocks
Settings Data	Audio, Graphics, Controls
Progress Data	Quest Progress, Achievement Progress

****Common Mistakes:****

- Save Data Corruption
- Save File Management Issues
- Auto Save Timing Issues
- Load Performance Issues

12.5.10 Win/Lose (jaya/paraaajaya)****kaii karatae habae:****

- Win Condition Implement karauna
- Lose Condition Implement karauna
- Win Screen taairai karauna
- Lose Screen taairai karauna
- Restart Functionality taairai karauna
- Game Flow paraiikassaaa karauna

****Win Conditions:****

All Collected saba Collectible sagaraha
 Boss Defeated Boss kae paraaajaita karaaa
 Time Survived nairadaissata samaya parayanata baechaе thaaakaaa

Level

****Lose Conditions:****

Fall Off Map thaekae padae yaaaauyaaa
 Time Out samaya shaessa haauyaaa
 Captured shatarau dabaaaraaa dharaaa padaaa

Health

****Common Mistakes:****

- Win/Lose Conditions asapassata
- Restart Functionality samasayaaa
- Game Flow samasayaaa
- Score Calculation samasayaaa

12.9 Exercise (anaushailana)**Exercise 1: Vertical Slice Planning**

aapanaaara parakalapaera janaya Vertical Slice Plan taairai karauna:

- kaona Level/Section baaanaabaena?
- kaii kaii Features thaaakabae?
- Timeline kaemana habae?
- Resources kaii kaii darakaaara?

Exercise 2: Vertical Slice Creation

aikatai Vertical Slice taairai karauna:

- Step-by-Step Guide anausarana karauna
- Checklist Follow karauna
- Timeline maenae chalauna
- Final Build taairai karauna

Exercise 3: Vertical Slice Evaluation

aapanaaara Vertical Slice Evaluate karauna:

- Quality Standard maenaechhae?
- Performance bhaaalao?
- Player Feedback kaemana?
- Production-aira janaya parasatauta?

12.10 Common Mistakes Summary (saaadhaaarana bhaula)

1. **Scope Too Big**: Vertical Slice khaubai bada karae phaelaaa
2. **No Blockout**: Greyboxing chhaadaaai saraasarai Art shaurau karaaa
3. **Placeholder Art**: Final Art-aira chaeyae Placeholder Art bayabahaaara karaaa
4. **Missing Systems**: kaonao System apaurana raaakhaaa
5. **No Polish**: Visual/Audio Polish naaa karaaa
6. **Performance Issues**: Performance Optimization naaa karaaa
7. **No Testing**: parayaaapata Testing naaa karaaa
8. **Timeline Overrun**: nairadhaaaraita samayaera baeshai samaya daeautyaaa
9. **No Documentation**: Progress naaa laekhaaa
10. **No Decision**: Vertical Slice-aira para saidadhaaanata naaa naeautyaaa

12.11 Pro Tips

1. **Start Small**: chhaota Section daiyae shaurau karauna
2. **Greybox First**: Blockout/Greybox parathamae karauna
3. **Final Art**: Final Art Style bayabahaaara karauna
4. **Complete Systems**: saba System Complete karauna
5. **Polish Everything**: sabakaichhau Polish karauna
6. **Test Thoroughly**: bhaalaobhaaaba Test karauna
7. **Document**: saba kaichhau laikhae raaakhauna
8. **Get Feedback**: Player/Team Feedback naina
9. **Be Decisive**: Vertical Slice-aira para saidadhaaanata naina
10. **Have Confidence**: aapanaaara gaemae baishabaaasa raaakhauna

12.12 Summary (saaarasakassaepa)

Vertical Slice halao Game Development-aira aikatai gaurautabapaurana Phase yaaa daekhaaaya gaematai

chauudaaanata rauupae kaemana habae aitari Prototype aira parae aiba Full Production aira aagae taairai karaaa haya

****mauula paaatha:****

- Vertical Slice halao aikatai chhaota kainatau samapauurana Playable Section
- Final Art, Audio, UI, VFX, Lighting sabakaichhau thaaakatae habae
- Save System aiba Win/Lose Conditions thaaakatae habae
- Performance Optimization karatae habae
- Timeline maenae chalauna
- Thoroughly Test karauna
- Documentation raaakhauna

manae raaakhabaena, *****A vertical slice is a promise to your players and your team.***** aitari daekhaaaya gaematai kaemana habae aiba taima kai karatae paaarabae

parabarataii adhayaaayae aamaraaa Game Design Mechanics samaparakae baisataaaraita aalaochanaaaa karabao

PART 13

adhayaaaya 13: GAME DESIGN MECHANICS

adhayaaaya 13: GAME DESIGN MECHANICS

shaekhaara udadaeshaya (Learning Goal)

aii adhayaaayae aapanai shaikhabaena parataitai Game Mechanics baisataaaraitabhaaaba Movement, Jump, Dash, Sprint, Crouch, Interact, Combat, Stealth, Puzzle, Crafting, Inventory, Dialogue, Quest, Skill Tree, Character Progression - parataitai Mechanics-aira Purpose, Input, Rules, Feedback, Reward, aiba Failure State aalaochanaaa karabao

13.1 Game Mechanics kaii?

Game Mechanics halao gaemaera naiyama aiba karaiyaaakalaaapa yaaa Player kae gaemae asha naitae saaahaaayaya karae aigaulao gaemaera mauula bhaitatai

GAME MECHANICS FRAMEWORK

MECHANICS		
CORE	COMBAT	PROGRESSION
* Movement	* Attack	* Skills
* Jump	* Defense	* Level
* Dash	* Stealth	* Items
* Sprint	* Puzzle	* Crafting
* Crouch	* Crafting	* Quest
* Interact		

13.2 Movement (chalaachala)

Purpose: Player Character kae gaema World-ai aika jaaayagaaa thaekae anaya jaaayagaaaya saraiyae naeayuaa

Input:

Controller	Left Analog Stick
Mobile	Virtual Joystick / Touch

Rules:

Direction	4 daika (Left, Right, Up, Down) athabaaa 8 daika
-----------	--

Keybo

Speed

Acceleration	gatai baridadhai paetae samaya
Deceleration	gatai kamatae samaya
Friction	Surface-ai gharassana
Slope	dhaaalaе chalaara naiyama
Water	paaanaitae chalaara naiyama

****Feedback:****

Audio	Footstep Sounds (Different Surfaces)
Physical	Camera Bob, Controller Vibration
UI	Speed Indicator (Optional)

****Reward:****

- natauna ailaaakaaa anausanadhaaana
- shatarau thaekae paaalaaanao
- Item sagaraha
- Level Complete

****Failure State:****

- gatai dhaira haauyaaa
- patha haaaraanao
- shataraura haaatae dharaaa padaaa
- samaya shaessa haauyaaa

****Examples:****

Dark Souls	Weight-based Movement
Celeste	Precise Platforming Movement
DOOM	Fast-paced Movement

13.3 Jump (laaapha)

****Purpose:**** Player Character kae uparae uthatae baaa baaadhaaa ataikarama karatae saaahaaayaya karaaa

****Input:****

Controller	A Button / X Button (Xbox/PS)
Mobile	Tap Jump Button

****Rules:****

Jump Distance	kata dauurataba ataikarama karabae
Air Control	baaataaasae daika paraibaratana karaaa yaaabae kainaaa
Coyote Time	palayaaatapharama thaekae padae gaelaeau kaichhaukassana Jump karaaa yaaaba
Jump Buffer	Jump Button aagae thaekae chaaapalaeau kaaaja karabae
Double Jump	baaataaasae aabaaara Jump karaaa yaaabae (yadai thaaakae)
Wall Jump	daeyaaalae laaaphaiyae Jump karaaa (yadai thaaakae)

****Feedback:****

Audio	Jump Sound, Land Sound
Physical	Camera Shake, Controller Vibration
UI	Jump Indicator (Optional)

****Reward:****

- uchacha sathaaanae paauchhaaanao
- baaadhaaa ataikarama
- shatarau aidaiyae yaaaauyaaa
- Collectible sagaraha

****Failure State:****

- Jump karatae naaa paaaraaa
- khaaaraaapa Timing
- padae yaaaauyaaa
- aabaaara chaessataaa karatae haauyaaa

****Examples:****

Celeste Ultra-precise Jump
 Hollow Knight Wall Jump, Double Jump
 Breath of Wild Climbing + Jump

Mario

13.4 Dash (darauta saraaanao)

****Purpose:**** Player Character kae darauta aikatai nairadaissata daikae saraiyae naeauyaaa

****Input:****

Controller B Button / Circle Button
 Mobile Swipe / Dash Button

Keybo

****Rules:****

Direction kaona daikae dayaaasha karabae
 Cooldown kata samaya para aabaaara dayaaasha karaaa yaaabae
 Invincibility dayaaasha samaya ajaeya kainaaa
 Air Dash baaataaasae dayaaasha karaaa yaaabae kainaaa
 Chain Dash aikaaadhaika dayaaasha aikasaaathae karaaa yaaabae kainaaa

Distar

****Feedback:****

Audio Dash Sound, Wind Sound
 Physical Screen Shake, Controller Vibration
 UI Cooldown Indicator

Visua

****Reward:****

- darauta paaalaaanao
- shataraura aakaramana aidaiyae yaaaauyaaa
- darauta aakaramana
- Gap Jump

****Failure State:****

- Cooldown-ai aatakae thaaakaaa
- khaaaraaapa Direction
- Wall-ai dhaaakakaaa

- padae yaaaauyaaa

****Examples:****

Celeste	Dash with Stamina
DOOM	Equipment Dash
Dead Cells	Roll/Dash

13.5 Sprint (daaudaaanao)

****Purpose:**** Player Character kae Walk-aira chaeyae darauta chalatae saaahaaayaya karaaa

****Input:****

Controller	Hold Left Analog Stick
Mobile	Hold Sprint Button

****Rules:****

Stamina Cost	kata Stamina kharacha habae
Stamina Regen	Stamina kata darauta phairae aasabae
Noise Level	Sprint karalae shatarau shaunabae kainaaa

****Feedback:****

Audio	Heavy Footsteps, Breathing Sound
Physical	Controller Vibration
UI	Stamina Bar

****Reward:****

- darauta ganatabayae paauchhaaanao
- shatarau thaekae paaalaaanao
- समयamatao kaaaja shaessa karaaa

****Failure State:****

- Stamina shaessa haauyaaa
- dhaiira hayae yaaaauyaaa
- shatarau daekhae phaelaaa
- Stealth bhaengae yaaaauyaaa

****Examples:****

Minecraft	Sprint Toggle
Fortnite	Sprint with Stamina
Assassin's Creed	Sprint for Parkour

13.6 Crouch (haatau gaedae thaaakaaa)

****Purpose:**** Player Character kae kama jaaayagaaaya dhaokaaara anaumatai daeauyaaa aiba Stealth-ai saaahaaayaya karaaa

****Input:****

Controller	Right Analog Stick Press
Mobile	Crouch Button

Keybo

****Rules:****

Speed	Crouch karalae gatai kamabae
Sound	Crouch karalae kama shabada habae
Detection	shataraura darissatai thaekae laukaaanao yaaabae
Peek	Wall-aira paechhana thaekae daekhaaa yaaabae

Heigh

****Feedback:****

Audio	Quiet Footsteps
Physical	None
UI	Crouch Indicator

Visua

****Reward:****

- chhaota jaaayagaaaya dhaokaaa
- Stealth karaaa
- shatarau aidaiyae yaaaauyaaa
- Cover haisaebae bayabahaaara

****Failure State:****

- gatai dhaira haauyaaa
- Attack Range kama haauyaaa
- shatarau dharae phaelaaa
- khaaaraaapa Position

****Examples:****

CS:GO	Crouch for Accuracy
Sly Cooper	Crouch + Sneak
Last of Us	Crouch + Cover

Metal

13.7 Interact (maithasakaraiyaaa)

****Purpose:**** Player Character kae gaema World-aira baibhainana Object-aira saaathae maithasakaraiyaaa karatae saaahaaayaya karaaa

****Input:****

Controller	X Button / Square Button
Mobile	Interact Button / Tap

Keybo

****Rules:****

Prompt	Interact karaaara samaya UI Prompt daekhaaaba
Animation	Interact karaaara samaya Animation chalaba
Time	Interact karatae kata samaya laaagabae
Result	Interact karalae kaii habae

Range

****Feedback:****

Audio	Interact Sound
Physical	None
UI	Interact Prompt, Progress Bar

****Reward:****

- Item sagaraha
- Door khaolaaa
- NPC-aira saaathae kathaaa balaaa
- Switch activate
- Information paaaauyaaa

****Failure State:****

- Range-aira baaairae
- Object Locked
- Interact samaya shaessa hayanai
- Wrong Object

****Examples:****

Zelda	A to Interact
Resident Evil	Examine + Interact
Portal	Portal Gun Interact

13.8 Combat (yaudadha)

****Purpose:**** Player Character kae shataraura saaathae yaudadha karatae saaahaaayaya karaaa

****Input:****

Controller	Right Trigger (Attack), Left Trigger (Block)
Mobile	Attack/Block Buttons

****Rules:****

Health	Player/Enemy-aira kata Health aachhae
Range	aakaramanaera paraisaiimaaa
Speed	aakaramanaera gatai
Combo	aikaaadhaika aakaramana aikasaaathae
Block/Dodge	aakaramana aidaaanao/paratairaodha
Critical	baishaessa aakaramana
Element	baishaessa Element-aira aakaramana

****Feedback:****

Audio	Attack Sound, Hit Sound, Block Sound
Physical	Screen Shake, Controller Vibration
UI	Health Bar, Damage Indicators

****Reward:****

- shatarau paraaajaita
- Item paaaauyaaa

- Experience paaaauyaaa
- Level Up
- Boss Defeat

****Failure State:****

- Player Death
- Health Low
- Weapon Broken
- Out of Ammo

****Examples:****

Devil May Cry	Combo-based Combat
Zelda	Sword + Shield Combat
DOOM	Fast-paced FPS Combat

13.9 Stealth (gaopanaiiyataaa)

****Purpose:**** Player Character kae shataraura darissatai thaekae laukaiyae thaaakatae saaahaaayaya karaaa

****Input:****

Controller	Crouch Button, Cover Button
Mobile	Stealth Button

****Rules:****

Noise Level	kata shabada karachhae
Detection	shatarau kata darauta khaujae paaabae
Cover System	laukaaanaora jaaayagaaa
Distraction	shataraukae baibharaaanata karaaa
Kill	gaopanaiiyabhaaabaes shataraukae hatayaaa

****Feedback:****

Audio	Quiet Footsteps, Ambient Sound
Physical	None
UI	Detection Indicator, Stealth Meter

****Reward:****

- shatarau aidaiyae yaaaauyaaa
- gaopanaiiya Kill
- Bonus Points
- Better Rating

****Failure State:****

- shatarau daekhae phaelaaa
- dharaaa padaaa
- Combat shaurau haauyaaa
- Mission Fail

****Examples:****

Hitman	Social Stealth
Assassin's Creed	Stealth Kill
Last of Us	Stealth + Scavenge

Metal

13.10 Puzzle (dhaaadhaaa)

****Purpose:**** Player Character kae samasayaaa samaaadhaaanae saaahaaayaya karaaa

****Input:****

Controller	Button Press, Analog Stick
Mobile	Touch, Drag

Keybo

****Rules:****

Sequence	kaona karamae karatae habae
Pattern	kaona Pattern khaujae baera karatae habae
Clue	kaothaaaya Clue aachhae
Time Limit	samaya aachhae kainaaa
Difficulty	katataaa kathaina

Logic

****Feedback:****

Audio	Correct/Wrong Sound
Physical	None
UI	Puzzle UI, Hint System

Visua

****Reward:****

- Door/Path Unlocked
- Item paaaauyaaa
- Secret Area
- Boss Weakness

****Failure State:****

- Wrong Answer
- Time Out
- Need Hint
- Restart Puzzle

****Examples:****

Portal	Physics-based Puzzles
Talos Principle	Logic Puzzles
Resident Evil	Inventory Puzzles

Zelda

13.11 Crafting (taairai karaaa)

****Purpose:**** Player Character kae natauna Item taairai karatae saaahaaayaya karaaa

****Input:****

Controller	Menu Button, Craft Button
Mobile	Craft Menu

****Rules:****

Recipe	kaona Recipe bayabahaaara karatae habae
Station	kaona Station-ai taaairai karatae habae
Time	taaairai karatae kata samaya laaagabae
Quality	taaairai karaaa Item-aira maaana
Skill	Crafting Skill-aira upara nairabhara karae

****Feedback:****

Audio	Crafting Sound
Physical	None
UI	Crafting Menu, Recipe List

****Reward:****

- natauna Weapon/Armor
- Better Stats
- Rare Items
- Crafting Skill Up

****Failure State:****

- Material naei
- Wrong Recipe
- Station naei
- Crafting Failed

****Examples:****

Skyrim	Recipe-based Crafting
Monster Hunter	Material-based Crafting
Terraria	Station-based Crafting

13.12 Inventory (inabhaenatarai)

****Purpose:**** Player Character kae Item sagaraha aiba bayabasathaaapanaaa karatae saaahaaayaya karaaa

****Input:****

Controller	Menu Button / Start Button
Mobile	Inventory Button

****Rules:****

Slots	kata dharanaera Item raaakhatae paaarabae
Weight	kata aujana bahana karatae paaarabae
Organization	Item kaiibhaaaba Sort karabae
Use	Item kaiibhaaaba bayabahaaara karabae
Drop	Item phaelae daeayaaa
Trade	Item baikarai/ exchange karaaa

****Feedback:****

Audio	Open/Close Sound, Equip Sound
Physical	None
UI	Inventory Grid, Item Details

****Reward:****

- Better Equipment
- More Options
- Resource Management
- Collection Complete

****Failure State:****

- Inventory Full
- Wrong Item
- Lost Item
- Cannot Carry

****Examples:****

Diablo	Bag-based Inventory
Minecraft	Simple Grid Inventory
Skyrim	Weight-based Inventory

13.11 Dialogue (salaaapa)

****Purpose:**** Player Character kae NPC-daera saaathae kathaaa balatae saaahaaayaya karaaa

****Input:****

Controller	X Button, D-Pad (Choices)
Mobile	Tap, Choice Buttons

****Rules:****

Branching	Choice-aira upara nairabhara karae Story paraibaratana habae
Tone	Dialogue-aira Tone kaemana
Information	kaii Information daeauyaaa hachachhae
Relationship	Choice-aira upara NPC-aira saaathae samaparaka paraibaratana habae

****Feedback:****

Audio	Voice Over (Optional), Text Sound
Physical	None
UI	Dialogue UI, Choice Buttons

****Reward:****

- Story Progress
- Information
- Items/Gold
- Relationship Up
- Quest Unlock

****Failure State:****

- Wrong Choice
- Relationship Down
- Quest Fail
- Information Lost

****Examples:****

Undertale	Moral Choices
Detroit	Branching Narrative
Witcher	Consequence-based Choices

13.12 Quest (abhaiyaaana)

****Purpose:**** Player Character kae Goal aiba Objective paradaaana karaaa

****Input:****

Controller	Menu Button, Quest Log
Mobile	Quest Button

****Rules:****

Steps	kaona karamae karatae habae
Reward	kaii paaabae
Time Limit	samaya aachhae kainaaa
Difficulty	katataaa kathaina
Tracking	Quest Track karaaa yaaabae

****Feedback:****

Audio	Quest Sound, Update Sound
Physical	None
UI	Quest UI, Map Markers

****Reward:****

- Experience
- Gold/Items
- Story Progress
- Unlock New Areas
- Relationship Changes

****Failure State:****

- Quest Fail
- Time Out
- Wrong Choice
- Objective Lost

****Examples:****

Witcher	Side Quest Design
Hollow Knight	Quest Board

13.13 Skill Tree (dakassataaa gaaachha)

Purpose: Player Character kae natauna Ability aiba Skill paetae saaahaaayaya karaaa

Input:

Controller	Menu Button, Skill Points
Mobile	Skill Menu

Rules:

Points	kata Skill Point darakaaara
Prerequisites	aagae kaona Skill shaikhatae habae
Reset	Skill Reset karaaa yaaabae kainaaa
Variety	kata dharanaera Skill aachhae

Feedback:

Audio	Skill Unlock Sound
Physical	None
UI	Skill Tree, Skill Details

Reward:

- New Abilities
- Better Stats
- Different Playstyles
- Character Growth

Failure State:

- Wrong Skill
- Not Enough Points
- Locked Skills
- No Progression

Examples:

Diablo	Class Skill Trees
Borderlands	Skill Trees + Combinations
Hollow Knight	Charm System

13.14 Character Progression (charaitara agaragatai)

Purpose: Player Character kae shakataishaaalaii karatae saaahaaayaya karaaa

Input:

Controller	Menu Button, Level Up
Mobile	Level Up Button

****Rules:****

Level	kata Level aachhae
Stats	kaona Stats baaadabae
Caps	Max Level kata
Scaling	parataitai Level-ai kata baaadabae

XP

****Feedback:****

Audio	Level Up Sound
Physical	None
UI	Level Up Screen, Stats Screen

Visua

****Reward:****

- Better Stats
- New Skills
- New Areas
- New Equipment
- Story Progress

****Failure State:****

- No Progress
- Slow Leveling
- Wrong Build
- Stuck at Level

****Examples:****

Dark Souls	Soul-based Progression
Monster Hunter	Rank-based Progression
Celeste	No Level, Skill-based

Final

13.15 Mechanics Summary Table

MECHANICS SUMMARY TABLE

Mechanic	Purpose	Key Element
Movement	Navigation	Speed, Control
Jump	Vertical Movement	Height, Timing
Dash	Quick Evasion	Direction, Cooldown
Sprint	Fast Travel	Stamina, Noise
Crouch	Stealth/Cover	Height, Detection
Interact	World Interaction	Range, Prompt
Combat	Enemy Defeat	Damage, Health
Stealth	Avoid Detection	Visibility, Noise
Puzzle	Problem Solving	Logic, Clues
Crafting	Item Creation	Materials, Recipe
Inventory	Item Management	Capacity, Slots
Dialogue	NPC Communication	Choices, Branching
Quest	Goal Setting	Objective, Reward
Skill Tree	Ability Growth	Branches, Points
Progression	Character Growth	XP, Level

13.16 Exercise (anaushailana)

Exercise 1: Mechanics Design

aikatai natauna Game Mechanics Design karauna:

- Purpose kii habae?
- Input kaemana habae?
- Rules kii kii thaaakabae?
- Feedback kaemana habae?
- Reward/Failure State kii habae?

Exercise 2: Mechanics Analysis

aikatai Published Game-aira Mechanics Analyze karauna:

- kaona Mechanics aachhae?
- kaiibhaaaba kaaaja karae?
- Player Feedback kaemana?
- kii bhaaalao/khaaaraaapa?

Exercise 3: Mechanics Combination

aikaaadhaika Mechanics aikasaaathae Combine karauna:

- Movement + Combat
- Stealth + Puzzle
- Crafting + Inventory
- Dialogue + Quest

13.17 Common Mistakes Summary (saaadhaaarana bhaula)

1. ****Too Many Mechanics****: atairaikata Mechanics yaoga karaaa
2. ****No Synergy****: Mechanics aikasaaathae kaaaja naaa karaaa
3. ****Poor Feedback****: Player kae sathaika Feedback naaa daeayuaa
4. ****No Depth****: Mechanics gabhaiirataaa naei
5. ****No Variety****: aikai Mechanics baaarabaaara bayabahaaara
6. ****Complex Controls****: kanataraola khaubai jataila
7. ****No Learning Curve****: dhairae dhairae shaekhaanao naei
8. ****Unbalanced****: Mechanics asama
9. ****No Purpose****: Mechanics aira kaonao udadaeshaya naei
10. ****No Fun****: Mechanics majaaara naaa

13.18 Pro Tips

1. ****Core First****: parathamae Core Mechanics baaanaana
2. ****Synergy****: Mechanics aikasaaathae bhaaalao kaaaja karauka

3. **Feedback**: sabasamaya Player Feedback daina
 4. **Depth**: Mechanics ai gabhaiirataaa daina
 5. **Variety**: baibhainana Mechanics bayabahaaara karauna
 6. **Balance**: Mechanics samaaana raaakhauna
 7. **Test**: parataitai Mechanics Test karauna
 8. **Iterate**: paraibaratana karae aabaaara paraikassaaa karauna
 9. **Player First**: Player Experience sarabaochacha
 10. **Have Fun**: majaaa karauna
-

13.19 Summary (saaarasakassaepa)

Game Mechanics halao gaemaera mauula bhaitatai parataitai Mechanics-aira aikatai nairadaissata Purpose, Input, Rules, Feedback, Reward, aiba Failure State aachhae

mauula paaatha:

- Mechanics gaulao shaikhauna aiba baujhauna
- parataitai Mechanics-aira Purpose baujhauna
- Player Feedback daina
- Mechanics gaulao aikasaaathae Synergy taairai karauna
- Balance raaakhauna
- Test karauna
- Iterate karauna

manae raaakhabaena, **"Mechanics are the language of games."** bhaaalao Mechanics aikatai gaemakae chamakaaara karae taolae

parabarataii adhayaaayae aamaraaa Horror Game Design samaparakae baisataaaraita aalaochanaaaa karabao

PART 14

adhayaaaya 14: HORROR GAME DESIGN

adhayaaaya 14: HORROR GAME DESIGN

shaekhaara udadaeshaya (Learning Goal)

aii adhayaaayae aapanai shaikhabaena kaiibhaaabaek aikatai Horror Game Design karatae haya Fear, Tension, Suspense, Uncertainty, Limited Information, Sound, Lighting, Enemy AI, Chase, Safe Zone, Puzzle, Environmental Storytelling, Jump Scare, Psychological Horror - sabakaichhau baisataaaraita aalaochanaaa karabao aamaraaa Horror Gameplay Loop Diagram, Fear Curve Diagram, aiba Real Horror Games-aira Examples daekhabao

14.1 Horror Game kaii?

Horror Game halao aimana aikatai Game yaaa Player-kae bhaya, utataejanaaaa, aiba aatangakaera anaubhauutai daeya aitai Player-kae asabasatai, bhaya, aiba maaanasaiika chaaapaera madhayae raaakhae

HORROR GAME SUB-GENRES

SURVIVAL HORROR

- Limited Resources (Ammo, Health)
- Inventory Management
- Puzzle-solving under Pressure
- Enemy Encounters
- Examples: Resident Evil, Silent Hill, Alone in the Dark

PSYCHOLOGICAL HORROR

- Mental Fear
- Unreliable Narrator
- Ambiguity & Uncertainty
- Mind-bending Story
- Examples: Silent Hill 2, Layers of Fear, Hellblade

SURVIVAL HORROR (Stealth)

- No Weapons
- Stealth-based Gameplay
- Helpless Protagonist
- Chase Sequences
- Examples: Outlast, Amnesia, Alien Isolation

SUPERNATURAL HORROR

- Ghost/Demon Enemies
- Paranormal Activity
- Exorcism Theme

Religious Horror
Examples: Phasmophobia, The Conjuring, Dead by Daylight

PSYCHOLOGICAL (Narrative)
Story-driven Horror
Moral Choices
Unreliable Reality
Mental Health Theme
Examples: SOMA, Observer, Layers of Fear

14.2 Horror Game Design Elements

14.2.1 Fear (bhaya)

Definition: Player-kae saraaasarai bhaya daekhhaanao

Types of Fear:

Jump Scare	hathaaa bhayangakara kaichhau daekhhaanao
Unknown Fear	yaaa daekhaaa yaaaya naaa taaa bhayaera
Loss of Control	naiyanatarana haaaraanao
Vulnerability	asahaaaya anaubhaba karaaa

How to Create Fear:

1. Sudden Events	asabaaabhaaabaika shabada
2. Unnatural Sounds	asabaaabhaaabaika shabada
3. Dark Environments	anadhakaaara paraibaesha
4. Limited Visibility	kama darishayamaaanaataaa
5. Unpredictable Events	anairadaissata ghatanaaa
6. Vulnerability	asahaaayataba
7. Loss of Control	naiyanatarana haaaraanao

Real Game Examples:

Outlast	Helplessness + Dark Environments
Alien Isolation	Unpredictable Alien AI
P.T.	Loop-based Psychological Horror

14.2.2 Tension (utataejanaaaa)

Definition: Player-kae utataejaita au udabaigana raaakhaaa

How to Create Tension:

1. Confined Spaces	saiimaita samapada
2. Limited Resources	saiimaita samapada
3. Close Proximity	shatarau kaaachhae aasachhae
4. Audio Cues	shabada daiyae aasanana baipadaera ingagaita
5. Environmental	paraibaesha paraibaratana
6. Stalking	shatarau taaadaaaa karachhae
7. No Safe Place	kaonao nairaaapada jaaayagaaa naei

Tension Techniques:

Visual Tension	Camera Shake, Distortion
----------------	--------------------------

Gameplay Tension Time Pressure, Limited Ammo
Environmental paraibaesha paraibaratana, Sound Cues

****Real Game Examples:****

Dead Space anadhakaaara Corridors-ai Tension
Amnesia Darkness-ai Tension
SOMA Underwater Pressure + Tension

14.2.3 Suspense (paratayaaashaaa)

****Definition:**** kaichhau bhayangakara hatae yaaachachhae aii anaubhauutai

****How to Create Suspense:****

2. Build-up dhairae dhairae Tension baaadaanao
3. False Safety nairaaapada manae haauyaaa kainatau naaa
4. Delayed Horror bhaya aisae paauchhaatae samaya naeauyaaa
5. Uncertainty kii habae jaaanaaa naei

****Suspense Building Techniques:****

Sound Design karamabaradhamaaaana shabada
Visual Cues aalaora paraibaratana
Environmental darajaaa khaolaaa/banadha, shabada
Player Anticipation Player naijaei bhaya paaachachhae

****Real Game Examples:****

Silent Hill Fog-ai Suspense
Outlast Camera Flash-ai Suspense
Amnesia Darkness-ai Suspense

14.2.4 Uncertainty (anaishachayataaa)

****Definition:**** Player-kae anaishachaita karae raaakhaaa

****How to Create Uncertainty:****

2. Unreliable Reality yaaa daekhachhaena taaa sataya naaaau hatae paaarae
3. Multiple Interpretations aikaaadhaika bayaaakhayaaa samabhaba
4. Missing Information tathaya apaurana raaakhauna
5. Moral Ambiguity kaonao sathaika utatara naei

****Uncertainty Techniques:****

Visual asapassata darishaya
Audio asapassata shabada
Gameplay anaishachaita Consequences
Environment paraibaesha paraibaratana

****Real Game Examples:****

SOMA Identity Crisis
Layers of Fear Unreliable Reality
Hellblade Mental Illness Representation

14.2.5 Limited Information (saiimaita tathaya)

Definition: Player-kae saiimaita tathaya daeauyaaa

How to Create Limited Information:

1. Limited Visibilitykama aalao, Fog, Darkness
2. Limited Mapsamapauurana Map naei
3. Limited Savekama Save Points
4. Limited Clueskama Clue

Limited Information Techniques:

- VisibilityFlashlight Battery, Darkness
- Mapsamapauurana Map naaa daeauyaaa
- Save Pointskama Save Points
- Clue SystemClue kama raaakhauna

Real Game Examples:

- OutlastCamera Battery Limited
- Alien Isolation Limited Save Points
- AmnesiaNo Weapons, Limited Light

14.2.6 Sound (shabada)

Definition: shabada Design daiyae Horror Atmosphere taairai karaaa

Sound Types in Horror:

- MusicBackground Music (Creepy, Tense)
- SFXSound Effects (Door Creak, Footsteps)
- Silencenaiirabataaa (Silence)
- Dynamic SoundPlayer Action-aira upara nairabhara karae shabada paraibaratana

Sound Design Techniques:

1. Distant Soundsdauuraera shabada
2. Sudden Soundshathaaa shabada
3. Silencenaiirabataaa
4. Binaural Audio3D shabada
5. Dynamic Musicparaisathaitaira upara nairabhara karae sagaiita
6. HeartbeatPlayer Heartbeat Sound

Real Game Examples:

- Alien Isolation Alien Sound Cues
- P.T.Baby Crying + Loop Sound
- AmnesiaMonster Sound Cues

14.2.7 Lighting (aalao)

Definition: aalao bayabahaaara karae Horror Atmosphere taairai karaaa

Lighting Techniques:

Flickering	jhalakaaanai aalao
Shadows	chhaayaaa
Limited Light	saiimaita aalao (Flashlight, Candle)
Color	rangaera bayabahaaara (Red, Blue, Green)
Contrast	ujajabala au anadhakaaaraera paaarathakaya

****Lighting Mood:****

Red Light	baipada, rakata, aagauna
Blue Light	shaiitalataaa, aikaaakaitaba
Green Light	asausathataaa, baissaaakata
Flickering	asathairataaa, anaishachayataaa

****Real Game Examples:****

Outlast	Camera Night Vision
Alien Isolation	Motion Tracker Light
Layers of Fear	Dynamic Lighting Changes

14.2.8 Enemy AI (shatarau AI)

****Definition:**** shataraukae bhayangakara karae AI Design karaaa

****Enemy AI Types:****

Chase AI	Player-kae taaadaaa karaaa
Search AI	Player-kae khaujae baera karaaa
Ambush AI	laukaiyae thaaakaaa aiba haaamalaaa
Smart AI	Player-aira kaaushala baujhae aacharana
Unpredictable	anairadaissata aacharana

****AI Design for Horror:****

- | | |
|----------------------|---|
| 2. Sound Cues | shabada daiyae upasathaitai jaaanaaanao |
| 3. Smart Pathfinding | Player-kae khaujae paaaauyaaa |
| 4. Adaptation | Player-aira kaaushala shaekhaaa |
| 5. Territorial | naijaera ailaakaaa rakassaaa |
| 6. Fear Factor | Player-kae bhaya daekhaaanao |

****Real Game Examples:****

Outlast	Enemies (Sound-based)
Amnesia	Monster (Darkness-based)
Dead by Daylight	Killer AI (Chase-based)

14.2.9 Chase (taaadaaa)

****Definition:**** shatarau Player-kae taaadaaa karae

****Chase Design Elements:****

Path	taaadaaara patha
Hiding Spots	laukaaanaora jaaayagaaa
Obstacles	baaadhaaa
Duration	taaadaaara samayakaaala

Consequence dharaaa padalae kaii habae

****Chase Techniques:****

- | | |
|------------------------|---------------------------------|
| 2. Smart Enemy | shatarau baudadhaimaaana |
| 3. Time Pressure | samayaera chaaapa |
| 4. Dynamic Environment | paraibaesha paraibaratana |
| 5. Multiple Enemies | aikaaadhaika shatarau |
| 6. Audio Cues | shabada daiyae aasanana baipada |

****Real Game Examples:****

Alien Isolation Alien Chase Sequences
Amnesia Monster Chase
Dead by Daylight Killer vs Survivors

14.2.10 Safe Zone (nairaaapada jaaayagaaa)

****Definition:**** Player-kae baisharaaama naeauyaaara jaaayagaaa daeauyaaa

****Safe Zone Functions:****

- | | |
|----------------|--|
| Rest Point | baisharaaama naeauyaaara jaaayagaaa |
| Resource | Resource paaaauyaaara jaaayagaaa |
| Story Point | galapa jaaanaaara jaaayagaaa |
| Puzzle Solving | dhaaadhaaa samaaadhaaanaera jaaayagaaa |

****Safe Zone Design:****

- | | |
|------------------------|----------------------------|
| 2. Distinct Appearance | aalaaadaaa daekhatae |
| 3. No Enemies | shatarau naei |
| 4. Calm Atmosphere | shaaanata paraibaesha |
| 5. Resource Supply | Resource paaaauyaaa yaaaya |

****Real Game Examples:****

Silent Hill	Save Points
Amnesia	Safe Areas
Dead Space	Save Stations

14.2.11 Puzzle (dhaaadhaaa)

****Definition:**** Horror Context-ai dhaaadhaaa samaaadhaaana karaaa

****Horror Puzzle Types:****

- | | |
|-----------------|----------------------------------|
| Code Puzzle | Code khaujae baera karaaa |
| Logic Puzzle | yaukatai parayaoga karaaa |
| Environmental | paraibaesha paraibaratana karaaa |
| Item Puzzle | Item bayabahaaara karaaa |
| Sequence Puzzle | sathaika karama anausarana |

****Horror Puzzle Design:****

- | | |
|-------------------|--|
| 2. Limited Clues | kama Clue |
| 3. Danger Present | baipada samayae dhaaadhaaa samaaadhaaana |

4. Resource Cost Resource kharacha
5. Environmental Danger paraibaeshaera baipada

****Real Game Examples:****

Silent Hill	Abstract Puzzles
Amnesia	Physics-based Puzzles
The Witness	Environmental Puzzles

14.2.12 Environmental Storytelling (paraibaeshagata galapa)

****Definition:**** paraibaesha daiyae galapa balaaa

****Environmental Storytelling Techniques:****

2. Audio Logs adaiau laga
3. Notes/Documents naota/dalaila
4. Environmental paraibaeshaera paraibaratana
5. Body Placement maritadaehaera abasathaaana
6. Destruction dhabasaera chaihana
7. Time Period samayakaaalaera nairadaesha

****Environmental Story Design:****

Imply	ingagaita daina
Fragmented	khanadaita galapa
Multiple Layers	aikaaadhaika satara
Player Discovery	Player naijae khaujae baera karauka

****Real Game Examples:****

Bioshock	Audio Logs + Visual Clues
Gone Home	House Exploration
What Remains of Edith Finch Family Story through Environment	

14.2.13 Jump Scare (hathaaa bhaya)

****Definition:**** hathaaa bhayangakara kaichhau daekhhaanao

****Jump Scare Types:****

Audio Jump Scare	hathaaa bhayangakara shabada
Combined	darishaya + shabada aikasaaathae
Fake-out	bhayaera aabhaaaasa kainatau kaichhau naya
Delayed	paratayaaashaaara para Jump Scare

****Jump Scare Design:****

2. Timing sathaika samayae daekhhaana
3. Sound shabada bayabahaaara karauna
4. Context parasangagaera saaathae mailaiyae daekhhaana
5. Rarity baaarabaaara naaa daekhhaana
6. Quality bhaaalao maaanaera daekhhaana

****Jump Scare Guidelines:****

Build Tension	Overuse
Use Sound	Show Too Early
Context	Cheap Feeling
Rare Usage	No Build-up
Quality	Predictable

****Real Game Examples:****

Outlast	Jump Scares during Chase
P.T.	Loop-based Jump Scares
Layers of Fear	Environmental Jump Scares

14.2.14 Psychological Horror (manaobaaijanyaaanaika bhaya)

****Definition:**** maaanasaika bhaya taairai karaaa

****Psychological Horror Elements:****

2. Guilt	aparaaadhabaodha
3. Paranoia	sanadaeoha
4. Identity Crisis	paraichaya sakata
5. Moral Dilemma	naaitaika dabaidhaaa
6. Unreliable Senses	anairabharayaogaya inadaraiya
7. Trauma	aaghaaata

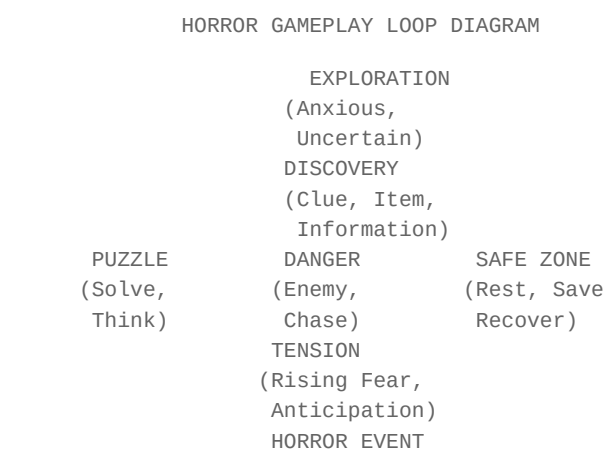
****Psychological Horror Techniques:****

2. Reality Shifts	baaasatabataaa paraibaratana
3. Symbolism	parataiikabaaaada
4. Isolation	aikaaakaitaba
5. Paranoia	sanadaeoha
6. Memory	samaritai

****Real Game Examples:****

Hellblade	Mental Illness
Layers of Fear	Unreliable Reality
SOMA	Identity + Philosophy

14.3 Horror Gameplay Loop Diagram



(Jump Scare,
Chase, Death)
RESOLUTION
(Survive, Die,
Learn, Adapt)
> (Loop Back to Exploration)

Key: Each Loop should Vary in Intensity and Duration

14.4 Fear Curve Diagram

FEAR CURVE DIAGRAM

Fear Level
Time

- Story Points:
- | | | |
|-----------------|-----------------|------------------|
| 1. Introduction | 4. First Chase | 7. Major Setback |
| 2. Discovery | 5. Brief Safety | 8. Climax |
| 3. First Scare | 6. Escalation | 9. Resolution |

Key: Fear should RISE and FALL, Never Stay Constant

- Fear Curve Rules:
- | | |
|--------------------|---|
| 1. Start Low | bhaya dhairae dhairae baaadaanao |
| 2. Peaks & Valleys | bhaya au baisharaama aikasaaathae |
| 3. Build to Climax | shaessaera daikae sabachaeyae baeshai bhaya |
| 4. Never Flat | aikai satarae naaa thaaakauka |
| 5. Surprise | Player-kae abaaaka karauna |
| 6. Vary Intensity | taibarataaa paraibaratana karauna |

14.5 Practical Tips from Real Horror Games

Resident Evil Series

- Tip 1: Limited Resources Create Tension
- Tip 2: Safe Rooms Provide Relief
- Tip 3: Save Points Create Anticipation
- Tip 4: Environmental Storytelling
- Tip 5: Jump Scares with Build-up

Outlast

- Tip 1: Helpless Protagonist Increases Fear
- Tip 2: Camera Battery Creates Resource Management
- Tip 3: Sound Design is Critical
- Tip 4: Chase Sequences Create Panic
- Tip 5: No Weapons = Vulnerability

Alien Isolation

- Tip 1: Unpredictable AI Creates Fear
- Tip 2: Sound Cues Build Tension
- Tip 3: Limited Save Points Increase Stakes
- Tip 4: Motion Tracker Creates Suspense
- Tip 5: Safe Zones Provide Relief

Silent Hill Series

- Tip 1: Psychological Horror Beats Physical
- Tip 2: Ambiguity Creates Discussion
- Tip 3: Fog Limits Visibility
- Tip 4: Symbolism Adds Depth
- Tip 5: Multiple Endings Add Replayability

Amnesia: The Dark Descent

- Tip 1: Darkness = Danger
- Tip 2: No Combat = Vulnerability
- Tip 3: Sanity System Adds Stress
- Tip 4: Environmental Storytelling
- Tip 5: Monster Design Less is More

14.6 Horror Game Design Checklist

HORROR GAME DESIGN CHECKLIST

CORE HORROR ELEMENTS:

- Fear Element
- Tension Building
- Suspense
- Uncertainty
- Limited Information
- Jump Scares (if needed)
- Psychological Horror

ATMOSPHERE:

- Sound Design
- Lighting Design
- Environmental Design
- Color Palette
- Music/Ambient
- Pacing

ENEMY DESIGN:

- Enemy AI
- Chase Sequences
- Enemy Appearance
- Enemy Behavior
- Enemy Sound
- Enemy Variations

GAMEPLAY:

- Player Vulnerability
- Resource Management
- Puzzle Design
- Save System
- Win/Lose Conditions
- Difficulty Balance

STORY:

- Environmental Storytelling
- Narrative Structure
- Character Development
- Multiple Endings (optional)
- Lore/Background
- Emotional Impact

PLAYER EXPERIENCE:

Fear Curve
Tension Curve
Safe Zones
Reward System
Player Agency
Replayability

14.7 Exercise (anaushailana)

Exercise 1: Horror Game Concept

aikatai Horror Game Concept Design karauna:

- kaona Sub-genre?
- kaa Setting?
- kaona Enemy?
- kaa Mechanics?

Exercise 2: Fear Curve Design

aapanaaara Horror Game-aira janaya Fear Curve Design karauna:

- kakhana bhaya baaadabae?
- kakhana baisharaaama naebae?
- kakhana Climax habae?

Exercise 3: Horror Analysis

aikatai Published Horror Game Analyze karauna:

- kaa Techniques bayabahaaara karaechhae?
 - Sound Design kaemana?
 - Lighting Design kaemana?
 - Enemy AI kaemana?
-

14.8 Common Mistakes Summary (saaadhaaarana bhaula)

1. ****Overuse Jump Scares****: atairaikata Jump Scare bayabahaaara karaaa
 2. ****No Build-up****: Jump Scare-aira aagae Tension naaa taairai karaaa
 3. ****Poor Sound Design****: khaaaraaapa Sound Design
 4. ****Too Much Light****: atairaikata aalao raaakhaaa
 5. ****No Safe Zones****: nairaaapada jaaayagaaa naaa raaakhaaa
 6. ****Weak AI****: shatarau AI daurabala raaakhaaa
 7. ****No Variation****: aikai dharanaera Horror baaarabaaara
 8. ****Ignoring Pacing****: Pacing upaekassaaa karaaa
 9. ****Too Many Enemies****: atairaikata shatarau
 10. ****No Story****: Horror-aira paechhanae galapa naaa thaaakaaa
-

14.9 Pro Tips

1. **Less is More**: kama daekhaana, baeshai bhaya daekhaana
2. **Sound is King**: Sound Design sabachaeyae gaurautabapauurana
3. **Build Tension**: Jump Scare-aira aagae Tension taairai karauna
4. **Vulnerability**: Player-kae asahaaaya karauna
5. **Psychological**: maaanasaiika bhaya shaaaraiika bhayaera chaeyae bhaaalao
6. **Environmental**: paraibaesha daiyae galapa balauna
7. **Pacing**: Pacing paraibaratana karauna
8. **Safe Zones**: nairaaapada jaaayagaaa daina
9. **Lessons from Masters**: bhaaalao Horror Games thaekae shaikhauna
10. **Test with Players**: Player-daera daiyae Test karauna

14.10 Summary (saaarasakassaepa)

Horror Game Design halao aikatai chayaaalaenyajai kainatau majaaara baissaya sathaika Technique bayabahaaara karalae Player-kae bhayaera anaubhauutai daeuyaaa samabhaba

mauula paaatha:

- Fear, Tension, Suspense, Uncertainty taairai karauna
- Sound Design aiba Lighting Design gaurautabapauurana
- Enemy AI bhayangakara karauna
- Player Vulnerability raaakhauna
- Environmental Storytelling bayabahaaara karauna
- Jump Scare kama bayabahaaara karauna
- Psychological Horror baeshai bayabahaaara karauna
- Fear Curve Follow karauna
- Safe Zones daina
- Test karauna

manae raaakhabaena, **"The scariest thing is what you don't see."** kama daekhaana, baeshai bhaya daekhaana

14.11 Final Words

Horror Game Design aikatai Art Form aitari shaekhaara janaya samaya aiba anaushailana darakaaara bhaaalao Horror Game taairai karatae Player Psychology baojhaa darakaaara

Remember:

- Horror is about FEELING, not just SHOWING
- Sound is MORE IMPORTANT than Visual
- Less is MORE in Horror
- Player Vulnerability is KEY
- Environmental Storytelling is POWERFUL

- Fear Curve should RISE and FALL
- Test with REAL PLAYERS

****Good luck with your Horror Game!****

aii adhayaaayaera maaadhayamae "GAME DEVELOPMENT MASTERBOOK" aira 14tai adhayaaaya samapanana halao aapanai aikhana Game Development-aira mauula baissayagaulao samaparakae jaaanaena aikhana samaya aisaechhae naijaera parathama Game baaanaanaora!

PART 15

adhayaaaya 15: LEVEL DESIGN -- laebhaela daijaaaina

adhayaaaya 15: LEVEL DESIGN -- laebhaela daijaaaina

shaekhaaara udadaeshaya (Learning Goal)

aii adhayaaaya shaessae aapanai jaaanabaena:

- Level Design kaii aiba kaena aitai gaemaera sabachaeyae gaurautabapaurana asha
 - Level Flow aiba Player Navigation kaibhaaaba kaaaja karae
 - Pacing aiba Difficulty Curve kaibhaaaba taairai karabaena
 - Encounter Design, Tutorial, Environmental Storytelling
 - Checkpoints, Secrets, Rewards saisataema
 - Text-based Level Layout taairai karaaa
-

15.1 Level Design kaii? (What is Level Design?)

Level Design halao gaemaera aikatai parithaka ailaaakaaa baaa Stage taairai karaaara parakaraiyaaa aitai shaudhau maaanachaitara aakaaa naya -- aitai halao aimanabhaaaba jaaayagaaa taairai karaaa yaekhaanae khaelaoyaaada aikatai nairadaissata abhaijanyataaa paaaya

Level Designer aira kaaaja:

- khaelaoyaaadake kaothaaaya yaetae habae taaa nairadaesha karaaa
- chayaaalaenyaja aiba baisharaamaera samanabaya ghataanao
- galapa balaaa (Environmental Storytelling)
- paurasakaaara aiba laukaaanao aayatataana (Secrets) sathaaapana karaaa
- khaelaoyaaadaera dakassataaa paraiikassaaa karaaa

Level Design aira mauulanaiitai:

1. **Readability** -- khaelaoyaaada baujhatae paaarabae kaothaaaya yaetae habae
 2. **Fairness** -- chayaaalaenyaja kathaina kainatau asamabhaba naya
 3. **Flow** -- laebhaelae aikatai sabaaabhaaabaika gatai thaaakatae habae
 4. **Reward** -- khaelaoyaaadaera kaaajaera janaya paurasakaaara thaaakatae habae
-

15.2 Level Flow aiba Player Navigation

Level Flow (laebhaela parabaaaha)

Level Flow halao khaelaoyaaadaera aikatai laebhaelae parabaesha thaekae shaessa parayanata yaaataraaapatha aikatai bhaaalao Level Flow khaelaoyaaadake sabaaabhaaabaikabhaaabaie aigaiyae naiyae yaaaya

Level Flow Diagram:

```
[Spawn] -> [Tutorial] -> [Exploration] -> [Puzzle] -> [Enemy]
                                         DOWN
[Exit] <- [Reward] <- [Mini Boss] <- [Checkpoint] <- [Combat]
```

****Level Flow aira dhaapasamauha:****

Player Navigation (khaelaoyaaada naebhaigaeshana)

khaelaoyaaadake sathaika daikae nairadaesha karaara padadhataisamauha:

****1. Visual Cues (darishayamaana sakaeta):****

- ujajabala rangaera jaaanaalaaa baaa aalao
- pathaera daikae ingagaitakaaaraii basatau
- lamabaaa kaaathaaamao yaaa dauura thaekae daekhaaa yaaaya

****2. Audio Cues (sharaaabaya sakaeta):****

- paaanaira shabada -> paaanaira kaaachhae yaaana
- baaataasaera shabada -> khaolaaa jaaayagaaa
- shataura shabada -> baipadaera kaaachhae

****3. Lighting (aalaoka bayabasathaaa):****

- ujajabala ailaaakaaa = patha
- anadhakaaara ailaaakaaa = baipada baaa laukaaanao jaaayagaaa
- laaala aalao = baipadaera sakaeta

****4. Architecture (sathaaapataya):****

- sarala patha = aigaiyae yaaaayaaa
- ghaurae ghaurae patha = anausanadhaana
- saidai = uparae uthaaa

15.3 Pacing aiba Difficulty Curve

Pacing (gatai naiyanatarana)

Pacing halao gaemaera utataejanaaa aiba baisharaamaera madhayae bhaaarasaaamaya rakassaaa karaaa aikatai gaema yadai sabasamaya utataejanaaamaya thaaakae, khaelaoyaaada kalaaanata hayae yaaabae aara yadai sabasamaya shaaanata thaaakae, khaelaoyaaada bairakata hayae yaaabae

****Pacing aira dharana:****

Pacing Curve Diagram:

taibarataaa (Intensity)

UP

-> samaya (Time)

[shaaanata] [utataejanaaa] [baisharaaama] [utataejanaaa] [shaaanata]

****Pacing aira udaaaharana:****

Difficulty Curve (kathainataaa bakararaekhaaa)

Difficulty Curve halao gaemaera kathainataaa kaibhaaaba samayaera saaathae baaadae taaara parakaaasha

Difficulty Curve:

kathainataaa (Difficulty)

UP

-> samaya (Time)

[sahaja] -> [maajhaaraai] -> [kathaina] -> [khauba kathaina]

****Difficulty Curve aira naiyama:****

1. ****shaurau sahaja**** -- khaelaoyaaadaka shaekhaanaora sauyaoga daina
2. ****dhairae dhairae baaadaana**** -- aikataanae kathaina karabaena naaa
3. ****chauudaaanata kathaina**** -- Boss baaa shaessa chayaaalaenyaja
4. ****baisharaaama daina**** -- kathaina parae sahaja asha daina
5. ****paunaraabaritai naya**** -- aikai kathainataaa baaarabaaara karabaena naaa

15.4 Encounter Design (ainakaaaunataaara daijaaaina)

Encounter Design halao khaelaoyaaada aiba shataraura madhayae ladaaaiyaera paraikalapanaaa karaaa

****Encounter Design aira dhaaapasamauha:****

1. ****shataraura dharana nairadhaaaraana**** -- kaona dharanaera shatarau thaaakabae
2. ****ailaakaaa parasatautai**** -- ladaaaiyaera jaaayagaaa taairai
3. ****abasathaaana nairadhaaaraana**** -- shataraura abasathaaana
4. ****kaaushala nairadhaaaraana**** -- khaelaoyaaadaka kaaushala bayabahaaara karatae habae
5. ****paurasakaaara nairadhaaaraana**** -- ladaaaiyaera para kaaipaaabae

****Encounter Types:****

15.5 Tutorial (taiutaoraiyaaala)

Tutorial halao khaelaoyaaadaka gaemaera naiyama aiba kaaushala shaekhaanaora padadhatai

****Tutorial aira dharana:****

****1. Explicit Tutorial (sapassata taiutaoraiyaaala):****

- taekasata bakasa daiyae nairadaeshanaaa
- UI uinadao daiyae shaekhhaanao
- udaaaharana: "WASD daiyae chalauna"

****2. Implicit Tutorial (asapassata taiutaoraiyaaala):****

- gaemapalaera madhayae shaekhhaanao
- parathama shatarau sahaja haya
- parathama samasayaaa sahaja haya

****3. Environmental Tutorial (paraibaeshagata taiutaoraiyaaala):****

- paraibaesha daiyae shaekhhaanao
- laaala ranga = baipada
- sabauja ranga = nairaaapada

****Tutorial Design aira naiyama:****

1. ****aikabaaarae aikatai shaekhhaana**** -- anaeka kaichhau aikasaaathae shaekhhaabaena naaa
2. ****anaushailanaera sauyaoga daina**** -- shaekhhaanaora para bayabahaaara karatae daina
3. ****kathainataaa dhairae baaadaana**** -- sahaja thaekae shaurau karauna
4. ****paunaraaabaritatai karauna**** -- shaekhhaanao kaaushala baaarabaaara bayabahaaara karauna
5. ****bairakata karabaena naaa**** -- khaelaoyaaadake thaaamaiyae raaakhabaena naaa

15.6 Environmental Storytelling (paraibaeshagata galapa balaaa)

Environmental Storytelling halao paraibaesha bayabahaaara karae galapa balaaa aikhaanae kaonao daaayaaalaga naei, shaudhau darishaya

****Environmental Storytelling aira padadhatai:********1. Props (upakarana):****

- bhaaangaaa jaaanaaalaaa -> aikhaanae kaichhau aikataaa ghataechhae
- khaaalai khaaachaaa -> kaeu paaalaiyae gaechhae
- chhadaanao bai -> hatayaaara sathaaana

****2. Architecture (sathaaapataya):****

- dhabasaparaaapata bhabana -> yaudadha hayaechhae
- saunadara baaagaaana -> aikhaanae saukhaii jaiibana chhaila
- anadhakaaara patha -> baipadaera ailaaakaaa

****3. Lighting (aalaoka bayabasathaaa):****

- ussana aalao -> nairaaapada ailaaakaaa
- thaaanadaaa aalao -> baipadaera ailaaakaaa
- laaala aalao -> rakatapaaata

****4. Audio (shabada):****

- shaaanata sagaiita -> nairaaapada
- utataejanaaapauurana sagaiita -> baipada

- naiirabataaa -> apaekassamaana baipada

****udaaaharana:****

- khaaalai khaaachaaa (kaeu paaalaiyae gaechhae)
- chhadaanao kaaagaja (hatayaaara sathaaana)
- bhaaangaaa jaaanaaalaaa (paaalaaanaora chaessataaa)
- aikatai chhaota khaelanaaa (shaishau chhaila)
- rakataera daaaga (yauadha hayaechhae)

aii basataugaulao mailaiyae khaelaoyaaada baujhatae paaarae:

"aikhaanae aikatai paraibaaara chhaila hathaa kaeu aisae aakaramana karaechhae
taaaaa paaalaaanaora chaessataaa karaechhae kainatau anaekae maaaaa gaechhae
aikatai shaishau chhaila yae khaelanaaa raekhae gaechhae"

aikat

15.7 Checkpoints, Secrets, aiba Rewards

Checkpoints (chaekapayaenata)

Checkpoint halao saei jaaayagaaa yaekhaanae khaelaoyaaada maaaaa gaelae saekhaana thaekae shaurau karae

****Checkpoint Design:****

- parataitai kathaina ladaaiyaera aagae Checkpoint raaakhauna
- Checkpoint nairaaapada jaaayagaaaya raaakhauna
- Checkpoint sapassata daekhaanao uchaita

Secrets (laukaanao aayatatana)

Secrets halao laukaanao jaaayagaaa baaa aayatatana yaaa khaelaoyaaadake anausanadhaana karatae haya

****Secrets aira dharana:****

- laukaanao kakassa
- gaopana patha
- atairaikata chayaaalaenyaja
- laukaanao aaitaema

****Secrets Design aira naiyama:****

1. ****sauyaoga daina**** -- khaelaoyaaadake sakaeta daina
2. ****paurasakaaara daina**** -- laukaanao aayatatana thaekae kaichau paaabae
3. ****kathaina karauna**** -- sahajae paaaauyaaa yaaaya naaa
4. ****paunaraabaritaitai karauna**** -- aikai dharanaera Secret baaarabaaara bayabahaaara karauna

Rewards (paurasakaaara)

Reward halao khaelaoyaaadaera kaaajaera janaya paurasakaaara

****Reward Types:****

15.8 Level Design Diagram

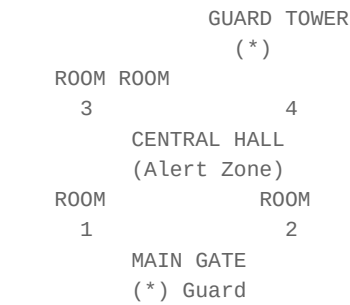
Complete Level Flow Diagram:

```

                                LEVEL 1: FOREST PATH
SPAWN > TUTORIAL > EXPLORATION> PUZZLE
*
    (WASD)                (Search)                (Switch)
EXIT  < REWARD  <  MINI BOSS  <
*
    (Loot)                (Guardian)
                                ENEMY FIGHT
                                (Wolf x 3)
                                CHECKPOINT
                                (Save Point)

```

Level Layout: "Enemy Base"



- Legend:
- * = Guard
 - = Wall
 - = Door

- Stealth Mechanics:
- Guards patrol in fixed routes
 - Light and shadow zones
 - Noise detection system
 - Camera angles

15.10 Practical Examples from Real Games

1. Super Mario Bros (Nintendo)

Level Design Principles:

- parataitai laebhaela natauna kaichhau shaekhaaaya
- shaurau sahaja, shaessae kathaina
- patha paraissakaaara
- paurasakaaara sapassata

Example: World 1-1

- > [First Jump] -> [Pipe Challenge] -> [Underground]
- > [Flagpole] -> [End]

2. Dark Souls (FromSoftware)

Level Design Principles:

- khauba kathaina
- anaeka Checkpoint (Bonfire)
- laukaaanao patha
- Interconnected World

Example: Undead Burg

- > [Bonfire] -> [Ladder] -> [Roof Combat]
- > [Bridge] -> [Mini Boss] -> [Bonfire]
- > [Boss Area] -> [Boss Fight]

3. The Legend of Zelda: Breath of the Wild (Nintendo)

****Level Design Principles:****

- Open World
- Player Choice
- Environmental Storytelling
- Multiple Solutions

****Example: Great Plateau****

-> [Magnesis Rune] -> [Cryonis Rune] -> [Bomb Rune]
 -> [Combat Training] -> [Shrine] -> [Paraglider]
 -> [Exit to World]

[Star

4. Half-Life 2 (Valve)

****Level Design Principles:****

- Linear with Exploration
- Environmental Storytelling
- Physics-based Puzzles
- Narrative Integration

****Example: Ravenholm****

-> [Rooftop Navigation] -> [Trap Usage] -> [Tunnel]
 -> [Minecart Sequence] -> [Exit to Coast]

[Enter

15.11 Common Level Design Mistakes (saaadhaaarana bhaula)

bhaula 1: Confusing Layout

****samasayaaa:**** khaelaoyaaada baujhatae paaarae naaa kaotaaaya yaetae habae

****samaaadhaaana:**** Visual Cues bayabahaaara karauna

bhaula 2: Too Much Combat

****samasayaaa:**** khaelaoyaaada kalaaanata hayae yaaaya

****samaaadhaaana:**** Pacing bayabahaaara karauna

bhaula 3: No Rewards

****samasayaaa:**** khaelaoyaaada anausanadhaaana karatae chaaaya naaa

****samaaadhaaana:**** Secrets aiba Rewards raaakhauna

bhaula : No Checkpoints

****samasayaaa:**** khaelaoyaaada maaaraaa gaelae anaeka dauura thaekae shaurau karatae haya

****samaaadhaaana:**** Checkpoints raaakhauna

bhaula 4: Too Many Enemies

****samasayaaa:**** khaelaoyaaada atairaikata chayaaalaenyajae haaaraiyae yaaaya
****samaaadhaaana:**** Encounter Design bayabahaaara karauna

bhaua 5: No Environmental Storytelling

****samasayaaa:**** laebhaela shaudhau khaaalai jaaayagaaa
****samaaadhaaana:**** Props, Lighting, Audio bayabahaaara karauna

15.12 Level Design Tips (paraaamarasha)

taipa 1: Playtest karauna

parataitai Level Playtest karauna anaya khaelaoyaaadadaera khaelatae daina

taipa 2: Reference sagaraha karauna

anayaaanaya gaemaera Level Design daekhauna kii kaaaja karaechhae, kii karaenai

taipa 3: Paper Prototype taairai karauna

kaaagajae aakauna kamapaiutaaarae aagae thaekae bhaaabauna

taipa 4: Iteration karauna

aikabaaarae paaaraphaekata habae naaa baaarabaaara paraibaratana karauna

taipa 5: Player Perspective bhaaabauna

khaelaoyaaada kii anaubhaba karabae taaa bhaaabauna

taipa : Feedback naina

khaelaoyaaadadaera kaaachha thaekae Feedback naina

taipa 6: Documentation karauna

parataitai Level aira dakaumaenataeshana raaakhauna

taipa 7: Modular Design bayabahaaara karauna

aikai basatau baaarabaaara bayabahaaara karauna aita samaya baaachae

taipa 8: Lighting gaurautabapauurana

Lighting daiyae patha daekhaana, maejaaaja taairai karauna

taipa 9: Audio bayabahaaara karauna

shabada daiyae nairadaeshanaaa daina, maejaaaja taairai karauna

taipa 10: Fun First

sabachaeyae gaurautabapauurana halao gaematai majaaara hatae habae

15.13 Summary (saaarasakassaepa)

Level Design halao gaemaera sabachaeyae gaurautabapauurana asha aikatai bhaaalao Level Designer hatae halae:

1. ****Level Flow**** baujhauna -- khaelaoyaaada kaibhaaabaie aikatai laebhaelae chalaachala karae
2. ****Pacing**** baujhauna -- utataejanaaa aiba baisharaamaera bhaaarasaaamaya
3. ****Difficulty Curve**** baujhauna -- kathainataaa kaibhaaabaie baaadaaabaena
4. ****Encounter Design**** shaikhauna -- shataraura saaathae ladaaaiyaera paraikalapanaaa
5. ****Tutorial**** shaikhauna -- khaelaoyaaadakaie kaibhaaabaie shaekhaaabaena
6. ****Environmental Storytelling**** shaikhauna -- paraibaesha daiyae galapa balauna
7. ****Checkpoints, Secrets, Rewards**** shaikhauna -- khaelaoyaaadakaie usaaahaita karauna
8. ****Practice**** karauna -- baaarabaaara laebhaela taairai karauna
9. ****Playtest**** karauna -- anaya khaelaoyaaadadaera khaelatae daina
10. ****Iterate**** karauna -- baaarabaaara paraibaratana karauna

****manae raaakhauna:****

"Level Design halao kaichhau naya, aitari halao abhaijanyataaa taairai karaaa aapanai shaudhau jaaayagaaa taairai karachhaena naaa, aapanai samaritari taairai karachhaena"

anaushailana (Exercises)

anaushailana 1: Level Flow taairai karauna

aikatai chhaota gaemaera Level Flow taairai karauna kamapakassae 5tai dhaapa thaaakatae habae

anaushailana 2: Text-Based Level Layout

aikatai Platform Game Level aira Text-Based Layout taairai karauna

anaushailana 3: Pacing Curve

aikatai Boss Level aira Pacing Curve aakauna

anaushailana 4: Encounter Design

aikatai Encounter Design karauna yaekhaanae 3tai dharanaera shatarau thaaakabae

anaushailana 5: Environmental Storytelling

aikatai gharaera barananaaa karauna yaekhaanae Environmental Storytelling bayabahaaara karaechhaena

anaushailana 6: Complete Level

aikatai samapauurana Level Design karauna yaaatae Tutorial, Exploration, Puzzle, Enemy, Boss, Reward, Exit thaaakabae

anaushailana 7: Secret Design

aikatai Secret Passage daijaaina karauna yaekhaanae laukaaanao aayatataa aachhae

anaushailana 1: Study Real Games

3tai bhainana gaemaera Level Design baishalaessana karauna

parabarataii adhayaaaya: [Chapter 16: CHARACTER DESIGN](16_character_design.md)

PART 16

adhayaaaya 16: CHARACTER DESIGN -- kayaaaraekataaara daijaaaina

adhayaaaya 16: CHARACTER DESIGN -- kayaaaraekataaara daijaaaina

shaekhaaara udadaeshaya (Learning Goal)

aii adhayaaaya shaessae aapanai jaaanabaena:

- Character Design aira samapauurana parakaraiyaaa
 - Concept, Silhouette, Personality, Backstory, Abilities
 - Movement, Animation, Clothing, Weapons, VFX
 - Character Design Sheet taairai karaaa
 - Character Archetypes aiba Color Psychology
 - Step-by-Step Process
-

16.1 Character Design kائي? (What is Character Design?)

****Character Design**** halao aikatai gaema kayaaaraekataaaraera chaehaaaraaa, bayakataitaba, aiba aacharana nairadhaaarana karaaara parakaraiyaaa aikatai bhaaalao Character Design khaelaoyaaadake kayaaaraekataaaraera saaathae samaparaka sathaaapana karatae saaahaaayaya karae

****Character Designer aira kaaaja:****

- kayaaaraekataaaraera chaehaaaraaa nairadhaaarana karaaa
- kayaaaraekataaaraera bayakataitaba taairai karaaa
- kayaaaraekataaaraera galapa balaaa
- kayaaaraekataaaraera kassamataaa nairadhaaarana karaaa
- kayaaaraekataaaraera ayaaanaimaeshana paraikalapananaa karaaa

****Character Design aira gaurautaba:****

- khaelaoyaaada kayaaaraekataaaraera saaathae samaparaka sathaaapana karae
 - gaemaera galapa shakataishaaalaa karae
 - gaema baikarai haya (Marketing)
 - khaelaoyaaada samarana raaakhae
-

16.2 Character Design Process (parakaraiyaaa)

dhaaapa 1: Concept (dhaaaranaaaa)

Concept halao kayaaaraekataaaraera mauula dhaaaranaaaa aitai kae? kii karae? kaena gaurautabapauurana?

Concept Development Questions:

- 1. kayaaaraekataaara kae? (naaama, bayasa, laingaga)
- 2. kaothaaaya baaasa karae? (parithaibaii, mahaaakaaasha, jaaadau jagata)
- 3. kii karae? (yaodadhaaa, yaaadaukara, parayaukataibaida)
- 4. kaena gaurautabapauurana? (galapae bhauumaikaaa)
- 5. kii samasayaaara samamaukhaiina? (Conflicts)
- 6. kaiibhaaaba samasayaaa samaaadhaana karae? (Abilities)

Example Concept:

bayasa: 25 bachhara
paeshaaa: paraaachaiina yaodadhaaa
baaasasathaaana: jaaadau jagata
samasayaaa: anadhakaaara shakataira bairaudadhae ladaaai
kassamataaa: aagaunaera yaaadau
lakassaya: jagatakae rakassaaa karaaa

naaama

dhaaapa 2: Silhouette (chhaayaaa chhabai)

Silhouette halao kayaaaraekataaaraera kaaalao chhabai aikatai bhaaalao kayaaaraekataaara shaudhau chhaayaaa daiyaei chaenaaa yaaaya

Silhouette Design Rules:

- 1. **Distinct Shape** -- sapassata aakaaara hatae habae
- 2. **Readability** -- dauura thaekaeau chaenaaa yaaaya
- 3. **Personality** -- chhaayaaa daiyae bayakataitaba baojhaaa yaaaya
- 4. **Variety** -- saba kayaaaraekataaara aikai rakama habae naaa

Silhouette Examples:

Warrior: Mage: Rogue:

dhaaapa 3: Personality (bayakataitaba)

Personality halao kayaaaraekataaaraera sababhaaaba aiba aacharana

Personality Types:

Personality Development:

- 1. **Strengths** -- kayaaaraekataaaraera shakatai kii?
- 2. **Weaknesses** -- kayaaaraekataaaraera daurabalataaa kii?
- 3. **Goals** -- kayaaaraekataaaraera lakassaya kii?
- 4. **Fears** -- kayaaaraekataaara kaisae bhaya paaaya?
- 5. **Motivation** -- kayaaaraekataaara kaena aitai karachhae?
- 6. **Flaws** -- kayaaaraekataaaraera tarautai kii?

dhaaapa 4: Backstory (patabhauumai)

Backstory halao kayaaaraekataaaraera ataiita aiba yae ghatanaaagaulao taaakae aikhanakaaara karae taulaechhae

Backstory Elements:

- 1. **Origin** -- kayaaaraekataaara kaothaaa thaekae aisaechhae?
- 2. **Key Events** -- kaona ghatanaaagaulao taaakae parabhaaabaita karaechhae?
- 3. **Relationships** -- kaaara saaathae samaparaka aachhae?
- 4. **Transforming Moment** -- kaona ghatanaaaya sae badalae gaechhae?
- 5. **Current Situation** -- aikhana kaothaaaya aachhae, kaii karachhae?

Example Backstory:

patabhauumai:
- aikatai dhabasa hayae yaaaauyaaa raaajayaera raaajapautara
- 10 bachhara bayasae anadhakaaara shakatai taaara paraibaaarakae dhabasa karaechhae
- aikatai baridadha yaodadhaa taaakae paaalaiyae ainaechhae
- 15 bachhara dharae parashaikassana naiyaechhae
- aikhana anadhakaaara shakataira bairaudadhae ladachhae
- taaara lakassaya: anadhakaaara shakataikae dhabasa karaaa

dhaaapa 5: Abilities (kassamataaa)

Abilities halao kayaaaraekataaaraera kassamataaa aiba dakassataaa

Ability Types:

Ability Design Rules:

- 1. **Balance** -- kassamataaa saussama hatae habae
- 2. **Synergy** -- kassamataaagaulao aikasaaathae kaaaja karabae
- 3. **Limitation** -- parataitai kassamataaara saiimaaabadadhataaa thaaakabae
- 4. **Growth** -- kassamataaa samayaera saaathae baaadabae

dhaaapa 6: Movement (gatai)

Movement halao kayaaaraekataaara kaibhaaabae chalaachala karae

Movement Types:

dhaaapa 7: Animation (ayaaanaimaeshana)

Animation halao kayaaaraekataaaraera chalaachalaera chaitaraayana

Animation States:

dhaaapa 8: Clothing (paoshaaaka)

Clothing halao kayaaaraekataaaraera paoshaaaka aiba paoshaaakaera upaaadaana

Clothing Design Principles:

- 1. **Functionality** -- paoshaaaka kaaajaera janaya upayaukata habae

- 2. **Personality** -- paoshaaaka bayakataitaba parakaaasha karabae
- 3. **Cultural** -- saaasakaritaika parabhaaaba thaaakabae
- 4. **Practical** -- yaudadhae baaadhaaa habae naaa

Clothing Types:

dhaaapa 9: Weapons (asatara)

Weapons halao kayaaaraekataaaraera asatara aiba yaudadha saranyajaaama

Weapon Types:

dhaaapa 10: VFX (bhaijauyaaala iphaekata)

VFX halao kayaaaraekataaaraera saaathae samaparakaita bhaijauyaaala iphaekata

VFX Types:

16.3 Character Design Sheet (kayaaaraekataaara daijaaaina shaita)

Character Design Sheet halao aikatai nathai yaekhaanae kayaaaraekataaaraera samapauurana tathaya thaaakae

CHARACTER DESIGN SHEET

Basic Information

Name: aarajauna (Arjun)

Age: 25

Gender: Male

Race: Human

Class: Fire Warrior

Alignment: Good

Silhouette

* <- Head with helmet

<- Shoulder armor

<- Body armor

<- Arms with gauntlets

<- Legs with boots

Personality

Trait: Brave, Determined, Compassionate

Flaw: Reckless, Hot-headed

Goal: Destroy Dark Forces

Fear: Losing loved ones

Motivation: Revenge for family

Backstory

- Prince of destroyed kingdom

- Family killed by Dark Forces at age 10

- Raised by old warrior master

- Trained for 15 years

- Now fights against darkness

- Seeks revenge and redemption

Abilities

Primary: Fire Sword

Secondary: Fire Shield
Special: Fire Storm (Ultimate)
Passive: Fire Resistance
Movement: Dash (Short teleport)

Combat Stats
Health: 100/100
Mana: 80/100
Attack: 70/100
Defense: 60/100
Speed: 90/100

Visual Design
Hair: Black, Long
Eyes: Brown
Skin: Light Brown
Height: 5'10"
Build: Athletic
Scars: Left cheek, Right arm

Clothing & Equipment
Head: Metal helmet with flame design
Chest: Red and gold armor
Arms: Metal gauntlets
Legs: Leather pants with armor plates
Feet: Metal boots
Weapon: Fire Sword (glowing red blade)
Shield: Fire Shield (round, flame pattern)

Color Palette
Primary: Red (#FF0000) - Passion, Fire
Secondary: Gold (#FFD700) - Royalty, Power
Accent: Black (#000000) - Mystery, Darkness
Skin: Tan (#D2B48C)
Hair: Black (#000000)

Animation States
Idle: Standing with sword ready
Walk: Confident stride
Run: Determined run with armor clanking
Jump: Dynamic leap
Attack 1: Horizontal slash
Attack 2: Vertical slash
Attack 3: Fire wave
Block: Shield forward
Hurt: Stagger back
Death: Fall to knees, then collapse

VFX
Sword Trail: Orange flame trail
Shield Glow: Red pulsing glow
Fire Storm: Massive fire explosion
Dash: Fire burst behind
Hit: Sparks and fire particles

16.4 Character Archetypes (kayaaaraekataaara aarakaitaaaipa)

Character Archetypes halao saaadhaaarana kayaaaraekataaara dharana yaaa baibhainana gaemae daekhaaa yaaaya

****Character Archetypes Table:****

aarakaitaaaipa 1: Hero (naaayaka)

****baibarana:**** galapaera mauula kayaaaraekataaara saaadhaaaranata saaahasaii, nayaaayaparaaayana aiba anayadaera saaahaaayaya karae

****udaaaharana:**** Mario, Link, Master Chief

****baaishaissataya:****

- saaahasaii
- nayaaayaparaaayana
- anayadaera saaahaaayaya karae
- samasayaaa samaaadhaana karae
- shataura bairaudadhae ladae

aarakaitaaaipa 2: Mentor (shaikassaka)

****baibarana:**** Hero kae shaekhaanao aiba paraamarasha daeuyaaa kayaaaraekataaara

****udaaaharana:**** Obi-Wan Kenobi, Gandalf, Alfred

****baaishaissataya:****

- janyaaanaii
- paraamarashadaataaa
- abhaijanya
- paraayai baridadha
- Hero kae gaaaida karae

aarakaitaaaipa 3: Villain (khalanaaayaka)

****baibarana:**** Hero aira shataura galapaera mauula biraodhaii

****udaaaharana:**** Bowser, Ganondorf, Sephiroth

****baaishaissataya:****

- shakataishaaalaii
- paraikalapanaaakaaaraii
- paraayai anadhakaaara
- kassamataaalaobhaii
- Hero kae paraajaita karatae chaaaya

aarakaitaaaipa 4: Sidekick (sahakaaaraii)

****baibarana:**** Hero aira banadhau aiba sahakaaaraii

****udaaaharana:**** Luigi, Tails, Ellie

****baaishaissataya:****

- baishabasata

- saaahaaayayakaaaraii
- paraaayai haaasayakara
- Hero aira paaashae thaaakae
- naijaeraau galapa thaaakae

aarakaitaaaipa 5: Trickster (parataaaraka)

****baibarana:**** haaasayakara aiba anaumaaana karaaa kathaina kayaaaraekataaara

****udaaaharana:**** Joker, Loki, Navi

****baaishaissataya:****

- haaasayakara
- anaumaaana karaaa kathaina
- naiyama bhaaangae
- paraaayai saaahaaayaya karae
- kainatau samasayaaaau sarissatai karae

aarakaitaaaipa 6: Anti-Hero (paratai-naaayaka)

****baibarana:**** naaayaka naya, kainatau galapae gaurautabapauurana bhauumaikaaa raaakhae

****udaaaharana:**** Kratos, Deadpool, Geralt

****baaishaissataya:****

- naaitaikabhaaabae dhauusara
- naijaera janaya kaaaja karae
- paraaayai haisara
- kainatau shaessae bhaaalao kaaaja karae
- gabhaiira bayakataitaba

16.5 Color Psychology for Characters (kayaaaraekataaaraera janaya rangaera manaobaijanyaaana)

ranga khaelaoyaaadaera manae nairadaissata anaubhauutai jaaagaiyae taolae

****Color Psychology Table:****

****Color Combinations for Characters:****

Hero Color Scheme:

Primary: Blue (#0000FF)

Secondary: White (FFFFFF)

Accent: Yellow (FFFF00)

Feeling: Trustworthy, Brave

Villain Color Scheme:

Primary: Red (FF0000)

Secondary: Black (#000000)
Accent: Purple (#800080)
Feeling: Dangerous, Powerful

Mage Color Scheme:

Primary: Purple (#800080)
Secondary: Blue (#0000FF)
Accent: Silver (#C0C0C0)
Feeling: Mysterious, Wise

16.6 Step-by-Step Character Design Process (dhaaapae dhaaapae parakaraiyaaa)

dhaaapa 1: Research (gabaessanaaaa)

- gaemaera parayaojanaiiyataaa baujhauna
- anayaaanaya gaemaera kayaaaraekataaara daekhauna
- Reference sagaraha karauna

dhaaapa 2: Concept (dhaaaranaaaa)

- kayaaaraekataaaraera mauula dhaaaranaaaa taairai karauna
- parashanaera utatara daina
- galapa laikhauna

dhaaapa 3: Sketch (khasadaaaa)

- anaeka khasadaaaa aakauna
- baibhainana dharana chaessataaaa karauna
- sabachaeyae bhaaalao baaachhaaai karauna

dhaaapa 4: Silhouette (chhaaayaaa)

- chhaaayaaa chhabai taairai karauna
- sapassata aakaaara naishachaita karauna
- dauura thaekae chaenaaa yaaaya kainaaa daekhauna

dhaaapa 5: Color (ranga)

- ranga nairadhaaarana karauna
- Color Psychology bayabahaaara karauna
- payaaalaeta taairai karauna

dhaaapa 6: Details (baisataaaraita)

- paoshaaaka yaoga karauna
- asatara yaoga karauna
- Accessory yaoga karauna

dhaaapa 7: Turnaround (ghaurae ghaurae)

- baibhainana kaona thaekae aakauna
- Front, Side, Back view
- taraimaaataraika rauupa naishachaita karauna

dhaaapa 1: Expression (baodhagamayataaa)

- baibhainana anaubhauutai daekhaana
- Happy, Sad, Angry, Fearful
- bayakataitaba parakaaasha karauna

dhaaapa 2: Sheet (shaita)

- Character Design Sheet taairai karauna
- samapauurana tathaya yaoga karauna
- dakaumaenataeshana samapauurana karauna

16.7 Common Character Design Mistakes (saaadhaaarana bhaula)

bhaula 1: Generic Design

****samasayaaa:**** kayaaaraekataaara anayaaanaya kayaaaraekataaaraera matao
****samaaadhaaana:**** ananaya baaishaissataya yaoga karauna

bhaula 2: Poor Silhouette

****samasayaaa:**** chhaayaaa chhabai sapassata naya
****samaaadhaaana:**** Silhouette daijaaaina unanata karauna

bhaula 3: No Personality

****samasayaaa:**** kayaaaraekataaara phaaakaaa
****samaaadhaaana:**** Personality yaoga karauna

bhaula 4: Over-design

****samasayaaa:**** khauba baeshai baisataaaraita
****samaaadhaaana:**** Simple aiba memorable raaakhauna

bhaula 5: Inconsistent Colors

****samasayaaa:**** ranga sausagata naya
****samaaadhaaana:**** Color Palette nairadhaaarana karauna

bhaula 6: No Backstory

****samasayaaa:**** kayaaaraekataaaraera galapa naei
****samaaadhaaana:**** Backstory taairai karauna

bhaulta 7: Poor Animation

****samasayaaa:**** ayaaanaimaeshana bhaaalao naya
****samaaadhaana:**** Animation States nairadhaaarana karauna

bhaulta 1: No Variation

****samasayaaa:**** saba kayaaaraekataaara aikai rakama
****samaaadhaana:**** Variety yaoga karauna

16.8 Character Design Tips (paraaamarasha)

taipa 1: Simple is Better

sahaja kayaaaraekataaara sabachaeyae samaranaiiya

taipa 2: Silhouette First

aagae chhaayaaa chhabai daijaaaina karauna

taipa 3: Color Psychology bayabahaaara karauna

ranga daiyae anaubhauutai jaaagaiyae taulauna

taipa 4: Backstory daina

kayaaaraekataaaraera galapa daina

taipa 5: Personality daina

kayaaaraekataarakae jaiibanata karauna

taipa 6: Reference sagaraha karauna

anayaaanaya gaema aiba chalachachaitara thaekae shaikhauna

taipa 7: Iteration karauna

baaarabaaara paraibaratana karauna

taipa 1: Playtest karauna

khaelaoyaaadadaera kaaachha thaekae Feedback naina

taipa 2: Memorability

kayaaaraekataaara samaranaiiya hatae habae

taipa 3: Function

kayaaaraekataaaraera kaaaja thaaakatae habae

16.9 Summary (saaarasakassaepa)

Character Design halao gaemaera anayatama gaurautabapauurana asha aikatai bhaaalao Character Designer hatae halae:

1. **Concept** taairai karauna -- kayaaaraekataaara kae, kii karae, kaena gaurautabapauurana
2. **Silhouette** daijaaaina karauna -- chhaayaa chhabai sapassata hatae habae
3. **Personality** taairai karauna -- bayakataitaba thaaakatae habae
4. **Backstory** laikhauna -- galapa thaaakatae habae
5. **Abilities** nairadhaaarana karauna -- kassamataaa thaaakatae habae
6. **Movement** paraikalapanaaa karauna -- chalaachala thaaakatae habae
7. **Animation** nairadhaaarana karauna -- ayaaanaimaeshana thaaakatae habae
8. **Clothing** daijaaaina karauna -- paoshaaaka thaaakatae habae
9. **Weapons** nairadhaaarana karauna -- asatara thaaakatae habae
10. **VFX** paraikalapanaaa karauna -- iphaekata thaaakatae habae
11. **Color Psychology** bayabahaaara karauna -- ranga daiyae anaubhauutai jaaagaiyae taulauna
12. **Character Design Sheet** taairai karauna -- dakaumaenataeshana samapauurana karauna

manae raaakhauna:

"aikatai bhaaalao Character halao yae kayaaaraekataaara khaelaoyaaadaera manae samaritai taairai karae khaelaoyaaada saei kayaaaraekataaarakae bhaulabashataau bhaulae yaaaya naaa"

anaushailana (Exercises)

anaushailana 1: Character Concept

aikatai natauna kayaaaraekataaaraera Concept taairai karauna

anaushailana 2: Silhouette Design

3tai bhainana kayaaaraekataaaraera Silhouette aakauna

anaushailana 3: Personality

aikatai kayaaaraekataaaraera Personality Traits, Flaws, Goals, Fears nairadhaaarana karauna

anaushailana 4: Backstory

aikatai kayaaaraekataaaraera Backstory laikhauna

anaushailana 5: Color Palette

aikatai kayaaaraekataaaraera Color Palette taairai karauna

anaushailana 6: Character Design Sheet

aikatai samapauurana Character Design Sheet taairai karauna

anaushailana 7: Turnaround

aikatai kayaaaraekataaaraera Front, Side, Back view aakauna

anaushaiilana 1: Expressions

aikatai kayaaaraekataaaraera 5tai bhainana Expression aakauna

****parabarataii adhayaaaya: [Chapter 17: ENEMY DESIGN](17_enemy_design.md)****

PART 17

adhayaaaya 17: ENEMY DESIGN -- shatarau daijaaaina

adhayaaaya 17: ENEMY DESIGN -- shatarau daijaaaina

shaekhaaara udadaeshaya (Learning Goal)

aii adhayaaaya shaessae aapanai jaaanabaena:

- Enemy Design Framework
- 10+ Enemy Archetypes with detailed descriptions
- Enemy Behavior, AI, Animation, Sound
- Counter-strategies
- Practical examples from real games

17.1 Enemy Design Framework (shatarau daijaaaina kaaathaaamao)

****Enemy Design**** halao gaemaera shatarau kayaaaraekataaaradaera daijaaaina karaaara parakaraiyaaa aikatai bhaaalao shatarau khaelaoyaaadake chayaaalaenyaja daeya kainatau asamabhaha kathaina haya naaa

****Enemy Design Framework:****

```

      ENEMY DESIGN FRAMEWORK
ROLE   > MOVEMENT > ATTACK
(bhauumaikaaa)   (gatai)   (aakaramana)
DEFENSE > WEAKNESS > STRENGTH
(paratairaodha)   (daurabalataaa)   (shakatai)
      AI   >ANIMATION > SOUND
(karitaraima baudadhai)   (ayaaanaimaeshana)   (shabada)
      REWARD (paurasakaaara)
```

Enemy Design aira upaaadaaanasamauha:

17.2 Enemy Archetypes (shatarau aarakaitaaaipa)

1. Grunt (phauta saainaika)

****baibarana:**** Grunt halao sabachaeyae saaadhaaarana aiba sakhayaaadhaika shatarau airaaa saaadhaaaranata daurabala kainatau anaeka thaaakae

****Behavior (aacharana):****

- sarala pathae haaatae
- khaelaoyaaadakaе daekhalae aigaiyae aasae
- aikataaanae aakaramana karae
- samasayaaa samaaadhaaanae dakassa naya

****Counter-strategy (paratairaodha kaaushala):****

- sahajaei maaaraaa yaaaya
- aikasaaathae anaeka thaaakalae sataraka hauna
- gaolaaakaaara aakaramana bayabahaaara karauna

****Example (udaaaharana):****

- Goomba (Super Mario)
- Zombie (Resident Evil)
- Basic Soldier (Call of Duty)

Grunt Profile:

Name: Grunt (phauta saainaika)
Health: Low (10-20)
Attack: Low (5-10)
Defense: Low (1-5)
Speed: Medium
AI: Simple (paratayakassa)
Reward: Low XP, Coins
Difficulty: *****

2. Brute (shakataishaaalaii)

****baibarana:**** Brute halao shakataishaaalaii aiba bhayakara shatarau airaaa dhaiira kainatau parataitai aakaramana shakataishaaalaii

****Behavior (aacharana):****

- dhaiirae haaatae
- shakataishaaalaii aakaramana karae
- dhaairaya dhaerae khaelaoyaaadaera apaekassaaa karae
- bhayakara shabada karae

****Counter-strategy (paratairaodha kaaushala):****

- darauta chalauna
- paechhanae thaekae aakaramana karauna
- taaara aakaramana aidaiyae chalauna

****Example (udaaaharana):****

- Bowser (Super Mario)
- Ganondorf (Zelda)
- Hulk (Marvel Games)

Brute Profile:

Name: Brute (shakataishaaalali)
 Health: High (100-200)
 Attack: Very High (50-100)
 Defense: High (30-50)
 Speed: Low
 AI: Simple (shakatai bayabahaaara)
 Reward: Medium XP, Rare Item
 Difficulty: *****

3. Sniper (sanaaaipaaara)

****baibarana:**** Sniper halao dauura thaekae aakaramana karaaa shatarau airaaa saaadhhaaranata uchau jaaayagaaaya thaaakae

****Behavior (aacharana):****

- dauurae thaaakae
- laejaaara baaa naishaaanaaa daekhaaaya
- sathaika samayae aakaramana karae
- khaelaoyaaada aigaiyae ailae paaalaiyae yaaaya

****Counter-strategy (paratairaodha kaaushala):****

- darauta aadaaala khaujauna
- laejaaara daekhae aidaiyae chalauna
- darauta aigaiyae yaaana

****Example (udaaaharana):****

- Sniper Wolf (Metal Gear Solid)
- Widowmaker (Overwatch)
- Hunter (Destiny)

Sniper Profile:

Name: Sniper (sanaaaipaaara)
 Health: Low (20-30)
 Attack: Very High (80-120)
 Defense: Low (5-10)
 Speed: Low
 AI: Strategic (naishaaanaaa karae)
 Reward: Medium XP, Ammo
 Difficulty: *****

4. Scout (sakaaauta)

****baibarana:**** Scout halao darauta aiba sataraka shatarau airaaa khaelaoyaaadakaе khaujaе baera karae aiba anaya shataraudaera sataraka karae

****Behavior (aacharana):****

- khauba darauta chalaachala karae
- khaelaoyaaadaka daekhalae paaalaiyae yaaaya
- anaya shataudaera sataraka karae
- daurabala kainatau baipadajanaka

****Counter-strategy (paratairaodha kaaushala):****

- parathamae maaarauna
- darauta shaessa karauna
- taaakae paaalaaatae daibaena naaa

****Example (udaaaharana):****

- Crawler (Gears of War)
- Scout (Team Fortress 2)
- Stalker (Warframe)

Scout Profile:

Name: Scout (sakaaauta)
 Health: Low (10-15)
 Attack: Low (5-10)
 Defense: Very Low (1-3)
 Speed: Very High
 AI: Alert (sataraka)
 Reward: Low XP, Intel
 Difficulty: *****

5. Tank (tayaaaka)

****baibarana:**** Tank halao atayanata shakataishaaalaa aiba bhaaarai shatarau airaaa dhaira kainatau paraaaya atala

****Behavior (aacharana):****

- khauba dhairae chalaachala karae
- khauba baeshai kassatai sahaya karatae paaarae
- shakataishaaalaa aakaramana karae
- paraaayai dhaaala bayabahaaara karae

****Counter-strategy (paratairaodha kaaushala):****

- dhairaya dharauna
- daurabalataaa khaujauna
- paechhane thaekae aakaramana karauna

****Example (udaaaharana):****

- Reinhardt (Overwatch)
- Heavy (Team Fortress 2)
- Brute (Halo)

Tank Profile:

Name: Tank (tayaaaka)
 Health: Very High (500+)

Attack: High (40-60)
 Defense: Very High (100+)
 Speed: Very Low
 AI: Aggressive (aakaramanaaataamaka)
 Reward: High XP, Rare Item
 Difficulty: *****

6. Healer (hailaaara)

****baibarana:**** Healer halao anaya shatarodaera saasatha karaaa shatarau airaaa saaadhaaaranata daurabala kainatau baipadajanaka

****Behavior (aacharana):****

- anaya shatarodaera paechhane thaaakae
- naiyamaita saasatha karaara chaessataaa karae
- naijae kama aakaramana karae
- anaya shatarodaera saathae samaparaka sathaaapana karae

****Counter-strategy (paratairoodha kaaushala):****

- parathamae maaarauna
- anaya shatarodaera thaekae aalaaadaaa karauna
- darauta shaessa karauna

****Example (udaaaharana):****

- Medic (Team Fortress 2)
- Mercy (Overwatch)
- Priest (World of Warcraft)

Healer Profile:

Name: Healer (hailaaara)
 Health: Low (20-30)
 Attack: Very Low (2-5)
 Defense: Low (5-10)
 Speed: Medium
 AI: Supportive (sahaaayaka)
 Reward: Medium XP, Heal Item
 Difficulty: *****

7. Swarm (paraanaanii jhaaaka)

****baibarana:**** Swarm halao chhaota chhaota shatarodaera jhaaaka airaaa sakhaayaaya shakataishaaalaa

****Behavior (aacharana):****

- aikasaaathae aakaramana karae
- khauba darauta chalaachala karae
- aikatai maaaralae aaraau aasae
- gaolaaakaaara aakaramanae kama kassataigarasata

****Counter-strategy (paratairoodha kaaushala):****

- gaolaaakaaara aakaramana bayabahaaara karauna
- ailaaakaaa aakaramana bayabahaaara karauna
- jabaaalaaanai baaa baisaphaoraka bayabahaaara karauna

****Example (udaaaharana):****

- Flood (Halo)
- Necromorphs (Dead Space)
- Zombies (Left 4 Dead)

Swarm Profile:

Name: Swarm (paraaanaii jhaaaka)
 Health: Very Low (1-5)
 Attack: Low (3-8)
 Defense: Very Low (0-1)
 Speed: High
 AI: Swarm (jhaaaka aacharana)
 Reward: Low XP per unit
 Difficulty: *****

8. Boss (basa)

****baibarana:**** Boss halao aikatai laebhaela baaa gaemaera sabachaeyae shakataishaaalarii shatarau airaaa saaadhaaaranata bada aiba bhayakara

****Behavior (aacharana):****

- anaeka dharanaera aakaramana karae
- parayaaayabhaitataika ladaai
- baishaessa kaaushala parayaojana
- galapae gaurautabapauurana

****Counter-strategy (paratairaodha kaaushala):****

- payaaataaarana shaikhauna
- dhaairaya dharauna
- sauyaogaera apaekassaaa karauna

****Example (udaaaharana):****

- Ganondorf (Zelda)
- Sephiroth (Final Fantasy)
- Bowser (Super Mario)

Boss Profile:

Name: Boss (basa)
 Health: Very High (1000+)
 Attack: Very High (100+)
 Defense: High (50-100)
 Speed: Variable
 AI: Complex (jataila)
 Reward: Very High XP, Item
 Difficulty: *****

9. Stealth (gaopana)

****baibarana:**** Stealth shatarau halao adarishaya baaa laukaiyae thaaakaaa shatarau airaaa hathaaa aakaramana karae

****Behavior (aacharana):****

- laukaiyae thaaakae
- khaelaoyaaada ajanyaaana thaaakalae aakaramana karae
- darauta paaalaiyae yaaaya
- chaihana raaakhae naaa

****Counter-strategy (paratairaodha kaaushala):****

- sataraka thaaakauna
- shabada shaunauna
- parataiphalana daekhauna

****Example (udaaaharana):****

- Hunter (Monster Hunter)
- Predator (Predator Games)
- Assassin (Assassin's Creed)

Stealth Profile:

Name: Stealth (gaopana)
Health: Medium (30-50)
Attack: High (40-70)
Defense: Low (5-10)
Speed: High
AI: Sneaky (gaopana)
Reward: Medium XP, Stealth
Difficulty: *****

10. Ranged (dauurapaaalalaaa)

****baibarana:**** Ranged shatarau halao dauura thaekae aakaramana karaaa shatarau airaaa saaadhhaaranata dhanauka, barashaaa, baaa yaaadau bayabahaaara karae

****Behavior (aacharana):****

- dauurae thaaakae
- naiyamaita aakaramana karae
- khaelaoyaaada kaaachhae ailae paaalaiyae yaaaya
- aadaaala bayabahaaara karae

****Counter-strategy (paratairaodha kaaushala):****

- darauta aigaiyae yaaana
- aadaaala bayabahaaara karauna
- taaakae paaalaaatae daibaena naaa

****Example (udaaaharana):****

- Archer (Clash of Clans)
- Trooper (Star Wars)
- Bandit (Borderlands)

Ranged Profile:

Name: Ranged (dauurapaaalalaaa)
 Health: Low-Medium (20-40)
 Attack: Medium (20-40)
 Defense: Low (5-15)
 Speed: Medium
 AI: Tactical (kaaushalagata)
 Reward: Medium XP, Ammo
 Difficulty: *****

17.3 Additional Enemy Types (atairaikata shatarau dharana)

11. Summoner (daaakau)

****baibarana:**** Summoner halao anaya shatarau daekae aanaaa shatarau airaaa naijae kama ladae kainatau jhaaaka taairai karae

****Behavior:****

- dauurae thaaakae
- naiyamaita natauna shatarau daekae aanae
- naijae kama aakaramana karae
- baishaessa saurakassaaa thaaakae

****Counter-strategy:****

- parathamae Summoner maaarauna
- daaakaaa shataraudaera aidaiyae chalauna
- darauta shaessa karauna

****Example:****

- Witch (Left 4 Dead)
- Necromancer (Diablo)
- Summoner (Final Fantasy)

12. Bomber (baomaaabaaaja)

****baibarana:**** Bomber halao baisaphaoraka bayabahaaara kaaaa shatarau airaaa aatamaghaaataii aakaramana karatae paaarae

****Behavior:****

- darauta khaelaoyaaadaera daikae chhautae yaaaya
- baisaphaoraka naikassaepa karae
- naijaeau baisaphaoraita hatae paaarae
- ailaaakaaa kassatai karae

****Counter-strategy:****

- dauurae thaaakauna
- naikhautabhaaabae aakaramana karauna
- समयमताओ पाालायाे याााना

****Example:****

- Creeper (Minecraft)
- Bomb Barrel (Uncharted)
- Kamikaze (Serious Sam)

13. Shapeshifter (rauupaaanatarakaaaraii)

****baibarana:**** Shapeshifter halao rauupa paraibaratana karaaa shatarau airaaa bhainana bhainana aakaramana karatae paaarae

****Behavior:****

- rauupa paraibaratana karae
- bhainana kassamataaa bayabahaaara karae
- anaumaaana karaaa kathaina
- paraibaratanaera samaya daurabala

****Counter-strategy:****

- rauupa paraibaratanaera samaya aakaramana karauna
- payaaataaarana shaikhauna
- dhaairaya dharauna

****Example:****

- Ditto (Pokemon)
- T-1000 (Terminator)
- Mimic (Dark Souls)

14. Turret (taaaraeta)

****baibarana:**** Turret halao sathaira aakaramana karaaa yanatara airaaa saaadhaaaranata aikatai jaaayagaaaya thaaakae

****Behavior:****

- aikatai jaaayagaaaya thaaakae
- sabayakaraiyabhaaabae aakaramana karae
- nairadaissata paraisaiimaaa aachhae
- dhabasa karaaa yaaaya

****Counter-strategy:****

- aadaaala bayabahaaara karauna
- nairadaissata kaona thaekae aakaramana karauna
- dhabasa karauna

****Example:****

- Turret (Portal)
- Sentry Gun (Team Fortress 2)
- Auto Turret (Fallout)

17.4 Enemy Behavior Patterns (shatarau aacharana payaaataaarana)

****Common Behavior Patterns:****

1. Patrol (paraharaii):
**
UP
DOWN
**
nairadaissata pathae chalaachala karae
2. Chase (anausarana):
* -> -> -> *
khaelaoyaaadae daekhae chhatae yaaaya
3. Ambush (aakaramana):
*
DOWN
* -> * -> *
DOWN
*
laukaiyaethaakaethathaaa aakaramana karae
4. Flank (paaarashaba aakaramana):
* -> -> -> *
* <- <- <- *
aikapaaashae aakaramana, anayapaaashae
paaarashaba aakaramana
5. Retreat (pashachaaadapasarana):
* -> -> -> *
DOWN
* <- <-
khaelaoyaaada kaaachhae ailae paaalaaaya

17.5 Enemy AI Systems (shatarau AI saisataema)

****AI Types:****

****State Machine Example:****

Enemy State Machine:

```
IDLE > PATROL > CHASE
(naiiraba) < (paraharaii) <(anausarana)
                ATTACK                RETREAT
                (aakaramana) <(pashachaaadapasarana)
                DEAD
> (maritayau)
```

17.6 Enemy Animation (shatarau ayaaanaimaeshana)

****Common Enemy Animations:****

17.7 Enemy Sound Design (shatarau shabada daijaaaina)

****Sound Types:****

17.8 Enemy Reward System (shatarau paurasakaaara saisataema)

****Reward Types:****

17.9 Enemy Design Tips (shatarau daijaaaina paraaamarasha)

taipa 1: Variety raaakhauna

baibhainana dharanaera shatarau raaakhauna

taipa 2: Fairness

shatarau kathaina kainatau asamabhaba naaa

taipa 2: Counter-strategy

parataitai shataraura aikatai Counter-strategy thaaakauka

taipa 3: Visual Clarity

shatarau sapassata daekhaaa yaaaya

taipa 4: Audio Cues

shataraura shabada sataraka karae

taipa 5: Reward

shatarau maaaralae paurasakaaara paaaauyaaa yaaaya

taipa 6: Pacing

shataraura sakhayaaa aiba shakatai saussama

taipa 7: Storytelling

shatarau galapaera asha

taipa 8: memorable

shatarau samaranaiiya haoka

taipa 9: Playtest

shatarau Playtest karauna

taipa 10: Iteration

baaarabaaara paraibaratana karauna

17.10 Summary (saaarasakassaepa)

Enemy Design halao gaemaera chayaaalaenyaja taairai karaaara asha aikatai bhaaalao Enemy Designer hatae halae:

1. ****Enemy Design Framework**** baujhauna -- Role, Movement, Attack, Defense, Weakness, Strength, AI, Animation, Sound, Reward
2. ****Enemy Archetypes**** shaikhauna -- Grunt, Brute, Sniper, Scout, Tank, Healer, Swarm, Boss, Stealth, Ranged
3. ****Behavior Patterns**** shaikhauna -- Patrol, Chase, Ambush, Flank, Retreat
4. ****AI Systems**** shaikhauna -- State Machine, Behavior Tree, GOAP
5. ****Animation**** shaikhauna -- baibhainana abasathaaara ayaaanaimaeshana
6. ****Sound Design**** shaikhauna -- shabada daiyae satarakataaa
7. ****Reward System**** shaikhauna -- paurasakaaara nairadhaaarana
8. ****Practice**** karauna -- baaarabaaara shatarau daijaaaina karauna
9. ****Playtest**** karauna -- anaya khaelaoyaaadadaera khaelatae daina
10. ****Iterate**** karauna -- baaarabaaara paraibaratana karauna

****manae raaakhauna:****

"aikatai bhaaalao shatarau halao yae shatarau khaelaoyaaadake chayaaalaenyaja daeya kainatau nayaaayaya khaelaoyaaada haaaraiyae gaelae bairakata haya naaa, bara aabaaara chaessataaa karatae chaaaya"

anaushailana (Exercises)

anaushailana 1: Enemy Concept

aikatai natauna shataraura Concept taairai karauna

anaushailana 2: Behavior Pattern

aikatai shataraura Behavior Pattern nairadhaaarana karauna

anaushailana 3: Counter-strategy

aikatai shataraura Counter-strategy laikhauna

anaushailana 4: AI Design

aikatai shataraura AI State Machine daijaaaina karauna

anaushaiilana 5: Animation States

aikatai shataura Animation States nairadhaaarana karauna

anaushaiilana 6: Sound Design

aikatai shataura Sound Types nairadhaaarana karauna

anaushaiilana 7: Reward System

aikatai shataura Reward System nairadhaaarana karauna

anaushaiilana 1: Complete Enemy

aikatai samapauurana Enemy Design Sheet taairai karauna

****parabarataii adhayaaaya: [Chapter 18: GAME ART PIPELINE](18_art_pipeline.md)****

PART 18

adhayaaaya 18: GAME ART PIPELINE -- gaema aarata paaaipalaaaina

adhayaaaya 18: GAME ART PIPELINE -- gaema aarata paaaipalaaaina

shaekhaaara udadaeshaya (Learning Goal)

aii adhayaaaya shaessae aapanai jaaanabaena:

- Complete Art Pipeline: Concept Art -> Reference -> Modeling -> Texturing -> Rigging -> Animation -> VFX -> Lighting -> Optimization -> Export -> Engine Integration
- Separate 2D and 3D workflows
- Tool recommendations for each step
- File format specifications
- Optimization tips

18.1 Game Art Pipeline kaii? (What is Game Art Pipeline?)

Game Art Pipeline halao gaemaera shailapakarama taairai karaaara samapauurana parakaraiyaaa aitai aikatai dhaaapae dhaaapae parakaraiyaaa yaekhaanae parataitai dhaaapa parabaratii dhaaapaera upara nairabhara karae

Pipeline Diagram:

Complete Art Pipeline:

```
CONCEPT >REFERENCE >MODELING >TEXTURING
ART          GATHER      (madaelai)      (taekasachaaara)
EXPORT <OPTIMIZE <LIGHTING < RIGGING
(aikasapaorata) (apataimaaaija) (aalao)      (raigai)
ENGINE <ANIMATION < VFX
INTEGRATION (ayaaanaimaeshana) (bhaijauyaaala)
```

18.2 2D Art Pipeline (2D aarata paaaipalaaaina)

2D Art Pipeline halao dabaimaaataraiika shailapakarama taairai karaaara parakaraiyaaa

2D Pipeline Steps:

2D Art Pipeline:

```

CONCEPT > SKETCH > LINEART > COLORING
ART      (khasadaaaa) (laaaina) (rangaina)
EXPORT < OPTIMIZE < EFFECTS < SHADING
(aikasapaorata) (apataimaaaaja) (iphaekata) (shaedai)

```

2D Art Tools:

2D File Formats:

18.3 3D Art Pipeline (3D aarata paaaipalaaaina)

3D Art Pipeline halao taraimaaataraiika shailapakarama taairai karaaara parakaraiyaaa

****3D Pipeline Steps:****

3D Art Pipeline:

```

CONCEPT > REFERENCE > MODELING > UV UNWRAP
ART      GATHERING (madaelai) (iubhai)
EXPORT < OPTIMIZE < LIGHTING < TEXTURING
(aikasapaorata) (apataimaaaaja) (aalao) (taekasachaaara)
ENGINE < ANIMATION < RIGGING
INTEGRATION (ayaaanaimaeshana) (raigai)

```

3D Art Tools:

3D File Formats:

18.4 Step 1: Concept Art (dhaaapa 1: kanasaepata aarata)

****Concept Art**** halao gaemaera darishaya, kayaaaraekataaara, aiba basataura paraaathamaiika chaitara

****Concept Art Types:****

****Concept Art Process:****

- **Research**** -- raephaaaraenasa sagaraha
- **Thumbnails**** -- chhaota chhaota khasadaaa
- **Sketches**** -- bada khasadaaa
- **Details**** -- baisataaaraita tathaya
- **Color**** -- ranga yaoga
- **Final**** -- chauudaaanata sasakarana

18.5 Step 2: Reference Gathering (dhaaapa 2: raephaaaraenasa

sagaraha)

****Reference**** halao parakarita jagataera chhabai baaa udaaaharana yaaa aamaraaa anausarana karai

****Reference Sources:****

****Reference Organization:****

Reference Folder Structure:

```
References/  
  Characters/  
    Hero/  
    Villain/  
    NPC/  
  Environment/  
    Forest/  
    City/  
    Dungeon/  
  Props/  
    Weapons/  
    Armor/  
    Items/  
  UI/  
    Buttons/  
    Icons/  
    Menus/  
  Mood/  
    Lighting/  
    Weather/
```

18.6 Step 3: Modeling (dhaaapa 3: madaelai)

****Modeling**** halao 3D madaela taairai karaaara parakaraiyaaa

****Modeling Types:****

****Modeling Workflow:****

Modeling Workflow:

1. Base Mesh (mauula madaela):
 - Simple
 - Shape
2. Detail Mesh (baisataaaraita madaela):
 - Detail
3. Low Poly (kama palaigana):
4. High Poly (baeshai palaigana):

****Modeling Tips:****

- sabasamaya Low Poly daiyae shaurau karauna
- Topology paraissakaaara raaakhauna

- Edge Flow sathaika raaakhauna
 - UV Unwrap karauna
 - Scale sathaika raaakhauna
-

18.7 Step 4: UV Unwrap (dhaaapa 4: iubhai aanarayaaapa)

****UV Unwrap**** halao 3D madaelakae 2D parissathae khaulae phaelaaara parakaraiyaaa aitai taekasachaaara laaagaaanaora janaya parayaojana

****UV Unwrap Process:****

UV Unwrap:

3D Model (taraimaaataraika madaela):

2D UV Map (dabaimaaataraika iubhai mayaaapa):

****UV Unwrap Tips:****

- Seams kama raaakhauna
 - UV Space samaaana bayabahaaara karauna
 - Overlap aidaiyae chalauna
 - Texel Density samaaana raaakhauna
-

18.8 Step 5: Texturing (dhaaapa 5: taekasachaaarai)

****Texturing**** halao 3D madaelae ranga aiba parissatha yaoga karaaara parakaraiyaaa

****PBR Texturing Maps:****

****Texturing Process:****

Texturing Process:

1. Base Color (mauula ranga):
Red

2. Normal Map (naramaaala mayaaapa):

3. Roughness Map (raaaphanaesa mayaaapa):

4. Metallic Map (maetaaalaika mayaaapa):

****Texturing Tools:****

- Substance Painter -- Industry Standard
 - Quixel Mixer -- Free
 - Mari -- High-End
 - Photoshop -- 2D Textures
-

18.9 Step 6: Rigging (dhaaapa 6: raigai)

****Rigging**** halao 3D madaelae haaada (Bone) yaoga karaaara parakaraiyaaa aitai ayaaanaimaeshanaera janaya parayaojana

****Rigging Components:****

****Rigging Process:****

Rigging Process:

1. Skeleton (kangakaaala):
 - o <- Head
 - <- Spine
 - <- Arms
 - <- Legs
2. Controllers (naiyanataraka):
 - <- Head Controller
 - <- Spine Controller
 - <- Hand Controllers
 - <- Foot Controllers

****Rigging Tips:****

- Hierarchy paraissakaaara raaakhauna
- Constraints bayabahaaara karauna
- Testing karauna
- Documentation karauna

18.10 Step 7: Animation (dhaaapa 7: ayaaanaimaeshana)

****Animation**** halao 3D madaelakae chalaachala karaaanaora parakaraiyaaa

****Animation Types:****

****Animation States:****

Animation States:

```

IDLE   >   WALK
(sathaira) < (haaataaa)
              RUN
              (daaudaaanao)
              JUMP
              (laaaphaaanao)
              ATTACK
              (aakaramana)
              HURT
              (aahata)
              DEATH
          > (maritayau)

```

18.11 Step 8: VFX (dhaaapa 8: bhaijauyaaala iphaekata)

****VFX**** halao bhaijauyaaala iphaekata yaaa gaemaera darishayakae aaraau saunadara karae

****VFX Types:****

****VFX Tools:****

- Houdini -- Industry Standard
 - Blender -- Open Source
 - Unity VFX Graph
 - Unreal Niagara
-

18.12 Step 9: Lighting (dhaaapa 9: aalaoka bayabasathaaa)

****Lighting**** halao gaemaera darishayae aalao yaoga karaaara parakaraiyaaa

****Light Types:****

****Lighting Tips:****

- Three-Point Lighting bayabahaaara karauna
 - Color Temperature baujhauna
 - Shadow Quality nairadhaaarana karauna
 - Mood taairai karauna
-

18.13 Step 10: Optimization (dhaaapa 10: apataimaaaiaeshana)

****Optimization**** halao gaemaera paaarapharamayaaanasa unanata karaaara parakaraiyaaa

****Optimization Tips:****

****Poly Count Guidelines:****

****Texture Size Guidelines:****

18.14 Step 11: Export (dhaaapa 11: aikasapaorata)

****Export**** halao madaelakae gaema inyajainaera janaya parasatauta karaaara parakaraiyaaa

****Export Settings:****

****Export Checklist:****

- ☐ Scale sathaika
- ☐ Rotation sathaika
- ☐ UV Unwrap sathaika
- ☐ Texture Maps sathaika

- [] Bones sathaika
- [] Animation sathaika
- [] LOD sathaika

18.15 Step 12: Engine Integration (dhaaapa 12: inyajaina inataigaraeshana)

****Engine Integration**** halao madaelakae gaema inyajainae yaoga karaaara parakaraiyaaa

****Engine Import Settings:****

****Unity:****

- Scale Factor: 1.0
- Mesh Compression: Off
- Read/Write: Enabled
- Generate Colliders: Yes
- Material Import: Yes

****Unreal Engine:****

- Import Mesh: Yes
- Import Materials: Yes
- Import Textures: Yes
- Generate Missing Collision: Yes
- Auto Generate LODs: Yes

****Engine Integration Tips:****

- Material sathaika saeta karauna
- Collision sathaika saeta karauna
- LOD saeta karauna
- Animation saeta karauna
- VFX saeta karauna

18.16 Art Pipeline Checklist

Art Pipeline Checklist:

- Concept Art
 - Character Design
 - Environment Design
 - Prop Design
 - UI Design
- Reference Gathering
 - Photo Reference
 - Art Reference
 - Video Reference
- Modeling
 - Base Mesh

- Detail Mesh
 - Low Poly
 - High Poly
- UV Unwrap
 - Seams
 - UV Space
 - Texel Density
- Texturing
 - Albedo Map
 - Normal Map
 - Roughness Map
 - Metallic Map
 - AO Map
- Rigging
 - Skeleton
 - Controllers
 - Constraints
 - IK/FK
- Animation
 - Idle
 - Walk
 - Run
 - Jump
 - Attack
 - Hurt
 - Death
- VFX
 - Particles
 - Trails
 - Glow
 - Distortion
- Lighting
 - Directional Light
 - Point Lights
 - Spot Lights
 - Ambient Light
- Optimization
 - Poly Count
 - Texture Size
 - Draw Calls
 - LOD
- Export
 - Format
 - Scale
 - Rotation
 - Textures
- Engine Integration
 - Import
 - Material
 - Collision
 - Animation

18.17 Summary (saaarasakassaepa)

Game Art Pipeline halao gaemaera shailapakarama taairai karaaara samapauurana parakaraiyaaa aikatai bhaaalao Game Artist hatae halae:

1. **Concept Art** shaikhauna -- mauula dhaaaranaaaa taairai karauna
2. **Reference** sagaraha karauna -- raephaaaraenasa jamaaa karauna
3. **Modeling** shaikhauna -- 3D madaela taairai karauna
4. **UV Unwrap** shaikhauna -- iubhai aanarayaaapa karauna
5. **Texturing** shaikhauna -- taekasachaaara yaoga karauna
6. **Rigging** shaikhauna -- haaada yaoga karauna
7. **Animation** shaikhauna -- chalaachala taairai karauna
8. **VFX** shaikhauna -- bhajauyaaala iphaekata yaoga karauna
9. **Lighting** shaikhauna -- aalao yaoga karauna
10. **Optimization** shaikhauna -- paaarapharamayaaanasa unanata karauna
11. **Export** shaikhauna -- aikasapaorata karauna
12. **Engine Integration** shaikhauna -- inyajinae yaoga karauna

manae raaakhauna:

"Game Art Pipeline halao aikatai daiiragha parakaraiyaaa parataitai dhaaapa gaurautabapauurana aikatai dhaaapae bhaula halae paura parakaraiyaaa bayaaahata haya"

anaushaailana (Exercises)

anaushaailana 1: Concept Art

aikatai kayaaaraekataaaraera Concept Art aakauna

anaushaailana 2: Reference

aikatai kayaaaraekataaaraera janaya Reference sagaraha karauna

anaushaailana 3: Modeling

aikatai saaadhhaarana 3D madaela taairai karauna

anaushaailana 4: UV Unwrap

aikatai 3D madaelaera UV Unwrap karauna

anaushaailana 5: Texturing

aikatai 3D madaelae Texturing karauna

anaushaailana 6: Rigging

aikatai 3D madaelae Rigging karauna

anaushaailana 7: Animation

aikatai saaadhhaarana Animation taairai karauna

anaushaiilana 8: Complete Pipeline

aikatai samapauurana Art Pipeline samapanana karauna

****parabarataii adhayaaaya: [Chapter 19: UNITY DEVELOPMENT](19_unity.md)****

PART 19

adhayaaaya 19: UNITY DEVELOPMENT -- iunaitai daebhaelapamaenata

adhayaaaya 19: UNITY DEVELOPMENT -- iunaitai daebhaelapamaenata

shaekhaara udadaeshaya (Learning Goal)

aii adhayaaaya shaessae aapanai jaaanabaena:

- Complete Unity Development Roadmap
 - Project Setup, Scenes, GameObjects, Prefabs, Scripts
 - Input System, Physics, Animation, UI, Audio
 - Particles, Lighting, Post Processing, Save System
 - Optimization, Build
 - Practical C# code examples
 - Unity-specific tips and best practices
-

19.1 Unity Development Roadmap

Unity Development Roadmap:

Level 1: Fundamentals

Project Setup
Editor Interface
GameObjects
Components
Basic Scripts

Level 2: Core Systems

Input System
Physics
Animation
UI System
Audio System

Level 3: Advanced

Particles
Lighting
Post Processing
Save System

AI Navigation

Level 4: Optimization

Profiling

LOD

Culling

Batching

Memory Management

Level 5: Build & Deploy

Build Settings

Platform Optimization

Testing

Release

19.2 Project Setup (parajaekata saetaaapa)

Unity Install aiba Project taairai:

Step 1: Unity Hub Install

- unity.com thaekae Unity Hub Download karauna
- Unity Editor Install karauna
- LTS Version bayabahaaara karauna

Step 2: New Project

- Unity Hub -> New Project
- Template: 2D baaa 3D
- Project Name: daina
- Location: nairadhhaarana karauna
- Create: kalaika karauna

Step 3: Editor Setup

- Layout: Window -> Layouts -> Default
- Scene: File -> New Scene
- Save: File -> Save Scene

Editor Interface:

Unity Editor Layout:

	Menu Bar	
Hierarchy	Scene View	Inspector
Panel		Panel
Project		Console
Panel		Panel
	Game View	

19.3 Scenes (saina)

Scene halao gaemaera aikatai parithaka paraibaesha parataitai Level aikatai Scene

****Scene Management:****

```
// Scene Management Example
using UnityEngine;
```



```
using UnityEngine.SceneManagement;

public class SceneManager : MonoBehaviour
{
    // natauna Scene laoda karauna
    public void LoadScene(string sceneName)
    {
        UnityEngine.SceneManagement.SceneManager.LoadScene(sceneName);
    }

    // parabarataii Scene laoda karauna
    public void LoadNextScene()
    {
        int nextSceneIndex =
            UnityEngine.SceneManagement.SceneManager.GetActiveScene().buildIndex + 1;
        UnityEngine.SceneManagement.SceneManager.LoadScene(nextSceneIndex);
    }

    // sainaera naaama daiyae laoda karauna
    public void LoadSceneByName(string sceneName)
    {
        UnityEngine.SceneManagement.SceneManager.LoadScene(sceneName);
    }

    // saina railaoda karauna
    public void ReloadScene()
    {
        UnityEngine.SceneManagement.SceneManager.LoadScene(
            UnityEngine.SceneManagement.SceneManager.GetActiveScene().name);
    }

    // baratamaaana sainaera naaama paaana
    public string GetCurrentSceneName()
    {
        return UnityEngine.SceneManagement.SceneManager.GetActiveScene().name;
    }
}
```

19.4 GameObjects (gaema abajaekata)

GameObject halao Unity aira mauula nairamaaana upaaadaaana saba kaichhau aikatai GameObject

****GameObject Operations:****

```
// GameObject Operations
using UnityEngine;

public class GameObjectExample : MonoBehaviour
{
    void Start()
    {
        // natauna GameObject taairai karauna
        GameObject newObject = new GameObject("MyObject");

        // Component yaoga karauna
        newObject.AddComponent<Rigidbody>();
        newObject.AddComponent<BoxCollider>();

        // Tag saeta karauna
        newObject.tag = "Player";
    }
}
```

```

// Layer saeta karauna
newObject.layer = LayerMask.NameToLayer("Default");

// Position saeta karauna
newObject.transform.position = new Vector3(0, 1, 0);

// Rotation saeta karauna
newObject.transform.rotation = Quaternion.Euler(0, 45, 0);

// Scale saeta karauna
newObject.transform.localScale = new Vector3(2, 2, 2);

// Active/Deactive karauna
newObject.SetActive(false);
newObject.SetActive(true);

// dhabasa karauna
Destroy(newObject, 5f); // 5 saekaenada parae
}

void Update()
{
    // GameObject khaujauna
    GameObject player = GameObject.FindWithTag("Player");

    // saba GameObject khaujauna
    GameObject[] enemies = GameObject.FindGameObjectsWithTag("Enemy");

    // Tag daiyae khaujauna
    GameObject enemy = GameObject.FindGameObjectWithTag("Enemy");

    // Component khaujauna
    Rigidbody rb = GetComponent<Rigidbody>();
    Collider col = GetComponent<Collider>();
}
}

```

19.5 Prefabs (paraiphayaaaba)

Prefab halao aikatai reusable GameObject aitai aikabaaara taairai karae baaarabaaara bayabahaaara karaaa yaaaya

****Prefab Operations:****

```

// Prefab Operations
using UnityEngine;

public class PrefabExample : MonoBehaviour
{
    public GameObject bulletPrefab; // Inspector thaekae saeta karauna

    void Start()
    {
        // Prefab thaekae natauna abajaekata taairai karauna
        GameObject bullet = Instantiate(bulletPrefab, transform.position, transform.rotati

        // nairadaissata position ai taairai karauna
        GameObject bullet2 = Instantiate(bulletPrefab, new Vector3(0, 0, 0), Quaternion.id

        // parent saeta karauna
        bullet.transform.parent = transform;

        // parent null karauna
    }
}

```

```

        bullet.transform.parent = null;

        // 5 saekaenada parae dhabasa karauna
        Destroy(bullet, 5f);
    }

    // Prefab Variant taairai
    // Project Window -> Create -> Prefab Variant

    // Prefab Override
    // Inspector -> Override -> Apply All
    // Inspector -> Override -> Revert
}

```

19.6 Scripts (sakaraipata)

Script halao C# code yaaa GameObject aira aacharana naiyanatarana karae

****Basic Script Structure:****

```

// Basic Script Structure
using UnityEngine;

public class PlayerController : MonoBehaviour
{
    // Variables (paraibaratanayaogaya)
    public float speed = 5f;
    public int health = 100;
    public bool isAlive = true;

    // Private Variables
    private Rigidbody rb;
    private Animator animator;

    // Awake: abajaekata taairai halae
    void Awake()
    {
        Debug.Log("Awake called");
    }

    // Start: parathama pharaemaera aagae
    void Start()
    {
        rb = GetComponent<Rigidbody>();
        animator = GetComponent<Animator>();
        Debug.Log("Start called");
    }

    // Update: parataitai pharaemae
    void Update()
    {
        if (isAlive)
        {
            Move();
            Rotate();
        }
    }

    // FixedUpdate: naiyamaita samaya inataaarabhaaalae
    void FixedUpdate()
    {

```

```
// Physics calculations
}

// LateUpdate: Update aira parae
void LateUpdate()
{
    // Camera follow
}

// Methods
void Move()
{
    float horizontal = Input.GetAxis("Horizontal");
    float vertical = Input.GetAxis("Vertical");

    Vector3 movement = new Vector3(horizontal, 0, vertical);
    transform.Translate(movement * speed * Time.deltaTime);
}

void Rotate()
{
    float mouseX = Input.GetAxis("Mouse X");
    transform.Rotate(0, mouseX, 0);
}

// Collision Detection
void OnCollisionEnter(Collision collision)
{
    if (collision.gameObject.tag == "Enemy")
    {
        TakeDamage(10);
    }
}

// Trigger Detection
void OnTriggerEnter(Collider other)
{
    if (other.gameObject.tag == "HealthPickup")
    {
        Heal(25);
        Destroy(other.gameObject);
    }
}

// Public Methods
public void TakeDamage(int damage)
{
    health -= damage;
    if (health <= 0)
    {
        Die();
    }
}

public void Heal(int amount)
{
    health += amount;
    if (health > 100)
    {
        health = 100;
    }
}

void Die()
```

```
{
    isAlive = false;
    animator.SetTrigger("Death");
    // Game Over logic
}
}
```

19.7 Input System (inapauta saisataema)

****Legacy Input System:****

```
// Legacy Input System
using UnityEngine;

public class InputExample : MonoBehaviour
{
    void Update()
    {
        // Keyboard Input
        if (Input.GetKeyDown(KeyCode.Space))
        {
            Jump();
        }

        if (Input.GetKey(KeyCode.W))
        {
            MoveForward();
        }

        // Mouse Input
        if (Input.GetMouseButtonDown(0))
        {
            Shoot();
        }

        // Axis Input
        float horizontal = Input.GetAxis("Horizontal");
        float vertical = Input.GetAxis("Vertical");

        // GetAxisRaw (smooth naya)
        float h = Input.GetAxisRaw("Horizontal");
        float v = Input.GetAxisRaw("Vertical");

        // Touch Input
        if (Input.touchCount > 0)
        {
            Touch touch = Input.GetTouch(0);
            if (touch.phase == TouchPhase.Began)
            {
                // Touch started
            }
        }
    }

    void Jump()
    {
        Debug.Log("Jump!");
    }

    void MoveForward()
    {

```

```
        transform.Translate(Vector3.forward * Time.deltaTime);
    }

    void Shoot()
    {
        Debug.Log("Shoot!");
    }
}
```

****New Input System (Recommended):****

```
// New Input System
using UnityEngine;
using UnityEngine.InputSystem;

public class NewInputSystem : MonoBehaviour
{
    private PlayerInput playerInput;
    private InputAction moveAction;
    private InputAction jumpAction;
    private InputAction shootAction;

    void Awake()
    {
        playerInput = GetComponent<PlayerInput>();
        moveAction = playerInput.actions["Move"];
        jumpAction = playerInput.actions["Jump"];
        shootAction = playerInput.actions["Shoot"];
    }

    void OnEnable()
    {
        jumpAction.performed += OnJump;
        shootAction.performed += OnShoot;
    }

    void OnDisable()
    {
        jumpAction.performed -= OnJump;
        shootAction.performed -= OnShoot;
    }

    void Update()
    {
        Vector2 moveInput = moveAction.ReadValue<Vector2>();
        Move(moveInput);
    }

    void Move(Vector2 input)
    {
        Vector3 movement = new Vector3(input.x, 0, input.y);
        transform.Translate(movement * 5f * Time.deltaTime);
    }

    void OnJump(InputAction.CallbackContext context)
    {
        Debug.Log("Jump!");
    }

    void OnShoot(InputAction.CallbackContext context)
    {
        Debug.Log("Shoot!");
    }
}
```

19.8 Physics (padaaarathabaijanyaaana)

****Rigidbody Physics:****

```
// Rigidbody Physics
using UnityEngine;

public class PhysicsExample : MonoBehaviour
{
    private Rigidbody rb;

    void Start()
    {
        rb = GetComponent<Rigidbody>();

        // Rigidbody Settings
        rb.mass = 1f;
        rb.drag = 0f;
        rb.angularDrag = 0.05f;
        rb.useGravity = true;
        rb.isKinematic = false;
    }

    void FixedUpdate()
    {
        // Force Apply
        rb.AddForce(Vector3.forward * 10f);

        // Velocity Set
        rb.velocity = new Vector3(0, 5, 0);

        // Torque Apply
        rb.AddTorque(Vector3.up * 10f);

        // Move Position
        rb.MovePosition(transform.position + Vector3.forward * Time.deltaTime);

        // Move Rotation
        rb.MoveRotation(transform.rotation * Quaternion.Euler(0, 90, 0));
    }

    // Collision Events
    void OnCollisionEnter(Collision collision)
    {
        Debug.Log("Hit: " + collision.gameObject.name);

        // Contact Point
        ContactPoint contact = collision.contacts[0];
        Debug.Log("Point: " + contact.point);
        Debug.Log("Normal: " + contact.normal);

        // Impact Force
        float impactForce = collision.relativeVelocity.magnitude;
        Debug.Log("Force: " + impactForce);
    }

    void OnCollisionStay(Collision collision)
    {
        // Continuous collision
    }

    void OnCollisionExit(Collision collision)
    {
    }
}
```

```

        // Collision ended
    }

    // Trigger Events
    void OnTriggerEnter(Collider other)
    {
        Debug.Log("Trigger: " + other.gameObject.name);
    }

    void OnTriggerStay(Collider other)
    {
        // Continuous trigger
    }

    void OnTriggerExit(Collider other)
    {
        // Trigger ended
    }
}

```

****Raycasting:****

```

// Raycasting
using UnityEngine;

public class RaycastExample : MonoBehaviour
{
    public float range = 100f;

    void Update()
    {
        // Single Raycast
        Ray ray = new Ray(transform.position, transform.forward);
        RaycastHit hit;

        if (Physics.Raycast(ray, out hit, range))
        {
            Debug.Log("Hit: " + hit.collider.gameObject.name);
            Debug.Log("Distance: " + hit.distance);
            Debug.Log("Point: " + hit.point);

            // Color Change
            hit.collider.gameObject.GetComponent<Renderer>().material.color = Color.red;
        }

        // Raycast from Mouse
        Ray mouseRay = Camera.main.ScreenPointToRay(Input.mousePosition);
        RaycastHit mouseHit;

        if (Physics.Raycast(mouseRay, out mouseHit))
        {
            Debug.Log("Mouse hit: " + mouseHit.collider.gameObject.name);
        }

        // SphereCast
        if (Physics.SphereCast(transform.position, 0.5f, transform.forward, out hit, range))
        {
            Debug.Log("Sphere hit: " + hit.collider.gameObject.name);
        }

        // BoxCast
        if (Physics.BoxCast(transform.position, Vector3.one * 0.5f,
            transform.forward, out hit, transform.rotation, range))
        {

```



```

        Debug.Log("Box hit: " + hit.collider.gameObject.name);
    }

    // Linecast
    if (Physics.Linecast(transform.position, Vector3.forward * 10f, out hit))
    {
        Debug.Log("Line hit: " + hit.collider.gameObject.name);
    }

    // Overlap Sphere
    Collider[] colliders = Physics.OverlapSphere(transform.position, 5f);
    foreach (Collider col in colliders)
    {
        Debug.Log("Overlap: " + col.gameObject.name);
    }
}

// Debug Draw
void OnDrawGizmos()
{
    Gizmos.color = Color.red;
    Gizmos.DrawRay(transform.position, transform.forward * range);
}
}

```

19.9 Animation (ayaaanaimaeshana)

****Animator Controller:****

```

// Animator Controller
using UnityEngine;

public class AnimationExample : MonoBehaviour
{
    private Animator animator;

    void Start()
    {
        animator = GetComponent<Animator>();
    }

    void Update()
    {
        // Set Float
        float speed = Input.GetAxis("Vertical");
        animator.SetFloat("Speed", speed);

        // Set Bool
        bool isGrounded = CheckGrounded();
        animator.SetBool("IsGrounded", isGrounded);

        // Set Int
        int health = 100;
        animator.SetInteger("Health", health);

        // Set Trigger
        if (Input.GetKeyDown(KeyCode.Space))
        {
            animator.SetTrigger("Jump");
        }
    }
}

```

```

        if (Input.GetKeyDown(KeyCode.LeftShift))
        {
            animator.SetTrigger("Attack");
        }

        // Get Current State Info
        AnimatorStateInfo stateInfo = animator.GetCurrentAnimatorStateInfo(0);
        if (stateInfo.IsName("Run"))
        {
            Debug.Log("Running!");
        }

        // Play Animation
        animator.Play("Idle", 0, 0f);

        // Cross Fade
        animator.CrossFade("Run", 0.2f);
    }

    bool CheckGrounded()
    {
        return Physics.Raycast(transform.position, Vector3.down, 1.1f);
    }
}

```

****Animation Events:****

```

// Animation Events
using UnityEngine;

public class AnimationEvents : MonoBehaviour
{
    // Animation Event Methods
    public void OnFootstep()
    {
        Debug.Log("Footstep sound!");
    }

    public void OnAttackHit()
    {
        Debug.Log("Attack hit!");
    }

    public void OnWeaponSwing()
    {
        Debug.Log("Weapon swing!");
    }

    public void OnAnimationEnd()
    {
        Debug.Log("Animation ended!");
    }
}

```

19.10 UI System (iuaai saisataema)

****Basic UI:****

```

// Basic UI
using UnityEngine;
using UnityEngine.UI;

```

```
using TMPro;

public class UIExample : MonoBehaviour
{
    public Text healthText;
    public Slider healthSlider;
    public Image healthBar;
    public TextMeshProUGUI scoreText;
    public GameObject gameOverPanel;
    public Button restartButton;

    private int score = 0;
    private float health = 100f;

    void Start()
    {
        // Button Click
        restartButton.onClick.AddListener(RestartGame);

        // Initial UI
        UpdateUI();
    }

    void Update()
    {
        // Score Update
        if (Input.GetKeyDown(KeyCode.Space))
        {
            score += 10;
            UpdateUI();
        }

        // Health Update
        if (Input.GetKeyDown(KeyCode.H))
        {
            health -= 10;
            UpdateUI();
        }
    }

    void UpdateUI()
    {
        // Text Update
        healthText.text = "Health: " + health.ToString();
        scoreText.text = "Score: " + score.ToString();

        // Slider Update
        healthSlider.value = health / 100f;

        // Image Update
        healthBar.fillAmount = health / 100f;

        // Color Change
        if (health > 50)
            healthBar.color = Color.green;
        else if (health > 25)
            healthBar.color = Color.yellow;
        else
            healthBar.color = Color.red;

        // Game Over
        if (health <= 0)
        {
            gameOverPanel.SetActive(true);
        }
    }
}
```

```

    }
}

void RestartGame()
{
    UnityEngine.SceneManagement.SceneManager.LoadScene(
        UnityEngine.SceneManagement.SceneManager.GetActiveScene().name);
}
}

```

****UI Animation:****

```

// UI Animation
using UnityEngine;
using UnityEngine.UI;
using TMPro;

public class UIAnimation : MonoBehaviour
{
    public TextMeshProUGUI titleText;
    public CanvasGroup fadePanel;

    void Start()
    {
        // Title Animation
        StartCoroutine(AnimateTitle());

        // Fade In
        StartCoroutine(FadeIn());
    }

    System.Collections.IEnumerator AnimateTitle()
    {
        float duration = 2f;
        float elapsed = 0f;

        while (elapsed < duration)
        {
            elapsed += Time.deltaTime;
            float t = elapsed / duration;

            titleText.transform.localScale = Vector3.one * Mathf.Lerp(0.5f, 1f, t);
            titleText.color = Color.Lerp(Color.white, Color.yellow, t);

            yield return null;
        }
    }

    System.Collections.IEnumerator FadeIn()
    {
        float duration = 1f;
        float elapsed = 0f;

        while (elapsed < duration)
        {
            elapsed += Time.deltaTime;
            float t = elapsed / duration;

            fadePanel.alpha = Mathf.Lerp(1f, 0f, t);

            yield return null;
        }

        fadePanel.gameObject.SetActive(false);
    }
}

```

```
}
```

19.11 Audio System (adaiau saisataema)

****Audio Management:****

```
// Audio Management
using UnityEngine;

public class AudioExample : MonoBehaviour
{
    public AudioSource musicSource;
    public AudioSource sfxSource;

    public AudioClip jumpSound;
    public AudioClip hitSound;
    public AudioClip coinSound;
    public AudioClip backgroundMusic;

    void Start()
    {
        // Play Music
        musicSource.clip = backgroundMusic;
        musicSource.loop = true;
        musicSource.volume = 0.5f;
        musicSource.Play();
    }

    void Update()
    {
        // Play SFX
        if (Input.GetKeyDown(KeyCode.Space))
        {
            PlaySound(jumpSound);
        }

        if (Input.GetKeyDown(KeyCode.Mouse0))
        {
            PlaySound(hitSound);
        }

        if (Input.GetKeyDown(KeyCode.C))
        {
            PlaySound(coinSound);
        }

        // Volume Control
        if (Input.GetKeyDown(KeyCode.UpArrow))
        {
            musicSource.volume += 0.1f;
        }

        if (Input.GetKeyDown(KeyCode.DownArrow))
        {
            musicSource.volume -= 0.1f;
        }

        // Pause/Resume
        if (Input.GetKeyDown(KeyCode.P))
        {
            if (musicSource.isPlaying)
```

```
        musicSource.Pause();
    else
        musicSource.UnPause();
    }
}

void PlaySound(AudioClip clip)
{
    sfxSource.PlayOneShot(clip);
}

void PlaySoundAtPosition(AudioClip clip, Vector3 position)
{
    AudioSource.PlayClipAtPoint(clip, position);
}
}
```

****3D Audio:****

```
// 3D Audio
using UnityEngine;

public class Audio3DExample : MonoBehaviour
{
    public AudioSource audioSource;

    void Start()
    {
        // 3D Audio Settings
        audioSource.spatialBlend = 1f; // 0 = 2D, 1 = 3D
        audioSource.minDistance = 1f;
        audioSource.maxDistance = 50f;
        audioSource.rolloffMode = AudioRolloffMode.Logarithmic;

        // Doppler Level
        audioSource.dopplerLevel = 1f;

        // Spread
        audioSource.spread = 0f;
    }

    void Update()
    {
        // Move sound source
        transform.position += transform.forward * Time.deltaTime * 5f;
    }
}
```

19.12 Particles (kanaaaa)

****Particle System:****

```
// Particle System
using UnityEngine;

public class ParticleExample : MonoBehaviour
{
    public ParticleSystem fireEffect;
    public ParticleSystem explosionEffect;
    public ParticleSystem smokeEffect;
```

```

void Start()
{
    // Play Particle
    fireEffect.Play();

    // Stop Particle
    // fireEffect.Stop();

    // Pause Particle
    // fireEffect.Pause();

    // Clear Particle
    // fireEffect.Clear();
}

void Update()
{
    // Trigger Explosion
    if (Input.GetKeyDown(KeyCode.E))
    {
        explosionEffect.transform.position = transform.position;
        explosionEffect.Play();
    }

    // Trigger Smoke
    if (Input.GetKeyDown(KeyCode.S))
    {
        smokeEffect.Play();
    }

    // Emit Particles
    if (Input.GetKeyDown(KeyCode.Space))
    {
        fireEffect.Emit(10); // 10 particles emit
    }

    // Check if playing
    if (fireEffect.isPlaying)
    {
        Debug.Log("Fire is playing");
    }
}

// Particle Collision
void OnParticleCollision(GameObject other)
{
    Debug.Log("Particle hit: " + other.name);
}
}

```

19.13 Lighting (aalaoka bayabasathaaa)

****Lighting Setup:****

```

// Lighting Setup
using UnityEngine;

public class LightingExample : MonoBehaviour
{
    public Light directionalLight;
    public Light pointLight;
}

```

```

public Light spotLight;

void Start()
{
    // Directional Light
    directionalLight.type = LightType.Directional;
    directionalLight.intensity = 1f;
    directionalLight.color = Color.white;
    directionalLight.shadows = LightShadows.Soft;

    // Point Light
    pointLight.type = LightType.Point;
    pointLight.intensity = 2f;
    pointLight.range = 10f;
    pointLight.color = Color.yellow;

    // Spot Light
    spotLight.type = LightType.Spot;
    spotLight.intensity = 3f;
    spotLight.range = 20f;
    spotLight.spotAngle = 45f;
    spotLight.color = Color.red;
}

void Update()
{
    // Animate Light
    float intensity = Mathf.Sin(Time.time) * 0.5f + 0.5f;
    pointLight.intensity = intensity * 3f;

    // Move Light
    pointLight.transform.position = transform.position +
        new Vector3(Mathf.Sin(Time.time) * 2f, 0, Mathf.Cos(Time.time) * 2f);
}
}

```

19.14 Post Processing (paosata parasaesai)

****Post Processing Effects:****

```

// Post Processing
using UnityEngine;
using UnityEngine.Rendering;
using UnityEngine.Rendering.Universal;

public class PostProcessingExample : MonoBehaviour
{
    public Volume volume;
    private Bloom bloom;
    private Vignette vignette;
    private ChromaticAberration chromaticAberration;

    void Start()
    {
        // Get Post Processing Effects
        if (volume.profile.TryGet<Bloom>(out bloom))
        {
            bloom.intensity.value = 2f;
            bloom.threshold.value = 1f;
        }
    }
}

```



```

        if (volume.profile.TryGet<Vignette>(out vignette))
        {
            vignette.intensity.value = 0.5f;
        }

        if (volume.profile.TryGet<ChromaticAberration>(out chromaticAberration))
        {
            chromaticAberration.intensity.value = 0.5f;
        }
    }

    void Update()
    {
        // Animate Bloom
        if (bloom != null)
        {
            bloom.intensity.value = Mathf.Sin(Time.time) * 2f + 3f;
        }

        // Vignette Effect
        if (Input.GetKeyDown(KeyCode.V))
        {
            vignette.intensity.value = 0.8f;
            StartCoroutine(ResetVignette());
        }
    }

    System.Collections.IEnumerator ResetVignette()
    {
        yield return new WaitForSeconds(0.5f);
        vignette.intensity.value = 0.5f;
    }
}

```

19.15 Save System (saebha saisataema)

****PlayerPrefs Save:****

```

// PlayerPrefs Save System
using UnityEngine;

public class PlayerPrefsSave : MonoBehaviour
{
    public void SaveGame()
    {
        // Save Data
        PlayerPrefs.SetInt("Score", 1000);
        PlayerPrefs.SetFloat("Health", 75.5f);
        PlayerPrefs.SetString("PlayerName", "Arjun");
        PlayerPrefs.SetInt("Level", 5);

        // Save Now
        PlayerPrefs.Save();
    }

    public void LoadGame()
    {
        // Load Data
        int score = PlayerPrefs.GetInt("Score", 0);
        float health = PlayerPrefs.GetFloat("Health", 100f);
    }
}

```

```

        string playerName = PlayerPrefs.GetString("PlayerName", "Default");
        int level = PlayerPrefs.GetInt("Level", 1);

        Debug.Log("Score: " + score);
        Debug.Log("Health: " + health);
        Debug.Log("Player Name: " + playerName);
        Debug.Log("Level: " + level);
    }

    public void DeleteSave()
    {
        PlayerPrefs.DeleteAll();
    }

    public bool HasSave()
    {
        return PlayerPrefs.HasKey("Score");
    }
}

```

****JSON Save System:****

```

// JSON Save System
using UnityEngine;
using System.IO;

[System.Serializable]
public class SaveData
{
    public int score;
    public float health;
    public string playerName;
    public int level;
    public float[] position;
}

public class JsonSaveSystem : MonoBehaviour
{
    private string savePath;

    void Start()
    {
        savePath = Application.persistentDataPath + "/savegame.json";
    }

    public void SaveGame()
    {
        SaveData data = new SaveData();
        data.score = 1000;
        data.health = 75.5f;
        data.playerName = "Arjun";
        data.level = 5;
        data.position = new float[] {
            transform.position.x,
            transform.position.y,
            transform.position.z
        };

        string json = JsonUtility.ToJson(data, true);
        File.WriteAllText(savePath, json);

        Debug.Log("Game saved to: " + savePath);
    }
}

```

```

public void LoadGame()
{
    if (File.Exists(savePath))
    {
        string json = File.ReadAllText(savePath);
        SaveData data = JsonUtility.FromJson<SaveData>(json);

        Debug.Log("Score: " + data.score);
        Debug.Log("Health: " + data.health);
        Debug.Log("Player Name: " + data.playerName);
        Debug.Log("Level: " + data.level);

        transform.position = new Vector3(
            data.position[0],
            data.position[1],
            data.position[2]
        );
    }
}

public void DeleteSave()
{
    if (File.Exists(savePath))
    {
        File.Delete(savePath);
    }
}
}

```

19.16 Optimization (apataimaaaijaeshana)

****Object Pooling:****

```

// Object Pooling
using UnityEngine;
using System.Collections.Generic;

public class ObjectPool : MonoBehaviour
{
    public GameObject prefab;
    public int poolSize = 10;

    private List<GameObject> pool = new List<GameObject>();

    void Start()
    {
        // Create Pool
        for (int i = 0; i < poolSize; i++)
        {
            GameObject obj = Instantiate(prefab);
            obj.SetActive(false);
            pool.Add(obj);
        }
    }

    public GameObject GetObject()
    {
        // Get inactive object
        foreach (GameObject obj in pool)
        {

```

```

        if (!obj.activeInHierarchy)
        {
            obj.SetActive(true);
            return obj;
        }
    }

    // Create new object if pool is empty
    GameObject newObj = Instantiate(prefab);
    pool.Add(newObj);
    return newObj;
}

public void ReturnObject(GameObject obj)
{
    obj.SetActive(false);
}
}

// Usage
public class BulletSpawner : MonoBehaviour
{
    public ObjectPool bulletPool;

    void Update()
    {
        if (Input.GetKeyDown(KeyCode.Space))
        {
            GameObject bullet = bulletPool.GetObject();
            bullet.transform.position = transform.position;
            bullet.transform.rotation = transform.rotation;

            // Return after 3 seconds
            StartCoroutine(ReturnBullet(bullet, 3f));
        }
    }

    System.Collections.IEnumerator ReturnBullet(GameObject bullet, float delay)
    {
        yield return new WaitForSeconds(delay);
        bulletPool.ReturnObject(bullet);
    }
}

```

****LOD (Level of Detail):****

```

// LOD System
using UnityEngine;

public class LODSystem : MonoBehaviour
{
    public GameObject highDetailModel;
    public GameObject mediumDetailModel;
    public GameObject lowDetailModel;

    public float highDetailDistance = 10f;
    public float mediumDetailDistance = 30f;

    private Transform player;

    void Start()
    {
        player = GameObject.FindGameObjectWithTag("Player").transform;
    }
}

```

```

void Update()
{
    float distance = Vector3.Distance(transform.position, player.position);

    // High Detail
    if (distance < highDetailDistance)
    {
        highDetailModel.SetActive(true);
        mediumDetailModel.SetActive(false);
        lowDetailModel.SetActive(false);
    }
    // Medium Detail
    else if (distance < mediumDetailDistance)
    {
        highDetailModel.SetActive(false);
        mediumDetailModel.SetActive(true);
        lowDetailModel.SetActive(false);
    }
    // Low Detail
    else
    {
        highDetailModel.SetActive(false);
        mediumDetailModel.SetActive(false);
        lowDetailModel.SetActive(true);
    }
}
}

```

****Culling:****

```

// Culling System
using UnityEngine;

public class CullingSystem : MonoBehaviour
{
    public float cullDistance = 50f;

    private Renderer[] renderers;
    private Transform player;

    void Start()
    {
        renderers = GetComponentsInChildren<Renderer>();
        player = GameObject.FindGameObjectWithTag("Player").transform;
    }

    void Update()
    {
        float distance = Vector3.Distance(transform.position, player.position);

        bool isVisible = distance < cullDistance;

        foreach (Renderer renderer in renderers)
        {
            renderer.enabled = isVisible;
        }
    }
}

```

19.17 Common Unity Mistakes (saaadhaaarana bhaula)

bhault 1: Find in Update

```
// BAD
void Update()
{
    GameObject player = GameObject.FindWithTag("Player");
}

// GOOD
private GameObject player;
void Start()
{
    player = GameObject.FindWithTag("Player");
}
```

bhault 2: String Concatenation

```
// BAD
void Update()
{
    Debug.Log("Score: " + score.ToString());
}

// GOOD
void Update()
{
    Debug.Log($"Score: {score}");
}
```

bhault 3: GC Allocation

```
// BAD
void Update()
{
    Vector3 pos = new Vector3(0, 0, 0);
}

// GOOD
private Vector3 pos = Vector3.zero;
void Update()
{
    pos.x = 0;
    pos.y = 0;
    pos.z = 0;
}
```

bhault 4: Coroutine Allocation

```
// BAD
void Update()
{
    StartCoroutine(DoSomething());
}

// GOOD
private bool isDoingSomething = false;
void Update()
{
    if (!isDoingSomething)
    {
        StartCoroutine(DoSomething());
    }
}
```

```
}
```

bhaulta 5: Null Check

```
// BAD
void Update()
{
    if (player != null)
    {
        player.Move();
    }
}

// GOOD
void Update()
{
    if (player)
    {
        player.Move();
    }
}
```

19.18 Build (bailada)

****Build Settings:****

Build Settings:

1. File -> Build Settings
2. Add Open Scenes
3. Select Platform
 - PC, Mac & Linux Standalone
 - Android
 - iOS
 - WebGL
4. Player Settings
 - Company Name
 - Product Name
 - Version
 - Resolution
 - Icon
5. Build

****Build Optimization:****

```
// Build Optimization Tips

// 1. Strip Engine Code
// Player Settings -> Other Settings -> Managed Stripping Level: High

// 2. Texture Compression
// Import Settings -> Compression: ASTC (Mobile), DXT (PC)

// 3. Audio Compression
// Import Settings -> Compression: Vorbis

// 4. Mesh Compression
// Import Settings -> Compression: Medium

// 5. Level of Detail
// Use LOD Groups for distant objects
```

```
// 6. Occlusion Culling
// Window -> Rendering -> Occlusion Culling

// 7. Batching
// Static Batching for static objects
// Dynamic Batching for small dynamic objects

// 8. GPU Instancing
// Material -> Enable GPU Instancing

// 9. Addressables
// Use Addressables for large assets

// 10. Profiling
// Window -> Analysis -> Profiler
```

19.19 Unity Tips and Best Practices

taipa 1: Naming Conventions

- Scripts: PascalCase (PlayerController)
- Variables: camelCase (playerHealth)
- Constants: UPPER_SNAKE_CASE (MAX_HEALTH)
- Prefabs: Descriptive (RedCar_01)
- Tags: Lowercase (player, enemy)

taipa 2: Folder Structure

```
Assets/
  Scripts/
    Player/
    Enemy/
    UI/
  Prefabs/
    Characters/
    Enemies/
    UI/
  Materials/
  Textures/
  Models/
  Animations/
  Audio/
  Scenes/
  Resources/
```

taipa 3: Code Organization

```
// Use Region
#region Variables
public float speed = 5f;
#endregion

#region Unity Methods
void Start() { }
void Update() { }
#endregion

#region Custom Methods
void Move() { }
void Rotate() { }
```



```
#endregion
```

taipa 4: Debug Tips

```
// Debug.Log
Debug.Log("Message");

// Debug.LogWarning
Debug.LogWarning("Warning");

// Debug.LogError
Debug.LogError("Error");

// Debug.DrawRay
Debug.DrawRay(transform.position, transform.forward * 10f, Color.red);

// Debug.Break
Debug.Break(); // Pause editor
```

taipa 5: Performance Tips

```
// Cache References
private Transform player;
void Start()
{
    player = GameObject.FindGameObjectWithTag("Player").transform;
}

// Avoid GetComponent in Update
private Rigidbody rb;
void Start()
{
    rb = GetComponent<Rigidbody>();
}

// Use Object Pooling
// Use LOD
// Use Occlusion Culling
// Use Static Batching
// Use GPU Instancing
```

19.20 Summary (saaarasakassaepa)

Unity Development halao gaema taairai karaaara aikatai shakataishaaalaili padadhatai aikatai bhaaalao Unity Developer hatae halae:

1. **Project Setup** shaikhauna -- parajaekata saetaaapa karauna
2. **Scenes** shaikhauna -- saina mayaaanaejamaenata
3. **GameObjects** shaikhauna -- abajaekata mayaaanaejamaenata
4. **Prefabs** shaikhauna -- reusable objects
5. **Scripts** shaikhauna -- C# programming
6. **Input System** shaikhauna -- inapauta mayaaanaejamaenata
7. **Physics** shaikhauna -- padaaarathabajanyaaana
8. **Animation** shaikhauna -- ayaaanaimaeshana
9. **UI** shaikhauna -- iujaaara inataaaraphaesa
10. **Audio** shaikhauna -- adaiau mayaaanaejamaenata
11. **Particles** shaikhauna -- kanaaa iphaekata

12. ****Lighting**** shaikhauna -- aalaoka bayabasathaaa
13. ****Post Processing**** shaikhauna -- paosata parasaesai
14. ****Save System**** shaikhauna -- saebha saisataema
15. ****Optimization**** shaikhauna -- apataimaaaijaeshana
16. ****Build**** shaikhauna -- bailada aiba daepalayamaenata

****manae raaakhauna:****

"Unity Development halao aikatai daiiragha yaaataraaa parataitai daina natauna kaichhau shaikhauna bhaula karauna, shaikhauna, aigaiyae yaaana"

anaushaiilana (Exercises)

anaushaiilana 1: Basic Script

aikatai Player Controller Script laikhauna

anaushaiilana 2: Input System

New Input System bayabahaaara karae aikatai Input Handler taairai karauna

anaushaiilana 3: Physics

aikatai Physics-based Movement System taairai karauna

anaushaiilana 4: UI

aikatai Health Bar UI taairai karauna

anaushaiilana 5: Save System

aikatai JSON Save System taairai karauna

anaushaiilana 6: Object Pooling

aikatai Bullet Pool System taairai karauna

anaushaiilana 7: Complete Game

aikatai samapauurana chhaota gaema taairai karauna

****parabarataii adhayaaaya: [Chapter 20: UNREAL ENGINE DEVELOPMENT](20_unreal.md)****

PART 20

adhayaaaya 20: UNREAL ENGINE DEVELOPMENT -- aanaraiyaaala inyajaina daebhaelapamaenata

adhayaaaya 20: UNREAL ENGINE DEVELOPMENT -- aanaraiyaaala inyajaina daebhaelapamaenata

shaekhaara udadaeshaya (Learning Goal)

aii adhayaaaya shaessae aapanai jaaanabaena:

- Complete Unreal Engine Roadmap
 - Blueprint Visual Scripting
 - C++ Basics for Unreal
 - Level, Actor, Component, Material
 - Niagara VFX, Animation, AI
 - UI, Lighting, Optimization, Packaging
 - Unreal-specific tips
-

20.1 Unreal Engine Roadmap

Unreal Engine Development Roadmap:

Level 1: Fundamentals

Engine Overview
Editor Interface
Blueprint Basics
Actors & Components
Basic Materials

Level 2: Core Systems

Blueprint Intermediate
C++ Basics
Input System
Physics & Collision
Animation Blueprint

Level 3: Advanced

Niagara VFX
AI System
UI System
Lighting System

Audio System

Level 4: Optimization

Profiling

LOD System

Culling

Memory Management

Performance Tips

Level 5: Packaging

Build Settings

Platform Optimization

Testing

Distribution

20.2 Engine Overview (inyajaina aubhaaarabhaiu)

****Unreal Engine Install:****

Step 1: Epic Games Launcher

- unrealengine.com thaekae Epic Games Launcher Download karauna
- Launcher Install karauna
- Epic Games Account taairai karauna

Step 2: Engine Install

- Epic Games Launcher -> Unreal Engine -> Library
- Engine Version Install karauna
- LTS Version bayabahaaara karauna

Step 3: New Project

- Launch Engine
- New Project Category
- Template Select: Third Person, First Person, Top Down
- Project Settings
- Create

****Editor Interface:****

Unreal Engine Editor Layout:

	Menu Bar	
Outliner	Viewport	Details
Panel		Panel
Content		Materials
Browser		Browser
	Sequencer	

20.3 Blueprint Basics (balauparainata baesaika)

Blueprint halao Unreal Engine aira Visual Scripting System aitari kaoda naaa laikhae gaema lajaika taairai karaaara padadhatai

****Blueprint Types:****

****Blueprint Editor:****

Blueprint Editor:



****Basic Blueprint Nodes:****

Common Blueprint Nodes:

1. Event Nodes:
 - Begin Play
 - Tick
 - Actor Begin Overlap
 - Actor End Overlap
2. Function Nodes:
 - Print String
 - Set Actor Location
 - Get Actor Location
 - Spawn Actor From Class
3. Variable Nodes:
 - Set (Variable Name)
 - Get (Variable Name)
 - Float + Float
 - Int + Int
4. Flow Control:
 - Branch (If)
 - For Loop
 - While Loop
 - Sequence
 - Flip Flop
5. Math:
 - Add
 - Subtract
 - Multiply
 - Divide
 - Lerp

20.4 Blueprint Examples

Player Movement Blueprint:

Player Movement Blueprint:

Event Graph:

```

graph TD
    BeginPlay[Begin Play] --> GetController[Get Controller -> Cast to PlayerController]
    GetController --> SetSpeed[Set Max Walk Speed: 600]
    SetSpeed --> EventTick[Event Tick]
  
```

The diagram shows the sequence of nodes in the Player Movement Blueprint's Event Graph: 'Begin Play' leads to 'Get Controller -> Cast to PlayerController', which then leads to 'Set Max Walk Speed: 600', and finally to 'Event Tick'.

```

Get Input Axis "MoveForward"
Get Input Axis "MoveRight"
Add Movement Input
InputAction Jump
Character Jump

```

Enemy AI Blueprint:

Enemy AI Blueprint:

```

Event Graph:
Event Tick
  Get Player Location
  Get Distance To Player
  Branch: Distance < 1000?
  True: Move To Player
  False: Patrol
On Component Begin Overlap
  Other Actor = Player?
  True: Attack

```

20.5 C++ Basics for Unreal (aanaraiyaaalaera janaya C++ baesaika)

****C++ Setup:****

```

C++ Project Setup:
1. New Project -> C++ Tab
2. Select Template: Basic Code
3. Create Project
4. Visual Studio/Rider Open
5. Edit Source Files
6. Compile

```

****Basic C++ Actor:****

```

// MyActor.h
#pragma once

#include "CoreMinimal.h"
#include "GameFramework/Actor.h"
#include "MyActor.generated.h"

UCLASS()
class MYPROJECT_API AMyActor : public AActor
{
    GENERATED_BODY()

public:
    AMyActor();

protected:
    virtual void BeginPlay() override;

public:
    virtual void Tick(float DeltaTime) override;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    float Speed = 100.0f;

    UPROPERTY(VisibleAnywhere)

```

```

    UStaticMeshComponent* MeshComponent;

    UFUNCTION(BlueprintCallable)
    void MoveForward();

    UFUNCTION(BlueprintCallable)
    void Rotate();
};

// MyActor.cpp
#include "MyActor.h"

AMyActor::AMyActor()
{
    PrimaryActorTick.bCanEverTick = true;

    MeshComponent = CreateDefaultSubobject<UStaticMeshComponent>(TEXT("Mesh"));
    RootComponent = MeshComponent;
}

void AMyActor::BeginPlay()
{
    Super::BeginPlay();

    UE_LOG(LogTemp, Warning, TEXT("Actor Begin Play!"));
}

void AMyActor::Tick(float DeltaTime)
{
    Super::Tick(DeltaTime);

    // Rotation
    FRotator Rotation = GetActorRotation();
    Rotation.Yaw += 45.0f * DeltaTime;
    SetActorRotation(Rotation);
}

void AMyActor::MoveForward()
{
    FVector Forward = GetActorForwardVector();
    FVector NewLocation = GetActorLocation() + Forward * Speed * GetWorld()->GetDeltaSecond();
    SetActorLocation(NewLocation);
}

void AMyActor::Rotate()
{
    AddActorWorldRotation(FRotator(0, 90, 0));
}

```

****C++ Character:****

```

// MyCharacter.h
#pragma once

#include "CoreMinimal.h"
#include "GameFramework/Character.h"
#include "MyCharacter.generated.h"

UCLASS()
class MYPROJECT_API AMyCharacter : public ACharacter
{
    GENERATED_BODY()

public:
    AMyCharacter();

```

```

protected:
    virtual void BeginPlay() override;

public:
    virtual void Tick(float DeltaTime) override;
    virtual void SetupPlayerInputComponent(class UInputComponent* PlayerInputComponent) ov

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    float Health = 100.0f;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    float MaxHealth = 100.0f;

    UFUNCTION(BlueprintCallable)
    void TakeDamage(float DamageAmount);

    UFUNCTION(BlueprintCallable)
    void Heal(float HealAmount);

    UFUNCTION(BlueprintPure)
    float GetHealthPercent() const;

private:
    void MoveForward(float Value);
    void MoveRight(float Value);
    void TurnAtRate(float Rate);
    void LookUpAtRate(float Rate);

    void StartJump();
    void StopJump();
    void StartSprint();
    void StopSprint();

    UPROPERTY(VisibleAnywhere)
    class USpringArmComponent* CameraBoom;

    UPROPERTY(VisibleAnywhere)
    class UCameraComponent* FollowCamera;

    float BaseTurnRate;
    float BaseLookUpRate;
    bool bIsSprinting;
};

// MyCharacter.cpp
#include "MyCharacter.h"
#include "GameFramework/SpringArmComponent.h"
#include "Camera/CameraComponent.h"
#include "GameFramework/CharacterMovementComponent.h"

AMyCharacter::AMyCharacter()
{
    PrimaryActorTick.bCanEverTick = true;

    BaseTurnRate = 45.0f;
    BaseLookUpRate = 45.0f;
    bIsSprinting = false;

    // Camera Boom
    CameraBoom = CreateDefaultSubobject<USpringArmComponent>(TEXT("CameraBoom"));
    CameraBoom->SetupAttachment(RootComponent);
    CameraBoom->TargetArmLength = 300.0f;
    CameraBoom->bUsePawnControlRotation = true;

    // Follow Camera

```



```

FollowCamera = CreateDefaultSubobject<UCameraComponent>(TEXT("FollowCamera"));
FollowCamera->SetupAttachment(CameraBoom, USpringArmComponent::SocketName);
FollowCamera->bUsePawnControlRotation = false;

// Movement
GetCharacterMovement()->MaxWalkSpeed = 600.0f;
GetCharacterMovement()->NavAgentProps.bCanCrouch = true;
}

void AMyCharacter::BeginPlay()
{
    Super::BeginPlay();
}

void AMyCharacter::Tick(float DeltaTime)
{
    Super::Tick(DeltaTime);
}

void AMyCharacter::SetupPlayerInputComponent(UInputComponent* PlayerInputComponent)
{
    Super::SetupPlayerInputComponent(PlayerInputComponent);

    // Movement
    PlayerInputComponent->BindAxis("MoveForward", this, &AMyCharacter::MoveForward);
    PlayerInputComponent->BindAxis("MoveRight", this, &AMyCharacter::MoveRight);

    // Look
    PlayerInputComponent->BindAxis("Turn", this, &APawn::AddControllerYawInput);
    PlayerInputComponent->BindAxis("LookUp", this, &APawn::AddControllerPitchInput);

    // Jump
    PlayerInputComponent->BindAction("Jump", IE_Pressed, this, &AMyCharacter::StartJump);
    PlayerInputComponent->BindAction("Jump", IE_Released, this, &AMyCharacter::StopJump);

    // Sprint
    PlayerInputComponent->BindAction("Sprint", IE_Pressed, this, &AMyCharacter::StartSprint);
    PlayerInputComponent->BindAction("Sprint", IE_Released, this, &AMyCharacter::StopSprint);
}

void AMyCharacter::MoveForward(float Value)
{
    if (Value != 0.0f)
    {
        const FRotator Rotation = Controller->GetControlRotation();
        const FRotator YawRotation(0, Rotation.Yaw, 0);
        const FVector Direction = FRotationMatrix(YawRotation).GetUnitAxis(EAxis::X);
        AddMovementInput(Direction, Value);
    }
}

void AMyCharacter::MoveRight(float Value)
{
    if (Value != 0.0f)
    {
        const FRotator Rotation = Controller->GetControlRotation();
        const FRotator YawRotation(0, Rotation.Yaw, 0);
        const FVector Direction = FRotationMatrix(YawRotation).GetUnitAxis(EAxis::Y);
        AddMovementInput(Direction, Value);
    }
}

void AMyCharacter::TurnAtRate(float Rate)
{

```

```

        AddControllerYawInput(Rate * BaseTurnRate * GetWorld()->GetDeltaSeconds());
    }

void AMyCharacter::LookUpAtRate(float Rate)
{
    AddControllerPitchInput(Rate * BaseLookUpRate * GetWorld()->GetDeltaSeconds());
}

void AMyCharacter::StartJump()
{
    Jump();
    bIsJumping = true;
}

void AMyCharacter::StopJump()
{
    StopJumping();
    bIsJumping = false;
}

void AMyCharacter::StartSprint()
{
    bIsSprinting = true;
    GetCharacterMovement()->MaxWalkSpeed = 1200.0f;
}

void AMyCharacter::StopSprint()
{
    bIsSprinting = false;
    GetCharacterMovement()->MaxWalkSpeed = 600.0f;
}

void AMyCharacter::TakeDamage(float DamageAmount)
{
    Health = FMath::Clamp(Health - DamageAmount, 0.0f, MaxHealth);

    if (Health <= 0.0f)
    {
        // Die
        UE_LOG(LogTemp, Warning, TEXT("Character Died!"));
    }
}

void AMyCharacter::Heal(float HealAmount)
{
    Health = FMath::Clamp(Health + HealAmount, 0.0f, MaxHealth);
}

float AMyCharacter::GetHealthPercent() const
{
    return Health / MaxHealth;
}

```

20.6 Level Design (laebhaela daijaaaina)

****Level Blueprint:****

Level Blueprint:

Event Graph:
Begin Play

```

Set Game Mode
Set Player Start
Spawn Enemy
Actor Begin Overlap
Other Actor = Player?
True: Load Next Level

```

****Level Streaming:****

```

// Level Streaming
#include "Kismet/GameplayStatics.h"

void AMyActor::LoadLevel(FName LevelName)
{
    UGameplayStatics::OpenLevel(GetWorld(), LevelName);
}

void AMyActor::LoadLevelAdditive(FName LevelName)
{
    FLatentActionInfo LatentInfo;
    UGameplayStatics::LoadStreamLevel(GetWorld(), LevelName, true, true, LatentInfo);
}

void AMyActor::UnloadLevel(FName LevelName)
{
    FLatentActionInfo LatentInfo;
    UGameplayStatics::UnloadStreamLevel(GetWorld(), LevelName, LatentInfo, true);
}

```

20.7 Actor and Component (ayaaakatara aiba kamapaonaenata)

****Actor Types:****

****Common Components:****

****Component Example:****

```

// Component Example
#include "Components/StaticMeshComponent.h"
#include "Components/BoxComponent.h"
#include "Components/SphereComponent.h"
#include "Components/AudioComponent.h"
#include "Components/ParticleSubUVComponent.h"

AMyActor::AMyActor()
{
    // Static Mesh
    UStaticMeshComponent* Mesh = CreateDefaultSubobject<UStaticMeshComponent>(TEXT("Mesh"))
    RootComponent = Mesh;

    // Box Collision
    UBoxComponent* Box = CreateDefaultSubobject<UBoxComponent>(TEXT("Box"));
    Box->SetupAttachment(RootComponent);
    Box->SetBoxExtent(FVector(100, 100, 100));

    // Sphere Collision
    USphereComponent* Sphere = CreateDefaultSubobject<USphereComponent>(TEXT("Sphere"));
    Sphere->SetupAttachment(RootComponent);
    Sphere->SetSphereRadius(100);
}

```

```

// Audio
UAudioComponent* Audio = CreateDefaultSubobject<UAudioComponent>(TEXT("Audio"));
Audio->SetupAttachment(RootComponent);

// Particle
UParticleSubUVComponent* Particle = CreateDefaultSubobject<UParticleSubUVComponent>(TE
Particle->SetupAttachment(RootComponent);
}

```

20.8 Material (mayaaataeraiyaaala)

****Material Editor:****

Material Editor:

Material Editor		
Details Panel	Node Graph	Preview Panel
	Texture Sample	
	Base Color >	
	Normal >	
	Roughness >	

****Material Properties:****

****Material Nodes:****

Common Material Nodes:

- Input Nodes:
 - Texture Sample
 - Constant
 - Constant2Vector
 - Constant3Vector
 - Constant4Vector
- Math Nodes:
 - Add
 - Subtract
 - Multiply
 - Divide
 - Lerp
 - Power
 - Sine
 - Cosine
- Utility Nodes:
 - If
 - Saturate
 - Clamp
 - Step
 - Smoothstep
- Coordinate Nodes:
 - TexCoord
 - Panner
 - Rotator
 - TextureCoordinate

20.9 Niagara VFX (naiyaaagaaaa VFX)

****Niagara System:****

Niagara Editor:

	Niagara Editor	
System	Emitter Graph	Preview
Overview		Panel
Emitter	Emitter Update	
Module	Spawn Rate	
	Initialize	
	Update Position	
	Render Sprite	

****Niagara Modules:****

****Niagara Example:****

```
// Spawn Niagara Effect
#include "NiagaraFunctionLibrary.h"
#include "NiagaraComponent.h"

void AMyActor::SpawnEffect()
{
    UNiagaraComponent* Effect = UNiagaraFunctionLibrary::SpawnSystemAtLocation(
        GetWorld(),
        FireEffect,
        GetActorLocation(),
        GetActorRotation(),
        FVector(1, 1, 1),
        true,
        true,
        true
    );
}
```

20.10 Animation (ayaaanaimaeshana)

****Animation Blueprint:****

Animation Blueprint:

Event Graph:

```
Event Blueprint Update Animation
Get Velocity
Get Speed (Length)
Set Speed Variable
Is Falling?
Set IsFalling Variable
```

AnimGraph:

```
State Machine
Idle -> Walk -> Run -> Jump -> Fall -> Land
Blend Poses by Speed
Speed 0: Idle
Speed 0-300: Walk
Speed 300-600: Run
```

****Animation Nodes:****

20.11 AI System (AI saisataema)

****AI Controller:****

```
// AI Controller
#include "AIController.h"
#include "NavigationSystem.h"
#include "Blueprint/AIBlueprintHelperLibrary.h"

void AAIController::BeginPlay()
{
    Super::BeginPlay();
}

void AAIController::MoveToPlayer()
{
    APawn* PlayerPawn = GetWorld()->GetFirstPlayerController()->GetPawn();
    if (PlayerPawn)
    {
        MoveToActor(PlayerPawn, 200.0f);
    }
}

void AAIController::MoveToLocation(FVector Location)
{
    MoveToLocation(Location, 200.0f);
}

void AAIController::Patrol()
{
    UNavigationSystemV1* NavSystem = UNavigationSystemV1::GetCurrent(GetWorld());
    if (NavSystem)
    {
        FVector RandomLocation;
        if (NavSystem->GetRandomReachablePointInRadius(GetPawn()->GetActorLocation(), 1000)
        {
            MoveToLocation(RandomLocation);
        }
    }
}
```

****Behavior Tree:****

Behavior Tree:

Root

Selector

Sequence (Attack)

Condition: Is Player Visible?

Condition: Is Player In Range?

Task: Move To Player

Task: Attack

Sequence (Chase)

Condition: Is Player Visible?

Task: Move To Player

Sequence (Patrol)

Task: Get Random Location

Task: Move To Location

****Blackboard:****

Blackboard Keys:

Key	Type	Description
TargetActor	Object	Target to attack
TargetLocation	Vector	Target location
LastKnownLocation	Vector	Last known location
CurrentState	Enum	AI state
PatrolIndex	Int	Current patrol point

20.12 UI System (UI saisataema)

****Widget Blueprint:****

Widget Blueprint:

Designer:

	Widget Designer	
Palette	Widget Canvas	Details Panel
Text		
Image	Health Bar	
Button		
Slider		
Progress	[Start Game]	
	[Settings]	
	[Quit]	

****Widget Example:****

```
// Widget Example
#include "Blueprint/UserWidget.h"

void AMyHUD::BeginPlay()
{
    Super::BeginPlay();

    if (HealthWidgetClass)
    {
        HealthWidget = CreateWidget<UUserWidget>(GetWorld(), HealthWidgetClass);
        if (HealthWidget)
        {
            HealthWidget->AddToViewport();
        }
    }
}

void AMyHUD::UpdateHealth(float HealthPercent)
{
    if (HealthWidget)
    {
        UProgressBar* HealthBar = Cast<UProgressBar>(HealthWidget->GetWidgetFromName("Heal
        if (HealthBar)
        {
            HealthBar->SetPercent(HealthPercent);
        }
    }
}
```

```
}  
}
```

20.13 Lighting System (aalaoka saisataema)

****Light Types:****

****Lighting Features:****

****Lighting Example:****

```
// Lighting Example  
#include "Components/PointLightComponent.h"  
#include "Components/SpotLightComponent.h"  
  
void AMyActor::SetupLighting()  
{  
    // Point Light  
    UPointLightComponent* PointLight = CreateDefaultSubobject<UPointLightComponent>(TEXT("PointLight"));  
    PointLight->SetupAttachment(RootComponent);  
    PointLight->SetIntensity(1000.0f);  
    PointLight->SetLightColor(FLinearColor(1, 0.8, 0.6));  
    PointLight->SetAttenuationRadius(500.0f);  
  
    // Spot Light  
    USpotLightComponent* SpotLight = CreateDefaultSubobject<USpotLightComponent>(TEXT("SpotLight"));  
    SpotLight->SetupAttachment(RootComponent);  
    SpotLight->SetIntensity(2000.0f);  
    SpotLight->SetLightColor(FLinearColor(1, 1, 1));  
    SpotLight->SetAttenuationRadius(1000.0f);  
    SpotLight->SetOuterConeAngle(45.0f);  
    SpotLight->SetInnerConeAngle(30.0f);  
}
```

20.14 Optimization (apataimaaaijaeshana)

****LOD System:****

LOD System:

LOD 0: High Detail
High
Distance: 0-10m

LOD 1: Medium Detail
Med
Distance: 10-30m

LOD 2: Low Detail
Low
Distance: 30-50m

LOD 3: Billboard
Distance: 50m+

****Optimization Tips:****


```
// Optimization Tips

// 1. Object Pooling
TArray<AActor*> Pool;
for (int i = 0; i < PoolSize; i++)
{
    AActor* Actor = GetWorld()->SpawnActor<AActor>(ActorClass);
    Actor->SetActorHiddenInGame(true);
    Actor->SetActorEnableCollision(false);
    Actor->SetActorTickEnabled(false);
    Pool.Add(Actor);
}

// 2. Culling
// Distance Cull Flag
// Min Draw Distance
// Max Draw Distance

// 3. LOD
// Auto Generate LODs
// LOD Group
// LOD Screen Size

// 4. Occlusion
// Occlusion Culling
// Distance Field Ambient Occlusion

// 5. instancing
// HISM (Hierarchical Instanced Static Mesh)
// GPU Instancing
```

****Performance Profiling:****

Performance Profiling:

1. stat fps - FPS daekhauna
2. stat unit - Frame Time daekhauna
3. stat game - Game Thread
4. stat gpu - GPU Time
5. stat scenerendering - Scene Rendering
6. stat rhi - RHI Stats

Profiling Tools:

- Session Frontend
- Unreal Insights
- GPU Visualizer
- Shader Complexity

20.15 Packaging (payaaakaejai)

****Build Settings:****

Build Settings:

1. File -> Package Project
2. Select Platform
 - Windows
 - macOS
 - Linux
 - Android
 - iOS
 - Console

3. Package

****Platform Optimization:****

****Packaging Checklist:****

Project Settings

- Game Mode
- Input Settings
- Audio Settings
- Video Settings

Assets

- Textures Compressed
- Meshes Optimized
- Materials Optimized
- Audio Compressed

Code

- C++ Compiled
- Blueprints Saved
- Debug Code Removed

Build

- Platform Selected
- Configuration Set
- Shipping Build

Testing

- Gameplay Tested
- Performance Tested
- Memory Tested
- Controls Tested

20.16 Unreal Engine Tips

taipa 1: Blueprint Tips

- Blueprint sakassaipata raaakhauna
- Function bayabahaaara karauna
- Macro bayabahaaara karauna
- Comment yaoga karauna

taipa 2: C++ Tips

- Header files sathaika raaakhauna
- UPROPERTY/UFUNCTION bayabahaaara karauna
- Forward Declaration bayabahaaara karauna
- Smart Pointers bayabahaaara karauna

taipa 3: Material Tips

- Material Instances bayabahaaara karauna
- Texture Atlases bayabahaaara karauna

- Shader Complexity kama raaakhauna

taipa 4: Lighting Tips

- Static Lighting bayabahaaara karauna
- Lightmaps apataimaaaija karauna
- Shadow Cascades saeta karauna

taipa 5: AI Tips

- Behavior Tree bayabahaaara karauna
- Navigation Mesh bayabahaaara karauna
- EQS (Environment Query System) bayabahaaara karauna

taipa 6: UI Tips

- Widget Animation bayabahaaara karauna
- Anchor System bayabahaaara karauna
- DPI Scaling bayabahaaara karauna

taipa 7: Performance Tips

- stat bayabahaaara karauna
- Profiling karauna
- Memory manaitara karauna

taipa 8: Debug Tips

- UE_LOG bayabahaaara karauna
- Draw Debug Lines bayabahaaara karauna
- Visual Logger bayabahaaara karauna

taipa 9: Version Control

- Git bayabahaaara karauna
- .gitignore saeta aapa karauna
- Branching bayabahaaara karauna

taipa 10: Documentation

- README laikhauna
- Code Comments yaoga karauna
- Wiki taairai karauna

20.17 Common Unreal Mistakes (saaadhaaarana bhaula)

bhaula 1: Blueprint Overuse

```
// BAD: Blueprint for everything
// GOOD: C++ for logic, Blueprint for data
```

bhaua 2: Tick Overuse

```
// BAD: Every actor ticks
// GOOD: Disable tick when not needed
PrimaryActorTick.bCanEverTick = false;
```

bhaua 3: String Comparison

```
// BAD: FString comparison
if (Name == "Player") { }

// GOOD: FName comparison
if (Name == FName("Player")) { }
```

bhaua 4: Memory Leak

```
// BAD: Raw pointers
AMyActor* Actor = new AMyActor();

// GOOD: Smart pointers
TSharedPtr<AMyActor> Actor = MakeShared<AMyActor>();
```

bhaua 5: Thread Safety

```
// BAD: Not thread safe
// GOOD: Use async tasks properly
```

20.18 Summary (saaarasakassaepa)

Unreal Engine Development halao gaema taairai karaaara aikatai shakataishaaalaili padadhatai aikatai bhaaalao Unreal Developer hatae halae:

1. **Blueprint** shaikhauna -- Visual Scripting
2. **C++** shaikhauna -- Programming
3. **Level Design** shaikhauna -- laebhaela taairai
4. **Materials** shaikhauna -- parissatha daijaaaina
5. **Niagara** shaikhauna -- VFX
6. **Animation** shaikhauna -- chalaachala
7. **AI** shaikhauna -- karitaraima baudadhahi
8. **UI** shaikhauna -- iujaaara inataaaraphaesa
9. **Lighting** shaikhauna -- aalaoka bayabasathaaa
10. **Optimization** shaikhauna -- paaarapharamayaaanasa
11. **Packaging** shaikhauna -- bailada aiba daepalayamaenata

manae raaakhauna:

"Unreal Engine Development halao aikatai shakataishaaalaili haaataiyaaara aitari bayabahaaara karae aapanai yaekaonao dharanaera gaema taairai karatae paaarabaena"

anaushailana (Exercises)

anaushaiilana 1: Blueprint Movement

aikatai Player Movement Blueprint taairai karauna

anaushaiilana 2: C++ Actor

aikatai C++ Actor taairai karauna

anaushaiilana 3: Material

aikatai mayaaataeraiyaaala taairai karauna

anaushaiilana 4: Niagara Effect

aikatai Niagara Effect taairai karauna

anaushaiilana 5: AI

aikatai AI Controller taairai karauna

anaushaiilana 6: UI

aikatai Widget Blueprint taairai karauna

anaushaiilana 7: Complete Game

aikatai samapauurana chhaota gaema taairai karauna

****GAME DEVELOPMENT MASTERBOOK samapauurana!****

****anayaaanaya adhayaaaya.****

- [Chapter 15: LEVEL DESIGN](15_level_design.md)
- [Chapter 16: CHARACTER DESIGN](16_character_design.md)
- [Chapter 17: ENEMY DESIGN](17_enemy_design.md)
- [Chapter 18: GAME ART PIPELINE](18_art_pipeline.md)
- [Chapter 19: UNITY DEVELOPMENT](19_unity.md)
- [Chapter 20: UNREAL ENGINE DEVELOPMENT](20_unreal.md)

PART 21

adhayaaaya 21: gaema paraogaraaamai aarakaitaekachaaara

adhayaaaya 21: gaema paraogaraaamai aarakaitaekachaaara

shaekhaara udadaeshaya

aii adhayaaayae aamaraaa gaema daebhaelapamaenatae sabachaeyae gaurautabapauurana baissaya -- paraogaraaamai aarakaitaekachaaara naiyae aalaochanaaa karaba aikatai bhaaalao aarakaitaekachaaara chhaadaaaa gaema daebhaelapamaenata asamabhaba aii adhayaaaya shaessae aapanai jaaanabaena kaiibhaaabaee kalaina kaoda laikhabaena, kaiibhaaabaee saisataema madaulaaara karabaena, aiba kaona kaona daijaaaina payaataaarana gaema daebhaelapamaenatae bayabahaarita haya

21.1 kalaina kaoda parainasaipala phara gaemasa

kalaina kaoda kaii?

kalaina kaoda halao aimana kaoda yaaa padaaaa sahaja, baojhaaa sahaja aiba maeinataeina karaaa sahaja gaema daebhaelapamaenatae kalaina kaoda atayanata gaurautabapauurana kaaarana gaema kaoda baaarabaaara paraibarataita haya

kalaina kaodaera mauula naiitaisamauuha:

****1. naaamakarana kanabhaenashana (Naming Convention)****

```
// khaaaraaapa naaamakarana
int hp = 100;
bool isD = true;
void U() { }

// bhaaalao naaamakarana
int currentHealth = 100;
bool isDead = true;
void UpdatePlayerPosition() { }
```

****2. aikatai phaaashana aikatai kaaaja karabae (Single Responsibility)****

```
// khaaaraaapa - aikatai phaaashanae anaeka kaaaja
void ProcessPlayer()
{
```

```

    MovePlayer();
    CheckHealth();
    UpdateUI();
    PlaySound();
    CheckCollision();
}

// bhaaalao - parataitai phaaashana aikatai kaaaja
void MovePlayer() { }
void CheckPlayerHealth() { }
void UpdateHealthUI() { }
void PlayHurtSound() { }
void CheckPlayerCollision() { }

```

****3. mayaaaajaika namabara aidaiyae chalauna (No Magic Numbers)****

```

// khaaaraapa
if (player.health <= 20) { }

// bhaaalao
const float CRITICAL_HEALTH_THRESHOLD = 20f;
if (player.health <= CRITICAL_HEALTH_THRESHOLD) { }

```

****4. kamaenata sataaaila****

```

// khaaaraapa kamaenata
// aii phaaashanatai palaeyaaarakae maubha karae
void MovePlayer() { }

// bhaaalao kamaenata - kaena aitai karaaa hayaechhae balauna
// palaeyaaakae samautha maubhamaenata daitae samauthadaemapai bayabahaaara karaaa hayaech
void SmoothMovePlayer() { }

```

21.2 madaulaaara saisataema daijaaaina

madaulaaara saisataema maaanae gaemaera parataitai asha sabaaadhainabhaaabaee kaaaja karabae

madaulaaara daijaaainaera saubaidhaaa:

- baaaga phaikasa karaaa sahaja
- natauna phaichaaara yaoga karaaa sahaja
- taimaera sadasayaraaa aikai saaathae kaaaja karatae paaarae
- kaoda raiiujabala haya

madaulaaara saisataemaera udaaaharana:

```

// parataitai saisataema sabaaadhaina
public class HealthSystem
{
    private float currentHealth;
    private float maxHealth;

    public void TakeDamage(float damage) { }
    public void Heal(float amount) { }
    public bool IsAlive() => currentHealth > 0;
}

```

```

public class MovementSystem
{
    private float speed;
    private CharacterController controller;

    public void Move(Vector3 direction) { }
    public void Jump() { }
}

public class CombatSystem
{
    private Weapon currentWeapon;

    public void Attack() { }
    public void Reload() { }
}

```

21.3 sataeta maeshaina payaaataaarana (State Machine Pattern)

sataeta maeshaina halao gaema daebhaelapamaenatae sabachaeyae baeshai bayabaharita payaaataaarana aital gaema abajaekataera baibhainana abasathaaa (state) mayaaanaeja karae

sataeta maeshaina daaayaaagaraama:

```

Idle
(alasa)
    shatarau daekhaaa gaela
Patrol
(paetaraola)
    shatarau sanaakata halao
Chase
(taaadaaaa)
    shataraura kaaachhae paauchhaaalao
Attack
(aakaramana)
    shatarau paaalaiyae gaela
Flee
(paaalaaanao)

```

sataeta maeshaina kaoda:

```

// sataeta inataaaraphaesa
public interface IState
{
    void Enter();
    void Update();
    void Exit();
}

// sataeta maeshaina
public class StateMachine
{
    private IState currentState;

    public void ChangeState(IState newState)
    {
        currentState?.Exit();
        currentState = newState;
    }
}

```



```

        currentState.Enter();
    }

    public void Update()
    {
        currentState?.Update();
    }
}

// payaaataraola sataeta
public class PatrolState : IState
{
    private Enemy enemy;
    private Transform[] waypoints;
    private int currentWaypointIndex;

    public PatrolState(Enemy enemy, Transform[] waypoints)
    {
        this.enemy = enemy;
        this.waypoints = waypoints;
    }

    public void Enter()
    {
        enemy.speed = 2f;
        Debug.Log("paetaraola shaurau halao");
    }

    public void Update()
    {
        // auyaepayaenataera daikae yaaaauyaaa
        Transform target = waypoints[currentWaypointIndex];
        enemy.transform.position = Vector3.MoveTowards(
            enemy.transform.position,
            target.position,
            enemy.speed * Time.deltaTime
        );

        // auyaepayaenatae paauchhaaalee parabarataii auyaepayaenatae yaaaau
        if (Vector3.Distance(enemy.transform.position, target.position) < 0.5f)
        {
            currentWaypointIndex = (currentWaypointIndex + 1) % waypoints.Length;
        }

        // shatarau sanaakata halae chaeja sataetae yaaaau
        if (enemy.DetectPlayer())
        {
            enemy.stateMachine.ChangeState(new ChaseState(enemy));
        }
    }

    public void Exit()
    {
        Debug.Log("paetaraola shaessa halao");
    }
}

// chaeja sataeta
public class ChaseState : IState
{
    private Enemy enemy;

    public ChaseState(Enemy enemy)
    {

```

```

        this.enemy = enemy;
    }

    public void Enter()
    {
        enemy.speed = 5f;
        Debug.Log("shataraukae taaadaaa karaaa shaurau halao!");
    }

    public void Update()
    {
        // palaeyaaarakae taaadaaa karao
        Transform player = enemy.targetPlayer;
        enemy.transform.position = Vector3.MoveTowards(
            enemy.transform.position,
            player.position,
            enemy.speed * Time.deltaTime
        );

        // aakaramanaera paraisaiimaaaya paauchhaaala
        if (Vector3.Distance(enemy.transform.position, player.position) < 2f)
        {
            enemy.stateMachine.ChangeState(new AttackState(enemy));
        }

        // shatarau adarishaya halae paetaraolae phairae yaaaau
        if (!enemy.DetectPlayer())
        {
            enemy.stateMachine.ChangeState(new PatrolState(enemy, enemy.waypoints));
        }
    }

    public void Exit()
    {
        Debug.Log("taaadaaa banadha halao");
    }
}

// aakaramana sataeta
public class AttackState : IState
{
    private Enemy enemy;
    private float attackCooldown = 1f;
    private float lastAttackTime;

    public AttackState(Enemy enemy)
    {
        this.enemy = enemy;
    }

    public void Enter()
    {
        Debug.Log("aakaramana shaurau!");
    }

    public void Update()
    {
        // aakaramanaera kauladaauna chaeka
        if (Time.time - lastAttackTime >= attackCooldown)
        {
            enemy.PerformAttack();
            lastAttackTime = Time.time;
        }
    }
}

```

```

// palaeyaaara dauurae gaelae chaeja sataetae phairae yaaaau
float distanceToPlayer = Vector3.Distance(
    enemy.transform.position,
    enemy.targetPlayer.position
);

if (distanceToPlayer > 5f)
{
    enemy.stateMachine.ChangeState(new ChaseState(enemy));
}
}

public void Exit()
{
    Debug.Log("aakaramana shaessa");
}
}

// ainaimai kalaaasa
public class Enemy : MonoBehaviour
{
    public StateMachine stateMachine;
    public Transform[] waypoints;
    public Transform targetPlayer;
    public float speed = 2f;

    void Start()
    {
        stateMachine = new StateMachine();
        stateMachine.ChangeState(new PatrolState(this, waypoints));
    }

    void Update()
    {
        stateMachine.Update();
    }

    public bool DetectPlayer()
    {
        float distance = Vector3.Distance(transform.position, targetPlayer.position);
        return distance < 10f;
    }

    public void PerformAttack()
    {
        Debug.Log("aakaramana karalaaama!");
    }
}

```

21.4 ibhaenata saisataema payaaataaarana (Event System Pattern)

ibhaenata saisataema daiyae aapanai bhaaaaitauyaaalai aimanabhaaaabae saisataema taairai karatae paaaraena yaekhaaanae aikatai saisataema anaya saisataemakae ibhaenata paaathaaaya aiba saei ibhaenataera upara bhaitatai karae saisataemagaulao kaaaja karae

ibhaenata saisataema kaoda:

```

// ibhaenata saisataema
public class EventManager

```

```

{
    // saingagalatana payaaataaarana
    private static EventManager instance;
    public static EventManager Instance
    {
        get
        {
            if (instance == null)
                instance = new EventManager();
            return instance;
        }
    }

    // ibhaenata daelaigaeta
    public delegate void GameEvent();
    public delegate void GameEvent<T>(T data);

    // ibhaenata daikashanaaarai
    private Dictionary<string, Delegate> events = new Dictionary<string, Delegate>();

    // ibhaenata saaabasakaraaaiba karao
    public void Subscribe(string eventName, Delegate handler)
    {
        if (events.ContainsKey(eventName))
            events[eventName] = Delegate.Combine(events[eventName], handler);
        else
            events[eventName] = handler;
    }

    // ibhaenata aanasaaabasakaraaaiba karao
    public void Unsubscribe(string eventName, Delegate handler)
    {
        if (events.ContainsKey(eventName))
            events[eventName] = Delegate.Remove(events[eventName], handler);
    }

    // ibhaenata taraigaaara karao
    public void TriggerEvent(string eventName)
    {
        if (events.ContainsKey(eventName))
            events[eventName]?.DynamicInvoke();
    }

    public void TriggerEvent<T>(string eventName, T data)
    {
        if (events.ContainsKey(eventName))
            events[eventName]?.DynamicInvoke(data);
    }
}

// ibhaenata naaama
public static class GameEvents
{
    public const string PLAYER_DIED = "PlayerDied";
    public const string ENEMY_KILLED = "EnemyKilled";
    public const string ITEM_COLLECTED = "ItemCollected";
    public const string LEVEL_COMPLETED = "LevelCompleted";
    public const string GAME_PAUSED = "GamePaused";
}

// ibhaenata bayabahaaaraera udaaaharana
public class ScoreManager : MonoBehaviour
{

```

```

private int score = 0;

void Start()
{
    // ainaimai maaaraaara ibhaenatae saaabasakaraaiba karao
    EventManager.Instance.Subscribe(GameEvents.ENEMY_KILLED, OnEnemyKilled);
    EventManager.Instance.Subscribe(GameEvents.ITEM_COLLECTED, OnItemCollected);
}

void OnEnemyKilled()
{
    score += 100;
    Debug.Log($"sakaora: {score}");
}

void OnItemCollected()
{
    score += 50;
}

void OnDestroy()
{
    EventManager.Instance.Unsubscribe(GameEvents.ENEMY_KILLED, OnEnemyKilled);
    EventManager.Instance.Unsubscribe(GameEvents.ITEM_COLLECTED, OnItemCollected);
}
}

```

21.5 mayaaanaejaaara payaaataaarana (Manager Pattern)

mayaaanaejaaara payaaataaarana halao gaema daebhaelapamaenatae sabachaeyae baeshai bayabaharita payaaataaarana parataitai mayaaanaejaaara aikatai nairadaissata kaaaja paraichaaalanaaa karae

mayaaanaejaaara payaaataaarana kaoda:

```

// gaema mayaaanaejaaara (saingagalatana)
public class GameManager : MonoBehaviour
{
    private static GameManager instance;
    public static GameManager Instance => instance;

    // gaema sataeta
    public enum GameState { Menu, Playing, Paused, GameOver }
    public GameState CurrentState { get; private set; }

    // gaema daaataaa
    public int Score { get; private set; }
    public int Level { get; private set; }
    public bool IsGameOver { get; private set; }

    void Awake()
    {
        if (instance == null)
        {
            instance = this;
            DontDestroyOnLoad(gameObject);
        }
        else
        {
            Destroy(gameObject);
        }
    }
}

```

```

    }
}

public void ChangeState(GameState newState)
{
    CurrentState = newState;

    switch (newState)
    {
        case GameState.Playing:
            Time.timeScale = 1f;
            break;
        case GameState.Paused:
            Time.timeScale = 0f;
            break;
        case GameState.GameOver:
            HandleGameOver();
            break;
    }

    EventManager.Instance.TriggerEvent("GameStateChanged");
}

public void AddScore(int points)
{
    Score += points;
    EventManager.Instance.TriggerEvent("ScoreUpdated");
}

public void CompleteLevel()
{
    Level++;
    EventManager.Instance.TriggerEvent("LevelCompleted");
}

private void HandleGameOver()
{
    IsGameOver = true;
    Debug.Log("gaema shaessa!");
}
}

// adaiau mayaaanaejaaara
public class AudioManager : MonoBehaviour
{
    private static AudioManager instance;
    public static AudioManager Instance => instance;

    [System.Serializable]
    public class Sound
    {
        public string name;
        public AudioClip clip;
        [Range(0f, 1f)]
        public float volume;
        [Range(0.5f, 1.5f)]
        public float pitch;
        public bool loop;
    }

    public Sound[] sounds;
    private Dictionary<string, AudioSource> audioSources;

    void Awake()

```

```

{
    if (instance == null)
    {
        instance = this;
        DontDestroyOnLoad(gameObject);
    }
    else
    {
        Destroy(gameObject);
        return;
    }

    audioSources = new Dictionary<string, AudioSource>();

    foreach (Sound s in sounds)
    {
        AudioSource source = gameObject.AddComponent<AudioSource>();
        source.clip = s.clip;
        source.volume = s.volume;
        source.pitch = s.pitch;
        source.loop = s.loop;
        audioSources[s.name] = source;
    }
}

public void PlaySound(string soundName)
{
    if (audioSources.ContainsKey(soundName))
    {
        audioSources[soundName].Play();
    }
}

public void StopSound(string soundName)
{
    if (audioSources.ContainsKey(soundName))
    {
        audioSources[soundName].Stop();
    }
}

public void PlayMusic(string soundName)
{
    StopAllMusic();
    PlaySound(soundName);
}

private void StopAllMusic()
{
    foreach (var source in audioSources.Values)
    {
        source.Stop();
    }
}
}

// laebhaela mayaaanaejaaara
public class LevelManager : MonoBehaviour
{
    private static LevelManager instance;
    public static LevelManager Instance => instance;

    public string[] levelNames;

```

```

private int currentLevelIndex;

void Awake()
{
    if (instance == null)
    {
        instance = this;
        DontDestroyOnLoad(gameObject);
    }
    else
    {
        Destroy(gameObject);
    }
}

public void LoadLevel(int levelIndex)
{
    if (levelIndex < levelNames.Length)
    {
        currentLevelIndex = levelIndex;
        SceneManager.LoadScene(levelNames[levelIndex]);
    }
}

public void LoadNextLevel()
{
    LoadLevel(currentLevelIndex + 1);
}

public void ReloadCurrentLevel()
{
    LoadLevel(currentLevelIndex);
}

public void LoadMainMenu()
{
    SceneManager.LoadScene("MainMenu");
}
}

```

21.6 daaataaaa-daraibhaena daijaaaina (Data-Driven Design)

daaataaaa-daraibhaena daijaaaina maaanae gaemaera tathaya (data) kaoda thaekae aalaaadaaaa raaakhaaa aitai karalae aapanai kaoda naaa paraibaratana karaeau gaemaera tathaya paraibaratana karatae paaarabaena

daaataaaa-daraibhaena daijaaainaera udaaaharana:

```

// JSON phaaaila thaekae tathaya padaaaa
[System.Serializable]
public class WeaponData
{
    public string weaponName;
    public int damage;
    public float fireRate;
    public int ammoCapacity;
    public float reloadTime;
}

[System.Serializable]

```



```

public class WeaponDatabase
{
    public WeaponData[] weapons;
}

public class WeaponManager : MonoBehaviour
{
    public TextAsset weaponDatabaseJSON;
    private WeaponDatabase weaponDatabase;

    void Start()
    {
        weaponDatabase = JsonUtility.FromJson<WeaponDatabase>(weaponDatabaseJSON.text);
    }

    public WeaponData GetWeapon(string weaponName)
    {
        foreach (WeaponData weapon in weaponDatabase.weapons)
        {
            if (weapon.weaponName == weaponName)
                return weapon;
        }
        return null;
    }
}

```

21.7 sakaraipataebala abajaekatasa kanasaepata (Scriptable Objects)

sakaraipataebala abajaekatasa halao Unity-aira aikatai phaichaaara yaaa daaataaa sataora karatae bayabaharita haya

sakaraipataebala abajaekatasa kaoda:

```

// sakaraipataebala abajaekata taairai karao
[CreateAssetMenu(fileName = "NewWeapon", menuName = "Game/Weapon Data")]
public class WeaponDataSO : ScriptableObject
{
    public string weaponName;
    public Sprite icon;
    public int damage;
    public float fireRate;
    public int ammoCapacity;
    public float reloadTime;
    public AudioClip fireSound;
    public GameObject bulletPrefab;
}

// sakaraipataebala abajaekata bayabahaaara karao
public class Weapon : MonoBehaviour
{
    public WeaponDataSO weaponData;
    private float lastFireTime;

    public void Fire()
    {
        if (Time.time - lastFireTime >= weaponData.fireRate)
        {
            // baulaeta taairai karao
            Instantiate(weaponData.bulletPrefab, transform.position, transform.rotation);
        }
    }
}

```

```

        // saaaunada palae karao
        AudioManager.Instance.PlaySound(weaponData.fireSound.name);

        lastFireTime = Time.time;
    }
}
}

```

21.8 saebha aarakaitaekachaaara (Save Architecture)

saebha aarakaitaekachaaara halao gaemaera paragaraesa saebha karaaara padadhatai

saebha saisataema kaoda:

```

// saebha daaataaa
[System.Serializable]
public class SaveData
{
    public string playerName;
    public int playerLevel;
    public int playerScore;
    public float[] playerPosition;
    public string lastSaveTime;
    public int[] inventory;
    public Dictionary<string, bool> unlockedLevels;
}

// saebha saisataema
public class SaveSystem
{
    private static string savePath = Application.persistentDataPath + "/savegame.json";

    public static void SaveGame(SaveData data)
    {
        data.lastSaveTime = System.DateTime.Now.ToString();

        string json = JsonUtility.ToJson(data, true);
        File.WriteAllText(savePath, json);

        Debug.Log("gaema saebha halao!");
    }

    public static SaveData LoadGame()
    {
        if (File.Exists(savePath))
        {
            string json = File.ReadAllText(savePath);
            SaveData data = JsonUtility.FromJson<SaveData>(json);
            Debug.Log("gaema laoda halao!");
            return data;
        }
        else
        {
            Debug.Log("saebha phaaaila paaaauyaaa yaaayanai!");
            return null;
        }
    }

    public static void DeleteSave()

```

```

    {
        if (File.Exists(savePath))
        {
            File.Delete(savePath);
            Debug.Log("saebha dailaita halao!");
        }
    }

    public static bool HasSaveFile()
    {
        return File.Exists(savePath);
    }
}

// saebha mayaaanaejaaara
public class SaveManager : MonoBehaviour
{
    private static SaveManager instance;
    public static SaveManager Instance => instance;

    public SaveData currentSaveData;

    void Awake()
    {
        if (instance == null)
        {
            instance = this;
            DontDestroyOnLoad(gameObject);
        }
        else
        {
            Destroy(gameObject);
        }
    }

    public void SaveCurrentGame()
    {
        currentSaveData = new SaveData();
        currentSaveData.playerName = "Player1";
        currentSaveData.playerLevel = GameManager.Instance.Level;
        currentSaveData.playerScore = GameManager.Instance.Score;

        SaveSystem.SaveGame(currentSaveData);
    }

    public void LoadLastGame()
    {
        currentSaveData = SaveSystem.LoadGame();

        if (currentSaveData != null)
        {
            GameManager.Instance.Level = currentSaveData.playerLevel;
            GameManager.Instance.Score = currentSaveData.playerScore;
        }
    }
}

```

21.9 daipadenadasai mayaaanaejamaenata (Dependency Management)

daipenadaenasai mayaaanaejamaenata halao kaodaera nairabharashaiilataaa paraichaaalanaaa karaaara padadhatai

daipenadaenasai inajaekashana:

```
// inataaaraphaesa
public interface IDamageable
{
    void TakeDamage(float damage);
    bool IsAlive { get; }
}

// daipenadaenasai inajaekashana
public class CombatSystem : MonoBehaviour
{
    private IDamageable damageable;

    // kanasataraaakatarera maaadhayamae daipenadaenasai inajaekashana
    public void Initialize(IDamageable target)
    {
        damageable = target;
    }

    public void Attack(float damage)
    {
        if (damageable != null && damageable.IsAlive)
        {
            damageable.TakeDamage(damage);
        }
    }
}

// palaeyaaara kalaaasa
public class Player : MonoBehaviour, IDamageable
{
    public float health = 100f;

    public bool IsAlive => health > 0;

    public void TakeDamage(float damage)
    {
        health -= damage;
        Debug.Log($"palaeyaaaraera haelatha: {health}");
    }
}

// bayabahaaaraera udaaaharana
public class EnemyAI : MonoBehaviour
{
    private CombatSystem combatSystem;

    void Start()
    {
        combatSystem = new CombatSystem();
        Player player = FindObjectOfType<Player>();
        combatSystem.Initialize(player);
    }

    public void AttackPlayer()
    {
        combatSystem.Attack(10f);
    }
}
```

21.10 aarakaitaekachaaara daaayaaagaraaama

GAME ARCHITECTURE		
GameManager (saingagalatana)	AudioManager (saingagalatana)	LevelManager (saingagalatana)
EVENT MANAGER (ibhaenata saisataema)		
Subscribe / Unsubscribe		
Trigger Event		
GAME SYSTEMS (gaema saisataema)		
Health System	Movement System	Combat System
Inventory System	AI System	UI System
DATA LAYER (daaataaa satara)		
Scriptable Objects	JSON Files	Save System

21.11 parayaaakataisa aikasaarasaaaija

aikasaarasaaaija 1: sataeta maeshaina taairai karao

aikatai palaeyaaara kayaaaraekataaaraera janaya sataeta maeshaina taairai karao yaaara sataetagaualao halao:

- Idle (alasa)
- Walking (haaataaaa)
- Running (daaudaaanao)
- Jumping (laaaphaaanao)
- Swimming (saaataaara)

aikasaarasaaaija 2: ibhaenata saisataema taairai karao

aikatai ibhaenata saisataema taairai karao yaaa naimanalaikhaita ibhaenata hayaaanadala karabae:

- PlayerDied
- ItemCollected
- LevelCompleted
- GamePaused

aikasaarasaaaija 3: saebha saisataema taairai karao

aikatai saebha saisataema taairai karao yaaa naimanalaikhaita tathaya saebha karabae:

- palaeyaaaraera naaama
- palaeyaaaraera laebhaela
- palaeyaaaraera sakaora
- palaeyaaaraera abasathaaana
- inabhaenatarai taaalaikaaa

21.12 bhaula yaaa aidaiyae chalatae habae

bhaua 1: saingagalatana payaaataaaranaera atairaikata bayabahaaara

```
// khaaaraaapa - saba kaichhau saingagalatana karaaa
public class PlayerManager : MonoBehaviour { }
public class EnemyManager : MonoBehaviour { }
public class UIManager : MonoBehaviour { }
public class SoundManager : MonoBehaviour { }
public class AnimationManager : MonoBehaviour { }
```

bhaua 2: taaaita kaaapalai (Tight Coupling)

```
// khaaaraaapa - kalaaasagaulao aikae aparaera aupara nairabharashaiila
public class Player
{
    public EnemyManager enemyManager;
    public UIManager uiManager;
    public AudioManager audioManager;

    public void TakeDamage()
    {
        health -= damage;
        enemyManager.CheckAggro();
        uiManager.UpdateHealthBar(health);
        audioManager.PlayHurtSound();
    }
}
```

bhaua 3: bada kalaaasa taairai karaaa

```
// khaaaraaapa - aikatai kalaaasae saba kaichhau
public class GameManager : MonoBehaviour
{
    public void HandlePlayerMovement() { }
    public void HandleCombat() { }
    public void HandleUI() { }
    public void HandleAudio() { }
    public void HandleAI() { }
    public void HandleInventory() { }
    public void HandleQuests() { }
    public void HandleSave() { }
}
```

21.13 taipasa au paraaamarasha

taipasa 1: saingagalatana payaaataaarana bayabahaaara karao

gaema daebhaelapamaenatae saingagalatana payaaataaarana atayanata upayaogaii aitai naishachaita karae yae aikatai kalaaasaera aikatai maaatara inasatayaaanasa thaaakabae

taipasa 2: ibhaenata saisataema bayabahaaara karao

ibhaenata saisataema daiyae aapanai saisataemagaulaokae sabaaadhaina raaakhatae paaarabaena aitai kaodakae kalaina raaakhae

taipasa 3: daaataaa-daraibhaena daijaaaina anausarana karao

gaemaera tathaya kaoda thaekae aalaaadaaa raaakhauna aitai daibaaagai aiba madaiphaikaeshana sahaja karae

taipasa 4: sakaraipataebala abajaekatasa bayabahaaara karao

Unity-aira sakaraipataebala abajaekatasa phaichaaara bayabahaaara karae daaataaa mayaaanaeja karauna

taipasa 5: madaulaaara kaoda laekhao

parataitai saisataema sabaaadhaiina raaakhauna aikatai saisataemaera paraibaratana anaya saisataemae parabhaaaba phaelatae naebae

21.14 saarasakassaepa

aaii adhayaaayae aamaraaa gaema paraogaraaamai aarakaitaekachaaaraera gaurautabapauurana baissayagaulao naiyae aalaochanaaa karaechhai mauula payaenatagaulao halao:

1. ****kalaina kaoda****: padaaa sahaja aiba maeinataeina karaaa sahaja kaoda laekhao
2. ****madaulaaara saisataema****: parataitai saisataema sabaaadhaiina raaakhao
3. ****sataeta maeshaina****: gaema abajaekataera baibhainana abasathaaa paraichaaalanaaa karao
4. ****ibhaenata saisataema****: saisataemagaulaokae sabaaadhaiina raaakhatae ibhaenata bayabahaaara karao
5. ****mayaaanaejaaara payaataaarana****: parataitai mayaaanaejaaara aikatai nairadaissata kaaaja paraichaaalanaaa karabae
6. ****daaataaa-daraibhaena daijaaaina****: gaemaera tathaya kaoda thaekae aalaaadaaa raaakhao
7. ****saebha aarakaitaekachaaara****: gaemaera paragaresa saebha karao
8. ****daipadenadaenasai mayaaanaejamaenata****: kaodaera nairabharashaiilataaa paraichaaalanaaa karao

aaii payaataaaranagaulao anausarana karalae aapanaaara gaema kaoda kalaina, madaulaaara aiba maeinataeinaebala habae

****parabarataii adhayaaaya: gaema AI -- kaiibhaaaba gaemae karitaraima baudadhaimatataaa bayabahaaara karabaena****

PART 22

adhayaaaya 22: gaema AI

adhayaaaya 22: gaema AI

shaekhaara udadaeshaya

aai adhayaaayae aamaraaa gaema daebhaelapamaenatae karitaraima baudadhaimatataaa (AI) naiyae aalaochanaaa karaba AI halao gaemaera sabachaeyae gaurautabapauurana asha yaaa khaelaoyaaadadaera abhaijanyataaakae samaridadha karae aii adhayaaaya shaessae aapanai jaaanabaena kaiibhaaabaee enemy AI taairai karabaena, kaiibhaaabaee pathfinding bayabahaaara karabaena, aiba baibhainana dharanaera gaemae AI kaiibhaaabaee kaaaja karae

22.1 ainaimai AI baihaebhaiyaaara (Enemy AI Behaviors)

ainaimai AI-aira baibhainana aacharana:

****1. paetaraola (Patrol)****

paetaraola maaanae ainaimai nairadhaaaraita pathae ghaurae ghaurae chalaee

```
public class PatrolBehavior : MonoBehaviour
{
    public Transform[] waypoints;
    public float speed = 2f;
    private int currentWaypointIndex = 0;

    void Update()
    {
        Transform target = waypoints[currentWaypointIndex];

        // auyaepayaenataera daikae yaaaau
        transform.position = Vector3.MoveTowards(
            transform.position,
            target.position,
            speed * Time.deltaTime
        );

        // auyaepayaenatae paauchhaaalaee parabarataaii auyaepayaenatae yaaaau
        if (Vector3.Distance(transform.position, target.position) < 0.5f)
        {
            currentWaypointIndex = (currentWaypointIndex + 1) % waypoints.Length;
        }
    }
}
```


****2. saaracha (Search)****

saaracha maaanae ainimai khaoja karachhae palaeyaaarakae

```
public class SearchBehavior : MonoBehaviour
{
    public float searchRadius = 10f;
    public float searchDuration = 5f;
    private float searchTimer;

    void StartSearch()
    {
        searchTimer = searchDuration;
    }

    void Update()
    {
        if (searchTimer > 0)
        {
            searchTimer -= Time.deltaTime;

            // baritataaakaaara pathae ghaurae khaoja karao
            transform.Rotate(Vector3.up, 90f * Time.deltaTime);

            // palaeyaaarakae khaujae paelae
            if (DetectPlayer())
            {
                // chaeja shaurau karao
                StartChase();
            }
        }
    }

    bool DetectPlayer()
    {
        // saenasara raenyajae palaeyaaara aachhae kainaaa chaeka karao
        Collider[] colliders = Physics.OverlapSphere(transform.position, searchRadius);
        foreach (Collider col in colliders)
        {
            if (col.CompareTag("Player"))
                return true;
        }
        return false;
    }
}
```

****3. daitaekashana (Detection)****

daitaekashana maaanae ainimai palaeyaaarakae sanaakata karachhae

```
public class DetectionBehavior : MonoBehaviour
{
    public float detectionRange = 15f;
    public float fieldOfView = 120f;
    public LayerMask playerLayer;

    public bool CanSeePlayer()
    {
        // palaeyaaaraera daikae kayaaasata karao
        Vector3 directionToPlayer = player.position - transform.position;
        float distanceToPlayer = Vector3.Distance(transform.position, player.position);

        // daitaekashana raenyajae aachhae kainaaa chaeka karao
    }
}
```

```

    if (distanceToPlayer <= detectionRange)
    {
        // phailada apha bhaiutae aachhae kainaaa chaeka karao
        float angle = Vector3.Angle(transform.forward, directionToPlayer);
        if (angle <= fieldOfView / 2f)
        {
            // abasataraaakashana chaeka karao
            if (!Physics.Raycast(transform.position, directionToPlayer, distanceToPlay
            {
                return true;
            }
        }
    }
    return false;
}
}

```

****4. ayaaalaaarata (Alert)****

ayaaalaaarata maaanae ainaimai sataraka hayae gaechhae

```

public class AlertBehavior : MonoBehaviour
{
    public float alertDuration = 3f;
    private float alertTimer;
    private bool isAlerted = false;

    public void OnPlayerSpotted()
    {
        isAlerted = true;
        alertTimer = alertDuration;

        // ayaaalaaarata saaaunada palae karao
        AudioManager.Instance.PlaySound("AlertSound");

        // anaya ainaimaidaera jaaanaaaaau
        NotifyOtherEnemies();
    }

    void Update()
    {
        if (isAlerted)
        {
            alertTimer -= Time.deltaTime;

            if (alertTimer <= 0)
            {
                isAlerted = false;
                // naramaaala sataetae phairae yaaaau
            }
        }
    }

    void NotifyOtherEnemies()
    {
        // salagana ainaimaidaera jaaanaaaaau
        Collider[] nearbyEnemies = Physics.OverlapSphere(transform.position, 20f, enemyLayer);
        foreach (Collider enemy in nearbyEnemies)
        {
            enemy.GetComponent<AlertBehavior>()?.OnPlayerSpotted();
        }
    }
}

```

****5. chaeja (Chase)****

chaeja maaanae ainaimai palaeyaaarakae taaadaaa karachhae

```
public class ChaseBehavior : MonoBehaviour
{
    public float chaseSpeed = 5f;
    public float chaseRange = 20f;
    public Transform player;

    void Update()
    {
        float distanceToPlayer = Vector3.Distance(transform.position, player.position);

        if (distanceToPlayer <= chaseRange)
        {
            // palaeyaaarakae taaadaaa karao
            transform.position = Vector3.MoveTowards(
                transform.position,
                player.position,
                chaseSpeed * Time.deltaTime
            );

            // palaeyaaaraera daikae taaakaaaau
            transform.LookAt(player.position);
        }
    }
}
```

****6. aakaramana (Attack)****

aakaramana maaanae ainaimai palaeyaaarakae aakaramana karachhae

```
public class AttackBehavior : MonoBehaviour
{
    public float attackDamage = 10f;
    public float attackRange = 2f;
    public float attackCooldown = 1f;
    private float lastAttackTime;

    void Update()
    {
        float distanceToPlayer = Vector3.Distance(transform.position, player.position);

        if (distanceToPlayer <= attackRange)
        {
            if (Time.time - lastAttackTime >= attackCooldown)
            {
                Attack();
                lastAttackTime = Time.time;
            }
        }
    }

    void Attack()
    {
        // aakaramana ayaanaimaeshana palae karao
        animator.SetTrigger("Attack");

        // palaeyaaaraera haelatha kamaaaaau
        player.GetComponent<PlayerHealth>().TakeDamage(attackDamage);

        // aakaramana saaaunada palae karao
        AudioManager.Instance.PlaySound("AttackSound");
    }
}
```

```

    }
}

```

7. phalai (Flee)

phalai maaanae ainimai paaalaiyae yaaachachhae

```

public class FleeBehavior : MonoBehaviour
{
    public float fleeSpeed = 6f;
    public float fleeDistance = 15f;

    public void Flee()
    {
        // palaeyaaaraera baiparaiita daikae yaaaau
        Vector3 fleeDirection = (transform.position - player.position).normalized;
        Vector3 fleeTarget = transform.position + fleeDirection * fleeDistance;

        // naebhaigaeshana saisataema bayabahaaara karao
        agent.SetDestination(fleeTarget);
        agent.speed = fleeSpeed;
    }
}

```

8. ritaaraana (Return)

ritaaraana maaanae ainimai taaara araijainaaala pajaishanae phairae yaaachachhae

```

public class ReturnBehavior : MonoBehaviour
{
    public Transform originalPosition;
    public float returnSpeed = 3f;

    public void ReturnToOriginalPosition()
    {
        // araijainaaala pajaishanae phairae yaaaau
        agent.SetDestination(originalPosition.position);
        agent.speed = returnSpeed;

        // araijainaaala pajaishanae paauchhaaala
        if (Vector3.Distance(transform.position, originalPosition.position) < 1f)
        {
            // paetaraola sataetae phairae yaaaau
            ChangeState(new PatrolState());
        }
    }
}

```

22.2 AI sataeta maeshaina daaayaaagaraama

```

AI STATE MACHINE

IDLE
(alasa)
    samaya paaara halao
PATROL <
(paetaraola)
    palaeyaaara sanaakata halao
DETECT
(sanaakata)
    naishachaita halao

```

```

ALERT
(sataraka)
    anaya ainaimaidaera jaaanaaalao
CHASE
(taaadaaaa) <
    aakaramanaera paraisaiimaaaya paauchhaaalao
ATTACK
(aakaramana)
    palaeyaaara dauurae gaela
    haelatha kama halao
FLEE
(paaalaaanao)
    nairaaapada halao
RETURN
(phairae yaaaau)

```

22.3 baihaebhaiyara tarai (Behavior Tree)

baihaebhaiyara tarai halao AI saisataemaera aikatai janaparaiya padadhatai aitari sataeta maeshainaera chaeyae baeshai phalaekasaibala

baihaebhaiyara taraira gathana:

```

        BEHAVIOR TREE
    ROOT (rauta)
        sabachaeyae uparaera naoda
    SEQUENCE      SELECTOR      PARALLEL
(karamaika)      (baaachhaai)    (samaaanataraaaala)
saba kaaaja      aikatai kaaaja  aikasaaathae
karatae habae    karatae habae  karatae habae
    ACTION      CONDITION      DECORATOR
(karama)        (sharata)      (sajajaaa)
kaaaja karao    chaeka karao    paraibaratana
                                karao

```

baihaebhaiyara tarai kaoda:

```

// naoda sataeta
public enum NodeState { Running, Success, Failure }

// baihaebhaiyara tarai naoda
public abstract class BTreeNode
{
    public abstract NodeState Evaluate();
}

// saikaoyaenasa naoda
public class BTSequence : BTreeNode
{
    private List<BTreeNode> children;

    public BTSequence(List<BTreeNode> children)
    {
        this.children = children;
    }

    public override NodeState Evaluate()
    {

```

```
        foreach (BTNode child in children)
        {
            NodeState state = child.Evaluate();

            if (state != NodeState.Success)
                return state;
        }
        return NodeState.Success;
    }
}

// sailaekatarata naoda
public class BTSelector : BTNode
{
    private List<BTNode> children;

    public BTSelector(List<BTNode> children)
    {
        this.children = children;
    }

    public override NodeState Evaluate()
    {
        foreach (BTNode child in children)
        {
            NodeState state = child.Evaluate();

            if (state != NodeState.Failure)
                return state;
        }
        return NodeState.Failure;
    }
}

// aikashana naoda
public class BTAction : BTNode
{
    private System.Func<NodeState> action;

    public BTAction(System.Func<NodeState> action)
    {
        this.action = action;
    }

    public override NodeState Evaluate()
    {
        return action();
    }
}

// kanadaishana naoda
public class BTCondition : BTNode
{
    private System.Func<bool> condition;

    public BTCondition(System.Func<bool> condition)
    {
        this.condition = condition;
    }

    public override NodeState Evaluate()
    {
        return condition() ? NodeState.Success : NodeState.Failure;
    }
}
```

```

}

// ainaimai AI baihaebhaiyara tarai
public class EnemyBehaviorTree
{
    private BTNode root;

    public EnemyBehaviorTree(Enemy enemy)
    {
        root = new BTSelector(new List<BTNode>
        {
            // saikaoyaenasa 1: palaeyaaaara daekhatae paaaralae taaadaaa karao
            new BTSequence(new List<BTNode>
            {
                new BTCondition(() => enemy.CanSeePlayer()),
                new BTAction(() =>
                {
                    enemy.ChasePlayer();
                    return NodeState.Running;
                })
            })),
            // saikaoyaenasa 2: paetaraola karao
            new BTSequence(new List<BTNode>
            {
                new BTCondition(() => !enemy.CanSeePlayer()),
                new BTAction(() =>
                {
                    enemy.Patrol();
                    return NodeState.Running;
                })
            })
        });
    }

    public void Update()
    {
        root.Evaluate();
    }
}

```

22.4 paaathaphaaainadai (Pathfinding)

A* ayaaalagaraidama

A* ayaaalagaraidama halao sabachaeyae janaparaiya paaathaphaaainadai ayaaalagaraidama aitai sabachaeyae chhaota patha khaujae baera karae

A* ayaaalagaraidamaera dhaaapasamauuha:

A* ALGORITHM

1. aupaena laisata aiba kalaojada laisata taaairai karao
2. sataaarata naodakae aupaena laisatae yaoga karao
3. yatakassana aupaena laisata khaaalai naei:
 - a. aupaena laisata thaekae sabachaeyae kama f sakaora auyaaalaaa naoda baera karao (current)
 - b. current kae kalaojada laisatae yaoga karao

- c. yadai current haya goal naoda, taaahalaе patha raitaaarana karao
- d. current aira saba neighbor aira janaya:
 - yadai neighbor kalaojada laisatae thaaakae, sakaipa karao
 - yadai neighbor aupaeana laisatae naaa thaaakae, yaoga karao
 - yadai natauna patha aagaera pathaera chaeyae bhaaalao haya, aapadaeta karao
- 4. patha raitaaarana karao

A* ayaaalagaraidama kaoda:

```
// A* naoda
public class AStarNode
{
    public Vector3 position;
    public float gCost; // sataaarata thaekae aii naoda parayanata dauurataba
    public float hCost; // aii naoda thaekae goal naoda parayanata dauurataba
    public float fCost => gCost + hCost; // maota kharacha
    public AStarNode parent;

    public AStarNode(Vector3 position)
    {
        this.position = position;
    }
}

// A* paaathaphaaainadaaara
public class AStarPathfinder
{
    public List<Vector3> FindPath(Vector3 start, Vector3 goal, List<Vector3> waypoints)
    {
        List<AStarNode> openList = new List<AStarNode>();
        List<AStarNode> closedList = new List<AStarNode>();

        // sataaarata naoda taairai karao
        AStarNode startNode = new AStarNode(start);
        startNode.gCost = 0;
        startNode.hCost = Vector3.Distance(start, goal);
        openList.Add(startNode);

        while (openList.Count > 0)
        {
            // sabachaeyae kama f sakaora auyaaalaaa naoda baera karao
            AStarNode currentNode = openList[0];
            for (int i = 1; i < openList.Count; i++)
            {
                if (openList[i].fCost < currentNode.fCost ||
                    (openList[i].fCost == currentNode.fCost &&
                     openList[i].hCost < currentNode.hCost))
                {
                    currentNode = openList[i];
                }
            }

            openList.Remove(currentNode);
            closedList.Add(currentNode);

            // goal naodae paauchhaaalaе patha raitaaarana karao
            if (currentNode.position == goal)
            {
                return RetracePath(currentNode);
            }
        }
    }
}
```



```

// parataibaeshaii naodagaulao parasaesa karao
foreach (Vector3 neighborPos in GetNeighbors(currentNode.position, waypoints))
{
    AStarNode neighbor = new AStarNode(neighborPos);

    if (closedList.Exists(n => n.position == neighbor.position))
        continue;

    float tentativeGCost = currentNode.gCost +
        Vector3.Distance(currentNode.position, neighbor.position);

    AStarNode existingNode = openList.Find(n => n.position == neighbor.position);

    if (existingNode == null)
    {
        neighbor.gCost = tentativeGCost;
        neighbor.hCost = Vector3.Distance(neighbor.position, goal);
        neighbor.parent = currentNode;
        openList.Add(neighbor);
    }
    else if (tentativeGCost < existingNode.gCost)
    {
        existingNode.gCost = tentativeGCost;
        existingNode.parent = currentNode;
    }
}

return null; // patha paaaauyaaa yaaayanai
}

List<Vector3> GetNeighbors(Vector3 position, List<Vector3> waypoints)
{
    List<Vector3> neighbors = new List<Vector3>();
    float connectionDistance = 5f;

    foreach (Vector3 waypoint in waypoints)
    {
        if (Vector3.Distance(position, waypoint) <= connectionDistance)
        {
            neighbors.Add(waypoint);
        }
    }

    return neighbors;
}

List<Vector3> RetracePath(AStarNode endNode)
{
    List<Vector3> path = new List<Vector3>();
    AStarNode currentNode = endNode;

    while (currentNode != null)
    {
        path.Add(currentNode.position);
        currentNode = currentNode.parent;
    }

    path.Reverse();
    return path;
}
}

```

22.5 baibhainana dharanaera gaemae AI

harara gaema AI

```
// harara gaemae ainaimai AI
public class HorrorEnemyAI : MonoBehaviour
{
    public enum AIState { Hunting, Listening, Ambushing, Patrolling }

    public AIState currentState = AIState.Patrolling;
    public float hearingRange = 15f;
    public float sightRange = 20f;

    void Update()
    {
        switch (currentState)
        {
            case AIState.Hunting:
                HuntPlayer();
                break;
            case AIState.Listening:
                ListenForPlayer();
                break;
            case AIState.Ambushing:
                WaitInAmbush();
                break;
            case AIState.Patrolling:
                PatrolArea();
                break;
        }
    }

    void ListenForPlayer()
    {
        // palaeyaaaraera shabada shaonaaa
        Collider[] players = Physics.OverlapSphere(transform.position, hearingRange);
        foreach (Collider player in players)
        {
            PlayerController pc = player.GetComponent<PlayerController>();
            if (pc != null && pc.isMoving)
            {
                // palaeyaaarakae shaunae paelae haaanatai sataetae yaaaau
                currentState = AIState.Hunting;
            }
        }
    }

    void WaitInAmbush()
    {
        // ayaamabaushae apaekassaaa karao
        // palaeyaaara kaaachhae ailae aakaramana karao
    }
}
```

FPS gaema AI

```
// FPS gaemae ainaimai AI
public class FPSEnemyAI : MonoBehaviour
{
```

```

public enum AIState { Flanking, TakingCover, Engaging, Retreating }

public AIState currentState = AIState.Engaging;
public float flankingDistance = 10f;

void Update()
{
    switch (currentState)
    {
        case AIState.Flanking:
            PerformFlankingManeuver();
            break;
        case AIState.TakingCover:
            FindAndTakeCover();
            break;
        case AIState.Engaging:
            EngagePlayer();
            break;
        case AIState.Retreating:
            RetreatToSafety();
            break;
    }
}

void PerformFlankingManeuver()
{
    // palaeyaaaraera paaasha daiyae yaaaau
    Vector3 flankPosition = CalculateFlankPosition();
    agent.SetDestination(flankPosition);
}

void FindAndTakeCover()
{
    // aasharaya khaujae baera karao
    Collider[] coverPoints = Physics.OverlapSphere(transform.position, 15f, coverLayer
    // sabachaeyae bhaaalao aasharaya nairabaaachana karao
    // saekhhaanae yaaaau
}
}

```

RPG gaema AI

```

// RPG gaemae ainaimai AI
public class RPGENemyAI : MonoBehaviour
{
    public enum AIState { Aggressive, Defensive, Supportive, Neutral }

    public AIState currentState = AIState.Aggressive;
    public RPGClass enemyClass;

    void Update()
    {
        switch (currentState)
        {
            case AIState.Aggressive:
                AggressiveBehavior();
                break;
            case AIState.Defensive:
                DefensiveBehavior();
                break;
            case AIState.Supportive:
                SupportiveBehavior();
                break;
        }
    }
}

```

```

        break;
    case AIState.Neutral:
        NeutralBehavior();
        break;
    }
}

void AggressiveBehavior()
{
    // saraaasaraai palaeyaaaraera upara aakaramana karao
    // shakataishaaalarii sakaila bayabahaaara karao
}

void DefensiveBehavior()
{
    // shailada bayabahaaara karao
    // haila sakaila bayabahaaara karao
    // palaeyaaarakae dhairae karao
}

void SupportiveBehavior()
{
    // anaya ainaimaidaera haila karao
    // baaapha daaaaau
    // daibaaapha daaaaau
}
}

```

satarayaaataejai gaema AI

```

// satarayaaataejai gaemae AI
public class StrategyGameAI : MonoBehaviour
{
    public enum AIPersonality { Aggressive, Defensive, Economic, Balanced }

    public AIPersonality personality = AIPersonality.Balanced;
    public int resources;
    public List<Unit> units;
    public List<Building> buildings;

    void Update()
    {
        // raisaorasa mayaaanaejamaenata
        ManageResources();

        // iunaita paraodaaakashana
        ProduceUnits();

        // bailadai kanasataraaakashana
        ConstructBuildings();

        // mailaitaarai apaaaraeshana
        ExecuteMilitaryOperations();
    }

    void ManageResources()
    {
        // raisaorasa sagaraha karao
        // kharacha karao
        // bayaaalaenasa raaakhao
    }

    void ProduceUnits()

```

```

{
    // kaona dharanaera iunaita taairai karabae saetaaa saidadhaaanata naaaau
    // bhayaaanaitai kaaaunataaara chaeka karao
}

void ExecuteMilitaryOperations()
{
    // ayaaataaaka palayaaana taaairai karao
    // daiphaenasa palayaaana taaairai karao
    // sakaaautai karao
}
}

```

22.6 parayaaakataisa taipasa

taipasa 1: saimapala shaurau karao

```

// saimapala AI daiyae shaurau karao
public class SimpleEnemyAI : MonoBehaviour
{
    public Transform player;
    public float detectionRange = 10f;
    public float attackRange = 2f;

    void Update()
    {
        float distanceToPlayer = Vector3.Distance(transform.position, player.position);

        if (distanceToPlayer <= attackRange)
        {
            Attack();
        }
        else if (distanceToPlayer <= detectionRange)
        {
            Chase();
        }
        else
        {
            Patrol();
        }
    }
}

```

taipasa 2: bhayaaaraaaibala bayabahaaara karao

```

// AI-kae bhayaaaraaaibala daiyae kanaphaigaaara karao
[System.Serializable]
public class AIConfig
{
    public float detectionRange = 10f;
    public float attackRange = 2f;
    public float chaseSpeed = 5f;
    public float patrolSpeed = 2f;
    public float attackDamage = 10f;
    public float attackCooldown = 1f;
}

```

taipasa 3: rayaaanadamanaesa yaoga karao

```
// AI-kae rayaaanadama karao yaaatae paraedaikataebala naaa haya
public void Patrol()
{
    // rayaaanadama samaya parae paetaraola patha paraibaratana karao
    if (Random.Range(0f, 1f) < 0.1f)
    {
        currentWaypointIndex = Random.Range(0, waypoints.Length);
    }
}
```

taipasa 4: phaidabayaaaka daiyae AI unanata karao

```
// palaeyaaaraera aacharana thaekae AI shaikhauka
public void LearnFromPlayer()
{
    // palaeyaaara kaona pathae baeshai yaaaya
    // palaeyaaara kaona satarayaaataejai bayabahaaara karae
    // AI saei anauyaaayaii naijaekae ayaaadaapata karauka
}
```

taipasa 5: paaarapharamayaaanasa baibaechanaaaa karao

```
// AI kaodakae apataimaaaija karao
public class OptimizedEnemyAI : MonoBehaviour
{
    public float updateInterval = 0.1f; // paratai 0.1 saekaenadae aapadaeta
    private float lastUpdateTime;

    void Update()
    {
        if (Time.time - lastUpdateTime >= updateInterval)
        {
            UpdateAI();
            lastUpdateTime = Time.time;
        }
    }
}
```

22.7 parayaaakataisa aikasaarasaaaija

aikasaarasaaaija 1: saimapala ainaimai AI taairai karao

aikatai saimapala ainaimai AI taairai karao yaaara aacharanagaulao halao:

- paetaraola karao
- palaeyaaarakae sanaakata karao
- palaeyaaarakae taaadaaa karao
- palaeyaaarakae aakaramana karao

aikasaarasaaaija 2: A* paaathaphaaainadaaara taairai karao

aikatai A* paaathaphaaainadaaara taairai karao yaaa:

- aikatai naoda thaekae anaya naodae patha khaujae baera karabae
- abasataraaakashana aidaiyae yaaabae
- sabachaeyae chhaota patha raitaaarana karabae

aikasaaarasaaaija 3: baihaebhaiyara tarai taairai karao

aikatai baihaebhaiyara tarai taairai karao yaaa:

- palaeyaaara daekhatae paaaralae taaadaaa karabae
- palaeyaaara daekhatae naaa paelae paetaraola karabae
- haelatha kama halae paaalaiyae yaaabae

22.8 bhaula yaaa aidaiyae chalatae habae

bhaula 1: atairaikata jataila AI taairai karaaa

```
// khaaaraaapa - atairaikata jataila AI
public class OverlyComplexAI : MonoBehaviour
{
    void Update()
    {
        // 100tai sataeta mayaaanaeja karaaa
        // kalaaanataikara aiba baaagaparabana
    }
}
```

bhaula 2: AI-kae atairaikata samaaarata karaaa

```
// khaaaraaapa - AI atairaikata samaaarata
public class OverlySmartAI : MonoBehaviour
{
    void Update()
    {
        // palaeyaaaraera saba kaaushala jaaanae
        // sabasamaya sathaika saidadhaaanata naeya
        // khaelaoyaaadaera janaya phaaanai hayae yaaaya
    }
}
```

bhaula 3: AI-kae atairaikata dhaiira karaaa

```
// khaaaraaapa - AI atairaikata dhaiira
public class SlowAI : MonoBehaviour
{
    public float reactionTime = 5f; // 5 saekaenada parataikaraiyaaa samaya

    void Update()
    {
        // AI atairaikata dhaiira, khaelaoyaaada bairakata haya
    }
}
```

22.9 saarasakassaepa

aai adhayaaayae aamaraaa gaema AI-aira baibhainana daika naiyae aalaochanaaa karaechhai mauula payaenatagaulao halao:

1. ****ainaimai AI baihaebhaiyaaara****: paetaraola, saaracha, daitaekashana, ayaaalaaarata, chaeja, aakaramana,

phalai, raitaaarana

2. ****AI sataeta maeshaina****: baibhainana sataetaera madhayae taraaanajaishana
3. ****baihaebhaiyara tarai****: phalaekasaibala AI saisataema
4. ****paaathaphaaainadai****: A* ayaaalagaraidama
5. ****baibhainana dharanaera gaemae AI****: harara, FPS, RPG, satarayaaataejai
6. ****parayaaakataisa taipasa****: saimapala shaurau karao, bhayaaaraaibala bayabahaaara karao, rayaaanadamanaesa yaoga karao

AI halao gaema daebhaelapamaenataera sabachaeyae aakarassanaiya asha bhaaalao AI taairai karatae samaya laaagae, kainatau phalaaaphala atayanata sanataossajanaka

****parabarataii adhayaaaya: UI/UX daijaaaina -- kaiibhaaabaee gaemae iujaaara inataaaraphaesa taairai karabaena****

PART 23

adhayaaaya 23: UI/UX daijaaaina

adhayaaaya 23: UI/UX daijaaaina

shaekhaara udadaeshaya

aai adhayaaayae aamaraaa gaema daebhaelapamaenatae UI/UX daijaaaina naiyae aalaochanaaa karaba UI (User Interface) halao yaaa khaelaoyaaada daekhae aiba inataaraayaaakata karae, aiba UX (User Experience) halao khaelaoyaaadaera saaamagaraika abhaijanyataaa aai adhayaaaya shaessae aapanai jaaanabaena kaiibhaaabaes aunadara aiba kaaarayakara UI taairai karabaena

23.1 maeina maenau daijaaaina

maeina maenau halao gaemaera parathama imaparaeshana aitai aunadara aiba bayabahaaarae sahaja hatae habae

maeina maenau upaaadaaanasamauha:

- gaemaera naaama aiba laogao
- Play baaatana, Settings baaatana, Quit baaatana
- bayaaakagaraaunada maiujaika aiba ayaaanaimaeshana

maeina maenau kaoda:

```
public class MainMenuController : MonoBehaviour
{
    public GameObject mainPanel;
    public GameObject settingsPanel;
    public Animator menuAnimator;

    void Start()
    {
        ShowMainPanel();
        AudioManager.Instance.PlayMusic("MainMenuMusic");
    }

    public void OnPlayButtonClicked()
    {
        menuAnimator.SetTrigger("FadeOut");
        StartCoroutine(LoadGameScene());
    }

    IEnumerator LoadGameScene()
```

```

{
    yield return new WaitForSeconds(1f);
    SceneManager.LoadScene("GameScene");
}

public void OnSettingsButtonClicked()
{
    mainPanel.SetActive(false);
    settingsPanel.SetActive(true);
}

public void OnQuitButtonClicked()
{
    #if UNITY_EDITOR
        EditorApplication.isPlaying = false;
    #else
        Application.Quit();
    #endif
}

public void ShowMainPanel()
{
    mainPanel.SetActive(true);
    settingsPanel.SetActive(false);
}
}

```

23.2 paja maenau daijaaaina

paja maenau khaelaoyaaadaka gaema bairatai daitae saaahaaayaya karae

paja maenauru upaaadaaanasamauuha:

- Resume baaatana, Settings baaatana, Main Menu baaatana, Quit baaatana

paja maenau kaoda:

```

public class PauseMenuController : MonoBehaviour
{
    public GameObject pauseMenuPanel;
    public GameObject settingsPanel;
    public bool isPaused = false;

    void Update()
    {
        if (Input.GetKeyDown(KeyCode.Escape))
        {
            if (isPaused) ResumeGame();
            else PauseGame();
        }
    }

    public void PauseGame()
    {
        pauseMenuPanel.SetActive(true);
        Time.timeScale = 0f;
        isPaused = true;
        Cursor.lockState = CursorLockMode.None;
    }
}

```

```

        Cursor.visible = true;
    }

    public void ResumeGame()
    {
        pauseMenuPanel.SetActive(false);
        Time.timeScale = 1f;
        isPaused = false;
        Cursor.lockState = CursorLockMode.Locked;
        Cursor.visible = false;
    }

    public void OnMainMenuButtonClicked()
    {
        Time.timeScale = 1f;
        SceneManager.LoadScene("MainMenu");
    }
}

```

23.3 HUD daijaaaina (Heads-Up Display)

HUD halao gaemaera sakarainae thaaakaaa tathaya yaaa khaelaoyaaadakaе gaema samaparakaе jaaanaaaya

HUD-aira upaaadaaanasamauha:

- haelatha baaara, mayaaanaaa baaara, sakaora, mainaimayaaapa, ayaaamao kaaaunata, maishana abajaekataibha

HUD kaoda:

```

public class HUDController : MonoBehaviour
{
    public Slider healthBar;
    public Slider manaBar;
    public Text scoreText;
    public Text ammoText;
    public Image damageVignette;

    public void UpdateHealthBar(float currentHealth, float maxHealth)
    {
        healthBar.value = currentHealth / maxHealth;
        if (currentHealth / maxHealth < 0.3f)
            healthBar.fillRect.GetComponent<Image>().color = Color.red;
        else if (currentHealth / maxHealth < 0.6f)
            healthBar.fillRect.GetComponent<Image>().color = Color.yellow;
        else
            healthBar.fillRect.GetComponent<Image>().color = Color.green;
    }

    public void UpdateManaBar(float currentMana, float maxMana)
    {
        manaBar.value = currentMana / maxMana;
    }

    public void UpdateScore(int score)
    {
        scoreText.text = "Score: " + score;
    }
}

```

```

public void UpdateAmmo(int currentAmmo, int totalAmmo)
{
    ammoText.text = currentAmmo + " / " + totalAmmo;
}

public void ShowDamageEffect()
{
    StartCoroutine(DamageVignetteEffect());
}

IEnumerator DamageVignetteEffect()
{
    damageVignette.gameObject.SetActive(true);
    yield return new WaitForSeconds(0.5f);
    damageVignette.gameObject.SetActive(false);
}
}

```

23.4 inabhaenatarai UI

inabhaenatarai UI khaelaoyaaadakaee taaara jainaisapatara daekhatae aiba bayabahaaara karatae saaahaaayaya karae

inabhaenatarai UI-aira upaaadaaanasamauuha:

- inabhaenatarai garaida, aaitaema salata, aaitaema daitaeilasa, darayaaaga ainada darapa

inabhaenatarai UI kaoda:

```

public class InventoryUI : MonoBehaviour
{
    public Transform inventoryGrid;
    public GameObject inventorySlotPrefab;
    public int inventorySize = 20;
    private InventorySlot[] slots;

    void Start()
    {
        CreateInventorySlots();
    }

    void CreateInventorySlots()
    {
        slots = new InventorySlot[inventorySize];
        for (int i = 0; i < inventorySize; i++)
        {
            GameObject slotObj = Instantiate(inventorySlotPrefab, inventoryGrid);
            slots[i] = slotObj.GetComponent<InventorySlot>();
            slots[i].slotIndex = i;
        }
    }

    public void UpdateInventory(List<Item> items)
    {
        foreach (InventorySlot slot in slots)
            slot.ClearSlot();
        for (int i = 0; i < items.Count && i < slots.Length; i++)
            slots[i].AddItem(items[i]);
    }
}

```

```

}

public class InventorySlot : MonoBehaviour
{
    public Image itemIcon;
    public Text itemQuantity;
    public int slotIndex;
    private Item currentItem;

    public void AddItem(Item item)
    {
        currentItem = item;
        itemIcon.sprite = item.icon;
        itemIcon.gameObject.SetActive(true);
        itemQuantity.text = item.quantity > 1 ? item.quantity.ToString() : "";
    }

    public void ClearSlot()
    {
        currentItem = null;
        itemIcon.gameObject.SetActive(false);
        itemQuantity.text = "";
    }

    public void OnSlotClicked()
    {
        if (currentItem != null)
            ItemManager.Instance.UseItem(currentItem);
    }
}

```

23.5 mayaaapa UI

mayaaapa UI khaelaoyaaadaka gaemaera maaanachaitara daekhatae saaahaaayaya karae

mayaaapa UI-aira upaaadaaanasamaauha:

- mayaaapa bhaiu, mayaaapa maaarakaaara, mayaaapa jauma, mayaaapa payaaana

mayaaapa UI kaoda:

```

public class MapUI : MonoBehaviour
{
    public RawImage mapImage;
    public RectTransform mapContainer;
    public GameObject mapMarkerPrefab;
    public float zoomSpeed = 0.1f;
    public float panSpeed = 1f;
    private float currentZoom = 1f;
    private Vector2 panOffset;
    private bool isDragging = false;

    void Update()
    {
        HandleZoom();
        HandlePan();
    }

    void HandleZoom()

```

```

{
    float scroll = Input.GetAxis("Mouse ScrollWheel");
    if (scroll != 0)
    {
        currentZoom += scroll * zoomSpeed;
        currentZoom = Mathf.Clamp(currentZoom, 0.5f, 2f);
        mapContainer.localScale = Vector3.one * currentZoom;
    }
}

void HandlePan()
{
    if (Input.GetMouseButtonDown(2)) isDragging = true;
    if (Input.GetMouseButtonUp(2)) isDragging = false;
    if (isDragging)
    {
        float h = Input.GetAxis("Mouse X") * panSpeed;
        float v = Input.GetAxis("Mouse Y") * panSpeed;
        panOffset += new Vector2(h, v);
        mapContainer.anchoredPosition = panOffset;
    }
}

public void AddMapMarker(Vector2 worldPos, Sprite icon, string label)
{
    GameObject marker = Instantiate(mapMarkerPrefab, mapContainer);
    MapMarker mm = marker.GetComponent<MapMarker>();
    mm.SetMarker(worldPos, icon, label);
}
}

public class MapMarker : MonoBehaviour
{
    public Image markerIcon;
    public Text markerLabel;
    public RectTransform rectTransform;

    public void SetMarker(Vector2 worldPos, Sprite icon, string label)
    {
        markerIcon.sprite = icon;
        markerLabel.text = label;
        rectTransform.anchoredPosition = worldPos * 0.1f;
    }
}

```

23.6 kauyaesata UI

kauyaesata UI khaelaoyaaadake maishana aiba abajaekataibha daekhatae saaahaaayaya karae

kauyaesata UI-aira upaaadaaanasamauuha:

- kauyaesata laisata, kauyaesata daitaelasa, kauyaesata tarayaaakaaara, kauyaesata maaarakaaara

kauyaesata UI kaoda:

```

public class QuestUI : MonoBehaviour
{
    public Transform questListContent;
}

```

```
public GameObject questItemPrefab;
public QuestDetailUI questDetailUI;

public void UpdateQuestList(List<Quest> quests)
{
    foreach (Transform child in questListContent)
        Destroy(child.gameObject);
    foreach (Quest quest in quests)
    {
        GameObject obj = Instantiate(questItemPrefab, questListContent);
        QuestItem qi = obj.GetComponent<QuestItem>();
        qi.SetQuest(quest);
        qi.OnQuestSelected += OnQuestSelected;
    }
}

void OnQuestSelected(Quest quest)
{
    questDetailUI.ShowQuestDetails(quest);
}

public class QuestItem : MonoBehaviour
{
    public Text questTitle;
    public Image questIcon;
    public Text questStatus;
    private Quest quest;
    public event System.Action<Quest> OnQuestSelected;

    public void SetQuest(Quest questData)
    {
        quest = questData;
        questTitle.text = quest.title;
        questIcon.sprite = quest.icon;
        questStatus.text = quest.isCompleted ? "Completed" : "Active";
    }

    public void OnClick()
    {
        OnQuestSelected?.Invoke(quest);
    }
}

public class QuestDetailUI : MonoBehaviour
{
    public Text questTitle;
    public Text questDescription;
    public Text questReward;
    public Transform objectiveList;
    public GameObject objectiveItemPrefab;

    public void ShowQuestDetails(Quest quest)
    {
        questTitle.text = quest.title;
        questDescription.text = quest.description;
        questReward.text = "Reward: " + quest.reward + " XP";
        foreach (Transform child in objectiveList)
            Destroy(child.gameObject);
        foreach (QuestObjective obj in quest.objectives)
            Instantiate(objectiveItemPrefab, objectiveList);
    }
}
```

23.7 saetaisa UI

saetaisa UI khaelaoyaaadaka gaema kanaphaigaaara karatae saaahaaayaya karae

saetaisa UI-aira upaaadaaanasamauha:

- adaiau saetaisa, bhaidaiu saetaisa, kanataraola saetaisa, jaenaaaraela saetaisa

saetaisa UI kaoda:

```
public class SettingsUI : MonoBehaviour
{
    public Slider masterVolumeSlider;
    public Slider musicVolumeSlider;
    public Slider sfxVolumeSlider;
    public Dropdown resolutionDropdown;
    public Dropdown qualityDropdown;
    public Toggle fullscreenToggle;
    public Slider sensitivitySlider;
    public Toggle invertYToggle;

    void Start()
    {
        LoadSettings();
        masterVolumeSlider.onValueChanged.AddListener(OnMasterVolumeChanged);
        musicVolumeSlider.onValueChanged.AddListener(OnMusicVolumeChanged);
        sfxVolumeSlider.onValueChanged.AddListener(OnSFXVolumeChanged);
        resolutionDropdown.onValueChanged.AddListener(OnResolutionChanged);
        qualityDropdown.onValueChanged.AddListener(OnQualityChanged);
        fullscreenToggle.onValueChanged.AddListener(OnFullscreenChanged);
    }

    void LoadSettings()
    {
        masterVolumeSlider.value = PlayerPrefs.GetFloat("MasterVolume", 1f);
        musicVolumeSlider.value = PlayerPrefs.GetFloat("MusicVolume", 0.8f);
        sfxVolumeSlider.value = PlayerPrefs.GetFloat("SFXVolume", 1f);
        Resolution[] resolutions = Screen.resolutions;
        resolutionDropdown.ClearOptions();
        List<string> options = new List<string>();
        int currentResIndex = 0;
        for (int i = 0; i < resolutions.Length; i++)
        {
            options.Add(resolutions[i].width + " x " + resolutions[i].height);
            if (resolutions[i].width == Screen.currentResolution.width &&
                resolutions[i].height == Screen.currentResolution.height)
                currentResIndex = i;
        }
        resolutionDropdown.AddOptions(options);
        resolutionDropdown.value = currentResIndex;
        qualityDropdown.ClearOptions();
        qualityDropdown.AddOptions(QualitySettings.names.ToList());
        qualityDropdown.value = QualitySettings.GetQualityLevel();
        fullscreenToggle.isOn = Screen.fullScreen;
    }

    void OnMasterVolumeChanged(float value)
    {
        AudioListener.volume = value;
    }
}
```



```

        PlayerPrefs.SetFloat("MasterVolume", value);
    }

    void OnMusicVolumeChanged(float value)
    {
        AudioManager.Instance.SetMusicVolume(value);
        PlayerPrefs.SetFloat("MusicVolume", value);
    }

    void OnSFXVolumeChanged(float value)
    {
        AudioManager.Instance.SetSFXVolume(value);
        PlayerPrefs.SetFloat("SFXVolume", value);
    }

    void OnResolutionChanged(int index)
    {
        Resolution r = Screen.resolutions[index];
        Screen.SetResolution(r.width, r.height, Screen.fullScreen);
    }

    void OnQualityChanged(int index)
    {
        QualitySettings.SetQualityLevel(index);
    }

    void OnFullscreenChanged(bool isFullscreen)
    {
        Screen.fullScreen = isFullscreen;
    }

    public void ResetSettings()
    {
        masterVolumeSlider.value = 1f;
        musicVolumeSlider.value = 0.8f;
        sfxVolumeSlider.value = 1f;
        fullscreenToggle.isOn = true;
    }
}

```

23.8 taiutaoraiyaaala UI

taiutaoraiyaaala UI khaelaoyaaadaka gaema shaekhatae saaahaaayaya karae

taiutaoraiyaaala UI-aira upaaadaaanamasamauha:

- taiutaoraiyaaala taekasata, haaailaaaita iphaekata, ayaaarao gaaaida, sakaipa baaatana

taiutaoraiyaaala UI kaoda:

```

public class TutorialUI : MonoBehaviour
{
    public GameObject tutorialPanel;
    public Text tutorialText;
    public Image highlightImage;
    public Button skipButton;
    private Queue<TutorialStep> tutorialSteps = new Queue<TutorialStep>();
    private bool isActive = false;
}

```

```

void Start()
{
    AddStep(new TutorialStep("WASD baaatana daiyae chalao", null, "Move"));
    AddStep(new TutorialStep("maaausae daika paraibaratana karao", null, "LookAround"));
    AddStep(new TutorialStep("laephata kalaika daiyae aakaramana karao", "AttackButton"));
    StartTutorial();
}

public void AddStep(TutorialStep step) { tutorialSteps.Enqueue(step); }

public void StartTutorial()
{
    if (tutorialSteps.Count > 0)
    {
        isActive = true;
        ShowNextStep();
    }
}

void ShowNextStep()
{
    if (tutorialSteps.Count > 0)
    {
        TutorialStep s = tutorialSteps.Dequeue();
        tutorialPanel.SetActive(true);
        tutorialText.text = s.text;
        if (s.highlightTarget != null) HighlightObject(s.highlightTarget);
        StartCoroutine(WaitForCompletion(s.completionCondition));
    }
    else EndTutorial();
}

void HighlightObject(string name)
{
    GameObject target = GameObject.Find(name);
    if (target != null)
    {
        highlightImage.gameObject.SetActive(true);
        highlightImage.rectTransform.position = Camera.main.WorldToScreenPoint(target.
    }
}

IEnumerator WaitForCompletion(string condition)
{
    bool done = false;
    while (!done)
    {
        switch (condition)
        {
            case "Move":
                done = Input.GetAxis("Horizontal") != 0 || Input.GetAxis("Vertical") != 0;
                break;
            case "LookAround":
                done = Input.GetAxis("Mouse X") != 0 || Input.GetAxis("Mouse Y") != 0;
                break;
            case "Attack":
                done = Input.GetMouseButtonDown(0);
                break;
        }
        yield return null;
    }
    ShowNextStep();
}

```

```

void EndTutorial()
{
    isActive = false;
    tutorialPanel.SetActive(false);
    highlightImage.gameObject.SetActive(false);
}

public void SkipTutorial()
{
    StopAllCoroutines();
    EndTutorial();
}
}

[System.Serializable]
public class TutorialStep
{
    public string text;
    public string highlightTarget;
    public string completionCondition;
    public TutorialStep(string t, string h, string c) { text = t; highlightTarget = h; com
}

```

23.9 naotaiphaikaeshana saisataema

naotaiphaikaeshana saisataema khaelaoyaaadaka gaurautabapauurana tathaya jaaanaaaya

naotaiphaikaeshana saisataema kaoda:

```

public enum NotificationType { Info, Warning, Success, Error }

[System.Serializable]
public class NotificationData
{
    public string message;
    public NotificationType type;
    public float timestamp;
}

public class NotificationSystem : MonoBehaviour
{
    public GameObject notificationPrefab;
    public Transform notificationContainer;
    public float notificationDuration = 3f;
    public float animationDuration = 0.5f;
    private Queue<NotificationData> queue = new Queue<NotificationData>();
    private bool isShowing = false;

    public void ShowNotification(string message, NotificationType type = NotificationType.
    {
        queue.Enqueue(new NotificationData { message = message, type = type, timestamp = T
        if (!isShowing) StartCoroutine(ShowNext());
    }

    IEnumerator ShowNext()
    {
        if (queue.Count > 0)
        {
            isShowing = true;

```

```

        NotificationData data = queue.Dequeue();
        GameObject obj = Instantiate(notificationPrefab, notificationContainer);
        NotificationUI n = obj.GetComponent<NotificationUI>();
        n.SetNotification(data);
        yield return StartCoroutine(AnimateIn(obj));
        yield return new WaitForSeconds(notificationDuration);
        yield return StartCoroutine(AnimateOut(obj));
        Destroy(obj);
        isShowing = false;
        if (queue.Count > 0) StartCoroutine>ShowNext());
    }
}

IEnumerator AnimateIn(GameObject obj)
{
    float t = 0;
    Vector3 start = obj.transform.position + Vector3.right * 200;
    Vector3 end = obj.transform.position;
    while (t < animationDuration)
    {
        t += Time.deltaTime;
        obj.transform.position = Vector3.Lerp(start, end, t / animationDuration);
        yield return null;
    }
}

IEnumerator AnimateOut(GameObject obj)
{
    float t = 0;
    Vector3 start = obj.transform.position;
    Vector3 end = obj.transform.position + Vector3.right * 200;
    while (t < animationDuration)
    {
        t += Time.deltaTime;
        obj.transform.position = Vector3.Lerp(start, end, t / animationDuration);
        yield return null;
    }
}

}

public class NotificationUI : MonoBehaviour
{
    public Text messageText;
    public Image backgroundImage;

    public void SetNotification(NotificationData data)
    {
        messageText.text = data.message;
        switch (data.type)
        {
            case NotificationType.Info: backgroundImage.color = Color.blue; break;
            case NotificationType.Warning: backgroundImage.color = Color.yellow; break;
            case NotificationType.Success: backgroundImage.color = Color.green; break;
            case NotificationType.Error: backgroundImage.color = Color.red; break;
        }
    }
}

```

23.10 UI haaayaaaraarakai daaayaaagaraama

```

    UI HIERARCHY
    CANVAS (kayaaanabhaaasa)
    SCREEN OVERLAY (sakaraina aubhaaaralae)
    HUD (haedasa-aapa daisapalae)
HealthBar ManaBar Score
MiniMap Ammo Mission
    MENUS (maenau)
    Main Menu
    Play Settings Quit
    Pause Menu
    Resume Settings Quit
    POPUPS (papaaapa)
Inventory Quest Log
Map Settings
```

23.11 UX phalao daaayaaagaraaama

```

    UX FLOW DIAGRAM
START > MENU > PLAY
(shaurau) (maenau) (khaelaaa)
          SETTINGS PAUSE
          (saetaisa) (paja)
                   GAME
                   OVER
                   (shaessa)
                   SCORE
                   (sakaora)
                   MENU
```

23.12 ranga tatataba (Color Theory for Games)

gaemae ranga bayabahaaaraera naiyama:

ranga payaaalaeta taairai karao:

```
[System.Serializable]
public class GameColorPalette
{
    public Color primaryColor = new Color(0.2f, 0.6f, 1f);
    public Color secondaryColor = new Color(0.9f, 0.3f, 0.3f);
    public Color accentColor = new Color(1f, 0.8f, 0.2f);
    public Color backgroundDark = new Color(0.1f, 0.1f, 0.15f);
    public Color backgroundLight = new Color(0.9f, 0.9f, 0.95f);
    public Color textPrimary = Color.white;
    public Color textSecondary = new Color(0.7f, 0.7f, 0.7f);
}
```

23.13 ayaaakasaesaibailaitai baibaechanaaaa (Accessibility)

ayaaakasaesaibailaitaira upaaadaaanasamauuha:

- kaaalaaara balaaainadanaesa saaapaorata
- phanata saaaija ayaaadajaaasatamaenata
- saaabataaaaitaela saaapaorata
- kanataraola raimayaaapai
- sakaraina raidaaara saaapaorata

ayaaakasaesaibailaitai kaoda:

```
public class AccessibilitySettings : MonoBehaviour
{
    public Slider fontSizeSlider;
    public Toggle colorBlindModeToggle;
    public Toggle subtitleToggle;
    public Toggle screenReaderToggle;
    public TMP_FontAsset[] fontAssets;

    void Start()
    {
        LoadAccessibilitySettings();
        fontSizeSlider.onValueChanged.AddListener(OnFontSizeChanged);
        colorBlindModeToggle.onValueChanged.AddListener(OnColorBlindModeChanged);
        subtitleToggle.onValueChanged.AddListener(OnSubtitleChanged);
    }

    void LoadAccessibilitySettings()
    {
        fontSizeSlider.value = PlayerPrefs.GetFloat("FontSize", 24f);
        colorBlindModeToggle.isOn = PlayerPrefs.GetInt("ColorBlindMode", 0) == 1;
        subtitleToggle.isOn = PlayerPrefs.GetInt("Subtitles", 1) == 1;
    }

    void OnFontSizeChanged(float value)
    {
        PlayerPrefs.SetFloat("FontSize", value);
        ApplyFontSize(value);
    }

    void OnColorBlindModeChanged(bool isOn)
    {
        PlayerPrefs.SetInt("ColorBlindMode", isOn ? 1 : 0);
        ApplyColorBlindFilter(isOn);
    }

    void OnSubtitleChanged(bool isOn)
    {
        PlayerPrefs.SetInt("Subtitles", isOn ? 1 : 0);
        ToggleSubtitles(isOn);
    }

    void ApplyFontSize(float size)
    {
        Text[] texts = FindObjectsOfType<Text>();
        foreach (Text t in texts)
            t.fontSize = (int)size;
    }

    void ApplyColorBlindFilter(bool enabled)
    {
        // kaaalaaara balaaainadanaesa phailataaara parayaoga karao
        if (enabled)
```

```

    {
        // Protanopia, Deutanopia, Tritanopia phailataaara
    }
}

void ToggleSubtitles(bool enabled)
{
    // saaabataaaaitaela payaaanaela daekhaaaau/laukaaau
}
}

```

23.14 UI/UX auyayaaarapharaema udaaaharana

maeina maenau auyayaaarapharaema:

```

    GAME TITLE
    (laogao)
    PLAY
    SETTINGS
    QUIT
    [bayaaakagaaaaunada aarata]

```

HUD auyayaaarapharaema:

```

<3<3<3<3<3      MISSION:
HP: 80/100      Defeat Boss      Ammo
    [GAME VIEW]
MP:50                      Score
                        12500

```

inabhaenatarai auyayaaarapharaema:

```

    INVENTORY
    x5      x20
x10      x3
Item: Health Potion
Restores 50 HP
[USE] [DROP] [EQUIP]

```

23.15 parayaaakataisa aikasaaarasaaaija

aikasaaarasaaaija 1: maeina maenau taairai karao

aikatai maeina maenau taairai karao yaaatae thaaakabae:

- gaemaera naaama aiba laogao
- Play, Settings, Quit baaatana
- bayaaakagaaaaunada maiujaika
- phaeda ina/aauta ayaaanaimaeshana

aikasaaarasaaaija 2: HUD taairai karao

aikatai HUD taairai karao yaaatae thaaakabae:

- haelatha baaara (ranga paraibaratana saha)
- mayaaanaaa baaara
- sakaora kaaaunataaara
- ayaaamao kaaaunataaara

aikasaaarasaaaija 3: saetaisa maenau taairai karao

aikatai saetaisa maenau taairai karao yaaatae thaaakabae:

- adaiau bhalaiuma kanataraola
- raejaolaiushana sailaekatarata
- phaulasakaraina tagala
- saetaisa saebha/laoda

23.16 bhaula yaaa aidaiyae chalatae habae

bhaulta 1: atairaikata jataila UI

```
// khaaaraaapa - atairaikata jataila maenau
// khaelaoyaaaada baibharaaanata haya
```

bhaulta 2: chhaota phanata saaaija

```
// khaaaraaapa - phanata khauba chhaota
// padaaa kathaina haya
```

bhaulta 3: kanataraasataera abhaaaba

```
// khaaaraaapa - taekasata aiba bayaaakagaaaaunadaera ranga aikai rakama
// padaaa asamabhaba haya
```

bhaulta 4: ayaaakasaesaibailaitai upaekassaaa karaaa

```
// khaaaraaapa - shaudhau ranga daiyae tathaya baojhaaanao
// kaaalaaara balaaainada khaelaoyaaadaraaa baujhatae paaarae naaa
```

23.17 taipasa au paraaamarasha

taipasa 1: UI kae saimapala raaakhao

```
// bhaaalao - saimapala aiba kalaina UI
// khaelaoyaaaada sahajae baujhatae paaarae
```

taipasa 2: kanasaisataenasai raaakhao

```
// bhaaalao - saba maenautae aikai ranga aiba sataaaila
// khaelaoyaaaada abhayasata haya
```

taipasa 3: phaidabayaaaka daiyae UI unanata karao

```
// bhaaalao - khaelaoyaaadadaera mataaamata naiyae UI paraibaratana karao
```


taipasa 4: ayaaanaimaeshana yaoga karao

```
// bhaaalao - UI ailaimaenatae sauukassama ayaaanaimaeshana
// inataaaraayaaakashana baaadae
```

taipasa 5: paaarapharamayaaanasa baibaechanaaaa karao

```
// bhaaalao - UI ailaimaenata kama raaakhao, Canvas Optimizer bayabahaaara karao
```

23.18 saaarasakassaepa

aii adhayaaayae aamaraaa UI/UX daijaaainaera baibhainana daika naiyae aalaochanaaaa karaechhai mauula payaenatagaulao halao:

1. ****maeina maenau****: gaemaera parathama imaparaeshana, saunadara aiba bayabahaaarae sahaja
2. ****paja maenau****: gaema bairatai daeayuaara saubaidhaaa
3. ****HUD****: khaelaoyaaadaka gaema samaparaka tathaya daeayuaa
4. ****inabhaenatarai UI****: jainaisapatara paraichaaalanaaa
5. ****mayaaapa UI****: maaanachaitara daekhaaara saubaidhaaa
6. ****kauyaesata UI****: maishana tarayaaakai
7. ****saetaisa UI****: gaema kanaphaigaaaraeshana
8. ****taiutaoraiyaaala UI****: natauna khaelaoyaaadadaera saaahaaayaya
9. ****naotaiphaikaeshana****: gaurautabapauurana tathaya jaaanaanao
10. ****ayaaakasaesaibailaitai****: saba dharanaera khaelaoyaaadadaera janaya UI

bhaaalao UI/UX daijaaaina gaemaera abhaijanyataaakae anaeka unanata karae

****parabarataii adhayaaaya: saaaunada daijaaaina -- kaiibhaaaba gaemae shabada taairai karabaena****

PART 24

adhayaaaya 24: saaaunada daijaaaina

adhayaaaya 24: saaaunada daijaaaina

shaekhaara udadaeshaya

aai adhayaaayae aamaraaa gaema daebhaelapamaenatae saaaunada daijaaaina naiyae aalaochanaaa karaba saaaunada daijaaaina halao gaemaera abhaijanyataaakae samaridadha karaaara aikatai gaurautabapauurana asha aii adhayaaaya shaessae aapanai jaaanabaena kaiibhaaabaee gaemae shabada taairai karabaena, kaiibhaaabaee shabada paraichaaalanaaa karabaena, aiba baibhainana dharanaera gaemae saaaunada daijaaaina kaiibhaaabaee kaaaja karae

24.1 gaema adaiau paaipalaaaina (Game Audio Pipeline)

gaema adaiau paaipalaaaina halao shabada taairai thaekae shaurau karae gaemae bayabahaaara karaaara paurao parakariyaaa

gaema adaiau paaipalaaainaera dhaapasamaauha:

```
GAME AUDIO PIPELINE
RECORDING > EDITING > MIXING
(raekaradai) (aidaitai) (maikasai)
PLAYBACK < MIDDLEWARE < MASTERING
(palaebyaaaaka) (maidalaauyayaaara) (maaasataaarai)
```

adaiau kayaaataagarai aiba taaadaera udadaeshaya:

24.2 maiujaika daijaaaina

maiujaika daijaaainaera dhaapasamaauha:

1. gaemaera dharana nairadhaaarana karao

- harara: bhayakara, asabasataikara
- ayaaakashana: utataejanaaapauurana, shakataishaaaliii
- ayaaadabhaenyachaaara: raomaaanyachakara, aabaegapauurana
- paaajala: shaaanata, chainataaashaiila

****2. maiujaika sataaaila nairabaaachana karao****

- arakaesataraaala
- ilaekataranaika
- chaipataiuna
- ayaaamabaiyaenata

maiujaika mayaaanaejaaara kaoda:

```
public class MusicManager : MonoBehaviour
{
    private static MusicManager instance;
    public static MusicManager Instance => instance;

    public AudioSource[] musicTracks;
    public float crossfadeDuration = 2f;
    private int currentTrackIndex = -1;

    void Awake()
    {
        if (instance == null)
        {
            instance = this;
            DontDestroyOnLoad(gameObject);
        }
        else Destroy(gameObject);
    }

    public void PlayMusic(int trackIndex)
    {
        if (trackIndex == currentTrackIndex) return;
        StartCoroutine(CrossfadeMusic(trackIndex));
    }

    IEnumerator CrossfadeMusic(int newIndex)
    {
        AudioSource oldTrack = currentTrackIndex >= 0 ? musicTracks[currentTrackIndex] : null;
        AudioSource newTrack = musicTracks[newIndex];

        if (oldTrack != null)
        {
            float t = 0;
            while (t < crossfadeDuration)
            {
                t += Time.deltaTime;
                oldTrack.volume = Mathf.Lerp(1f, 0f, t / crossfadeDuration);
                yield return null;
            }
            oldTrack.Stop();
        }

        newTrack.volume = 0f;
        newTrack.Play();
        float t2 = 0;
        while (t2 < crossfadeDuration)
        {
            t2 += Time.deltaTime;
            newTrack.volume = Mathf.Lerp(0f, 1f, t2 / crossfadeDuration);
            yield return null;
        }
    }
}
```

```

        currentTrackIndex = newIndex;
    }

    public void StopMusic()
    {
        StartCoroutine(FadeOutMusic());
    }

    IEnumerator FadeOutMusic()
    {
        if (currentTrackIndex < 0) yield break;
        AudioSource track = musicTracks[currentTrackIndex];
        float t = 0;
        while (t < crossfadeDuration)
        {
            t += Time.deltaTime;
            track.volume = Mathf.Lerp(1f, 0f, t / crossfadeDuration);
            yield return null;
        }
        track.Stop();
        currentTrackIndex = -1;
    }
}

```

24.3 ayaaamabaiyaenasa daijaaaina

ayaaamabaiyaenasa daijaaainaera upaaadaaanasamaauha:

- baaataasaera shabada
- paaanaira shabada
- paaakhaira daaaka
- paokaaamaakadaera shabada
- barissataira shabada
- bajarapaaataera shabada

ayaaamabaiyaenasa mayaaanaejaaara kaoda:

```

public class AmbienceManager : MonoBehaviour
{
    private static AmbienceManager instance;
    public static AmbienceManager Instance => instance;

    [System.Serializable]
    public class AmbienceZone
    {
        public string zoneName;
        public AudioClip[] ambienceClips;
        public float[] volumes;
        public float fadeDuration = 2f;
    }

    public AmbienceZone[] zones;
    private AudioSource[] ambienceSources;
    private int currentZoneIndex = -1;

    void Awake()

```

```

{
    if (instance == null)
    {
        instance = this;
        DontDestroyOnLoad(gameObject);
    }
    else Destroy(gameObject);

    InitializeAmbienceSources();
}

void InitializeAmbienceSources()
{
    ambienceSources = new AudioSource[10];
    for (int i = 0; i < ambienceSources.Length; i++)
    {
        ambienceSources[i] = gameObject.AddComponent<AudioSource>();
        ambienceSources[i].loop = true;
        ambienceSources[i].playOnAwake = false;
    }
}

public void ChangeZone(int zoneIndex)
{
    if (zoneIndex == currentZoneIndex) return;
    StartCoroutine(CrossfadeZone(zoneIndex));
}

IEnumerator CrossfadeZone(int newZoneIndex)
{
    // pauraanao jaona phaeda aauta
    if (currentZoneIndex >= 0)
    {
        AmbienceZone oldZone = zones[currentZoneIndex];
        float t = 0;
        while (t < oldZone.fadeDuration)
        {
            t += Time.deltaTime;
            for (int i = 0; i < oldZone.ambienceClips.Length; i++)
                ambienceSources[i].volume = Mathf.Lerp(oldZone.volumes[i], 0f, t / old
            yield return null;
        }
        for (int i = 0; i < oldZone.ambienceClips.Length; i++)
            ambienceSources[i].Stop();
    }

    // natauna jaona phaeda ina
    AmbienceZone newZone = zones[newZoneIndex];
    for (int i = 0; i < newZone.ambienceClips.Length; i++)
    {
        ambienceSources[i].clip = newZone.ambienceClips[i];
        ambienceSources[i].volume = 0f;
        ambienceSources[i].Play();
    }

    float t2 = 0;
    while (t2 < newZone.fadeDuration)
    {
        t2 += Time.deltaTime;
        for (int i = 0; i < newZone.ambienceClips.Length; i++)
            ambienceSources[i].volume = Mathf.Lerp(0f, newZone.volumes[i], t2 / newZon
        yield return null;
    }
}

```

```

        currentZoneIndex = newZoneIndex;
    }
}

```

24.4 phautasataepa saaaunada daijaaaina

phautasataepa saaaunadaera dharanasamauuha:

- maaataira upara haaataaa
- paaatharaera upara haaataaa
- kaaathaera upara haaataaa
- paaanaitae haaataaa
- taussaaaraera upara haaataaa
- dhaaatabaera upara haaataaa

phautasataepa saisataema kaoda:

```

public class FootstepSystem : MonoBehaviour
{
    [System.Serializable]
    public class SurfaceType
    {
        public string surfaceName;
        public AudioClip[] footstepClips;
        public float volume = 1f;
        public float pitchVariation = 0.1f;
    }

    public SurfaceType[] surfaceTypes;
    public float footstepInterval = 0.5f;
    private float lastFootstepTime;
    private AudioSource audioSource;

    void Start()
    {
        audioSource = GetComponent<AudioSource>();
    }

    public void PlayFootstep()
    {
        if (Time.time - lastFootstepTime < footstepInterval) return;

        SurfaceType surface = GetSurfaceType();
        if (surface != null && surface.footstepClips.Length > 0)
        {
            AudioClip clip = surface.footstepClips[Random.Range(0, surface.footstepClips.Length)];
            audioSource.pitch = 1f + Random.Range(-surface.pitchVariation, surface.pitchVariation);
            audioSource.PlayOneShot(clip, surface.volume);
            lastFootstepTime = Time.time;
        }
    }

    SurfaceType GetSurfaceType()
    {
        RaycastHit hit;
        if (Physics.Raycast(transform.position, Vector3.down, out hit, 2f))

```

```

    {
        foreach (SurfaceType surface in surfaceTypes)
        {
            if (hit.collider.CompareTag(surface.surfaceName))
                return surface;
        }
    }
    return surfaceTypes.Length > 0 ? surfaceTypes[0] : null;
}
}

```

24.5 auyaepana saaaunada daijaaaina

auyaepana saaaunadaera dharanasamauuha:

- phaaayaaarai saaaunada
- railaoda saaaunada
- imapayaaakata saaaunada
- halao saaaunada
- daraaaga saaaunada

auyaepana saaaunada kaoda:

```

public class WeaponSoundSystem : MonoBehaviour
{
    [System.Serializable]
    public class WeaponSoundSet
    {
        public string weaponName;
        public AudioClip fireSound;
        public AudioClip reloadSound;
        public AudioClip impactSound;
        public AudioClip holSound;
        public float fireVolume = 1f;
        public float firePitchVariation = 0.05f;
    }

    public WeaponSoundSet[] weaponSounds;
    private AudioSource audioSource;
    private WeaponSoundSet currentWeapon;

    void Start()
    {
        audioSource = GetComponent<AudioSource>();
    }

    public void SetWeapon(string weaponName)
    {
        foreach (WeaponSoundSet ws in weaponSounds)
        {
            if (ws.weaponName == weaponName)
            {
                currentWeapon = ws;
                break;
            }
        }
    }
}

```

```

    }

    public void PlayFireSound()
    {
        if (currentWeapon == null) return;
        audioSource.pitch = 1f + Random.Range(-currentWeapon.firePitchVariation, currentWeapon.firePitchVariation);
        audioSource.PlayOneShot(currentWeapon.fireSound, currentWeapon.fireVolume);
    }

    public void PlayReloadSound()
    {
        if (currentWeapon == null) return;
        audioSource.PlayOneShot(currentWeapon.reloadSound);
    }

    public void PlayImpactSound()
    {
        if (currentWeapon == null) return;
        audioSource.PlayOneShot(currentWeapon.impactSound);
    }
}

```

24.6 UI saaaunada daijaaaina

UI saaaunadaera dharanasamauha:

- baaatana kalaika
- maenau aupaena/kalaoja
- haobhaara saaaunada
- sailaekashana saaaunada
- naotaiphaikaeshana saaaunada

UI saaaunada kaoda:

```

public class UISoundManager : MonoBehaviour
{
    private static UISoundManager instance;
    public static UISoundManager Instance => instance;

    public AudioClip buttonClickSound;
    public AudioClip menuOpenSound;
    public AudioClip menuCloseSound;
    public AudioClip hoverSound;
    public AudioClip selectSound;
    public AudioClip notificationSound;
    public AudioClip errorSound;

    private AudioSource audioSource;

    void Awake()
    {
        if (instance == null)
        {
            instance = this;
            DontDestroyOnLoad(gameObject);
        }
        else Destroy(gameObject);
    }
}

```



```

        audioSource = gameObject.AddComponent<AudioSource>();
    }

    public void PlayButtonClick()
    {
        audioSource.PlayOneShot(buttonClickSound);
    }

    public void PlayMenuOpen()
    {
        audioSource.PlayOneShot(menuOpenSound);
    }

    public void PlayMenuClose()
    {
        audioSource.PlayOneShot(menuCloseSound);
    }

    public void PlayHover()
    {
        audioSource.PlayOneShot(hoverSound);
    }

    public void PlaySelect()
    {
        audioSource.PlayOneShot(selectSound);
    }

    public void PlayNotification()
    {
        audioSource.PlayOneShot(notificationSound);
    }

    public void PlayError()
    {
        audioSource.PlayOneShot(errorSound);
    }
}

```

24.7 ainaimai saaaunada daijaaaina

ainaimai saaaunadaera dharanasamauuha:

- garajana/aagaunaera shabada
- aakaramanaera shabada
- maritayaura shabada
- ayaaalaaarata shabada
- paaayaera shabada

ainaimai saaaunada kaoda:

```

public class EnemySoundSystem : MonoBehaviour
{
    [System.Serializable]
    public class EnemySoundSet
    {
        public string enemyType;
    }
}

```

```

        public AudioClip[] growlSounds;
        public AudioClip[] attackSounds;
        public AudioClip[] deathSounds;
        public AudioClip alertSound;
        public float volume = 1f;
    }

    public EnemySoundSet[] enemySounds;
    private AudioSource audioSource;
    private EnemySoundSet currentSoundSet;

    void Start()
    {
        audioSource = GetComponent<AudioSource>();
    }

    public void SetEnemyType(string enemyType)
    {
        foreach (EnemySoundSet ess in enemySounds)
        {
            if (ess.enemyType == enemyType)
            {
                currentSoundSet = ess;
                break;
            }
        }
    }

    public void PlayGrowl()
    {
        if (currentSoundSet == null || currentSoundSet.growlSounds.Length == 0) return;
        AudioClip clip = currentSoundSet.growlSounds[Random.Range(0, currentSoundSet.growl
        audioSource.PlayOneShot(clip, currentSoundSet.volume);
    }

    public void PlayAttack()
    {
        if (currentSoundSet == null || currentSoundSet.attackSounds.Length == 0) return;
        AudioClip clip = currentSoundSet.attackSounds[Random.Range(0, currentSoundSet.atta
        audioSource.PlayOneShot(clip, currentSoundSet.volume);
    }

    public void PlayDeath()
    {
        if (currentSoundSet == null || currentSoundSet.deathSounds.Length == 0) return;
        AudioClip clip = currentSoundSet.deathSounds[Random.Range(0, currentSoundSet.death
        audioSource.PlayOneShot(clip, currentSoundSet.volume);
    }

    public void PlayAlert()
    {
        if (currentSoundSet == null) return;
        audioSource.PlayOneShot(currentSoundSet.alertSound, currentSoundSet.volume);
    }
}

```

24.8 ainabhaaayaranamaenata saaaunada daijaaaina

ainabhaaayaranamaenata saaaunadaera dharanasamauha:

- darajaaa khaolaaa/banadha
- kaaacha bhaaangaaa
- dhaaatabaera shabada
- aagaunaera shabada
- paaanaira shabada

ainabhaaayaranamaenata saaaunada kaoda:

```
public class EnvironmentSoundSystem : MonoBehaviour
{
    [System.Serializable]
    public class EnvironmentSound
    {
        public string soundName;
        public AudioClip clip;
        public float volume = 1f;
        public float spatialBlend = 1f;
        public float minDistance = 1f;
        public float maxDistance = 50f;
    }

    public EnvironmentSound[] environmentSounds;
    private Dictionary<string, EnvironmentSound> soundDict;

    void Start()
    {
        soundDict = new Dictionary<string, EnvironmentSound>();
        foreach (EnvironmentSound es in environmentSounds)
            soundDict[es.soundName] = es;
    }

    public void PlayEnvironmentSound(string soundName, Vector3 position)
    {
        if (!soundDict.ContainsKey(soundName)) return;

        EnvironmentSound es = soundDict[soundName];
        GameObject tempObj = new GameObject("EnvSound");
        tempObj.transform.position = position;
        AudioSource source = tempObj.AddComponent<AudioSource>();
        source.clip = es.clip;
        source.volume = es.volume;
        source.spatialBlend = es.spatialBlend;
        source.minDistance = es.minDistance;
        source.maxDistance = es.maxDistance;
        source.Play();
        Destroy(tempObj, es.clip.length + 0.5f);
    }
}
```

24.9 harara gaema saaaunada daijaaaina (baishaessa asha)

harara gaemae saaaunadaera gaurautaba:

harara gaemae saaaunada halao sabachaeyae gaurautabapauurana upaaadaaana bhaaalao saaaunada daijaaaina khaelaoyaaadake bhaya daekhaaatae paaarae

harara saaaunadaera dharanasamaauha:

****1. ayaaamabaiyaenata saaaunada****

- jhakajhakae shabada
- gabhaiira shabaaasaparashabaaasa
- dauuraera garajana
- kaaachaera taukarao padaaara shabada

****2. jaaamapa sakayaaara saaaunada****

- achaenaaa shabada
- hathaaa garajana
- chaikaaara

****3. ayaaatamasaphaeraika saaaunada****

- dhaiirae dhaiirae baaadatae thaaakaaa shabada
- laukaiyae thaaakaaa shabada
- adarishaya shabada

harara saaaunada mayaaanaejaaara kaoda:

```
public class HorrorSoundManager : MonoBehaviour
{
    private static HorrorSoundManager instance;
    public static HorrorSoundManager Instance => instance;

    [Header("Ambient Sounds")]
    public AudioClip[] ambientClips;
    public float ambientInterval = 5f;

    [Header("Jump Scare Sounds")]
    public AudioClip[] jumpScareClips;
    public float jumpScareVolume = 1.5f;

    [Header("Atmospheric Sounds")]
    public AudioClip[] atmosphericClips;
    public float atmosphericInterval = 10f;

    private AudioSource ambientSource;
    private AudioSource atmosphericSource;
    private AudioSource jumpScareSource;

    void Awake()
    {
        if (instance == null)
        {
            instance = this;
            DontDestroyOnLoad(gameObject);
        }
        else Destroy(gameObject);

        InitializeAudioSources();
        StartCoroutine(PlayAmbientSounds());
        StartCoroutine(PlayAtmosphericSounds());
    }

    void InitializeAudioSources()
```

```

{
    ambientSource = gameObject.AddComponent<AudioSource>();
    ambientSource.loop = true;
    ambientSource.volume = 0.3f;

    atmosphericSource = gameObject.AddComponent<AudioSource>();
    atmosphericSource.loop = false;

    jumpScareSource = gameObject.AddComponent<AudioSource>();
    jumpScareSource.volume = jumpScareVolume;
}

IEnumerator PlayAmbientSounds()
{
    while (true)
    {
        yield return new WaitForSeconds(ambientInterval + Random.Range(-2f, 2f));
        AudioClip clip = ambientClips[Random.Range(0, ambientClips.Length)];
        ambientSource.clip = clip;
        ambientSource.Play();
    }
}

IEnumerator PlayAtmosphericSounds()
{
    while (true)
    {
        yield return new WaitForSeconds(atmosphericInterval + Random.Range(-3f, 3f));
        AudioClip clip = atmosphericClips[Random.Range(0, atmosphericClips.Length)];
        atmosphericSource.PlayOneShot(clip);
    }
}

public void PlayJumpScare()
{
    AudioClip clip = jumpScareClips[Random.Range(0, jumpScareClips.Length)];
    jumpScareSource.PlayOneShot(clip);
}

public void PlayHeartbeat(float intensity)
{
    // haridasapanadanaera shabada, intensity anauyaayaii bhalaiuma baaadaanaao
}

public void PlayWhisper()
{
    // phaisaphaisaaanaira shabada
}
}

```

24.10 adaiau maidalaauyayaaara kanasaepata (FMOD, Wwise)

FMOD kanasaepata:

FMOD halao aikatai janaparaiya adaiau maidalaauyayaaara yaaa gaema daebhaelapamaenatae bayabaharita haya

```

// FMOD bayabahaaaraera udaaaharana
using FMODUnity;
using FMOD.Studio;

```

```
public class FMODAudioManager : MonoBehaviour
{
    [EventRef]
    public string musicEventPath;
    public string sfxEventPath;

    private EventInstance musicInstance;
    private EventInstance sfxInstance;

    void Start()
    {
        musicInstance = RuntimeManager.CreateInstance(musicEventPath);
        musicInstance.start();
    }

    public void PlaySFX(string eventPath)
    {
        EventInstance sfx = RuntimeManager.CreateInstance(eventPath);
        sfx.start();
        sfx.release();
    }

    public void SetMusicParameter(string parameterName, float value)
    {
        musicInstance.setParameterByName(parameterName, value);
    }

    void OnDestroy()
    {
        musicInstance.release();
    }
}
```

Wwise kanasaepata:

```
// Wwise bayabahaaaraera udaaaharana
using UnityEngine;

public class WwiseAudioManager : MonoBehaviour
{
    public uint musicPlayingID;
    public uint sfxPlayingID;

    void Start()
    {
        AkSoundEngine.SetListenerPosition(Camera.main.transform.position);
    }

    public void PlayMusic(string eventName)
    {
        musicPlayingID = AkSoundEngine.PostEvent(eventName, gameObject);
    }

    public void PlaySFX(string eventName)
    {
        sfxPlayingID = AkSoundEngine.PostEvent(eventName, gameObject);
    }

    public void SetRTPC(string paramName, float value)
    {
        AkSoundEngine.SetRTPCValue(paramName, value, gameObject);
    }
}
```

24.11 adaiau apataimaaaijaeshana taipasa

adaiau apataimaaaijaeshanaera kaaushala:

1. adaiau kamaparaeshana

```
// MP3: chhaota saaija, kama kaoyaaalaitai
// OGG: maaajhaaarai saaija, bhaaalao kaoyaaalaitai
// WAV: bada saaija, saeraaa kaoyaaalaitai
```

2. adaiau sataraimai

```
// chhaota adaiau phaaailaera janaya kamaparaesada bayabahaaara karao
```

3. adaiau paulai

```
// maemarai saebha karao
```

4. adaiau kaoyaaalaitai saetaisa

```
// maobaaaaila: 22050 Hz, Mono
// PC: 44100 Hz, Stereo
```

24.12 saupaaaraishakarita taulasa

adaiau aidaitai taulasa:

- **Audacity**: pharai, aupaena saorasa
- **Adobe Audition**: paraphaeshanaaala
- **Reaper**: saaasharayaii mauulayaera
- **FL Studio**: sagaiita taairaira janaya

saaaunada iphaekata laaaibaraera:

- **Freesound.org**: pharai saaaaunada iphaekata
- **Zapsplat**: pharai aiba paeida
- **Soundsnap**: paeida
- **BBC Sound Effects**: pharai

adaiau maidalaauyayaaara:

- **FMOD**: pharai (inadai daebhaelapaaaradaera janaya)
- **Wwise**: pharai (inadai daebhaelapaaaradaera janaya)
- **Unity Audio**: bailata-ina

24.13 parayaaakataisa aikasaarasaaaija

aikasaaarasaaaaja 1: phautasataepa saisataema taairai karao

aikatai phautasataepa saisataema taairai karao yaaa:

- baibhainana parissathatalaera janaya aalaaadaaa shabada bayabahaaara karabae
- haaataaa aiba daaudaaanaora janaya aalaaadaaa shabada daebae
- shabadaera paichae bhayaaariyaeshana yaoga karabae

aikasaaarasaaaaja 2: harara saaaunada saisataema taairai karao

aikatai harara saaaunada saisataema taairai karao yaaa:

- ayaaamabaiyaenata saaaunada palae karabae
- jaaamapa sakayaaara saaaunada palae karabae
- ayaaatamasaphaeraika saaaunada palae karabae

aikasaaarasaaaaja 3: maiujaika karasaphaeda saisataema taairai karao

aikatai maiujaika karasaphaeda saisataema taairai karao yaaa:

- dautai maiujaikaera madhayae samautha taraaanajaishana karabae
- baibhainana jaonaera janaya aalaaadaaa maiujaika palae karabae

24.14 bhaula yaaa aidaiyae chalatae habae

bhaula 1: atairaikata shabada bayabahaaara karaaa

// khaaaraaapa - aikai saaathae anaeka shabada palae karaaa
// khaelaoyaaaada bairakata haya

bhaula 2: shabadaera bhalaiuma samaaana naaa raaakhaaa

// khaaaraaapa - kaichhau shabada khauba jaoraaalao, kaichhau khauba maridau
// khaelaoyaaaada shaunatae paaaya naaa

bhaula 3: raipaitaitaibha shabada bayabahaaara karaaa

// khaaaraaapa - aikai shabada baaarabaaara palae karaaa
// khaelaoyaaaada kalaaanata haya

bhaula 4: adaiau apataimaaaijaeshana upaekassaaa karaaa

// khaaaraaapa - bada adaiau phaaaila bayabahaaara karaaa
// maemarai samasayaaa haya

24.15 taipasa au paraaamarasha

taipasa 1: saaaunada daijaaaina shaurau thaekae karao

// bhaaalao - gaema daijaaainaera saaathae saaathae saaaunada daijaaaina karao

taipasa 2: bhayaaaraaishana yaoga karao

// bhaaalao - aikai dharanaera shabadaera janaya aikaaadhaika bhayaaaraiyaeshana raaakhao

taipasa 3: 3D saaaunada bayabahaaara karao

// bhaaalao - sapayaaashaaala balaenada daiyae 3D iphaekata taaairai karao

taipasa 4: adaiau paraophaaailai karao

// bhaaalao - paaarapharamayaaanasa manaitara karao aiba apataimaaaija karao

taipasa 5: palaetaesatai karao

// bhaaalao - khaelaoyaaadadaera kaaachha thaekae phaidabayaaaka naiyae saaaunada unanata

24.16 saarasakassaepa

aai adhayaaayae aamaraaa saaaunada daijaaainaera baibhainana daika naiyae aalaochanaaa karaechhai mauula payaenatagaulao halao:

1. ****gaema adaiau paaaipalaaaina****: raekaradai thaekae palaebayaaaka parayanata parakaraiyaaa
2. ****maiujaiika daijaaaina****: gaemaera paraibaesha aiba aabaega taaairai
3. ****ayaaamabaiyaenasa daijaaaina****: paraibaeshaera shabada
4. ****phautasataepa saaaunada****: chalaachalaera shabada
5. ****aulyaepana saaaunada****: aakaramanaera shabada
6. ****UI saaaunada****: inataaaraayaaakashanaera shabada
7. ****ainaimai saaaunada****: shataraura shabada
8. ****ainabhaaayaranamaenata saaaunada****: paraibaeshaera shabada
9. ****harara saaaunada****: bhayakara shabada
10. ****adaiau apataimaaaijaeshana****: paaarapharamayaaanasa unanata karaaa

saaaunada daijaaaina halao gaemaera abhaijanyataaakae samaridadha karaaara aikatai gaurautabapauurana upaaadaana

****parabarataii** adhayaaaya: apataimaaaijaeshana -- kaiibhaaabaee gaemaera paaarapharamayaaanasa unanata karabaena******

PART 25

adhayaaaya 25: apataimaaaijaeshana

adhayaaaya 25: apataimaaaijaeshana

shaekhaara udadaeshaya

aii adhayaaayae aamaraaa gaema daebhaelapamaenatae apataimaaaijaeshana naiyae aalaochanaaa karaba apataimaaaijaeshana halao gaemaera paaarapharamayaaanasa unanata karaara parakariyaaa aii adhayaaaya shaessae aapanai jaaanabaena kaiibhaaabaee FPS baaadaaabaena, kaiibhaaabaee CPU/GPU bayabahaaara kamaaabaena, aiba kaiibhaaabaee maemarai apataimaaaija karabaena

25.1 FPS apataimaaaijaeshana (FPS Optimization)

FPS (Frames Per Second) halao gaemaera samauthanaesa nairadhaaarana karae

FPS baaadaaanaora kaaushala:

****1. dara kala raidaaakashana****

```
// Static Batching bayabahaaara karao
// Dynamic Batching bayabahaaara karao
// GPU Instancing bayabahaaara karao
```

// da

****2. LOD (Level of Detail)****

// da

```
public class LODController : MonoBehaviour
{
    public LODGroup lodGroup;
    public float[] lodDistances;

    void Update()
    {
        float distance = Vector3.Distance(transform.position, Camera.main.transform.position);
        for (int i = 0; i < lodDistances.Length; i++)
        {
            if (distance < lodDistances[i])
            {
                lodGroup.ForceLOD(i);
                break;
            }
        }
    }
}
```

****3. akalaushana kaaalai****

```
// Occlusion Culling saetaaapa karao
// Dynamic Occludee bayabahaaara karao
```

// ada

25.2 CPU apataimaaaijaeshana

CPU apataimaaaijaeshanaera kaaushala:

****1. kaoda apataimaaaijaeshana****

```
void Update()
{
    GameObject player = GameObject.Find("Player");
}

// bhaaalao - raephaaaraenasa saebha karao
private GameObject player;
void Start()
{
    player = GameObject.Find("Player");
}
void Update()
{
    // player bayabahaaara karao
}
```

// kha

****2. karautaina apataimaaaijaeshana****

```
void Update()
{
    StartCoroutine(MyCoroutine());
}

// bhaaalao - aikabaaara shaurau karao
void Start()
{
    StartCoroutine(MyCoroutine());
}
```

// kha

****3. GC (Garbage Collection) mainaimaaaija****

```
void Update()
{
    string temp = "Score: " + score;
}

// bhaaalao - StringBuilder bayabahaaara karao
private StringBuilder sb = new StringBuilder();
void Update()
{
    sb.Clear();
    sb.Append("Score: ");
    sb.Append(score);
    scoreText.text = sb.ToString();
}
```

// kha

****4. abajaekata paulai****

```

public class ObjectPool : MonoBehaviour
{
    public GameObject prefab;
    public int poolSize = 10;
    private Queue<GameObject> pool = new Queue<GameObject>();

    void Start()
    {
        for (int i = 0; i < poolSize; i++)
        {
            GameObject obj = Instantiate(prefab);
            obj.SetActive(false);
            pool.Enqueue(obj);
        }
    }

    public GameObject GetObject()
    {
        if (pool.Count > 0)
        {
            GameObject obj = pool.Dequeue();
            obj.SetActive(true);
            return obj;
        }
        return Instantiate(prefab);
    }

    public void ReturnObject(GameObject obj)
    {
        obj.SetActive(false);
        pool.Enqueue(obj);
    }
}

```

25.3 GPU apataimaaaijaeshana

GPU apataimaaaijaeshanaera kaaushala:

1. shaedaaara apataimaaaijaeshana

```

// apataimaaaijada shaedaaara bayabahaaara karao
// LOD shaedaaara bayabahaaara karao

```

2. taekasachaaara apataimaaaijaeshana

```

// taekasachaaara kamaparaeshana bayabahaaara karao
// Mipmaps bayabahaaara karao
// taekasachaaara ayaaatalaaasa bayabahaaara karao

```

3. dara kala raidaaakashana

```

// bayaaaachai bayabahaaara karao
// GPU Instancing bayabahaaara karao

```

25.4 maemarai apataimaaaijaeshana

maemarai apataimaaaijaeshanaera kaaushala:

****1. taekasachaaara maemarai****

```
// taekasachaaara sataraimai bayabahaaara karao
// taekasachaaara paulai bayabahaaara karao
```

****2. adaiiau maemarai****

```
// adaiiau sataraimai bayabahaaara karao
// adaiiau paulai bayabahaaara karao
```

****3. abajaekata maemarai****

```
// abajaekata raiiuja karao
// abajaekata daisataraya karaaara samaya naotaisha karao
```

25.5 dara kala raidaaakashana

dara kala kamaaanaora kaaushala:

```
// Static Batching
// Unity saetaisae Static Batching ainaaabala karao

// Dynamic Batching
// chhaota maeshaera janaya Dynamic Batching bayabahaaara karao

// GPU Instancing
public class GPUInstancing : MonoBehaviour
{
    public Mesh mesh;
    public Material material;
    public int instanceCount = 1000;
    private Matrix4x4[] matrices;

    void Start()
    {
        matrices = new Matrix4x4[instanceCount];
        for (int i = 0; i < instanceCount; i++)
        {
            Vector3 position = new Vector3(Random.Range(-50f, 50f), 0, Random.Range(-50f, 50f));
            matrices[i] = Matrix4x4.TRS(position, Quaternion.identity, Vector3.one);
        }
    }

    void Update()
    {
        Graphics.DrawMeshInstanced(mesh, 0, material, matrices);
    }
}

// Material Batching
// aikai mayaaataeraiyaaala bayabahaaara karao
```

25.6 LOD (Level of Detail)

LOD saisataema kaoda:

```
[System.Serializable]
public class LODLevel
{
    public Mesh mesh;
    public float distance;
    public Material[] materials;
}

public class LODSystem : MonoBehaviour
{
    public LODLevel[] lodLevels;
    public float updateInterval = 0.5f;
    private float lastUpdateTime;
    private MeshFilter meshFilter;
    private MeshRenderer meshRenderer;
    private int currentLOD = -1;

    void Start()
    {
        meshFilter = GetComponent<MeshFilter>();
        meshRenderer = GetComponent<MeshRenderer>();
    }

    void Update()
    {
        if (Time.time - lastUpdateTime >= updateInterval)
        {
            UpdateLOD();
            lastUpdateTime = Time.time;
        }
    }

    void UpdateLOD()
    {
        float distance = Vector3.Distance(transform.position, Camera.main.transform.position);
        int newLOD = 0;

        for (int i = 0; i < lodLevels.Length; i++)
        {
            if (distance >= lodLevels[i].distance)
                newLOD = i;
        }

        if (newLOD != currentLOD)
        {
            currentLOD = newLOD;
            ApplyLOD(currentLOD);
        }
    }

    void ApplyLOD(int lodIndex)
    {
        if (lodIndex < lodLevels.Length)
        {
            meshFilter.mesh = lodLevels[lodIndex].mesh;
            meshRenderer.materials = lodLevels[lodIndex].materials;
        }
    }
}
```

25.7 akalaushana kaaalai

akalaushana kaaalai kaoda:

```
// akalaushana kaaalai saetaaapa
public class OcclusionCulling : MonoBehaviour
{
    public float checkInterval = 1f;
    private float lastCheckTime;
    private Renderer[] renderers;
    private bool isVisible = true;

    void Start()
    {
        renderers = GetComponentsInChildren<Renderer>();
    }

    void Update()
    {
        if (Time.time - lastCheckTime >= checkInterval)
        {
            CheckVisibility();
            lastCheckTime = Time.time;
        }
    }

    void CheckVisibility()
    {
        Plane[] planes = GeometryUtility.CalculateFrustumPlanes(Camera.main);
        Bounds bounds = new Bounds(transform.position, Vector3.one * 10f);

        bool visible = GeometryUtility.TestPlanesAABB(planes, bounds);

        if (visible != isVisible)
        {
            isVisible = visible;
            SetRenderersVisible(isVisible);
        }
    }

    void SetRenderersVisible(bool visible)
    {
        foreach (Renderer r in renderers)
            r.enabled = visible;
    }
}
```

25.8 taekasachaaara saaaija apataimaaaijaeshana

taekasachaaara apataimaaaijaeshana kaoda:

```
// taekasachaaara apataimaaaijaeshana
public class TextureOptimizer : MonoBehaviour
{
    public Texture2D[] textures;
    public int maxTextureSize = 1024;
    public TextureImporterFormat format = TextureImporterFormat.ASTC_6x6;
```

```

void Start()
{
    OptimizeTextures();
}

void OptimizeTextures()
{
    foreach (Texture2D tex in textures)
    {
        // taekasachaaara saaaija kamaaaau
        if (tex.width > maxTextureSize || tex.height > maxTextureSize)
        {
            ResizeTexture(tex, maxTextureSize);
        }

        // taekasachaaara kamaparaeshana parayaoga karao
        // Unity Editor-ai saeta karao
    }
}

void ResizeTexture(Texture2D tex, int maxSize)
{
    int newWidth = tex.width > maxSize ? maxSize : tex.width;
    int newHeight = tex.height > maxSize ? maxSize : tex.height;

    Texture2D resized = new Texture2D(newWidth, newHeight, tex.format, false);
    Graphics.CopyTexture(tex, resized);

    // raisaaaijada taekasachaaara bayabahaaara karao
}
}

```

25.9 laaaitai apataimaaaijaeshana

laaaitai apataimaaaijaeshana kaoda:

```

// laaaitai apataimaaaijaeshana
public class LightingOptimizer : MonoBehaviour
{
    public Light[] lights;
    public float shadowDistance = 50f;
    public int shadowResolution = 1024;

    void Start()
    {
        OptimizeLighting();
    }

    void OptimizeLighting()
    {
        // shayaaadao daisataenasa saeta karao
        QualitySettings.shadowDistance = shadowDistance;

        // shayaaadao raejaolaiushana saeta karao
        QualitySettings.shadowResolution = ShadowResolution.Medium;

        // laaaita kauilaitai saeta karao
        foreach (Light light in lights)
        {
            if (light.type == LightType.Directional)

```



```

        {
            light.shadows = LightShadows.Soft;
            light.shadowStrength = 0.8f;
        }
        else
        {
            light.shadows = LightShadows.None;
        }
    }
}

void Update()
{
    // dauurae thaaakaaa laaaita banadha karao
    foreach (Light light in lights)
    {
        float distance = Vector3.Distance(transform.position, Camera.main.transform.po
        light.enabled = distance < shadowDistance;
    }
}
}

```

25.10 phaijaikasa apataimaaaijaeshana

phaijaikasa apataimaaaijaeshana kaoda:

```

// phaijaikasa apataimaaaijaeshana
public class PhysicsOptimizer : MonoBehaviour
{
    public float fixedTimestep = 0.02f;
    public int defaultSolverIterations = 6;
    public int defaultSolverVelocityIterations = 1;

    void Start()
    {
        OptimizePhysics();
    }

    void OptimizePhysics()
    {
        // Fixed Timestep saeta karao
        Time.fixedDeltaTime = fixedTimestep;

        // Solver Iterations saeta karao
        Physics.defaultSolverIterations = defaultSolverIterations;
        Physics.defaultSolverVelocityIterations = defaultSolverVelocityIterations;

        // Physics Layer Matrix saeta karao
        // aparayaojanaiiya kaolaishana banadha karao
    }

    void Update()
    {
        // dauurae thaaakaaa phaijaikasa abajaekata daisaebala karao
        Collider[] colliders = FindObjectsOfType<Collider>();
        foreach (Collider col in colliders)
        {
            float distance = Vector3.Distance(col.transform.position, Camera.main.transfor
            col.enabled = distance < 100f;
        }
    }
}

```

```

    }
}
}

```

25.11 paaarataikaelasa apataimaaaijaeshana

paaarataikaelasa apataimaaaijaeshana kaoda:

```

// paaarataikaelasa apataimaaaijaeshana
public class ParticleOptimizer : MonoBehaviour
{
    public ParticleSystem[] particles;
    public float maxDistance = 50f;

    void Update()
    {
        OptimizeParticles();
    }

    void OptimizeParticles()
    {
        foreach (ParticleSystem ps in particles)
        {
            float distance = Vector3.Distance(ps.transform.position, Camera.main.transform.position);

            if (distance > maxDistance)
            {
                ps.Stop();
            }
            else
            {
                if (!ps.isPlaying)
                {
                    ps.Play();

                    // dauurataba anauyaaayaii paaarataikaela sakhayaaa kamaaaau
                    var main = ps.main;
                    main.maxParticles = Mathf.Lerp(100, 10, distance / maxDistance);
                }
            }
        }
    }
}

```

25.12 abajaekata paulai

abajaekata paulai kaoda:

```

// abajaekata paulai saisataema
public class ObjectPoolSystem : MonoBehaviour
{
    private static ObjectPoolSystem instance;
    public static ObjectPoolSystem Instance => instance;

    [System.Serializable]
    public class Pool
    {

```

```

        public string tag;
        public GameObject prefab;
        public int size;
    }

    public List<Pool> pools;
    private Dictionary<string, Queue<GameObject>> poolDictionary;

    void Awake()
    {
        if (instance == null)
        {
            instance = this;
            DontDestroyOnLoad(gameObject);
        }
        else Destroy(gameObject);

        poolDictionary = new Dictionary<string, Queue<GameObject>>();
        foreach (Pool pool in pools)
        {
            Queue<GameObject> objectPool = new Queue<GameObject>();
            for (int i = 0; i < pool.size; i++)
            {
                GameObject obj = Instantiate(pool.prefab);
                obj.SetActive(false);
                objectPool.Enqueue(obj);
            }
            poolDictionary[pool.tag] = objectPool;
        }
    }

    public GameObject SpawnFromPool(string tag, Vector3 position, Quaternion rotation)
    {
        if (!poolDictionary.ContainsKey(tag))
        {
            Debug.LogWarning("Pool with tag " + tag + " doesn't exist.");
            return null;
        }

        GameObject objectToSpawn = poolDictionary[tag].Dequeue();
        objectToSpawn.SetActive(true);
        objectToSpawn.transform.position = position;
        objectToSpawn.transform.rotation = rotation;

        IPooledObject pooledObj = objectToSpawn.GetComponent<IPooledObject>();
        if (pooledObj != null)
        {
            pooledObj.OnObjectSpawn();
        }

        poolDictionary[tag].Enqueue(objectToSpawn);
        return objectToSpawn;
    }
}

public interface IPooledObject
{
    void OnObjectSpawn();
}

// bayabahaaaraera udaaaharana
public class Bullet : MonoBehaviour, IPooledObject
{
    public float speed = 20f;

```

```

public float lifetime = 3f;
private float spawnTime;

public void OnObjectSpawn()
{
    spawnTime = Time.time;
    GetComponent<Rigidbody>().velocity = transform.forward * speed;
}

void Update()
{
    if (Time.time - spawnTime >= lifetime)
    {
        ObjectPoolSystem.Instance.SpawnFromPool("Bullet", transform.position, Quaterni
    }
}
}

```

25.13 apataimaaaijaeshana chaekalaisata

OPTIMIZATION CHECKLIST

FPS apataimaaaijaeshana
 Draw Call raidaaakashana
 LOD saisataema saetaaapa
 Occlusion Culling saetaaapa
 raenadaaarai apataimaaaijaeshana
 CPU apataimaaaijaeshana
 kaoda apataimaaaijaeshana
 GC mainaimaaaija
 abajaekata paulai
 karautaina apataimaaaijaeshana
 GPU apataimaaaijaeshana
 shaedaaara apataimaaaijaeshana
 taekasachaaara apataimaaaijaeshana
 mayaaataeraiyaaala bayaaachai
 GPU Instancing
 maemarai apataimaaaijaeshana
 taekasachaaara saaaija apataimaaaija
 adaiau apataimaaaijaeshana
 abajaekata raiiuj
 maemarai laika chaeka
 phaijaikasa apataimaaaijaeshana
 Fixed Timestep saetaisa
 Solver Iterations
 Physics Layer Matrix
 kaolaishana apataimaaaijaeshana
 paaarataikaelasa apataimaaaijaeshana
 paaarataikaela sakhayaaa kamaaanao
 dauurataba anauyaaayaii kamaiunaitai
 LOD bayabahaaara
 laaaitai apataimaaaijaeshana
 shayaaadao daisataenasa
 laaaita kauilaitai
 baekada laaaitai
 adaiau apataimaaaijaeshana
 adaiau kamaparaeshana
 adaiau sataraimai
 adaiau paulai

25.14 paraophaaaailai taulasa saupaaaraisha

Unity paraophaaaailai taulasa:

- **Unity Profiler**: CPU, GPU, Memory, Audio paraophaaaailai
- **Frame Debugger**: Draw Call baishalaessana
- **Memory Profiler**: maemarai baishalaessana

baaairaera taulasa:

- **RenderDoc**: GPU paraophaaaailai
- **Intel GPA**: paaarapharamayaaanasa ayaaanaaalaaisaisa
- **NVIDIA Nsight**: GPU apataimaaaijaeshana
- **AMD Radeon GPU Profiler**: GPU apataimaaaijaeshana

paraophaaaailai kaoda:

```
// saimapala paaarapharamayaaanasa manaitara
public class PerformanceMonitor : MonoBehaviour
{
    public Text fpsText;
    public Text drawCallsText;
    public Text trianglesText;
    public Text memoryText;
    private float fps;
    private float deltaTime;

    void Update()
    {
        deltaTime += (Time.unscaledDeltaTime - deltaTime) * 0.1f;
        fps = 1.0f / deltaTime;
        fpsText.text = "FPS: " + Mathf.Ceil(fps);

        // Draw Calls
        drawCallsText.text = "Draw Calls: " + UnityStats.drawCalls;

        // Triangles
        trianglesText.text = "Triangles: " + UnityStats.triangles;

        // Memory
        long memory = System.GC.GetTotalMemory(false);
        memoryText.text = "Memory: " + (memory / 1024 / 1024) + " MB";
    }
}
```

25.15 palayaaatapharama-sapaesaiphaika apataimaaaijaeshana

maobaaaaila apataimaaaijaeshana:

```
// maobaaaaila apataimaaaijaeshana
public class MobileOptimizer : MonoBehaviour
{
    void Start()
```

```

{
    // raejaolaiushana kamaaaaau
    Screen.SetResolution(1280, 720, true);

    // kaoyaaalaitai saetaisa
    QualitySettings.SetQualityLevel(0);

    // shayaaadao banadha karao
    QualitySettings.shadows = ShadowQuality.Disable;

    // ayaanaanaisaotarapaika phailataaarai kamaaaaau
    QualitySettings.anisotropicFiltering = AnisotropicFiltering.Disable;
}
}

```

PC apataimaaaijaeshana:

```

// PC apataimaaaijaeshana
public class PCOptimizer : MonoBehaviour
{
    void Start()
    {
        // haaai kaoyaaalaitai saetaisa
        QualitySettings.SetQualityLevel(QualitySettings.names.Length - 1);

        // shayaaadao ainaaabala
        QualitySettings.shadows = ShadowQuality.All;

        // ayaanaanaisaotarapaika phailataaarai ainaaabala
        QualitySettings.anisotropicFiltering = AnisotropicFiltering.ForceEnable;
    }
}

```

kanasaola apataimaaaijaeshana:

```

// kanasaola apataimaaaijaeshana
// phaikasada haaaradaauyayaaaraera janaya apataimaaaija karao
// 30 FPS baaa 60 FPS taaaragaeta raaakhao
// maemarai laimaita maenae chalao

```

25.16 parayaaakataisa aikasaaarasaaaija

aikasaaarasaaaija 1: dara kala raidaakashana

aikatai gaema sainae dara kala raidaakashana karao:

- Static Batching bayabahaaara karao
- Dynamic Batching bayabahaaara karao
- GPU Instancing bayabahaaara karao

aikasaaarasaaaija 2: abajaekata paulai

aikatai baulaeta paulai saisataema taairai karao:

- baulaeta paula taairai karao
- baulaeta sapana karao
- baulaeta raitaarana karao

aikasaaarasaaaija 3: LOD saisataema

aikatai LOD saisataema taairai karao:

- tainatai LOD laebhaela taairai karao
- dauurataba anauyaaayaii LOD paraibaratana karao
- paaarapharamayaaanasa manaitara karao

25.17 bhaula yaaa aidaiyae chalatae habae

bhaula 1: paraimayaaachaura apataimaaaijaeshana

```
// khaaaraaapa - gaema taairai naaa karaei apataimaaaija karaaa
// parathamae phaichaaara taairai karao, taaarapara apataimaaaija karao
```

bhaula 2: paraophaaaailai chhaadaaaa apataimaaaijaeshana

```
// khaaaraaapa - anaumaaana karae apataimaaaija karaaa
// parathamae paraophaaaaila karao, taaarapara apataimaaaija karao
```

bhaula 3: saba kaichhau apataimaaaija karaaara chaessataaa karaaa

```
// khaaaraaapa - saba kaichhau apataimaaaija karaaara chaessataaa karaaa
// gaurautabapaurana ashae phaokaaasa karao
```

bhaula 4: apataimaaaijaeshanaera para taesata naaa karaaa

```
// khaaaraaapa - apataimaaaijaeshanaera para taesata naaa karae saebha karaaa
// sabasamaya taesata karao
```

25.18 taipasa au paraaamarasha

taipasa 1: paraophaaaailai daiyae shaurau karao

```
// bhaaalao - parathamae paraophaaaaila karao, taaarapara apataimaaaija karao
// kaothaaaya samasayaaa aachhae saetaaa baujhao
```

taipasa 2: baenyachamaaaraka karao

```
// bhaaalao - taaaragaeta palayaaatapharamae baenyachamaaaraka karao
// 30 FPS baaa 60 FPS taaaragaeta raaakhao
```

taipasa 3: dhaaapae dhaaapae apataimaaaija karao

```
// bhaaalao - aikabaaarae aikatai karae apataimaaaija karao
// parataitai paraibaratanaera para taesata karao
```

taipasa 4: dakaumaenataeshana raaakhao

```
// bhaaalao - apataimaaaijaeshanaera naota raaakhao
// bhabaissayatae raephaaaraenasa haisaebae kaaaja karabae
```

taipasa 5: taimaera saaathae shaeyaaara karao

```
// bhaaalao - apataimaaaijaeshanaera abhaijanyataaa shaeyaaara karao
// taimaera sabaaai shaikhabae
```

25.19 saaarasakassaepa

aii adhayaayae aamaraaa apataimaaaijaeshanaera baibhainana daika naiyae aalaochanaaa karaechhai mauula payaenatagaulao halao:

1. ****FPS apataimaaaijaeshana****: dara kala raidaakashana, LOD, Occlusion Culling
2. ****CPU apataimaaaijaeshana****: kaoda apataimaaaijaeshana, GC mainaimaaaija, abajaekata paulai
3. ****GPU apataimaaaijaeshana****: shaedaaara, taekasachaaara, mayaaataeraiyaaala
4. ****maemarai apataimaaaijaeshana****: taekasachaaara, adaiau, abajaekata
5. ****phaijaikasa apataimaaaijaeshana****: Fixed Timestep, Solver Iterations
6. ****paaarataikaelasa apataimaaaijaeshana****: paaarataikaela sakhayaaa, dauurataba
7. ****laaaitai apataimaaaijaeshana****: shayaaadao, laaaita kauilaitai
8. ****apataimaaaijaeshana chaekalaisata****: saba kaichhau chaeka karao
9. ****paraophaaailai taulasa****: Unity Profiler, Frame Debugger
10. ****palayaaatapharama-sapaesaiphaika****: maobaaaila, PC, kanasaola

apataimaaaijaeshana halao gaema daebhaelapamaenataera aikatai gaurautabapauurana asha sathaika apataimaaaijaeshana gaemakae samautha aiba paaarapharamayaaanata karae

****parabarataii adhayaayae**: QA aiba taesatai -- kaiibhaaaba gaemaera maaana yaaachaaai karabaena**

PART 26

adhayaaaya 26: QA aiba taesatai

adhayaaaya 26: QA aiba taesatai

shaekhaara udadaeshaya

aai adhayaaayae aamaraaa gaema daebhaelapamaenatae QA (Quality Assurance) aiba taesatai naiyae aalaochanaaa karaba QA halao gaemaera maaana naishachaita karaara parakaraiyaaa aii adhayaaaya shaessae aapanai jaaanabaena kaiibhaaaba gaema taesata karabaena, kaiibhaaaba baaaga raipaorata karabaena, aiba kaiibhaaaba QA taima paraichaaalanaaa karabaena

26.1 taesataiyaera dharanasamauuha (Testing Types)

1. phaaashanaaala taesatai (Functional Testing)

gaemaera phaichaaaragaulao sathaiabhaaaba kaaaja karachhae kainaaa yaaachaaa karaaa

```
public class FunctionalTest : MonoBehaviour
{
    public PlayerController player;
    public HealthSystem health;

    void Start() { RunTests(); }

    void RunTests()
    {
        TestPlayerMovement();
        TestHealthSystem();
    }

    void TestPlayerMovement()
    {
        Vector3 startPos = player.transform.position;
        player.Move(Vector3.forward);
        float distance = Vector3.Distance(startPos, player.transform.position);
        Debug.Log(distance > 0 ? "maubhamaenata: PASS" : "maubhamaenata: FAIL");
    }

    void TestHealthSystem()
    {
        float initial = health.currentHealth;
        health.TakeDamage(10);
        Debug.Log(health.currentHealth == initial - 10 ? "haelatha: PASS" : "haelatha: FAI");
    }
}
```

2. gaemapalae taesatai (Gameplay Testing)

gaemaera khaelaaara abhaijanyataaa, bayaaalaenasa, kathainataaa yaaachaaai karaaa

3. paaarapharamayaaanasa taesatai (Performance Testing)

FPS, maemarai, CPU/GPU bayabahaaara yaaachaaai karaaa

4. kamapayaaataibailaitai taesatai (Compatibility Testing)

baibhainana palayaaatapharamae gaema chalachhae kainaaa yaaachaaai karaaa

5. UI taesatai (UI Testing)

UI ailaimaenatagaulao sathaikabhaaabaee kaaaja karachhae kainaaa yaaachaaai karaaa

6. saebha/laoda taesatai (Save/Load Testing)

gaema saebha aiba laoda sathaikabhaaabaee kaaaja karachhae kainaaa yaaachaaai karaaa

7. sataresa taesatai (Stress Testing)

gaemaera saiimaaanaaa paraiikassaaa karaaa

8. baaaga taesatai (Bug Testing)

baaaga khaujae baera karaaa aiba raipaorata karaaa

26.2 paraphaeshanaaala baaaga raipaorata taemapalaeta

```
BUG REPORT TEMPLATE

baaaga aaidai: BUG-001
shairaonaaama: palaeyaaara daeyaaalaera madhayae dhaukae yaaachachhae
saebhaeraitai: High (Critical)
satayaaataaasa: Open
ayaaasaaainada: Developer Name
raipaorataeda: 2024-01-15
barananaaa:
palaeyaaara WASD bayabahaaara karae daeyaaalaera madhayae dhaukae
yaaachachhae aitari aikatai gaurautara baaaga
paunaraupaaadanaera dhaaapa:
1. gaema shaurau karao
2. palaeyaaarakae daeyaaalaera kaaachhae naiyae yaaaau
3. WASD daiyae daeyaaalaera daikae yaaaau
4. palaeyaaara daeyaaalaera madhayae dhaukae yaaabae
paratayaaashaita aacharana:
palaeyaaarakae daeyaaalae thaaamatae habae
parakarita aacharana:
palaeyaaara daeyaaalaera madhayae dhaukae yaaaya
ainabhaaayaranamaenata:
OS: Windows 10
GPU: NVIDIA GTX 1060
RAM: 16GB
Unity Version: 2022.3.15f1
sakarainashata/bhaidaiu:
[sakarainashata yaoga karao]
atairaikata naota:
```

aii baaagatai palaeyaaarakae gaemaera baaairae yaetae daeya

baaaga raipaorata kaoda:

```
[System.Serializable]
public class BugReport
{
    public string bugId;
    public string title;
    public BugSeverity severity;
    public BugStatus status;
    public string description;
    public string[] reproductionSteps;
    public string expectedBehavior;
    public string actualBehavior;
    public string environment;
    public string screenshotPath;
    public string reporterName;
    public string reportDate;
    public string assignee;
}

public enum BugSeverity { Low, Medium, High, Critical }
public enum BugStatus { Open, InProgress, Fixed, Verified, Closed }

public class BugReportManager : MonoBehaviour
{
    public List<BugReport> bugReports = new List<BugReport>();

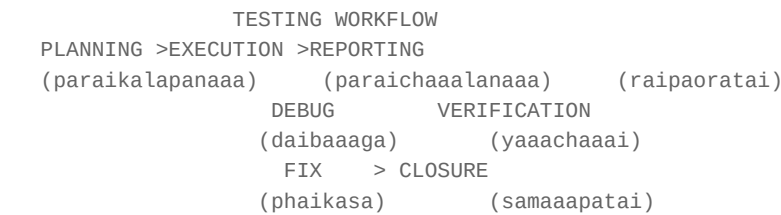
    public void CreateBugReport(string title, BugSeverity severity, string description,
        string[] steps, string expected, string actual)
    {
        BugReport report = new BugReport
        {
            bugId = "BUG-" + (bugReports.Count + 1).ToString("D3"),
            title = title,
            severity = severity,
            status = BugStatus.Open,
            description = description,
            reproductionSteps = steps,
            expectedBehavior = expected,
            actualBehavior = actual,
            environment = SystemInfo.deviceModel + " | " + SystemInfo.operatingSystem,
            reporterName = "QA Tester",
            reportDate = System.DateTime.Now.ToString(),
            assignee = ""
        };

        bugReports.Add(report);
        Debug.Log("baaaga raipaorata taairai halao: " + report.bugId);
    }

    public void UpdateBugStatus(string bugId, BugStatus newStatus)
    {
        BugReport report = bugReports.Find(b => b.bugId == bugId);
        if (report != null)
        {
            report.status = newStatus;
            Debug.Log(bugId + " satayaaataaasa aapadaeta halao: " + newStatus);
        }
    }
}
```

```
public void AssignBug(string bugId, string assignee)
{
    BugReport report = bugReports.Find(b => b.bugId == bugId);
    if (report != null)
    {
        report.assignee = assignee;
        Debug.Log(bugId + " ayaasaaaaina halao: " + assignee);
    }
}
```

26.3 taesatai auyaaarakaphalao



26.4 baaaga saebhaeritai laebhaela

26.5 palayaaatapharama-sapaesaiphaika taesatai chaekalaisata

PC taesatai chaekalaisata:

Windows 10/11 taesatai
macOS taesatai
Linux taesatai
baibhainana raejaolaiushana taesatai
phaulasakaraina/uinadaoda maoda taesatai
kanataraolaaara saaapaorata taesatai
kaiibaorada/maausa taesatai

maobaaaila taesatai chaekalaisata:

Android taesatai
iOS taesatai
baibhainana sakaraina saaaija taesatai
taaacha kanataraola taesatai
bayaaataaarai bayabahaaara taesatai
maemarai bayabahaaara taesatai

kanasaola taesatai chaekalaisata:

PlayStation taesatai
Xbox taesatai
Nintendo Switch taesatai
kanataraolaaara taesatai
saaarataiphaikaeshana raikauyaaaramaenatasa

26.6 ataomaetaeda taesatai kanasaepata

ataomaetaeda taesatai kaoda:

```
using UnityEngine.TestTools;
using NUnit.Framework;
using System.Collections;
using UnityEngine;

public class AutomatedTests
{
    [Test]
    public void PlayerHealthDecreasesOnDamage()
    {
        PlayerHealth health = new PlayerHealth();
        health.maxHealth = 100;
        health.currentHealth = 100;
        health.TakeDamage(25);
        Assert.AreEqual(75, health.currentHealth);
    }

    [Test]
    public void PlayerDiesAtZeroHealth()
    {
        PlayerHealth health = new PlayerHealth();
        health.maxHealth = 100;
        health.currentHealth = 100;
        health.TakeDamage(100);
        Assert.IsFalse(health.IsAlive());
    }

    [UnityTest]
    public IEnumerator PlayerMovementTest()
    {
        GameObject player = new GameObject();
        PlayerController pc = player.AddComponent<PlayerController>();
        Vector3 startPos = player.transform.position;
        pc.Move(Vector3.forward);
        yield return new WaitForSeconds(0.1f);
        Assert.Greater(
            Vector3.Distance(startPos, player.transform.position), 0f);
        Object.Destroy(player);
    }

    [UnityTest]
    public IEnumerator EnemyAIPatrolTest()
    {
        GameObject enemy = new GameObject();
        EnemyAI ai = enemy.AddComponent<EnemyAI>();
        ai.state = EnemyState.Patrol;
        yield return new WaitForSeconds(1f);
        Assert.AreEqual(EnemyState.Patrol, ai.state);
        Object.Destroy(enemy);
    }

    [Test]
    public void InventoryAddItemTest()
    {
        InventorySystem inventory = new InventorySystem(10);
        Item sword = new Item("Sword", 1);
```

```

        bool added = inventory.AddItem(sword);
        Assert.IsTrue(added);
        Assert.AreEqual(1, inventory.Count);
    }

    [Test]
    public void ScoreIncreasesOnEnemyKill()
    {
        ScoreManager score = new ScoreManager();
        score.AddScore(100);
        Assert.AreEqual(100, score.currentScore);
    }
}

```

26.7 palaetaesatai gaaaida

palaetaesataiyaera dhaapasamaauha:

1. paraikalapanaaa (Planning)

- taesataiyaera udadaeshaya nairadhaaarana karao
- taesataaara nairabaaachana karao
- taesatai sainaaraiau taairai karao

2. parasatautai (Preparation)

- gaema bailada taairai karao
- taesatai pharama taairai karao
- phaidabayaaaka saisataema saetaaapa karao

3. paraichaaalanaaa (Execution)

- taesataaaradaera gaema khaelatae daaaaau
- parayabaekassana karao
- naota naaaaau

4. baishalaessana (Analysis)

- phaidabayaaaka sagaraha karao
- samasayaaa shanaaakata karao
- samaaadhaaana parasataaaba karao

5. raipaoratai (Reporting)

- raipaorata taairai karao
- taimaera saaathae shaeyaaara karao
- phalaoaapa karao

26.8 QA taima sataraaakachaaara

QA TEAM STRUCTURE
QA LEAD

(QA laida)
 SR. QA QA ENGINEER QA ANALYST
 (sainaiyara QA) (QA inyajainaiyaaara) (QA ayaaanaaaisata)
 MANUAL QA AUTOMATION PERFORMANCE
 (mayaaanauyaaala QA) (ataomaeshana) (paaarapharamayaaanasa)

QA taimaera bhauumaikaaa:

26.9 taesatai taulasa

taesatai taulasaera taaalaikaaa:

Unit Testing:

- NUnit
- Unity Test Runner
- Moq

Automated Testing:

- Unity Automated QA
- Selenium (Web)
- Appium (Mobile)

Performance Testing:

- Unity Profiler
- RenderDoc
- Intel GPA

Bug Tracking:

- Jira
- Bugzilla
- Trello
- GitHub Issues

Test Management:

- TestRail
- Zephyr
- qTest

26.10 parayaaakataisa aikasaaarasaaaija

aikasaaarasaaaija 1: baaaga raipaorata taairai karao

aikatai baaaga raipaorata taairai karao yaaatae thaaakabae:

- baaaga aaidai

- shairaonaaama
- saebhaeraitai
- paunaraupaaadanaera dhaaapa
- paratayaaashaita aiba parakarita aacharana

aikasaaarasaaaaja 2: taesata kaesa laekhao

palaeyaaara maubhamaenataera janaya 5tai taesata kaesa laekhao

aikasaaarasaaaaja 3: ataomaetaeda taesata laekhao

aikatai saimapala ataomaetaeda taesata laekhao yaaa:

- palaeyaaara haelatha chaeka karabae
- sakaora chaeka karabae
- inabhaenatarai chaeka karabae

26.11 bhaula yaaa aidaiyae chalatae habae

bhula 1: taesatai shaurau thaekae naaa karaaa

```
// khaaaraaapa - gaema shaessae taesatai karaaa
// bhaaalao - dhaapae dhaapae taesatai karao
```

bhula 2: shaudhau daebhaelapaaaradaera taesatai karaaanao

```
// khaaaraaapa - daebhaelapaaararaaa naijaedaera kaoda naijaeraaa taesata karae
// bhaaalao - aalaaadaaa QA taima thaaakauka
```

bhula 3: baaaga raipaorata naaa karaaa

```
// khaaaraaapa - baaaga daekhaeau kaichhau naaa karaaa
// bhaaalao - saba baaaga raipaorata karao
```

bhula 4: paraaayaoraitai naaa daeayyaaa

```
// khaaaraaapa - saba baaaga aikai rakama bhaaabae taraita karaaa
// bhaaalao - saebhaeraitai anayaaayaii paraaayaoraitai daaaau
```

26.12 taipasa au paraaamarasha

taipasa 1: chhaota chhaota inakaraimaenatae taesata karao

```
// bhaaalao - parataitai phaichaaara taairaira para taesata karao
// baaaga dharatae sahaja haya
```

taipasa 2: ataomaeshana bayabahaaara karao

```
// bhaaalao - raipaitaitaibha taesata ataomaeta karao
// samaya saebha haya
```

taipasa 3: bayaaapaka taesatai karao


```
// bhaaalao - baibhainana paraisathaitaitae taesata karao
// saba baaaga dharaaa padae
```

taipasa 4: dakaumaenataeshana raaakhao

```
// bhaaalao - saba taesata raejaaalata dakaumaenata karao
// bhabaissayatae raephaaraenasa haya
```

taipasa 5: khaelaoyaaadadaera saaathae yaogaaayaoga raaakhao

```
// bhaaalao - khaelaoyaaadadaera phaidabayaaaka naaaau
// baaasataba samasayaaa dharaaa padae
```

26.13 saarasakassaepa

aai adhayaaayae aamaraaa QA aiba taesataiyaera baibhainana daika naiyae aalaochanaaa karaechhai mauula payaenatagaulao halao:

1. ****taesataiyaera dharana****: phaaashanaaala, gaemapalae, paaarapharamayaaanasa, kamapayaaataibailaitai, UI, saebha/laoda, sataraesaa, baaaga
2. ****baaaga raipaorata****: paraphaeshanaaala taemapalaeta aiba kaoda
3. ****taesatai auyaaarakaphalao****: paraikalapanaaa thaekae samaaapatai parayanata
4. ****saebhaeritai laebhaela****: Critical, High, Medium, Low
5. ****palayaaatapharama-sapaesaiphaika taesatai****: PC, maobaaaaila, kanasaola
6. ****ataomaetaeda taesatai****: ataomaeshana kaaushala aiba kaoda
7. ****palaetaesatai****: khaelaoyaaadadaera saaathae taesatai
8. ****QA taima sataraaakachaaara****: bhauumaikaaa aiba daaayaitaba
9. ****taesatai taulasa****: baibhainana taulasaera taaalaikaaa
10. ****parayaaakataisa****: baaasataba paraisathaitaitae taesatai

QA aiba taesatai halao gaema daebhaelapamaenataera aikatai aparaihaaaraya asha bhaaalao QA taima gaemaera maaana naishachaita karae

****gaema daebhaelapamaenata maaasataaarabauka samapanana! shaubhaechachhaaa!****

PART 27

adhayaaaya 27: gaema ikaonamai (GAME ECONOMY)

adhayaaaya 27: gaema ikaonamai (GAME ECONOMY)

shaekhhaara udadaeshaya (Learning Goal)

aii adhayaaayae aapanai shaikhabaena kaiibhaaabaek aikatai gaemaera arathanaiitai taairai karatae haya, yaaatae khaelaoyaaadaraaa daiiraghasamaya gaemae aakarissata thaaakae aiba gaemaera madhayae sausatha arathanaaitaika bhaaarasaaamaya bajaaaya thaaakae aapanai shaikhabaena Currency System, Reward System, Loot Box, aiba Economy Balancing aira baaijanyaaanaika padadhatai

baishada bayaaakhayaaa (Detailed Explanation)

27.1 gaema ikaonamai kaii? (What is Game Economy?)

gaema ikaonamai halao gaemaera madhayae samapadaera (Resources) upaaadana, bayabahaaara aiba bainaimayaera aikatai samanabaita bayabasathaaa aitai nairadhaaarana karae khaelaoyaaadaraaa kaiibhaaabaek gaemaera madhayae maudaraaa (Currency) aaya karabae, kharacha karabae aiba kaiibhaaabaek taaadaera agaragatai (Progression) ghatabae

****gaema ikaonamaira mauula upaaadaaanamasamauha:****

GAME ECONOMY ECOSYSTEM		
SOURCE	> FLOW	> SINK
(usa)	(parabaaaha)	(garata)
* Quests	* Trading	* Upgrades
* Enemies	* Crafting	* Purchases
* Daily Bonus	* Gifting	* Repairs
* Achievements	* Market	* Taxes

****usa (Source) kaaaraaa?*** -- yaekhhaana thaekae samapada aasae

****garata (Sink) kaaaraaa?*** -- yaekhhaanae samapada nassata baaa bayabaharita haya

****parabaaaha (Flow) kaii?*** -- samapada kaiibhaaabaek aika jaaayagaaa thaekae anaya jaaayagaaaya yaaaya

27.2 maudaraaa bayabasathaaa (Currency Systems)

parataitai gaemae aika baaa aikaaadhaika dharanaera maudaraaa thaaakatae paaarae sathaika maudaraaa bayabasathaaa khaelaoyaaadadaera gaemae aakarissata raaakhae

ka) aikaka maudaraaa bayabasathaaa (Single Currency System)

```
# aikaka maudaraaa bayabasathaaaara udaaaharana
class SingleCurrencySystem:
    def __init__(self):
        self.gold = 0 # shaudhaumaaatara aikatai maudaraaa

    def earn_gold(self, amount, source):
        """saorasa thaekae gaolada aaya"""
        self.gold += amount
        print(f"+{amount} Gold from {source}")
        print(f"Total Gold: {self.gold}")

    def spend_gold(self, amount, purpose):
        """gaolada kharacha"""
        if self.gold >= amount:
            self.gold -= amount
            print(f"-{amount} Gold for {purpose}")
            return True
        else:
            print("Not enough Gold!")
            return False

# bayabahaaara
game = SingleCurrencySystem()
game.earn_gold(100, "Quest Complete")
game.earn_gold(50, "Enemy Kill")
game.spend_gold(75, "Weapon Upgrade")
# Output: +100 Gold from Quest Complete
# Output: +50 Gold from Enemy Kill
# Output: -75 Gold for Weapon Upgrade
# Output: Total Gold: 75
```

****aikaka maudaraaaara saubaidhaaa:****

- sahaja baojhaaa yaaaya
- khaelaoyaaadadaera janaya paraissakaaara
- daebhaelapaaaradaera janaya sahaja bayaaalaaanasai

****aikaka maudaraaaara asaubaidhaaa:****

- saiiimaita monetization sauyaoga
- darauta inflation hatae paaarae

kha) bahau maudaraaa bayabasathaaa (Multi-Currency System)

```
# bahau maudaraaa bayabasathaaaara udaaaharana
class MultiCurrencySystem:
    def __init__(self):
        self.currencies = {
            "gold": {"free": 0, "paid": 0}, # pharai maudaraaa
            "gems": {"free": 0, "paid": 0}, # paeida maudaraaa
            "tokens": {"free": 0, "paid": 0}, # ibhaenata maudaraaa
            "prestige": {"free": 0, "paid": 0} # paraesataija maudaraaa
        }

    def earn(self, currency, amount, is_paid=False):
```

```

        """maudaraaa aaya"""
        category = "paid" if is_paid else "free"
        self.currencies[currency][category] += amount
        self.currencies[currency]["free"] += amount
        print(f"+{amount} {currency.upper()} (Total: {self.get_total(currency)})")

    def spend(self, currency, amount):
        """maudaraaa kharacha"""
        total = self.get_total(currency)
        if total >= amount:
            # parathamae pharai mudaraaa kharacha habae
            if self.currencies[currency]["free"] >= amount:
                self.currencies[currency]["free"] -= amount
            else:
                remaining = amount - self.currencies[currency]["free"]
                self.currencies[currency]["free"] = 0
                self.currencies[currency]["paid"] -= remaining
            return True
        return False

    def get_total(self, currency):
        """maota mudaraaa"""
        return (self.currencies[currency]["free"] +
                self.currencies[currency]["paid"])

# bayabahaaara
economy = MultiCurrencySystem()
economy.earn("gold", 500)          # pharai gaolada
economy.earn("gems", 10, is_paid=True) # paeida jaemasa
economy.spend("gold", 200)

**bahau mudaraaaara dharana:**

```

ga) mudaraaaara mauulaya samaparaka (Currency Value Relationship)

```

# mudaraaaara mauulaya samaparaka taairai karaaa
class CurrencyExchange:
    def __init__(self):
        # mauulaya samaparaka saaranaii
        self.rates = {
            "gold_to_gems": 0.01,      # 100 Gold = 1 Gem
            "gems_to_tokens": 10,      # 1 Gem = 10 Tokens
            "gold_to_tokens": 1000     # 1000 Gold = 10 Tokens
        }

    def convert(self, from_currency, to_currency, amount):
        """maudaraaa rauupaaanatarata"""
        rate_key = f"{from_currency}_to_{to_currency}"
        reverse_key = f"{to_currency}_to_{from_currency}"

        if rate_key in self.rates:
            return amount * self.rates[rate_key]
        elif reverse_key in self.rates:
            return amount / self.rates[reverse_key]
        else:
            return None

# mauulaya pairaamaaida
"""
        GEMS      <- sabachaeyae mauulayabaaana (paeida)
        (1$)
        GOLD      <- maaajhaaarai mauulaya

```

```

        (100)
    WOOD/      <- sabachaeyae kama mauulaya
    IRON
        (1000)
"""

```

27.3 paurasakaaara bayabasathaaa (Reward Systems)

khaelaoyaaadadaera gaemae aakarissata raaakhatae baibhainana dharanaera paurasakaaara bayabasathaaa parayaojana

ka) daainaika paurasakaaara (Daily Rewards)

```

# daainaika paurasakaaara bayabasathaaa
class DailyRewardSystem:
    def __init__(self):
        self.login_streak = 0
        self.last_login = None
        self.rewards = {
            1: {"gold": 100, "item": None},
            2: {"gold": 150, "item": None},
            3: {"gold": 200, "item": "Health Potion"},
            4: {"gold": 250, "item": None},
            5: {"gold": 300, "item": None},
            6: {"gold": 400, "item": "Rare Weapon"},
            7: {"gold": 500, "item": "Epic Chest"},
            # sapataaaha pauraotaaa jamaaa halae baishaessa paurasakaaara
        }

    def claim_daily_reward(self):
        """daainaika paurasakaaara daaabai"""
        today = datetime.now().date()

        if self.last_login and (today - self.last_login).days == 1:
            self.login_streak += 1
        elif self.last_login and (today - self.last_login).days > 1:
            self.login_streak = 1 # sataaraika bhaengae yaaaya
        else:
            self.login_streak = 1

        # 7 daina para sataaraika raisaeta
        if self.login_streak > 7:
            self.login_streak = 1

        reward = self.rewards.get(self.login_streak, self.rewards[7])
        self.last_login = today

        print(f"Day {self.login_streak} Login Streak!")
        print(f"Reward: {reward['gold']} Gold")
        if reward['item']:
            print(f"Bonus: {reward['item']}")

        return reward

# bayabahaaara
daily = DailyRewardSystem()
for day in range(8):
    daily.claim_daily_reward()

```

****daainaika paurasakaaaraera sauuchai:****

DAILY LOGIN REWARDS

Day 1	Day 2	Day 3	Day 4	Day 5	Day 6-7
100 Gold	150 Gold	200 Gold	250 Gold	300 Gold	500 Gold
		+Potion			+Epic Item

kha) arajana paurasakaaara (Achievement Rewards)

```
# arajana paurasakaaara bayabasathaaa
class AchievementSystem:
    def __init__(self):
        self.achievements = {
            "first_blood": {
                "name": "First Blood",
                "description": "parathama enemy kae hatayaaa karauna",
                "reward": {"xp": 100, "gold": 50},
                "unlocked": False
            },
            "collector": {
                "name": "Collector",
                "description": "100tai item sagaraha karauna",
                "reward": {"xp": 500, "gold": 200, "item": "Crown"},
                "unlocked": False
            },
            "speedrunner": {
                "name": "Speed Runner",
                "description": "Level 10 10 mainaitaera madhayae shaessa karauna",
                "reward": {"xp": 1000, "gems": 10},
                "unlocked": False
            }
        }

    def check_achievement(self, achievement_id, condition_met):
        """arajana paraiikassaaa"""
        if condition_met and not self.achievements[achievement_id]["unlocked"]:
            self.achievements[achievement_id]["unlocked"] = True
            reward = self.achievements[achievement_id]["reward"]
            print(f" Achievement Unlocked: {self.achievements[achievement_id]['name']}")
            print(f"   Reward: {reward}")
            return reward
        return None
```

****arajana paurasakaaaraera dharana:****

ga) Level paurasakaaara (Level Rewards)

```
# Level paurasakaaara bayabasathaaa
class LevelRewardSystem:
    def __init__(self):
        self.level_rewards = {
            5: {"gold": 500, "item": "Iron Sword"},
            10: {"gold": 1000, "item": "Steel Armor", "unlock": "Arena"},
            15: {"gold": 1500, "gems": 20, "unlock": "Guild"},
            20: {"gold": 2000, "item": "Epic Mount"},
            25: {"gold": 3000, "gems": 50, "unlock": "Raid"},
            50: {"gold": 10000, "item": "Legendary Weapon", "title": "Veteran"}
        }

    def level_up(self, new_level):
        """Level Up ai paurasakaaara"""
        if new_level in self.level_rewards:
```

```

        reward = self.level_rewards[new_level]
        print(f" Level Up! You are now Level {new_level}")
        print(f"    Rewards: {reward}")

        # aanalaka karaaa phaichaaara
        if "unlock" in reward:
            print(f"    New Feature Unlocked: {reward['unlock']}")
        if "title" in reward:
            print(f"    New Title: {reward['title']}")

        return reward
    return None

```

****Level Progression Curve:****

XP Required per Level (Exponential Curve)

```

Level 1: 100 XP
Level 5: 500 XP
Level 10: 2,000 XP
Level 15: 5,000 XP
Level 20: 10,000 XP
Level 25: 20,000 XP
Level 50: 100,000 XP

```

Formula: $XP\ Required = Base \times Level^{Multiplier}$

Example: $100 \times Level^{1.5}$

27.4 kharacha bayaaalaaanasai (Cost Balancing)

ka) kharachaera sauutara (Cost Formula)

```

# kharachaera ganaitaika sauutara
class CostCalculator:
    def __init__(self, base_cost=100, multiplier=1.15, max_level=100):
        self.base_cost = base_cost
        self.multiplier = multiplier
        self.max_level = max_level

    def get_upgrade_cost(self, current_level, upgrade_count=1):
        """aapagaraeda kharacha gananaaa"""
        total_cost = 0
        for i in range(upgrade_count):
            level = current_level + i
            cost = self.base_cost * (self.multiplier ** level)
            total_cost += cost
        return int(total_cost)

    def get_total_investment(self, target_level):
        """aikatai aaitaemae maota bainaiyaoga"""
        total = 0
        for level in range(1, target_level):
            total += self.base_cost * (self.multiplier ** level)
        return int(total)

# bayabahaaara
calc = CostCalculator(base_cost=100, multiplier=1.15)

# Level 1 thaekae Level 10 aira kharacha
print(f"Level 1->2: {calc.get_upgrade_cost(1)} Gold")
print(f"Level 5->6: {calc.get_upgrade_cost(5)} Gold")

```

```
print(f"Level 9->10: {calc.get_upgrade_cost(9)} Gold")
print(f"Total to Level 10: {calc.get_total_investment(10)} Gold")
```

****kharachaera baridadhaira sauuchai (Exponential Growth):****

UPGRADE COST CURVE		
Level	Cost (Gold)	Visual
1 -> 2	100	
2 -> 3	115	
3 -> 4	132	
4 -> 5	152	
5 -> 6	175	
6 -> 7	201	
7 -> 8	231	
8 -> 9	266	
9 -> 10	306	
10 -> 15	600+	
15 -> 20	1200+	

kha) mauulaya nairadhaaarana (Pricing Strategy)

```
# aaitaema mauulaya nairadhaaarana
class ItemPricing:
    def __init__(self):
        self.rarity_multiplier = {
            "common": 1.0,
            "uncommon": 2.5,
            "rare": 6.0,
            "epic": 15.0,
            "legendary": 50.0
        }

        self.base_prices = {
            "weapon": 100,
            "armor": 80,
            "potion": 10,
            "mount": 500,
            "cosmetic": 200
        }

    def calculate_price(self, item_type, rarity, player_level):
        """aaitaemaera mauulaya gananaaa"""
        base = self.base_prices.get(item_type, 100)
        rarity_mult = self.rarity_multiplier.get(rarity, 1.0)
        level_mult = 1 + (player_level * 0.05) # 5% per level

        price = base * rarity_mult * level_mult
        return int(price)

# mauulaya saaraanaii
pricing = ItemPricing()
for rarity in ["common", "uncommon", "rare", "epic", "legendary"]:
    price = pricing.calculate_price("weapon", rarity, 10)
    print(f"{rarity.capitalize()} Weapon (Level 10): {price} Gold")

**raeyaaaraitai bhaitataika mauulaya:**
```

27.5 agaragatai samatala (Progression Curves)

ka) raaikhaika agaragatai (Linear Progression)

```
# raaikhaika agaragatai
def linear_progression(level):
    """parataitai level ai samaana XP parayaojana"""
    xp_required = 1000 # parataitai level ai 1000 XP
    total_xp = level * xp_required
    return total_xp

# saubaidhaaa: paraikalapanaayaogaya, paraimaapayaogaya
# asaubaidhaaa: darauta nairasatara hayae yaaaya
```

kha) ghaataka agaragatai (Exponential Progression)

```
# ghaataka agaragatai
import math

def exponential_progression(level, base=100, exponent=1.5):
    """parataitai level ai baeshai XP parayaojana"""
    xp_required = base * math.pow(level, exponent)
    return int(xp_required)

# saubaidhaaa: daiiraghasamaya khaelaara paraeranaa daeya
# asaubaidhaaa: shaessa level gaulao atayanata kathaina

# udaaharana:
for level in [1, 5, 10, 20, 50]:
    xp = exponential_progression(level)
    print(f"Level {level}: {xp} XP required")
```

ga) lagaaaraidamaika agaragatai (Logarithmic Progression)

```
# lagaaaraidamaika agaragatai
import math

def logarithmic_progression(level, base=100, growth_rate=200):
    """shauratae darauta, parae dhairae dhairae"""
    xp_required = base + growth_rate * math.log(level + 1)
    return int(xp_required)

# saubaidhaaa: shauratae sahaja, parae chayaalaenyajai
# asaubaidhaaa: daiiraghasamaya khaelaara kama paraeranaa
```

****agaragatai taulanaa:****

Progression Curves Comparison
XP Required

Level	Linear	Exponential	Logarithmic
1	1,000	100	139
5	5,000	1,118	422
10	10,000	3,162	560
20	20,000	8,944	699
50	50,000	35,355	890
100	100,000	100,000	1,021

27.6 maudaraasaphaitai paratairaodha (Inflation Prevention)

ka) maudaraasaphaitai kaii? (What is Inflation?)

```
# maudaraasaphaiitai samasayaaara udaaaharana
class InflationProblem:
    def __init__(self):
        self.gold_supply = 1000000 # maota gaolada saaapalaaai
        self.gold_earned_per_hour = 500 # paratai ghanataaaya aaya
        self.gold_spent_per_hour = 300 # paratai ghanataaaya kharacha
        self.inflation_rate = self.gold_earned_per_hour - self.gold_spent_per_hour
        # paratai ghanataaaya 200 Gold baeshai hachachhae!

    def calculate_inflation(self, hours):
        """maudaraasaphaiitai gananaaa"""
        excess_gold = self.inflation_rate * hours
        new_price_multiplier = 1 + (excess_gold / self.gold_supply)
        return new_price_multiplier

# samasayaaa: gaolada baaadachhae, kainatauu mauulaya aparaibarataita
# phalaaaphala: khaelaoyaaadaraaa atairaikata dhanaii hayae yaaaya
```

kha) maudaraasaphaiitai paratairaodhaera upaaaya

```
# maudaraasaphaiitai paratairaodha bayabasathaaa
class AntiInflationSystem:
    def __init__(self):
        self.gold_supply = 1000000
        self.sink_mechanisms = []

    def add_gold_sink(self, name, cost, frequency):
        """gaolada saingaka yaoga karauna"""
        self.sink_mechanisms.append({
            "name": name,
            "cost": cost,
            "frequency": frequency
        })

    def calculate_sink_effectiveness(self):
        """saingakaera kaaarayakaaaraitaaa gananaaa"""
        total_gold_removed = 0
        for sink in self.sink_mechanisms:
            daily_cost = sink["cost"] * sink["frequency"]
            total_gold_removed += daily_cost
        return total_gold_removed

# samaaadhaana 1: Gold Sinks
anti_inflation = AntiInflationSystem()
anti_inflation.add_gold_sink("Item Repair", 50, 10) # parataidaina 500 gaolada
anti_inflation.add_gold_sink("Fast Travel", 100, 5) # parataidaina 500 gaolada
anti_inflation.add_gold_sink("Tax at Market", 0.1, 100) # 10% tayaaakasa
anti_inflation.add_gold_sink("Gem Crafting", 1000, 2) # parataidaina 2000 gaolada
```

****maudaraasaphaiitai paratairaodhaera kaaushala:****

ANTI-INFLATION STRATEGIES

1. GOLD SINKS (gaolada shaossana)
 - Item Repair (aaitaema maeraaamata)
 - Market Tax (baajaara tayaaakasa)
 - Fast Travel (darauta bharamana)
 - Consumables (bayabahaaarayagaya aaitaema)
2. LIMITED SUPPLY (saiimaita saaapalaaai)
 - Daily Cap (daainaika saiimaaa)
 - Cooldown Timer (kauladaauna taaaimaara)
 - Energy System (shakatai bayabasathaaa)
3. DEPRECIATION (mauulayaharaasa)

- Item Degradation (aaitaema kassaya)
- Time-Limited Items (saiimaita samayaera aaitaema)
- Seasonal Reset (maausaumi raisaeta)
- 4. BALANCED ECONOMY (bhaaarasaaamayapauurana arathanaiitai)
 - Source = Sink (usa = garata)
 - Regular Monitoring (naiyamaita parayabaekassana)
 - Dynamic Adjustment (gataishaiila samanabaya)

27.7 aapagaraeda bayabasathaaa (Upgrade Systems)

ka) sarala aapagaraeda (Simple Upgrade)

```
# sarala aapagaraeda bayabasathaaa
class SimpleUpgrade:
    def __init__(self, name, base_stat, base_cost):
        self.name = name
        self.level = 1
        self.max_level = 50
        self.base_stat = base_stat
        self.base_cost = base_cost
        self.multiplier = 1.15

    def get_current_stat(self):
        """baratamaana satayaaata"""
        return self.base_stat * (1.2 ** (self.level - 1))

    def get_upgrade_cost(self):
        """aapagaraeda kharacha"""
        return int(self.base_cost * (self.multiplier ** (self.level - 1)))

    def get_next_level_stat(self):
        """parabarataii level aira satayaaata"""
        return self.base_stat * (1.2 ** self.level)

    def upgrade(self):
        """aapagaraeda karauna"""
        if self.level >= self.max_level:
            return False, "Max level reached!"

        cost = self.get_upgrade_cost()
        current_stat = self.get_current_stat()
        next_stat = self.get_next_level_stat()

        self.level += 1
        return True, {
            "cost": cost,
            "stat_before": current_stat,
            "stat_after": next_stat,
            "improvement": next_stat - current_stat
        }

# bayabahaaara
weapon = SimpleUpgrade("Iron Sword", base_stat=10, base_cost=100)

for i in range(5):
    success, info = weapon.upgrade()
    if success:
        print(f"Level {weapon.level}: Stat {info['stat_before']:.1f} -> {info['stat_after']:.1f}")

**aapagaraeda saisataemaera dharana:**
```

kha) shaaakhaayaukata aapagaraeda (Branching Upgrade)

```
# shaaakhaayaukata aapagaraeda bayabasathaaa
class BranchingUpgrade:
    def __init__(self):
        self.skill_tree = {
            "Warrior": {
                "Level 1": {"name": "Power Strike", "stat": "ATK", "bonus": 10},
                "Level 5": {"name": "Critical Hit", "stat": "CRIT", "bonus": 5},
                "Level 10": {"name": "Berserker Rage", "stat": "ATK", "bonus": 25}
            },
            "Mage": {
                "Level 1": {"name": "Fireball", "stat": "MAG", "bonus": 15},
                "Level 5": {"name": "Mana Shield", "stat": "DEF", "bonus": 10},
                "Level 10": {"name": "Meteor", "stat": "MAG", "bonus": 40}
            },
            "Rogue": {
                "Level 1": {"name": "Backstab", "stat": "ATK", "bonus": 8},
                "Level 5": {"name": "Evasion", "stat": "SPD", "bonus": 12},
                "Level 10": {"name": "Assassinate", "stat": "CRIT", "bonus": 20}
            }
        }

    def get_available_skills(self, class_name, level):
        """paraaapatayaogaya sakaila"""
        available = []
        for req_level, skill in self.skill_tree[class_name].items():
            req = int(req_level.split()[1])
            if level >= req:
                available.append(skill)
        return available
```

27.8 lauta bayabasathaaa (Loot Systems)

ka) rayaaanadama darapa (Random Drops)

```
import random

# rayaaanadama darapa bayabasathaaa
class LootDropSystem:
    def __init__(self):
        self.loot_table = {
            "common": {
                "items": ["Wood", "Iron Ore", "Herb"],
                "drop_rate": 0.50, # 50%
                "gold_range": (10, 50)
            },
            "uncommon": {
                "items": ["Steel Ingot", "Magic Crystal"],
                "drop_rate": 0.30, # 30%
                "gold_range": (50, 200)
            },
            "rare": {
                "items": ["Rare Sword", "Enchanted Armor"],
                "drop_rate": 0.15, # 15%
                "gold_range": (200, 1000)
            },
            "epic": {
```

```

        "items": ["Epic Staff", "Dragon Scale"],
        "drop_rate": 0.04,      # 4%
        "gold_range": (1000, 5000)
    },
    "legendary": {
        "items": ["Excalibur", "Godslayer"],
        "drop_rate": 0.01,      # 1%
        "gold_range": (5000, 20000)
    }
}

def roll_loot(self):
    """lauta raola"""
    roll = random.random()
    cumulative = 0

    for rarity, data in self.loot_table.items():
        cumulative += data["drop_rate"]
        if roll <= cumulative:
            item = random.choice(data["items"])
            gold = random.randint(*data["gold_range"])
            return {
                "rarity": rarity,
                "item": item,
                "gold": gold
            }

    # Default to common
    return {
        "rarity": "common",
        "item": random.choice(self.loot_table["common"]["items"]),
        "gold": random.randint(*self.loot_table["common"]["gold_range"])
    }

# bayabahaaara
loot = LootDropSystem()
for i in range(10):
    result = loot.roll_loot()
    print(f"Drop {i+1}: {result['rarity'].upper()} - {result['item']} ({result['gold']} Go

```

****lauta taebaila (Loot Table):****

LOOT DROP RATES		
Rarity	Drop %	Visual Distribution
Common	50%	
Uncommon	30%	
Rare	15%	
Epic	4%	
Legendary	1%	

kha) lauta bakasa (Loot Boxes)

```

# lauta bakasa bayabasathaaa
class LootBoxSystem:
    def __init__(self):
        self.box_types = {
            "bronze": {
                "cost": 100,
                "guaranteed_rarity": "common",
                "possible_rarities": ["common", "uncommon"],
                "pity_counter": 10
            },

```

```

        "silver": {
            "cost": 500,
            "guaranteed_rarity": "uncommon",
            "possible_rarities": ["uncommon", "rare", "epic"],
            "pity_counter": 20
        },
        "gold": {
            "cost": 2000,
            "guaranteed_rarity": "rare",
            "possible_rarities": ["rare", "epic", "legendary"],
            "pity_counter": 30
        }
    }
    self.pity_counters = {}

def open_box(self, box_type):
    """lauta bakasa khaulauna"""
    box = self.box_types[box_type]

    # Pity system
    if box_type not in self.pity_counters:
        self.pity_counters[box_type] = 0

    self.pity_counters[box_type] += 1

    # Pity guarantee
    if self.pity_counters[box_type] >= box["pity_counter"]:
        self.pity_counters[box_type] = 0
        return {"rarity": box["guaranteed_rarity"], "pity": True}

    # Normal roll
    roll = random.random()
    if roll < 0.01: # 1% Legendary
        return {"rarity": "legendary"}
    elif roll < 0.10: # 10% Epic
        return {"rarity": "epic"}
    elif roll < 0.30: # 20% Rare
        return {"rarity": "rare"}
    else:
        return {"rarity": box["guaranteed_rarity"]}

# bayabahaaara
loot_box = LootBoxSystem()
for i in range(5):
    result = loot_box.open_box("silver")
    print(f"Box {i+1}: {result['rarity'].upper()} {'(PITY!)' if result.get('pity') else ''}

```

****Pity System (pитай saisataema):****

```

PITY SYSTEM
Box Opening Count: 1 2 3 4 5 6 7 8 9 10
Guaranteed Drop:   C C C C C C C C C *
(C = Common, * = Guaranteed Rare+)
After 10 failed attempts -> Guaranteed Rare+ drop

```

ga) lauta taebaila daijaaaina (Loot Table Design)

```

# unanata lauta taebaila
class AdvancedLootTable:
    def __init__(self):
        self.items = {
            "common": [
                {"name": "Health Potion", "drop_rate": 0.30},

```

```

        {"name": "Mana Potion", "drop_rate": 0.25},
        {"name": "Iron Ore", "drop_rate": 0.20},
        {"name": "Herb", "drop_rate": 0.15},
        {"name": "Arrow", "drop_rate": 0.10}
    ],
    "uncommon": [
        {"name": "Steel Ingot", "drop_rate": 0.25},
        {"name": "Magic Crystal", "drop_rate": 0.20},
        {"name": "Enchanted Herb", "drop_rate": 0.15},
        {"name": "Rare Gem", "drop_rate": 0.10},
        {"name": "Ancient Relic", "drop_rate": 0.05}
    ],
    "rare": [
        {"name": "Dragon Scale", "drop_rate": 0.15},
        {"name": "Phoenix Feather", "drop_rate": 0.10},
        {"name": "Void Crystal", "drop_rate": 0.05},
        {"name": "Eternal Weapon", "drop_rate": 0.02}
    ]
}

def get_drop(self, rarity):
    """nairadaissata raeyaaaraitai thaekae darapa"""
    items = self.items.get(rarity, [])
    roll = random.random()
    cumulative = 0

    for item in items:
        cumulative += item["drop_rate"]
        if roll <= cumulative:
            return item["name"]

    return items[0]["name"] if items else None

```

27.9 bayaaalaaanasa taiunai (Balance Tuning)

```

# gaema ikaonamai bayaaalaaanasai saisataema
class EconomyBalancer:
    def __init__(self):
        self.metrics = {
            "gold_inflow": 0,
            "gold_outflow": 0,
            "avg_player_gold": 0,
            "avg_items_per_player": 0,
            "inflation_rate": 0
        }

    def calculate_gold_flow_ratio(self):
        """saonaaara parabaaahaera anaupaaata"""
        if self.metrics["gold_outflow"] == 0:
            return float('inf')
        return self.metrics["gold_inflow"] / self.metrics["gold_outflow"]

    def check_balance(self):
        """bayaaalaaanasa paraiikassaaa"""
        ratio = self.calculate_gold_flow_ratio()

        if ratio > 1.2:
            return "INFLATION WARNING: Too much gold entering"
        elif ratio < 0.8:
            return "DEFLATION WARNING: Too much gold leaving"
        else:

```

```

        return "BALANCED: Economy is healthy"

def suggest_adjustments(self):
    """samanabaya paraaamarasha"""
    ratio = self.calculate_gold_flow_ratio()
    suggestions = []

    if ratio > 1.2:
        suggestions.append("Increase gold sinks")
        suggestions.append("Reduce gold rewards")
        suggestions.append("Add more items to buy")
    elif ratio < 0.8:
        suggestions.append("Decrease gold sinks")
        suggestions.append("Increase gold rewards")
        suggestions.append("Reduce item costs")

    return suggestions

**ikaonamai bayaaalaaanasa chaekalaisata:**

ECONOMY BALANCE CHECKLIST
[X] Gold Flow Ratio (saonaaara parabaaaha anaupaaata)
    Target: 1.0 +/- 0.2 (Inflow Outflow)
[X] Time to Max Level (sarabaochacha level parayanata samaya)
    Target: 100-200 hours for casual games
[X] Time to Best Items (saeraaa aaitaema paetae samaya)
    Target: 80% of max level
[X] Free vs Paid Advantage (pharai vs paeida saubaidhaaa)
    Target: Paid gives 20-30% advantage
[X] Early vs Late Game Economy (shaurau vs shaessa gaema)
    Target: Both feel rewarding
[X] New Player Experience (natauna khaelaoyaaadaera abhaijanyataaa)
    Target: Fun from minute 1

```

27.10 namaunaaa gaema ikaonamai saisataema (Sample Game Economy)

```

# samapauurana namaunaaa gaema ikaonamai
class GameEconomy:
    def __init__(self):
        # maudaraaa bayabasathaaa
        self.currencies = {
            "gold": 0,
            "gems": 0,
            "tokens": 0
        }

        # aaitaema inabhaenatarai
        self.inventory = {}

        # palaeyaaaara satayaaata
        self.player_level = 1
        self.player_xp = 0

        # ikaonamai maetaraikasa
        self.total_gold_earned = 0
        self.total_gold_spent = 0
        self.items_crafted = 0

    def earn_gold(self, amount, source):
        """gaolada aaya"""
        self.currencies["gold"] += amount

```



```

        self.total_gold_earned += amount
        print(f" +{amount} Gold from {source}")
        self._check_inflation()

def spend_gold(self, amount, purpose):
    """gaolada kharacha"""
    if self.currencies["gold"] >= amount:
        self.currencies["gold"] -= amount
        self.total_gold_spent += amount
        print(f" -{amount} Gold for {purpose}")
        return True
    print(" Not enough Gold!")
    return False

def earn_xp(self, amount):
    """XP aaya"""
    self.player_xp += amount
    xp_needed = self._xp_for_level(self.player_level + 1)

    while self.player_xp >= xp_needed:
        self.player_xp -= xp_needed
        self.player_level += 1
        print(f" Level Up! Level {self.player_level}")
        self._grant_level_reward()
        xp_needed = self._xp_for_level(self.player_level + 1)

def _xp_for_level(self, level):
    """Level aira janaya parayaojanaiiya XP"""
    return int(100 * (level ** 1.5))

def _grant_level_reward(self):
    """Level aapa paurasakaaara"""
    rewards = {
        5: {"gold": 500, "item": "Iron Sword"},
        10: {"gold": 1000, "gems": 10, "item": "Steel Armor"},
        15: {"gold": 1500, "item": "Rare Mount"},
        20: {"gold": 2000, "gems": 25, "item": "Epic Weapon"}
    }

    if self.player_level in rewards:
        reward = rewards[self.player_level]
        if "gold" in reward:
            self.earn_gold(reward["gold"], "Level Up")
        if "gems" in reward:
            self.currencies["gems"] += reward["gems"]
        if "item" in reward:
            self.inventory[reward["item"]] = 1
            print(f" Received: {reward['item']}")

def craft_item(self, item_name, gold_cost, materials):
    """aaitaema taairai"""
    if self.spend_gold(gold_cost, f"Crafting {item_name}"):
        # mayaataeraiaaala chaeka
        for mat, count in materials.items():
            if self.inventory.get(mat, 0) < count:
                print(f" Need {count} {mat}")
                return False
            self.inventory[mat] -= count

        self.inventory[item_name] = self.inventory.get(item_name, 0) + 1
        self.items_crafted += 1
        print(f" Crafted: {item_name}")
        return True

```

```

        return False

def _check_inflation(self):
    """maudaraaasaphaitai paraikassaaa"""
    if self.total_gold_earned > 0:
        ratio = self.total_gold_spent / self.total_gold_earned
        if ratio < 0.3:
            print(" WARNING: Low gold sink ratio")

def get_economy_report(self):
    """arathanaaitaika raipaorata"""
    print("\n" + "="*50)
    print("          ECONOMY REPORT")
    print("="*50)
    print(f"Player Level: {self.player_level}")
    print(f"Gold: {self.currencies['gold']}")
    print(f"Gems: {self.currencies['gems']}")
    print(f"Tokens: {self.currencies['tokens']}")
    print(f"Total Gold Earned: {self.total_gold_earned}")
    print(f"Total Gold Spent: {self.total_gold_spent}")
    print(f"Items Crafted: {self.items_crafted}")

    if self.total_gold_earned > 0:
        spend_ratio = self.total_gold_spent / self.total_gold_earned * 100
        print(f"Spend Ratio: {spend_ratio:.1f}%")
    print("="*50)

# gaema chaaalaaanao
game = GameEconomy()

# khaelaaa saimaulaeshana
for i in range(20):
    game.earn_xp(150)
    game.earn_gold(random.randint(50, 200), "Quest Complete")

    if game.currencies["gold"] >= 100:
        game.spend_gold(100, "Health Potion")

# raipaorata
game.get_economy_report()

**namaunaaa ikaonamai saaaranaii:**

```

SAMPLE GAME ECONOMY TABLE

SOURCE (usa)	AMOUNT	FREQUENCY	DAILY TOTAL
Quest Rewards	100-500	10/day	2,000
Enemy Kills	10-50	50/day	1,500
Daily Login	100-500	1/day	300
Achievements	50-1000	2/day	500
TOTAL INFLOW			4,300/day
SINK (garata)	AMOUNT	FREQUENCY	DAILY TOTAL
Item Upgrade	100-1000	5/day	2,000
Item Repair	50-200	10/day	1,000
Market Tax	10%	All trades	500
Fast Travel	100	3/day	300
Consumables	50-100	5/day	400
TOTAL OUTFLOW			4,200/day
GOLD FLOW RATIO: $4,300 / 4,200 = 1.02$ [x] BALANCED			

27.11 ganaitaika sauutarasamauuha (Mathematical Formulas)

ka) XP Progression Formula

XP Required = Base x Level^{Exponent}

udaaaharana:

Base = 100, Exponent = 1.5

Level 1: $100 \times 1^{1.5} = 100$ XP

Level 5: $100 \times 5^{1.5} = 1,118$ XP

Level 10: $100 \times 10^{1.5} = 3,162$ XP

Level 20: $100 \times 20^{1.5} = 8,944$ XP

Level 50: $100 \times 50^{1.5} = 35,355$ XP

kha) Upgrade Cost Formula

Cost = Base x Multiplier^{Level}

udaaaharana:

Base = 100, Multiplier = 1.15

Level 1: $100 \times 1.15^1 = 115$ Gold

Level 5: $100 \times 1.15^5 = 201$ Gold

Level 10: $100 \times 1.15^{10} = 405$ Gold

Level 20: $100 \times 1.15^{20} = 1,637$ Gold

ga) Loot Drop Probability

$P(\text{Rarity}) = \text{Drop Rate} \times (1 + \text{Pity Bonus})$

Pity System:

After N attempts without rare drop:

Pity Bonus = N / Pity Threshold

udaaaharana:

Drop Rate = 0.01 (1%)

Pity Threshold = 100

After 50 attempts: Pity Bonus = 0.5

$P(\text{Rare}) = 0.01 \times (1 + 0.5) = 0.015$ (1.5%)

ga) Inflation Rate

Inflation Rate = (Gold Inflow - Gold Outflow) / Total Gold Supply

Target: |Inflation Rate| < 0.05 (5%)

If Inflation Rate > 0.05:

-> Economy is inflating (too much gold)

-> Action: Increase sinks or decrease sources

If Inflation Rate < -0.05:

-> Economy is deflating (too little gold)

-> Action: Decrease sinks or increase sources

janaparaiya gaema thaekae udaaaharana (Examples from Popular Games)

1. kalayaaasha apha kalayaaanasa (Clash of Clans)

CLASH OF CLANS ECONOMY

CURRENCIES:

Gold (Free): Buildings, Walls
Elixir (Free): Troops, Spells
Dark Elixir (Rare): Dark Troops
Gems (Paid): Speed-ups, Exclusive

SOURCES:

Raiding (Gold + Elixir)
Collectors (Passive Income)
Clan Wars (Bonus Rewards)
Daily Bonuses

SINKS:

Building Upgrades (Main Sink)
Troop Training
Wall Upgrades (Major Gold Sink)
Laboratory Research

KEY INSIGHT: Different currencies for different purposes

Gold = Defense, Elixir = Offense

2. gaenashaina imapayaaakata (Genshin Impact)

GENSHIN IMPACT ECONOMY

CURRENCIES:

Mora (Gold): Universal currency
Primogems (Premium): Wish system
Genesis Crystals (Paid): Convert to Primogems
Master XI: Talent level-up
Hero's Wit: Character level-up

WISH SYSTEM (Loot Box):

Base Rate: 0.6% for 5-star
Pity: Guaranteed 5-star at 90 wishes
pity: 50% chance at 75 wishes
Hard pity: 100% at 90 wishes

KEY INSIGHT: Complex multi-currency with gacha mechanics

3. phaoratanaaaaita (Fortnite)

FORTNITE ECONOMY

CURRENCIES:

V-Bucks (Paid Only): Everything cosmetic

MONETIZATION:

Battle Pass: \$9.50 for season rewards
Item Shop: Daily rotating cosmetics
Save the World: PvE mode (separate purchase)

ECONOMY DESIGN:

No gameplay advantage (100% cosmetic)
Battle Pass encourages daily play
FOMO (Fear of Missing Out) on items
Free V-Bucks in Battle Pass (earn next pass)

KEY INSIGHT: Cosmetic-only monetization builds trust

4. maaainakaraaaphata (Minecraft)

MINECRAFT ECONOMY

NO FORMAL CURRENCY:

Barter System (Trading with Villagers)

ECONOMY TYPES:

Survival: Resource gathering & crafting
Creative: Unlimited resources

SMP: Player-driven economy
 DESIGN PHILOSOPHY:
 Player-created value
 No official monetization
 Community-driven markets
 Simple but deep
 KEY INSIGHT: Player agency creates emergent economy

daaayaaagaraama (Diagrams)

gaema ikaonamaira chakara (Game Economy Cycle)

GAME ECONOMY CYCLE
 PLAYER

EARN		SPEND
Quest	Items	Upgrades
Enemy	Market	Crafting
Event	Shop	Trading

FEEDBACK LOOP:
 Earn -> Spend -> Progress -> More Challenges -> Earn More

maudaraaasaphaiitai chakara (Inflation Cycle)

INFLATION CYCLE

Too Much Gold <- Sources > Sinks
 Available

Prices Rise <- Players willing to pay more
 New Players <- Can't afford anything
 Struggle

Player Churn <- Players leave
 Increases

SOLUTION: Add Gold Sinks to Balance the Cycle

anaushaiilana (Exercises)

anaushaiilana 1: maudaraaa bayabasathaaa daijaaaina

aikatai RPG gaemaera janaya aikatai maudaraaa bayabasathaaa daijaaaina karauna:

- kamapakassae 3tai dharanaera maudaraaa thaaakatae habae
- parataitai maudaraaara janaya usa aiba bayabahaaara nairadhaaarana karauna
- maudaraaasaphaiitai paratairaodhaera bayabasathaaa yaoga karauna
- aikatai maudaraaa saaranaii taairai karauna

anaushaiilana 2: Loot Table daijaaaina

aikatai gaemaera janaya loot table daijaaaina karauna:

- kamapakassae 5tai raeyaaaraitai laebhaela
- parataitai raeyaaaraitaira janaya 3tai aaitaema

- darapa raeta nairadhaaarana karauna
- Pity system yaoga karauna

anaushailana 3: Progression Curve gananaaa

aikatai 50 level aira RPG gaemaera janaya:

- parataitai level aira janaya XP gananaaa karauna
- maota XP gananaaa karauna
- aanaumaaanaika khaelaaara samaya nairadhaaarana karauna
- aikatai garaaapha taairai karauna

anaushailana 4: Gold Flow Analysis

aikatai gaemaera janaya gold flow analysis karauna:

- daainaika gold inflow gananaaa karauna
- daainaika gold outflow gananaaa karauna
- Inflation rate gananaaa karauna
- samasayaaa thaaakalae samaaadhaaana parasataaaba karauna

saaadhaaarana bhaula (Common Mistakes)

1. atairaikata maudaraaa sasathaaana (Too Much Currency)

```
# bhaula: 10taira baeshai maudaraaa
bad_currencies = ["gold", "silver", "copper", "gems", "crystals",
                  "tokens", "points", "stars", "hearts", "diamonds"]

# sathaika: 2-4tai maudaraaa
good_currencies = ["gold", "gems", "tokens"]
```

2. Pity System naei (No Pity System)

```
# bhaula: 0.01% darapa raeta, kaonao guarantee naei
# khaelaoyaaadaraaa hataaasha hayae yaaaya

# sathaika: Pity system yaoga karauna
def pity_system(attempts, base_rate=0.01, threshold=100):
    """100 chaessataaara para guarantee"""
    if attempts >= threshold:
        return 1.0 # 100% guarantee
    return base_rate * (1 + attempts / threshold)
```

3. Inflation upaekassaaa karaaa (Ignoring Inflation)

```
# bhaula: Gold sink naei
# khaelaoyaaadaraaa atairaikata dhanaii hayae yaaaya
# aaitaemaera mauulaya arathahaiina hayae yaaaya

# sathaika: Gold sink yaoga karauna
sinks = ["item_repair", "fast_travel", "market_tax", "crafting"]
```

4. Pay-to-Win (P2W)

```
# bhaula: paeida aaitaema gaemapalae saubaidhaaa daeya
# pharai khaelaoyaaadaraaa asaubaidhaaaya padae

# sathaika: shaudhau kasamaetaika monetization
# paeida aaitaema shaudhau daekhatae bhaaalao, gameplay advantage naya
```

5. jataila bayabasathaaa (Complex System)

```
# bhaula: 10tai bhainana maudaraaaa, 50tai aaitaema kayaaataaagarai
# khaelaoyaaadaraaa baujhatae paaarae naaa

# sathaika: sarala aiba paraissakaaara
# khaelaoyaaadaraaa sahajae baujhatae paaarae
```

taipasa (Tips)

1. shaurau karauna saralataaa thaekae (Start Simple)

```
Phase 1: Basic Currency (Gold only)
Phase 2: Add Premium Currency (Gold + Gems)
Phase 3: Add Event Currency (Gold + Gems + Tokens)
Phase 4: Fine-tune based on data
```

2. Data-Driven Decisions

```
# daetaaa sagaraha karauna
analytics = {
    "avg_gold_per_session": 0,
    "avg_gold_spent_per_session": 0,
    "top_gold_sinks": [],
    "top_gold_sources": [],
    "player_retention": 0,
    "monetization_rate": 0
}

# daetaaa thaekae saidadhaaanata naina
# anaumaaana naya, paramaaana thaekae
```

3. Player Psychology baujhauna

- Loss Aversion: haaaraanaora chaeyae paaaauyaaa baeshai gaurautabapauurana
- Sunk Cost: yaaa kharacha karaechhae taaa chhaedae daitae chaaaya naaa
- FOMO: saiimaita samayaera ibhaenata aagaraha baaadaaaya
- Progress Bar: 80% samapanana halae baaakai 20% shaessa karatae chaaaya

4. naiyamaita aapadaeta karauna (Regular Updates)

```
Weekly: Fine-tune drop rates
Monthly: Add new items, adjust prices
Seasonally: New currency sinks, events
Yearly: Major economy overhaul if needed
```

5. Playtest karauna (Playtest)

- anatata 100 jana khaelaoyaaada daiyae playtest karauna
- taaadaera mataaamata naina
- taaadaera kharacha karaaara dharana parayabaekassana karauna
- daetaaa thaekae shaikhauna

saaarasakassaepa (Summary)

mauula payaenata:

gaema ikaonamai halao samapadaera upaaadana, bayabahaaara aiba bainaimayaera bayabasathaa

maudaraaa bayabasathaaa: 2-4tai mudaraaa raaakhauna (pharai + paeida)

paurasakaaara: daainaika, arajana, Levelbhaitataika paurasakaaara daina

kharacha: Exponential curve bayabahaaara karauna

agaragatai: Linear, Exponential, baaa Logarithmic baechhae naina

maudaraaasaphaiitai: Gold sink yaoga karauna

lauta: Pity system yaoga karauna

bayaaalaaanasa: Data-driven decisions naina

samapauurana ikaonamai saisataemaera kaaathaaamao:

- COMPLETE ECONOMY SYSTEM
1. CURRENCY SYSTEM

Primary: Gold (Free)

Premium: Gems (Paid)

Event: Tokens (Limited)
2. REWARD SYSTEM

Daily Login Rewards

Achievement Rewards

Level Up Rewards

Quest Rewards
3. COST SYSTEM

Upgrade Costs (Exponential)

Item Prices (Rarity-based)

Service Costs (Repairs, Taxes)
4. PROGRESSION SYSTEM

XP Curve

Level Milestones

Unlock Schedule
5. LOOT SYSTEM

Drop Rates

Loot Tables

Pity System
6. ANTI-INFLATION

Gold Sinks

Supply Limits

Regular Monitoring

parabarataii adhayaaaya: [Chapter 28: Monetization](28_monetization.md)

GAME DEVELOPMENT MASTERBOOK - Chapter 27

PART 28

adhayaaaya 28: manaitaaaijaeshana (MONETIZATION)

adhayaaaya 28: manaitaaaijaeshana (MONETIZATION)

shaekhaara udadaeshaya (Learning Goal)

aii adhayaayae aapanai shaikhabaena gaema thaekae aayaera baibhainana padadhatai, yaaara madhayae Premium, Free-to-Play, DLC, Battle Pass, Ads, aiba Subscription anatarabhaukata aapanai shaikhabaena kaona dharanaera gaemaera janaya kaona monetization madaela sabachaeyae upayaukata

baishada bayaaakhayaaa (Detailed Explanation)

28.1 manaitaaaijaeshana kaii? (What is Monetization?)

manaitaaaijaeshana halao gaema thaekae aayaera kaaushala aitai nairadhaaarana karae kaiibhaaaba khaelaoyaaadaraaa gaemae aratha bayaya karabae aiba daebhaelapaaararaaa kaiibhaaaba aaya karabae

MONETIZATION MODELS OVERVIEW		
PREMIUM (Buy)	FREE TO PLAY	SUBSCRIPTION (Monthly)
One-time Payment	Multiple Payments	Recurring Payment
DLC (Content)	BATTLE PASS	ADS (Free)
Extra Content Purchase	Seasonal Rewards	Watch Ads for Rewards

28.2 paraimaiyaaama madaela (Premium Model)

paraimaiyaaama madaelae khaelaoyaaadaraaa gaema kaenaaara para samapauurana gaema paaaya kaonao atairaikata kharacha naei

```
class PremiumGame:
    def __init__(self, name, price):
        self.name = name
        self.price = price
        self.purchases = 0
```

```

        self.revenue = 0

    def sell(self):
        self.purchases += 1
        self.revenue += self.price
        print(f"Sold {self.name} for ${self.price}")

game = PremiumGame("The Witcher 3", 39.99)
game.sell()

```

****paraimaiyaaama gaemaera udaaaharana:****

****saubaidhaaa:**** sahaja baojhaaa yaaaya, aikabaaara kainalaei samapauurana gaema, No Pay-to-Win
****asaubaidhaaa:**** aikabaaaraei saba aaya, daiiraghasamaya khaelaoyaaada aakarissata thaaakae naaa

28.3 pharai-tau-palae madaela (Free-to-Play Model)

khaelaoyaaadaraaa gaema bainaaamauulayae khaelatae paaarae, kainatau atairaikata saaamagaraiira janaya aratha daitae haya

```

class FreeToPlayGame:
    def __init__(self, name):
        self.name = name
        self.players = 0
        self.paying_players = 0
        self.revenue = 0

    def add_player(self):
        self.players += 1

    def make_purchase(self, amount):
        self.paying_players += 1
        self.revenue += amount

    def get_metrics(self):
        conversion_rate = (self.paying_players / self.players * 100) if self.players > 0 else 0
        arpu = self.revenue / self.players if self.players > 0 else 0
        print(f"Players: {self.players}, Paying: {self.paying_players}")
        print(f"Conversion: {conversion_rate:.2f}%, ARPU: ${arpu:.2f}")

```

****F2P aira dharana:****

F2P MONETIZATION TYPES

1. COSMETIC (kasamaetaika)
 - Skins, Emotes, Outfits
 - No gameplay advantage
 - Example: Fortnite, League of Legends
2. CONSUMABLE (bayabahaaarayaogaya)
 - Potions, Boosts, Energy
 - Temporary advantage
 - Example: Candy Crush, Clash of Clans
3. CONTENT GATE (baissayabasatau gaeta)
 - Unlock levels, Characters
 - Core gameplay locked
4. GACHA (gaaachaaa)
 - Random item pulls
 - Example: Genshin Impact
5. BATTLE PASS (bayaaatala paaasa)
 - Seasonal rewards

Free + Premium track

****F2P gaurautabapauurana maetaraikasa:****

F2P KEY METRICS			
METRIC	GOOD	AVERAGE	POOR
Conversion Rate	>5%	2-5%	<2%
ARPU (Daily)	>\$0.50	\$0.20	<\$0.20
ARPPU	>\$50	\$20-50	<\$20
Retention (Day 1)	>40%	25-40%	<25%
Retention (Day 7)	>20%	10-20%	<10%
Retention (Day 30)	>10%	5-10%	<5%
LTV	>\$10	\$5-10	<\$5

28.4 DLC aiba aikasapaaanashana payaaaka (DLC & Expansion Packs)

```
class DLCSystem:
    def __init__(self, base_game_name, base_price):
        self.base_game = base_game_name
        self.base_price = base_price
        self.dlcs = []

    def add_dlc(self, name, price, content):
        self.dlcs.append({"name": name, "price": price, "content": content})

    def calculate_bundle_price(self):
        total = self.base_price
        for dlc in self.dlcs:
            total += dlc["price"]
        return total * 0.8 # 20% bundle discount

game = DLCSystem("Skyrim", 29.99)
game.add_dlc("Dawnguard", 19.99, ["New Quests", "Vampire Powers"])
game.add_dlc("Dragonborn", 19.99, ["New Island", "Dragon Shouts"])
```

****DLC vs Expansion Pack:****

DLC: chhaota kanataenata, \$5-\$20, 2-10 ghanataaa

Expansion: bada kanataenata, \$20-\$40, 20-50+ ghanataaa

28.5 kasamaetaika manaitaaaijaeshana (Cosmetic Monetization)

kasamaetaika manaitaaaijaeshanae shaudhau daekhatae bhaaalao aaitaema baikarai karaaa haya, gaemapalae kaonao saubaidhaaa daeya naaa

****kasamaetaika aaitaemaera udaaaharana:****

****saubaidhaaa:**** No Pay-to-Win, Player Expression, Ethical, Long-term Revenue

28.6 bayaaatala paaasa saisataema (Battle Pass System)

```
class BattlePass:
    def __init__(self, season_name, price):
        self.season_name = season_name
        self.price = price
```

```
self.levels = 100
self.is_premium = False

def purchase_premium(self):
    self.is_premium = True
    print(f"Premium Battle Pass purchased for ${self.price}")

def claim_reward(self, level):
    print(f"Reward claimed at Level {level}")
    if self.is_premium:
        print("Premium reward also claimed!")
```

****bayaaatala paaasa kaaathaaamao:****

BATTLE PASS STRUCTURE		
Level	Free Track	Premium Track
1	100 Gold	Rare Skin
5	500 Gold	Epic Emote
10	Health Potion x10	Rare Banner
15	1000 Gold	Epic Skin
20	Rare Weapon	Legendary Emote
30	2000 Gold	Epic Skin
50	5000 Gold	Legendary Mount
100	Legendary Weapon	Exclusive Legendary
Price: \$9.99 Duration: 90 Days		
Free Track: 30% of rewards Premium: 70%		

28.7 baijanyaaapana manaitaaaijaeshana (Ad Monetization)

```
class AdSystem:
    def __init__(self):
        self.ad_types = {
            "rewarded": {"cpm": 10.00, "ux": "positive"},
            "interstitial": {"cpm": 5.00, "ux": "neutral"},
            "banner": {"cpm": 1.00, "ux": "negative"}
        }
        self.revenue = 0

    def show_rewarded_ad(self, reward_amount):
        self.revenue += self.ad_types["rewarded"]["cpm"] / 1000
        print(f"Rewarded Ad shown - Player gets {reward_amount} Gems")
        return reward_amount
```

****baijanyaaapanaera taulanaaa:****

AD TYPES COMPARISON			
TYPE	CPM	UX	FREQUENCY
Rewarded	\$8-15	Good	Player chooses
Interstitial	\$4-8	Neutral	Every few mins
Banner	\$0.50-2	Bad	Always visible
Video	\$10-20	Neutral	Between levels
BEST PRACTICE: Use Rewarded Ads primarily			

28.8 saaabasakaraipashana madaela (Subscription Model)

```
class SubscriptionModel:
    def __init__(self, game_name):
        self.game_name = game_name
```

```
self.tiers = {
    "basic": {"price": 4.99, "benefits": ["Remove Ads", "500 Gems/Month"]},
    "premium": {"price": 9.99, "benefits": ["Remove Ads", "1000 Gems", "Exclusive
    "ultimate": {"price": 19.99, "benefits": ["Remove Ads", "2000 Gems", "Early Ac
}
self.subscribers = {"basic": 0, "premium": 0, "ultimate": 0}

def subscribe(self, tier):
    self.subscribers[tier] += 1
    print(f"New {tier} subscriber: ${self.tiers[tier]['price']}/month")

def calculate_monthly_revenue(self):
    total = 0
    for tier, count in self.subscribers.items():
        total += count * self.tiers[tier]["price"]
    return total

**saaabasakaraipashana taaayaaara:**

BASIC ($4.99/month): Remove Ads, 500 Gems/Month
PREMIUM ($9.99/month): + Exclusive Skins, Early Access
ULTIMATE ($19.99/month): + Priority Support, Beta Access
```

28.9 dhaaaraaa-janaita dharana taulanaaaa (Genre Comparison)

MONETIZATION BY GENRE		
GENRE	BEST MODEL	EXAMPLES
RPG	Premium/DLC	Witcher, Skyrim
Mobile Casual	F2P + Ads	Candy Crush
Mobile Strategy	F2P + IAP	Clash of Clans
Battle Royale	F2P + Cosmetic	Fortnite, Apex
MMO	Subscription	WoW, FFXIV
Sports	Premium + DLC	FIFA, NBA 2K
Racing	Premium + DLC	Forza, Gran Turismo
Indie	Premium	Hollow Knight
Puzzle	F2P + Ads	Wordle, 2048

```
genre_monetization = {
    "RPG": {"primary": "Premium", "secondary": ["DLC", "Expansion"]},
    "Mobile_Casual": {"primary": "F2P", "secondary": ["Ads", "IAP"]},
    "Battle_Royale": {"primary": "F2P", "secondary": ["Battle Pass", "Cosmetic"]},
    "MMO": {"primary": "Subscription", "secondary": ["Cosmetic Shop", "Expansion"]},
    "Indie": {"primary": "Premium", "secondary": ["DLC"]}
}
```

28.10 raebhainaiu kayaaalakaulaeshana (Revenue Calculation)

```
class RevenueCalculator:
    def calculate_premium_revenue(self, price, copies_sold):
        gross = price * copies_sold
        platform_fee = gross * 0.30
        net = gross - platform_fee
        return {"gross": gross, "platform_fee": platform_fee, "net": net}

    def calculate_f2p_revenue(self, players, conversion_rate, avg_spend):
        paying = players * (conversion_rate / 100)
        total = paying * avg_spend
        return {"paying_players": paying, "total_revenue": total}
```

```
def calculate_battle_pass_revenue(self, players, purchase_rate, price):
    purchasers = players * purchase_rate
    return purchasers * price

calc = RevenueCalculator()
premium = calc.calculate_premium_revenue(29.99, 100000)
print(f"Gross: ${premium['gross']}, Net: ${premium['net']}")

**raebhainaiu taulanaaa (1 mailaiyana khaelaoyaaada):**

Premium ($30): $30M gross (one-time)
F2P (5% conv, $10): $500K/month (recurring)
Subscription ($10): $10M/month (if 10% subscribe)
Battle Pass ($10): $1M/month (if 10% purchase)
Ads (1M DAU): $100K/month
```

28.11 naaitaika manaitaaaijaeshana (Ethical Monetization)

```
ETHICAL MONETIZATION CHECKLIST

[X] No Pay-to-Win (P2W naei)
[X] Transparent Pricing (sabachachha mauulaya)
[X] Fair Odds (nayaaayaya samabhaaabananaa)
[X] No Predatory Mechanics (parataaaranaaamaulaka kaaushala naei)
[X] Spending Limits (kharachaera saimaaa)
[X] Parental Controls (abhaibhaaabaka naiyanatarana)
[X] Easy Refund (sahaja raiphaanada)
[X] Free Player Experience (pharai khaelaoyaaadaera abhaijanyataaa)
```

28.12 khaelaoyaaada manaobaijanyaaana (Player Psychology)

```
PLAYER PSYCHOLOGY IN MONETIZATION

1. LOSS AVERSION: "Limited time offer ending soon!"
2. SUNK COST: "You've already invested 50 hours!"
3. FOMO: "Exclusive skin available this week only!"
4. SOCIAL PROOF: "10,000 players bought this today!"
5. SCARCITY: "Only 100 left in stock!"

USE RESPONSIBLY - Don't exploit players!
```

daaayaaagaraaama (Diagrams)

manaitaaaijaeshana madaela taulanaaa

```
MONETIZATION MODEL COMPARISON

REVENUE POTENTIAL:
Low      Premium
Med      DLC + Expansion
High     F2P + IAP
V.High   F2P + Sub + Battle Pass
```

anaushailana (Exercises)

anaushailana 1: manaitaaaijaeshana madaela nairabaaachana

aapanaaara gaemaera janaya saeraaa monetization madaela baechhae naina:

- gaemaera dharana nairadhaaarana karauna
- taaaragaeta adaiyaenasa chainauna
- Revenue projection taairai karauna

anaushailana 2: Battle Pass daijaaaina

aikatai Battle Pass daijaaaina karauna:

- 100 laebhaela
- pharai aiba paraimaiyaaama tarayaaaka
- mauulaya nairadhaaarana

anaushailana 3: Revenue Calculation

aikatai F2P gaemaera janaya revenue calculation karauna:

- 1 mailaiyana khaelaoyaaada
- 5% conversion rate
- \$10 average spend

saaadhaaarana bhaula (Common Mistakes)

1. Pay-to-Win haauyaaa

- # bhaula: paeida aaitaema gaemapalae saubaidhaaa daeya
- # sathaika: shaudhau kasamaetaika monetization

2. atairaikata baijanyaaapana

- # bhaula: paratai 30 saekaenadae baijanyaaapana
- # sathaika: Rewarded ads bayabahaaara karauna, player choice daina

3. jataila mauulaya kaaathaaamao

- # bhaula: 10tai bhainana currency
- # sathaika: 2-3tai sahaja maudaraaa

4. FOMO shaossana

- # bhaula: saba samaya "Limited Time" balaaa
- # sathaika: satayaikaaaraera scarcity bayabahaaara karauna

taipasa (Tips)

1. Player-First Approach

khaelaoyaaadadaera abhaijanyataaakae paraaadhhaanaya daina, shaudhau aayakae naya

2. A/B Testing

baibhainana monetization strategy paraiikassaaa karauna

3. Data-Driven Decisions

daetaaa thaekae saidadhhaanata naina, anaumaaana naya

4. Long-term Thinking

daiiraghasamayaera samaparaka taairai karauna, aikabaaaraera aaya naya

5. Community Feedback

khaelaoyaaadadaera mataaamata shaunauna aiba samanabaya karauna

saaarasakassaepa (Summary)

Premium Model: aikabaaarae kaenauna, samapauurana gaema paaana
F2P Model: bainaamaulayae khaelauna, atairaikata saaamagaraii kaenauna
DLC: atairaikata kanataenata kaenauna
Battle Pass: maausaumai paurasakaaara paaana
Ads: baijanyaaapana daekhae paurasakaaara paaana
Subscription: maaasaika phai daiyae saubaidhaaa paaana
Cosmetic: shaudhau daekhatae bhaaalao aaitaema kaenauna
Ethical: naaitaikabhhaabae manaitaaaija karauna

GAME DEVELOPMENT MASTERBOOK - Chapter 28

PART 29

bhauLa: "aitai saeraaa gaema habae!"

adhayaaaya 29: gaema maaarakaetai (GAME MARKETING)

shaekhaaara udadaeshaya (Learning Goal)

aii adhayaaayae aapanai shaikhabaena kaiibhaaabaee aapanaaara gaemakae parachaaara karatae haya, yaaara madhayae Pre-launch Marketing, Social Media, Steam Page Optimization, Influencer Outreach aiba Community Building anatarabhaukata

baishada bayaaakhayaaa (Detailed Explanation)

29.1 parai-lacha maaarakaetai raodamayaaapa (Pre-launch Marketing Roadmap)

PRE-LAUNCH MARKETING TIMELINE

12 MONTHS BEFORE LAUNCH:

- Create brand identity
- Set up social media accounts
- Start building community

9 MONTHS BEFORE:

- Release announcement trailer
- Start posting development updates
- Create Steam page

6 MONTHS BEFORE:

- Release gameplay trailer
- Start press outreach
- Build influencer list

3 MONTHS BEFORE:

- Release demo
- Launch wishlist campaign
- Ramp up social media

1 MONTH BEFORE:

- Final trailer
- Review copies to press
- Community events

LAUNCH WEEK:

- Release day posts
- Live streams
- Monitor reviews

POST-LAUNCH:

Thank you posts
Patch notes
Community feedback

29.2 barayaaanada taairai (Brand Creation)

```
class GameBrand:
    def __init__(self, name):
        self.name = name
        self.colors = {
            "primary": "#FF5733",
            "secondary": "#33FF57",
            "accent": "#3357FF"
        }
        self.voice = {
            "tone": "Professional yet friendly",
            "language": "English",
            "keywords": ["Adventure", "Epic", "Discovery"]
        }
        self.assets = {
            "logo": "logo.png",
            "banner": "banner.png",
            "icon": "icon.png"
        }

    def create_style_guide(self):
        print(f"Brand Style Guide for {self.name}")
        print(f"Colors: {self.colors}")
        print(f"Voice: {self.voice}")

brand = GameBrand("Dragon Quest")
brand.create_style_guide()
```

****barayaaanadai ailaimaenata:****

BRAND ELEMENTS

1. LOGO (laogao)
 - Simple, memorable, scalable
 - Works in different sizes
 - Black and white version needed
2. COLORS (ra)
 - Primary color (1)
 - Secondary color (1)
 - Accent colors (2-3)
3. VOICE (kanatha)
 - Professional, Casual, Humorous?
 - Consistent across all platforms
4. TYPOGRAPHY (phanata)
 - Title font (unique)
 - Body font (readable)
5. MASCOT (maaasakata)
 - Optional but memorable

29.3 taraeilaaara taairai (Trailer Creation)

****taraeilaaaraera dharana:****

****taraeilaaara taairaira taipasa:****

TRAILER CREATION TIPS

DO:

Start with a hook (first 5 seconds)
Show best gameplay moments
Keep it short (60-90 seconds for launch)
Use exciting music
End with call to action
Include release date

DON'T:

Too much story exposition
Low quality footage
Too many cuts
Generic music
Missing release date

29.4 sakarainashata nairadaeshaikaaa (Screenshot Guidelines)

```
class ScreenshotGuide:
    def __init__(self):
        self.best_practices = [
            "High resolution (1920x1080 minimum)",
            "Show diverse environments",
            "Include UI elements (optional)",
            "Show action moments",
            "Use good lighting",
            "No text overlays (save for key art)"
        ]
        self.quantity = {
            "steam_page": "5-10 screenshots",
            "press_kit": "10-20 screenshots",
            "social_media": "3-5 per post"
        }
```

****sakarainashata taulanaaa:****

GOOD Screenshots:

- Action-packed moments
- Diverse environments
- Good lighting
- Clear composition
- High resolution

BAD Screenshots:

- Empty rooms
- Dark/blurry
- Too much UI
- Low resolution
- Boring composition

29.5 Steam Page Optimization

```
class SteamPageOptimizer:
    def __init__(self):
        self.elements = {
            "capsule_art": "Eye-catching header image",
```

```

        "screenshots": "5-10 high quality screenshots",
        "trailer": "60-90 second gameplay trailer",
        "description": "Compelling game description",
        "tags": "Relevant genre tags",
        "features": "Key features list",
        "system_requirements": "Minimum and recommended specs"
    }

```

```

def optimize(self):
    print("Steam Page Optimization Checklist:")
    for element, desc in self.elements.items():
        print(f" [x] {element}: {desc}")

```

****Steam Page apataimaaaijaeshana:****

```

        STEAM PAGE OPTIMIZATION
CAPSULE ART:
    460x215 pixels (main capsule)
    374x448 pixels (vertical capsule)
    616x353 pixels (header capsule)
SCREENSHOTS:
    1920x1080 or 1280x720
    Minimum 5 screenshots
    Maximum 15 screenshots
TRAILER:
    MP4 format
    1920x1080 resolution
    60-90 seconds length
DESCRIPTION:
    First line should hook readers
    Bullet points for features
    Clear call to action
TAGS:
    Choose 1-5 relevant tags
    Most important tag first
    Research competitor tags

```

29.6 Store Description Writing

```

class StoreDescription:
    def __init__(self, game_name):
        self.game_name = game_name
        self.structure = {
            "hook": "First paragraph - grab attention",
            "features": "Bullet points - key features",
            "story": "Brief story overview",
            "gameplay": "What players do",
            "call_to_action": "Buy now / Wishlist"
        }

    def write_hook(self):
        return f"""
        {self.game_name} is an epic adventure that combines
        stunning visuals with deep gameplay mechanics.
        """

    def write_features(self):
        return """
        Key Features:
        - Open world exploration

```

```

- Deep combat system
- 50+ hours of content
- Multiple endings
- Co-op multiplayer
"""

```

```

description = StoreDescription("Dragon Quest")
print(description.write_hook())
print(description.write_features())

```

****Store Description taemapalaeta:****

```

[HOOK - First Sentence]
Game Name is a [genre] game that [unique selling point].

```

```

[FEATURES - Bullet Points]
Key Features:
- Feature 1
- Feature 2
- Feature 3

```

```

[STORY - Brief Overview]
In a world where [premise], you must [objective].

```

```

[GAMEPLAY - What Players Do]
Explore vast landscapes, battle enemies, and uncover secrets.

```

```

[CALL TO ACTION]
Wishlist now and be ready for launch!

```

29.7 Social Media Strategy

```

class SocialMediaStrategy:
    def __init__(self):
        self.platforms = {
            "twitter": {
                "best_time": "9 AM - 12 PM",
                "frequency": "3-5 posts/day",
                "content": ["Screenshots", "Dev updates", "Memes"]
            },
            "instagram": {
                "best_time": "11 AM - 2 PM",
                "frequency": "1-2 posts/day",
                "content": ["Screenshots", "Behind the scenes", "Stories"]
            },
            "tiktok": {
                "best_time": "7 PM - 10 PM",
                "frequency": "1-3 posts/day",
                "content": ["Short clips", "Devlogs", "Trends"]
            },
            "youtube": {
                "best_time": "2 PM - 5 PM",
                "frequency": "1-2 videos/week",
                "content": ["Devlogs", "Trailers", "Gameplay"]
            }
        }

    def get_content_calendar(self):
        print("Weekly Content Calendar:")
        print("Monday: Development Update")
        print("Tuesday: Screenshot Tuesday")

```

```

print("Wednesday: Work in Progress")
print("Thursday: Throwback / Lore")
print("Friday: Community Spotlight")
print("Saturday: Fun / Memes")
print("Sunday: Rest / Light engagement")

```

****Social Media kanataenata kayaaalaenadaaara:****

```

WEEKLY CONTENT CALENDAR
MONDAY:    Development Update
TUESDAY:    Screenshot Tuesday
WEDNESDAY:  Work in Progress
THURSDAY:   Lore / Throwback
FRIDAY:     Community Spotlight
SATURDAY:   Fun / Memes
SUNDAY:     Rest / Light engagement
DAILY:
- Reply to comments
- Engage with community
- Share relevant content

```

29.8 Discord Community Building

```

class DiscordCommunity:
    def __init__(self, server_name):
        self.server_name = server_name
        self.channels = {
            "general": "General chat",
            "announcements": "Official announcements",
            "dev-updates": "Development updates",
            "screenshots": "Share screenshots",
            "suggestions": "Player suggestions",
            "bug-reports": "Bug reports",
            "off-topic": "Off-topic chat"
        }
        self.roles = ["Admin", "Moderator", "Developer", "Early Supporter", "Member"]

    def create_server(self):
        print(f"Creating Discord server: {self.server_name}")
        for channel, desc in self.channels.items():
            print(f"  #{channel}: {desc}")

discord = DiscordCommunity("Game Dev Community")
discord.create_server()

```

****Discord kaaathaaamao:****

```

DISCORD SERVER STRUCTURE
ANNOUNCEMENTS
#announcements
#patch-notes
COMMUNITY
#general
#introductions
#off-topic
GAME DISCUSSION
#gameplay
#screenshots
#suggestions
DEVELOPMENT

```

```
#dev-updates
#bug-reports
#feedback
EVENTS
#events
#giveaways
ROLES:
- Admin (Red)
- Moderator (Blue)
- Developer (Green)
- Early Supporter (Gold)
- Member (Default)
```

29.9 Demo Strategy

```
class DemoStrategy:
    def __init__(self, game_name):
        self.game_name = game_name
        self.timing = {
            "ideal": "3-6 months before launch",
            "too_early": "More than 9 months before",
            "too_late": "Less than 1 month before"
        }
        self.contents = [
            "First level or chapter",
            "Core mechanics showcase",
            "Limited content (2-4 hours)",
            "Save transfer to full game (optional)"
        ]

    def calculate_impact(self, wishlist_before, wishlist_after):
        conversion = (wishlist_after - wishlist_before) / wishlist_before * 100
        print(f"Wishlist increase: {conversion:.1f}%")

demo = DemoStrategy("Dragon Quest")
demo.calculate_impact(1000, 2500)
```

****Demo Strategy taulanaaa:****

```
DEMO STRATEGY COMPARISON

TIMING:
3-6 months before: IDEAL [x]
9+ months before: Too early
<1 month before: Too late

CONTENT:
First level/chapter
Core mechanics
2-4 hours of gameplay
Save transfer option

PLATFORMS:
Steam (Steam Next Fest)
itch.io
Dedicated demo page

BENEFITS:
Increases wishlists
Builds community
Gets feedback
Creates buzz
```

29.10 Wishlist Strategy

```
class WishlistStrategy:
    def __init__(self):
        self.targets = {
            "minimum": 7000,
            "good": 20000,
            "great": 50000,
            "viral": 100000
        }

    def estimate_launch_sales(self, wishlists):
        conversion_rate = 0.15 # 15% conversion
        launch_sales = wishlists * conversion_rate
        return launch_sales

strategy = WishlistStrategy()
for level, count in strategy.targets.items():
    sales = strategy.estimate_launch_sales(count)
    print(f"[level]: {count} wishlists -> {sales:.0f} launch sales")
```

****Wishlist Targets:****

Minimum: 7,000 wishlists
 Good: 20,000 wishlists
 Great: 50,000 wishlists
 Viral: 100,000+ wishlists

Conversion Rate: ~15% on launch

29.11 Press Kit Creation

```
class PressKit:
    def __init__(self, game_name):
        self.game_name = game_name
        self.contents = {
            "game_description": "Short and long descriptions",
            "key_features": "5-10 key features",
            "screenshots": "10-20 high quality screenshots",
            "trailers": "Announcement and gameplay trailers",
            "logo": "High resolution logo files",
            "screenshots": "Various gameplay screenshots",
            "fact_sheet": "Quick facts about the game",
            "contact_info": "Press contact details"
        }

    def create_kit(self):
        print(f"Press Kit for {self.game_name}:")
        for item, desc in self.contents.items():
            print(f"  [x] {item}: {desc}")

press_kit = PressKit("Dragon Quest")
press_kit.create_kit()
```

****Press Kit kanataenata:****

PRESS KIT CONTENTS

GAME DESCRIPTION

Short description (1-2 paragraphs)

Long description (3-4 paragraphs)

One-liner (for quick mentions)
 VISUAL ASSETS
 Logo (PNG, SVG, high-res)
 Screenshots (10-20, various scenes)
 Key art (hero images)
 GIFs (for social media)
 VIDEOS
 Announcement trailer
 Gameplay trailer
 Press-specific footage
 FACT SHEET
 Developer name
 Release date
 Platform
 Price
 Genre
 Key features (bullet points)
 CONTACT
 Press contact email
 Social media links
 Steam page URL

29.12 Influencer/YouTuber Outreach

```

class InfluencerOutreach:
    def __init__(self):
        self.tiers = {
            "micro": {"followers": "10K-100K", "cost": "Free game", "response_rate": "30%"},
            "mid": {"followers": "100K-500K", "cost": "$500-2000", "response_rate": "20%"},
            "large": {"followers": "500K-1M", "cost": "$2000-10000", "response_rate": "10%"},
            "mega": {"followers": "1M+", "cost": "$10000+", "response_rate": "5%"}
        }

    def find_influencers(self, game_genre):
        print(f"Finding influencers for {game_genre}:")
        print("1. Search YouTube/Twitch for similar games")
        print("2. Check their engagement rate")
        print("3. Verify audience alignment")
        print("4. Send personalized outreach")

outreach = InfluencerOutreach()
outreach.find_influencers("RPG")
  
```

****Influencer Outreach kaaushala:****

INFLUENCER OUTREACH STRATEGY

STEP 1: RESEARCH
 Find influencers in your genre
 Check their content quality
 Verify engagement rate
 Ensure audience alignment
 STEP 2: OUTREACH
 Personalized email/message
 Include key game info
 Offer free game key
 No pressure for coverage
 STEP 3: FOLLOW UP
 Send reminder after 1 week
 Provide additional assets
 Be responsive

TIER STRATEGY:

Micro (10K-100K): Free game, high response

Mid (100K-500K): \$500-2000, decent response

Large (500K-1M): \$2000-10000, lower response

Mega (1M+): \$10000+, lowest response

daaayaaagaraaama (Diagrams)

maaraketai phaaanaela

MARKETING FUNNEL

AWARENESS
(100,000)

INTEREST
(30,000)

CONSIDER
(10,000)

WISHLIST
(5,000)

PURCHASE
(1,000)

Conversion Rates:

Awareness -> Interest: 30%

Interest -> Consider: 33%

Consider -> Wishlist: 50%

Wishlist -> Purchase: 20%

saoshayaaala maidaiyaaa kanataenata maikasa

SOCIAL MEDIA CONTENT MIX

Content Type	Percentage	Frequency
Development Updates	30%	2x/week
Screenshots/Art	25%	3x/week
Community Features	20%	2x/week
Behind the Scenes	15%	1x/week
Memes/Fun	10%	1x/week

Platform Mix:

Twitter: 40% (Short updates, engagement)

Instagram: 25% (Visual content)

TikTok: 20% (Short clips, trends)

YouTube: 15% (Long-form content)

anaushailana (Exercises)

anaushailana 1: maaraketai palayaaana taairai

- aapanaaara gaemaera janaya aikatai maaraketai palayaaana taairai karauna:
- 12 maaasaera raodamayaaapa
 - saoshayaaala maidaiyaaa kaaushala
 - baaajaeta paraikalapanaaa

anaushailana 2: Steam Page apataimaaaijaeshana

aapanaaara gaemaera Steam page apataimaaaija karauna:

- Capsule art daijaaaina karauna
- Screenshot nairabaaachana karauna
- Description laikhauna

anaushailana 3: Influencer List taairai

aapanaaara gaemaera janaya influencer list taairai karauna:

- 10 jana micro-influencer
- 5 jana mid-tier influencer
- Outreach email laikhauna

saaadhaaarana bhaula (Common Mistakes)

1. atairaikata parataisharautai

- # bhaula: "aitai saeraaa gaema habae!"
- # sathaika: baaasatabasamamata paratayaaashaaa taairai karauna

2. saoshayaaala maidaiyaaa upaekassaaa

- # bhaula: shaudhau gaema taairai, maaarakaetai naya
- # sathaika: saoshayaaala maidaiyaaaya sakaraiya thaaakauna

3. daemao naaa daeautyaaa

- # bhaula: daemao chhaaadaaaai lacha
- # sathaika: 3-6 maaasa aagae daemao daina

4. Press Kit anaupasathaitai

- # bhaula: Press kit taairai naaa karaaa
- # sathaika: samapauurana press kit taairai karauna

taipasa (Tips)

1. Early and Often

yata taaadaaataaadai samabhaba maaarakaetai shaurau karauna

2. Authentic Voice

satayaikaaaraera aiba sa kanathae kathaaa balauna

3. Community First

kamaiunaitaika paraaadhaanaya daina

. Visual Storytelling

chhabai aiba bhaidaiu daiyae galapa balauna

5. Consistency

naiyamaita aiba dhaaaraabaaahaika thaaakauna

saaarasakassaepa (Summary)

Pre-launch: 12 maaasa aagae shaurau karauna
Brand: laogao, ra, kanatha nairadhaaarana karauna
Trailer: sakassaipata aiba aakarassanaiiya raaakhauna
Steam Page: samapauurana apataimaaaija karauna
Social Media: naiyamaita paosata karauna
Discord: kamaiunaitai taairai karauna
Demo: 3-6 maaasa aagae railaija karauna
Wishlist: kamapakassae 7,000 taaaragaeta karauna
Press Kit: samapauurana kit taairai karauna
Influencers: dhaaapae dhaaapae outreach karauna

GAME DEVELOPMENT MASTERBOOK - Chapter 29

PART 30

adhayaaaya 30: Steam Launch

adhayaaaya 30: Steam Launch

shaekhaara udadaeshaya (Learning Goal)

aai adhayaaayae aapanai shaikhabaena Steam-ai gaema lacha karaara samapauurana parakaraiyaaa, yaaara madhayae Steam Page setup, Store Assets, Pricing, Review Management aiba Post-launch Updates anatarabhaukata

baishada bayaaakhayaaa (Detailed Explanation)

30.1 Steam Launch Checklist

COMPLETE STEAM LAUNCH CHECKLIST

PRE-LAUNCH (3-6 months before):

- ☐ Create Steamworks account
- ☐ Set up Steam page
- ☐ Upload capsule art and screenshots
- ☐ Write store description
- ☐ Set pricing and release date
- ☐ Enable wishlists
- ☐ Set up Steam Cloud (if needed)
- ☐ Configure achievements
- ☐ Set up Steam Trading Cards (optional)

LAUNCH PREPARATION (1 month before):

- ☐ Send review copies to press
- ☐ Set up demo (if applicable)
- ☐ Final trailer
- ☐ Community announcement
- ☐ Prepare launch day content
- ☐ Test Steam integration

LAUNCH DAY:

- ☐ Monitor store page
- ☐ Respond to reviews
- ☐ Social media announcements
- ☐ Community engagement
- ☐ Monitor crash reports

POST-LAUNCH (First week):

- ☐ Respond to all reviews
- ☐ Fix critical bugs
- ☐ Patch notes
- ☐ Thank you posts

[] Monitor sales data

30.2 Steam Page Setup

```
class SteamPageSetup:
    def __init__(self, game_name):
        self.game_name = game_name
        self.required_assets = {
            "capsule_460x215": "Main capsule image",
            "capsule_374x448": "Vertical capsule",
            "capsule_616x353": "Header capsule",
            "screenshots": "5-10 screenshots (1920x1080)",
            "trailer": "60-90 second video",
            "logo": "Game logo"
        }
        self.store_items = {
            "short_description": "1-3 sentences",
            "long_description": "Detailed description",
            "features": "Key features list",
            "tags": "1-5 genre tags",
            "system_requirements": "Min/Recommended specs"
        }

    def validate_assets(self):
        print(f"Validating Steam assets for {self.game_name}:")
        for asset, desc in self.required_assets.items():
            print(f" [x] {asset}: {desc}")

steam = SteamPageSetup("Dragon Quest")
steam.validate_assets()
```

****Steam Page kaaathaaamao:****

STEAM PAGE STRUCTURE

HEADER:

- Capsule Art (460x215)
- Game Title
- Developer/Publisher Name
- Release Date

MEDIA:

- Trailer (video)
- Screenshots (5-10 images)

DESCRIPTION:

- Short description (visible in search)
- Long description (detailed)
- Features list

SIDEBAR:

- Price
- Release date
- Developer
- Publisher
- Tags
- Reviews

ADDITIONAL:

- System requirements
- Metacritic score (if available)
- Similar games

30.3 Store Assets

```
class StoreAssets:
    def __init__(self):
        self.specs = {
            "capsule_main": {"width": 460, "height": 215, "format": "PNG"},
            "capsule_vertical": {"width": 374, "height": 448, "format": "PNG"},
            "capsule_header": {"width": 616, "height": 353, "format": "PNG"},
            "screenshot": {"width": 1920, "height": 1080, "format": "PNG/JPG"},
            "trailer": {"resolution": "1080p", "format": "MP4", "max_length": "90s"}
        }

    def check_compliance(self, asset_type, width, height):
        spec = self.specs[asset_type]
        if width == spec["width"] and height == spec["height"]:
            print(f"[x] {asset_type} is compliant")
        else:
            print(f" {asset_type} needs to be {spec['width']}x{spec['height']}")

assets = StoreAssets()
assets.check_compliance("capsule_main", 460, 215)
assets.check_compliance("screenshot", 1920, 1080)
```

****Store Assets saaranaii:****

STORE ASSETS SPECIFICATIONS			
ASSET	SIZE	FORMAT	NOTES
Main Capsule	460x215	PNG	Most important
Vertical Capsule	374x448	PNG	Mobile display
Header Capsule	616x353	PNG	Store header
Screenshots	1920x1080	PNG	Min 5, Max 15
Trailer	1080p	MP4	60-90 seconds
Logo	400x400	PNG	Transparent bg
CAPSULE ART TIPS:			
High contrast colors			
Readable at small sizes			
Include game title			
Show key visual element			

30.4 Description Writing

```
class SteamDescription:
    def __init__(self, game_name):
        self.game_name = game_name

    def write_short_description(self):
        return f"""
        {self.game_name} is an epic adventure featuring
        stunning visuals, deep gameplay, and an unforgettable
        story. Explore vast worlds, battle enemies, and
        become the hero.
        """

    def write_long_description(self):
        return f"""
        ## About {self.game_name}

        {self.game_name} is a [genre] game that combines
        [unique feature 1] with [unique feature 2].
        """
```



```

## Key Features

- **Feature 1**: Description
- **Feature 2**: Description
- **Feature 3**: Description
- **Feature 4**: Description
- **Feature 5**: Description

## Story

In a world where [premise], you must [objective].

## Gameplay

Experience [gameplay description].

## Join Our Community

Discord: [link]
Twitter: [link]
"""

```

```

desc = SteamDescription("Dragon Quest")
print(desc.write_short_description())

```

****Description taemapalaeta:****

SHORT DESCRIPTION (1-3 sentences):
 [Game Name] is a [genre] game featuring [unique selling point].
 [Key feature] and [key feature] await you.

LONG DESCRIPTION:
 ## About [Game Name]
 [1-2 paragraphs about the game]

```

## Key Features
- **Feature 1**: Description
- **Feature 2**: Description
- **Feature 3**: Description

```

```

## Story
[Brief story overview]

```

```

## Community
[Social links]

```

30.5 Tag Selection

```

class TagSelection:
    def __init__(self):
        self.tag_categories = {
            "genre": ["RPG", "Action", "Adventure", "Strategy", "Simulation"],
            "theme": ["Fantasy", "Sci-Fi", "Horror", "Historical", "Modern"],
            "feature": ["Singleplayer", "Multiplayer", "Co-op", "Online"],
            "style": ["Pixel Graphics", "3D", "2D", "Hand-drawn"]
        }

    def select_tags(self, game_features):
        selected = []
        for feature in game_features:
            for category, tags in self.tag_categories.items():
                if feature in tags:
                    selected.append(feature)

```

```

        return selected[:5] # Max 5 tags

tags = TagSelection()
selected = tags.select_tags(["RPG", "Fantasy", "Singleplayer", "3D"])
print(f"Selected tags: {selected}")

```

****Tag Strategy:****

```

TAG SELECTION STRATEGY

TAG HIERARCHY:
1. Primary Genre (most important)
2. Secondary Genre
3. Theme/Setting
4. Feature
5. Style
EXAMPLES:
Action RPG: Action, RPG, Fantasy, Singleplayer, 3D
Puzzle Game: Puzzle, Casual, Singleplayer, 2D
Strategy: Strategy, Real-Time Strategy, Multiplayer
TIPS:
Research competitor tags
Use tags players search for
Don't use irrelevant tags
Max 5 tags recommended

```

30.6 Pricing Strategy

```

class PricingStrategy:
    def __init__(self):
        self.price_points = {
            "indie_small": {"usd": 4.99, "eur": 4.99, "inr": 349},
            "indie_medium": {"usd": 14.99, "eur": 14.99, "inr": 999},
            "indie_large": {"usd": 24.99, "eur": 24.99, "inr": 1699},
            "aaa": {"usd": 59.99, "eur": 59.99, "inr": 3999}
        }
        self.discount_tiers = {
            "launch": 0.10, # 10% launch discount
            "sale": 0.25, # 25% during sales
            "deep": 0.50 # 50% deep discount
        }

    def recommend_price(self, playtime_hours, content_size):
        if playtime_hours < 5:
            return "indie_small"
        elif playtime_hours < 15:
            return "indie_medium"
        elif playtime_hours < 30:
            return "indie_large"
        else:
            return "aaa"

pricing = PricingStrategy()
recommendation = pricing.recommend_price(20, "large")
print(f"Recommended price point: {recommendation}")

```

****Pricing Strategy:****

```

PRICING STRATEGY

PRICE POINTS:
$4.99: Small indie (2-5 hours)

```

\$14.99: Medium indie (5-15 hours)
 \$24.99: Large indie (15-30 hours)
 \$59.99: AAA (30+ hours)
 DISCOUNT STRATEGY:
 Launch: 10% discount (first week)
 Seasonal Sales: 25% off
 Deep Sales: 50% off (anniversary)
 Bundle: 20-30% off with other games
 REGIONAL PRICING:
 Adjust for regional economies
 Steam suggests regional prices
 Consider PPP (Purchasing Power Parity)
 TIPS:
 Don't underprice your game
 Research competitor pricing
 Consider perceived value
 Price reflects quality

30.7 Demo Release Strategy

```

class DemoStrategy:
    def __init__(self):
        self.timing = {
            "steam_next_fest": "Best opportunity for demos",
            "3_months_before": "Good timing",
            "6_months_before": "Early but acceptable",
            "1_month_before": "Too late for maximum impact"
        }

    def calculate_impact(self, wishlists_before, wishlists_after):
        increase = wishlists_after - wishlists_before
        percentage = (increase / wishlists_before) * 100
        print(f"Wishlists: {wishlists_before} -> {wishlists_after}")
        print(f"Increase: +{increase} ({percentage:.1f}%)")

demo = DemoStrategy()
demo.calculate_impact(1000, 3000)
  
```

****Demo Strategy:****

DEMO STRATEGY
 BEST TIMING:
 Steam Next Fest (bi-annual event)
 3-6 months before launch
 During major gaming events
 DEMO CONTENT:
 First level/chapter
 Core mechanics showcase
 2-4 hours of gameplay
 Save transfer option (optional)
 DEMO BEST PRACTICES:
 End with a cliffhanger
 Include feedback survey
 Limit content to avoid spoilers
 Make it polished
 STEAM NEXT FEST:
 Apply early
 Prepare demo 1 month before
 Be active during the event
 Engage with players

30.8 Wishlist Building

```
class WishlistBuilder:
    def __init__(self):
        self.targets = {
            "minimum": 7000,
            "good": 20000,
            "great": 50000,
            "viral": 100000
        }
        self.conversion_rate = 0.15 # 15% average

    def estimate_launch_sales(self, wishlists):
        return wishlists * self.conversion_rate

    def get_strategy(self):
        return {
            "social_media": "Regular posts with wishlist links",
            "demo": "Include wishlist CTA at end",
            "trailers": "End with wishlist reminder",
            "community": "Discord wishlist events",
            "press": "Include wishlist in press kit"
        }

builder = WishlistBuilder()
for level, count in builder.targets.items():
    sales = builder.estimate_launch_sales(count)
    print(f"{level}: {count} wishlists -> {sales:.0f} estimated sales")
```

****Wishlist Strategy:****

```
WISHLIST BUILDING STRATEGY

TARGETS:
Minimum: 7,000 wishlists
Good: 20,000 wishlists
Great: 50,000 wishlists
Viral: 100,000+ wishlists

STRATEGIES:
Social media posts with wishlist links
Demo with wishlist CTA
Trailers ending with wishlist reminder
Discord community events
Press kit with wishlist information
Steam Next Fest participation

CONVERSION:
Average conversion: 15%
High conversion: 20-25%
Factors: Game quality, marketing, timing
```

30.9 Playtest Program

```
class PlaytestProgram:
    def __init__(self):
        self.phases = {
            "alpha": {"players": 20, "duration": "2 weeks", "focus": "Core mechanics"},
            "beta": {"players": 100, "duration": "1 month", "focus": "Balance"},
            "stress_test": {"players": 1000, "duration": "1 week", "focus": "Performance"}
        }
```

```
def setup_playtest(self, phase):
    config = self.phases[phase]
    print(f"Setting up {phase} playtest:")
    print(f"  Players: {config['players']}")
    print(f"  Duration: {config['duration']}")
    print(f"  Focus: {config['focus']}")
```

```
playtest = PlaytestProgram()
playtest.setup_playtest("beta")
```

****Playtest Program:****

```
PLAYTEST PROGRAM

PHASES:
Alpha: 20 players, 2 weeks, core mechanics
Beta: 100 players, 1 month, balance
Stress Test: 1000 players, 1 week, performance
FEEDBACK COLLECTION:
Surveys
Bug reports
Gameplay feedback
Analytics data
STEAM PLAYTEST:
Apply through Steamworks
Set up playtest build
Invite players
Collect feedback
TIPS:
Start small, scale up
Be responsive to feedback
Fix critical issues quickly
Document everything
```

30.10 Launch Discount Strategy

```
class LaunchDiscountStrategy:
    def __init__(self):
        self.discount_tiers = {
            "launch_week": 0.10,    # 10% first week
            "first_sale": 0.20,     # 20% first major sale
            "deep_discount": 0.40,  # 40% after 6 months
            "permanent": 0.50       # 50% after 1 year
        }

    def calculate_revenue(self, base_price, copies, discount):
        discounted_price = base_price * (1 - discount)
        revenue = discounted_price * copies
        return revenue

strategy = LaunchDiscountStrategy()
base_price = 24.99
copies = 10000

for tier, discount in strategy.discount_tiers.items():
    revenue = strategy.calculate_revenue(base_price, copies, discount)
    print(f"{tier}: {discount*100}% off -> ${revenue:,.2f}")
```

****Launch Discount:****

```
LAUNCH DISCOUNT STRATEGY

DISCOUNT TIERS:
```

Launch Week: 10% off (first 7 days)
 First Sale: 20% off (3 months)
 Deep Discount: 40% off (6 months)
 Permanent: 50% off (1 year+)
 REVENUE EXAMPLE (10,000 copies):
 No discount: \$249,900
 10% off: \$224,910
 20% off: \$199,920
 50% off: \$124,950
 TIPS:
 Don't discount too early
 Monitor sales data
 Consider player sentiment
 Balance revenue vs. accessibility

30.11 Review Management

```

class ReviewManagement:
    def __init__(self):
        self.review_status = {
            "positive": 0,
            "negative": 0,
            "mixed": 0
        }

    def calculate_score(self):
        total = sum(self.review_status.values())
        if total == 0:
            return 0
        positive_rate = self.review_status["positive"] / total
        return positive_rate * 100

    def respond_to_review(self, review_type, response):
        print(f"Responding to {review_type} review: {response}")

reviews = ReviewManagement()
reviews.review_status["positive"] = 850
reviews.review_status["negative"] = 150
print(f"Review Score: {reviews.calculate_score():.1f}%")
  
```

****Review Management:****

REVIEW MANAGEMENT
 REVIEW SCORES:
 Overwhelmingly Positive: 95%+
 Very Positive: 80-95%
 Mostly Positive: 70-80%
 Mixed: 40-70%
 Negative: <40%
 RESPONDING TO REVIEWS:
 Always respond to negative reviews
 Be professional and helpful
 Acknowledge issues
 Offer solutions
 Thank positive reviewers
 IMPROVING REVIEWS:
 Fix bugs quickly
 Respond to community feedback
 Regular updates
 Transparent communication

REVIEW COPY STRATEGY:

Send to press 1-2 weeks before launch
 Include key features info
 Be responsive to press
 Don't pressure for positive reviews

30.12 Community Engagement

```
class CommunityEngagement:
    def __init__(self):
        self.channels = {
            "steam_community": "Forums, announcements",
            "discord": "Real-time chat",
            "twitter": "Quick updates",
            "reddit": "Community discussion"
        }

    def engagement_strategy(self):
        return {
            "daily": "Respond to comments, social media",
            "weekly": "Community updates, Q&A",
            "monthly": "Developer logs, surveys",
            "quarterly": "Major updates, events"
        }

community = CommunityEngagement()
print("Engagement Strategy:")
for frequency, actions in community.engagement_strategy().items():
    print(f" {frequency}: {actions}")
```

Community Engagement:

```
COMMUNITY ENGAGEMENT

CHANNELS:
Steam Community: Forums, announcements
Discord: Real-time chat
Twitter: Quick updates
Reddit: Community discussion
FREQUENCY:
Daily: Respond to comments, social media
Weekly: Community updates, Q&A
Monthly: Developer logs, surveys
Quarterly: Major updates, events
BEST PRACTICES:
Be transparent
Respond quickly
Acknowledge feedback
Share development progress
Celebrate milestones
```

30.13 Post-launch Updates

```
class PostLaunchUpdates:
    def __init__(self):
        self.update_schedule = {
            "day_1_patch": "Critical fixes",
            "week_1": "Bug fixes, minor improvements",
            "month_1": "Quality of life updates",
```

```

        "month_3": "Major content update",
        "month_6": "Expansion or major update"
    }

    def plan_updates(self):
        print("Post-Launch Update Schedule:")
        for timing, content in self.update_schedule.items():
            print(f"  {timing}: {content}")

updates = PostLaunchUpdates()
updates.plan_updates()

```

****Post-Launch Schedule:****

```

        POST-LAUNCH UPDATE SCHEDULE
DAY 1: Critical fixes, Day 1 patch
WEEK 1: Bug fixes, minor improvements
MONTH 1: Quality of life updates
MONTH 3: Major content update
MONTH 6: Expansion or major update
YEAR 1: Anniversary update
UPDATE TYPES:
  Bug Fixes: Address critical issues
  Balance: Gameplay adjustments
  Content: New features, items, levels
  Quality of Life: UI improvements, convenience
  Events: Seasonal content, limited time
COMMUNICATION:
  Detailed patch notes
  Community announcements
  Developer blogs

```

30.14 Steam Algorithm Explanation

```

class SteamAlgorithm:
    def __init__(self):
        self.factors = {
            "wishlists": "Number of wishlists",
            "reviews": "Review score and count",
            "engagement": "Community engagement",
            "sales": "Recent sales performance",
            "visibility": "Store page views"
        }

    def visibility_factors(self):
        return {
            "new_releases": "Boosted in first 2 weeks",
            "popular_upcoming": "Based on wishlists",
            "trending": "Based on recent sales",
            "discovery_queue": "Personalized recommendations"
        }

algorithm = SteamAlgorithm()
print("Steam Visibility Factors:")
for factor, desc in algorithm.visibility_factors().items():
    print(f"  {factor}: {desc}")

```

****Steam Algorithm:****

```

        STEAM ALGORITHM FACTORS
VISIBILITY BOOSTS:

```


New Releases: Boosted first 2 weeks
 Popular Upcoming: Based on wishlists
 Trending: Based on recent sales
 Discovery Queue: Personalized recommendations
 ALGORITHM FACTORS:
 Wishlists count
 Review score and count
 Community engagement
 Recent sales performance
 Store page views
 OPTIMIZATION:
 Build wishlists before launch
 Get positive reviews early
 Be active in community
 Regular updates
 Good store page optimization
 DISCOVERY QUEUE:
 Players browse recommended games
 Based on play history
 Tags and genre matching
 Wishlist activity

30.15 Steam-specific Optimization Tips

```

class SteamOptimization:
    def __init__(self):
        self.tips = {
            "capsule_art": "Eye-catching, readable at small sizes",
            "screenshots": "Show best moments first",
            "trailer": "Hook in first 5 seconds",
            "description": "First line should grab attention",
            "tags": "Research competitor tags",
            "pricing": "Consider regional pricing",
            "release_timing": "Avoid major releases",
            "demo": "Include during Steam Next Fest"
        }

    def optimize_page(self):
        print("Steam Page Optimization Tips:")
        for area, tip in self.tips.items():
            print(f" {area}: {tip}")

optimization = SteamOptimization()
optimization.optimize_page()
  
```

****Steam Optimization:****

STEAM OPTIMIZATION TIPS
 CAPSULE ART:
 High contrast colors
 Readable at small sizes
 Include game title
 Show key visual element
 SCREENSHOTS:
 Best moments first
 Diverse environments
 Show gameplay variety
 High resolution
 TRAILER:
 Hook in first 5 seconds

Show best gameplay
Keep it short (60-90 seconds)
End with call to action

DESCRIPTION:

First line should grab attention
Bullet points for features
Clear and concise
Include social links

RELEASE TIMING:

Avoid major game releases
Consider Steam sales schedule
Tuesday-Thursday recommended
Check competitor releases

daaayaaagaraaama (Diagrams)

Steam Launch Timeline

STEAM LAUNCH TIMELINE

12 MONTHS: Create Steamworks account, plan marketing
9 MONTHS: Create Steam page, start building wishlists
6 MONTHS: Release trailers, start press outreach
3 MONTHS: Release demo, Steam Next Fest
1 MONTH: Final trailer, review copies
LAUNCH DAY: Monitor, engage, respond
POST-LAUNCH: Updates, community, reviews

anaushailana (Exercises)

anaushailana 1: Steam Page Setup

- aapanaaara gaemaera janaya Steam page setup karauna:
- Capsule art daijaaaina karauna
 - Screenshots nairabaaachana karauna
 - Description laikhauna
 - Tags nairadhaaarana karauna

anaushailana 2: Pricing Strategy

- aapanaaara gaemaera janaya pricing strategy taairai karauna:
- Base price nairadhaaarana karauna
 - Regional pricing saeta karauna
 - Discount strategy paraikalapanaaa karauna

anaushailana 3: Launch Checklist

- Steam launch checklist taairai karauna:
- Pre-launch tasks

- Launch day tasks
 - Post-launch tasks
-

saaadhaaarana bhaula (Common Mistakes)

1. aparayaaapata Wishlists

```
# bhaula: 7,000 aira kama wishlists naiyae lacha
# sathaika: kamapakassae 7,000 wishlists aanauna
```

2. Poor Store Page

```
# bhaula: khaaaraaapa capsule art aiba screenshots
# sathaika: uchacha maaanaera assets taairai karauna
```

3. No Demo

```
# bhaula: daemao chhaaadaaaai lacha
# sathaika: Steam Next Fest-ai daemao daina
```

4. Ignoring Reviews

```
# bhaula: raibhaiu upaekassaaa karaaa
# sathaika: saba raibhaiu padauna aiba utatara daina
```

taipasa (Tips)

1. Build Wishlists Early

yata taaadaaataaadai samabhava wishlists taairai karauna

2. Polish Your Store Page

Store page kae saunadara aiba aakarassanaiiya karauna

3. Engage with Community

kamaiunaitaira saaathae sakaraiyabhaaabaе yaукata thaaakauna

4. Respond to Reviews

saba raibhaiu padauna aiba utatara daina

5. Regular Updates

naiyamaita aapadaeta daina

saaarasakassaepa (Summary)

Steamworks: ayaaakaaaunata taairai karauna
Store Page: samapauurana apataimaaaia karauna
Assets: uchacha maaanaera chhabai aiba bhaidaiu daina
Description: aakarassanaiya barananaaa laikhauna
Tags: sathaika tayaaaga baaachhaai karauna
Pricing: baudadhaimaanaii mauulaya nairadhaaarana
Demo: Steam Next Fest-ai daemao daina
Wishlists: kamapakassae 7,000 taaaragaeta
Reviews: saba raibhaiu padauna aiba utatara daina
Community: sakaraiyabhaaabaе yaukata thaaakauna
Updates: naiyamaita aapadaeta daina

GAME DEVELOPMENT MASTERBOOK - Chapter 30

PART 31

adhayaaaya 31: Android Game Launch

adhayaaaya 31: Android Game Launch

shaekhaara udadaeshaya (Learning Goal)

aai adhayaaayae aapanai shaikhabaena Google Play Store-ai gaema lacha karaaara samapauurana parakaraiyaaa, yaaara madhayae ASO (App Store Optimization), Ads Integration, Analytics, aiba Retention Strategies anatarabhaukata

baishada bayaaakhayaaa (Detailed Explanation)

31.1 Google Play Store Optimization (ASO)

```
class ASOOptimization:
    def __init__(self, app_name):
        self.app_name = app_name
        self.aso_factors = {
            "title": "App name (30 characters max)",
            "short_description": "80 characters max",
            "long_description": "4000 characters max",
            "icon": "512x512 pixels",
            "screenshots": "Min 2, Max 8",
            "video": "30 sec - 2 min"
        }

    def optimize_title(self, keywords):
        optimized = f"{self.app_name}: {' '.join(keywords[:3])}"
        if len(optimized) > 30:
            optimized = optimized[:30]
        return optimized

    def optimize_description(self, features):
        desc = f"Download {self.app_name} now!\n\n"
        desc += "Features:\n"
        for feature in features:
            desc += f"- {feature}\n"
        return desc

aso = ASOOptimization("Dragon Quest")
print(aso.optimize_title(["RPG", "Adventure", "Fantasy"]))

**ASO phayaaakatara:**
```

ASO OPTIMIZATION FACTORS

TITLE (30 characters):

Include primary keyword

Brand name first

Clear and memorable

SHORT DESCRIPTION (80 characters):

Hook in first line

Include key features

Call to action

LONG DESCRIPTION (4000 characters):

Keywords naturally integrated

Bullet points for features

Social proof

Regular updates mentioned

ICON (512x512):

Simple and recognizable

High contrast

No text (or minimal)

Test at small sizes

SCREENSHOTS:

Best screenshots first

Show key features

Include captions

Test different orders

VIDEO:

30 seconds - 2 minutes

Hook in first 5 seconds

Show gameplay

31.2 App Icon Design Tips

```
class AppIconDesign:
    def __init__(self):
        self.specs = {
            "size": "512x512 pixels",
            "format": "PNG",
            "background": "Transparent or solid"
        }
        self.best_practices = [
            "Simple and recognizable",
            "High contrast colors",
            "Minimal text",
            "Test at small sizes",
            "Follow platform guidelines"
        ]

    def design_checklist(self):
        print("App Icon Design Checklist:")
        for i, practice in enumerate(self.best_practices, 1):
            print(f" {i}. {practice}")

icon = AppIconDesign()
icon.design_checklist()
```

App Icon Design:

APP ICON DESIGN TIPS

SPECIFICATIONS:

Size: 512x512 pixels

Format: PNG

Background: Transparent or solid

BEST PRACTICES:

- Simple and recognizable
- High contrast colors
- Minimal text
- Test at small sizes (32x32, 48x48)
- Follow Material Design guidelines

COMMON MISTAKES:

- Too much detail
- Low contrast
- Text that's unreadable at small sizes
- Inconsistent with brand

TOOLS:

- Adobe Illustrator/Photoshop
- Figma
- Canva
- Android Asset Studio

31.3 Screenshot Guidelines

```
class ScreenshotGuidelines:
    def __init__(self):
        self.specs = {
            "min_screenshots": 2,
            "max_screenshots": 8,
            "recommended": 5,
            "size": "16:9 or 9:16 aspect ratio"
        }
        self.best_practices = [
            "Show best gameplay first",
            "Include captions",
            "Show diverse features",
            "High quality images",
            "Tell a story"
        ]

    def create_screenshot_set(self):
        return [
            "1. Main gameplay moment",
            "2. Key feature showcase",
            "3. Character/weapon showcase",
            "4. Environment/world",
            "5. Social/multiplayer features"
        ]

screenshots = ScreenshotGuidelines()
print("Recommended Screenshot Set:")
for item in screenshots.create_screenshot_set():
    print(f" {item}")
```

****Screenshot Guidelines:****

SCREENSHOT GUIDELINES

SPECIFICATIONS:

- Min: 2 screenshots
- Max: 8 screenshots
- Recommended: 5 screenshots
- Aspect ratio: 16:9 or 9:16

BEST PRACTICES:

- Show best gameplay first

```

Include captions
Show diverse features
High quality images
Tell a story
SCREENSHOT ORDER:
1. Main gameplay moment
2. Key feature showcase
3. Character/weapon showcase
4. Environment/world
5. Social/multiplayer features
TOOLS:
Screenshot tools (built-in)
Photoshop/Figma
Android Device Manager

```

31.4 Trailer Creation

```

class TrailerCreation:
    def __init__(self):
        self.specs = {
            "length": "30 seconds - 2 minutes",
            "resolution": "1080p minimum",
            "format": "MP4",
            "max_size": "100MB"
        }
        self.structure = {
            "hook": "First 5 seconds",
            "gameplay": "20-30 seconds",
            "features": "10-15 seconds",
            "call_to_action": "Last 5 seconds"
        }

    def create_trailer_script(self):
        return """
[0-5s] HOOK: Exciting gameplay moment
[5-25s] GAMEPLAY: Show best features
[25-35s] FEATURES: Highlight key selling points
[35-40s] CTA: "Download Now!"
        """

trailer = TrailerCreation()
print(trailer.create_trailer_script())

```

****Trailer Specs:****

```

                TRAILER CREATION SPECS
SPECIFICATIONS:
Length: 30 seconds - 2 minutes
Resolution: 1080p minimum
Format: MP4
Max size: 100MB
STRUCTURE:
0-5s: Hook (exciting moment)
5-25s: Gameplay showcase
25-35s: Feature highlights
35-40s: Call to action
BEST PRACTICES:
Start with action
Show variety
Keep it short

```


Good music
 End with download prompt
 TOOLS:
 OBS Studio
 Adobe Premiere
 DaVinci Resolve
 Mobile screen recorders

31.5 Description Writing

```
class DescriptionWriting:
    def __init__(self, app_name):
        self.app_name = app_name

    def write_short_description(self, keywords):
        return f"Download {self.app_name} - {keywords[0]} game with {keywords[1]} features"

    def write_long_description(self, features, story):
        desc = f"## About {self.app_name}\n\n{story}\n\n"
        desc += "## Features\n\n"
        for feature in features:
            desc += f"- {feature}\n"
        desc += "\n## What's New\n\nRegular updates with new content!\n"
        return desc

desc = DescriptionWriting("Dragon Quest")
print(desc.write_short_description(["RPG", "Adventure"]))
```

****Description Template:****

```
SHORT DESCRIPTION (80 chars):
[Game Name] - [Genre] game with [Key Feature]!

LONG DESCRIPTION:
## About [Game Name]
[1-2 paragraphs about the game]

## Features
- Feature 1
- Feature 2
- Feature 3

## What's New
Regular updates with new content!

## Contact
[Social links]
```

31.6 Keyword Research

```
class KeywordResearch:
    def __init__(self):
        self.tools = [
            "Google Keyword Planner",
            "App Annie",
            "Sensor Tower",
            "AppTweak",
            "Mobile Action"
        ]
```

```

        self.factors = {
            "relevance": "How related to your game",
            "volume": "Search volume",
            "difficulty": "Competition level"
        }

    def research_keywords(self, game_genre):
        print(f"Researching keywords for {game_genre}:")
        print("1. Check competitor keywords")
        print("2. Use keyword tools")
        print("3. Analyze search volume")
        print("4. Select top 5-10 keywords")

research = KeywordResearch()
research.research_keywords("RPG")

```

****Keyword Research:****

```

                                KEYWORD RESEARCH
TOOLS:
Google Keyword Planner
App Annie
Sensor Tower
AppTweak
Mobile Action
FACTORS:
Relevance: How related to your game
Volume: Search volume
Difficulty: Competition level
PROCESS:
1. Check competitor keywords
2. Use keyword tools
3. Analyze search volume
4. Select top 5-10 keywords
5. Integrate naturally
KEYWORD PLACEMENT:
Title (most important)
Short description
Long description
Developer name

```

31.7 Pricing Strategy

```

class AndroidPricing:
    def __init__(self):
        self.price_points = {
            "free": 0,
            "budget": 0.99,
            "mid": 2.99,
            "premium": 4.99,
            "high": 9.99
        }
        self.iap_types = {
            "consumable": "Coins, gems, energy",
            "non_consumable": "Remove ads, extra content",
            "subscription": "Monthly passes"
        }

    def recommend_strategy(self, game_type):
        if game_type == "casual":

```

```

        return "Free with ads + IAP"
    elif game_type == "midcore":
        return "Free with IAP"
    elif game_type == "hardcore":
        return "Premium or Free with IAP"

pricing = AndroidPricing()
print(pricing.recommend_strategy("casual"))

```

****Pricing Strategy:****

```

        ANDROID PRICING STRATEGY

PRICE POINTS:
Free: $0 (most common)
Budget: $0.99
Mid: $2.99
Premium: $4.99
High: $9.99
STRATEGY BY GAME TYPE:
Casual: Free with ads + IAP
Midcore: Free with IAP
Hardcore: Premium or Free with IAP
Indie: Premium or Free with IAP
IAP TYPES:
Consumable: Coins, gems, energy
Non-consumable: Remove ads, extra content
Subscription: Monthly passes
TIPS:
Test different price points
Monitor conversion rates
Consider regional pricing
Balance free vs paid content

```

31.8 Ads Integration (AdMob)

```

class AdMobIntegration:
    def __init__(self, app_id):
        self.app_id = app_id
        self.ad_types = {
            "banner": {"position": "Bottom/Top", "size": "320x50"},
            "interstitial": {"frequency": "Every 3-5 actions"},
            "rewarded": {"reward": "Coins/lives", "frequency": "Player choice"},
            "native": {"integration": "In-feed"}
        }

    def setup_ads(self):
        print(f"Setting up AdMob for {self.app_id}")
        for ad_type, config in self.ad_types.items():
            print(f"  {ad_type}: {config}")

    def show_rewarded_ad(self, callback):
        print("Showing rewarded ad...")
        # On completion
        callback()

admob = AdMobIntegration("com.example.game")
admob.setup_ads()

```

****AdMob Integration:****

```

        ADMOB INTEGRATION

```

AD TYPES:

Banner: 320x50, always visible
 Interstitial: Full screen, every 3-5 actions
 Rewarded: Player opts in, gets reward
 Native: In-feed, less intrusive

SETUP STEPS:

1. Create AdMob account
2. Add app to AdMob
3. Create ad units
4. Integrate SDK
5. Test ads
6. Publish

BEST PRACTICES:

Use rewarded ads primarily
 Don't show too many ads
 Test ad placement
 Monitor eCPM
 Optimize for fill rate

COMMON ISSUES:

Low fill rate
 Low eCPM
 Ad fatigue
 User complaints

31.9 Analytics Setup (Firebase)

```
class FirebaseAnalytics:
    def __init__(self, project_id):
        self.project_id = project_id
        self.events = {
            "app_open": "User opens app",
            "level_complete": "User completes level",
            "purchase": "User makes purchase",
            "session_start": "User starts session",
            "session_end": "User ends session"
        }

    def setup(self):
        print(f"Setting up Firebase Analytics for {self.project_id}")
        print("Events to track:")
        for event, desc in self.events.items():
            print(f"  {event}: {desc}")

    def log_event(self, event_name, params=None):
        print(f"Logging event: {event_name}")
        if params:
            print(f"  Parameters: {params}")

firebase = FirebaseAnalytics("my-game-project")
firebase.setup()
firebase.log_event("level_complete", {"level": 5, "score": 1000})
```

****Firebase Analytics:****

FIREBASE ANALYTICS SETUP

SETUP STEPS:

1. Create Firebase project
2. Add Android app
3. Download google-services.json
4. Add SDK to project

```

5. Initialize Firebase
6. Log events
KEY EVENTS:
app_open: User opens app
level_complete: User completes level
purchase: User makes purchase
session_start: User starts session
session_end: User ends session
custom_events: Your own events
KEY METRICS:
DAU/MAU: Daily/Monthly Active Users
Session length: How long users play
Retention: How many users return
Conversion: Purchase rate
Revenue: Total earnings
DASHBOARD:
Real-time users
Event tracking
Audience demographics
Acquisition channels

```

31.10 Retention Strategies

```

class RetentionStrategies:
    def __init__(self):
        self.strategies = {
            "daily_rewards": "Login bonuses",
            "push_notifications": "Reminders",
            "events": "Limited time events",
            "social": "Friend invites",
            "progression": "Clear goals"
        }

    def calculate_retention(self, day1, day7, day30):
        print(f"Day 1 Retention: {day1}%")
        print(f"Day 7 Retention: {day7}%")
        print(f"Day 30 Retention: {day30}%")
        if day7 > 20:
            print("Good retention!")
        else:
            print("Needs improvement")

retention = RetentionStrategies()
retention.calculate_retention(40, 25, 15)

```

****Retention Strategies:****

```

RETENTION STRATEGIES
STRATEGIES:
Daily Rewards: Login bonuses
Push Notifications: Reminders
Events: Limited time content
Social: Friend invites, leaderboards
Progression: Clear goals, milestones
RETENTION TARGETS:
Day 1: 40%+ (Good)
Day 7: 20%+ (Good)
Day 30: 10%+ (Good)
IMPROVING RETENTION:
Onboarding tutorial

```

- Regular content updates
- Community features
- Personalized experience
- Bug fixes and optimization

ANALYTICS:

- Track where users drop off
- A/B test different strategies
- Monitor session length
- Analyze user behavior

31.11 Crash Reporting (Crashlytics)

```
class CrashlyticsSetup:
    def __init__(self):
        self.crash_types = {
            "fatal": "App crashes",
            "non_fatal": "Errors that don't crash",
            "anr": "Application Not Responding"
        }

    def setup(self):
        print("Setting up Crashlytics:")
        print("1. Add Firebase SDK")
        print("2. Initialize Crashlytics")
        print("3. Test crash reporting")
        print("4. Monitor dashboard")

    def log_error(self, error_type, message):
        print(f"Logging {error_type}: {message}")

crashlytics = CrashlyticsSetup()
crashlytics.setup()
crashlytics.log_error("fatal", "Null pointer exception")
```

Crashlytics Setup:

```
CRASHLYTICS SETUP

SETUP STEPS:
1. Add Firebase SDK
2. Initialize Crashlytics
3. Test crash reporting
4. Monitor dashboard

CRASH TYPES:
Fatal: App crashes
Non-fatal: Errors that don't crash
ANR: Application Not Responding

MONITORING:
Crash rate
Crash-free users
Device/OS breakdown
Stack traces

BEST PRACTICES:
Fix crashes quickly
Monitor trends
Test on multiple devices
Use version tracking
```

31.12 Update Strategy

```

class UpdateStrategy:
    def __init__(self):
        self.update_schedule = {
            "week_1": "Bug fixes, performance",
            "month_1": "Content updates",
            "month_3": "Major features",
            "quarterly": "Large updates"
        }

    def plan_updates(self):
        print("Update Schedule:")
        for timing, content in self.update_schedule.items():
            print(f" {timing}: {content}")

updates = UpdateStrategy()
updates.plan_updates()

```

****Update Strategy:****

```

UPDATE STRATEGY
SCHEDULE:
Week 1: Bug fixes, performance
Month 1: Content updates
Month 3: Major features
Quarterly: Large updates
UPDATE TYPES:
Bug fixes: Critical issues
Performance: Optimization
Content: New levels, items
Features: New mechanics
Events: Limited time content
COMMUNICATION:
Detailed changelogs
Store page updates
Social media announcements
In-game messages
BEST PRACTICES:
Regular schedule
Test before release
Monitor after release
Respond to feedback

```

31.13 Android-specific Optimization

```

class AndroidOptimization:
    def __init__(self):
        self.optimizations = {
            "apk_size": "Keep under 100MB",
            "memory": "Optimize RAM usage",
            "battery": "Minimize battery drain",
            "compatibility": "Support Android 5.0+",
            "performance": "Target 60fps"
        }

    def optimize(self):
        print("Android Optimization Checklist:")
        for area, tip in self.optimizations.items():
            print(f" {area}: {tip}")

android = AndroidOptimization()

```

```
android.optimize()
```

****Android Optimization:****

ANDROID OPTIMIZATION

APK SIZE:

- Keep under 100MB (Google Play limit 150MB)
- Use AAB (Android App Bundle)
- Compress textures
- Remove unused resources

MEMORY:

- Optimize RAM usage
- Use object pooling
- Minimize allocations
- Profile memory usage

BATTERY:

- Minimize background processing
- Optimize GPS usage
- Reduce network calls
- Use efficient algorithms

COMPATIBILITY:

- Support Android 5.0+ (API 21)
- Test on multiple screen sizes
- Handle different orientations
- Test on various devices

PERFORMANCE:

- Target 60fps
- Optimize draw calls
- Use efficient shaders
- Profile regularly

31.14 Google Play Console Walkthrough

```
class GooglePlayConsole:
    def __init__(self):
        self.sections = {
            "dashboard": "Overview of app performance",
            "production": "Release management",
            "store_listing": "App store page",
            "pricing": "Price and distribution",
            "acquisition": "User acquisition reports",
            "engagement": "User engagement",
            "monetization": "Revenue reports",
            "quality": "Crash and ANR reports"
        }

    def walkthrough(self):
        print("Google Play Console Walkthrough:")
        for section, desc in self.sections.items():
            print(f" {section}: {desc}")

console = GooglePlayConsole()
console.walkthrough()
```

****Google Play Console:****

GOOGLE PLAY CONSOLE WALKTHROUGH

SECTIONS:

- Dashboard: Overview of performance
- Production: Release management

Store Listing: App store page
Pricing: Price and distribution
Acquisition: User acquisition reports
Engagement: User engagement
Monetization: Revenue reports
Quality: Crash and ANR reports

KEY FEATURES:
A/B testing
Staged rollouts
Pre-registration
Store listing experiments
User feedback

REPORTS:
Installs and uninstalls
Active users
Revenue
Ratings and reviews
Crash reports

daaayaaagaraama (Diagrams)

Android Launch Timeline

ANDROID LAUNCH TIMELINE

PRE-DEVELOPMENT:
Market research
Competitor analysis
ASO keyword research
DEVELOPMENT:
Build game
Integrate analytics
Integrate ads
Set up Crashlytics
PRE-LAUNCH:
Create store listing
Design icon and screenshots
Create trailer
Test on multiple devices
LAUNCH:
Publish to Play Store
Monitor performance
Respond to reviews
Fix critical bugs
POST-LAUNCH:
Regular updates
Community engagement
Monitor analytics
Optimize monetization

anaushailana (Exercises)

anaushailana 1: ASO Optimization

aapanaaara gaemaera janaya ASO optimization karauna:

- Title optimization
- Description writing
- Keyword research

anaushailana 2: Store Listing

Google Play Store listing taairai karauna:

- Icon design
- Screenshots
- Trailer

anaushailana 3: Analytics Setup

Firebase Analytics saeta aapa karauna:

- Project creation
- Event tracking
- Dashboard setup

saaadhaaarana bhaula (Common Mistakes)

1. Large APK Size

- # bhaula: 150MB+ APK
- # sathaika: 100MB aira kama raaakhauna

2. No Analytics

- # bhaula: Analytics chhaaadaaaai lacha
- # sathaika: Firebase Analytics saeta aapa karauna

3. Poor Store Listing

- # bhaula: khaaaraaapa icon aiba screenshots
- # sathaika: uchacha maaanaera assets taairai karauna

4. No Crash Reporting

- # bhaula: Crashlytics chhaaadaaaai lacha
- # sathaika: Crashlytics saeta aapa karauna

taipasa (Tips)

1. Optimize APK Size

APK saaaija yata chhaota habae tata bhaaalao

2. Test on Multiple Devices

baibhainana daibhaaaisae taesata karauna

3. Monitor Analytics

Analytics data naiyamaita parayabaekassana karauna

4. Respond to Reviews

saba raibhaiu padauna aiba utatara daina

5. Regular Updates

naiyamaita aapadaeta daina

saaarasakassaepa (Summary)

ASO: Title, Description, Keywords apataimaaaija karauna
Icon: 512x512, Simple, Recognizable
Screenshots: 5-8, High Quality, Best First
Trailer: 30 sec - 2 min, Hook First
Keywords: Research and Integrate
Pricing: Free with IAP or Premium
Ads: AdMob Integration
Analytics: Firebase Analytics
Crash Reporting: Crashlytics
Retention: Daily Rewards, Push Notifications
Updates: Regular Schedule
Optimization: APK Size, Memory, Battery
Play Console: Monitor Performance

GAME DEVELOPMENT MASTERBOOK - Chapter 31

PART 32

adhayaaaya 32: gaema baijanaesa (GAME BUSINESS)

adhayaaaya 32: gaema baijanaesa (GAME BUSINESS)

shaekhaara udadaeshaya (Learning Goal)

aii adhayaaayae aapanai shaikhabaena gaema daebhaelapamaenataera bayabasaaayaika daika, yaaara madhayae Budget Planning, Cost Estimation, Revenue Projections, ROI Calculation aiba Business Structure anatarabhaukata

baishada bayaaakhayaaa (Detailed Explanation)

32.1 Development Budget Planning

```
class DevelopmentBudget:
    def __init__(self, game_name):
        self.game_name = game_name
        self.categories = {
            "personnel": 0,
            "software": 0,
            "hardware": 0,
            "marketing": 0,
            "outsourcing": 0,
            "contingency": 0
        }

    def add_cost(self, category, amount):
        if category in self.categories:
            self.categories[category] += amount

    def get_total(self):
        return sum(self.categories.values())

    def get_breakdown(self):
        total = self.get_total()
        print(f"\n{self.game_name} Development Budget")
        print("=" * 40)
        for category, amount in self.categories.items():
            percentage = (amount / total * 100) if total > 0 else 0
            print(f"{category.capitalize():20} ${amount:>10,.2f} ({percentage:.1f}%)")
        print("-" * 40)
```

```
print(f"{'TOTAL':20} ${total:>10,.2f}")

budget = DevelopmentBudget("Indie RPG")
budget.add_cost("personnel", 50000)
budget.add_cost("software", 5000)
budget.add_cost("hardware", 3000)
budget.add_cost("marketing", 10000)
budget.add_cost("outsourcing", 8000)
budget.add_cost("contingency", 5000)
budget.get_breakdown()
```

****baajaeta kaaathaaamao:****

DEVELOPMENT BUDGET BREAKDOWN		
CATEGORY	AMOUNT	PERCENTAGE
Personnel	\$50,000	62.5%
Software	\$5,000	6.3%
Hardware	\$3,000	3.8%
Marketing	\$10,000	12.5%
Outsourcing	\$8,000	10.0%
Contingency	\$5,000	6.3%
TOTAL	\$81,000	100%

32.2 Marketing Budget Planning

```
class MarketingBudget:
    def __init__(self):
        self.channels = {
            "social_media_ads": 0,
            "influencer_marketing": 0,
            "press_outreach": 0,
            "events": 0,
            "content_creation": 0,
            "community_management": 0
        }

    def add_cost(self, channel, amount):
        if channel in self.channels:
            self.channels[channel] += amount

    def get_total(self):
        return sum(self.channels.values())

    def calculate_cpi(self, total_cost, installs):
        """Cost Per Install"""
        return total_cost / installs if installs > 0 else 0

    def calculate_roas(self, revenue, ad_spend):
        """Return on Ad Spend"""
        return revenue / ad_spend if ad_spend > 0 else 0

marketing = MarketingBudget()
marketing.add_cost("social_media_ads", 3000)
marketing.add_cost("influencer_marketing", 5000)
marketing.add_cost("press_outreach", 1000)
marketing.add_cost("events", 2000)
marketing.add_cost("content_creation", 1500)
marketing.add_cost("community_management", 1000)

total = marketing.get_total()
print(f"Total Marketing Budget: ${total}"))
```

```
print(f"CPI (if 10,000 installs): ${marketing.calculate_cpi(total, 10000):.2f}")
print(f"ROAS (if $50,000 revenue): {marketing.calculate_roas(50000, total):.2f}x")
```

****Marketing Budget:****

MARKETING BUDGET BREAKDOWN		
CHANNEL	AMOUNT	% OF TOTAL
Social Media Ads	\$3,000	25.0%
Influencer Marketing	\$5,000	41.7%
Press Outreach	\$1,000	8.3%
Events	\$2,000	16.7%
Content Creation	\$1,500	12.5%
Community Management	\$1,000	8.3%
TOTAL	\$13,500	100%
KEY METRICS:		
CPI (10K installs): \$1.35		
ROAS (\$50K revenue): 3.7x		

32.3 Asset Cost Estimation

```
class AssetCostEstimation:
    def __init__(self):
        self.asset_costs = {
            "2d_art": {"per_hour": 50, "hours": 200},
            "3d_modeling": {"per_hour": 75, "hours": 300},
            "animation": {"per_hour": 60, "hours": 150},
            "sound_effects": {"per_hour": 40, "hours": 50},
            "music": {"per_hour": 80, "hours": 100},
            "voice_acting": {"per_hour": 100, "hours": 30}
        }

    def calculate_costs(self):
        print("Asset Cost Estimation:")
        print("-" * 50)
        total = 0
        for asset, data in self.asset_costs.items():
            cost = data["per_hour"] * data["hours"]
            total += cost
            print(f"{asset:20} ${data['per_hour']}/hr x {data['hours']}hrs = ${cost:,}")
        print("-" * 50)
        print(f"{'TOTAL':20} ${total:,}")
        return total

assets = AssetCostEstimation()
total = assets.calculate_costs()
```

****Asset Cost saaraanaii:****

ASSET COST ESTIMATION			
ASSET TYPE	RATE/HR	HOURS	TOTAL COST
2D Art	\$50	200	\$10,000
3D Modeling	\$75	300	\$22,500
Animation	\$60	150	\$9,000
Sound Effects	\$40	50	\$2,000
Music	\$80	100	\$8,000
Voice Acting	\$100	30	\$3,000
TOTAL		830	\$54,500
NOTE: Rates vary by region and experience			

32.4 Software Cost (Engine, Tools, Licenses)

```
class SoftwareCosts:
    def __init__(self):
        self.software = {
            "unity": {"license": "Free (Personal)", "cost": 0},
            "unreal": {"license": "Free (until $1M revenue)", "cost": 0},
            "godot": {"license": "Free (Open Source)", "cost": 0},
            "visual_studio": {"license": "Community (Free)", "cost": 0},
            "blender": {"license": "Free (Open Source)", "cost": 0},
            "photoshop": {"license": "Subscription", "cost": 240},
            "maya": {"license": "Subscription", "cost": 1785},
            "fmod": {"license": "Free (Indie)", "cost": 0},
            "wwise": {"license": "Free (Indie)", "cost": 0}
        }

    def calculate_annual_costs(self):
        print("Software Costs (Annual):")
        print("-" * 50)
        total = 0
        for software, data in self.software.items():
            total += data["cost"]
            print(f"{software:20} {data['license']:30} ${data['cost']:,}")
        print("-" * 50)
        print(f"{'TOTAL':20} {'':30} ${total:,}")
        return total

software = SoftwareCosts()
total = software.calculate_annual_costs()
```

****Software Costs:****

SOFTWARE COSTS		
SOFTWARE	LICENSE	ANNUAL COST
Unity	Personal (Free)	\$0
Unreal Engine	Free (until \$1M)	\$0
Godot	Open Source	\$0
Visual Studio	Community (Free)	\$0
Blender	Open Source	\$0
Photoshop	Subscription	\$240
Maya	Subscription	\$1,785
FMOD	Indie (Free)	\$0
Wwise	Indie (Free)	\$0
TOTAL		\$2,025

NOTE: Free tools available for most tasks

32.5 Team Cost (Salary, Contract)

```
class TeamCosts:
    def __init__(self):
        self.roles = {
            "game_designer": {"hourly_rate": 50, "hours_per_week": 40},
            "programmer": {"hourly_rate": 60, "hours_per_week": 40},
            "artist_2d": {"hourly_rate": 45, "hours_per_week": 40},
            "artist_3d": {"hourly_rate": 55, "hours_per_week": 40},
            "animator": {"hourly_rate": 50, "hours_per_week": 40},
            "sound_designer": {"hourly_rate": 40, "hours_per_week": 20},
            "writer": {"hourly_rate": 35, "hours_per_week": 20},
            "qa_tester": {"hourly_rate": 25, "hours_per_week": 40}
```

```

    }
    self.duration_weeks = 52 # 1 year

def calculate_team_cost(self):
    print("Team Cost (1 Year):")
    print("-" * 60)
    total = 0
    for role, data in self.roles.items():
        annual_cost = data["hourly_rate"] * data["hours_per_week"] * self.duration_weeks
        total += annual_cost
        print(f"{role:20} ${data['hourly_rate']}/hr x {data['hours_per_week']}hrs x {self.duration_weeks}s")
    print("-" * 60)
    print(f"{'TOTAL':20} {'':35} ${total:>10,}")
    return total

team = TeamCosts()
total = team.calculate_team_cost()

```

****Team Cost saaranai:****

TEAM COST (1 YEAR)			
ROLE	RATE	HRS/WK	TOTAL ANNUAL
Game Designer	\$50	40	\$104,000
Programmer	\$60	40	\$124,800
2D Artist	\$45	40	\$93,600
3D Artist	\$55	40	\$114,400
Animator	\$50	40	\$104,000
Sound Designer	\$40	20	\$41,600
Writer	\$35	20	\$36,400
QA Tester	\$25	40	\$52,000
TOTAL			\$670,800

NOTE: Rates vary by location and experience

32.6 Revenue Projections

```

class RevenueProjection:
    def __init__(self):
        self.scenarios = {
            "conservative": {"copies": 10000, "price": 19.99},
            "moderate": {"copies": 50000, "price": 19.99},
            "optimistic": {"copies": 200000, "price": 19.99}
        }
        self.platform_fee = 0.30 # 30%

    def calculate_revenue(self, scenario):
        data = self.scenarios[scenario]
        gross = data["copies"] * data["price"]
        fee = gross * self.platform_fee
        net = gross - fee
        return {"gross": gross, "fee": fee, "net": net}

    def print_projections(self):
        print("Revenue Projections:")
        print("-" * 60)
        for scenario in self.scenarios:
            rev = self.calculate_revenue(scenario)
            copies = self.scenarios[scenario]["copies"]
            print(f"{scenario.capitalize():15} {copies:>8,} copies x ${self.scenarios[scenario]['price']:.2f}")
            print(f"{'':15} Gross: ${rev['gross']:>10,.2f}")
            print(f"{'':15} Fee:   ${rev['fee']:>10,.2f}")

```



```
print(f"{'':15} Net:    ${rev['net']:>10,.2f}")
print()
```

```
revenue = RevenueProjection()
revenue.print_projections()
```

****Revenue Projections:****

REVENUE PROJECTIONS				
SCENARIO	COPIES	GROSS	FEE	NET
Conservative	10,000	\$199,900	\$59,970	\$139,930
Moderate	50,000	\$999,500	\$299,850	\$699,650
Optimistic	200,000	\$3,998,000	\$1,199,400	\$2,798,600

NOTE: Platform fee is 30% (Steam, Google Play, App Store)

32.7 Profit Calculation

```
class ProfitCalculation:
    def __init__(self):
        self.revenue = 0
        self.costs = {
            "development": 0,
            "marketing": 0,
            "operations": 0,
            "taxes": 0
        }

    def set_revenue(self, amount):
        self.revenue = amount

    def add_cost(self, category, amount):
        if category in self.costs:
            self.costs[category] += amount

    def get_total_costs(self):
        return sum(self.costs.values())

    def calculate_profit(self):
        return self.revenue - self.get_total_costs()

    def calculate_profit_margin(self):
        if self.revenue == 0:
            return 0
        return (self.calculate_profit() / self.revenue) * 100

    def print_profit_statement(self):
        print("Profit Statement:")
        print("-" * 40)
        print(f"{'Revenue':20} ${self.revenue:>10,.2f}")
        print("-" * 40)
        for category, amount in self.costs.items():
            print(f"{category.capitalize():20} ${amount:>10,.2f}")
        print("-" * 40)
        print(f"{'Total Costs':20} ${self.get_total_costs():>10,.2f}")
        print("-" * 40)
        print(f"{'Net Profit':20} ${self.calculate_profit():>10,.2f}")
        print(f"{'Profit Margin':20} {self.calculate_profit_margin():>9.1f}%")

profit = ProfitCalculation()
profit.set_revenue(699650) # Moderate scenario
profit.add_cost("development", 81000)
```

```
profit.add_cost("marketing", 13500)
profit.add_cost("operations", 5000)
profit.add_cost("taxes", 100000)
profit.print_profit_statement()
```

****Profit Statement:****

PROFIT STATEMENT	
Revenue	\$699,650.00
Development	\$81,000.00
Marketing	\$13,500.00
Operations	\$5,000.00
Taxes	\$100,000.00
Total Costs	\$199,500.00
Net Profit	\$500,150.00
Profit Margin	71.5%

32.8 Break-even Analysis

```
class BreakEvenAnalysis:
    def __init__(self):
        self.fixed_costs = 0
        self.variable_cost_per_unit = 0
        self.price_per_unit = 0

    def set_parameters(self, fixed, variable, price):
        self.fixed_costs = fixed
        self.variable_cost_per_unit = variable
        self.price_per_unit = price

    def calculate_break_even(self):
        contribution_margin = self.price_per_unit - self.variable_cost_per_unit
        if contribution_margin <= 0:
            return float('inf')
        return self.fixed_costs / contribution_margin

    def calculate_break_even_revenue(self):
        units = self.calculate_break_even()
        return units * self.price_per_unit

analysis = BreakEvenAnalysis()
analysis.set_parameters(fixed=81000, variable=0, price=19.99)
units = analysis.calculate_break_even()
revenue = analysis.calculate_break_even_revenue()
print(f"Break-even Point: {units:,.0f} copies")
print(f"Break-even Revenue: ${revenue:,.2f}")
```

****Break-even Analysis:****

BREAK-EVEN ANALYSIS

FORMULA:

$$\text{Break-even Units} = \frac{\text{Fixed Costs}}{(\text{Price} - \text{Variable Cost})}$$

EXAMPLE:

Fixed Costs: \$81,000
 Price: \$19.99
 Variable Cost: \$0
 Break-even: 4,052 copies

BREAK-EVEN CHART:

Revenue

Profit Zone

> Units

0 4,052
Break-even Point

32.9 ROI (Return on Investment) Calculation

```
class ROICalculation:
    def __init__(self):
        self.investment = 0
        self.return_amount = 0

    def set_investment(self, amount):
        self.investment = amount

    def set_return(self, amount):
        self.return_amount = amount

    def calculate_roi(self):
        if self.investment == 0:
            return 0
        return ((self.return_amount - self.investment) / self.investment) * 100

    def calculate_payback_period(self, monthly_revenue):
        if monthly_revenue <= 0:
            return float('inf')
        return self.investment / monthly_revenue

roi = ROICalculation()
roi.set_investment(81000)
roi.set_return(699650)
roi_value = roi.calculate_roi()
payback = roi.calculate_payback_period(58304) # Monthly revenue
print(f"ROI: {roi_value:.1f}%")
print(f"Payback Period: {payback:.1f} months")
```

****ROI Calculation:****

ROI CALCULATION

FORMULA:

$ROI = ((Return - Investment) / Investment) \times 100$

EXAMPLE:

Investment: \$81,000

Return: \$699,650

ROI: 763.8%

Payback Period: 1.4 months

ROI INTERPRETATION:

>100%: Excellent

50-100%: Good

0-50%: Moderate

<0%: Loss

32.10 Sample Indie Game Budget

```
class SampleIndieBudget:
    def __init__(self):
        self.budget = {
            "development": {
                "salaries": 50000,
                "freelance": 15000,
```

```

        "software": 2000,
        "hardware": 3000,
        "total": 70000
    },
    "marketing": {
        "ads": 5000,
        "influencer": 3000,
        "press_kit": 500,
        "events": 2000,
        "total": 10500
    },
    "operations": {
        "hosting": 1200,
        "support": 3000,
        "legal": 2000,
        "total": 6200
    },
    "contingency": 5000
}

```

```

def print_budget(self):
    print("Sample Indie Game Budget")
    print("=" * 60)
    grand_total = 0
    for category, items in self.budget.items():
        print(f"\n{category.upper()}:")
        if isinstance(items, dict):
            for item, amount in items.items():
                if item != "total":
                    print(f"    {item:20} ${amount:>8,}")
                print(f"    {'Subtotal':20} ${items['total']:>8,}")
            grand_total += items['total']
        else:
            print(f"    {category:20} ${items:>8,}")
            grand_total += items
    print("\n" + "=" * 60)
    print(f"{'GRAND TOTAL':20} ${grand_total:>8,}")

```

```

budget = SampleIndieBudget()
budget.print_budget()

```

****Sample Indie Budget:****

SAMPLE INDIE GAME BUDGET	
DEVELOPMENT:	
Salaries	\$50,000
Freelance	\$15,000
Software	\$2,000
Hardware	\$3,000
Subtotal	\$70,000
MARKETING:	
Ads	\$5,000
Influencer	\$3,000
Press Kit	\$500
Events	\$2,000
Subtotal	\$10,500
OPERATIONS:	
Hosting	\$1,200
Support	\$3,000
Legal	\$2,000
Subtotal	\$6,200
CONTINGENCY:	\$5,000

GRAND TOTAL: \$91,700

32.11 Financial Planning Templates

```
class FinancialTemplates:
    def __init__(self):
        self.templates = {
            "monthly_budget": "Track monthly expenses",
            "cash_flow": "Monitor income/expenses",
            "pnl": "Profit and Loss statement",
            "balance_sheet": "Assets vs Liabilities"
        }

    def create_monthly_budget(self, month, income, expenses):
        print(f"\n{month} Budget:")
        print("-" * 40)
        print(f"{'Income':20} ${income:>10,.2f}")
        for category, amount in expenses.items():
            print(f"{'category':20} ${amount:>10,.2f}")
        total_expenses = sum(expenses.values())
        print("-" * 40)
        print(f"{'Total Expenses':20} ${total_expenses:>10,.2f}")
        print(f"{'Net':20} ${income - total_expenses:>10,.2f}")

templates = FinancialTemplates()
templates.create_monthly_budget(
    "January 2024",
    50000,
    {"development": 30000, "marketing": 5000, "operations": 2000}
)
```

****Financial Templates:****

- FINANCIAL PLANNING TEMPLATES
1. MONTHLY BUDGET:

Track income vs expenses

Category breakdown

Variance analysis

2. CASH FLOW STATEMENT:

Operating activities

Investing activities

Financing activities

3. PROFIT & LOSS (P&L):

Revenue

Cost of Goods Sold

Gross Profit

Operating Expenses

Net Profit

4. BALANCE SHEET:

Assets (what you own)

Liabilities (what you owe)

Equity (net worth)

32.12 Tax Considerations

```
class TaxConsiderations:
    def __init__(self):
        self.tax_items = {
```

```

        "income_tax": "Tax on business income",
        "sales_tax": "Tax on game sales",
        "vat": "Value Added Tax (EU)",
        "self_employment": "Self-employment tax",
        "deductions": "Business expense deductions"
    }

    def calculate_estimated_tax(self, income, tax_rate):
        return income * tax_rate

    def print_tax_info(self):
        print("Tax Considerations:")
        for item, desc in self.tax_items.items():
            print(f" {item}: {desc}")

tax = TaxConsiderations()
tax.print_tax_info()
estimated = tax.calculate_estimated_tax(100000, 0.25)
print(f"\nEstimated Tax (25% rate): ${estimated:,.2f}")

```

****Tax Considerations:****

```

        TAX CONSIDERATIONS

TAX TYPES:
Income Tax: Tax on business income
Sales Tax: Tax on game sales (US)
VAT: Value Added Tax (EU)
Self-Employment: Self-employment tax
Deductions: Business expense deductions

COMMON DEDUCTIONS:
Home office
Equipment purchases
Software licenses
Marketing expenses
Travel expenses
Professional services

TIPS:
Keep detailed records
Separate personal/business accounts
Consult a tax professional
File quarterly estimated taxes
Track all expenses

```

32.13 Business Structure Options

```

class BusinessStructure:
    def __init__(self):
        self.structures = {
            "sole_proprietorship": {
                "description": "Single owner, simplest form",
                "pros": ["Easy to set up", "Full control", "Simple taxes"],
                "cons": ["Unlimited liability", "Hard to raise capital"]
            },
            "llc": {
                "description": "Limited Liability Company",
                "pros": ["Limited liability", "Flexible taxes", "Credibility"],
                "cons": ["More paperwork", "State fees"]
            },
            "corporation": {
                "description": "Separate legal entity",

```

```
        "pros": ["Limited liability", "Easy to raise capital", "Perpetual existenc
        "cons": ["Complex setup", "Double taxation", "More regulations"]
    }
}

def print_comparison(self):
    print("Business Structure Comparison:")
    print("=" * 60)
    for structure, data in self.structures.items():
        print(f"\n{structure.upper()}:")
        print(f"  {data['description']}")
        print(f"  Pros: {' '.join(data['pros'])}")
        print(f"  Cons: {' '.join(data['cons'])}")

biz = BusinessStructure()
biz.print_comparison()
```

****Business Structure:****

BUSINESS STRUCTURE OPTIONS

SOLE PROPRIETORSHIP:

Description: Single owner, simplest form

Pros: Easy setup, full control, simple taxes

Cons: Unlimited liability, hard to raise capital

LLC (Limited Liability Company):

Description: Separate legal entity

Pros: Limited liability, flexible taxes, credibility

Cons: More paperwork, state fees

CORPORATION:

Description: Separate legal entity, shareholders

Pros: Limited liability, easy to raise capital

Cons: Complex setup, double taxation

RECOMMENDATION FOR INDIES:

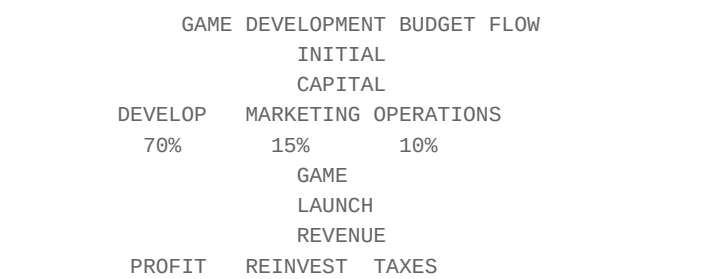
Start as Sole Proprietorship

Form LLC when revenue increases

Consult a lawyer for specific advice

daaayaaagaraaama (Diagrams)

Game Development Budget Flow



anaushailana (Exercises)

anaushailana 1: Budget Planning

aapanaara gaemaera janaya baaajaeta paraikalapanaaa karauna:

- Development costs
- Marketing costs
- Operations costs
- Contingency

anaushailana 2: Revenue Projection

Revenue projection taairai karauna:

- Conservative scenario
- Moderate scenario
- Optimistic scenario

anaushailana 3: Break-even Analysis

Break-even analysis karauna:

- Fixed costs nairadhaaarana karauna
- Break-even point gananaaa karauna
- Break-even chart taairai karauna

anaushailana 4: ROI Calculation

ROI calculation karauna:

- Investment amount
- Expected return
- ROI percentage
- Payback period

saaadhaaarana bhaula (Common Mistakes)

1. Underestimating Costs

- # bhaula: kharacha kama manae karaaa
- # sathaika: baaasatabasamamata baaajaeta taairai karauna

2. No Contingency

- # bhaula: Emergency fund naei
- # sathaika: 10-20% contingency raaakhauna

3. Ignoring Taxes

- # bhaula: tayaaakasa upaekassaaa karaaa
- # sathaika: tayaaakasa paraikalapanaaa karauna

4. Poor Financial Tracking

bhaula: aarathaika haisaaaba naaa raaakhaaa
sathaika: naiyamaita haisaaaba raaakhauna

taipasa (Tips)

1. Start Small

chhaota baaajaeta daiyae shaurau karauna

2. Track Everything

saba kharacha aiba aaya tarayaaaka karauna

3. Plan for Taxes

tayaaakasa janaya aagae thaekae paraikalapanaaa karauna

4. Emergency Fund

Emergency fund raaakhauna

5. Consult Professionals

parayojanae paeshaaadaaaradaera saaathae kathaaa balauna

saaarasakassaepa (Summary)

Development Budget: Personnel, Software, Hardware, Marketing
Marketing Budget: Ads, Influencer, Press, Events
Asset Costs: 2D, 3D, Animation, Sound, Music
Software Costs: Engine, Tools, Licenses
Team Costs: Salaries, Contractors
Revenue Projections: Conservative, Moderate, Optimistic
Profit Calculation: Revenue - Costs = Profit
Break-even Analysis: Fixed Costs / (Price - Variable Cost)
ROI Calculation: ((Return - Investment) / Investment) x 100
Sample Budget: \$91,700 for indie game
Financial Planning: Budget, Cash Flow, P&L
Tax Considerations: Income, Sales, VAT, Deductions
Business Structure: Sole Proprietorship, LLC, Corporation

GAME DEVELOPMENT MASTERBOOK - Chapter 32

PART 33

Chapter 33: GAME DEVELOPMENT TIMELINE

Chapter 33: GAME DEVELOPMENT TIMELINE

gaema daebhaelapamaenata taaaimalaaaina

shaekhaara udadaeshaya (Learning Goal)

aai adhayaaayae aapanai shaikhabaena kaiibhaaaba gaema daebhaelapamaenata parajaekataera janaya baaasatabasamamata taaaimalaaaina taairai karatae haya baibhainana sakaelaera gaema, Solo Developer aiba Small Team aira janaya aalaaadaabhaaaba Roadmap palayaaanai, Milestone saetai, aiba Risk Assessment karatae shaikhabaena

33.1 gaema daebhaelapamaenata taaaimalaaainaera gaurautaba

taaaimalaaaina kaena darakaaara?

taaaimalaaaina chhaaadaa gaema daebhaelapamaenata = anadhakaaarae gaaadai chaaalaaanao

taaaimalaaaina palayaaanaiyaera mauulanaiitai

1. Realistic Estimation - baaasatabasamamata anaumaaana
2. Buffer Time - atairaikata samaya raaakhauna (20-30%)
3. Milestone-Based - Milestone bhaitataika paraikalapanaaa
4. Risk Assessment - jhaukai mauulayaaayana
5. Flexible Planning - namanaiya paraikalapanaaa

33.2 3-maaasaera taaaimalaaaina (Small Mobile/Casual Game)

Solo Developer aira janaya

3-MONTH TIMELINE (Solo Developer)
Small Mobile/Casual Game

MONTH 1: PRE-PRODUCTION + CORE DEVELOPMENT

Week 1-2: Concept + GDD + Art Style

- Game Concept Document (1-2 days)

- Basic GDD (1-2 days)
- Art Style Guide (2-3 days)
- Prototype Setup (1-2 days)

Week 3-4: Core Gameplay

- Core Mechanic Implementation
- Basic Player Controller
- Core Game Loop
- Basic UI

MONTH 2: CONTENT CREATION + POLISH

Week 5-6: Content Building

- Level Design (5-10 levels)
- Art Asset Creation
- Sound Effects
- Background Music

Week 7-8: Integration + Polish

- All Systems Integration
- UI Polish
- Visual Effects
- Performance Optimization

MONTH 3: TESTING + LAUNCH

Week 9-10: Testing + Bug Fixing

- Internal Testing
- Beta Testing
- Bug Fixing
- Performance Testing

Week 11-12: Marketing + Launch

- Store Page Setup
- Trailer Creation
- Social Media Marketing
- Soft Launch
- Full Launch
- = Active Development
- = Planning/Waiting

Milestones aiba Deliverables

Buffer Time saupaaaraisha

Total Timeline: 12 weeks

Buffer Time: 2-3 weeks (20-25%)

Recommended Buffer Distribution:

- Month 1: 1 week buffer (Concept changes)
- Month 2: 1 week buffer (Content delays)
- Month 3: 1-2 weeks buffer (Testing/Bug fixing)

Risk Assessment

33.3 6-maaasaera taaaimalaaaina (Medium Indie Game)

Solo Developer aira janaya

6-MONTH TIMELINE (Solo Developer)
Medium Indie Game

- MONTH 1-2: PRE-PRODUCTION
- Complete GDD (Week 1-2)
 - Art Style Development (Week 2-3)
 - Technical Prototype (Week 3-4)
 - Core Mechanic Proof of Concept (Week 4-6)
 - Vertical Slice Planning (Week 6-8)
- MONTH 3-4: CORE DEVELOPMENT
- Vertical Slice Creation (Week 9-12)
 - Core Systems Implementation (Week 9-14)
 - Main Gameplay Loop (Week 12-16)
 - Basic AI Implementation (Week 14-16)
- MONTH 5: CONTENT + POLISH
- Full Level Design (Week 17-20)
 - Art Asset Pipeline (Week 17-20)
 - Audio Integration (Week 19-20)
 - UI/UX Polish (Week 20)
- MONTH 6: TESTING + LAUNCH
- QA Testing (Week 21-22)
 - Beta Testing (Week 22-23)
 - Bug Fixing + Optimization (Week 23-24)
 - Marketing Push (Week 24)
 - Launch (Week 24-25)

Milestones aiba Deliverables

Buffer Time saupaaaraisha

- Total Timeline: 24 weeks (6 months)
- Buffer Time: 4-5 weeks (17-20%)
- Recommended Buffer Distribution:
- Month 1-2: 1 week (Concept validation)
 - Month 3-4: 2 weeks (Core development delays)
 - Month 5: 1 week (Content creation)
 - Month 6: 1-2 weeks (Testing/Bug fixing)

Risk Assessment

33.4 12-maaasaera taaaimalaaaina (Large Indie Game)

Small Team (3-5 jana) aira janaya

- 12-MONTH TIMELINE (Team: 3-5)
- Large Indie Game
- QUARTER 1 (MONTH 1-3): PRE-PRODUCTION
- Month 1: Concept + Planning
- Week 1: Game Vision Document
 - Week 2: Core Gameplay Document
 - Week 3: Art Style Development
 - Week 4: Technical Architecture
- Month 2: Core Prototype
- Week 5-6: Core Mechanic Prototype
 - Week 7: Art Pipeline Setup
 - Week 8: Technical Foundation

Month 3: Vertical Slice
Week 9-10: Vertical Slice Development
Week 11: Vertical Slice Polish
Week 12: Review + Approval
QUARTER 2 (MONTH 4-6): CORE DEVELOPMENT
Month 4: Core Systems
Week 13-14: Game Systems Implementation
Week 15: AI System Foundation
Week 16: Physics + Collision
Month 5: Gameplay Systems
Week 17-18: Combat/Interaction System
Week 19: Inventory/Progression System
Week 20: Quest/Story System
Month 6: Integration
Week 21-22: Systems Integration
Week 23: First Playable Build
Week 24: Internal Review
QUARTER 3 (MONTH 7-9): CONTENT CREATION
Month 7: Level Design Phase 1
Week 25-26: Main Hub/World Design
Week 27-28: First Level Blockout
Month 8: Content Pipeline
Week 29-30: Art Asset Production
Week 31-32: Audio Assets
Month 9: Level Design Phase 2
Week 33-34: Additional Levels
Week 35-36: Content Integration
QUARTER 4 (MONTH 10-12): POLISH + LAUNCH
Month 10: Polish
Week 37-38: Visual Polish
Week 39-40: Audio Polish
Month 11: Testing
Week 41-42: Internal QA
Week 43-44: External Beta Testing
Month 12: Launch
Week 45-46: Bug Fixing + Optimization
Week 47-48: Marketing Push + Launch

Team Role Distribution

Buffer Time saupaaaraisha

Total Timeline: 48 weeks (12 months)
Buffer Time: 8-10 weeks (17-21%)
Recommended Distribution:
- Q1 (Pre-Production): 2 weeks buffer
- Q2 (Core Dev): 3 weeks buffer
- Q3 (Content): 2 weeks buffer
- Q4 (Polish/Launch): 3 weeks buffer

Risk Assessment

33.5 18-maasaera taaimalaaaina (Ambitious Indie Game)

Small Team (3-5 jana) aira janaya

18-MONTH TIMELINE (Team: 3-5)

Ambitious Indie Game

PHASE 1: FOUNDATION (MONTH 1-4)

Month 1-2: Vision + Research

- Game Vision Document
- Market Research
- Competitive Analysis
- Art Style Exploration
- Technical Research

Month 3-4: Core Prototype

- Core Mechanic Prototype
- Art Pipeline Setup
- Technical Architecture
- Vertical Slice Planning

PHASE 2: PROOF OF CONCEPT (MONTH 5-8)

Month 5-6: Vertical Slice

- 1 Complete Level
- Core Systems Working
- Basic AI
- UI/UX Foundation

Month 7-8: Core Systems Expansion

- All Major Systems
- Progression System
- Save/Load System
- Settings System

PHASE 3: CORE DEVELOPMENT (MONTH 9-12)

Month 9-10: Feature Development

- All Gameplay Systems
- AI Development
- Combat System
- Progression System

Month 11-12: Integration

- Systems Integration
- First Full Build
- Internal Testing
- Feature Review

PHASE 4: CONTENT CREATION (MONTH 13-15)

Month 13-14: Content Production

- Level Design (All Levels)
- Art Asset Production
- Audio Production
- UI/UX Design

Month 15: Content Integration

- All Content Integrated
- Full Game Playable
- Content Review

PHASE 5: POLISH + LAUNCH (MONTH 16-18)

Month 16: Polish

- Visual Polish
- Audio Polish
- UI Polish
- Performance Optimization

Month 17: Testing

- Internal QA
- External Beta
- Performance Testing
- Platform Testing

Month 18: Launch

Bug Fixing
Marketing Push
Store Page Setup
Launch

Major Milestones

Buffer Time saupaaaraisha

Total Timeline: 72 weeks (18 months)
Buffer Time: 12-14 weeks (17-19%)
Recommended Distribution by Phase:
- Foundation: 2 weeks
- Proof of Concept: 3 weeks
- Core Development: 3 weeks
- Content Creation: 2 weeks
- Polish + Launch: 3 weeks

Risk Assessment

33.6 Solo Developer vs Team Comparison

taaaimalaaaaina taulanaaa

SOLO DEVELOPER vs TEAM TIMELINE COMPARISON	
Small Game (3 Months):	
Solo:	(3 months)
Team:	(2 months)
Medium Game (6 Months):	
Solo:	(6 months)
Team:	(4 months)
Large Game (12 Months):	
Solo:	(12 mo)
Team:	(9 months)
= Active Development	
= Buffer/Buffer Time	

Solo Developer aira saubaidhaaa aiba saiimaaabadadhataaa

saubaidhaaa:

- + Full Creative Control
- + No Communication Overhead
- + Flexible Schedule
- + Lower Cost
- + Quick Decisions

saiimaaabadadhataaa:

- Slower Production
- Limited Skill Set
- No Backup
- Burnout Risk
- Limited Testing

Team aira saubaidhaaa aiba saiimaaabadadhataaa

saubaidhaaa:

- + Faster Production
- + Diverse Skills
- + Built-in Testing
- + Shared Burden
- + Better Quality

saiimaaabadadhataaa:

- Higher Cost
- Communication Overhead
- Creative Conflicts
- Coordination Challenges
- Shared Vision Required

33.7 Milestone Planning kaaushala

SMART Milestones

S - Specific: nairadaissata aiba sapassata

M - Measurable: paraimaaapayaogaya

A - Achievable: samabhaba

R - Relevant: paraaasangagaika

T - Time-bound: samayasaiimaaabadadha

Milestone Template

Milestone: [Name]

Target Date: [Date]

Duration: [Weeks]

Deliverables:

- [] [Deliverable 1]
- [] [Deliverable 2]
- [] [Deliverable 3]

Success Criteria:

- [] [Criteria 1]
- [] [Criteria 2]

Dependencies:

- [Dependency 1]
- [Dependency 2]

Risks:

- [Risk 1] - Mitigation: [Action]
- [Risk 2] - Mitigation: [Action]

Milestone Tracking

33.8 Buffer Time baishalaessana

kaena Buffer Time darakaaara?

Reality Check:

- parathama anaumaaana sabasamaya bhaula haya
- samasayaaa anaeka samaya aparatayaaashaita aasae
- maaanaiyae naeauyaaara samaya darakaaara
- Quality maintain karatae samaya laaagae

Buffer Time Formula

Buffer Time = Base Estimate x Buffer Percentage

Buffer Percentage nairadhaaarana:

- Well-understood project: 15-20%
- New technology: 25-30%
- Complex project: 30-40%
- Unprecedented project: 40-50%

Buffer Distribution Strategy

BUFFER TIME DISTRIBUTION

Total Project: 12 months

Total Buffer: 3 months (25%)

Phase 1 (Pre-Production): 15% of buffer = 1.5 weeks

Phase 2 (Production): 50% of buffer = 6 weeks

Phase 3 (Testing/Launch): 35% of buffer = 4.5 weeks

33.9 Risk Assessment Framework

Risk Matrix

RISK ASSESSMENT MATRIX

IMPACT

	High	Medium	Low	
Critical	****	***	**	*
High	****	***	**	*
Med	***	**	*	0
	***	**	*	0
Low	**	*	0	0
	**	*	0	0

PROBABILITY ->

* = Risk Items

o = Acceptable Risks

Common Risks and Mitigation

33.10 Exercise

anaushailana 1: naijaera gaemaera janaya taaaimalaaaina taairai karauna

1. aapanaaara gaemaera sakaela nairadhaaarana karauna
2. uparaera taaimalaaaina thaekae aikatai baechhae naina
3. naijaera paraisathaitai anauyaaayaii kaaasatamaaaaija karauna
4. Milestones saeta karauna
5. Buffer Time yaoga karauna

anaushaiilana 2: Risk Assessment karauna

1. aapanaaara parajaekataera janaya 10tai Risk laikhauna
2. parataitaira janaya Probability aiba Impact nairadhaaarana karauna
3. Mitigation Strategy laikhauna
4. Risk Matrix taairai karauna

anaushaiilana 3: Milestone Document taairai karauna

1. aapanaaara parajaekataera janaya 5tai Milestone saeta karauna
2. parataitaira janaya Deliverables laikhauna
3. Success Criteria saeta karauna
4. Dependencies chaihanaite karauna

33.11 saaadhaaarana bhaula (Common Mistakes)

bhaula 1: Optimistic Estimation

bhaula: "aamai aitaana 1 maaasae shaessa karaba"
sathaika: "aitaana 1 maaasa + 1 sapataaaha samaya naitae paaarae"

bhaula 2: Buffer Time naaa raaakhaaa

bhaula: Timeline = Actual Work Time
sathaika: Timeline = Work Time + Buffer Time (20-30%)

bhaula 3: Scope Creep maaanaaa

bhaula: "aitaukau phaichaaara yaoga karalaei tao habae"
sathaika: GDD anauyaaayaii thaaakauna, paraibaratanara parae karauna

bhaula 4: Regular Review naaa karaaa

bhaula: Milestone shaessa haauyaaara aagae raibhaiu naaa karaaa
sathaika: sapataaahae aikabaaara Progress Review karauna

bhaula 5: Risk Ignore karaaa

bhaula: "samasayaaa halae daekhaaa yaaabae"
sathaika: Risk Assessment aagae thaekae karauna

33.12 taipasa (Tips)

taipasa 1: Start Small

chhaota gaema daiyae shaurau karauna, bada gaemae yaaana

taipasa 2: Use Templates

Timeline Template bayabahaaara karauna, shauunaya thaekae shaurau naaa karauna

taipasa : Track Daily

parataidaina Progress tarayaaaka karauna, sapataaahae aikabaaara naya

taipasa 4: Be Flexible

Timeline paraibaratana hatae paaarae, aitaana sabaaabhaaabaika

taipasa 5: Celebrate Milestones

parataitai Milestone shaessa halae udayaaapana karauna

33.13 saarasakassaepa (Summary)

aai adhayaaayae aamaraaa shaikhaechhai:

1. ****taaimalaaainaera gaurautaba**** - gaema daebhaelapamaenatae sathaika taaaimalaaaina katataaa jaraurai
2. ****3-maasaera taaaimalaaaina**** - chhaota maobaaaila/kayaaajauyaaala gaemaera janaya
3. ****6-maasaera taaaimalaaaina**** - maaajhaaarai inadai gaemaera janaya
4. ****12-maasaera taaaimalaaaina**** - bada inadai gaemaera janaya
5. ****18-maasaera taaaimalaaaina**** - mahasaaahasaika inadai gaemaera janaya
6. ****Solo vs Team**** - dauto padadhataira taulanaaa
7. ****Milestone Planning**** - SMART Milestones kaiibhaaaba saeta karabaena
8. ****Buffer Time**** - kaena aiba kaiibhaaaba Buffer Time raaakhabaena
9. ****Risk Assessment**** - jhaukai mauulayaaayanaera kaaushala

manae raaakhauna:

taaimalaaaina aikatai gaaaida, aikatai baedaaa naya
baasatabasamamata paraikalapanaaa karauna, namanaiiya thaaakauna

****parabarataai adhayaaaya:**** Chapter 34: Daily/Weekly Development System

PART 34

natauna feature branch taairai karauna

Chapter 34: DAILY/WEEKLY DEVELOPMENT SYSTEM

daainaika/saaapataaahaika daebhaelapamaenata saisataema

shaekhaara udadaeshaya (Learning Goal)

aii adhayaaayae aapanai shaikhabaena kaiibhaaaba gaema daebhaelapamaenatae daainaika aiba saaapataaahaika kaaajaera saisataema taairai karatae haya Daily Task System, Weekly Sprint, Monthly Milestone, Backlog Management, Kanban Board, Git Workflow, aiba Productivity Tips samaparaka baaisataaaraita shaikhabaena

34.1 Daily Task System for Solo Developer

daainaika kaaajaera dharana

```
DAILY TASK SYSTEM (Solo Developer)
sakaala (Morning) - 9:00 AM - 12:00 PM
9:00 - 9:15 : Daily Planning + Review
9:15 - 10:30 : Deep Work Block 1 (Core Development)
10:30 - 10:45: Break
10:45 - 12:00: Deep Work Block 2 (Core Development)
daupura (Afternoon) - 1:00 PM - 5:00 PM
1:00 - 1:15 : Afternoon Review
1:15 - 2:30 : Deep Work Block 3 (Secondary Tasks)
2:30 - 2:45 : Break
2:45 - 4:00 : Deep Work Block 4 (Secondary Tasks)
4:00 - 4:30 : Communication/Email
4:30 - 5:00 : Daily Review + Planning Tomorrow
sanadhayaaa (Evening) - Optional
6:00 - 7:00 : Learning/Research (Optional)
7:00 - 8:00 : Admin Tasks (Optional)
Note: parataidaina 6-8 ghanataaaa kaaaja yathaessata baeshai karabaena naaa
```

daainaika Task Template

```
## aajakaera kaaaja - [taaaraikha]
```

```
### agaraaadhaikaaara 1 (High Priority):
- [ ] [kaaaja 1] - aanaumaaanaika samaya: [X] mainaita
- [ ] [kaaaja 2] - aanaumaaanaika samaya: [X] mainaita

### agaraaadhaikaaara 2 (Medium Priority):
- [ ] [kaaaja 3] - aanaumaaanaika samaya: [X] mainaita
- [ ] [kaaaja 4] - aanaumaaanaika samaya: [X] mainaita

### agaraaadhaikaaara 3 (Low Priority):
- [ ] [kaaaja 5] - aanaumaaanaika samaya: [X] mainaita

### samaya tarayaaakai:
- maota kaaajaera samaya: [X] ghanataaa
- Productive Time: [X] ghanataaa
- Break Time: [X] mainaita

### samasayaaa/baaadhaaa:
- [samasayaaa 1]
- [samasayaaa 2]

### aagaaamaiikaaalaera janaya:
- [kaaaja 1]
- [kaaaja 2]
```

Task Prioritization Method (Eisenhower Matrix)

EISENHOWER MATRIX			
jaraurai		jaraurai naya	
gaurautabapauurana	karauna (DO)	paraikalapanaaa	karauna
jaraurai + gaurautabapauurana		gaurautabapauurana + jaraurai naya	
udaaaharana:		udaaaharana:	
- Bug Fix		- Feature Dev	
- Crash Fix		- Optimization	
- Deadline		- Learning	
gaurautabapauurana naya	aparatainaidhaitaba karauna	baaada daina (DELETE)	
jaraurai + gaurautabapauurana naya		jaraurai naya + gaurautabapauurana na	
udaaaharana:		udaaaharana:	
- Some Emails		- Social Media	
- Some Meetings		- Distractions	
- Some Calls		- Time Wasters	

34.2 Weekly Sprint Methodology

Sprint kaii?

Sprint = aikatai nairadaissata samayakaaala (saaadhaaaranata 1 sapataaaha) yaekhaaanae nai

Weekly Sprint Structure

```
WEEKLY SPRINT STRUCTURE
shanaibaaara/rabaibaaara: SPRINT PLANNING
- gata sapataaahaera Review
- aii sapataaahaera Goal saeta karaaa
- Tasks baraadada karaaa
- Priority saeta karaaa
saomabaaara - barihasapataibaaara: SPRINT EXECUTION
- daainaika kaaaja samapanana karaaa
```

- Progress tarayaaaka karaaa
 - samasayaaa samaaadhaaana karaaa
 - Daily Standup (naijaera saaathae)
- shaukarabaaaara: SPRINT REVIEW + RETROSPECTIVE
- Sprint Goal pauurana hayaechhae kainaaa chaeka karaaa
 - Completed Tasks raibhaiu karaaa
 - Incomplete Tasks baishalaessana karaaa
 - Retrospective: kiii bhaaalao hayaechhae, kiii bhaaalao hayanai
 - parabarataii sapataaahaera janaya paraikalapananaa

Sprint Board Template

```
## Sprint [Number]: [taaaraikha thaekae] thaekae [taaaraikha] parayanata

### Sprint Goal:
[aii sapataaahae kiii arajana karatae habae]

### Backlog (karaaara kaaaja):
| Task | Priority | Status | Assignee | Notes |
|-----|-----|-----|-----|-----|
| [kaaaja 1] | High | | Self | |
| [kaaaja 2] | Medium | | Self | |
| [kaaaja 3] | Low | | Self | |

### In Progress (chalamaana):
| Task | Started | Expected | Status |
|-----|-----|-----|-----|
| [kaaaja] | [taaaraikha] | [taaaraikha] | |

### Completed (samapanana):
| Task | Completed | Notes |
|-----|-----|-----|
| [kaaaja] | [taaaraikha] | |

### Blocked (baaadhaaagarasata):
| Task | Blocker | Resolution |
|-----|-----|-----|
| [kaaaja] | [kaaarana] | [samaaadhaaana] |
```

Sample Weekly Sprint

```
SAMPLE WEEKLY SPRINT
Sprint 5: 1 jaaanauyaaarai - 7 jaaanauyaaarai
SPRINT GOAL: Combat System aira mauula phaichaaara samapanana karaaa
MONDAY (1 jaaanauyaaarai):
[ ] Basic Attack Implementation
[ ] Attack Animation Setup
[ ] Damage System
TUESDAY (2 jaaanauyaaarai):
[ ] Enemy Health System
[ ] Hit Detection
[ ] Visual Feedback (Hit Effects)
WEDNESDAY (3 jaaanauyaaarai):
[ ] Special Attack Implementation
[ ] Cooldown System
[ ] UI for Special Attack
THURSDAY (4 jaaanauyaaarai):
[ ] Enemy AI - Basic Behavior
[ ] Enemy Attack Pattern
[ ] Enemy Death Animation
FRIDAY (5 jaaanauyaaarai):
```

```
[ ] Integration Testing
[ ] Bug Fixing
[ ] Sprint Review + Retrospective
COMPLETED:
[x] Basic Attack Implementation
[x] Attack Animation Setup
[x] Damage System
[x] Enemy Health System
[x] Hit Detection
[x] Visual Feedback
[x] Special Attack Implementation
[x] Cooldown System
[x] UI for Special Attack
[x] Enemy AI - Basic Behavior
[x] Enemy Attack Pattern
INCOMPLETE:
  Enemy Death Animation (Next Sprint)
  Integration Testing (Next Sprint)
RETROSPECTIVE:
  What went well: Good planning, realistic estimates
  What to improve: More buffer for animation work
  Action items: Use more asset store animations
```

34.3 Monthly Milestone Planning

Monthly Milestone Structure

```
## Monthly Milestone: [maaasaera naaama] [bachhara]
```

```
### Overall Goal:
```

```
[aai maaasae kaa arajana karatae habae]
```

```
### Key Deliverables:
```

1. [Deliverable 1] - Due: [taaaraikha]
2. [Deliverable 2] - Due: [taaaraikha]
3. [Deliverable 3] - Due: [taaaraikha]

```
### Weekly Breakdown:
```

```
#### Week 1:
```

- [] [kaaaja 1]
- [] [kaaaja 2]

```
#### Week 2:
```

- [] [kaaaja 3]
- [] [kaaaja 4]

```
#### Week 3:
```

- [] [kaaaja 5]
- [] [kaaaja 6]

```
#### Week 4:
```

- [] [kaaaja 7]
- [] [kaaaja 8]

```
### Progress Tracking:
```

- Week 1: []% Complete
- Week 2: []% Complete
- Week 3: []% Complete
- Week 4: []% Complete

```
### Risks:
- [Risk 1] - Mitigation: [Action]
- [Risk 2] - Mitigation: [Action]
```

Sample Monthly Plan

```
MONTHLY MILESTONE EXAMPLE
January 2026 - Core Combat System
GOAL: Combat System aira mauula phaichaaara samapanana karaaa
DELIVERABLES:
1. Player Attack System - Due: Jan 15
2. Enemy AI System - Due: Jan 25
3. Combat UI - Due: Jan 31
WEEKLY BREAKDOWN:
Week 1 (Jan 1-7): Player Attack System
Tasks: Basic Attack, Animation, Damage
Week 2 (Jan 8-14): Enemy Health + Hit Detection
Tasks: Enemy Health, Hit Detection, Feedback
Week 3 (Jan 15-21): Enemy AI System
Tasks: Enemy AI, Attack Pattern, Death
Week 4 (Jan 22-31): Integration + UI
Tasks: Integration, Combat UI, Testing
PROGRESS:
Week 1: 100% [x]
Week 2: 100% [x]
Week 3: 100% [x]
Week 4: 75% (In Progress)
RISKS:
1. Enemy AI Complexity - Mitigation: Start simple, iterate
2. Animation Delays - Mitigation: Use asset store
```

34.4 Backlog Management

Backlog kائي?

Backlog = saba kaaajaera taaalaikaaa yaaa karaaa darakaaara, kainatau aikhanao shaurau hay

Backlog Structure

```
## Game Development Backlog

### Epic 1: Core Gameplay
#### Feature 1: Player Movement
- [ ] Basic Movement (Walk, Run, Jump)
- [ ] Animation System
- [ ] Collision Detection
- [ ] Input Handling

#### Feature 2: Combat System
- [ ] Basic Attack
- [ ] Special Attacks
- [ ] Enemy AI
- [ ] Health System

### Epic 2: UI/UX
#### Feature 1: Main Menu
- [ ] Menu Layout
```


- [] Menu Navigation
- [] Settings Menu
- [] Credits

Feature 2: HUD

- [] Health Bar
- [] Score Display
- [] Mini Map
- [] Inventory UI

Epic 3: Content

Feature 1: Level 1

- [] Level Layout
- [] Art Assets
- [] Enemy Placement
- [] Puzzle Design

Feature 2: Level 2

- [] Level Layout
- [] Art Assets
- [] Enemy Placement
- [] Puzzle Design

Backlog Prioritization

BACKLOG PRIORITIZATION

P0 - CRITICAL (aikhanai karatae habae):

Core Gameplay Systems
Game Breaking Bugs
Essential Features

P1 - HIGH (shaiigharai karatae habae):

Important Features
Quality of Life
Performance Issues

P2 - MEDIUM (maaaajhaaarai agaraaadhaikaaara):

Nice to Have Features
Visual Polish
Audio Enhancement

P3 - LOW (kama agaraaadhaikaaara):

Optional Features
Cosmetic Changes
Future Updates

Backlog Review Process

sapataaahae aikabaaara Backlog Review karauna:

1. New Tasks yaoga karauna
2. Priority raibhaiu karauna
3. Completed Tasks saraaana
4. Blocked Tasks chaihanaita karauna
5. Estimate raibhaiu karauna
6. Sprint Planning aira janaya Ready karauna

34.5 Kanban Board Concept

Kanban Board kii?

Kanban Board = kaaajaera abasathaaa daekhaaaanaora bhaijayauyaaala taula

Physical Kanban Board

KANBAN BOARD

BACKLOG
(karaaara kaaaja)

IN PROGRESS
(chalamaana)

REVIEW

DONE
(raibhaiu)

(samapanana)

Task 1
->

Task 5
->

Task 8
->

Task 10

Task 2
->

Task 6
->

Task 9
->

Task 11

Task 3
->

Task 7

Task 4

WIP LIMITS:

- Backlog: No Limit

- In Progress: 3 Tasks Max

- Review: 2 Tasks Max

- Done: No Limit

Digital Kanban (Trello/Notion)

DIGITAL KANBAN SETUP

TRELLO BOARD:

Lists (Columns):

1. Backlog

2. To Do

3. In Progress

4. Review

5. Done

Cards (Tasks):

- Title: Task Name

- Description: Details

- Labels: Priority (High/Medium/Low)

- Due Date: Deadline

- Checklist: Subtasks

- Attachments: Screenshots, Documents

NOTION SETUP:

Database: Game Development

Properties:

- Task Name (Title)

- Status (Select: Backlog, To Do, In Progress, Review, Done)

- Priority (Select: High, Medium, Low)

- Due Date (Date)

- Assignee (Person)

- Tags (Multi-select)

- Notes (Text)

Trello Workflow Setup

- Step 1: Board taairai karauna
- Step 2: Lists taairai karauna (Backlog, To Do, In Progress, Review, Done)
- Step 3: Labels taairai karauna (High, Medium, Low Priority)
- Step 4: Cards yaoga karauna (Tasks)
- Step 5: Due Dates saeta karauna
- Step 6: Checklists yaoga karauna (Subtasks)
- Step 7: Attachments yaoga karauna (Screenshots, Docs)
- Step 8: Members yaoga karauna (Team Members)

Notion Workflow Setup

Step 1: Database taairai karauna
Step 2: Properties yaoga karauna
Step 3: Views taairai karauna (Board, List, Calendar, Timeline)
Step 4: Templates taairai karauna
Step 5: Filters saeta karauna
Step 6: Sorts saeta karauna
Step 7: Linked Databases yaoga karauna
Step 8: Automations saeta karauna

34.6 Git Workflow for Game Development

Git kii?

Git = Version Control System yaaa kaodaera paraibaratana tarayaaaka karae

Git Branch Strategy

```
GIT BRANCH STRATEGY
main (Production)
develop (Development)
  feature/combat-system
  feature/enemy-ai
  feature/ui-system
  feature/level-1
hotfix/critical-bug
release/v1.0
BRANCH WORKFLOW:
1. develop thaekae feature branch taairai karauna
2. feature branch ai kaaaja karauna
3. develop ai merge karauna
4. feature branch delete karauna
```

Git Commands for Game Development

```
# natauna feature branch taairai karauna
git checkout -b feature/combat-system develop

# kaaaja shaessa halae commit karauna
git add .
git commit -m "feat: implement basic combat system"

# develop ai merge karauna
git checkout develop
git merge feature/combat-system

# feature branch delete karauna
git branch -d feature/combat-system

# remote repository ai push karauna
git push origin develop
```

Commit Message Convention

```

feat: natauna phaichaaara yaoga karaaa
fix: bug fix karaaa
docs: documentation aapadaeta karaaa
style: code style paraibaratana karaaa
refactor: code refactor karaaa
test: test yaoga karaaa/aapadaeta karaaa
chore: build process paraibaratana karaaa

```

Gitignore for Game Development

```

# Unity
[LL]ibrary/
[Tt]emp/
[Oo]bj/
[Bb]uild/
[Bb]uilds/
[LL]ogs/
[Uu]ser[Ss]ettings/

# Unreal
Binaries/
DerivedDataCache/
Intermediate/
Saved/

# Godot
.godot/
export.cfg
export_presets.cfg

# IDE
.vs/
.vscode/
*.sln
*.suo
*.user
*.userprefs

# OS
.DS_Store
Thumbs.db

```

34.7 Sample Weekly Schedule

Solo Developer Weekly Schedule

```

SAMPLE WEEKLY SCHEDULE
Solo Developer - Game Development

SHATIBAHAR (SATURDAY) - PLANNING DAY
10:00 - 11:00: Review Last Week
11:00 - 12:00: Plan This Week
12:00 - 1:00: Lunch
1:00 - 2:00: Backlog Management
2:00 - 4:00: Learning/Research
4:00 - 5:00: Admin Tasks
RAVIBAHAR (SUNDAY) - REST DAY
Rest Day - No Development Work
Optional: Play Games, Read, Learn

```

SOMBAHAR (MONDAY) - CORE DEVELOPMENT
 9:00 - 9:15: Daily Planning
 9:15 - 12:00: Core Development Block 1
 12:00 - 1:00: Lunch
 1:00 - 4:00: Core Development Block 2
 4:00 - 5:00: Review + Planning Tomorrow
 MANGALBAHAR (TUESDAY) - CORE DEVELOPMENT
 9:00 - 9:15: Daily Planning
 9:15 - 12:00: Core Development Block 1
 12:00 - 1:00: Lunch
 1:00 - 4:00: Core Development Block 2
 4:00 - 5:00: Review + Planning Tomorrow
 BUDHBAHAR (WEDNESDAY) - SECONDARY TASKS
 9:00 - 9:15: Daily Planning
 9:15 - 12:00: Secondary Tasks Block 1
 12:00 - 1:00: Lunch
 1:00 - 4:00: Secondary Tasks Block 2
 4:00 - 5:00: Communication/Email
 BIBAHAR (THURSDAY) - TESTING + BUG FIXING
 9:00 - 9:15: Daily Planning
 9:15 - 12:00: Testing Block 1
 12:00 - 1:00: Lunch
 1:00 - 4:00: Bug Fixing Block
 4:00 - 5:00: Review + Planning Tomorrow
 SHUKRABAHAR (FRIDAY) - REVIEW + POLISH
 9:00 - 9:15: Daily Planning
 9:15 - 12:00: Polish Block
 12:00 - 1:00: Lunch
 1:00 - 3:00: Weekly Review
 3:00 - 4:00: Retrospective
 4:00 - 5:00: Next Week Planning

34.8 Task Prioritization Methods

MoSCoW Method

MoSCoW METHOD

MUST HAVE:

- Core Gameplay Systems
- Minimum Viable Product
- Essential Features

SHOULD HAVE:

- Important Features
- Quality of Life
- Performance Optimization

COULD HAVE:

- Nice to Have Features
- Visual Polish
- Extra Content

WON'T HAVE (THIS TIME):

- Future Features
- Nice to Have but Not Essential
- Out of Scope

Value vs Effort Matrix

VALUE vs EFFORT MATRIX

	LOW EFFORT	HIGH EFFORT
HIGH VALUE	QUICK WINS (DO FIRST)	MAJOR PROJECTS (PLAN CAREFULLY)
	- Basic Attack	- Full Combat
	- Menu System	- AI System
	- Save System	- Level Editor
LOW VALUE	FILL-INS (DO IF TIME)	THANKLESS TASKS (AVOID/DELAY)
	- Minor UI	- Over-engineering
	- Small Polish	- Unnecessary
	- Optional Features	- Features

34.9 Productivity Tips for Solo Developers

Productivity System

PRODUCTIVITY TIPS

1. TIME BLOCKING
nairadaissata samayae nairadaissata kaaaja karauna
parataitai balakaera janaya aalaaadaaa kaaaja saeta karauna
Break samaya nairadhaaarana karauna
2. POMODORO TECHNIQUE
25 mainaita kaaaja karauna
5 mainaita baisharaaama naina
4tai Pomodoro aira para 15-30 mainaita baisharaaama
parataidaina 8-12tai Pomodoro taaaragaeta karauna
3. TWO-MINUTE RULE
2 mainaitaera kama samaya laaagae aimana kaaaja aikhanai karauna
dairagha kaaajaera aagae chhaota chhaota kaaaja shaessa karauna
aitae momentum taairai haya
4. SINGLE TASKING
aikabaaarae aikatai kaaaja karauna
Multitasking avoid karauna
Focus bajaaya raaakhauna
5. DAILY TOP 3
parataidaina 3tai gaurautabapaurana kaaaja saeta karauna
saegaulao aagae shaessa karauna
baaakai kaaaja taaarapara karauna

Focus Techniques

FOCUS TECHNIQUES

1. DEEP WORK
naotaiphaikaeshana banadha karauna
Distraction-free environment taairai karauna
2-4 ghanataaara Deep Work Session karauna
parataidaina anatata 1tai Deep Work Session karauna
2. FLOW STATE
Challenge aiba Skill aira bhaarasaaamaya raaakhauna
Clear Goals saeta karauna
Immediate Feedback daina
Flow triggers bayabahaaara karauna
3. ENVIRONMENT
paraissakaaara paraichachhanana workspace raaakhauna
Comfortable setup taairai karauna
Distractions saraiyae phaelauna

Music/White noise bayabahaaara karauna

4. ENERGY MANAGEMENT

Peak energy time ai kathaina kaaaja karauna

Low energy time ai sahaja kaaaja karauna

Regular breaks naina

Sleep, Exercise, Nutrition manaoyaoga daina

34.10 Burnout Prevention Strategies

Burnout kii?

Burnout = maaanasaike aiba shaaarairaike kalaaanatai yaaa daiiraghamaeyaaadai sataraesa t

Burnout Prevention System

BURNOUT PREVENTION

1. SUSTAINABLE PACE

parataidaina 6-8 ghanataaa kaaaja karauna (baeshai naya)

sapataaahae 1 daina pauraopaurai baisharaaama naina

maaasae 2-3 daina pauraopaurai baisharaaama naina

Break samayae kaaaja samaparake bhaaababaena naaa

2. BOUNDARIES

kaaaja aiba bayakataigata jaiibana aalaaadaaa raaakhauna

Work hours nairadhaaarana karauna

Weekend ai kaaaja karabaena naaa (baaa khauba kama)

Vacation naina

3. PASSION MAINTENANCE

yae gaematai baaanaaachachhaena saetai khaelauna

anaya gaema khaelauna, anaya gaemaera thaekae shaikhauna

Game Development Community tae yaoga daina

Progress udayaaapana karauna

4. SELF-CARE

parayaaapata ghauma naina (7-8 ghanataaa)

Exercise karauna

Balanced diet khaana

Friends/Family aira saaathae samaya kaaataaana

Hobbies raaakhauna (gaema chhaaadaaa)

5. MENTAL HEALTH

Stress chahanaite karauna

parayaojanae saaahaaayaya naina

Mindfulness/ Meditation anaushailana karauna

Positive self-talk karauna

Burnout Warning Signs

Warning Signs:

1. kaaajae aagaraha haaaraanao
2. karamaagata kalaaanatai anaubhaba karaaa
3. Productivity haraasa paaaayaaa
4. Irritability baridadhai
5. Sleep problems
6. Concentration kamae yaaaayaaa
7. Physical symptoms (headache, stomach issue)
8. Isolation from others
9. Cynicism about the project
10. Feeling overwhelmed

Recovery Strategies

Recovery Strategies:

1. kaichhaudaina pauraopaurai baisharaaama naina
 2. Scope kamaiyae aanauna
 3. saaahaayaya naina (Team Member, Freelancer)
 4. paraaathamaikataaa paunaranairadhaaarana karauna
 5. chhaota chhaota jaya arajana karauna
 6. saaapaorata saisataema taairai karauna
 7. Professional help naina parayaojanae
-

34.11 Exercise

anaushailana 1: Weekly Schedule taairai karauna

1. aapanaaara jaiibanayaaapanaera saaathae mailaiyae Weekly Schedule taairai karauna
2. Deep Work Blocks saeta karauna
3. Break Time nairadhaaarana karauna
4. Review Time yaoga karauna
5. aika sapataaaha aii Schedule anausarana karauna

anaushailana 2: Kanban Board taairai karauna

1. Trello baaa Notion ai baorada taairai karauna
2. Lists/Columns yaoga karauna
3. 10tai Task Cards yaoga karauna
4. Priority Labels saeta karauna
5. Due Dates yaoga karauna
6. aika sapataaaha aii Board bayabahaaara karauna

anaushailana 3: Git Repository saeta aapa karauna

1. GitHub/GitLab ai Repository taairai karauna
2. develop branch taairai karauna
3. 3tai feature branch taairai karauna
4. parataitaitae kaaaja karauna aiba merge karauna
5. commit message convention anausarana karauna

anaushailana 4: Burnout Prevention Plan

1. aapanaaara baratamaaana pace mauulayaaayana karauna
 2. Warning Signs chaihanaite karauna
 3. Prevention Strategy taairai karauna
 4. Sustainable Pace saeta karauna
 5. Regular check-in karauna
-

34.12 saaadhaaarana bhaula (Common Mistakes)

bhaula 1: Overwork

bhaula: parataidaina 10+ ghanataaa kaaaja karaaa
sathaika: 6-8 ghanataaa kaaaja karauna, baaakaitaaa baisharaaama

bhaula 2: No Breaks

bhaula: aikataaanaaa kaaaja karaaa
sathaika: paratai 90 mainaitae 15-20 mainaita baraeka naina

bhaula 3: Multitasking

bhaula: aikasaaathae anaeka kaaaja karaaa
sathaika: aikabaaarae aikatai kaaajae focus karauna

bhaula 4: No Planning

bhaula: sakaaalae uthae bhaaabaaa "aaja kaii karaba?"
sathaika: aagaera daina sanadhayaaya paradainaera paraikalapanaaa karauna

bhaula 5: Ignoring Warning Signs

bhaula: Burnout warning signs ignore karaaa
sathaika: satarakataaa chaihanaite karauna, parayaojanae baisharaaama naina

34.13 taipasa (Tips)

taipasa 1: Start Small

chhaota daiyae shaurau karauna, dhairae dhairae system taairai karauna

taipasa 2: Consistency

parataidaina alapa karae kaaaja karauna, maaajhae maaajhae baeshai naya

taipasa 3: Track Everything

sabakaichhau track karauna - samaya, progress, problems

taipasa : Review Regularly

paratai sapataaahae review karauna, paraibaratana parayaojanae karauna

taipasa 5: Be Kind to Yourself

naijaera paratai sahaaanaubhauutaishaiila hana, perfect hatae habae naaa

34.14 saarasakassaepa (Summary)

aai adhayaaayae aamaraaa shaikhaechhai:

1. ****Daily Task System**** - daainaika kaaajaera saisataema taairai karaaa
2. ****Weekly Sprint**** - saaapataaahaika Sprint Methodology
3. ****Monthly Milestone**** - maaasaika Milestone Planning

4. ****Backlog Management**** - kaaajaera taaalaikaaa paraichaaalanaaa
5. ****Kanban Board**** - bhajayauyaaala Task Management
6. ****Git Workflow**** - Version Control kaaushala
7. ****Sample Schedule**** - namaunaaa Weekly Schedule
8. ****Task Prioritization**** - kaaajaera agaraaadhaikaaara nairadhaaarana
9. ****Productivity Tips**** - Productivity baridadhaira kaaushala
10. ****Burnout Prevention**** - Burnout paratairaodha

manae raaakhauna:

System taairai karauna, kainatau saetai maaanauna
Consistency > Intensity

****parabarataii adhayaaaya:**** Chapter 35: Common Failure

PART 35

Chapter 35: COMMON FAILURE

Chapter 35: COMMON FAILURE

saaadhaaarana bayarathataaa

shaekhaara udadaeshaya (Learning Goal)

aaii adhaayaayae aapanai shaikhabaena gaema daebhaelapamaenatae kaena gaema bayaratha haya 10tai saaadhaaarana bayarathataaara kaaarana baishalaessana karabaena, parakarita udaaaharana daekhabaena, aiba parataitai samasayaaara samaaadhaaana aiba paratairaodha kaaushala shaikhabaena

35.1 gaema daebhaelapamaenatae kaena gaema bayaratha haya

bayarathataaara paraisakhayaaana

GAME DEVELOPMENT FAILURE STATISTICS

maota inadaai gaemaera madhayae:

90% gaema profit taairai karae naaa

70% gaema launch aira 1 bachharaera madhayae banadha hayae yaaaya

50% gaema shaurau karaaara para shaessa haya naaa

30% gaema launch aira para 3 maaasaera madhayae banadha haya

Steam-ai paratai maaasae:

500+ natauna gaema release haya

gadae 50 gaemai 100+ reviews paaaya

gadae 10 gaemai 1000+ reviews paaaya

bayarathataaara mauula kaaarana:

40% Scope samasayaaa

25% baaajaaara gabaessanaaara abhaaaba

20% parayaukataigata samasayaaa

15% anayaaanaya kaaarana

bayarathataaara dhaaapa

FAILURE PROGRESSION

Phase 1: CONCEPTION

bhaula: khaaaraaapa dhaaarananaa naiyae shaurau karaaa

phalaaaphala: shaurau thaekaei samasayaaa

Phase 2: PLANNING

bhaula: paraikalapananaa naaa karaaa baaa khaaaraaapa paraikalapananaa

phalaaaphala: abayabasathaaapaita daebhaelapamaenata

Phase 3: PRODUCTION

bhaulta: Scope Creep, Burnout, Technical Debt

phalaaaphala: samaya aiba baaajaeta ataikarama

Phase 4: POLISH

bhaulta: parayaaapata polish naaa karaaa

phalaaaphala: khaaaraaapa parathama impression

Phase 5: LAUNCH

bhaulta: khaaaraaapa marketing, bhaulta samayae launch

phalaaaphala: kaeu daekhae naaa

Phase 6: POST-LAUNCH

bhaulta: saaapaorata naaa daeayyaaa

phalaaaphala: Negative reviews, community loss

35.2 Failure Reason 1: Scope Too Large

bayaaakhayaaa

Scope Too Large = gaemaera janaya atairaikata phaichaaara aiba kanataenata paraikalapanaaa

parakarita udaaaharana

EXAMPLE: STAR CITIZEN

gaema: Star Citizen

Developer: Cloud Imperium Games

samayakaaala: 2012 - baratamaana (14+ bachhara)

baajaeta: \$700 mailaiyana+ (karaaaudaphaaanadai)

samasayaaa:

mauula gaemaera scope baaadatae thaaakaechhae

natauna phaichaaara parataidaina yaoga hachachhae

14 bachhara paraeau pauurana gaema release hayana

Players asanataussata

shaekhaara paaatha:

Scope nairadhhaarana karauna aiba saetai maenae chalauna

Feature Freeze saeta karauna

MVP (Minimum Viable Product) daiyae shaurau karauna

samaadhaana

Scope Too Large aira samaadhaana:

1. MVP APPROACH

sabachaeyae gaurautabapauurana phaichaaara chahanaita karauna

shaudhau saegaulao karauna

baaakaigaulao "Nice to Have" haisaebae raaakhauna

2. FEATURE PRIORITIZATION

MoSCoW Method bayabahaaara karauna

Must Have, Should Have, Could Have, Won't Have

Must Have chhaaadaaa anaya kaichhau karabaena naaa

3. SCOPE FREEZE

GDD phaaainaaalaaaija karauna

taaarapara scope freeze karauna

natauna phaichaaara parae yaoga karauna

4. TIME-BOXING

parataitai phaichaaaraera janaya samayasaiimaaa saeta karauna
 samaya shaessa halae phaichaaara shaessa naaa halaeau aigaiyae yaaana
 Iteration ai phaerata aasauna

paratairaodha

Prevention:

1. chhaota daiyae shaurau karauna
2. GDD tae scope limit saeta karauna
3. Weekly scope review karauna
4. "No" balatae shaikhauna
5. Team members kae scope maaanatae balauna

35.3 Failure Reason 2: Bad Design

bayaaakhayaaa

Bad Design = gaemaera gameplay, mechanics, aiba user experience khaaaraapabhaaabaie daijaa

parakarita udaaaharana

EXAMPLE: BALAN WONDERWORLD

gaema: Balan Wonderworld
 Developer: Arzest
 Publisher: Square Enix
 samayakaaala: 2021
 baaajaeta: ajanyaaata (bada baaajaeta)
 samasayaaa:
 aikai samayae aikatai baaatana chaaapaaanaora matao sarala gameplay
 samasata kassamataaa aikatai baaatanae naiyanataraita
 kaonao strategic depth naei
 Platforming controls khaaaraaapa
 Review Score: 1/10 (multiple outlets)
 shaekhaara paaatha:
 Gameplay mechanics gabhaiirabhaaabaie bhaaabauna
 Playtesting karauna
 Player feedback naina

samaaadhaaana

Bad Design aira samaaadhaaana:

1. PLAYER-CENTERED DESIGN
 palaeyaaaraera darissataikaona thaekae bhaaabauna
 karii majaaa laaagabae saetai baibaechanaaa karauna
 palaeyaaaraera paratayaaashaaa baujhauna
2. PROTOTYPING
 chhaota prototype taairai karauna
 paraiikassaaa karauna
 paraibaratana karauna
3. PLAYTESTING
 anaya maaanaussakae khaelatae daina
 Feedback naina
 karauna

4. DESIGN DOCUMENTATION
 - GDD taairai karauna
 - Mechanics laikhauna
 - Iteratively improve karauna

paratairaodha

Prevention:

1. gaema daijaaaaina shaikhauna
2. anaya gaema khaelauna aiba baishalaessana karauna
3. Playtesting early aiba often karauna
4. Feedback garahana karauna
5. Iterate karauna

35.4 Failure Reason 3: No Market Research

bayaaakhayaaa

No Market Research = baaajaaara gabaessanaaa naaa karae gaema taairai karaaa

parakarita udaaaharana

EXAMPLE: HYENAS (Cancelled)

gaema: Hyenas
Developer: Creative Assembly
Publisher: Sega
samayakaaala: 2022-2023 (Cancelled)
samasayaaa:
Extraction shooter genre ai parabaesha karala
Tarkov, DMZ aira matao competitors aachhae
kaonao unique selling point naei
Market timing bhaula
shaessae game cancelled halao
shaekhaaara paaatha:
baaajaaara gabaessanaaa karauna
Competitors baishalaessana karauna
Unique selling point khaujauna

samaaadhaaana

Market Research aira samaaadhaaana:

1. COMPETITIVE ANALYSIS
 - aikai genre aira gaema khaujauna
 - taaadaera saphalataaa aiba bayarathataaa baishalaessana karauna
 - taaadaera thaekae shaikhauna
 - Gap khaujauna
2. AUDIENCE ANALYSIS
 - kaaara janaya gaema baaanaachachhaena saetai nairadhaaarana karauna
 - taaadaera pachhanada/apachhanada baujhauna
 - taaadaera baaajaeta baujhauna
 - taaadaera platform baujhauna
3. TREND ANALYSIS
 - baratamaaana trends daekhauna

Future trends anaumaaana karauna
saei anauyaaayaii paraikalapanaaa karauna

4. VALIDATION

Concept validation karauna
Market validation karauna
Pre-launch ai feedback naina

paratairaodha

Prevention:

1. gaema taairaira aagae market research karauna
2. Steam, itch.io ai similar games daekhauna
3. Gaming communities ai research karauna
4. Surveys paaathaaana
5. Competitors kae study karauna

35.5 Failure Reason 4: Poor Optimization

bayaaakhayaaa

Poor Optimization = gaemaera performance khaaaraaapa haauyaaa, lag, crash, aiba low FPS

parakarita udaaaharana

EXAMPLE: CYBERPUNK 2077 (Launch)

gaema: Cyberpunk 2077
Developer: CD Projekt Red
samayakaaala: 2020 Launch
samasayaaa (PlayStation 4/Xbox One):
paraaayai crash hatao
Frame rate 10-15 FPS ai naemae yaetao
Texture loading samaya baeshai naeauyaaatao
AI kaaaja karata naaa
PlayStation Store thaekae remove karaaa halao
parainaaama:
CDPR aira stock price 75% kamae gaelao
lakassa lakassa refund halao
bachharaera para bachhara patch karatae halao
Reputation kassataigarasata halao
shaekharaa paaatha:
Early optimization karauna
Target platform ai test karauna
Performance budget saeta karauna

samaaadhaaana

Optimization aira samaaadhaaana:

1. PROFILING
parataitai dhaaapae profile karauna
Bottlenecks khaujauna
saegaulao fix karauna
2. PERFORMANCE BUDGET
Target FPS saeta karauna (e.g., 60 FPS)

Frame time budget saeta karauna (16.67ms)
parataitai system ai budget baraaadada karauna

3. ASSET OPTIMIZATION

Texture sizes optimize karauna
Polygon count control karauna
LOD (Level of Detail) bayabahaaara karauna
Object pooling bayabahaaara karauna

4. CODE OPTIMIZATION

Profiler bayabahaaara karauna
Memory leaks fix karauna
Unnecessary calculations avoid karauna
Caching bayabahaaara karauna

5. PLATFORM TESTING

parataitai target platform ai test karauna
Minimum specs ai test karauna
Performance regression test karauna

paratairaodha

Prevention:

1. Shaurau thaekaei optimization manae raaakhauna
2. Regular profiling karauna
3. Performance budget saeta karauna
4. Target platform ai early test karauna
5. Optimization aira janaya time raaakhauna

35.6 Failure Reason 5: No Testing

bayaaakhayaaa

No Testing = parayaaapata paraiikassaaa naaa karae launch karaaa

parakarita udaaaharana

EXAMPLE: ANTHEM

gaema: Anthem
Developer: BioWare
Publisher: Electronic Arts
samayakaaala: 2019
samasayaaa:
asakhaya bugs launch ai chhaila
Server issues chhaila
Loading screens chhaila
loot system chhaila
Content chhaila
"Anthem Next" cancelled halao
parainaaama:
BioWare aira reputation kassataigarasata
EA aira baishabaaasa haaaraaalao
Players chalaе gaela
Game essentially dead
shaekhaaara paaatha:
parayaaapata testing karauna
Beta testing karauna

QA team raaakhauna

samaaadhaana

Testing aira samaaadhaana:

1. TESTING PYRAMID
 - Unit Tests: chhaota chhaota components test karauna
 - Integration Tests: components aikasaaathae test karauna
 - System Tests: paurao saisataema test karauna
 - Acceptance Tests: user perspective thaekae test karauna
2. TESTING TYPES
 - Functional Testing: phaichaaara kaaaja karachhae kainaaa
 - Performance Testing: speed, memory, CPU usage
 - Compatibility Testing: different platforms
 - Usability Testing: user experience
 - Regression Testing: aagaera bugs phairae aisaechhae kainaaa
 - Playtesting: real players daiyae test
3. TESTING TOOLS
 - Automated Testing Framework
 - Bug Tracking System
 - Performance Profiler
 - Memory Profiler
 - Platform-specific tools
4. TESTING SCHEDULE
 - parataitai sprint ai test karauna
 - Regular regression test karauna
 - Beta test karauna
 - Launch aira aagae comprehensive test karauna

paratairaodha

Prevention:

1. Testing culture taaairai karauna
2. Early testing shaurau karauna
3. Automated testing bayabahaaara karauna
4. External testers naina
5. Bug tracking system bayabahaaara karauna

35.7 Failure Reason 6: Weak Marketing

bayaaakhayaaa

Weak Marketing = gaemaera parachaaarananaa daurabala haauyaaa

parakarita udaaaharana

EXAMPLE: MANY INDIE GAMES

samasayaaa:

Steam-ai paratai maaasae 500+ gaema release haya
 baeshairabhaaaga inadai gaemaera kaonao marketing naei
 Trailer naei, screenshots naei
 Social media presence naei

Launch dainae kaeu jaaanae naaa gaematai kai
phalaaaphala:
Steam-ai visibility paaaya naaa
Reviews paaaya naaa
Revenue paaaya naaa
Game effectively invisible
shaekhhaara paaatha:
Marketing early shaurau karauna
Community building karauna
Steam page early saeta aapa karauna

samaaadhaana

Marketing aira samaaadhaana:

1. PRE-LAUNCH MARKETING (6 maaasa aagae)
 - Steam page saeta aapa karauna
 - Wishlists sagaraha karauna
 - Social media shaurau karauna
 - Trailer taaairai karauna
 - Press kit taaairai karauna
2. COMMUNITY BUILDING
 - Discord server taaairai karauna
 - Twitter/Reddit presence taaairai karauna
 - Devlogs laikhauna
 - Demo release karauna
3. LAUNCH MARKETING
 - Launch day plan karauna
 - Press outreach karauna
 - Streamers kae contact karauna
 - Social media campaign chaaalaaana
4. POST-LAUNCH MARKETING
 - Updates daina
 - Community engagement raaakhauna
 - Sale events ai participate karauna
 - Word of mouth encourage karauna
5. MARKETING MATERIALS
 - High-quality screenshots
 - Professional trailer
 - Press kit
 - Store page description
 - Social media assets

paratairaodha

Prevention:

1. Day 1 thaekae marketing shaurau karauna
2. Community building aira janaya time raaakhauna
3. Marketing budget raaakhauna
4. Marketing plan taaairai karauna
5. Regular content post karauna

35.8 Failure Reason 7: Poor Production Planning

bayaaakhayaaa

Poor Production Planning = upaadana paraikalapanaaa daurabala haauyaaa

parakarita udaaaharana

EXAMPLE: DUKE NUKEM FOREVER

gaema: Duke Nukem Forever
Developer: 3D Realms / Gearbox
samayakaaala: 1997-2011 (14 bachhara!)
samasayaaa:
 kaonao production plan chhaila naaa
 Scope baaarabaaara paraibaratana halao
 Technology baaarabaaara paraibaratana halao
 Team members chalaе gaela
 14 bachhara para khaaaraaapa gaema haisaebae release halao
shaekhaaara paaatha:
 Production plan taairai karauna
 Milestones saeta karauna
 Regular reviews karauna

samaaadhaana

Production Planning aira samaaadhaana:

1. PRODUCTION PIPELINE
 - Pre-production: GDD, Art Bible, Tech Design
 - Production: Core development, Content creation
 - Polish: Bug fixing, Optimization, Polish
 - Launch: Marketing, Release, Post-launch
2. MILESTONE PLANNING
 - chhaota chhaota milestones saeta karauna
 - parataitai milestone aira deliverables nairadhaaarana karauna
 - Timeline saeta karauna
 - Regular reviews karauna
3. RESOURCE PLANNING
 - Team members aira skills baujhauna
 - Time allocation karauna
 - Budget planning karauna
 - Tools and software saeta aapa karauna
4. RISK MANAGEMENT
 - Risks identify karauna
 - Mitigation strategies taairai karauna
 - Contingency plans raaakhauna
 - Regular risk review karauna

paratairaodha

Prevention:

1. Production plan aagae thaekae taairai karauna
2. Milestones saeta karauna
3. Regular status meetings karauna
4. Progress tracking karauna
5. Flexible planning raaakhauna

35.9 Failure Reason 8: Feature Creep

bayaaakhayaaa

Feature Creep = dhairae dhairae atairaikata phaichaaara yaoga karaaa yaaa mauula paraika

parakarita udaaaharana

EXAMPLE: BROKEN AGE

gaema: Broken Age
 Developer: Double Fine Productions
 samayakaaala: 2012-2015 (Kickstarter)
 samasayaaa:
 Kickstarter-ai \$3.3 mailaiyana paelao
 mauula scope bada chhaila
 dhairae dhairae aaraau phaichaaara yaoga halao
 Act 1 shaessa halao, kainatau Act 2 aira janaya baaajaeta kamae gaelao
 shaessae dautai aalaaadaaa game haisaebae release halao
 Act 2 chhaota aiba incomplete chhaila
 shaekhaara paaatha:
 Scope control karauna
 Feature freeze karauna
 MVP daiyae shaurau karauna

samaaadhaana

Feature Creep aira samaaadhaana:

1. FEATURE PRIORITIZATION
 - MoSCoW method bayabahaaara karauna
 - Must Have, Should Have, Could Have, Won't Have
 - Must Have chhaadaaa anaya kaichhau karabaena naaa
2. FEATURE FREEZE
 - GDD phaaaanaalaaaija karauna
 - taaarapara feature freeze karauna
 - natauna phaichaaara "version 2" aira janaya raaakhauna
3. CHANGE CONTROL
 - parataitai natauna phaichaaaraera janaya approval parayaojana
 - Impact analysis karauna
 - Cost-benefit analysis karauna
 - Decision documentation karauna
4. MVP APPROACH
 - sabachaeyae gaurautabapaurana phaichaaara chahanaita karauna
 - shaudhau saegaulao karauna
 - baaakaigaulao "Nice to Have" haisaebae raaakhauna

paratairaodha

Prevention:

1. Clear scope definition
2. Regular scope reviews
3. Change control process
4. Team discipline
5. "No" culture develop karauna

35.10 Failure Reason 9: Burnout

bayaaakhayaaa

Burnout = daiiraghamaeyaaadai sataraesera phalae maaanasaika aiba shaaaraiiraika kalaaana

parakarita udaaaharana

EXAMPLE: TERRY CAVANEH (Cave Story)

Developer: Daisuke "Pixel" Amaya

gaema: Cave Story

samayakaaala: 2004-2006 (5 bachhara solo development)

abhaijanyataaa:

5 bachhara solo development karaechhaena

paraaayai 20+ ghanataaa kaaaja karataena

paraaayai ghauma kamaataena

Burnout anaubhaba karaechhaena

shaessae sabaaasathaya samasayaaa hayaechhae

shaekhaara paaatha:

Sustainable pace maintain karauna

Breaks naina

Health priority daina

samaaadhaana

Burnout aira samaaadhaana:

1. SUSTAINABLE PACE

parataidaina 6-8 ghanataaa kaaaja karauna (baeshai naya)

sapataaahae 1 daina pauraopurai baisharaaama naina

maaasae 2-3 daina pauraopurai baisharaaama naina

Break samayae kaaaja samaparakae bhaaababaena naaa

2. BOUNDARIES

kaaaja aiba bayakataigata jaiibana aalaaadaaa raaakhauna

Work hours nairadhaaarana karauna

Weekend ai kaaaja karabaena naaa (baaa khauba kama)

Vacation naina

3. SELF-CARE

parayaaapata ghauma naina (7-8 ghanataaa)

Exercise karauna

Balanced diet khaaana

Friends/Family aira saaathae samaya kaaataaana

Hobbies raaakhauna (gaema chhaaadaaa)

4. SCOPE MANAGEMENT

Scope kamaiyae aanauna

Priorities paunaranairadhaaarana karauna

Help naina

Timeline extend karauna

5. MENTAL HEALTH

Stress chaihanaite karauna

parayaojanae saaahaaayaya naina

Mindfulness/ Meditation anaushaailana karauna

Positive self-talk karauna

paratairaodha

Prevention:

1. Sustainable pace daiyae shaurau karauna
2. Regular breaks naina
3. Health monitoring karauna
4. Support system taairai karauna
5. Warning signs chaihanaaita karauna

35.11 Failure Reason 10: Budget Problem

bayaaakhayaaa

Budget Problem = parayaaapata baaajaeta naaa thaaakaaa baaa baaajaeta apachaya karaaa

parakarita udaaaharana

EXAMPLE: FORTNITE SAVE THE WORLD

gaema: Fortnite - Save the World

Developer: Epic Games

samayakaaala: 2011-2017 (6 bachhara development)

samasayaaa:

\$100 mailaiyana+ development cost

6 bachhara development time

Early Access ai release halao

maula game concept paraibaratana halao

Battle Royale mode yaoga halao

Save the World mode gaauna hayae gaelao

shaekhhaara paaatha:

Budget tracking karauna

Cost control karauna

Revenue planning karauna

samaaadhaaana

Budget Problem aira samaaadhaaana:

1. BUDGET PLANNING
 - Detailed budget taairai karauna
 - Categories bhaaaga karauna (Personnel, Software, Marketing, etc.)
 - Buffer budget raaakhauna (20-30%)
 - Regular budget review karauna
2. COST CONTROL
 - parataitai kharacha track karauna
 - Unnecessary expenses kamaaana
 - Free tools/software bayabahaaara karauna
 - Outsource vs in-house cost comparison karauna
3. REVENUE PLANNING
 - Sales projections taairai karauna
 - Multiple revenue streams plan karauna
 - Funding options khaujauna (Kickstarter, Grants, Publishers)
 - Pre-sales aira plan karauna
4. COST OPTIMIZATION

Asset stores bayabahaaara karauna
 Open source tools bayabahaaara karauna
 Freelancers bayabahaaara karauna
 Remote work optimize karauna

paratairaodha

Prevention:

1. Realistic budget taairai karauna
2. Regular tracking karauna
3. Emergency fund raaakhauna
4. Cost-benefit analysis karauna
5. Revenue diversification plan karauna

35.12 Recovery Strategies

bayarathataaa thaekae Recovery

RECOVERY STRATEGIES

1. ASSESSMENT
 - samasayaatai kii saetai chaihanaite karauna
 - kaaarana baishalaessana karauna
 - Impact assessment karauna
 - Options taairai karauna
2. SCOPE REDUCTION
 - gaurautabapaurana phaichaaara chaihanaite karauna
 - Optional features cut karauna
 - MVP ai phairae yaaana
 - "Version 2" aira janaya features save karauna
3. RESOURCE OPTIMIZATION
 - Team restructure karauna
 - Tools optimize karauna
 - Outsourcing consider karauna
 - Remote work bayabahaaara karauna
4. TIMELINE ADJUSTMENT
 - Timeline extend karauna
 - Milestones reprioritize karauna
 - Buffer time add karauna
 - Launch date push karauna
5. MARKET REVALIDATION
 - Market research paunaraaya karauna
 - Audience feedback naina
 - Competitors analyze karauna
 - Pivot if necessary

35.13 Exercise

anaushailana 1: naijaera parajaekataera Risk Assessment

1. aapanaaara gaema daebhaelapamaenata parajaekataera janaya 10tai samabhaaabaya samasayaaa laikhauna
2. parataitaira janaya Probability aiba Impact nairadhaaarana karauna
3. Mitigation strategies taairai karauna

4. Prevention plan taairai karauna

anaushailana 2: Failure Analysis

1. aikatai bayaratha gaema khaujauna (Steam ai negative reviews daekhauna)
2. bayarathataaara kaaarana baishalaessana karauna
3. kaiibhaaabaie aidaaanao yaeta saetai laikhauna
4. aapanaaara parajaekatae kaiibhaaabaie apply karabaena saetai barananaaa karauna

anaushailana 3: Recovery Plan

1. aikatai kaaalapanaika samasayaaa taairai karauna (e.g., "aamaaara gaemaera scope 3 gauna baedae gaechhae")
2. Recovery strategy taairai karauna
3. Step-by-step plan laikhauna
4. Timeline adjust karauna

35.14 saaadhaaarana bhaula (Common Mistakes)

bhaula 1: Warning Signs Ignore karaaa

bhaula: samasayaaara lakassana daekhaeau udaasaaiina thaaakaaa
sathaika: satarakataaa chaihanaite karauna, darauta parataikaaara karauna

bhaula 2: Perfectionism

bhaula: paaaraphaekata hatae chaessataaa karaaa
sathaika: "Good enough" daiyae aigaiyae yaaana, iterate karauna

bhaula 3: Isolation

bhaula: aikaaa saba kaichhau karaaara chaessataaa karaaa
sathaika: saaahaaayaya naina, community tae yaoga daina

bhaula 4: No Learning

bhaula: bhaula thaekae shaikhaaa naaa yaaaauyaaa
sathaika: Post-mortem karauna, shaikhauna, improve karauna

bhaula : Giving Up Too Early

bhaula: parathama samasayaaaya haaala chhaedae daeauyaaa
sathaika: Challenges sabaaabhaaabaika, persevere karauna

35.15 taipasa (Tips)

taipasa 1: Learn from Others

anayadaera bhaula thaekae shaikhauna, naijae karae daekhabaena naaa

taipasa 2: Early Detection

samasayaaa yata taaadaaataaadaai dharaaa yaaaya, tata sahajae samaaadhaaana haya

taipasa 3: Have Backup Plans

sabasamaya Plan B raaakhauna

taipasa 4: Stay Humble

naijaekae sarabajanya manae karabaena naaa, shaikhatae thaaakauna

taipasa 5: Celebrate Small Wins

chhaota chhaota saphalataaa udayaaapana karauna, aita motivation thaaakae

35.16 saarasakassaepa (Summary)

aii adhayaaayae aamaraaa shaikhaechhai:

1. ****Scope Too Large**** - atairaikata scope aidaiyae chalauna
2. ****Bad Design**** - parayaaapata playtesting karauna
3. ****No Market Research**** - baaajaaara gabaessanaaa karauna
4. ****Poor Optimization**** - early optimization karauna
5. ****No Testing**** - parayaaapata testing karauna
6. ****Weak Marketing**** - early marketing shaurau karauna
7. ****Poor Production Planning**** - paraikalapananaaa karauna
8. ****Feature Creep**** - scope control karauna
9. ****Burnout**** - sustainable pace maintain karauna
10. ****Budget Problem**** - budget tracking karauna

manae raaakhauna:

bayarathataaa shaekhaaara sauyaoga
parataitai bhaula thaekae shaikhauna, improve karauna

****parabarataii adhayaaaya:**** Chapter 36: Case Studies

PART 36

Chapter 36: CASE STUDIES

Chapter 36: CASE STUDIES

kaesa sataaadaija

shaekhaara udadaeshaya (Learning Goal)

aai adhayaaayae aapanai 10tai saphala inadai gaemaera baisataaaraita baishalaessana daekhabaena parataitai gaemaera Development Approach, Unique Idea, Marketing Strategy, Launch Details, aiba Success Factors samaparakaе shaikhabaena aai kaesa sataaadaija thaekae aapanaaara naijaera gaema daebhaelapamaenatae kaiibhaaabaе apply karatae paaaraena saetai shaikhabaena

36.1 Case Study 1: Stardew Valley

gaema tathaya

STARDEW VALLEY

Game Name: Stardew Valley
Developer: ConcernedApe (Eric Barone)
Genre: Farming Simulation / RPG
Platform: PC, Switch, PS4, Xbox, Mobile
Release: February 2016
Development Time: 4.5 years (2012-2016)
Team Size: 1 person (Eric Barone)
Engine: MonoGame/XNA
Budget: \$0 (Self-funded)
Revenue: \$100+ Million (Lifetime)
Reviews: 250,000+ (Steam, 97% Positive)

Development Approach

Eric Barone aira Development Approach:

1. SELF-TAUGHT

kaonao formal education chhaila naaa game development ai
Online tutorials daiyae shaikhaechhaena
parataitai skill (programming, art, music) naijaе shaikhaechhaena
4.5 bachhara dharae aikaaa kaaaja karaechhaena

2. ITERATIVE DEVELOPMENT

parathamae basic prototype taairai karaechhaena
dhaiirae dhaiirae features yaoga karaechhaena
Regular playtesting karaechhaena
Feedback aira bhaitataitae improve karaechhaena

3. PASSION-DRIVEN

naijaera pachhanadaera gaema baaanaiyaechhaena (Harvest Moon aira matao)
parataidaina anatata 8+ ghanataaa kaaaja karaechhaena
kakhanao give up karaenanai
Quality aira paratai committed chhailaena

Unique Idea

Unique Selling Points:

1. NOSTALGIA + INNOVATION

Harvest Moon aira nostalgia dharae raekhaechhaena
natauna features yaoga karaechhaena (Mines, Combat, etc.)
pauraonao aiba natauna players daujanaei enjoy karatae paaarae

2. DEPTH

gabhaiira gameplay mechanics
anaeka baeshai content
Replayability high
Modding support

3. COMMUNITY

Multiplayer yaoga karaechhaena
Modding community develop karaechhaena
Regular updates daiyaechhaena

Marketing Strategy

Marketing Approach:

1. WORD OF MOUTH

parathamae kaonao marketing budget chhaila naaa
Players naijaeraaai recommend karaechhae
Steam forums ai positive buzz taairai hayaechhae
Reviews bhaaalao haauyaaya visibility baedaechhae

2. STREAMER EXPOSURE

Popular streamers khaelaechhaena
Twitch ai viral hayaechhae
YouTube ai playthroughs popular hayaechhae

3. TIMING

February 2016 ai release (Winter, cozy game time)
No major competing releases
Perfect timing for the genre

Launch Details

Launch Timeline:

2012: Development shaurau
2013: First screenshots shared
2014: Greenlight campaign
2015: Early Access release
2016 February: Full release
2016+: Updates, ports, multiplayer

Key Launch Metrics:

- Day 1: Steam Top Sellers
- Week 1: 100,000+ copies sold
- Month 1: 500,000+ copies sold
- Year 1: 1,000,000+ copies sold

What Made It Successful

Success Factors:

1. QUALITY PRODUCT
 - Polished gameplay
 - Deep mechanics
 - Beautiful art style
 - Excellent music
 - Great value for money
2. MARKET FIT
 - Niche market (farming sim)
 - High demand, low supply
 - Perfect timing
 - Nostalgia factor
3. COMMUNITY
 - Active modding community
 - Regular updates
 - Developer communication
 - Fan content creation
4. LONGEVITY
 - Regular content updates
 - New platforms/ports
 - Multiplayer addition
 - Still selling years later

What We Can Learn

Key Takeaways:

1. Solo development possible (with dedication)
2. Quality > Quantity
3. Niche markets can be very profitable
4. Community building is crucial
5. Passion drives success
6. Patience (4.5 years development)
7. Self-taught is possible
8. Word of mouth is powerful
9. Regular updates keep players engaged
10. Value proposition matters

36.2 Case Study 2: Hollow Knight

gaema tathaya

HOLLOW KNIGHT

Game Name: Hollow Knight

Developer: Team Cherry (3 people)

Genre: Metroidvania
Platform: PC, Switch, PS4, Xbox
Release: February 2017
Development Time: 3 years (2014-2017)
Team Size: 3 people (Ari Gibson, William Pellen, David Kazi)
Engine: Unity
Budget: Kickstarter (\$57,000) + Self-funded
Revenue: \$50+ Million (Lifetime)
Reviews: 200,000+ (Steam, 97% Positive)

Development Approach

Team Cherry's Development Approach:

1. SMALL TEAM, BIG VISION
maaataa 3 jana daiyae bada gaema taairai karaechhae
Each member wore multiple hats
Ari: Art + Animation
William: Code + Design
David: Sound + Music
2. KICKSTARTER FUNDING
Kickstarter campaign chaaalaiyaechhae
\$57,000 raised (goal: \$57,000)
Community support paeyaechhae
Backer rewards fulfill karaechhae
3. ITERATIVE DESIGN
Prototype daiyae shaurau karaechhae
Playtesting karaechhae
Feedback aira bhaitataitae improve karaechhae
Scope expand karaechhae (free DLC)

Unique Idea

Unique Selling Points:

1. ATMOSPHERE
Dark, mysterious world
Beautiful hand-drawn art
Immersive sound design
Emotional storytelling
2. EXPLORATION
Large interconnected world
Metroidvania progression
Hidden secrets everywhere
Rewarding exploration
3. CHALLENGE
Difficult combat
Fair difficulty
Sense of accomplishment
Boss fights memorable

Marketing Strategy

Marketing Approach:

1. KICKSTARTER
Kickstarter campaign ai community build karaechhae

Regular updates daiyaechhae
Backers kae engaged raekhaechhae

2. DEMO

Free demo release karaechhae
PAX ai showcase karaechhae
Press coverage paeyaeachhae

3. WORD OF MOUTH

Quality product chhaila
Players recommend karaechhae
Streamers khaelaechhae
Reviews excellent chhaila

Launch Details

Launch Timeline:

2014: Kickstarter campaign
2015: Development updates
2016: Demo release, PAX showcase
2017 February: Full release
2017+: Free DLC (Hidden Dreams, The Grimm Troupe, Godmaster)

Key Launch Metrics:

- Day 1: Steam Top Sellers
- Week 1: 50,000+ copies sold
- Month 1: 250,000+ copies sold
- Year 1: 1,000,000+ copies sold
- Free DLC kept players engaged

What Made It Successful

Success Factors:

1. EXCEPTIONAL QUALITY

Beautiful hand-drawn art
Excellent gameplay
Immersive world
Great value (\$15)

2. METROIDVANIA NICHE

Popular genre
High demand
Limited competition
Perfect execution

3. FREE DLC

3 major free DLCs
Kept players engaged
Extended game life
Built goodwill

4. COMMUNITY

Active fanbase
Modding community
Speedrun community
Fan art/content

What We Can Learn

Key Takeaways:

1. Small team can create masterpiece
2. Kickstarter can validate + fund
3. Quality art is crucial
4. Free DLC builds loyalty
5. Metroidvania is profitable niche
6. Atmosphere matters
7. Challenge can be a selling point
8. Regular updates keep interest
9. Value proposition (\$15 for 40+ hours)
10. Patience and iteration

36.3 Case Study 3: Celeste

gaema tathaya

CELESTE

Game Name: Celeste
 Developer: Maddy Makes Games (Maddy Thorson + Noel Berry)
 Genre: Platformer
 Platform: PC, Switch, PS4, Xbox
 Release: January 2018
 Development Time: ~2 years (2016-2018)
 Team Size: 2 core + contractors
 Engine: MonoGame/XNA
 Budget: Funded by Matt Makes Games Inc.
 Revenue: \$10+ Million (Estimated)
 Reviews: 50,000+ (Steam, 97% Positive)

Development Approach

Celeste aira Development Approach:

1. GAME JAM ORIGIN
 - 2016 Celeste Game Jam version taairai hayaechhaila
 - Game Jam ai concept taairai hayaechhaila
 - Jam version bhaaalao reception paeyaeachhaila
 - Full game haisaebae develop karaaa hayaechhaila
2. COLLABORATION
 - Maddy Thorson: Design + Programming
 - Noel Berry: Art + Level Design
 - External contractors: Music, Sound
 - Small, efficient team
3. ACCESSIBLE DIFFICULTY
 - Assist Mode yaoga karaaa hayaechhaila
 - sabaaara janaya playable
 - Challenge optional
 - Story accessible

Unique Idea

Unique Selling Points:

1. EMOTIONAL STORY

- Mental health themes
- Depression, anxiety representation
- Personal, relatable story
- Positive message

2. ACCESSIBLE DIFFICULTY

- Assist Mode for struggling players
- Optional challenges
- B-Sides, C-Sides for hardcore
- Everyone can enjoy

3. TIGHT CONTROLS

- Precise platforming
- Responsive controls
- Fair difficulty
- Satisfying gameplay

Marketing Strategy

Marketing Approach:

1. INDIE SHOWCASES

- PAX, E3 ai showcase karaaa hayaechhaila
- Press coverage paaaauyaaa gaechhaila
- Demo available karaaa hayaechhaila

2. AWARD SEASON

- The Game Awards 2018 ai Best Indie Game jaitaechhae
- Multiple award nominations
- Awards drove sales

3. WORD OF MOUTH

- Quality product
- Emotional impact
- Players recommending
- Streamer coverage

Launch Details

Launch Timeline:

- 2016: Game Jam version
- 2017: Full development
- 2018 January: Full release
- 2018 November: The Game Awards (Best Indie Game)
- 2019+: DLC (Farewell), Ports

Key Launch Metrics:

- Day 1: Strong Steam sales
- Awards season: Sales spike
- Year 1: 500,000+ copies sold
- Still selling years later

What Made It Successful

Success Factors:

1. EMOTIONAL RESONANCE

- Story touched players
- Mental health representation
- Positive message

- Personal connection
- 2. EXCELLENT GAMEPLAY
 - Tight controls
 - Fair difficulty
 - Rewarding progression
 - High replayability
- 3. ACCESSIBILITY
 - Assist Mode
 - Everyone can play
 - Optional challenges
 - Inclusive design
- 4. AWARDS RECOGNITION
 - Multiple awards
 - Critical acclaim
 - Media coverage
 - Sales boost

What We Can Learn

Key Takeaways:

1. Game jams can lead to full games
2. Emotional storytelling matters
3. Accessibility is important
4. Small teams can succeed
5. Awards drive sales
6. Personal stories resonate
7. Tight gameplay is crucial
8. Difficulty can be optional
9. Collaboration works
10. Quality over quantity

36.4 Case Study 4: Among Us

gaema tathaya

AMONG US

Game Name: Among Us
 Developer: InnerSloth (3 people)
 Genre: Social Deduction
 Platform: PC, Mobile, Switch, PS4, Xbox
 Release: June 2018 (Original), 2020 (Viral)
 Development Time: ~1 year (2017-2018)
 Team Size: 3 people (Marcus Bromander, Forest Willard, Amy Liu)
 Engine: Unity
 Budget: Low (Small indie team)
 Revenue: \$500+ Million (Lifetime, Mobile + PC)
 Reviews: 500,000+ (Steam, 95% Positive)

Development Approach

InnerSloth's Development Approach:

1. SMALL TEAM

maaatarara 3 jana
Marcus: Art + Design
Forest: Programming
Amy: Programming + QA

2. SIMPLE CONCEPT

Social deduction gameplay
Easy to learn
Hard to master
Party game appeal

3. MOBILE-FIRST

Mobile-first design
Free-to-play model
In-app purchases
Cross-platform later

Unique Idea

Unique Selling Points:

1. SOCIAL DEDUCTION

Mafia/Werewolf style gameplay
Digital adaptation
Voice chat integration
Perfect for groups

2. ACCESSIBILITY

Free on mobile
Simple controls
Short game sessions
Anyone can play

3. CONTENT CREATORS

Streamer-friendly
Great for content
Viral potential
Community-driven

Marketing Strategy

Marketing Approach:

1. INITIAL RELEASE (2018)

Mobile app stores
Small marketing budget
Moderate success
Some streamer coverage

2. VIRAL MOMENT (2020)

Korean streamers discovered game
English streamers followed
COVID-19 lockdown timing
Perfect for quarantine gaming
Explosive growth

3. CONTENT CREATOR FOCUS

Streamers loved it
YouTube videos viral
Free marketing
Community-driven growth

Launch Details

Launch Timeline:

2018 June: Mobile release (iOS/Android)
2018 August: PC release (Steam)
2018-2019: Moderate success
2020 July: Viral explosion
2020 September: 100M+ downloads
2021: Console ports, updates

Key Metrics:

- 2018: 1M+ downloads
- 2020: 100M+ downloads
- Peak: 500M+ players worldwide
- Revenue: \$500M+ (Mobile + PC)

What Made It Successful

Success Factors:

1. PERFECT TIMING
 - COVID-19 lockdown
 - People needed social games
 - gaming demand
 - Perfect storm
2. STREAMER APPEAL
 - Great for content
 - Social deduction = drama
 - Easy to stream
 - Free marketing
3. ACCESSIBILITY
 - Free on mobile
 - Simple to learn
 - Works on low-end devices
 - Cross-platform
4. SOCIAL ASPECT
 - Play with friends
 - Voice chat
 - Party game
 - Community building

What We Can Learn

Key Takeaways:

1. Timing matters (COVID-19)
2. Streamers are powerful marketing
3. Simple concepts can be huge
4. Mobile-first can work
5. Social games are popular
6. Free-to-play can be profitable
7. Small teams can create hits
8. Community-driven growth
9. Cross-platform is important
10. Content creator focus

36.5 Case Study 5: Hades

game tathaya

HADES

Game Name: Hades
 Developer: Supergiant Games (20 people)
 Genre: Roguelike / Action RPG
 Platform: PC, Switch, PS4, PS5, Xbox
 Release: September 2020 (1.0), Early Access 2018
 Development Time: ~3 years (2017-2020)
 Team Size: ~20 people
 Engine: Proprietary
 Budget: Funded by Supergiant Games
 Revenue: \$50+ Million (Estimated)
 Reviews: 200,000+ (Steam, 97% Positive)

Development Approach

Supergiant Games aira Development Approach:

1. EARLY ACCESS MODEL
 - 2018 air Early Access air release karaechhae
 - Community feedback naiyaechhae
 - Iteratively improved
 - 2020 air 1.0 release karaechhae
2. EXPERIENCED TEAM
 - Previously made Bastion, Transistor, Pyre
 - Proven track record
 - Professional team
 - Quality pipeline
3. INNOVATIVE NARRATIVE
 - Roguelike + Story integration
 - Greek mythology
 - Character relationships
 - Narrative progression

Unique Idea

Unique Selling Points:

1. ROGUELIKE + STORY
 - Roguelike genre air narrative integration
 - Death = story progression
 - Character relationships
 - Emotional depth
2. GREEK MYTHOLOGY
 - Popular theme
 - Rich characters
 - Epic story
 - Beautiful art
3. POLISH
 - Excellent animation
 - Tight controls
 - Great music

Professional quality

Marketing Strategy

Marketing Approach:

1. EARLY ACCESS
 - 2018 Early Access release
 - Community feedback
 - Regular updates
 - Word of mouth
2. GAME AWARDS
 - 2020 The Game Awards
 - Won Best Game Direction
 - Won Best Narrative
 - Won Best Action Game
 - Huge visibility boost
3. CRITICAL ACCLAIM
 - 97% positive on Steam
 - Multiple awards
 - Media coverage
 - Best of 2020 lists

Launch Details

Launch Timeline:

2017: Development announced
2018 November: Early Access release
2019-2020: Regular updates
2020 September: 1.0 release
2020 November: Game Awards
2021: Console ports, DLC

Key Metrics:

- Early Access: Strong community
- 1.0 Release: 70,000+ concurrent
- Year 1: 1,000,000+ copies sold
- Awards: 50+ awards

What Made It Successful

Success Factors:

1. EARLY ACCESS SUCCESS
 - Community feedback valuable
 - Iterative improvement
 - Built anticipation
 - Quality product
2. NARRATIVE INNOVATION
 - Roguelike + story
 - Death as progression
 - Character depth
 - Emotional investment
3. PROFESSIONAL QUALITY
 - Excellent animation
 - Tight gameplay

- Great audio
 - Polish level
4. AWARDS RECOGNITION
- Multiple awards
 - Critical acclaim
 - Media coverage
 - Sales boost

What We Can Learn

- Key Takeaways:
1. Early Access can work
 2. Community feedback is valuable
 3. Narrative innovation matters
 4. Professional quality is crucial
 5. Awards drive sales
 6. Roguelike genre is popular
 7. Character depth adds value
 8. Iterative development works
 9. Greek mythology appeals
 10. Polish level matters

36.6 Case Study 6: Undertale

gaema tathaya

UNDERTALE
Game Name: Undertale
Developer: Toby Fox (1 person)
Genre: RPG
Platform: PC, Switch, PS4, PS5, Mobile
Release: September 2015
Development Time: 2.5 years (2013-2015)
Team Size: 1 person (Toby Fox) + contractors
Engine: GameMaker Studio
Budget: \$50,000 (Kickstarter)
Revenue: \$50+ Million (Lifetime)
Reviews: 250,000+ (Steam, 96% Positive)

Development Approach

- Toby Fox aira Development Approach:
1. KICKSTARTER
 - 2013 ai Kickstarter campaign
 - \$50,000 raised
 - Community support
 - Backer engagement
 2. SOLO DEVELOPMENT
 - aikaaa programming
 - aikaaa design
 - aikaaa writing
 - Music composition

3. UNIQUE DESIGN

- Bullet hell + RPG
- Moral choices
- Meta-narrative
- Player agency

Unique Idea

Unique Selling Points:

1. META-NARRATIVE

- Game aware of player actions
- Save/load affects story
- Moral choices matter
- Multiple endings

2. CHARACTERS

- Memorable characters
- Emotional depth
- Humor + Drama
- Player attachment

3. GAMEPLAY INNOVATION

- Bullet hell dodge mechanics
- Peaceful resolution options
- Unique combat system
- Player choice focused

Marketing Strategy

Marketing Approach:

1. KICKSTARTER

- Kickstarter campaign
- Community building
- Backer updates
- Word of mouth

2. INDIE COMMUNITY

- Reddit presence
- Twitter engagement
- Fan content
- Community-driven

3. VIRAL MOMENTS

- Streamers played it
- YouTube playthroughs
- Memes created
- Cultural impact

Launch Details

Launch Timeline:

- 2013: Kickstarter campaign
- 2013-2015: Development
- 2015 September: PC release
- 2016: Console ports
- 2018: Switch release
- 2021: Deltarune chapters

Key Metrics:

- Kickstarter: \$50,000 raised
- Day 1: Strong sales
- Year 1: 1,000,000+ copies
- Lifetime: 5,000,000+ copies
- Cultural phenomenon

What Made It Successful

Success Factors:

1. NARRATIVE EXCELLENCE
 - Unique story
 - Memorable characters
 - Emotional impact
 - Multiple playthroughs
2. GAMEPLAY INNOVATION
 - Unique combat system
 - Player choice
 - Peaceful options
 - Replayability
3. COMMUNITY
 - Fan content explosion
 - Memes
 - Speedruns
 - Cultural impact
4. CULTURAL PHENOMENON
 - References in other games
 - Fan art explosion
 - Music popularity
 - Lasting impact

What We Can Learn

Key Takeaways:

1. Kickstarter can fund + build community
2. Solo development possible
3. Unique gameplay matters
4. Narrative depth is crucial
5. Community drives success
6. Cultural impact possible
7. Memorable characters matter
8. Player agency is important
9. Replayability adds value
10. Indie can become mainstream

36.7 Case Study 7: Cuphead

gaema tathaya

CUPHEAD

Game Name: Cuphead

Developer: Studio MDHR (10-20 people)

Genre: Run and Gun / Boss Rush
Platform: PC, Switch, PS4, Xbox, Mobile
Release: September 2017
Development Time: ~4 years (2013-2017)
Team Size: 10-20 people
Engine: Unity
Budget: Self-funded + Microsoft deal
Revenue: \$100+ Million (Lifetime)
Reviews: 100,000+ (Steam, 96% Positive)

Development Approach

Studio MDHR's Development Approach:

1. ART-DRIVEN
 - 1930s cartoon style
 - Hand-drawn animation
 - Traditional techniques
 - Unique visual identity
2. DIFFICULTY FOCUS
 - Challenging boss fights
 - Fair difficulty
 - Skill-based gameplay
 - Satisfaction from mastery
3. INDUSTRY PARTNERSHIP
 - Microsoft partnership
 - Xbox console exclusive
 - Marketing support
 - Distribution deal

Unique Idea

Unique Selling Points:

1. VISUAL STYLE
 - 1930s cartoon aesthetic
 - Hand-drawn animation
 - Unique look
 - Nostalgia appeal
2. DIFFICULTY
 - Challenging gameplay
 - Boss rush format
 - Skill-based
 - Accomplishment feeling
3. CO-OP
 - 2-player co-op
 - Fun with friends
 - Party game potential
 - Shared experience

Marketing Strategy

Marketing Approach:

1. E3 REVEAL
 - 2015 E3 Xbox conference
 - Stunning trailer

- Immediate buzz
- Press coverage
- 2. MICROSOFT PARTNERSHIP
 - Xbox exclusive marketing
 - Game Pass inclusion
 - Marketing support
 - Distribution
- 3. VISUAL APPEAL
 - Screenshots viral
 - Art style unique
 - Social media sharing
 - Press coverage

Launch Details

Launch Timeline:

2013: Development started
2015: E3 reveal
2016: Release date announced
2017 September: PC/Xbox release
2018: Switch release
2019: PS4 release
2020: DLC announced

Key Metrics:

- E3 2015: Immediate buzz
- Day 1: Xbox Top Sellers
- Year 1: 3,000,000+ copies
- Lifetime: 10,000,000+ copies
- Netflix show announced

What Made It Successful

Success Factors:

1. VISUAL EXCELLENCE
 - Unique art style
 - Hand-drawn quality
 - Nostalgia appeal
 - Social media friendly
2. GAMEPLAY QUALITY
 - Challenging but fair
 - Satisfying combat
 - Boss variety
 - Skill-based
3. PARTNERSHIP
 - Microsoft deal
 - Marketing support
 - Distribution
 - Game Pass
4. CULTURAL IMPACT
 - Netflix show
 - Merchandise
 - Brand expansion
 - Lasting appeal

What We Can Learn

Key Takeaways:

1. Visual style can be unique selling point
2. Partnerships can help
3. Difficulty can be appealing
4. Art quality matters
5. Industry connections help
6. Co-op adds value
7. Brand expansion possible
8. Nostalgia can work
9. Traditional techniques can be modern
10. Patience (4 years) pays off

36.8 Case Study 8: Dead Cells

gaema tathaya

DEAD CELLS

Game Name: Dead Cells
Developer: Motion Twin (7 people)
Genre: Roguelite / Metroidvania
Platform: PC, Switch, PS4, Xbox, Mobile
Release: August 2017 (1.0), Early Access 2017
Development Time: ~2 years (2015-2017)
Team Size: 7 people
Engine: Heaps.io
Budget: Self-funded
Revenue: \$50+ Million (Lifetime)
Reviews: 150,000+ (Steam, 95% Positive)

Development Approach

Motion Twin's Development Approach:

1. EARLY ACCESS
 - 2017 Early Access release
 - Regular updates
 - Community feedback
 - Iterative improvement
2. GENRE BLEND
 - Roguelite + Metroidvania
 - Procedural generation
 - Permanent progression
 - High replayability
3. SMALL TEAM
 - 7 people
 - Each member multiple roles
 - Efficient workflow
 - Quality focus

Unique Idea

Unique Selling Points:

1. GENRE BLEND
 - Roguelite + Metroidvania
 - Unique combination
 - Procedural + hand-crafted
 - High replayability
2. COMBAT
 - Fast-paced combat
 - Variety of weapons
 - Skill-based
 - Satisfying feel
3. PROGRESSION
 - Permanent upgrades
 - Meta-progression
 - Motivation to replay
 - Long-term engagement

Marketing Strategy

Marketing Approach:

1. EARLY ACCESS
 - Early Access buzz
 - Community building
 - Regular updates
 - Word of mouth
2. GENRE APPEAL
 - Roguelite fans
 - Metroidvania fans
 - Action fans
 - Broad appeal
3. CRITICAL ACCLAIM
 - Excellent reviews
 - Awards
 - Media coverage
 - Sales boost

Launch Details

Launch Timeline:

2015: Development started
2017 March: Early Access release
2017 August: 1.0 release
2018: DLC (The Bad Seed)
2019: DLC (Fatal Falls)
2021: DLC (The Queen and the Sea)

Key Metrics:

- Early Access: Strong community
- 1.0 Release: 10,000+ concurrent
- Year 1: 2,000,000+ copies
- Lifetime: 10,000,000+ copies
- Multiple DLC expansions

What Made It Successful

Success Factors:

1. GENRE INNOVATION
 - Unique genre blend
 - Roguelite + Metroidvania
 - High replayability
 - Broad appeal
2. GAMEPLAY QUALITY
 - Fast combat
 - Weapon variety
 - Skill-based
 - Satisfying
3. EARLY ACCESS SUCCESS
 - Community feedback
 - Iterative improvement
 - Regular updates
 - Player investment
4. LONG-TERM ENGAGEMENT
 - DLC expansions
 - Meta-progression
 - Multiple runs
 - Content updates

What We Can Learn

Key Takeaways:

1. Genre blending can be successful
2. Early Access works for roguelites
3. Replayability is crucial
4. Combat feel matters
5. Meta-progression adds value
6. DLC can extend life
7. Small teams can succeed
8. Community feedback valuable
9. Procedural generation works
10. High production value

36.9 Case Study 9: Slay the Spire

gaema tathaya

SLAY THE SPIRE

Game Name: Slay the Spire
Developer: Mega Crit (4 people)
Genre: Deckbuilder / Roguelike
Platform: PC, Switch, PS4, Xbox, Mobile
Release: January 2019 (1.0), Early Access 2017
Development Time: ~2 years (2017-2019)
Team Size: 4 people
Engine: LibGDX
Budget: Self-funded
Revenue: \$50+ Million (Lifetime)
Reviews: 150,000+ (Steam, 97% Positive)

Development Approach

Mega Crit aira Development Approach:

1. GENRE INNOVATION
 - Deckbuilder + Roguelike
 - Unique combination
 - Strategic depth
 - High replayability
2. EARLY ACCESS
 - 2017 Early Access
 - Regular updates
 - Community balance
 - Iterative design
3. MODDING SUPPORT
 - Steam Workshop support
 - Community mods
 - Extended life
 - Community engagement

Unique Idea

Unique Selling Points:

1. DECKBUILDER + ROGUELIKE
 - Unique genre blend
 - Strategic depth
 - Procedural runs
 - High replayability
2. STRATEGIC DEPTH
 - Complex card interactions
 - Multiple strategies
 - Build variety
 - Decision-making
3. ACCESSIBILITY
 - Easy to learn
 - Hard to master
 - Turn-based
 - Anyone can play

Marketing Strategy

Marketing Approach:

1. EARLY ACCESS
 - Early Access success
 - Community building
 - Regular updates
 - Word of mouth
2. MODDING COMMUNITY
 - Steam Workshop
 - Community content
 - Extended playtime
 - Free marketing
3. CRITICAL ACCLAIM

Excellent reviews
Awards
Media coverage
Sales boost

Launch Details

Launch Timeline:

2017: Early Access release
2017-2019: Regular updates
2019 January: 1.0 release
2019: Console ports
2021: Mobile release
2022: Slay the Spire 2 announced

Key Metrics:

- Early Access: Strong community
- 1.0 Release: 15,000+ concurrent
- Year 1: 2,000,000+ copies
- Lifetime: 10,000,000+ copies
- Mobile: 10,000,000+ downloads

What Made It Successful

Success Factors:

1. GENRE INNOVATION
 - Unique combination
 - Strategic depth
 - High replayability
 - Broad appeal
2. MODDING SUPPORT
 - Steam Workshop
 - Community content
 - Extended life
 - Free marketing
3. GAMEPLAY DEPTH
 - Card interactions
 - Multiple strategies
 - Build variety
 - Decision-making
4. ACCESSIBILITY
 - Easy to learn
 - Hard to master
 - Turn-based
 - Anyone can play

What We Can Learn

Key Takeaways:

1. Genre innovation can succeed
2. Modding extends game life
3. Strategic depth matters
4. Accessibility is important
5. Early Access works
6. Turn-based can be popular

7. Small teams can succeed
8. Community content is valuable
9. Replayability is crucial
10. Sequels possible with success

36.10 Case Study 10: Valheim

gaema tathaya

VALHEIM

Game Name: Valheim
Developer: Iron Gate AB (5 people)
Genre: Survival / Open World
Platform: PC (Early Access)
Release: February 2021 (Early Access)
Development Time: ~2 years (2019-2021)
Team Size: 5 people
Engine: Unity
Budget: Self-funded
Revenue: \$100+ Million (Lifetime)
Reviews: 300,000+ (Steam, 95% Positive)

Development Approach

Iron Gate AB aia Development Approach:

1. EARLY ACCESS
 - 2021 Early Access release
 - Low price (\$20)
 - Regular updates planned
 - Community-driven development
2. SMALL TEAM
 - maatara 5 jana
 - Efficient workflow
 - Quality focus
 - Personal touch
3. CO-OP FOCUS
 - 1-10 player co-op
 - Shared world
 - Community building
 - Social experience

Unique Idea

Unique Selling Points:

1. VIKING THEME
 - Popular theme
 - Norse mythology
 - Exploration focus
 - Cultural appeal
2. SURVIVAL + BUILDING
 - Base building
 - Crafting systems

Survival mechanics
Progression

3. CO-OP EXPERIENCE

1-10 players
Shared worlds
Community servers
Social gaming

Marketing Strategy

Marketing Approach:

1. WORD OF MOUTH

Players loved it
Friends recommended
Social media buzz
Viral growth

2. STREAMER EXPOSURE

Streamers played it
YouTube content
Twitch popularity
Free marketing

3. LOW PRICE

\$20 price point
High value proposition
Accessible
Impulse buy

Launch Details

Launch Timeline:

2019: Development started
2021 February: Early Access release
2021: Explosive growth
2021: 10,000,000+ copies sold
2022: Major updates
2023: More content planned

Key Metrics:

- Day 1: 500,000 copies
- Week 1: 2,000,000 copies
- Month 1: 5,000,000 copies
- Year 1: 10,000,000+ copies
- Peak: 500,000+ concurrent

What Made It Successful

Success Factors:

1. PERFECT TIMING

COVID-19 lockdown
Co-op gaming demand
gaming
Perfect storm

2. CO-OP APPEAL

Play with friends

- Social experience
- Community building
- Shared worlds

3. VALUE PROPOSITION
- \$20 price
 - 100+ hours content
 - High value
 - Impulse buy

4. SURVIVAL GENRE
- Popular genre
 - Base building
 - Crafting
 - Progression

What We Can Learn

Key Takeaways:

1. Timing matters
2. Co-op is powerful
3. Value proposition matters
4. Small teams can succeed
5. Survival genre is popular
6. Word of mouth is crucial
7. Streamers drive sales
8. Low price can work
9. Early Access can explode
10. Community focus is key

36.11 Comparison Table

Game	CASE STUDY COMPARISON				
	Dev Time	Team	Budget	Revenue	Genre
Stardew Valley	4.5 yrs	1	\$0	\$100M+	Farm
Hollow Knight	3 yrs	3	\$57K	\$50M+	Metr.
Celeste	2 yrs	2	Funded	\$10M+	Plat.
Among Us	1 yr	3	Low	\$500M+	Social
Hades	3 yrs	20	Funded	\$50M+	Rog.
Undertale	2.5 yrs	1	\$50K	\$50M+	RPG
Cuphead	4 yrs	20	Funded	\$100M+	Run
Dead Cells	2 yrs	7	Self	\$50M+	Rog.
Slay the Spire	2 yrs	4	Self	\$50M+	Deck
Valheim	2 yrs	5	Self	\$100M+	Surv.
Avg Dev Time: 2.5 years					
Avg Team Size: 7 people					
Avg Revenue: \$100M+					

36.12 Common Success Patterns

- COMMON SUCCESS PATTERNS
1. QUALITY OVER QUANTITY
 - All games are polished
 - High production value
 - Player satisfaction

2. UNIQUE SELLING POINT
 - Each game has something special
 - Visual style, gameplay, or theme
 - Stands out from competition
 3. COMMUNITY FOCUS
 - Player feedback valued
 - Community building
 - Word of mouth marketing
 4. TIMING
 - Market timing important
 - Genre popularity
 - External factors (COVID-19)
 5. VALUE PROPOSITION
 - Fair price for content
 - High replayability
 - Long-term engagement
-

36.13 Exercise

anaushailana 1: Case Study Analysis

1. uparaera 10tai gaemaera madhayae aikatai baechhae naina
2. baisataaaraita baishalaessana karauna
3. aapanaaaara gaemae kaiibhaaabaee apply karatae paaaraena saetai laikhauna

anaushailana 2: Success Pattern Identification

1. uparaera success patterns daekhauna
2. aapanaaaara paraaekatae kaonagaulao apply karatae paaarabaena
3. kaonagaulao apply karatae paaarabaena naaa
4. paraikalapanaaa taairai karauna

anaushailana 3: Marketing Strategy

1. aikatai case study aira marketing strategy baishalaessana karauna
 2. aapanaaaara gaemaera janaya similar strategy taairai karauna
 3. Timeline saeta karauna
 4. Budget nairadhaaarana karauna
-

36.14 saaadhaaarana bhaula (Common Mistakes)

bhaula 1: Copying Without Understanding

bhaula: anayaera gaema kapai karaaa baujhae
sathaika: kaena saphala hayaechhae saetai baujhauna, aapanaaaara naijaera kaaushala taairai

bhaula 2: Ignoring Market Context

bhaula: 2020-aira success 2026-ai replicate karaaara chaessataaa karaaa
sathaika: baratamaaana market context baujhauna

bhaulta 3: Unrealistic Expectations

bhaulta: "aamaaaraau \$100M revenue habae"

sathaika: Realistic expectations raaakhauna, chhaota daiyae shaurau karauna

bhaulta 4: Ignoring Quality

bhaulta: shaudhau quantity aira paechhanae laaagaaa

sathaika: Quality sarabaochacha agaraaadhaikaaara

36.15 taipasa (Tips)

taipasa 1: Study Success

saphala gaema khaelauna, baishalaessana karauna, shaikhauna

taipasa 2: Learn from Failures

bayaratha gaema thaekaeau shaikhauna, kائي bhaulta hayaechhae

taipasa 3: Find Your Niche

aapanaaara naijaera niche khaujauna, copy naya

taipasa 4: Quality Focus

sabasamaya quality aira daikae manaoyaoga daina

taipasa 5: Community Building

daina 1 thaekae community build karauna

36.16 saarasakassaepa (Summary)

aai adhayaaayae aamaraaa 10tai saphala inada gaemaera baishalaessana daekhaechhai:

1. ****Stardew Valley**** - Solo development, quality, community
2. ****Hollow Knight**** - Small team, Kickstarter, free DLC
3. ****Celeste**** - Game jam origin, accessibility, awards
4. ****Among Us**** - Timing, streamers, social gaming
5. ****Hades**** - Early Access, narrative innovation, polish
6. ****Undertale**** - Kickstarter, meta-narrative, cultural impact
7. ****Cuphead**** - Visual style, partnerships, difficulty
8. ****Dead Cells**** - Genre blend, Early Access, replayability
9. ****Slay the Spire**** - Genre innovation, modding, depth
10. ****Valheim**** - Co-op, timing, value proposition

manae raaakhauna:

parataitai saphalataaa unique, kainatau common patterns aachhae

aapanaaara naijaera path khaujauna, quality aira daikae manaoyaoga daina

****parabarataii adhayaaaya:**** Chapter 37: Complete Game Project Example

PART 37

Chapter 37: COMPLETE GAME PROJECT EXAMPLE

Chapter 37: COMPLETE GAME PROJECT EXAMPLE

samapauurana gaema parajaekata udaaaharana

shaekhaara udadaeshaya (Learning Goal)

aai adhayaaayae aapanai aikatai samapauurana Indie Horror Game "THE LAST ROOM" aira baisataaaraita udaaaharana daekhabaena dhaaaranaaa thaekae shaurau karae Launch parayanata samapauurana parakaraiyaaa daekhabaena GDD, Gameplay Loop, Story, Characters, Art, Audio, Programming, Testing, Marketing, Budget, aiba Timeline sabakaichhau baisataaaraitabhaaabaek daekhabaena

37.1 gaema tathaya (Game Information)

THE LAST ROOM
daya laaasata rauma
Game Name: The Last Room
Genre: Psychological Horror
Platform: PC (Steam)
Target Audience: 18+ (Horror fans, Atmospheric game lovers)
Engine: Unity (HDRP)
Development: Solo Developer
Development Time: 12 months
Target Price: \$14.99
Target Length: 3-5 hours (single playthrough)
Replay Value: Multiple endings, Secret areas

37.2 Concept (dhaaaranaaa)

gaemaera dhaaaranaaa

CONCEPT DOCUMENT

High Concept:
aikajana maaanaussa aikatai adabhauta haotaelae jaegae authae aiba sae kaiibhaaabaek saekha

Elevator Pitch:

"The Last Room aikatai Psychological Horror Game yaekhaanae aapanai aikatai adabhauta hao

Unique Selling Points:

1. Atmosphere-driven horror (galapa naya, paraibaesha)
2. Multiple endings based on choices
3. No jump scares - pure psychological tension
4. Short, intense experience (3-5 hours)
5. Replayability with different paths

Inspiration:

- Silent Hill 2 (Atmosphere, Story)
- PT (Horror, Loop)
- The Shining (Setting, Isolation)
- Jacob's Ladder (Psychological horror)

Target Audience

TARGET AUDIENCE

Primary Audience:

- Age: 18-35
- Gender: All genders
- Interest: Horror games, Atmospheric games
- Platform: PC gamers
- Budget: \$10-20 range

Secondary Audience:

- Horror movie fans
- Indie game enthusiasts
- Streamers/Content creators
- Achievement hunters

Player Persona:

- Name: Rahul
- Age: 25
- Plays horror games regularly
- Enjoys story-driven games
- Watches horror game streams
- Values atmosphere over jump scares

37.3 Pitch (paicha)

Pitch Document

PITCH DOCUMENT

THE LAST ROOM

A Psychological Horror Experience

THE PROBLEM:

Horror games rely too much on jump scares. Players want genuine fear, not cheap tricks.

THE SOLUTION:

The Last Room creates fear through atmosphere, ambiguity, and psychological tension. Every

UNIQUE APPROACH:

- No traditional jump scares
- Environment tells the story

- Player choices create multiple endings
- Short, intense experience (3-5 hours)
- High replayability

MARKET OPPORTUNITY:

- Horror games are popular on Steam
- Atmospheric horror is underserved
- Streamer-friendly content
- Low price point (\$14.99)

TEAM:

- Solo developer
- 12 months development
- Unity HDRP

FINANCIALS:

- Development cost: \$5,000
- Target revenue: \$100,000+
- Break-even: 3,500 copies

37.4 GDD (gaema daijaaaina dakaumaenata)

GDD Overview

GAME DESIGN DOCUMENT

Version: 1.0

Date: January 2026

Author: [Developer Name]

Table of Contents:

1. Game Overview
2. Gameplay Mechanics
3. Story & Characters
4. Level Design
5. Art Style
6. Audio Design
7. UI/UX
8. Technical Specifications

Game Overview

GAME OVERVIEW

Title: The Last Room

Genre: Psychological Horror

Platform: PC (Steam)

Engine: Unity HDRP

Target Frame Rate: 60 FPS

Resolution: 1920x1080 (scalable)

Game Flow:

START -> Wake up in hotel -> Explore rooms ->

Make choices -> Discover truth ->

Reach ending -> Credits

Core Loop:

Explore Room -> Discover Clue ->

Make Choice -> Affect Story ->

Unlock New Room -> Repeat

Gameplay Mechanics

GAMEPLAY MECHANICS

1. MOVEMENT
- WASD movement

- Mouse look

- Sprint (Shift)

- Crouch (Ctrl)

- Interact (E)

- Flashlight (F)
2. EXPLORATION
- Free movement in rooms

- Object interaction

- Item collection

- Clue discovery
3. CHOICE SYSTEM
- Binary choices (2 options)

- Consequence tracking

- Multiple endings based on choices

- Moral ambiguity
4. FEAR MECHANICS
- Darkness affects visibility

- Sound creates tension

- Environment changes subtly

- Psychological manipulation
5. PROGRESSION
- Linear room progression

- Key items unlock doors

- Clues reveal story

- Choices affect available rooms

37.5 Gameplay Loop (gaemapalae laupa)

Core Loop

CORE GAMEPLAY LOOP

START

Wake up

EXPLORE

Room

DISCOVER

Clue/Item

CHOOSE

Option A/B

AFFECT

Story

UNLOCK

New Room

(Repeat)

ENDING

Based on
Choices

Detailed Loop

DETAILED GAMEPLAY LOOP

Room 1: Lobby (Tutorial)

Explore: Learn movement, interaction
Discover: First clue (newspaper clipping)
Choose: Follow hint or ignore
Unlock: Room 2 (Corridor)

Room 2: Corridor

Explore: Find locked doors, note on wall
Discover: Key under carpet
Choose: Use key on left or right door
Unlock: Room 3a or 3b

Room 3a: Old Bedroom

Explore: Find photo, diary entry
Discover: Truth about previous guest
Choose: Accept truth or deny
Unlock: Room 4a or 4b

Room 3b: Bathroom

Explore: Mirror reflection changes
Discover: Hidden message
Choose: Trust reflection or destroy mirror
Unlock: Room 4c or 4d

... (continues for 8-10 rooms)

Final Room: The Last Room

Explore: All clues come together
Discover: The truth about the hotel
Choose: Final decision
Ending: Based on accumulated choices

37.6 Story (galapa)

Story Overview

STORY OVERVIEW

Setting:

A mysterious, seemingly abandoned hotel called "Hotel Echo" located in an isolated area. The hotel appears normal at first glance but reveals disturbing details as you explore.

Plot:

You wake up in a room in Hotel Echo with no memory of how you got there. The hotel appears empty, but something feels wrong. As you explore different rooms, you discover clues about the hotel's dark past and your connection to it.

The Truth (Spoiler):

You are actually dead. The hotel is a limbo between life and death. Each room represents a stage of your life. The choices

you make determine whether you accept your death and move on,
or remain trapped in denial forever.

Themes:

- Acceptance vs Denial
- Memory and Identity
- Isolation and Loneliness
- The nature of reality

Story Structure

STORY STRUCTURE

Act 1: Confusion (Rooms 1-3)

- Wake up with no memory
- Explore basic rooms
- Find first clues
- Establish atmosphere

Act 2: Discovery (Rooms 4-7)

- Clues reveal hotel's history
- Player's connection emerges
- Choices become harder
- Reality becomes unstable

Act 3: Revelation (Rooms 8-9)

- Full truth revealed
- Final choice presented
- All paths converge
- Emotional climax

Act 4: Resolution (Room 10 - The Last Room)

- Based on choices:
 - Ending A: Acceptance (Peace)
 - Ending B: Denial (Trapped)
 - Ending C: Resistance (Escape?)
- Secret Ending: True Understanding

Dialogue Examples

DIALOGUE EXAMPLES

Room 1 - Lobby:

[Wall text appears]

"Welcome to Hotel Echo.

Check-in: [Unknown]

Check-out: [Never]"

Room 3 - Old Bedroom:

[Diary entry found]

"Day 1: The hotel seems nice.

Day 7: I can't find the exit.

Day 14: The rooms are changing.

Day ???: I think I'm not alone."

Room 7 - Mirror Room:

[Mirror speaks]

"You've been here before.

You just don't remember.

Do you want to remember?"

Room 10 - The Last Room:

[Narration]
"This is the last room.
This is where it ends.
Or begins again.
What do you choose?"

37.7 Characters (charaitara)

Main Character

MAIN CHARACTER

Name: Protagonist (Unnamed)
Age: 30s
Background: Unknown (revealed through story)

Character Arc:

- Start: Confused, scared, seeking escape
- Middle: Discovering truth, facing fears
- End: Accepting or denying reality

Player Connection:

- First-person perspective
- No visible body (only hands when interacting)
- Internal monologue through text
- Choices define personality

Voice:

- No voice acting (text-based)
- Internal thoughts appear on screen
- Emotional state shown through text style

NPCs

NON-PLAYER CHARACTERS

1. The Manager (Ghost)
 - Appears in mirrors
 - Speaks in riddles
 - Guides or misleads player
 - True nature: Part of protagonist's psyche
 2. The Previous Guest (Ghost)
 - Found in specific rooms
 - Shares their story
 - Warning or encouragement
 - True nature: Another soul in limbo
 3. The Child (Ghost)
 - Appears in certain paths
 - Represents innocence
 - Key to secret ending
 - True nature: Protagonist's lost innocence
 4. The Shadow
 - Visible in peripheral vision
 - Never directly confronted
 - Represents fear/denial
 - True nature: Protagonist's inner demon
-

37.8 Enemies (shatarau)

Enemy Design

ENEMY DESIGN

NOTE: This game does NOT have traditional enemies. The horror comes from atmosphere and psychology.

"Enemies" are abstract concepts:

1. DARKNESS
 - Limited flashlight
 - Darkness affects visibility
 - Creates tension
 - Not an actual enemy
2. THE SHADOW
 - Visible in corners
 - Never attacks
 - Represents fear
 - Flees when confronted
3. MIRRORS
 - Show disturbing reflections
 - Speak truths player doesn't want to hear
 - Can be avoided
 - Part of psychological horror
4. THE HOTEL ITSELF
 - Rooms change subtly
 - Doors lock/unlock
 - Environment shifts
 - Represents protagonist's mind

Fear Mechanics

FEAR MECHANICS

Instead of enemies, the game uses:

1. ATMOSPHERE
 - Dim lighting
 - Eerie sounds
 - Unsettling visuals
 - Environmental storytelling
2. PSYCHOLOGICAL TENSION
 - Unreliable narration
 - Reality shifts
 - Memory fragments
 - Moral choices
3. SOUND DESIGN
 - Creaking floors
 - Distant whispers
 - Heartbeat effects
 - Silence used for tension
4. VISUAL EFFECTS
 - Subtle distortions

- Peripheral movement
- Reflection changes
- Light flickering

37.9 Levels (laebhaela)

Level Overview

LEVEL OVERVIEW

Total Rooms: 10

Playtime: 3-5 hours

Linearity: Semi-linear (choices affect path)

Room List:

1. Lobby (Tutorial)
2. Corridor
3. Old Bedroom OR Bathroom
4. Library OR Dining Room
5. Ballroom OR Kitchen
6. Office OR Nursery
7. Mirror Room OR
8. attic OR Garden
9. Penthouse
10. The Last Room (Final)

Each room has:

- Unique theme
- Clues to discover
- Choice to make
- Atmosphere elements

Room Design Examples

ROOM DESIGN EXAMPLES

ROOM 1: LOBBY

Size: Medium (10x10 meters)

Theme: Grand but decayed

Lighting: Dim chandelier, flashlight

Props: Reception desk, old paintings, dust

Clues: Newspaper clipping on desk

Choice: Read note or explore immediately

Atmosphere: Dust particles, creaking sounds

Unlock: Corridor door

ROOM 5: BALLROOM

Size: Large (20x15 meters)

Theme: Faded grandeur

Lighting: Moonlight through windows

Props: Dance floor, orchestra pit, mirrors

Clues: Ghost couple dancing, music box

Choice: Join dance or watch from distance

Atmosphere: Faint music, cold wind

Unlock: Two different paths based on choice

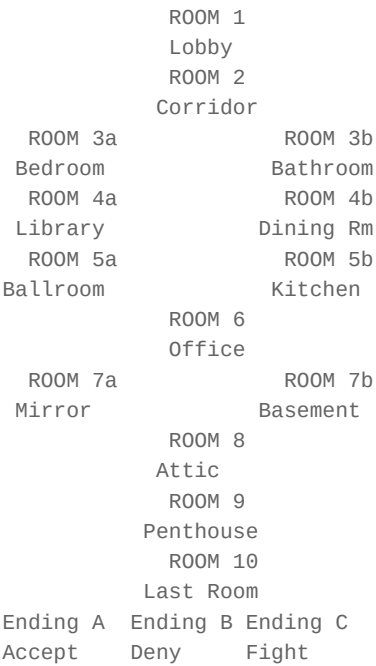
ROOM 10: THE LAST ROOM

Size: Small (5x5 meters)

Theme: Minimalist, symbolic
Lighting: Single light bulb
Props: Chair, table, mirror
Clues: All previous clues come together
Choice: Final decision (accept/deny/fight)
Atmosphere: Complete silence
Unlock: Ending sequence

Level Progression Map

LEVEL PROGRESSION MAP



37.10 Art (shailapa)

Art Style Guide

ART STYLE GUIDE

Overall Style: Photorealistic with stylized elements

Color Palette:

- Primary: Dark blues, grays, blacks
- Secondary: Warm yellows (flashlight)
- Accent: Red (danger, important clues)
- Avoid: Bright colors, saturated hues

Lighting:

- Low key lighting
- High contrast
- Flashlight as main light source
- Moonlight through windows
- Flickering lights

Textures:

- Realistic materials

Worn, aged surfaces
Dust and decay details
High resolution (2K-4K)
PBR materials

Visual Effects:

Dust particles
Light rays
Reflections (water, mirrors)
Fog and haze
Subtle distortions

Asset List

ASSET LIST

Characters:

Protagonist (first-person hands only)
Ghost NPCs (3 types)
Shadow entity
Mirror reflections

Environment:

Hotel rooms (10 unique themes)
Hallways and corridors
Furniture (beds, desks, chairs)
Decorations (paintings, mirrors)
Props (books, papers, items)
Architectural details

Items:

Flashlight
Keys (various)
Notes and documents
Photos
Key story items

Visual Effects:

Dust particles
Light rays
Fog
Reflections
Screen effects (distortion)
Transition effects

UI:

Menu screens
HUD elements
Choice UI
Inventory UI
Loading screens

Art Production Pipeline

ART PRODUCTION PIPELINE

Phase 1: Concept (Month 1-2)

Concept art for rooms
Color palette exploration
Mood boards
Style guide finalization

Phase 2: Blockout (Month 2-3)

- 3D greybox rooms
- Basic lighting
- Scale and proportion
- Playtesting

Phase 3: Production (Month 3-8)

- High-poly modeling
- Low-poly retopology
- UV mapping
- Texturing (Substance Painter)
- Material creation
- Asset integration

Phase 4: Polish (Month 8-10)

- Lighting finalization
- Visual effects
- Post-processing
- Optimization
- Quality assurance

37.11 Audio (adaiau)

Audio Design

AUDIO DESIGN

Overall Approach: Minimalist, atmospheric

Music:

- Ambient tracks (no traditional music)
- Drones and textures
- Emotional cues at key moments
- Silence used as a tool
- Adaptive music based on choices

Sound Effects:

- Environmental (footsteps, creaks, wind)
- Interaction (doors, items, switches)
- Atmospheric (heartbeat, breathing)
- UI (menu sounds, choice selection)
- Story cues (whispers, distant sounds)

Voice:

- No voice acting
- Text-based dialogue
- Sound effects for text appearance
- Breathing sounds for protagonist

Audio Assets

AUDIO ASSETS

Music (6-8 tracks):

- Main Theme (2:00)
- Lobby Atmosphere (3:00)
- Exploration Tension (4:00)
- Discovery Moment (1:30)
- Choice Sequence (2:00)

Revelation (3:00)
Ending A (2:30)
Ending B (2:30)
Ending C (2:30)

Sound Effects (100+ sounds):
Footsteps (8 variations)
Door sounds (6 variations)
Object interactions (20+)
Environmental ambience (15+)
UI sounds (10+)
Horror elements (20+)
Story cues (10+)

Ambience:
Hotel background hum
Wind outside
Distant traffic
Electrical buzz
Silence tracks

37.12 Programming (paraogaraaamai)

Technical Architecture

TECHNICAL ARCHITECTURE

Engine: Unity 2022 LTS (HDRP)
Language: C#
Version Control: Git (GitHub)

Core Systems:
Player Controller
Interaction System
Choice System
Story Manager
Room Manager
Audio Manager
Save System
UI Manager
Settings Manager

Third-Party Assets:
TextMeshPro (Text rendering)
DOTween (Animations)
Rewired (Input)
Steamworks.NET (Steam integration)

Core Scripts

CORE SCRIPTS

PlayerController.cs
Movement (WASD, Mouse)
Sprint, Crouch
Flashlight toggle
Interaction detection

InteractionSystem.cs

- Raycast detection
- Object highlighting
- Interaction prompts
- Action execution

ChoiceSystem.cs

- Choice presentation
- Timer (if applicable)
- Consequence tracking
- Story progression

StoryManager.cs

- Story state tracking
- Ending calculation
- Save/Load story
- Event triggers

RoomManager.cs

- Room loading
- Room transitions
- Room state management
- Clue tracking

AudioManager.cs

- Music playback
- SFX management
- Ambience control
- Spatial audio

SaveSystem.cs

- Save game data
- Load game data
- Auto-save
- Multiple save slots

UIManager.cs

- Menu management
- HUD updates
- Choice UI
- Settings menu
- Pause menu

SettingsManager.cs

- Graphics settings
- Audio settings
- Input settings
- Accessibility options
- Save settings

Save System

SAVE SYSTEM

Save Data Structure:

```
{
  "saveVersion": "1.0",
  "timestamp": "2026-01-15T10:30:00",
  "currentRoom": "Room_5_Ballroom",
  "choices": [
    {"room": "Room_3", "choice": "A", "consequence": "truth"},
    {"room": "Room_4", "choice": "B", "consequence": "denial"},
    {"room": "Room_5", "choice": "A", "consequence": "accept"}
  ],
}
```

```
"itemsCollected": ["newspaper", "diary", "photo"],
"roomsVisited": ["Room_1", "Room_2", "Room_3a", "Room_4a", "Room_5a"],
"storyProgress": 0.5,
"settings": {
  "quality": "High",
  "volume": 0.8,
  "fullscreen": true
}
```

Auto-save triggers:

- Room entry
- Choice made
- Key item collected
- Every 5 minutes

37.13 Prototype (paraotaotaaaipa)

Prototype Goals

PROTOTYPE GOALS

Phase 1: Basic Movement (Week 1-2)

Player controller
First-person camera
Basic interaction
Flashlight

Phase 2: Room System (Week 3-4)

Room loading
Object interaction
Basic UI
Placeholder art

Phase 3: Choice System (Week 5-6)

Choice presentation
Consequence tracking
Story progression
Basic ending

Phase 4: Horror Elements (Week 7-8)

Lighting system
Sound integration
Atmosphere creation
Tension building

Prototype Milestones

PROTOTYPE MILESTONES

Week 2: Basic movement working

Week 4: One room playable

Week 6: Choice system working

Week 8: Horror atmosphere achieved

Success Criteria:

- Player can move and interact
- Choices affect story
- Horror atmosphere is effective

- Game is playable from start to finish
-

37.14 Vertical Slice

Vertical Slice Definition

VERTICAL SLICE

A complete, polished section of the game that demonstrates:

- All core mechanics
- Final art quality
- Final audio quality
- Story integration
- Horror atmosphere

Vertical Slice Content:

Room 1: Lobby (Tutorial)
Room 2: Corridor
Room 3: Old Bedroom
Choice system
Basic save/load
Ending preview

Duration: 30-45 minutes

Quality: Final release quality

Purpose: Proof of concept for full game

37.15 Production (upaaadana)

Production Timeline

PRODUCTION TIMELINE (12 MONTHS)

Month 1-2: PRE-PRODUCTION

GDD finalization
Art style exploration
Technical setup
Prototype planning
Asset list creation

Month 3-4: PROTOTYPE

Basic movement system
Interaction system
Choice system
Room system
Basic UI

Month 5-6: VERTICAL SLICE

Room 1-3 polished
Art integration
Audio integration
Horror atmosphere
Playtesting

Month 7-9: CORE DEVELOPMENT

All rooms built

All systems working
Art assets created
Audio assets created
Story complete

Month 10-11: POLISH

Bug fixing
Performance optimization
Visual polish
Audio polish
QA testing

Month 12: LAUNCH PREPARATION

Store page setup
Trailer creation
Marketing push
Launch
Post-launch support

Weekly Schedule

WEEKLY SCHEDULE

Monday: Planning + Core Development

9:00 - 9:30: Daily planning
9:30 - 12:00: Core development
12:00 - 1:00: Lunch
1:00 - 4:00: Core development
4:00 - 5:00: Review + next day planning

Tuesday: Programming

9:00 - 9:30: Daily planning
9:30 - 12:00: System development
12:00 - 1:00: Lunch
1:00 - 4:00: Bug fixing
4:00 - 5:00: Code review

Wednesday: Art Production

9:00 - 9:30: Daily planning
9:30 - 12:00: Asset creation
12:00 - 1:00: Lunch
1:00 - 4:00: Asset integration
4:00 - 5:00: Art review

Thursday: Audio + Level Design

9:00 - 9:30: Daily planning
9:30 - 12:00: Audio production
12:00 - 1:00: Lunch
1:00 - 4:00: Level design
4:00 - 5:00: Playtesting

Friday: Testing + Polish

9:00 - 9:30: Daily planning
9:30 - 12:00: Testing
12:00 - 1:00: Lunch
1:00 - 3:00: Bug fixing
3:00 - 4:00: Weekly review
4:00 - 5:00: Retrospective

37.16 Testing (paraiikassaaa)

Testing Plan

TESTING PLAN

Testing Types:

- Functional Testing
- Playtesting
- Performance Testing
- Compatibility Testing
- Bug Testing
- User Experience Testing

Testing Schedule:

- Weekly internal testing
- Monthly external playtesting
- Beta testing (Month 10)
- Final QA (Month 11)
- Platform testing (Month 12)

Bug Tracking:

- Tool: Unity Bug Tracker / Excel
- Categories: Critical, Major, Minor, Polish
- Priority: High, Medium, Low
- Status: Open, In Progress, Fixed, Verified

Test Cases

TEST CASES

Functional Tests:

- Player movement works correctly
- Flashlight toggles properly
- Interaction system detects objects
- Choices present correctly
- Story progresses based on choices
- Save/Load works properly
- Settings save correctly
- Multiple endings accessible
- Game completes without crashes

Playtesting:

- Horror atmosphere effective
- Pacing feels right
- Choices are meaningful
- Story is engaging
- Difficulty is appropriate
- Length is satisfying
- Replay value exists

Performance Tests:

- 60 FPS on minimum specs
- Loading times acceptable
- Memory usage stable
- No memory leaks
- Crash rate acceptable

37.17 Marketing (maaraketai)

Marketing Plan

MARKETING PLAN

Phase 1: Pre-Development (Month 1-2)

- Steam page creation
- Social media setup
- Devlog website
- Press kit creation

Phase 2: Development (Month 3-10)

- Regular devlogs
- Screenshots/videos
- Community building
- Demo release (Month 8)
- Press outreach

Phase 3: Launch (Month 11-12)

- Trailer release
- Streamer outreach
- Launch day plan
- Social media campaign
- Press coverage

Marketing Materials:

- Steam page assets
- Trailer (2-3 minutes)
- Screenshots (10+)
- Press kit
- Social media assets
- Demo build

Steam Page

STEAM PAGE

Title: The Last Room

Short Description: A psychological horror game where every room tells a story and every ch

Tags:

- Psychological Horror
- Atmospheric
- Story Rich
- Exploration
- Multiple Endings
- First-Person
- Mystery
- Indie

System Requirements:

- Minimum: i5, 8GB RAM, GTX 1050
- Recommended: i7, 16GB RAM, GTX 1070
- Storage: 5GB

Features:

- Atmospheric horror
- Multiple endings
- Choice-driven story
- No jump scares
- 3-5 hours gameplay
- High replay value

Trailer

TRAILER STRUCTURE

Duration: 2:30 - 3:00 minutes

Section 1 (0:00-0:30): Hook

- Quick cuts of atmosphere

- Text: "What if you couldn't remember?"

- Eerie music

- Hotel shots

Section 2 (0:30-1:30): Gameplay

- Exploration footage

- Interaction examples

- Choice moments

- Story reveals

- Horror elements

Section 3 (1:30-2:15): Climax

- Intense moments

- Multiple ending hints

- Emotional beats

- Final reveal

Section 4 (2:15-2:30): Call to Action

- Release date

- Steam page

- Wishlist now

- Logo

37.18 Steam Launch

Launch Plan

LAUNCH PLAN

Pre-Launch (2 weeks before):

- Final trailer release

- Press outreach

- Streamer keys sent

- Social media campaign

- Community engagement

- Store page finalized

Launch Day:

- Game goes live

- Social media announcement

- Press coverage

- Streamer coverage

- Community support

- Bug monitoring

Post-Launch (1 week after):

- Bug fixes

- Player feedback

- Community engagement

- Sales tracking

- Review monitoring

Post-mortem planning

Launch Checklist

LAUNCH CHECKLIST

- Game build finalized
- Store page complete
- Trailer uploaded
- Press kit sent
- Streamer keys distributed
- Social media scheduled
- Community notified
- Support channels ready
- Bug tracking ready
- Analytics set up
- Backup plan ready
- Launch team briefed
- Post-launch plan ready

37.19 Budget (baajaeta)

Development Budget

DEVELOPMENT BUDGET

Total Budget: \$5,000

Breakdown:

Software & Tools

- Unity License: \$0 (Personal)
- Adobe Creative Cloud: \$600/year
- Substance Painter: \$200
- FL Studio: \$200
- Other tools: \$200
- Total: \$1,200

Assets

- Sound effects: \$300
- Music (if outsourced): \$500
- Textures: \$200
- Total: \$1,000

Marketing

- Trailer creation: \$500
- Press outreach: \$200
- Social media ads: \$500
- Total: \$1,200

Testing

- QA testers: \$500
- Total: \$500

Platform Fees

- Steam fee: ~\$100
- Total: \$100

Contingency

- Total: \$1,000

GRAND TOTAL: \$5,000

Revenue Projections

REVENUE PROJECTIONS

Price: \$14.99

Steam cut: 30%

Net per sale: \$10.49

Break-even point:

$\$5,000 / \$10.49 = 477$ copies

Conservative (Year 1):

2,000 copies

Revenue: \$20,980

Profit: \$15,980

Moderate (Year 1):

5,000 copies

Revenue: \$52,450

Profit: \$47,450

Optimistic (Year 1):

10,000 copies

Revenue: \$104,900

Profit: \$99,900

Factors affecting sales:

Reviews quality

Marketing effectiveness

Genre popularity

Competition

Timing

Streamer coverage

37.20 Timeline (samayaraekhaaa)

Master Timeline

MASTER TIMELINE (12 MONTHS)

MONTH 1: SETUP

Week 1-2: Project setup, GDD

Week 3-4: Art exploration, prototype planning

Milestone: Project ready

MONTH 2: PROTOTYPE START

Week 1-2: Player controller, interaction

Week 3-4: Room system, basic UI

Milestone: Basic prototype working

MONTH 3: PROTOTYPE COMPLETE

Week 1-2: Choice system, story manager

Week 3-4: Audio integration, horror elements

Milestone: Prototype playable

MONTH 4: VERTICAL SLICE START

Week 1-2: Room 1-2 polished

Week 3-4: Room 3 polished

Milestone: Vertical slice progress

MONTH 5: VERTICAL SLICE COMPLETE

Week 1-2: Vertical slice polish
Week 3-4: Playtesting, iteration
Milestone: Vertical slice ready

MONTH 6: CORE DEVELOPMENT START

Week 1-2: Room 4-6 built
Week 3-4: Room 7-8 built
Milestone: Core rooms built

MONTH 7: CORE DEVELOPMENT CONTINUE

Week 1-2: Room 9-10 built
Week 3-4: Systems integration
Milestone: All rooms built

MONTH 8: CORE DEVELOPMENT COMPLETE

Week 1-2: All systems working
Week 3-4: Demo release, marketing start
Milestone: Feature complete

MONTH 9: POLISH START

Week 1-2: Art polish
Week 3-4: Audio polish
Milestone: Polish progress

MONTH 10: POLISH COMPLETE

Week 1-2: Bug fixing
Week 3-4: Performance optimization
Milestone: Polish complete

MONTH 11: TESTING

Week 1-2: QA testing
Week 3-4: Beta testing, final fixes
Milestone: Release candidate

MONTH 12: LAUNCH

Week 1-2: Launch preparation
Week 3: Launch
Week 4: Post-launch support
Milestone: Game launched

37.21 Exercise

anaushailana 1: naijaera gaemaera GDD laikhauna

1. uparaera GDD template bayabahaaara karauna
2. naijaera gaemaera janaya anaukauulaita karauna
3. parataitai section baisataaaraita bharauna
4. aikajana banadhaukae daiyae review karaaana

anaushailana 2: Budget taairai karauna

1. naijaera gaemaera janaya budget taairai karauna
2. parataitai category tae kharacha anaumaaana karauna
3. Revenue projection taairai karauna
4. Break-even point nairanaya karauna

anaushailana 3: Timeline taairai karauna

1. 12 maaasaera timeline taairai karauna
 2. Monthly milestones saeta karauna
 3. Weekly tasks plan karauna
 4. Buffer time yaoga karauna
-

37.22 saaadhaaarana bhaula (Common Mistakes)

bhaula 1: Over-scoping

bhaula: atairaikata phaichaaara yaoga karaaa
sathaika: MVP daiyae shaurau karauna, parae expand karauna

bhaula 2: No Planning

bhaula: GDD naaa laikhae saraasarai kaoda shaurau karaaa
sathaika: parathamae GDD laikhauna, taaarapara implement karauna

bhaula 3: Ignoring Testing

bhaula: shaessa parayanata testing naaa karaaa
sathaika: Regular testing karauna, early bug detection

bhaula 4: No Marketing

bhaula: gaema baaanaiyae phaelae maaarakaetai bhaaabaaa
sathaika: Day 1 thaekae marketing shaurau karauna

37.23 taipasa (Tips)

taipasa 1: Start Small

aikatai room daiyae shaurau karauna, sabakaichhau aikasaaathae naya

taipasa 2: Vertical Slice First

parathamae vertical slice baaanaana, taaarapara full game

taipasa 3: Regular Playtesting

paratai sapataaahae anatata aikabaaara playtest karauna

taipasa 4: Document Everything

sabakaichhau laikhae raaakhauna, parae kaaajae laaagabae

taipasa 5: Take Breaks

bairatai naina, burnout aidaiyae chalauna

37.24 saaarasakassaepa (Summary)

aii adhayaayae aamaraaa aikatai samapauurana Indie Horror Game "THE LAST ROOM" aira udaaaharana daekhaechhai:

1. **Concept** - gaemaera mauula dhaaarananaa
2. **Pitch** - paicha dakaumaenata
3. **GDD** - samapauurana gaema daijaaaina dakaumaenata
4. **Gameplay Loop** - gaemapalae laupa
5. **Story** - galapa aiba characters
6. **Enemies** - shatarau (abstract concepts)
7. **Levels** - laebhaela daijaaaina
8. **Art** - shailapa aiba bhajayauyaaala
9. **Audio** - adaiau daijaaaina
10. **Programming** - paraogaraaamai aarakaitaekachaaara
11. **Prototype** - paraotaotaaaiipa
12. **Vertical Slice** - bhaaarataikaaala salaaaisa
13. **Production** - upaaadana parakaraiyaaa
14. **Testing** - paraiikassaaa paraikalapananaa
15. **Marketing** - maaarakaetai paraikalapananaa
16. **Steam Launch** - Steam launch paraikalapananaa
17. **Budget** - baaajaeta
18. **Timeline** - samayaraekhaaa

manae raaakhauna:

parataitai gaema unique, kainatau process similar
paraikalapananaa karauna, implement karauna, test karauna, launch karauna

parabarataii adhayaaya: Chapter 38: Complete Production Checklist

PART 38

Chapter 38

Chapter 38: COMPLETE PRODUCTION CHECKLIST

samapauurana upaadana chaekalaisata

shaekhaara udadaeshaya (Learning Goal)

aai adhaayaayae aapanai aikatai samapauurana Game Development Production Checklist paaabaena aitai aikatai practical tracking tool yaaa aapanaaara gaema daebhaelapamaenata parataitai dhaapae tarayaaaka karatae saaahaaayaya karabae parataitai item aira janaya Task, Status, Priority, aiba Notes anatarabhaukata rayaechhae

38.1 chaekalaisata bayabahaaaraera nairadaeshanaaa

kaiibhaaabaee aii chaekalaisata bayabahaaara karabaena

CHECKLIST USAGE GUIDE

Status Icons:

- = Not Started (shaurau hayanaai)
- = In Progress (chalamaana)
- [x] = Complete (samapanana)
- = Blocked/Issue (baaadhaaagarasata/samasayaaa)

Priority Levels:

- * High (uchacha) - agaraadhaikaaara sarabaochacha
- Medium (maajhaaraai) - gaurautabapauurana
- o Low (kama) - aichachhaika

How to Use:

- parataitai Phase aira janaya checklist daekhauna
 - parataitai task aira status update karauna
 - Priority anauyaaayaii kaaaja karauna
 - Notes ai gaurautabapauurana tathaya laikhauna
 - Regularly update karauna (sapataaahae aikabaaara)
-

38.2 Phase 1: Idea Phase (dhaaaranaaa parayaaaya)

IDEA PHASE CHECKLIST

dhaaaranaaa parayaaaya chaekalaisata

CATEGORY: Concept Development

- High Game concept written
Notes: gaemaera mauula dhaaaranaaa laikhauna
- High Genre selected
Notes: kaona genre nairadhhaarana karaechhaena
- High Target audience defined
Notes: kaaara janaya gaema baaanaaachachhaena
- High Platform selected
Notes: PC, Mobile, Console - kaonatai
- Medium Unique selling points identified
Notes: gaematai kainaaa aalaaadaaa
- Medium Core gameplay loop defined
Notes: gaemapalae laupa kaii
- Low Inspiration games listed
Notes: kaona gaema thaekae inspiration naiyaechhaena

CATEGORY: Initial Research

- High Market research completed
Notes: baaajaaara gabaessanaaa karaechhaena kainaaa
- High Competitor analysis done
Notes: parataiyaogaii gaema baishalaessana
- Medium Platform requirements reviewed
Notes: Platform-aira requirements
- Medium Target hardware specs defined
Notes: Minimum/Recommended specs
- Low Similar games played and analyzed
Notes: anaurauupa gaema khaelae baishalaessana

CATEGORY: Initial Planning

- High Development timeline estimated
Notes: aanaumaaanaika समयारेकहा
- High Budget estimated
Notes: aanaumaaanaika baaajaeta
- Medium Team size determined
Notes: Solo naakai Team
- Medium Technology stack selected
Notes: Engine, tools, software
- Low Development environment setup planned
Notes: kaotaaaya kaaaja karabaena

38.3 Phase 2: Research Phase (gabaessanaaa parayaaaya)

RESEARCH PHASE CHECKLIST

gabaessanaaa parayaaaya chaekalaisata

CATEGORY: Market Research

- High Steam market analysis completed
Notes: Steam-ai baaajaaara baishalaessana
- High Genre popularity assessed
Notes: Genre-aira janapariyataaa
- High Price point analysis done
Notes: daaama nairadhhaaranaera gabaessanaaa
- Medium Audience demographics researched
Notes: darashakadaera baaishaissataya
- Medium Trend analysis completed
Notes: baratamaana trends
- Low International market considered
Notes: aanatarajaaataika baaajaaara

CATEGORY: Technical Research

- High Engine evaluation completed

- Notes: Unity, Unreal, Godot - comparison
- High Platform requirements documented
 - Notes: parataitai platform-aira requirements
- Medium Technical feasibility assessed
 - Notes: parayaukataigata samabhaabanaaa
- Medium Performance targets defined
 - Notes: FPS, loading times, etc.
- Low Third-party tools evaluated
 - Notes: Middleware, plugins, etc.

CATEGORY: Art Research

- High Art style reference collected
 - Notes: Art style aira reference images
- High Color palette defined
 - Notes: rangaera payaaalaeta nairadhaaarana
- Medium Asset requirements listed
 - Notes: kiai kiai assets darakaaara
- Medium Art pipeline established
 - Notes: Art taairaira parakaraiyaaa
- Low Outsourcing needs assessed
 - Notes: baaairae thaekae karatae habae kainaaa

CATEGORY: Audio Research

- High Audio style defined
 - Notes: adaiau-aira dharana nairadhaaarana
- Medium Music approach decided
 - Notes: Original, stock, or outsourced
- Medium Sound effect sources identified
 - Notes: SFX aira usa
- Low Audio middleware evaluated
 - Notes: FMOD, Wwise, etc.

38.4 Phase 3: GDD Phase (gaema daijaaaina dakaumaenata parayaaaya)

GDD PHASE CHECKLIST

jaidaidai parayaaaya chaekalaisata

CATEGORY: Core Design

- High Game Overview section complete
 - Notes: gaemaera saaamagaraika barananaaa
- High Gameplay Mechanics documented
 - Notes: gaemapalae mechanics laikhae phaelaechhaena
- High Core Loop defined
 - Notes: mauula gameplay loop
- High Player Actions defined
 - Notes: Player-aira actions
- Medium Game Feel described
 - Notes: gaema feel kaemana habae
- Low Difficulty curve planned
 - Notes: kathainataaara parakaritai

CATEGORY: Story & Characters

- High Story outline written
 - Notes: galapaera rauuparaekhaaa
- High Main characters defined
 - Notes: paradhaaana characters
- Medium Character bios written
 - Notes: Character details
- Medium Dialogue system designed
 - Notes: salaaapa padadhatai
- Low Cutscene plan created

- Notes: Cutscene aira paraikalapanaaa
- CATEGORY: Level Design
- High Level list created
- Notes: katagaulao level, kائي kائي level
- High Level flow diagram
- Notes: Level progression map
- Medium Level themes defined
- Notes: parataitai level-aira theme
- Medium Puzzle mechanics designed
- Notes: Puzzle-aira dharana
- Low Secret areas planned
- Notes: gaopana area

- CATEGORY: Systems Design
- High Save/Load system designed
- Notes: saebha/laoda padadhatai
- High Menu system designed
- Notes: maenau padadhatai
- Medium Settings system designed
- Notes: Settings options
- Medium Achievement system designed
- Notes: Achievement-aira plan
- Low Leaderboard system designed
- Notes: Leaderboard plan

- CATEGORY: Technical Design
- High Architecture document created
- Notes: Technical architecture
- High Data structures defined
- Notes: Data structures
- Medium API design documented
- Notes: API design
- Medium Performance requirements defined
- Notes: Performance targets
- Low Scalability plan created
- Notes: Scalability plan

38.5 Phase 4: Prototype Phase (paraotaotaaaipa parayaaaya)

- PROTOTYPE PHASE CHECKLIST
- paraotaotaaaipa parayaaaya chaekalaisata
- CATEGORY: Setup
- High Project created in engine
- Notes: Engine-ai project setup
- High Version control setup
- Notes: Git repository setup
- High Development environment configured
- Notes: IDE, tools, etc.
- Medium Project structure established
- Notes: Folder structure
- Low Build pipeline setup
- Notes: Build process
- CATEGORY: Core Mechanics
- High Player controller implemented
- Notes: Movement, camera, interaction
- High Core gameplay mechanic working
- Notes: mauula mechanic prototype
- High Basic UI implemented
- Notes: Menu, HUD basics
- Medium Save system prototype
- Notes: Basic save/load

Low Settings menu prototype
Notes: Basic settings

CATEGORY: Testing

High Core mechanics playtested
Notes: Mechanics kaaaja karachhae kainaaa

High Game feel evaluated
Notes: Game feel kaemana

Medium Performance baseline established
Notes: Baseline performance

Low Technical limitations identified
Notes: kائي kائي samabhaba naya

CATEGORY: Documentation

Medium Prototype notes documented
Notes: Prototype-aira notes

Medium Feedback collected
Notes: Feedback sagaraha

Low Lessons learned documented
Notes: yaaa shaikhaechhaena saetai laikhauna

38.6 Phase 5: Art Production (shailapa upaaadana)

ART PRODUCTION CHECKLIST shailapa upaaadana chaekalaisata

CATEGORY: Concept Art

High Character concept art
Notes: Character design sheets

High Environment concept art
Notes: Level/environment designs

Medium Prop concept art
Notes: Props and objects

Low UI concept art
Notes: UI elements

CATEGORY: 2D Art

High Character sprites/models
Notes: Character art assets

High Environment art
Notes: Backgrounds, tiles, etc.

Medium Animation frames
Notes: Sprite sheets, animations

Low Particle effects
Notes: Visual effects

CATEGORY: 3D Art (if applicable)

High 3D models created
Notes: Characters, props, environment

High Textures created
Notes: PBR textures

Medium Materials created
Notes: Material shaders

Medium Lighting setup
Notes: Light maps, real-time lighting

Low Post-processing effects
Notes: Bloom, color grading, etc.

CATEGORY: UI Art

High Main menu art
Notes: Menu background, buttons

High HUD elements
Notes: Health, score, etc.

Medium Icons and symbols
Notes: Game icons

Low

Loading screen

Notes: Loading art

38.7 Phase 6: Programming (paraogaraaamai)

PROGRAMMING CHECKLIST

paraogaraaamai chaekalaisata

- CATEGORY: Core Systems
- High
- Player Controller
- Notes: Movement, camera, input
- High
- Game Manager
- Notes: Game state management
- High
- Scene Manager
- Notes: Scene loading/transitions
- High
- Audio Manager
- Notes: Sound playback, music
- Medium
- Save/Load System
- Notes: Data persistence
- Medium
- Settings System
- Notes: Options, preferences
- Low
- Achievement System
- Notes: Achievement tracking
- CATEGORY: Gameplay Systems
- High
- Interaction System
- Notes: Object interaction
- High
- Inventory System (if applicable)
- Notes: Item management
- High
- Combat System (if applicable)
- Notes: Combat mechanics
- Medium
- AI System (if applicable)
- Notes: Enemy/NPC AI
- Medium
- Dialogue System (if applicable)
- Notes: Conversation system
- Low
- Quest System (if applicable)
- Notes: Quest tracking
- CATEGORY: UI Systems
- High
- Main Menu
- Notes: Start, options, quit
- High
- HUD
- Notes: In-game UI
- Medium
- Pause Menu
- Notes: Pause functionality
- Medium
- Settings Menu
- Notes: Graphics, audio, controls
- Low
- Credits Screen
- Notes: Credits display
- CATEGORY: Technical Systems
- High
- Input System
- Notes: Keyboard, mouse, gamepad
- High
- Collision System
- Notes: Physics, collision detection
- Medium
- Pooling System
- Notes: Object pooling
- Low
- Event System
- Notes: Event handling

38.8 Phase 7: Audio Production (adaiau upaaadana)

AUDIO PRODUCTION CHECKLIST adaiau upaaadana chaekalaisata

CATEGORY: Music

- High Main theme composed
Notes: paradhaaana thaima sagaiita
- High Menu music composed
Notes: maenau sagaiita
- High Gameplay music composed
Notes: gaemapalae sagaiita
- Medium Boss battle music (if applicable)
Notes: Boss battle music
- Medium Ambient tracks
Notes: paraibaesha sagaiita
- Low Victory/defeat music
Notes: jaya/paraaajaya sagaiita

CATEGORY: Sound Effects

- High Player sound effects
Notes: Footsteps, actions
- High UI sound effects
Notes: Button clicks, menu sounds
- High Environmental sounds
Notes: Ambient sounds
- Medium Combat sounds (if applicable)
Notes: Attack, hit, death sounds
- Medium Interaction sounds
Notes: Pick up, use, open sounds
- Low Notification sounds
Notes: Achievement, level up, etc.

CATEGORY: Audio Integration

- High Music integrated in game
Notes: Music playback working
- High SFX integrated in game
Notes: Sound effects working
- Medium Audio settings implemented
Notes: Volume controls
- Low Audio optimization
Notes: Memory, streaming

38.9 Phase 8: Level Design (laebhaela daijaaaina)

LEVEL DESIGN CHECKLIST laebhaela daijaaaina chaekalaisata

CATEGORY: Planning

- High Level flow diagram created
Notes: Level progression map
- High Level themes defined
Notes: parataitai level-aira theme
- Medium Difficulty curve planned
Notes: kathainataaara parakaritai
- Low Secret areas planned
Notes: gaopana area

CATEGORY: Blockout

- High Level 1 blockout
Notes: parathama level-aira blockout
- High Level 2 blockout

Notes: dabaitaiiya level-aira blackout
 Medium Level 3 blackout
 Notes: taritaiiya level-aira blackout
 Medium Level 4 blackout
 Notes: chatauratha level-aira blackout
 Low Level 5 blackout
 Notes: panyachama level-aira blackout
 CATEGORY: Production
 High Level 1 art pass
 Notes: Art integration
 High Level 2 art pass
 Notes: Art integration
 Medium Level 3 art pass
 Notes: Art integration
 Medium Level 4 art pass
 Notes: Art integration
 Low Level 5 art pass
 Notes: Art integration
 CATEGORY: Polish
 High Lighting pass
 Notes: Lighting setup
 Medium Atmosphere pass
 Notes: Fog, particles, effects
 Low Optimization pass
 Notes: Performance optimization

38.10 Phase 9: UI/UX (iuaai/iuaikasa)

UI/UX CHECKLIST iuaai/iuaikasa chaekalaisata

CATEGORY: Main Menu
 High Start Game button
 Notes: natauna gaema shaurau
 High Continue Game button
 Notes: chalamaana gaema chaaalaiyae yaaaauyaaa
 High Options button
 Notes: Settings menu
 Medium Quit Game button
 Notes: gaema banadha karaaa
 Low Credits button
 Notes: Credits display
 CATEGORY: HUD
 High Health display
 Notes: Health bar/number
 High Score display
 Notes: Score counter
 Medium Timer display
 Notes: Time counter (if applicable)
 Low Minimap
 Notes: Minimap (if applicable)
 CATEGORY: Settings Menu
 High Graphics settings
 Notes: Resolution, quality, fullscreen
 High Audio settings
 Notes: Volume controls
 High Input settings
 Notes: Key bindings, sensitivity
 Medium Accessibility settings
 Notes: Color blind, subtitles, etc.

- Low Language settings
 Notes: Localization
- CATEGORY: In-Game UI
- High Pause menu
 Notes: Pause functionality
- High Inventory screen
 Notes: Item display (if applicable)
- Medium Map screen
 Notes: Full map (if applicable)
- Low Journal/log screen
 Notes: Story/log (if applicable)

38.11 Phase 10: Save System (saebha saisataema)

- SAVE SYSTEM CHECKLIST
- saebha saisataema chaekalaisata
- CATEGORY: Core Save System
- High Save game data structure defined
 Notes: kائي kائي data save habae
- High Save file location determined
 Notes: kaotaaaya save habae
- High Save function implemented
 Notes: Save functionality
- High Load function implemented
 Notes: Load functionality
- Medium Auto-save implemented
 Notes: Automatic save
- Medium Multiple save slots
 Notes: aikaaadhaika save slot
- Low Save file versioning
 Notes: Version compatibility
- CATEGORY: Save Data
- High Player position saved
 Notes: Player location
- High Player stats saved
 Notes: Health, inventory, etc.
- Medium World state saved
 Notes: Environment changes
- Medium Quest progress saved
 Notes: Quest state
- Low Settings saved
 Notes: Player preferences
- CATEGORY: Save System Polish
- Medium Save confirmation UI
 Notes: Save success message
- Medium Load confirmation UI
 Notes: Load success message
- Low Save file corruption handling
 Notes: Error handling
- Low Save file backup
 Notes: Backup system

38.12 Phase 11: QA Testing (kaiuai paraiikassaaa)

- QA TESTING CHECKLIST
- kaiuai paraiikassaaa chaekalaisata

CATEGORY: Functional Testing

- High Main menu functions correctly
Notes: Menu navigation
- High Game starts properly
Notes: Game launch
- High Gameplay works correctly
Notes: Core mechanics
- High Save/Load works correctly
Notes: Save functionality
- Medium Settings work correctly
Notes: Options functionality
- Medium Quit game works
Notes: Exit functionality
- Low All buttons responsive
Notes: UI responsiveness

CATEGORY: Bug Testing

- High No game-breaking bugs
Notes: Critical bugs fixed
- High No progression blockers
Notes: Can complete game
- Medium Minor bugs documented
Notes: Minor issues logged
- Low Cosmetic issues logged
Notes: Visual issues

CATEGORY: Performance Testing

- High FPS meets target
Notes: Frame rate stable
- High Loading times acceptable
Notes: Load speed
- Medium Memory usage stable
Notes: No memory leaks
- Low CPU/GPU usage acceptable
Notes: Resource usage

CATEGORY: Compatibility Testing

- High Works on minimum specs
Notes: Low-end hardware
- Medium Works on recommended specs
Notes: Mid-range hardware
- Low Works on high-end specs
Notes: High-end hardware

CATEGORY: User Experience Testing

- High Controls feel good
Notes: Input responsiveness
- High Game is intuitive
Notes: Easy to understand
- Medium Difficulty is appropriate
Notes: Not too hard/easy
- Low Enjoyable experience
Notes: Fun factor

38.13 Phase 12: Optimization (apataimaaaijaeshana)

OPTIMIZATION CHECKLIST
apataimaaaijaeshana chaekalaisata

CATEGORY: Graphics Optimization

- High Texture optimization
Notes: Texture sizes, compression
- High Model optimization
Notes: Polygon count reduction

Medium Draw call optimization
Notes: Batch rendering

Medium LOD implementation
Notes: Level of detail

Low Occlusion culling
Notes: Hidden object culling

CATEGORY: Code Optimization

High Memory leak fixes
Notes: No memory leaks

High CPU optimization
Notes: Algorithm optimization

Medium Object pooling
Notes: Reuse objects

Low Code profiling
Notes: Performance profiling

CATEGORY: Loading Optimization

High Scene loading optimized
Notes: Fast scene loads

Medium Asset loading optimized
Notes: Async loading

Low Memory management
Notes: Efficient memory usage

38.14 Phase 13: Trailer/Marketing Materials (taraeilaaara/maarakataetai mayaaataeraiyaaala)

TRAILER/MARKETING MATERIALS CHECKLIST

taraeilaaara/maarakataetai mayaaataeraiyaaala chaekalaisata

CATEGORY: Trailer

High Trailer concept written
Notes: Trailer-aira dhauuparaekhaaa

High Trailer footage captured
Notes: Gameplay footage

High Trailer edited
Notes: Editing samapanana

Medium Trailer music selected
Notes: Trailer music

Medium Trailer text/titles added
Notes: Text overlays

Low Trailer rendered
Notes: Final render

CATEGORY: Screenshots

High 10+ high-quality screenshots
Notes: Screenshot collection

High Screenshots show variety
Notes: Different scenes

Medium Screenshots optimized
Notes: Resolution, file size

Low Screenshots organized
Notes: File organization

CATEGORY: Press Kit

High Press release written
Notes: Press release

High Game description written
Notes: Short/long descriptions

Medium Developer bio written
Notes: About the developer

Medium Contact information provided

Notes: Contact details

Low Press kit packaged

Notes: ZIP file ready

CATEGORY: Social Media Materials

Medium Social media graphics created

Notes: Banners, posts

Medium Social media copy written

Notes: Posts, updates

Low Social media schedule planned

Notes: Posting schedule

38.15 Phase 14: Store Page (sataora paeja)

STORE PAGE CHECKLIST

sataora paeja chaekalaisata

CATEGORY: Steam Page

High Game title set

Notes: gaemaera naaama

High Short description written

Notes: chhaota barananaaa (300 shabada)

High Long description written

Notes: baisataaaraita barananaaa

High Tags selected

Notes: Genres, features

High System requirements set

Notes: Min/recommended specs

Medium Release date set

Notes: Launch date

Medium Price set

Notes: Game price

Low Developer info added

Notes: About developer

CATEGORY: Store Assets

High Header image uploaded

Notes: Store header

High Screenshots uploaded

Notes: 5+ screenshots

High Trailer uploaded

Notes: Store trailer

Medium Library image uploaded

Notes: Library capsule

Low Logo uploaded

Notes: Store logo

CATEGORY: Additional Platforms

Medium itch.io page set up

Notes: itch.io listing

Low Epic Games Store page

Notes: Epic listing

Low GOG page

Notes: GOG listing

38.16 Phase 15: Marketing Campaign (maarakatai kayaaamapaeina)

MARKETING CAMPAIGN CHECKLIST

maarakatai kayaaamapaeina chaekalaisata

CATEGORY: Pre-Launch Campaign

High Steam page live (6 months before launch)
Notes: Steam page with wishlisting

High Social media accounts created
Notes: Twitter, Discord, etc.

High Regular devlogs started
Notes: Development updates

Medium Demo released (3 months before launch)
Notes: Demo build

Medium Press outreach started
Notes: Press contacts

Low Streamer outreach started
Notes: Streamer contacts

CATEGORY: Launch Campaign

High Launch trailer released
Notes: Final trailer

High Press release sent
Notes: Launch announcement

High Social media blitz
Notes: Launch day posts

Medium Streamer keys sent
Notes: Review copies

Medium Launch day plan executed
Notes: Launch activities

Low Community engagement
Notes: Discord, Reddit

CATEGORY: Post-Launch Campaign

Medium Bug fixes released
Notes: Day 1 patch

Medium Community feedback monitored
Notes: Reviews, comments

Low Sale participation
Notes: Steam sales

Low Content updates planned
Notes: Future updates

38.17 Phase 16: Launch Preparation (lanyacha parasatautai)

LAUNCH PREPARATION CHECKLIST

lanyacha parasatautai chaekalaisata

CATEGORY: Final Build

High Final build created
Notes: Release build

High Build tested on target platform
Notes: Platform testing

High All bugs fixed
Notes: No known bugs

Medium Performance optimized
Notes: Meets performance targets

Low Final polish applied
Notes: Final touches

CATEGORY: Launch Checklist

High Store page finalized
Notes: All assets uploaded

High Build uploaded to store
Notes: Steam build uploaded

High Release date confirmed
Notes: Launch date set

Medium Marketing materials ready
Notes: Trailer, screenshots

- Medium Press kit distributed
 - Notes: Sent to press
- Low Launch day plan reviewed
 - Notes: Plan finalized
- CATEGORY: Technical Preparation
 - High Day 1 patch prepared
 - Notes: Launch patch ready
 - High Bug tracking system ready
 - Notes: Bug reporting setup
 - Medium Support channels ready
 - Notes: Email, Discord support
 - Low Analytics set up
 - Notes: Tracking setup

38.18 Phase 17: Post-Launch (paosata-lanyacha)

- POST-LAUNCH CHECKLIST
 - paosata-lanyacha chaekalaisata
- CATEGORY: Immediate Post-Launch (Week 1)
 - High Monitor player feedback
 - Notes: Reviews, comments
 - High Fix critical bugs
 - Notes: Day 1-7 patches
 - High Respond to community
 - Notes: Community engagement
 - Medium Track sales data
 - Notes: Sales analytics
 - Low Social media engagement
 - Notes: Thank players
- CATEGORY: Short-Term Post-Launch (Month 1)
 - High Address major issues
 - Notes: Important fixes
 - Medium Release stability patch
 - Notes: Performance fixes
 - Medium Analyze player data
 - Notes: Player behavior
 - Low Plan first update
 - Notes: Content update plan
- CATEGORY: Long-Term Post-Launch (Month 2-6)
 - Medium Content updates
 - Notes: New content
 - Medium Community events
 - Notes: Community activities
 - Low DLC planning (if applicable)
 - Notes: DLC concepts
 - Low Post-mortem analysis
 - Notes: What worked, what didn't

38.19 How to Use This Checklist

daainaika bayabahaaara

DAILY USAGE

sakaaalae:

- 1. Checklist khaulauna
- 2. aajakaera kaaaja daekhauna
- 3. Priority anauyaaayaii kaaaja karauna

sanadhayaaaya:

- 1. Status update karauna
- 2. Notes yaoga karauna
- 3. paradainaera kaaaja plan karauna

saaapataaahaika bayabahaaara

WEEKLY USAGE

shaukarabaaara:

- 1. sapataaahaera progress review karauna
- 2. Completed tasks tick karauna
- 3. Blocked items identify karauna
- 4. Next week plan karauna
- 5. Priorities adjust karauna

maaasaika bayabahaaara

MONTHLY USAGE

maaasa shaessae:

- 1. Monthly milestone check karauna
- 2. Overall progress assess karauna
- 3. Timeline adjust karauna
- 4. Budget review karauna
- 5. Goals reset karauna

38.20 Customization

naijaera janaya kaaasatamaaija karauna

CUSTOMIZATION GUIDE

- 1. RELEVANT ITEMS
 - aapanaaara gaemaera janaya relevant items baaachhauna
 - Irrelevant items remove karauna
 - natauna items yaoga karauna
 - 2. PRIORITY ADJUSTMENT
 - aapanaaara priority anauyaaayaii adjust karauna
 - Game-specific priorities set karauna
 - samayaera saaathae update karauna
 - 3. NOTES CUSTOMIZATION
 - naijaera notes yaoga karauna
 - Specific details add karauna
 - paraibaratana track karauna
 - 4. STATUS TRACKING
 - Regular update karauna
 - Progress track karauna
 - Milestones mark karauna
-

38.21 Exercise

anaushailana 1: Checklist Customize karauna

1. uparaera checklist daekhauna
2. aapanaaara gaemaera janaya relevant items baaachhauna
3. Irrelevant items remove karauna
4. natauna items yaoga karauna
5. aikatai customized checklist taairai karauna

anaushailana 2: baratamaaana sathaitai mauulayaaayana karauna

1. aapanaaara baratamaaana parajaekata daekhauna
2. kaona phase ai aachhaena saetai nairadhhaarana karauna
3. kii kii samapanana hayaechhae saetai tick karauna
4. kii kii baaakai aachhae saetai identify karauna
5. parabarataii steps plan karauna

anaushailana 3: Weekly Review karauna

1. aii checklist bayabahaaara karae aika sapataaaha kaaaja karauna
2. parataidaina status update karauna
3. shaukarabaaarae weekly review karauna
4. Progress assessment karauna
5. Next week plan karauna

38.22 saaadhaaarana bhaula (Common Mistakes)

bhaura 1: Not Using the Checklist

bhaura: Checklist taairai karae bayabahaaara naaa karaaa
sathaika: parataidaina checklist update karauna

bhaura 2: Skipping Items

bhaura: kaichhau items skip karaaa
sathaika: parataitai item check karauna

bhaur 3: Not Updating Status

bhaura: Status update naaa karaaa
sathaika: parataidaina status update karauna

bhaura 4: Ignoring Priority

bhaura: Priority ignore karae kaaaja karaaa
sathaika: Priority anauyaaayaii kaaaja karauna

bhaura 5: No Regular Review

bhaura: Regular review naaa karaaa

sathaika: sapataaahae aikabaaara review karauna

38.23 taipasa (Tips)

taipasa 1: Start Early

chaekalaisata yata taaadaataaadai shaurau karabaena tata bhaaalao

taipasa 2: Be Consistent

parataidaina update karauna, maaajhae maaajhae naya

taipasa 3: Review Regularly

sapataaahae aikabaaara review karauna

taipasa 4: Customize

naijaera parayaojana anauyaaayaii customize karauna

taipasa 5: Celebrate Progress

parataitai completed item celebrate karauna

38.24 saarasakassaepa (Summary)

aai adhayaaayae aamaraaa aikatai samapaurana Game Development Production Checklist daekhaechhai:

samapaurana Phases:

1. **Idea Phase** - dhaarananaa parayaaaya
2. **Research Phase** - gabaessanaa parayaaaya
3. **GDD Phase** - gaema daijaaaina dakaumaenata parayaaaya
4. **Prototype Phase** - paraotaotaaipa parayaaaya
5. **Art Production** - shailapa upaadana
6. **Programming** - paraogaraamai
7. **Audio Production** - adaiau upaadana
8. **Level Design** - laebhaela daijaaaina
9. **UI/UX** - iuaai/iuaikasa
10. **Save System** - saebha saisataema
11. **QA Testing** - kaiuai paraiikassaaa
12. **Optimization** - apataimaaaijaeshana
13. **Trailer/Marketing** - taraeilaaara/maaraketai
14. **Store Page** - sataora paeja
15. **Marketing Campaign** - maaaraketai kayaaamapaeina
16. **Launch Preparation** - lanyacha parasatautai
17. **Post-Launch** - paosata-lanyacha

manae raaakhauna:

aii checklist aikatai tool, aikatai rule naya
aapanaaara parayaojana anauyaaayaii customize karauna
Regular update karauna, progress track karauna

****aitai chhaila Chapter 38: Complete Production Checklist****

****GAME DEVELOPMENT MASTERBOOK - COMPLETE!****

PART 39

Chapter 39

adhayaaaya 39: TEMPLATES -- raiiuaebala taemapalaeta laaibaraera

paraichaitai

aia adhayaaayae aamaraa 15tai baiaataaraita raiiuaebala (Reusable) taemapalaeta taairai karaba yaegaulao gaema daebhaelapamaenataera parataitai parayaaayae bayabahaaara karaa yaaabae parataitai taemapalaeta inadaasatarai satayaaanadaaarada anausarana karae taairai karaa hayaechhae aiba aita samapauurana nairadaeshanaa au udaaharana maaana anatarabhaukata rayaechhae

taemapalaeta 1: Game Idea Template (gaema aaidaiyaaa taemapalaeta)

udadaeshaya

natauna gaema aaidaiyaaa sagathaita karaara janaya aia taemapalaeta bayabahaaara karauna aita apanaara aaidaiyaaakae aikatai paraissakaaara au sausagathaita pharamayaaatae upasathaaapana karabae

taemapalaeta

GAME IDEA TEMPLATE
(gaema aaidaiyaaa taemapalaeta)

1. gaemaera naama (Game Name)

udaaharana: "Mystic Realms: The Lost Kingdom"

nairadaeshanaa: aimana naama daina yaetai gaemaera thaima parataiphalaeta karae

2. dharana (Genre)

paraathamaika: Action RPG

saekaanadaarai: Exploration, Puzzle

nairadaeshanaa: mauula jaenaaara aiba saaaba-jaenaaara ubhayai ulalaekha karauna

3. palayaaatapharama (Platform)

PC (Steam/Epic)

PlayStation 5

Xbox Series X|S

Nintendo Switch

Mobile (iOS/Android)

Web Browser

nairadaeshanaaa: saba palayaaatapharama chaihanaite karauna

4. taaaragaeta adaiyaenasa (Target Audience)

bayasa: 13-35 bachhara

jaenadaaara: sakala

gaemai abhaijanyataaa: maaajhaaarai thaekae unanata

aagaraha: galapa-chaalaita abhaijanyataaa, chayaaalaenyajai kamabayaaata

nairadaeshanaaa: aapanaara aadaraya khaelaoyaaadaera paraophaaaila taairai karauna

5. kaora maekaaanaika (Core Mechanic)

paraaathamaika: Elemental Magic Combat System

saekaaenadaaarai: Dungeon Exploration, Crafting

nairadaeshanaaa: sabachaeyae gaurautabapauurana khaelaara maekaaanaika barananaaa karauna

6. iunaika hauka (Unique Hook)

"Elemental shakatai paraibaratana karae paaajala au yaudadha samaaadhaana karauna"

nairadaeshanaaa: kaena aii gaema anayadaera thaekae aalaaadaaa?

7. aarata sataaaila (Art Style)

sataaaila: Stylized Fantasy

raephaaraenasa: Genshin Impact, Breath of the Wild

payaaalaeta: Pastel colors with vibrant magical effects

nairadaeshanaaa: bhajayauyaaala aaidaenataitai barananaaa karauna

8. thaima (Theme)

mauula thaima: Discovery and Mastery

saaaba-thaima: Nature vs Corruption, Friendship

nairadaeshanaaa: gaemaera gabhaiiratama aratha baaa baaarataaa kaii?

9. saetai (Setting)

samayakaaala: Ancient Fantasy Era

abasathaaana: Floating Islands, Ancient Ruins, Dark Forests

baishabaera aaina: Elemental Magic System

nairadaeshanaaa: gaemaera baishabaera paraekassaaapata barananaaa karauna

10. kaora phayaaanataasai (Core Fantasy)

"aapanai aikajana haaaraanao Elementalist yainai paraaachaiina shakatai

phairaiyae ainae baishabakae rakassaaa karabaena"

nairadaeshanaaa: khaelaoyaaada kaii anaubhaba karatae chaaana?

11. palaeyaaara maotaibhaeshana (Player Motivation)

Mastery (dakassataaa arajana)
Exploration (anausanadhaaana)
Story (galapa)
Collection (sagaraha)
Social (saaamaajaika)
Competition (parataiyaogaitaaa)
nairadaeshanaaa: khaelaoyaaadadaera kae khaelatae udabaudadha karabae?

12. USP (Unique Selling Proposition)

"Elemental shakatai paraibaratana karae paaajala au yaudadha
samaaadhaaana karaaara ananaya abhaijanyataaa"
nairadaeshanaaa: baaajaaarae aii gaemaera sabachaeyae shakataishaaalarii payaenata kaii?

bayabahaaaraera nairadaeshanaaa

1. parataitai phailada samapauurana karauna
2. udaaaharana maaana shaudhau raephaaaraenasa haisaebae bayabahaaara karauna
3. aapanaaara naijaera aaidaiyaaa anauyaaayaii paraibaratana karauna
4. saba phailada phaaakaaa raaakhabaena naaa

taemapalaeta 2: Game Pitch Template (gaema paicha taemapalaeta)

udadaeshaya

aapanaaara gaema aaidaiyaaakae paaabalaishaaara, inabhaesatara baaa taima maemabaaaradaera kaaachhae
upasathaaapana karaaara janaya aii taemapalaeta bayabahaaara karauna

taemapalaeta

GAME PITCH TEMPLATE
(gaema paicha taemapalaeta)

1. samasayaaa (Problem)

baratamaaanae baaajaaarae Element-based RPG gaulao shaudhau yaudadhae
Element bayabahaaara karae, kainatau paaajala samaaadhaaanae bayabahaaara karae naaa
khaelaoyaaadaraaa aikai dharanaera abhaijanyataaa paaachachhaena

nairadaeshanaaa: baaajaaarae kaona samasayaaa baaa phaaaka aachhae kai?

2. phayaaanataaasai (Fantasy)

"aapanai aikajana Elementalist haisaebae paraaachaiina shakatai naiyanatarana

karae paaajala samaaadhaana karauna aiba shataudaera paraajaita karauna
parataitai Element aikatai natauna samabhaabanaaa khaulae daebae"

nairadaeshanaaa: aika baaakayae gaemaera saaraaasha daina

3. gaemapalae (Gameplay)

- raiyaela-taaaima kamabayaaata saisataema
- Element sauichai maekaaanaika
- ainabhaayaranamaenataala paaajala
- Character Progression System
- Crafting aiba Upgrades

nairadaeshanaaa: mauula gaemapalae ailaimaenatagaulao taaalaikaaabhaukata karauna

4. iunaika phaichaaara (Unique Feature)

"Elemental Synergy System - dautai Element aikasaaathae
bayabahaara karalae natauna shakatai taairai haya"

nairadaeshanaaa: parataiyaogaiidaera thaekae aalaaadaaa karae taolaaara phaichaaara

5. adaiyaenasa (Audience)

paraaathamaika: 16-30 bachhara bayasaii RPG bhakataraaa
saekaenadaaarai: Action-Adventure khaelaoyaaadaraaa
TAM: 50 mailaiyana+

nairadaeshanaaa: kaaaraaa aii gaema khaelabae taaa nairadhaaarana karauna

6. maaarakaeta aparachaunaitai (Market Opportunity)

baaajaaaraera mauulaya: bailaiyana (2024)
baridadhaira haaara: XX% CAGR
parataiyaogaitaaa: maaajhaaarai (Medium Competition)
sauyaoga: Element-based paaajala RPG aira chaaahidaaa baaadachhae

nairadaeshanaaa: baaajaaaraera samabhaabanaaa baishalaessana karauna

7. baijanaesa madaela (Business Model)

Premium (.99)
Free-to-Play (IAP)
Subscription
Hybrid (Premium + DLC)
aayaera anaumaaana: mailaiyana (bachhara 1)

nairadaeshanaaa: kaiibhaabae taaakaaa aaya karabaena taaa ulalaekha karauna

paicha taaaimalaaaina

- **60 saekænada paicha**: shaudhau phayaaanataaasai au iunaika phaichaaara
- **5 mainaita paicha**: saba phailada kabhaaara karauna
- **15 mainaita paicha**: daemao saha baisataaaraita bayaaakhayaaa

taemapalaeta 3: GDD Template (Game Design Document Template)

udadaeshaya

samapauurana Game Design Document taairai karaara janaya aii 15-saekashana taemapalaeta bayabahaaara karauna

taemapalaeta

GAME DESIGN DOCUMENT (GDD) TEMPLATE
(gaema daijaaaina dakaumaenata taemapalaeta)

SECTION 1: OVERVIEW (saaaraaasha)

1.1 Executive Summary (nairabaaahaii saaaraaasha)

- gaemaera aika parissathaaara saaaraaasha
- mauula phaichaaara au saubaidhaaa
- lakassaya adaiyaenasa

1.2 Game Overview (gaemaera saaaraaasha)

- Game Title: _____
- Version: _____
- Genre: _____
- Platform: _____
- Players: Single/Multiplayer
- Estimated Play Time: _____

1.3 High Concept (uchacha dhaaaranaaa)

- gaemaera mauula dhaaaranaaa aika baaakayae
- kaena aitai gaurautabapauurana

1.4 Unique Selling Points (ananaya baikaraya payaenata)

- USP 1: _____
- USP 2: _____
- USP 3: _____

1.5 Target Audience (lakassaya adaiyaenasa)

- Primary: _____
- Secondary: _____
- Demographics: _____

SECTION 2: STORY AND LORE (galapa au laora)

2.1 Story Synopsis (galapaera saaaraaasha)

- mauula galapaera sauuchanaaa, maaajhhaarai au shaessa
- Protagonist: _____
- Antagonist: _____
- Central Conflict: _____

2.2 World Lore (baishabaera laora)

- History: _____
- Geography: _____
- Factions: _____
- Magic System: _____
- Technology Level: _____

2.3 Characters (charaitara)

- Main Characters: _____
- Supporting Characters: _____
- Character Arcs: _____

2.4 Dialogue System (salaaapa saisataema)

- Branching Dialogue: Yes/No
- Voice Acting: Yes/No
- Localization: _____

SECTION 3: GAMEPLAY (gaemapalae)

3.1 Core Gameplay Loop (mauula gaemapalae laupa)

- Explore -> Fight -> Loot -> Upgrade -> Repeat
- parataitai dhaapaera baisataaaraita barananaaa

3.2 Game Mechanics (gaema maekaaanaika)

- Movement: _____
- Combat: _____
- Inventory: _____
- Progression: _____
- Crafting: _____

3.3 Controls (kanataraola)

- Keyboard/Mouse Layout: _____
- Controller Layout: _____
- Touch Controls: _____

3.4 Game Flow (gaema phalao)

- Main Menu -> Gameplay -> Pause -> Options
- Win Conditions: _____
- Lose Conditions: _____

3.5 Difficulty System (kathainataaa saisataema)

- Easy: _____
- Normal: _____
- Hard: _____
- Adaptive Difficulty: Yes/No

3.6 Tutorial System (taiutaoraiyaaala saisataema)

- In-game Tutorials: _____
- Tooltips: _____
- Practice Mode: _____

SECTION 4: LEVEL DESIGN (laebhaela daijaaaina)

4.1 World Map (baishabaera mayaaapa)

- Regions: _____
- Biomes: _____
- Fast Travel Points: _____

4.2 Level List (laebhaela taaalaikaaa)

- Level 1: _____ (baisataaaraita)
- Level 2: _____ (baisataaaraita)
- Level 3: _____ (baisataaaraita)

4.3 Environment Art (paraibaesha aarata)

- Art Style Guide: _____
- Color Palette: _____
- Lighting: _____
- Weather System: _____

4.4 Props and Interactables (paraopasa au inataaaraayaaakataebala)

- Destructible Objects: _____
- Interactive Objects: _____
- Collectibles: _____

SECTION 5: COMBAT SYSTEM (kamabayaaata saisataema)

5.1 Combat Overview (kamabayaaata saaaraaasha)

- Type: Turn-based/Real-time/Hybrid
- Camera: First/Third Person/Top-down
- Complexity: _____

5.2 Weapons and Equipment (asatara au saranyajaaama)

- Melee Weapons: _____
- Ranged Weapons: _____
- Armor Types: _____
- Accessories: _____

5.3 Abilities and Skills (kassamataaaa au dakassataaaa)

- Active Abilities: _____
- Passive Abilities: _____
- Ultimate Abilities: _____
- Skill Trees: _____

5.4 Enemy Design (shatarau daijaaaina)

- Enemy Types: _____
- Boss Design: _____
- AI Behavior: _____

5.5 Damage System (kassatai saisataema)

- Damage Types: _____
- Resistances: _____
- Critical Hits: _____
- Status Effects: _____

SECTION 6: PROGRESSION (paragaraeshana)

6.1 Character Progression (charaitara paragaraeshana)

- Level Cap: _____
- XP System: _____
- Stats: _____
- Perks: _____

6.2 Equipment Progression (saranyajaaama paragaraeshana)

- Rarity Tiers: _____
- Upgrade System: _____
- Set Bonuses: _____

6.3 Unlock System (aanalaka saisataema)

- Content Unlocks: _____
- Achievement System: _____
- Reward Schedule: _____

SECTION 7: UI/UX (iuaai/iuaikasa)

7.1 Main Menu (maeina maenau)

- Layout: _____
- Options: _____
- Animation: _____

7.2 HUD (Heads-Up Display)

- Health Bar: _____
- Minimap: _____
- Quest Tracker: _____

- Quick Slots: _____

7.3 Inventory UI (inabhaenatarai UI)

- Layout: _____
- Sorting: _____
- Filtering: _____

7.4 Accessibility (ayaaakasaesaibailaitai)

- Colorblind Mode: _____
- Subtitles: _____
- Control Remapping: _____
- Screen Reader Support: _____

SECTION 8: AUDIO (adaiaiu)

8.1 Music (sangagaiita)

- Main Theme: _____
- Area Themes: _____
- Combat Music: _____
- Dynamic Music System: _____

8.2 Sound Effects (saaaunada iphaekata)

- UI Sounds: _____
- Combat Sounds: _____
- Environmental Sounds: _____
- Footsteps: _____

8.3 Voice Acting (bhayaesa ayaaakatai)

- Languages: _____
- Character Voices: _____
- Narrator: _____

SECTION 9: TECHNICAL (parayaukataigata)

9.1 Engine and Tools (inyajaina au taulasa)

- Game Engine: _____
- Level Editor: _____
- Version Control: _____
- Build System: _____

9.2 Performance Targets (paaarapharamayaaanasa taaaragaeta)

- Target FPS: _____
- Resolution: _____
- Load Times: _____
- Memory Usage: _____

9.3 Networking (naetaauyaaarakai)

- Online Features: _____
- Multiplayer: _____
- Server Architecture: _____

9.4 Platform Requirements (palayaaatapharama parayaojanaiiyataaa)

- PC Specs: _____
- Console Specs: _____
- Mobile Specs: _____

SECTION 10: MONETIZATION (manaitaaajaeshana)

10.1 Revenue Model (aayaera madaela)

- Base Price: _____
- DLC Plan: _____
- Microtransactions: _____
- Season Pass: _____

10.2 In-Game Economy (ina-gaema arathanaiitai)

- Currencies: _____
- Shop Items: _____
- Drop Rates: _____

10.3 Content Roadmap (kanataenata raodamayaaapa)

- Launch Content: _____
- Post-Launch Updates: _____
- DLC Schedule: _____

SECTION 11: MULTIPLAYER (maaalataipalaeyaaara)

11.1 Multiplayer Modes (maaalataipalaeyaaara maoda)

- Co-op: _____
- PvP: _____
- Raids: _____
- Guilds: _____

11.2 Social Features (saaamaajaika phaichaaara)

- Friends List: _____
- Chat System: _____
- Trading: _____
- Leaderboards: _____

SECTION 12: PRODUCTION (paraodaaakashana)

12.1 Team Structure (taima gathana)

- Team Size: _____
- Roles: _____

- Tools: _____

12.2 Development Timeline (daebhaelapamaenata taaaimalaaaaina)

- Pre-Production: _____
- Production: _____
- Post-Production: _____
- Total Duration: _____

12.3 Budget (baaajaeta)

- Total Budget: _____
- Breakdown: _____
- Funding Source: _____

SECTION 13: MARKETING (maarakatai)

13.1 Marketing Strategy (maarakatai kaaushala)

- Pre-Launch: _____
- Launch: _____
- Post-Launch: _____

13.2 PR and Media (PR au maidaiyaaa)

- Press Kit: _____
- Influencer Outreach: _____
- Social Media: _____

SECTION 14: RISK ASSESSMENT (jhaukai mauulayaaayana)

14.1 Technical Risks (parayaukataigata jhaukai)

- Engine Limitations: _____
- Performance Issues: _____
- Platform Certification: _____

14.2 Design Risks (daijaaaina jhaukai)

- Scope Creep: _____
- Player Retention: _____
- Balance Issues: _____

14.3 Business Risks (bayabasaaayaika jhaukai)

- Market Competition: _____
- Budget Overrun: _____
- Timeline Delays: _____

SECTION 15: APPENDIX (paraishaissata)

15.1 Reference Games (raephaaraenasa gaema)

- Game 1: _____ (kaena raephaaraenasa)

- Game 2: _____ (kaena raephaaaraenasa)
- Game 3: _____ (kaena raephaaaraenasa)

15.2 Glossary (shabadakaossa)

- Term 1: _____
- Term 2: _____
- Term 3: _____

15.3 Change Log (paraibaratanaera taaalaikaaa)

- Version | Date | Changes | Author
- 1.0 | _____ | _____ | _____

GDD laekhaaara nairadaeshanaaa

- 1. **Iterative Approach**: GDD aikabaaarae samapauurana laekhaaara chaessataaa karabaena naaa
- 2. **Living Document**: aital dhaaaraaabaahaikabhaaabaee aapadaeta karauna
- 3. **Visual Aids**: Wireframes, concept art yaoga karauna
- 4. **Consensus**: taimaera sabaaara saaathae yaaachaaai karauna
- 5. **Version Control**: parataitai paraibaratana tarayaaaka karauna

taemapalaeta 4: Level Design Template (laebhaela daijaaaina taemapalaeta)

udadaeshaya

parataitai laebhaelaera janaya baisataaaraita paraikalapananaa taairai karaaara janaya

taemapalaeta

LEVEL DESIGN TEMPLATE

(laebhaela daijaaaina taemapalaeta)

Level Name: _____

Level Number: _____

Duration: _____

1. Theme (thaima)

Main Theme: _____

Environment: Forest/Dungeon/City/Cave/Lava/Ice

Weather: Clear/Rain/Snow/Fog/Storm

Time: Day/Night/Dawn/Dusk

2. Difficulty (kathainataaa)

Overall: Easy/Medium/Hard/Expert

Combat: _____

Puzzle: _____

Player Level Range: _____

3. Enemies (shatarau)

Enemy Name	Level	Count	Location
_____	_____	_____	_____

Boss: _____

4. Puzzles (paaajala)

Puzzle Name	Type	Difficulty	Location
-------------	------	------------	----------

Death: _____
6. Voice (kanatha)
Type: _____
Lines: _____
Language: _____
7. Equipment (saranyajaaama)
Weapon: _____
Armor: _____
Accessory: _____
8. Personality (bayakataitaba)
Traits: _____
Quote: " _____ "
9. Gameplay Role (gaemapalae bhauumaikaaa)
Style: _____
Purpose: _____

taemapalaeta 6: Enemy Template (shatarau taemapalaeta)

ENEMY TEMPLATE
(shatarau taemapalaeta)

Enemy Name: _____
Type: Regular/Elite/Boss/Mini-Boss
Genre: Humanoid/Beast/Machine/Undead/Magic

1. Behavior (aacharana)
Normal: _____
Alert: _____
Attack: _____
Flee: _____
Group: _____

2. Attack Pattern (aakaramana payaaataaarana)

Attack Name	Damage	Range	Cooldown
_____	_____	_____	_____

Special: _____

3. Weakness (daurabalataaaa)
Damage Type: _____
Resistance: _____
Status Effect: _____

4. Stats (paraisakhayaaaana)
Health: _____
Attack: _____
Defense: _____
Speed: _____
XP: _____
Gold: _____
Drop Rate: _____

5. AI State (aiaai abasathaaa)
Idle: _____
Patrol: _____
Chase: _____
Attack: _____
Detection: _____
Aggro: _____

6. Loot (paurasakaaara)

Item	Drop %	Rarity
_____	_____	_____

7. Visual (bhaijayauyaaala)
Model: _____
Texture: _____
Animation: _____

Size: _____

taemapalaeta 7: Quest Template (kaoyaesata taemapalaeta)

QUEST TEMPLATE

(kaoyaesata taemapalaeta)

Quest Name: _____

Quest ID: _____

Type: Main/Side/Chain/Daily/Weekly/Event

1. Objective (udadaeshaya)

Primary: _____

Secondary: _____

Optional: _____

2. Description (baibarana)

Brief: _____

Detailed: _____

Lore: _____

3. Rewards (paurasakaaara)

XP: _____

Gold: _____

Item: _____

Reputation: _____

Unlock: _____

Title: _____

4. Prerequisites (pauurabasharata)

Level: _____

Quest: _____

Item: _____

Faction: _____

5. Dialog (salaaapa)

NPC: _____

Accept: _____

Progress: _____

Complete: _____

6. Enemies (shatarau)

Enemy	Level	Count	Location
_____	_____	_____	_____

Boss: _____

7. Map (maaanachaitara)

Start: _____

Objectives: _____

End: _____

8. Steps (dhaaapa)

Step 1: _____

Step 2: _____

Step 3: _____

Step 4: _____

Step 5: _____

9. Timeline (samayakaaala)

Time Limit: _____

Cooldown: _____

taemapalaeta 8: Mechanic Template (maekaaanaika taemapalaeta)

MECHANIC TEMPLATE

(maekaaanaika taemapalaeta)

Mechanic Name: _____

Category: Core/Secondary/Support/Utility

1. Purpose (udadaeshaya)

Why Needed: _____

Adds to Game: _____

Importance: _____

2. Input (inapauta)

Button/Key: _____

Mouse: _____

Controller: _____

Touch: _____

3. Rules (naiyama)

Activation: _____

Duration: _____

Cooldown: _____

Cost: _____

Restrictions: _____

4. Feedback (phaidabayaaaka)

Visual: _____

Audio: _____

Haptic: _____

UI: _____

5. Reward (paurasakaaara)

Direct: _____

Indirect: _____

Progression: _____

6. Failure State (bayarathataaa)

Condition: _____

Consequence: _____

Recovery: _____

7. Variables (bhaeraiyaebala)

Variable	Type	Default	Range
_____	_____	_____	_____

8. Balance (bayaaalaenasa)

Power Level: _____

Skill Ceiling: _____

Skill Floor: _____

Fun Factor: _____

taemapalaeta

9: Bug Report Template (baaaga raipaorata taemapalaeta)

BUG REPORT TEMPLATE

(baaaga raipaorata taemapalaeta)

Bug ID: BUG-XXXX-XXX

Title: _____

Reporter: _____

Date: _____

1. Description (baibarana)

What: _____

Where: _____

When: _____

How Often: _____

2. Steps to Reproduce (paunaraupaaadanaera dhaaapa)

Step 1: _____

Step 2: _____

Step 3: _____

Step 4: _____

Step 5: _____

3. Expected Result (paratayaaashaita phalaaaphala)

What should happen: _____

4. Actual Result (parakarita phalaaaphala)

What happened: _____

5. Severity (gaurautaba)

Critical (Game Crash)

Major (Feature Broken)

Minor (Minor Issue)

Trivial (Cosmetic)

6. Priority (agaraaadhaikaaara)

P1 - Immediate

P2 - High

P3 - Medium

P4 - Low

7. Platform (palayaaatapharama)

OS: _____

Hardware: _____

Game Version: _____

8. Screenshot/Video (sakarainashata/bhaidaiu)

Screenshot: [File Path]

Video: [File Path]

Log: [File Path]

9. Additional Info (atairaikata tathaya)

Workaround: _____

Related Bugs: _____

Notes: _____

10. Status (satayaaataaasa)

New Assigned In Progress

Fixed Verified Closed Reopened

Assignee: _____

Fix Version: _____

taemapalaeta 10: Sprint Template (saparainata taemapalaeta)

SPRINT TEMPLATE

(saparainata taemapalaeta)

Sprint Number: Sprint XX

Duration: XX Days (Usually 2 Weeks)

Start Date: _____

End Date: _____

1. Sprint Goal (saparainata gaola)

2. Task List (taaasaka taaalaikaaa)

ID	Task	Assignee	Points	Status
T-01	_____	_____	_____	_____
T-02	_____	_____	_____	_____
T-03	_____	_____	_____	_____
T-04	_____	_____	_____	_____
T-05	_____	_____	_____	_____

3. Story Points Summary

Planned: _____

Completed: _____

Velocity: _____

4. Daily Standup Notes

Day 1: Yesterday: ____ Today: ____ Blockers: ____

Day 2: Yesterday: ____ Today: ____ Blockers: ____

Day 3: Yesterday: ____ Today: ____ Blockers: ____

Day 4: Yesterday: ____ Today: ____ Blockers: ____

Day 5: Yesterday: ____ Today: ____ Blockers: ____

5. Sprint Review (saparainata raibhaiu)

Demo: _____

Feedback: _____

Criteria Met: _____

6. Retrospective (raetaraosapaekataibha)

What went well: _____

Improve: _____

Actions: _____

taemapalaeta

11: Production Schedule

(paraodaaakashana shaidaiula)

PRODUCTION SCHEDULE

(paraodaaakashana shaidaiula)

Project Name: _____

Total Duration: _____

PHASE 1: PRE-PRODUCTION (parai-paraodaaakashana)

ID	Task	Start	End	Status
PP-01	Concept Art	_____	_____	_____
PP-02	GDD Writing	_____	_____	_____
PP-03	Prototype	_____	_____	_____
PP-04	Tech Research	_____	_____	_____
PP-05	Team Setup	_____	_____	_____

Duration: _____ weeks

PHASE 2: VERTICAL SLICE (bhaaarataikayaaala salaaaisa)

VS-01	Core Mechanics	_____	_____	_____
VS-02	One Level	_____	_____	_____
VS-03	Art Pipeline	_____	_____	_____
VS-04	Audio Pipeline	_____	_____	_____
VS-05	QA Testing	_____	_____	_____

PHASE 3: PRODUCTION (paraodaaakashana)

PR-01	Levels	_____	_____	_____
PR-02	Characters	_____	_____	_____
PR-03	Enemies	_____	_____	_____
PR-04	Quests	_____	_____	_____
PR-05	UI	_____	_____	_____
PR-06	Audio	_____	_____	_____
PR-07	VFX	_____	_____	_____
PR-08	Animation	_____	_____	_____

PHASE 4: ALPHA (aalaphaaa)

AL-01	Feature Complete	_____	_____	_____
AL-02	Content Complete	_____	_____	_____
AL-03	Internal QA	_____	_____	_____
AL-04	Bug Fixing	_____	_____	_____
AL-05	Performance Opt.	_____	_____	_____

PHASE 5: BETA (baitaaa)

BE-01	Beta Testing	_____	_____	_____
BE-02	Bug Fixing	_____	_____	_____
BE-03	Balance Tuning	_____	_____	_____
BE-04	Polish	_____	_____	_____
BE-05	Certification	_____	_____	_____

PHASE 6: LAUNCH (lanyacha)

LA-01	Gold Master	_____	_____	_____
LA-02	Platform Submit	_____	_____	_____
LA-03	Marketing Push	_____	_____	_____
LA-04	Launch Support	_____	_____	_____

PHASE 7: POST-LAUNCH (paosata-lanyacha)

PL-01 Day-1 Patch	_____	_____	_____
PL-02 Week-1 Support	_____	_____	_____
PL-03 Month-1 Update	_____	_____	_____
PL-04 DLC Planning	_____	_____	_____

taemapalaeta 12: Budget Template (baajaeta taemapalaeta)

BUDGET TEMPLATE
(baajaeta taemapalaeta)

Project Name: _____

Total Budget: \$ _____

1. PERSONNEL (karamaii)

Category	Monthly	Duration	Total
Game Director	\$ _____	_____	\$ _____
Lead Programmer	\$ _____	_____	\$ _____
Programmers (X)	\$ _____	_____	\$ _____
Lead Artist	\$ _____	_____	\$ _____
Artists (X)	\$ _____	_____	\$ _____
Lead Designer	\$ _____	_____	\$ _____
Designers (X)	\$ _____	_____	\$ _____
QA Lead	\$ _____	_____	\$ _____
QA Testers (X)	\$ _____	_____	\$ _____
Producer	\$ _____	_____	\$ _____
Audio Engineer	\$ _____	_____	\$ _____
Writer	\$ _____	_____	\$ _____
SUBTOTAL			\$ _____

2. SOFTWARE AND TOOLS (saphataaayayaaara)

Item	Cost	Priority	Status
Game Engine	\$ _____	_____	_____
3D Software	\$ _____	_____	_____
2D Software	\$ _____	_____	_____
Audio Software	\$ _____	_____	_____
Version Control	\$ _____	_____	_____
Project Mgmt	\$ _____	_____	_____
SUBTOTAL			\$ _____

3. OUTSOURCING (aautasaorasai)

Service	Cost	Provider	Status
Voice Acting	\$ _____	_____	_____
Music	\$ _____	_____	_____
Sound Effects	\$ _____	_____	_____
Motion Capture	\$ _____	_____	_____
Localization	\$ _____	_____	_____
QA Testing	\$ _____	_____	_____
SUBTOTAL			\$ _____

4. MARKETING (maarakatai)

Item	Cost	Timeline	Status
Trailer	\$ _____	_____	_____
Ad Campaign	\$ _____	_____	_____
Events	\$ _____	_____	_____
PR Agency	\$ _____	_____	_____
Social Media	\$ _____	_____	_____
Influencer	\$ _____	_____	_____
SUBTOTAL			\$ _____

5. HARDWARE (haaradaayayaaara)

Item	Cost	Qty	Total
Dev PCs	\$ _____	_____	\$ _____
Consoles	\$ _____	_____	\$ _____
Mobile Devices	\$ _____	_____	\$ _____
Servers	\$ _____	_____	\$ _____

SUBTOTAL		\$ _____	
6. OTHER (anayaaanaya)			
Item	Cost	Notes	Status
Office	\$ _____	_____	_____
Legal	\$ _____	_____	_____
Insurance	\$ _____	_____	_____
Travel	\$ _____	_____	_____
Contingency (10%)	\$ _____	_____	_____
SUBTOTAL		\$ _____	
7. TOTAL SUMMARY (saaamagaraika saaaraaasha)			
Category	Total		
Personnel	\$ _____		
Software	\$ _____		
Outsourcing	\$ _____		
Marketing	\$ _____		
Hardware	\$ _____		
Other	\$ _____		
GRAND TOTAL	\$ _____		

taemapalaeta 13: Marketing Plan (maarakatai palayaaana)

MARKETING PLAN			
(maarakatai palayaaana)			
Project Name: _____			
Launch Date: _____			
Total Budget: \$ _____			
1. SOCIAL MEDIA (saoshayaaala maidaiyaaa)			
Platform	Strategy	Budget	KPI
Twitter/X	_____	\$ _____	_____
Instagram	_____	\$ _____	_____
TikTok	_____	\$ _____	_____
YouTube	_____	\$ _____	_____
Discord	_____	\$ _____	_____
Reddit	_____	\$ _____	_____
Facebook	_____	\$ _____	_____
2. CONTENT MARKETING (kanataenata maarakatai)			
Content Type	Timeline	Budget	Status
Developer Blog	_____	\$ _____	_____
Behind the Scenes	_____	\$ _____	_____
Tutorial Videos	_____	\$ _____	_____
Lore Videos	_____	\$ _____	_____
Interviews	_____	\$ _____	_____
Newsletter	_____	\$ _____	_____
3. PAID ADVERTISING (paeida baijanyaaapana)			
Platform	Budget	Duration	KPI
Google Ads	\$ _____	_____	_____
Facebook Ads	\$ _____	_____	_____
Twitter Ads	\$ _____	_____	_____
YouTube Ads	\$ _____	_____	_____
Reddit Ads	\$ _____	_____	_____
Twitch Ads	\$ _____	_____	_____
4. INFLUENCER MARKETING (inaphalauyaenasaaara)			
Influencer	Budget	Platform	Status
_____	\$ _____	_____	_____
_____	\$ _____	_____	_____
5. PR AND MEDIA (PR au maidaiyaaa)			
Press Kit: _____			
Press Releases: _____			
Media Outreach: _____			

6. EVENTS (ibhaenata)
Conventions: _____
Online Events: _____
tournaments: _____
7. TIMELINE (samayaraekhaaa)
6 Months Before: _____
3 Months Before: _____
1 Month Before: _____
Launch Week: _____
Post-Launch: _____

taemapalaeta 14: Launch Checklist (lanyacha chaekalaisata)

LAUNCH CHECKLIST
(lanyacha chaekalaisata)

Project: _____
Launch Date: _____

PRE-LAUNCH (lanyachaera aagae)
Gold Master Build Ready
All Platforms Certified
Day-1 Patch Ready
Server Infrastructure Ready
Customer Support Ready
Press Kit Sent
Marketing Materials Ready
Store Pages Live
Trailer Released
Social Media Campaign Started
Influencer Copies Sent
Review Copies Sent
Community Ready
Legal Clearance
Rating Board Approval

LAUNCH DAY (lanyacha daina)
Build Deployed
Servers Live
Monitoring Active
Support Team Online
Social Media Active
Performance Monitoring
Bug Reports Collected
Player Feedback Monitored

POST-LAUNCH (lanyachaera para)
Day-1 Patch Deployed
Day-3 Review
Week-1 Analysis
Critical Bugs Fixed
Player Feedback Addressed
Performance Optimization
Community Management
Sales Analysis
Next Update Planned

TASK TRACKING (taaasaka tarayaaakai)

Task	Owner	Status	Due
_____	_____	_____	_____
_____	_____	_____	_____
_____	_____	_____	_____

taemapalaeta 15: Post-Launch Plan (paosata-lanyacha palayaaana)

POST-LAUNCH PLAN

(paosata-lanyacha palayaaana)

Project: _____

Launch Date: _____

Plan Duration: 12 Months

MONTH 1 (parathama maaasa)

Update Content

Day-1 Patch

Week-1 Hotfix

Balance Adjustments

Bug Fixes

Priority

High

High

Medium

High

Status

MONTH 2-3 (dabaitaiiya-taritaiiya maaasa)

Update Content

Quality of Life Updates

New Cosmetics

Performance Optimization

Community Features

Priority

Medium

Low

High

Medium

Status

MONTH 4-6 (chatauratha-ssassatha maaasa)

Update Content

Major Content Update

New Game Mode

New Characters

Seasonal Event

Priority

High

Medium

Medium

Low

Status

MONTH 7-9 (sapatama-nabama maaasa)

Update Content

DLC Planning

Expansion Content

Cross-Platform Support

Mod Support

Priority

High

High

Medium

Low

Status

MONTH 10-12 (dashama-dabaaadasha maaasa)

Update Content

DLC Release

Anniversary Event

Year 2 Planning

Community Survey

Priority

High

Medium

High

Medium

Status

KPI TRACKING (KPI tarayaaakai)

KPI

DAU

MAU

Retention D1

Retention D7

Retention D30

Revenue

Reviews

Target

Actual

Status

saaarasakassaepa (Summary)

aii adhayaaayae aamaraaa 15tai raiiujalebala taemapalaeta taairai karaechhai:

bayabahaaaraera taipasa

1. ****Customize****: parataitai taemapalaeta aapanaaara parakalapaera anauyaaayaii kaaasatamaaaaija karauna
2. ****Iterate****: aikabaaarae saba phailada pauurana karaaara chaessataaa karabaena naaa

3. **Collaborate**: taima maemabaaaradaera saaathae shaeyaaara karauna
4. **Update**: naiyamaita aapadaeta karauna
5. **Version Control**: sasakarana naiyanatarana bajaaaya raaakhauna

PART 40

Chapter 40

adhayaaaya 40: FINAL MASTER WORKFLOW -- chauudaaanata maaasataaara auyaaarakaphalao

paraichaitai

aai adhayaaayae aamaraaa gaema daebhaelapamaenataera samapauurana auyaaarakaphalao taairai karaba aitai aikatai baisataaaraita raodamayaaapa yaetai aaidaiyaaa thaekae shaurau karae paosata-lanyacha parayanata sabakaichhau kabhaaara karae

chauudaaanata maaasataaara auyaaarakaphalao daaayaaagaraama

GAME DEVELOPMENT MASTER WORKFLOW									
(gaema daebhaelapamaenata maaasataaara auyaaarakaphalao)									
IDEA	>	RESEARCH	>	PITCH	>	GDD			
(aaidaiyaaa)		(gabaessanaaa)		(paicha)			(daijaaaaina)		
							PROTOTYPE		
							(paraotaotaaaipa)		
							VERTICAL		
							SLICE		
							(bhaaarataikayaaala		
							salaaaaisa)		
							PRODUCTION (paraodaaakashana)		
Levels		Characters		Enemies		Quests		UI/Audio	
(laebhaela)		(charaitara)		(shatarau)		(kaoyaesata)		(UI/adaiaiu)	
							ALPHA		
							(aalaphaaa)		
							BETA		
							(baitaaa)		
							QA PHASE (QA parayaaaya)		
Testing		Bug Fix		Balance		Polish			
(taesatai)		(baaaga phaikasa)		(bayaaalaenasa)		(palaisha)			
							OPTIMIZATION (apataimaaaaijaeshana)		
Perform.		Memory		Loading		Platform			
(paaarapharamayaaanasa)		(maemarai)		(laodai)		(palayaaatapharama)			
							MARKETING (maarakataetai)		
Social		Content		Paid		PR/Events			
(saoshayaaala)		(kanataenata)		(paeida)		(PR/ibhaenata)			
							LAUNCH		
							(lanyacha)		
							POST-LAUNCH (paosata-lanyacha)		
Support		Updates		DLC		New Game			
(saaapaorata)		(aapadaeta)		(DLC)		(natauna gaema)			

parayaaaya 1: IDEA (aaidaiyaaa)

mauula kaaarayakarama (Key Activities)

- gaemaera mauula dhaaaranaaa taairai
- iunaika hauka nairadhaaarana
- taaaragaeta adaiyaenasa chaihanaaitakarana
- paraaathamaika jaenaaara nairadhaaarana

samayakaaala (Duration)

- **1-2 sapataaaha**

parayaojanaiiya taima (Team Required)

- Game Designer
- Creative Director
- Producer

taulasa (Tools)

- paepaaara au paenasaila
- Miro/FigJam
- Google Docs
- Notion

daelaibhaaaraebala (Deliverables)

- Game Concept Document
- Unique Hook Statement
- Target Audience Profile
- Initial Genre Selection

saaaphalayaera maaanadanada (Success Criteria)

- kalaiyaaara gaema kanasaepata nairadhaaaraita
- iunaika saelai payaenata chaihanaaita
- taaaragaeta adaiyaenasa sapassata

jhaukai (Risk)

- jhaukai: asapassata kanasaepata
- samaaadhaaana: baaarabaaara bhaebae chaeka karauna, phaidabayaaaka naina

parayaaaya 2: RESEARCH (gabaessanaaa)

mauula kaaarayakarama (Key Activities)

- baaajaaara gabaessanaaa

- parataiyaogaii baishalaessana
- taekanaikayaaaala phaejaibailaitai
- baaajaeta anaumaaana

samayakaaala (Duration)

- **2-4 sapataaaha**

parayaojanaiiya taima (Team Required)

- Game Designer
- Producer
- Market Research Analyst

taulasa (Tools)

- Steam/PlayStation/Xbox Store
- Newzoo/Nielsen
- Google Trends
- Social Media Analytics

daelaibhaaraebala (Deliverables)

- Market Research Report
- Competitor Analysis
- Technical Feasibility Study
- Preliminary Budget

saaaphalayaera maaanadanada (Success Criteria)

- baaajaaaraera chaaahidaaa yaaachaaai
- parayaukataigata samabhaaabananaa nairadhaaraaita
- baaajaeta anaumaaana taairai

jhaukai (Risk)

- jhaukai: bhaula baaajaaara baishalaessana
- samaaadhaana: baishabasata saorasa bayabahaaara karauna

parayaaaya 3: PITCH (paicha)

mauula kaaarayakarama (Key Activities)

- paicha dakaumaenata taairai
- daemao parasatautai
- paaabalaishaaara/inabhaesataradaera saaathae saaakassaaa

- phaidabayaaaka sagaraha

samayakaaala (Duration)

- **2-4 sapataaaha**

parayaojanaiiya taima (Team Required)

- Game Director
- Producer
- Lead Designer

taulasa (Tools)

- PowerPoint/Keynote
- Video Editing Software
- Pitch Deck Template

daelaibhaaraebala (Deliverables)

- Game Pitch Deck
- 60-Second Pitch Video
- Business Plan
- Financial Projections

saaaphalayaera maaanadanada (Success Criteria)

- paaabalaishaaara/inabhaesataradaera aagaraha arajana
- phaaanadai naishachaita

jhaukai (Risk)

- jhaukai: paicha raijaekataeda
- samaaadhaana: baikalapa paaabalaishaaara khaujauna, sabaaadhaiina parakaaasha baibaechanaaaa karauna

parayaaaya 4: GDD (gaema daijaaaina dakaumaenata)

mauula kaaarayakarama (Key Activities)

- samapauurana GDD laekhaaa
- maekaaanaika barananaaa
- laebhaela daijaaaina
- charaitara daijaaaina
- UI/UX paraikalapanaaa

samayakaaala (Duration)

- **4-8 sapataaaha**

parayaojanaiiya taima (Team Required)

- Lead Designer
- Game Designers
- Concept Artists
- Technical Director

taulasa (Tools)

- Google Docs/Confluence
- Miro/FigJam
- Photoshop/Blender
- Figma

daelaibhaaaraebala (Deliverables)

- Complete GDD (15+ Sections)
- Concept Art Package
- Technical Design Document
- Art Bible

saaaphalayaera maaanadanada (Success Criteria)

- GDD samapauurana au kalaiyaaara
- taima sabaaai baujhatae paaarae
- paraibaratanaera janaya phalaekasaibala

jhaukai (Risk)

- jhaukai: GDD atairaikata daiiragha baaa asapassata
- samaaadhaana: sakassaipata au sapassata raaakhauna

parayaaaya 5: PROTOTYPE (paraotaotaaaipa)

mauula kaaarayakarama (Key Activities)

- kaora maekaaanaika baaasatabaaayana
- baesaika palaeyabailaitai taesata
- parayaukatai bhayaaalidaeshana
- paraaathamaika phaidabayaaaka

samayakaaala (Duration)

- **4-8 sapataaaha**

parayaojanaiiya taima (Team Required)

- Lead Programmer
- Programmers
- Level Designer
- QA Tester

taulasa (Tools)

- Game Engine (Unity/Unreal)
- Version Control (Git/Perforce)
- Placeholder Art
- Basic Audio

daelaibhaaraebala (Deliverables)

- Working Prototype
- Technical Documentation
- Performance Metrics
- Player Feedback Report

saaaphalayaera maaanadanada (Success Criteria)

- kaora maekaaanaika kaaaja karae
- khaelatae majaaa laaagae
- parayaukataigata samasayaaa naei

jhaukai (Risk)

- jhaukai: paraotaotaaaipa kaaaja karae naaa
- samaaadhaana: chhaota chhaota sataepae aigaona, baaarabaaara taesata karauna

parayaaaya 6: VERTICAL SLICE (bhaaarataikayaaala salaaaaisa)

mauula kaaarayakarama (Key Activities)

- aikatai samapauurana laebhaela taairai
- aarata paaaipalaaaaina parataissathaaa
- adaiau paaaipalaaaaina parataissathaaa
- paraodaaakashana auyaaarakaphalao bhayaaalaidaeta

samayakaaala (Duration)

- **8-12 sapataaaha**

parayaojanaiiya taima (Team Required)

- Full Team (Core Members)
- Artists
- Programmers
- Designers
- Audio Engineers

taulasa (Tools)

- Game Engine
- 3D/2D Software
- Audio Software
- Version Control

daelaibhaaraebala (Deliverables)

- Complete Vertical Slice Level
- Art Pipeline Documentation
- Audio Pipeline Documentation
- Production Workflow

saaaphalayaera maaanadanada (Success Criteria)

- aikatai samapauurana laebhaela khaelaaa yaaaya
- aarata/adaiau paaipalaaaaina kaaaja karae
- paraodaaakashana parakaraiyaaa samautha

jhaukai (Risk)

- jhaukai: paaipalaaaaina samasayaaa
- samaaadhaana: aagae thaekae paraosaesa satayaaanadaaaradaaaija karauna

parayaaaya 7: PRODUCTION (paraodaaakashana)

mauula kaaarayakarama (Key Activities)

- saba laebhaela taairai
- saba charaitara taairai
- saba shatarau taairai
- saba kaoyaesata baaasatabaaayana
- UI/UX baaasatabaaayana
- adaiau baaasatabaaayana
- ayaaanaimaeshana
- VFX taairai

samayakaaala (Duration)

- **6-12 maaasa**

parayaojanaiiya taima (Team Required)

- Full Production Team
- All Departments
- QA Team
- Producer

taulasa (Tools)

- Game Engine
- All Production Software
- Bug Tracking System
- Project Management Tools

daelaibhaaaraebala (Deliverables)

- All Game Content
- All Assets
- All Systems
- All Features

saaaphalayaera maaanadanada (Success Criteria)

- saba kanataenata taairai
- saba phaichaaara kaaaja karae
- samayamatao aigaochachhae

jhaukai (Risk)

- jhaukai: sakaopa karaipa, daelaaai
- samaaadhhaana: sataraikata sakaopa mayaaanaejamaenata

parayaaaya 8: ALPHA (aalaphaaa)

mauula kaaarayakarama (Key Activities)

- saba phaichaaara samapauurana
- saba kanataenata ayaaadaeda
- anataragata QA paraiikassaaa
- baaaga phaikasai shaurau

samayakaaala (Duration)

- **4-8 sapataaaha**

parayaojanaiiya taima (Team Required)

- Full Team
- QA Team (bada)
- Producer

taulasa (Tools)

- Bug Tracking System
- Automated Testing
- Performance Profiling

daelaibhaaraebala (Deliverables)

- Feature Complete Build
- Content Complete Build
- QA Report
- Bug List

saaaphalayaera maaanadanada (Success Criteria)

- saba phaichaaara aachhae
- saba kanataenata aachhae
- gaema khaelaaa yaaaya

jhaukai (Risk)

- jhaukai: anaeka baaaga
- samaaadhaana: paraayaoraitai baesada baaaga phaikasai

parayaaaya 9: BETA (baitaaa)

mauula kaaarayakarama (Key Activities)

- baaairaera taesataaaradaera paraiikassaaa
- bayaaalaenasa taiunai
- paaarapharamayaaanasa apataimaaaijaeshana
- palaisha

samayakaaala (Duration)

- **4-8 sapataaaha**

parayaojanaiiya taima (Team Required)

- QA Team
- Balance Designers

- Performance Engineers
- Producer

taulasa (Tools)

- Beta Testing Platform
- Analytics Tools
- Performance Profiling
- Feedback Collection

daelaibhaaraebala (Deliverables)

- Beta Build
- Balance Report
- Performance Report
- Player Feedback Analysis

saaaphalayaera maaanadanada (Success Criteria)

- baaairaera taesataaararaaa khaushai
- bayaaalaenasa bhaaalao
- paaarapharamayaaanasa bhaaalao

jhaukai (Risk)

- jhaukai: khaaaraapa raibhaiu
- samaadhhaana: samayatao samasayaaa samaadhhaana karauna

parayaaaya 10: QA (kaoyaalaitai ayaaasauraenasa)

mauula kaaarayakarama (Key Activities)

- baisataaaraita taesatai
- saba palayaaatapharamae taesata
- saaarataiphaikaeshana parasatautai
- phaaainaaala baaaga phaikasa

samayakaaala (Duration)

- **4-8 sapataaaha**

parayaojanaiiya taima (Team Required)

- QA Team (paurana)
- Certification Team
- Producer

taulasa (Tools)

- Automated Testing
- Manual Testing
- Certification Checklists

daelaibhaaraebala (Deliverables)

- QA Report
- Certification Build
- Final Bug List
- Compliance Documents

saaaphalayaera maaanadanada (Success Criteria)

- karaitaikayaaala baaaga naei
- saaarataiphaikaeshana paaasa
- saba palayaaatapharamae kaaaja karae

jhaukai (Risk)

- jhaukai: saaarataikaeshana phaeila
- samaaadhaana: aagae thaekae chaekalaisata phalao karauna

parayaaaya 11: OPTIMIZATION (apataimaaaijaeshana)

mauula kaaarayakarama (Key Activities)

- paaarapharamayaaanasa apataimaaaija
- maemarai apataimaaaija
- laodai taaaima kamaanao
- palayaaatapharama-sapaesaiphaika apataimaaaija

samayakaaala (Duration)

- **4-8 sapataaaha**

parayaojanaiiya taima (Team Required)

- Performance Engineers
- Programmers
- Technical Artists

taulasa (Tools)

- Profiling Tools
- Memory Analyzers

- Platform-Specific Tools

daelaibhaaraebala (Deliverables)

- Optimized Build
- Performance Report
- Memory Report
- Platform Compliance

saaaphalayaera maaanadanada (Success Criteria)

- taaaragaeta FPS arajaita
- taaaragaeta raejaolaiushana
- darauta laodai

jhaukai (Risk)

- jhaukai: apataimaaaijaeshanae samaya baeshai laaagae
- samaaadhaana: shaudhau gaurautabapauurana jaaayagaaaya phaokaaasa karauna

parayaaaya 12: MARKETING (maarakatai)

mauula kaaarayakarama (Key Activities)

- taelaara taairai
- saoshayaaala maidaiyaaa kayaaamapaeina
- paraesa kaita taairai
- inaphalauyaenasaaara aautaraicha
- ibhaenata palayaaanai

samayakaaala (Duration)

- **3-6 maaasa (lanyachaera aagae thaekae)**

parayaojanaiya taima (Team Required)

- Marketing Team
- PR Team
- Social Media Manager
- Content Creator

taulasa (Tools)

- Video Editing Software
- Social Media Platforms
- Email Marketing

- Analytics Tools

daelaibhaaraebala (Deliverables)

- Launch Trailer
- Social Media Content
- Press Kit
- Marketing Calendar

saaaphalayaera maaanadanada (Success Criteria)

- baishaissata anausaranakaaarii sakhayaaa baaadae
- paraesa kabhaaraeja paaaauyaaa yaaaya
- auyaaaishalaisata sakhayaaa baaadae

jhaukai (Risk)

- jhaukai: khaaraaapa maaarakaetai
- samaadhaana: taaragaetaeda kayaaamapaeina chaaalaaana

parayaaaya 13: LAUNCH (lanyacha)

mauula kaaarayakarama (Key Activities)

- gaolada maaasataaara taairai
- palayaaatapharama saaabamaita
- lanyacha dae saaapaorata
- manaitarai

samayakaaala (Duration)

- **1-2 sapataaaha**

parayaojanaiiya taima (Team Required)

- Full Team
- Support Team
- Community Manager
- Producer

taulasa (Tools)

- Build System
- Analytics
- Support Tools
- Monitoring Tools

daelaibhaaraebala (Deliverables)

- Gold Master Build
- Store Pages
- Day-1 Patch
- Support Documentation

saaaphalayaera maaanadanada (Success Criteria)

- saphala lanyacha
- kama baaaga
- bhaaalao paaarapharamayaanasa

jhaukai (Risk)

- jhaukai: lanyacha daijaaasataaara
 - samaaadhaana: Day-1 Patch raedai raaakhauna
-

parayaaaya 14: POST-LAUNCH (paosata-lanyacha)

mauula kaaarayakarama (Key Activities)

- saaapaorata paradaaana
- baaaga phaikasa
- aapadaeta parakaaasha
- DLC paraikalapanaaa
- natauna gaema paraikalapanaaa

samayakaaala (Duration)

- **12+ maaasa**

parayaojanaiiya taima (Team Required)

- Support Team
- Community Manager
- Producer
- Development Team (chhaota)

taulasa (Tools)

- Support Tools
- Analytics
- Community Platforms
- Development Tools

daelaibhaaaraebala (Deliverables)

- Regular Updates
- Bug Fixes
- New Content
- DLC

saaaphalayaera maaanadanada (Success Criteria)

- khaelaoyaaadaraaa sanataussata
- raebhaiu bhaaalao
- raebhainaiu bhaaalao

jhaukai (Risk)

- jhaukai: khaelaoyaaada haaaraiyae yaaaya
- samaaadhaaana: dhaaaraabaaahaika aapadaeta daina

darauta shauraura nairadaeshanaaa (Quick Start Guide for Beginners)

dhaaapa 1: aaidaiyaaa (1 sapataaaha)

- [] gaemaera naaama daina
- [] jaenaaara nairadhaaarana karauna
- [] mauula maekaaanaika barananaaa karauna
- [] iunaika hauka laikhauna
- [] taaaragaeta adaiyaenasa chaihanaita karauna

dhaaapa 2: gabaessanaaa (2 sapataaaha)

- [] baaajaaara gabaessanaaa karauna
- [] parataiyaogaii baishalaessana karauna
- [] parayaukatai gabaessanaaa karauna
- [] baaajaeta anaumaaana karauna

dhaaapa 3: paicha (2 sapataaaha)

- [] paicha daeka taairai karauna
- [] 60 saekaenada bhaidaiu baaanaaana
- [] baijanaesa palayaaana laikhauna
- [] paaabalaishaaaradaera saaathae yaogaaayaoga karauna

dhaaapa 4: GDD (4 sapataaaha)

- [] GDD shaurau karauna
- [] maekaaanaika barananaaa karauna
- [] laebhaela daijaaaaina karauna
- [] charaitara daijaaaaina karauna
- [] taimaera saaathae yaaachaaai karauna

dhaaapa 5: paraotaotaaaipa (4 sapataaaha)

- [] inyajaina saetaaapa karauna
- [] kaora maekaaanaika baaasatabaaayana karauna

- [] baesaika aarata yaoga karauna
- [] taesata karauna
- [] phaidabayaaaka naina

samapauurana raodamayaaapa saaarasakassaepa (Complete Roadmap Summary)

maota samayakaaala: 18-36 maaasa

COMPLETE ROADMAP SUMMARY	
(samapauurana raodamayaaapa saaarasakassaepa)	
Phase 1: IDEA (aaidaiyaaa)	[1-2 Weeks]
Game Concept	
Unique Hook	
Target Audience	
Phase 2: RESEARCH (gabaessanaaa)	[2-4 Weeks]
Market Research	
Competitor Analysis	
Feasibility Study	
Phase 3: PITCH (paicha)	[2-4 Weeks]
Pitch Deck	
Demo Video	
Business Plan	
Phase 4: GDD (daijaaaina)	[4-8 Weeks]
Complete GDD	
Concept Art	
Technical Design	
Phase 5: PROTOTYPE (paraotaotaaaipa)	[4-8 Weeks]
Core Mechanics	
Basic Playability	
Tech Validation	
Phase 6: VERTICAL SLICE (bhaaarataikayaaala salaaaisa)	[8-12 Weeks]
Complete Level	
Art Pipeline	
Audio Pipeline	
Phase 7: PRODUCTION (paraodaaakashana)	[6-12 Months]
All Levels	
All Characters	
All Enemies	
All Quests	
UI/UX	
Audio	
Animation/VFX	
Phase 8: ALPHA (aalaphaaa)	[4-8 Weeks]
Feature Complete	
Content Complete	
Internal QA	
Phase 9: BETA (baitaaa)	[4-8 Weeks]
External Testing	
Balance Tuning	
Performance Optimization	
Phase 10: QA (kaoyaaalaitai ayaaasauranasa)	[4-8 Weeks]
Comprehensive Testing	
Certification	
Final Bug Fixes	
Phase 11: OPTIMIZATION (apataimaaaijaeshana)	[4-8 Weeks]
Performance Optimization	

Memory Optimization	
Platform Optimization	
Phase 12: MARKETING (maarakatai)	[3-6 Months]
Trailer	
Social Media	
PR	
Events	
Phase 13: LAUNCH (lanyacha)	[1-2 Weeks]
Gold Master	
Platform Submission	
Launch Support	
Phase 14: POST-LAUNCH (paosata-lanyacha)	[12+ Months]
Support	
Updates	
DLC	
New Game Planning	

jhaukai mauulayaaayana saaarasakassaepa (Risk Assessment Summary)

RISK ASSESSMENT SUMMARY
(jhaukai mauulayaaayana saaarasakassaepa)
HIGH RISK (uchacha jhaukai):
Scope Creep (sakaopa karaipa)
Budget Overrun (baajaeta atakarama)
Timeline Delays (samayasaiimaaa bailamaba)
Technical Issues (parayaukataigata samasayaaa)
Team Turnover (tama paraibaratana)
MEDIUM RISK (maajhaaraai jhaukai):
Market Changes (baajaaara paraibaratana)
Competition (parataiyaogaitaaa)
Platform Changes (palayaaatapharama paraibaratana)
Player Expectations (khaelaoyaaadaera paratayaaashaaa)
LOW RISK (kama jhaukai):
Minor Bugs (chhaota baaaga)
Feature Requests (phaichaaara anauraodha)
Cosmetic Issues (kasamaetaika samasayaaa)
MITIGATION STRATEGIES (shamana kaaushala):
Regular Reviews (naiyamaita parayaaalaochanaaa)
Clear Communication (sapassata yaogaaayaoga)
Flexible Planning (namanaiiya paraikalapananaaa)
Buffer Time (baaphaaara samaya)
Contingency Plans (kanatainajaenasai palayaaana)

gaurautabapauurana taipasa (Important Tips)

1. paraikalapananaa karauna

- parataitai parayaaayaera janaya sapassata paraikalapananaa karauna
- samayasaiimaaa nairadhaaarana karauna
- raisaorasa palayaaanai karauna

2. yaogaaayaoga karauna

- taimaera saaathae naiyamaita yaogaaayaoga raaakhauna
- satayaaataaasa aapadaeta daina
- samasayaaa samaaadhaaanae sahaaayataaa karauna

3. phalaekasaibala thaaakauna

- paraibaratanaera janaya parasatauta thaaakauna
- natauna aaidaiyaaa garahana karauna
- aparatayaaashaita samasayaaara samaaadhaaana karauna

4. maaanadanada bajaaaya raaakhauna

- parataitai parayaaayaera janaya maaanadanada nairadhaaarana karauna
- naiyamaita parayaaalaochanaaa karauna
- maaana kamaaabaena naaa

5. dakaumaenataeshana raaakhauna

- sabakaichhau dakaumaenata karauna
- sasakarana naiyanatarana bayabahaaara karauna
- janyaaana shaeyaaara karauna

chauudaaanata kathaaa (Final Words)

gaema daebhaelapamaenata aikatai daiiragha au jataila yaaataraaa aii maaasataaara auyaaarakaphalao aapanaaakae parataitai parayaaayae nairadaeshananaa daebae manae raaakhabaena:

1. ****Shaurau karauna****: sabachaeyae bada chayaaalaenyaja halao shaurau karaaa
2. ****aigaiyae chalauna****: parataidaina aikatau aigaiyae yaaana
3. ****shaikhauna****: bhaula thaekae shaikhauna
4. ****bhaaaga karauna****: abhaijanyataaa anayadaera saaathae bhaaaga karauna
5. ****majaaa karauna****: gaema daebhaelapamaenata upabhaoga karauna

****saphalataaa kaaamanaaa karai!****